CoderArmy

Introduction to Spring & SpringBoot

1. The Starting Point: How Web Applications Communicate

Before understanding Spring, Spring Boot, Servlet, or any backend framework, we first need to understand one basic question:

> How does a user sitting on one machine communicate with code running on another machine?

This is the foundation of web development.

When you open a browser and type:

```
www.amazon.com
```

your browser is running on your laptop or mobile phone, while Amazon's application is running on Amazon's server somewhere else.

At the most basic level:

```
Your Browser  ---- talks to ----  Amazon Server
```

This structure is called **Client–Server Architecture**.

---

2. Client–Server Architecture

What is a Client?

A **client** is the side that asks for something.

Examples of clients:

CoderArmy

```
Browser
Mobile app
Postman
Frontend React app
Android app
iOS app
```

When you open a website, your browser becomes the client.

It sends a request like:

> "Hey server, give me this webpage."

What is a Server?

A **server** is the side that receives requests, processes them, and sends back a response.

Examples of servers:

```
Amazon server
YouTube server
Bank server
Your Spring Boot application
```

A server usually performs tasks like:

```
Checking login details
Fetching data from a database
Applying business rules
Saving information
Returning a response
```

The basic idea is simple:

```
Client asks.
Server responds.
```

CoderArmy

3. HTTP: The Language of the Web

Now the important question is:

> How does the browser know how to ask?
>
> How does the server understand what the browser is asking?

For communication to happen properly, both sides need a common language.

That common language is called **HTTP**.

What is HTTP?

HTTP stands for:

```
HyperText Transfer Protocol
```

In simple words:

> HTTP is the rulebook for communication between a client and a server.

HTTP defines:

```
How a request should look
How a response should look
Which method is being used
Which URL is being called
What data is being sent
Which status code is returned
```

So the browser and server do not randomly exchange text. They follow a proper format.

HTTP is an **application-layer protocol** that works on top of TCP/IP.

---

4. Request–Response Cycle

Every web interaction follows the same basic pattern:

CoderArmy

```
Client sends request
Server processes request
Server sends response
Client displays or uses the response
```

Example:

```
www.coderarmy.in/courses
```

The browser may send an HTTP request like:

```
GET /courses
Host: www.coderarmy.in
```

The server receives the request and understands:

```
The user wants the courses page.
Fetch the courses data.
Prepare the response.
Send it back to the browser.
```

The server may send back:

```
Status: 200 OK
Body: course data / HTML page / JSON
```

The browser receives the response and displays the page.

---

5. Anatomy of an HTTP Request

An HTTP request usually contains four main parts:

```
Method
URL or path
```

CoderArmy

```
Headers
Body
```

Example:

```
POST /login
Content-Type: application/json

{
  "email": "abc@gmail.com",
  "password": "12345"
}
```

Meaning:

```
POST        → I am sending data
/login      → I want to call the login functionality
Headers     → Extra information about the request
Body        → Actual data being sent
```

HTTP Methods

The first line of an HTTP request is called the **request line**.

It contains the method, path, and HTTP version.

The method tells the server what action the client wants to perform.

Method	Meaning	Example Use
`GET`	Read data	Fetch a list of orders
`POST`	Create data	Place a new order
`PUT`	Replace data completely	Update an entire user profile
`PATCH`	Update data partially	Change only the phone number
`DELETE`	Remove data	Cancel an order

HTTP Headers

CoderArmy

Headers are key-value pairs that provide extra information about the request.

They tell the server things like:

```
What format the client can understand
What format the request body is in
Who the client is
Which host the client is trying to reach
```

Common examples:

```
Accept: application/json
Content-Type: application/json
Authorization: Bearer token
Host: www.coderarmy.in
```

Headers are very important in real Spring applications because we frequently work with authentication, JSON data, API communication, and request metadata.

HTTP Body

The body carries the actual data being sent by the client.

It is commonly used with:

```
POST
PUT
PATCH
```

Example body:

```
{
  "name": "Rohit",
  "email": "rohit@example.com"
}
```

In modern APIs, the request body is usually sent in JSON format.

CoderArmy

`GET` requests usually do not have a body because they are mainly used to fetch data.

---

6. Anatomy of an HTTP Response

An HTTP response usually contains:

```
Status code
Headers
Body
```

Example:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "message": "Login successful"
}
```

The most important part of the response is the **status code**.

Status Code	Meaning
`200`	Request successful
`404`	Resource not found
`500`	Internal server error

We will study status codes in more detail later.

---

7. JVM: A Java Program Runs Locally

Now let's compare this web communication model with a normal Java program.

Example:

```java
public class Main {
    public static void main(String[] args) {
```

```
        System.out.println("Hello World");
    }
}
```

When we run this program:

```
java Main
```

it runs inside the **JVM**, which means **Java Virtual Machine**.

Your `.java` source file is compiled by `javac` into platform-independent bytecode stored in `.class` files.

The JVM loads this bytecode and executes it.

This is the reason Java is known for:

```
Write once, run anywhere
```

The same bytecode can run on Windows, macOS, or Linux as long as the JVM is available.

---

8. Normal Java Program vs Web Application

A normal Java program usually follows this pattern:

```
Start
Run instructions
Finish
Exit
```

But a website or backend application follows a very different pattern:

```
Start
Keep running
Wait for requests
Process requests
```

CoderArmy

CoderArmy

```
Send responses
Continue running
```

A website is not like a program that runs once and exits.

A web server is a program that stays alive continuously and keeps listening for incoming requests.

---

9. Why Core Java Alone Is Not Enough for Web Applications

Core Java understands concepts like:

```
Classes
Objects
Inheritance
Collections
Threads
Files
```

But Core Java does not automatically understand web concepts like:

```
HTTP requests
URLs
Headers
Cookies
Sessions
REST APIs
```

These are web-related concepts, not basic Java language concepts.

Even if we keep a Java program running using something like:

```java
while (true) {
    // keep program alive
}
```

it still does not automatically understand a request like:

CoderArmy

```
GET /hello
```

Someone has to read that request, understand it, and map it to the correct Java logic.

---

10. Can Java Open Network Connections?

Yes, Java can do networking.

Java has had the `java.net` package since Java 1.0.

For example:

```
ServerSocket server = new ServerSocket(8080);
```

This allows a Java program to listen on port `8080`.

So technically, Java can communicate over a network.

But there is still a problem.

When a browser or Postman sends an HTTP request, Java receives a raw TCP connection — basically a stream of bytes.

Example request:

```
GET /users HTTP/1.1
Host: localhost:8080
```

To the JVM, this is just data.

It does not automatically understand:

```
GET means fetch data
/users is an endpoint
Host is a header
```

Someone has to interpret all of this manually.

CoderArmy

11. What We Would Need to Do Manually in Core Java

If we tried to build a web application using only Core Java, we would have to handle many things ourselves.

We would need to:

1. Open a port using `ServerSocket`
2. Read raw input streams
3. Parse the HTTP request manually
4. Extract the method, URL, headers, and body
5. Route the request to the correct Java logic
6. Create the HTTP response in the correct format
7. Manage multiple users using threads
8. Handle errors, malformed requests, and connection behavior

Example of manual routing:

```
if (url.equals("/users")) {
    // call user logic
} else if (url.equals("/orders")) {
    // call order logic
}
```

This code has nothing to do with actual business logic.

It is repeated technical work that every Java web application would need.

That is why a standard solution was needed.

---

12. The Problem Before Servlets

We had this gap:

CoderArmy

```
Browser
   |
HTTP Request
   |
   v
 ???
   |
Java Code
```

The browser sends HTTP requests.

Java contains classes and methods.

But something is needed between HTTP and Java to translate web requests into Java method calls.

For example, if a browser sends:

```
GET /hello HTTP/1.1
Host: localhost:8080
```

how should this request trigger a Java method like:

```
sayHello();
```

This translation is handled by:

```
Servlet Container
Servlet
```

---

13. Java Servlets

A **Servlet** is a Java object that can handle HTTP requests.

In simple words:

> A Servlet is a special Java class designed for web applications.

CoderArmy

Conceptually:

```
HTTP Request
      |
      v
   Servlet
      |
      v
  Java Code
```

Servlets were one of the first standard Java technologies created specifically for building web applications.

A Servlet runs inside a **Servlet Container**.

Examples of Servlet Containers:

```
Apache Tomcat
Jetty
Undertow
```

---

14. What Does a Servlet Container Do?

A Servlet Container sits between the outside web world and your Java code.

It handles the low-level web work for you.

A Servlet Container is responsible for:

```
Opening a port such as 8080
Listening for HTTP requests
Reading TCP bytes
Parsing HTTP requests
Creating request and response objects
Managing threads
Calling servlet methods
Sending HTTP responses
Handling connection behavior
```

CoderArmy

Instead of manually reading bytes and parsing HTTP text, the Servlet Container gives us Java-friendly objects.

For example, it can call methods like:

```
doGet()
doPost()
```

So the request:

```
GET /hello
```

can eventually be handled by Java code inside a servlet.

---

15. Why Was Spring Needed Then?

Servlets solved an important problem.

They made it possible for Java applications to handle HTTP requests properly.

But building large enterprise applications directly with Servlets became difficult over time.

Common problems included:

```
Too much boilerplate code
Too many configurations
Tight coupling between classes
Difficult testing
Difficult maintenance in large applications
Repeated code across projects
```

As applications became bigger, developers needed a better way to organize code, manage objects, handle dependencies, and reduce configuration complexity.

This is where the **Spring Framework** became important.

---

16. What is Spring Framework?

CoderArmy

Spring Framework is one of the most popular frameworks in the Java world.

It was created to make enterprise Java development easier, cleaner, and more maintainable.

Spring introduced important concepts like:

```
IoC
Dependency Injection
Bean Management
Configuration
Loose Coupling
```

We will study these concepts in depth later in the series.

For now, remember:

> Spring helps us build Java applications in a cleaner and more manageable way.

---

17. Spring is an Ecosystem

Spring is not just one small library.

Spring is a large ecosystem of projects and frameworks.

Different Spring projects solve different problems.

Some important parts of the Spring ecosystem are:

```
Spring Core
Spring MVC
Spring Data
Spring Security
Spring AOP
Spring Boot
Spring AI
```

---

18. Spring Core

Spring Core is the foundation of the Spring ecosystem.

CoderArmy

It provides the most basic and important features of Spring.

Spring Core includes:

```
IoC
Dependency Injection
Bean Management
Configuration
ApplicationContext
```

Without Spring Core, other Spring projects would not exist.

Spring Core is the base on which many other Spring modules are built.

---

19. Spring MVC

Spring MVC is used to build web applications and REST APIs.

It is built on top of:

```
Servlets
+
Spring Core
```

Spring MVC makes it easier to handle web requests.

Instead of writing Servlet code directly, we can use clean annotations and controller classes.

For example, later we will write code like:

```java
@GetMapping("/hello")
public String sayHello() {
    return "Hello World";
}
```

Spring MVC internally uses Servlet technology, but it gives developers a cleaner programming model.

CoderArmy

20. Spring Data

Most applications need to store data permanently.

For that, applications usually need a database.

Earlier, Java developers commonly used **JDBC**.

With JDBC, developers had to:

```
Write SQL manually
Open database connections
Execute queries
Handle result sets
Close resources
Manage repetitive database code
```

Later, frameworks like **Hibernate** made database work easier.

Hibernate maps Java objects to database tables.

This concept is called **Object-Relational Mapping**, or ORM.

JPA and Hibernate

JPA stands for:

```
Java Persistence API
```

JPA is a specification.

That means it defines rules and guidelines for how Java objects should be mapped to database tables.

But JPA itself does not provide the actual working implementation.

Hibernate is one of the most popular implementations of JPA.

In simple words:

```
JPA tells what should be done.
Hibernate actually does it.
```

CoderArmy

Spring Data JPA goes one step further and reduces even more boilerplate code.

The flow looks like this:

```
Spring Data JPA → Hibernate → JDBC → Database
```

A simple way to remember:

> JDBC crawled so Hibernate could walk, and Hibernate walked so Spring Data JPA could fly.

---

21. Spring Security

Spring Security is used for authentication and authorization.

It helps with features like:

```
Login
JWT
OAuth
Roles
Permissions
Password encoding
CSRF protection
Access control
```

Without Spring Security, developers would have to manually write a lot of sensitive and repetitive security code.

Spring Security gives a standard and powerful way to secure Java applications.

---

22. Spring AOP

AOP stands for:

```
Aspect-Oriented Programming
```

CoderArmy

Spring AOP helps us separate cross-cutting concerns from business logic.

Examples of cross-cutting concerns:

```
Logging
Security checks
Transaction management
Performance tracking
Exception handling
```

These are things that may be needed across many parts of an application.

We will study AOP properly later in the series.

---

23. Spring AI

Spring AI is a newer part of the Spring ecosystem.

It helps Java developers integrate AI features into Spring applications.

It can work with:

```
OpenAI
Gemini
Anthropic
Vector databases
RAG systems
Embeddings
AI chat models
```

This is useful when building AI-powered applications using Java and Spring.

---

24. What is Spring Boot?

Spring Boot is not a replacement for Spring.

Spring Boot is an automation layer on top of Spring.

It helps developers create Spring applications faster by providing:

CoderArmy

```
Auto-configuration
Starter dependencies
Embedded servers
Sensible defaults
Production-ready features
Less manual configuration
```

In simple words:

> Spring Boot configures Spring for us so we can start building applications quickly.

---

25. Spring Boot is Opinionated

Spring Boot makes many default assumptions.

These assumptions are called **opinions**.

An opinionated framework provides sensible defaults so developers do not have to configure everything manually.

For example, if we add a web dependency, Spring Boot assumes that we want to build a web application.

So it can automatically configure things like:

```
Embedded Tomcat
Spring MVC setup
Default application structure
JSON support
Basic error handling
```

This saves a lot of time.

But the real skills are still in understanding:

```
Spring Core
Spring MVC
Dependency Injection
IoC
Beans
```

```
Servlets
HTTP
Database concepts
Security concepts
```

Spring Boot makes development faster, but Spring fundamentals make you a stronger developer.

---

26. Spring Boot vs Spring Framework

A common confusion is:

> Are Spring and Spring Boot the same?

No.

Spring Framework provides the core features and different modules.

Spring Boot makes Spring easier to use by reducing configuration and setup work.

Simple comparison:

Spring Framework	Spring Boot
Provides core features and modules	Provides auto-configuration and quick setup
Requires more manual configuration	Reduces manual configuration
Gives flexibility	Gives sensible defaults
Foundation of the ecosystem	Built on top of Spring

So we can say:

```
Spring Boot uses Spring.
Spring Boot does not replace Spring.
```

---

27. Where Do Microservices Fit?

Microservices are not a separate Spring module.

Microservices are an **architecture style**.

CoderArmy

CoderArmy

In a microservices architecture, a large application is divided into smaller independent services.

For example:

```
User Service
Order Service
Payment Service
Notification Service
Product Service
```

Each service can be developed, deployed, and scaled independently.

Spring Boot is commonly used to build microservices because it makes it easy to create independent production-ready applications.

So the idea is:

```
Microservices = Architecture style
Spring Boot = Tool commonly used to build them
```

---

28. Complete Flow: From Browser to Spring Boot

Now we can connect the whole journey.

```
Browser sends HTTP request
        |
        v
Servlet Container receives request
        |
        v
Servlet technology handles web communication
        |
        v
Spring MVC gives a cleaner web programming model
        |
        v
Spring Core manages objects and dependencies
        |
        v
```

CoderArmy

```
Spring Boot auto-configures everything
        |
        v
Developer writes business logic
```

This is the bigger picture behind modern Java backend development.

---

29. Why Understanding This History Matters

Many beginners start directly with Spring Boot and write code like:

```
@RestController
@GetMapping("/hello")
```

The code works, but they do not understand what is happening behind the scenes.

When we understand the journey from:

```
Client–Server Architecture
HTTP
JVM
Core Java networking
Servlets
Spring Framework
Spring Boot
```

Spring Boot no longer feels magical.

It starts making sense.

We understand why each technology came into the picture and what problem it solved.

---

30. Final Summary

The complete evolution looks like this:

CoderArmy

```
Client–Server Architecture
        ↓
HTTP communication
        ↓
Core Java limitation for web apps
        ↓
Java networking with sockets
        ↓
Manual HTTP parsing problem
        ↓
Servlets and Servlet Containers
        ↓
Spring Framework
        ↓
Spring ecosystem
        ↓
Spring Boot
```

Key takeaways:

```
Client sends a request.
Server sends a response.
HTTP defines the communication format.
Core Java does not automatically understand HTTP.
Servlets helped Java handle web requests.
Servlet Containers handle low-level web work.
Spring made enterprise Java development cleaner.
Spring Core manages objects and dependencies.
Spring MVC simplifies web development.
Spring Data simplifies database access.
Spring Security handles authentication and authorization.
Spring Boot auto-configures Spring applications.
Microservices are an architecture style, not a Spring module.
```

The goal of this series is not just to write Spring Boot code.

The goal is to understand how modern Java backend development actually works.

Once this foundation is clear, Spring Boot becomes much easier to learn, debug, and use in real projects.

Coder Army

Lecture 2: Writing Our First Spring Boot Application

1. What Are We Doing Today?

In this lecture, we are going to write our first Spring Boot application.

The goal is not to understand every internal concept immediately. That is intentional.

Spring Boot hides a lot of complexity in the beginning so that developers can quickly start building applications. In this lecture, we will experience how fast Spring Boot helps us create a working web application, and in later lectures, we will slowly understand what is happening behind the scenes.

---

2. What Are We Building?

We are going to build a simple web application where the browser sends a request to our Spring Boot application, and the application sends back a response.

The flow will look like this:

```
Browser
   |
   | Request
   v
Spring Boot Application
   |
   | Response
   v
Browser
```

We will create one endpoint:

```
localhost:8080/hello
```

When we open this URL in the browser, it will return:

Coder Army

```
Hello World
```

At first, this looks very simple.

But many important questions are hidden behind this simple output:

```
How does the browser talk to Java?
How does Java understand URLs?
Who starts the server?
Who listens on port 8080?
How does /hello get connected to our Java method?
```

We will not answer all of these deeply today, but we will create curiosity for the upcoming lectures.

---

3. What is a Port?

Before running our Spring Boot application, we need to understand the idea of a **port**.

Suppose your laptop is connected to a network.

Your laptop has an IP address.

Example:

```
192.168.1.10
```

Now imagine many applications are running on the same laptop:

```
Chrome
WhatsApp
Zoom
Spotify
Spring Boot application
```

If some data arrives at your laptop, how will the operating system know which application should receive that data?

Coder Army

This is where **port numbers** come in.

IP Address vs Port Number

An **IP address** identifies the machine.

Example:

```
192.168.1.10
```

A **port number** identifies the application running inside that machine.

Example:

```
192.168.1.10:8080
```

Meaning:

```
Machine      = 192.168.1.10
Application  = Listening on port 8080
```

So the IP address brings the data to the correct machine, and the port number helps the operating system deliver that data to the correct application.

Visual Example

```
Internet
   |
   v

192.168.1.10
--------------------------------
| Chrome              Port 5000 |
| WhatsApp            Port 5222 |
| Zoom                Port 8801 |
| Spring Boot App     Port 8080 |
--------------------------------
```

Coder Army

The IP address tells us **which machine** to reach.

The port number tells us **which application** on that machine should receive the request.

---

4. What is Localhost?

Students often get confused between `localhost`, IP address, and port number.

In simple words:

```
localhost = your own machine
```

`localhost` is equivalent to:

```
127.0.0.1
```

So when we write:

```
localhost:8080
```

it means:

```
Send the request to my own machine on port 8080.
```

If our Spring Boot application is running on port `8080`, then the browser can send a request to it using:

```
http://localhost:8080
```

Coder Army

5. How Does the Browser Know Which Port to Use?

When we type a normal website URL like:

```
www.coderarmy.in
```

we usually do not mention any port number.

That is because browsers use default ports.

For example:

Protocol	Default Port
`http`	`80`
`https`	`443`

So when we write:

```
https://www.coderarmy.in
```

the browser automatically assumes port `443`.

When we write:

```
http://www.coderarmy.in
```

the browser automatically assumes port `80`.

But when we are running a local Spring Boot application, it usually runs on port `8080`, so we mention it manually:

```
http://localhost:8080
```

Important point:

Coder Army

> DNS gives the IP address of a domain.
>
> The port is either mentioned in the URL or decided by the protocol default.

For example:

```
https://www.coderarmy.in
```

Here:

```
DNS resolves www.coderarmy.in to an IP address.
HTTPS tells the browser to use port 443 by default.
```

---

6. Development Setup

For this series, we will use **IntelliJ IDEA**.

You can use any editor, but IntelliJ is widely preferred for Java and Spring Boot development because it has strong support for:

```
Java
Maven
Spring Boot
Code navigation
Auto-completion
Project structure
Debugging
```

Before creating the project, make sure Java is installed and working on your system.

You can verify Java installation using:

```
java -version
```

If Java is not installed, install it first before moving ahead.

Coder Army

7. Why Does Spring Initializr Exist?

Imagine starting a Java web project from scratch.

Before writing even a single line of business logic, we would need to do many setup tasks:

```
Create folder structure
Create Maven configuration
Select Spring Boot version
Select Java version
Add required dependencies
Configure package structure
Prepare project metadata
```

This setup work is repetitive.

Every Spring Boot project needs a similar basic structure.

To solve this problem, the Spring team created **Spring Initializr**.

What is Spring Initializr?

Spring Initializr is a project generator for Spring applications.

It creates the basic project skeleton for us so that we can quickly start writing application logic.

In simple words:

> Spring Initializr helps us create a ready-to-use Spring Boot project.

It allows us to choose:

```
Project type
Programming language
Spring Boot version
Project metadata
Packaging type
Java version
Dependencies
```

Coder Army

8. What is a Dependency?

When we build a Java project, we often need external libraries.

For example, if we want our Java application to connect with MySQL, we need a MySQL connector library.

That reusable external code is called a **library**.

When our project needs that library to work, it becomes a **dependency**.

Simple definition:

> A dependency is an external library required by our project.

Examples:

```
Spring Web
MySQL Connector
Spring Data JPA
Spring Security
Lombok
Hibernate
```

In Maven projects, dependencies are added inside the `pom.xml` file.

---

9. Choosing the Right Spring Boot Version

On Spring Initializr, we may see versions like:

```
Stable release
SNAPSHOT
RC
```

SNAPSHOT

A `SNAPSHOT` version is a work-in-progress version.

It may contain bugs and is not considered final.

Coder Army

RC

`RC` means **Release Candidate**.

It is almost ready but still not the final stable release.

Stable Release

For beginners and real projects, we should prefer stable releases.

A stable release is tested and suitable for normal development.

---

10. JAR Packaging in Spring Boot

Modern Spring Boot applications usually use **JAR** packaging.

JAR stands for:

```
Java Archive
```

A Spring Boot JAR can contain our application code along with the required setup to run the application.

This is one reason Spring Boot applications are easy to run.

Instead of manually deploying to an external server, we can run the application directly.

---

11. Project Tour

After creating the project from Spring Initializr and opening it in IntelliJ, we will see a standard Spring Boot project structure.

Common important files and folders include:

```
src/main/java
src/main/resources
pom.xml
```

```
application.properties
Main application class
```

`src/main/java`

This folder contains our Java source code.

`src/main/resources`

This folder contains configuration files and static resources.

`pom.xml`

This is the Maven configuration file.

It contains project information and dependencies.

`application.properties`

This file is used to configure our Spring Boot application.

For example, we can change the server port from here.

Main Application Class

This is the class from where the Spring Boot application starts.

It usually contains the `main()` method.

Example:

```
@SpringBootApplication
public class FirstSpringBootApplication {

    public static void main(String[] args) {
        SpringApplication.run(FirstSpringBootApplication.clas
s, args);
    }
}
```

Coder Army

Coder Army

For now, we only need to understand that this class starts our Spring Boot application.

We will understand `@SpringBootApplication` deeply later.

---

12. What is a Controller?

A **Controller** is the entry point for incoming web requests.

Simple explanation:

> A controller is like the receptionist of our application.

When a request comes to the application, the controller receives it and decides what response should be sent back.

For example:

```
Browser sends request to /hello
Controller receives the request
Controller returns Hello World
```

---

13. What is `@RestController`?

`@RestController` is an annotation in Spring.

It tells Spring:

```
This class can handle HTTP requests and return data directly as a response.
```

Example:

```java
@RestController
public class HelloController {

}
```

Coder Army

This annotation makes the class eligible to receive web requests.

We will understand annotations and controllers in detail later.

---

14. Creating Our First Endpoint

Now we can create a simple controller.

Example:

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World";
    }
}
```

Here:

```
@RestController
```

tells Spring that this class can handle HTTP requests.

```
@GetMapping("/hello")
```

tells Spring that this method should run when a GET request comes to `/hello`.

```
sayHello()
```

is the Java method that gets executed.

```
return "Hello World";
```

Coder Army

is the response sent back to the browser.

So when we open:

```
http://localhost:8080/hello
```

the browser displays:

```
Hello World
```

---

15. What Happened When We Ran the Application?

When we run the Spring Boot application, we may see a message like:

```
Tomcat started on port 8080
```

This is very important.

We did not install Tomcat separately.

We did not write socket programming code.

We did not manually create a server.

Still, Tomcat started.

Why?

Because Spring Boot includes and configures an embedded Tomcat server for us when we use the web dependency.

Spring Boot starts the application and also starts the embedded Tomcat server.

Tomcat is the actual server listening on the port.

So the correct understanding is:

Coder Army

```
Spring Boot starts and configures the application.
Embedded Tomcat listens for HTTP requests.
Spring MVC maps the request to the correct controller method.
```

We will understand this complete flow properly in upcoming lectures.

---

16. Changing the Port Number

By default, Spring Boot runs on port:

```
8080
```

We can change the port using the `application.properties` file.

Example:

```
server.port=9090
```

After changing this and restarting the application, the app will run on:

```
http://localhost:9090
```

Now our endpoint will be:

```
http://localhost:9090/hello
```

Important point:

> Spring Boot reads the configuration, but the embedded Tomcat server listens on the configured port.

So if we set:

Coder Army

```
server.port=9090
```

Tomcat starts listening on port `9090`.

---

17. Complete Flow of Our First Spring Boot Application

The complete flow looks like this:

```
Browser sends request to localhost:8080/hello
        |
        v
Request reaches embedded Tomcat
        |
        v
Spring MVC checks the available mappings
        |
        v
/hello is matched with sayHello()
        |
        v
sayHello() returns "Hello World"
        |
        v
Response goes back to browser
        |
        v
Browser displays Hello World
```

This is what happens behind one simple line in the browser:

```
localhost:8080/hello
```

---

18. Why This Feels So Fast

In less than a few minutes, we created a working web application.

We wrote very little code:

Coder Army

```java
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World";
    }
}
```

But behind this simple code, Spring Boot did many things for us:

```
Created application context
Started embedded Tomcat
Configured Spring MVC
Detected controller class
Registered /hello endpoint
Handled HTTP request and response
Returned data to browser
```

This is the power of Spring Boot.

It allows developers to focus more on business logic instead of repeating infrastructure setup again and again.

---

19. Then Why Learn Internals?

A common question is:

> If Spring Boot does everything automatically, why should we learn Tomcat, Servlets, Maven, Spring Core, MVC, IoC, and Dependency Injection?

The answer is simple:

Spring Boot helps us start fast, but internals help us solve real problems.

Writing basic annotations is easy.

For example:

Coder Army

```
@RestController
@GetMapping("/hello")
```

Many tools can generate this code today.

But real engineering starts when something does not work.

---

20. Real Problems Require Internal Understanding

Suppose we create an endpoint, but the browser shows:

```
404 Not Found
```

Now we need to ask:

```
Did Spring detect my controller?
Is the package structure correct?
Did component scanning happen?
Was the bean created?
Did Spring MVC register the mapping?
Is the URL correct?
Did Tomcat receive the request?
```

These questions cannot be answered by memorizing annotations.

They require understanding the internal flow.

Example: Application Starts Slowly

Suppose the application takes 10 seconds to start.

Spring Boot will start the application, but we still need to understand what may be happening:

```
Too many beans being created
Database connection taking time
```

Coder Army

```
Auto-configuration doing extra work
Heavy startup logic running
External service call blocking startup
```

To debug this, we need to understand Spring Boot internals.

Example: Controller Does Not Receive Request

Suppose the browser sends a request, but the controller never receives it.

Possible reasons could be:

```
Wrong port
Wrong URL
Application not running
Controller not detected
Incorrect annotation
Context path configured
Request blocked by security
```

Again, solving this requires understanding the layers behind Spring Boot.

21. The Industry Reality

In real projects, companies do not only pay engineers for writing:

```
@RestController
```

That part is easy.

AI tools, code generators, and IDEs can write basic code.

Companies pay engineers for understanding:

```
Why something is not working
How different layers communicate
How to debug production issues
How to design maintainable systems
How to make the application reliable
```

Coder Army

That is why learning the internals is important.

Spring Boot gives speed.

Spring fundamentals give control.

---

22. Final Summary

In this lecture, we built our first Spring Boot web application.

We learned:

```
A browser sends a request to a server.
A port identifies the application running inside a machine.
localhost means our own machine.
localhost:8080 means our own machine on port 8080.
Spring Initializr helps us generate a Spring Boot project quickly.
A dependency is an external library required by our project.
@RestController marks a class as capable of handling web requests.
@GetMapping("/hello") maps a GET request to a Java method.
Spring Boot starts and configures the application.
Embedded Tomcat listens for HTTP requests.
The browser receives the response and displays it.
```

The most important thing to remember:

> Today we built a web application very quickly, but many powerful concepts were working behind the scenes.

In the upcoming lectures, we will slowly understand those concepts one by one:

```
Maven
Servlets
Tomcat
Spring Core
IoC
Dependency Injection
Beans
Spring MVC
Spring Boot auto-configuration
```

Once these foundations are clear, Spring Boot will no longer feel magical.

It will start making logical sense.

Coder Army

Coder Army

Lecture 3 Notes: Maven Deep Dive

> **Topic:** JAR Files, Maven, `pom.xml`, Dependencies, Repositories, and Maven Lifecycle
>
> **Goal:** Understand why Maven exists and how it helps Java projects manage structure, dependencies, builds, and packaging.

---

1. The Basic Journey of Java Code

Before understanding Maven, we first need to understand the basic flow of a Java program.

```
Hello.java  -- compiled by javac -->  Hello.class  -- run by JVM --> output
```

At the core Java level:

```
.java file   = source code written by the developer
.class file  = compiled bytecode understood by the JVM
```

So, Java code is not directly executed as `.java` files. First, it is compiled into `.class` files, and then the JVM runs those `.class` files.

---

2. What Happens in a Real Java Project?

A real project usually does not have only one Java class.

It may have many files like:

```
User.java
Order.java
Payment.java
EmailService.java
```

Coder Army

```
InvoiceGenerator.java
Main.java
```

After compilation, each Java file becomes a `.class` file:

```
User.class
Order.class
Payment.class
EmailService.class
InvoiceGenerator.class
Main.class
```

Now imagine sharing this project with another developer.

Sending many separate `.class` files is not practical because:

- Files can get misplaced.
- Package/folder structure can break.
- Some files may be missed.
- Resource files may not be included.
- Running the project becomes confusing.

Java needed a standard way to bundle compiled code and related files together.

That standard bundle is called a **JAR file**.

---

3. What is a JAR File?

JAR stands for **Java Archive**.

A JAR file is basically a packaged bundle for Java projects. You can think of it like a ZIP file designed specifically for Java.

A JAR file can contain:

```
.class files
folders/packages
images
properties files
```

Coder Army

```
configuration files
metadata
```

In simple words:

> A JAR file is a standard way to package Java compiled code and related resources into one file.

---

4. Why Do We Need a JAR File?

There are two major reasons why JAR files are important.

Reason 1: To Share Java Code Easily

Suppose you create a useful calculator library.

Instead of sending all compiled `.class` files separately, you can package them into:

```
calculator.jar
```

Now another developer can add this JAR to their project and use your compiled code.

This is how many Java libraries are shared.

---

5. Library vs Application

Before moving further, it is important to understand the difference between a **library** and an **application**.

Term	Meaning
Library	Code created for other developers to use inside their projects. It usually does not run independently.
Application	Code created for the end user. It can run as a complete program, usually with a `main()` method or an application entry point.

Example:

Coder Army

```
calculator.jar
```

If this JAR only provides reusable calculator functions, it is a **library JAR**.

But if it contains a complete runnable program, it is an **application JAR**.

---

6. Reason 2: To Use External Libraries

When we use libraries like:

```
Spring Core
MySQL Connector
Hibernate
Jackson
```

we are usually using someone else's JAR files.

For example:

> When we say, "I am using Spring Core", practically our project is using Spring Core JAR files.

Similarly:

> When we say, "I am using MySQL driver", our project is using the MySQL driver JAR.

So, a library is often just compiled Java code packaged as a JAR file.

---

7. Important Note: Library JAR vs Spring Boot Executable JAR

A normal library JAR usually does **not** contain all of its dependency JARs inside it.

For example, if you create a reusable library, its JAR may contain your compiled classes, but its dependency information is usually managed separately through Maven using `pom.xml`.

Coder Army

However, a Spring Boot application is commonly packaged as an **executable JAR**. In that case, Spring Boot can package the application code along with required dependencies so that the application can run as one complete executable JAR.

So:

```
Normal library JAR
→ Usually contains your compiled code, not all external dependency JARs inside it.

Spring Boot executable JAR
→ Can contain application code + required dependencies, making it runnable as one JAR.
```

This is why we often call Spring Boot output a **fat JAR** or **executable JAR**.

---

8. External Libraries and Classpath

Suppose your Java code uses an external JAR.

Conceptually, the flow looks like this:

```
Your Java code
     +
External JAR file
     ↓
Compiled together
     ↓
Run by JVM
```

During compile time and runtime, Java must know where to find the required classes.

For example, Java may need to know:

> "This class is not in my project. It is inside this external JAR."

This connection is handled using something called the **classpath**.

---

9. What is Classpath?

The **classpath** is the place where Java searches for classes.

Coder Army

Java searches for classes in:

- Your own project classes
- External JAR files
- Other configured class locations

If the class is present in your project, Java can find it there.

If the class is inside an external JAR, Java must know the location of that JAR.

In simple words:

> Classpath tells Java where to search for required classes.

---

10. Where Maven Comes Into the Picture

If a project needs only one external JAR, we can manually download it and add it to the project.

But real projects usually need many third-party libraries.

Then many problems appear:

- Where should we download these JARs from?
- Which version should we use?
- What if one JAR depends on another JAR?
- What if two teammates use different versions?
- What if we forget to add one required JAR?
- What if version 1 works, but version 2 breaks compatibility?

This is exactly where **Maven** becomes useful.

Maven says:

> Do not manually download and manage JAR files. Tell me what dependency your project needs, and I will download and manage the required JARs for you.

---

11. What is Maven?

Coder Army

Maven is a project management and build automation tool for Java projects.

It is important to understand that Maven does not exist only because of Spring.

Maven is independent of Spring.

You can use Maven with:

- Core Java projects
- Spring projects
- Spring Boot projects
- Hibernate projects
- Many other Java-based projects

In simple words:

> Maven helps us manage project structure, dependencies, builds, packaging, and project configuration in a standard way.

---

12. What Maven Does Practically

Maven mainly helps with these things:

1. It gives a standard project structure.
2. It compiles Java code.
3. It runs tests.
4. It creates JAR/WAR files.
5. It downloads external JAR libraries.
6. It manages dependency versions.
7. It uses plugins to perform build-related tasks.

This makes Java projects more consistent and easier to work with, especially in teams.

---

13. Maven and IDEs

Coder Army

Maven can be used from the command line.

For example:

```
mvn clean install
```

But most modern IDEs also support Maven directly, such as:

- IntelliJ IDEA
- Eclipse
- VS Code

Every IDE may have its own project structure, but Maven gives us a standard structure that works across tools.

That is why Maven is helpful in real projects.

---

14. Convention Over Configuration

Maven follows a principle called:

> Convention over Configuration

This means Maven already assumes a standard project structure.

If we follow that structure, we do not need to manually configure everything.

For example, Maven expects main Java code inside:

```
src/main/java
```

and test code inside:

```
src/test/java
```

Because of this standard convention, Maven automatically knows where to find source code, test code, resources, and where to place generated output.

Coder Army

15. Standard Maven Project Structure

A typical Maven project looks like this:

```
my-maven-project/
|
├── pom.xml
|
├── src/
|   ├── main/
|   |   ├── java/
|   |   └── resources/
|   |
|   └── test/
|       ├── java/
|       └── resources/
|
└── target/
```

Now let us understand each part.

`pom.xml`

This file contains information about the complete Maven project.

It tells Maven:

- Project name
- Project version
- Required dependencies
- Plugins
- Build configuration
- Parent configuration, if any

`src/main/java`

This folder contains the main Java source code of the application.

Example:

Coder Army

```
Controller classes
Service classes
Repository classes
Utility classes
Main application class
```

`src/main/resources`

This folder contains non-Java files needed by the main application.

Example:

```
application.properties
application.yml
configuration files
static files
templates
```

`src/test/java`

This folder contains test code.

Example:

```
unit tests
integration tests
```

`src/test/resources`

This folder contains resources needed only during testing.

Example:

```
test configuration files
test data files
```

`target`

Coder Army

The `target` folder is created by Maven during the build process.

It can contain:

```
compiled .class files
test reports
generated sources
final JAR/WAR file
temporary build files
```

If we delete the `target` folder, Maven can recreate it during the next build.

So, `target` is generated output, not source code.

---

16. The Most Important File in Maven: `pom.xml`

Every Maven project has a file called:

```
pom.xml
```

POM stands for **Project Object Model**.

The `pom.xml` file is the heart of a Maven project.

It tells Maven:

- What the project is
- Which version the project has
- What type of output should be created
- Which external libraries are needed
- Which plugins should be used
- Whether there is any parent configuration

---

17. Basic Structure of `pom.xml`

Every `pom.xml` starts with a root `<project>` tag.

A simple POM may look like this:

Coder Army

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instanc
e"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.
0 https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.coderarmy</groupId>
    <artifactId>calculator-app</artifactId>
    <version>1.0.0</version>

</project>
```

The extra `xmlns` lines define the XML namespace and structure.

For beginners, we do not need to deeply focus on them at this stage.

---

18. `<modelVersion>`

```
<modelVersion>4.0.0</modelVersion>
```

This tells Maven which POM model version this file follows.

Most Maven projects use:

```
4.0.0
```

---

19. Maven Coordinates

These three tags are very important:

```
<groupId>com.coderarmy</groupId>
<artifactId>calculator-app</artifactId>
```

Coder Army

```
<version>1.0.0</version>
```

Together, they are called **Maven coordinates**.

Maven uses these coordinates to uniquely identify a project or dependency.

`<groupId>`

`groupId` usually represents the organization, company, or domain that owns the project.

Example:

```
<groupId>com.coderarmy</groupId>
```

`<artifactId>`

`artifactId` is the name of the project or module.

Example:

```
<artifactId>calculator-app</artifactId>
```

`<version>`

`version` tells which version of the project is being built.

Example:

```
<version>1.0.0</version>
```

---

20. What is `SNAPSHOT` in Maven?

Sometimes we see versions like:

```
<version>1.0.0-SNAPSHOT</version>
```

`SNAPSHOT` means the project is still under development.

Coder Army

It is not considered a final stable release yet.

In simple words:

```
1.0.0          → stable release version
1.0.0-SNAPSHOT → development version
```

---

21. `<packaging>` Tag

The `<packaging>` tag tells Maven what type of output should be created.

Example:

```
<packaging>jar</packaging>
```

Common packaging types:

Packaging	Meaning
`jar`	Creates a JAR file
`war`	Creates a WAR file, usually for traditional web applications
`pom`	Used for parent or aggregator projects

If we do not mention packaging, Maven defaults to:

```
jar
```

---

22. `<properties>` Tag

The `<properties>` tag is used to define reusable values in the POM.

Example:

```
<properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.source
```

```
Encoding>
</properties>
```

Here, we are telling Maven:

- Use Java 17 source code.
- Compile it for Java 17.
- Use UTF-8 encoding.

Properties make the POM cleaner because the same value can be reused in multiple places.

---

23. `<dependencies>` Tag

The `<dependencies>` section contains external libraries required by the project.

Example:

```
<dependencies>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>8.x.x</version>
    </dependency>
</dependencies>
```

Each dependency is also identified using Maven coordinates:

```
groupId + artifactId + version
```

When Maven reads this dependency, it downloads the required JAR file from a Maven repository.

---

24. Dependency Example: Hibernate

A Hibernate dependency may look like this:

Coder Army

Coder Army

```
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.x.x</version>
</dependency>
```

This tells Maven that the project needs Hibernate Core.

Maven will download the Hibernate JAR and also manage its required supporting dependencies.

---

25. Dependency Example: Spring Boot Starter

In Spring Boot projects, we commonly use starter dependencies.

Example:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

A starter dependency does not mean only one JAR.

It usually brings a group of related dependencies needed for a particular feature.

For example, `spring-boot-starter-web` brings dependencies related to web application development.

---

26. Transitive Dependencies

Sometimes one dependency needs other dependencies to work.

For example:

```
Your project needs A
A needs B
B needs C
```

Coder Army

In Maven, if you add dependency A, Maven can automatically download B and C also.

These are called **transitive dependencies**.

In simple words:

> Transitive dependencies are dependencies of your dependencies.

This is one of the biggest reasons Maven is useful.

Without Maven, we would have to manually find and download every required JAR.

---

27. Maven Plugins

Dependencies are libraries that our project uses.

Plugins are tools that Maven uses to perform tasks.

Maven plugins can help with:

- Compiling code
- Running tests
- Creating JAR files
- Creating reports
- Running a Spring Boot application
- Packaging the application

For example, the Spring Boot Maven plugin helps package and run Spring Boot applications.

In simple words:

```
Dependency → used by our application code
Plugin     → used by Maven during the build process
```

---

28. Where Does Maven Download JAR Files From?

Coder Army

Maven downloads dependencies from Maven repositories.

A **Maven repository** is a storage place where Maven artifacts are kept.

Artifacts can include:

```
JAR files
WAR files
POM files
plugins
other build artifacts
```

---

29. Types of Maven Repositories

At a beginner level, remember these three types:

1. Local repository
2. Maven Central repository
3. Remote repository

---

30. Local Repository

The local repository is a folder on your own computer.

Default location:

```
Mac/Linux:
~/.m2/repository

Windows:
C:\Users\<your-username>\.m2\repository
```

The local repository has two main jobs:

1. It caches downloaded dependencies.
2. It stores your own locally installed artifacts.

When Maven downloads a dependency for the first time, it stores it in the local repository.

Coder Army

Because of this, the first build may be slower, but later builds are faster because dependencies are already cached.

---

31. What Happens During `mvn install`?

When we run:

```
mvn install
```

Maven builds the project and stores the final artifact in the local Maven repository.

Example location:

```
~/.m2/repository/com/example/MavenDemo/1.0/
```

This allows other local Maven projects on the same machine to use that artifact.

---

32. Maven Central Repository

Maven Central is a huge public repository where many open-source Java libraries are published.

Libraries like these are commonly available through Maven Central:

```
Spring
Hibernate
Jackson
JUnit
MySQL Connector
```

When we add a public dependency in `pom.xml`, Maven usually downloads it from Maven Central if it is not already available locally.

---

33. Remote Repository

A remote repository is any Maven repository that is not on your local machine.

It may be available through:

Coder Army

- The internet
- A company network
- A private repository server

Examples:

```
Maven Central
Company Nexus repository
Company Artifactory repository
GitHub Packages
```

---

34. Why Companies Use Private Repositories

In companies, not every JAR is public.

A company may create internal libraries that should only be used inside the organization.

For such cases, companies use private Maven repositories like:

```
Nexus
Artifactory
GitHub Packages
```

A private repository can be configured in `pom.xml` like this:

```xml
<repositories>
    <repository>
        <id>company-repo</id>
        <url>https://repo.company.com/maven2</url>
    </repository>
</repositories>
```

---

35. How Maven Searches for a Dependency

Suppose our `pom.xml` has this dependency:

Coder Army

```xml
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.0</version>
</dependency>
```

Maven roughly follows this process:

```
1. Read pom.xml
2. See that a dependency is required
3. Check the local repository first
4. If found locally, use it
5. If not found locally, check remote repositories
6. Download the JAR and POM
7. Store them in the local repository
8. Use them in the project build
```

That is why the first build is usually slower than later builds.

The first time, Maven downloads dependencies.

After that, Maven uses the cached dependencies from the local `.m2` repository.

---

36. What is `settings.xml`?

`settings.xml` is Maven's user-level configuration file.

It is usually found inside the `.m2` folder.

Common location:

```
~/.m2/settings.xml
```

It can contain user-specific Maven settings such as:

- Repository credentials
- Proxy settings
- Mirror configuration

Coder Army

- Custom local repository location

For beginners, it is enough to remember:

> `pom.xml` is project-level configuration, while `settings.xml` is user/machine-level Maven configuration.

---

37. Why Deleting `.m2/repository` Sometimes Fixes Dependency Errors

Sometimes dependency files inside the local Maven repository may become corrupted or incomplete.

For example:

- Download failed in between
- Internet disconnected during dependency download
- Partial JAR file got saved
- Metadata became inconsistent

In such cases, deleting the problematic dependency folder from `.m2/repository` forces Maven to download it again.

That is why deleting `.m2/repository` or a specific dependency folder sometimes fixes Maven dependency errors.

However, deleting the entire `.m2/repository` means Maven will need to download all dependencies again.

---

38. Maven Lifecycle

A Java project has a build journey.

For example:

```
Check project → Compile code → Run tests → Create JAR → Store/Deploy artifact
```

Maven calls this ordered build journey a **lifecycle**.

CoderArmy

In simple words:

> A Maven lifecycle is a standard sequence of steps Maven follows to build and manage a project.

---

39. The Most Important Rule of Maven Lifecycle

The heart of Maven lifecycle is this rule:

> When we run a Maven phase, Maven automatically runs all earlier phases of that lifecycle before it.

For example, if we run:

```
mvn package
```

Maven does not only run `package`.

It runs the earlier required phases first, such as:

```
validate → compile → test → package
```

Similarly, if we run:

```
mvn install
```

Maven runs:

```
validate → compile → test → package → verify → install
```

---

40. The Three Main Maven Lifecycles

Maven has three main lifecycles:

```
1. Clean Lifecycle
2. Default Lifecycle
```

```
3. Site Lifecycle
```

At beginner level, we can understand them like this:

Lifecycle	Purpose
Clean	Cleans old build output
Default	Builds, tests, packages, installs, and deploys the project
Site	Generates project documentation and reports

---

41. Clean Lifecycle

The clean lifecycle is used to remove old build output.

Common command:

```
mvn clean
```

Usually, this deletes the `target` folder.

This is useful when we want a fresh build.

---

42. Default Lifecycle

The default lifecycle is the main build lifecycle.

Important phases:

```
validate → compile → test → package → verify → install → deploy
```

Each phase represents one step in the build journey.

---

43. Important Default Lifecycle Phases

Phase	Meaning
`validate`	Checks whether the project is valid
`compile`	Compiles `.java` files into `.class` files

Coder Army

Coder Army

Phase	Meaning
`test`	Runs test cases
`package`	Creates JAR/WAR file
`verify`	Performs additional checks on the packaged output
`install`	Stores artifact in local `.m2` repository
`deploy`	Uploads artifact to a remote Maven repository

44. `validate` Phase

The `validate` phase checks whether the project is structurally valid.

It checks things like:

```
Is pom.xml present?
Is the POM readable?
Are required project details available?
Is the project structure valid enough to continue?
```

Command:

```
mvn validate
```

45. `compile` Phase

The `compile` phase compiles the main Java code.

Command:

```
mvn compile
```

Flow:

```
src/main/java  →  target/classes
```

Coder Army

This means Java source files are compiled into `.class` files and placed inside `target/classes`.

---

46. `test` Phase

The `test` phase runs the test cases of the project.

Command:

```
mvn test
```

Maven compiles and runs tests from:

```
src/test/java
```

This helps verify that the code works as expected before packaging.

---

47. `package` Phase

The `package` phase creates the final packaged output.

Command:

```
mvn package
```

For a normal Java project, this usually creates a JAR file inside the `target` folder.

Example:

```
target/MavenDemo-1.0.jar
```

For web applications, it may create a WAR file depending on the packaging type.

---

48. `verify` Phase

The `verify` phase checks whether the packaged output is valid.

Coder Army

Command:

```
mvn verify
```

In bigger projects, this phase can include:

```
integration tests
code quality checks
coverage checks
extra validation
```

For beginners, remember:

> `verify` is used to perform additional checks before installing or deploying the artifact.

---

49. `install` Phase

The `install` phase builds the project and stores the final artifact in the local Maven repository.

Command:

```
mvn install
```

The artifact is stored in:

```
~/.m2/repository
```

This is useful when another local project needs to use this project as a dependency.

---

50. `deploy` Phase

The `deploy` phase builds the project and uploads the final artifact to a remote Maven repository.

Coder Army

Command:

```
mvn deploy
```

Remote repositories can include:

```
company Nexus
company Artifactory
GitHub Packages
remote Maven repository
```

This is different from pushing code to GitHub.

GitHub usually stores source code.

A Maven remote repository stores built artifacts like JAR files so that other projects can use them as dependencies.

---

51. Site Lifecycle

The site lifecycle is used to generate project documentation and reports.

Command:

```
mvn site
```

Output usually goes inside:

```
target/site
```

This is separate from the normal build flow.

---

52. What Happens When We Run `mvn clean install`?

When we run:

```
mvn clean install
```

Coder Army

Maven runs two lifecycle commands together:

```
clean + install
```

First, Maven runs the clean lifecycle and deletes old build output, usually the `target` folder.

Then, Maven runs the default lifecycle up to the `install` phase:

```
validate → compile → test → package → verify → install
```

So the complete idea is:

```
mvn clean install
= clean old output
+ build the project again
+ run tests
+ package the project
+ install the artifact into local .m2 repository
```

---

53. Maven Archetypes

A Maven archetype is a project template.

It provides a ready-made structure for creating a new project.

In simple words:

> A Maven archetype is a starter template with predefined folder structure and configuration.

It can help generate projects quickly instead of creating everything manually.

Example idea:

```
Template → project structure + basic configuration + boilerplate files
```

Modern Spring Boot projects are usually created using Spring Initializr, but Maven archetypes are still useful to understand as a Maven concept.

Coder Army

Final Revision Summary

In this lecture, we understood the journey from basic Java compilation to Maven project management.

Key points:

- Java source code is compiled into `.class` files.
- A JAR file packages compiled Java code and resources.
- External libraries are usually available as JAR files.
- Java uses classpath to find project classes and external classes.
- Manually managing many JAR files is difficult.
- Maven solves this by managing project structure, dependencies, builds, and packaging.
- `pom.xml` is the heart of a Maven project.
- Maven dependencies are identified using `groupId`, `artifactId`, and `version`.
- Maven downloads dependencies from repositories.
- The local repository is stored inside `.m2/repository`.
- Maven Central is a public repository for open-source Java libraries.
- Remote repositories can be public or private.
- Maven lifecycle defines the standard build flow.
- Running a Maven phase also runs all earlier phases in that lifecycle.
- `mvn clean install` cleans old output and builds the project up to the install phase.
- Maven archetypes are templates for creating projects quickly.

Coder Army

One-Line Takeaway

Maven exists because real Java projects need a standard way to manage structure, dependencies, builds, packaging, and sharing of artifacts.

Coder Army

Lecture 4 : Dependency Injection, IoC & Loosely Coupled Design

1. Why Do We Need Spring Core?

Java already gives us many powerful building blocks:

- Classes
- Objects
- Constructors
- Interfaces
- Inheritance
- Packages

So a natural question appears:

> If Java already gives us all these features, why do we need Spring Core?

Spring Core is not mainly about building web applications. That part comes later with Spring MVC and Spring Boot.

The core idea of Spring is simpler:

> Spring helps us create objects, manage dependencies, and connect objects together in a clean and scalable way.

In small programs, object creation looks easy. But in real applications, one class depends on another class, which depends on another class, and this chain keeps growing.

That is where Spring Core becomes useful.

Coder Army

2. Understanding Dependency

Suppose we have an `EmailService` class:

```
class EmailService {

    public void sendEmail() {
        System.out.println("Email sent");
    }
}
```

Now suppose `OrderService` needs to send an email after placing an order.

```
class OrderService {

    private EmailService emailService = new EmailService();

    public void placeOrder() {
        System.out.println("Order placed");
        emailService.sendEmail();
    }
}
```

Here, `OrderService` depends on `EmailService`.

In simple words:

> A dependency is something a class needs in order to complete its work.

So in this example:

```
private EmailService emailService = new EmailService();
```

means:

> `OrderService` is creating its own dependency.

This works, but it creates a design problem.

Coder Army

3. The Problem: Tight Coupling

There are two broad design styles:

```
Tightly Coupled Design  → Hard to change
Loosely Coupled Design  → Easier to change
```

Real-Life Example

Imagine a person wants to travel from Delhi to Chandigarh.

Tightly Coupled Thinking

> "I can travel only by this one specific car, with this one specific driver, from this one specific company."

If that car is unavailable, the person is stuck.

Loosely Coupled Thinking

> "I need transportation. It can be a car, bus, train, or cab."

Here, the person is not dependent on one fixed option. The person only cares about the service: transportation.

---

4. Tight Coupling in Code

In our previous example, `OrderService` directly creates `EmailService`.

```java
class OrderService {

    private EmailService emailService = new EmailService();

    public void placeOrder() {
        System.out.println("Order placed");
```

```
        emailService.sendEmail();
    }
}
```

This means:

```
OrderService
    |
    | directly creates
    v
EmailService
```

Now suppose the requirement changes.

Instead of email, we now need to send an SMS.

```
public class SmsService {

    public void sendSms() {
        System.out.println("SMS notification sent to custome
r");
    }
}
```

Now we must modify `OrderService`.

```
public class OrderService {

    private SmsService smsService = new SmsService();

    public void placeOrder() {
        System.out.println("Order placed successfully");
        smsService.sendSms();
    }
}
```

The problem is not that the notification requirement changed.

Coder Army

Coder Army

The real problem is:

> A change in notification logic forced us to modify `OrderService`.

That means `OrderService` is tightly coupled to a specific notification class.

---

5. What Exactly Is Tight Coupling?

Tight coupling means:

> One class is directly dependent on a specific concrete class.

In our case:

```
OrderService is directly dependent on EmailService.
```

This makes the code harder to change, test, and reuse.

---

6. First Improvement: Use an Interface

To make the design better, we can introduce an interface.

```java
public interface NotificationService {

    void sendNotification();
}
```

Now different notification services can implement this interface.

```java
public class EmailService implements NotificationService {

    @Override
    public void sendNotification() {
        System.out.println("Email sent");
```

Coder Army

```
    }
}
```

```
public class SmsService implements NotificationService {

    @Override
    public void sendNotification() {
        System.out.println("SMS sent");
    }
}
```

Now `OrderService` can depend on the interface instead of a concrete class.

```
public class OrderService {

    private NotificationService notificationService = new Ema
ilService();

    public void placeOrder() {
        System.out.println("Order placed successfully");
        notificationService.sendNotification();
    }
}
```

This is better than before because the variable type is now an interface:

```
NotificationService
```

But there is still one issue.

---

7. Interface Alone Is Not Enough

Even though we are using an interface, this line is still a problem:

Coder Army

```
private NotificationService notificationService = new EmailSe
rvice();
```

Why?

Because the object creation is still concrete:

```
new EmailService()
```

So `OrderService` is still deciding which implementation to use.

The important point is:

> Creating objects is not the problem.
>
> Creating them in the wrong place is the problem.

`OrderService` is a business class. Its job should be order-related logic, not deciding which notification object to create.

Currently, `OrderService` is doing two things:

1. Handling order-related logic
2. Creating and deciding the notification service

This breaks two important design principles:

```
S → Single Responsibility Principle
O → Open-Close Principle
```

The main idea is:

> Dependency is normal. Tight dependency is the problem.

---

8. The Solution: Dependency Injection

Instead of creating the dependency inside `OrderService`, we provide the dependency from outside.

Coder Army

```
public class OrderService {

    private NotificationService notificationService;

    public OrderService(NotificationService notificationServi
ce) {
        this.notificationService = notificationService;
    }

    public void placeOrder() {
        System.out.println("Order placed successfully");
        notificationService.sendNotification();
    }
}
```

Now `OrderService` does not create `EmailService` or `SmsService`.

It simply says:

> "Give me something that can send a notification."

That "something" is represented by the `NotificationService` interface.

---

9. Object Creation Happens Outside

Now we create the objects in `Main`.

```
public class Main {

    public static void main(String[] args) {

        NotificationService notificationService = new EmailSe
rvice();

        OrderService orderService = new OrderService(notifica
tionService);
```

```
        orderService.placeOrder();
    }
}
```

Now the dependency is provided from outside.

This means:

```
OrderService depends only on NotificationService.
OrderService does not create EmailService.
OrderService receives the dependency from outside.
```

This is called **Dependency Injection**.

---

10. What Is Dependency Injection?

Dependency Injection means:

> A class receives the objects it depends on from outside, instead of creating those objects itself.

A simpler way to remember it:

> Do not create your dependency. Receive your dependency.

Or:

> A class should ask for what it needs, not build everything by itself.

Dependency Injection is not only a Spring concept.

It is a design principle that we can use in plain Java also.

Spring did not invent Dependency Injection. Spring automates it.

---

11. Benefits of Dependency Injection

Coder Army

Coder Army

Benefit 1: Easy to Change Implementation

If we want to switch from email to SMS, we do not need to change `OrderService`.

We only change the object passed from outside.

```
NotificationService notificationService = new SmsService();
OrderService orderService = new OrderService(notificationServ
ice);
```

`OrderService` remains unchanged.

Benefit 2: Easier to Test

For testing, we may not want to send a real email or SMS.

So we can create a fake implementation.

```
public class FakeNotificationService implements NotificationS
ervice {

    @Override
    public void sendNotification() {
        System.out.println("Fake notification for testing");
    }
}
```

Now we can test `OrderService` without depending on real email or SMS logic.

Benefit 3: More Reusable Code

The same `OrderService` can work with multiple implementations:

```
EmailService
SmsService
WhatsAppService
FakeNotificationService
```

Coder Army

This makes the design flexible and reusable.

12. Types of Dependency Injection

There are mainly three common types of Dependency Injection.

1. Constructor Injection

In constructor injection, the dependency is provided through the constructor.

```
public class OrderService {

    private NotificationService notificationService;

    public OrderService(NotificationService notificationServi
ce) {
        this.notificationService = notificationService;
    }
}
```

Constructor injection is usually preferred because it makes required dependencies clear.

If `OrderService` cannot work without `NotificationService`, constructor injection is a good choice.

2. Setter Injection

In setter injection, the dependency is provided through a setter method.

```
public class OrderService {

    private NotificationService notificationService;

    public void setNotificationService(NotificationService no
tificationService) {
```

Coder Army

```
        this.notificationService = notificationService;
    }

    public void placeOrder() {
        System.out.println("Order placed successfully");
        notificationService.sendNotification();
    }
}
```

Setter injection is useful when the dependency is optional or can be changed later.

---

3. Field Injection

In Spring, we will also discuss about field injection later.

---

13. What Is IoC?

IoC stands for **Inversion of Control**.

To understand IoC, first ask:

> In the earlier code, who was controlling the creation of `EmailService`?

The answer is:

```
OrderService was controlling object creation.
```

So the control was inside `OrderService`.

After Dependency Injection, `OrderService` no longer creates `EmailService`.

The object is created outside and passed to `OrderService`.

This movement of control is called **Inversion of Control**.

---

14. Simple Definition of IoC

Coder Army

Earlier:

> The class created what it needed.

Now:

> The class receives what it needs.

That reversal is called **Inversion of Control**.

It is called “inversion” because the normal control flow is reversed.

Earlier:

```
OrderService
    |
    | creates
    v
EmailService
```

After Dependency Injection:

```
Main
 |
 | creates
 v
EmailService

Main provides EmailService
 |
 | to
 v
OrderService
```

So the control moved from inside the class to outside the class.

---

15. Relationship Between IoC and DI

IoC and DI are closely related, but they are not exactly the same.

Coder Army

```
IoC → Principle
DI  → Technique to achieve that principle
```

In simple words:

> IoC is the idea.
>
> Dependency Injection is one way to implement that idea.

When we give dependencies from outside instead of creating them inside the class, we are using Dependency Injection to achieve Inversion of Control.

---

16. Where Does Spring Fit In?

In plain Java, we moved object creation outside `OrderService`.

But now `Main` is doing all the object creation and wiring.

```
NotificationService notificationService = new EmailService();
OrderService orderService = new OrderService(notificationServ
ice);
```

This is fine for small applications.

But in large applications, there may be hundreds of classes and dependencies.

If `Main` creates and connects everything manually, it becomes messy.

So we need something that can:

```
Create objects
Manage objects
Connect objects together
```

That is where Spring comes in.

---

17. Spring IoC Container

Coder Army

Spring provides an external system called the **Spring IoC Container**.

The Spring IoC Container is responsible for:

```
Creating objects
Managing objects
Injecting dependencies
Connecting classes together
```

In plain Java, `Main` was acting like a small container.

In Spring, the Spring IoC Container does this work automatically.

---

18. What Does Spring Core Do?

At a high level, Spring Core does three major things:

```
1. Creates objects
2. Manages objects
3. Connects objects together
```

This is the foundation of Spring.

Before learning Spring MVC, Spring Boot, Spring Data JPA, or Spring Security, it is important to understand this core idea.

---

19. What Is a Bean?

In normal Java, we call them objects.

In Spring, objects managed by Spring are called **beans**.

Important point:

> Every Spring bean is an object, but every object is not necessarily a Spring bean.

Example:

Coder Army

```
EmailService emailService = new EmailService();
```

This is a normal Java object.

But if Spring creates and manages `EmailService`, then it becomes a Spring bean.

---

20. Manual Java Flow vs Spring Flow

Plain Java Flow

```
Main
 |
 | creates
 v
EmailService

Main
 |
 | creates
 v
OrderService

Main injects EmailService into OrderService
```

In this flow, `Main` is responsible for creating and connecting objects.

Spring Flow

```
Spring IoC Container
 |
 | creates
 v
EmailService Bean

Spring IoC Container
 |
 | creates and injects dependency
 v
OrderService Bean
```

In this flow, Spring creates the objects and connects them automatically.

21. Final Summary

Dependency Injection helps us design classes that do not create their own dependencies.

Instead, dependencies are provided from outside.

This makes the code:

```
Easier to change
Easier to test
Easier to reuse
More loosely coupled
```

IoC means the control of object creation moves from the class itself to an external system.

DI is one way to achieve IoC.

Spring takes this idea and automates it using the Spring IoC Container.

At the core level, Spring is mainly helping us with object creation, object management, and object wiring.

That is the foundation of Spring Core.

Coder Army

Coder Army

Spring IoC Container, Beans & Annotation-Based Configuration

1. Core Idea: Classes Should Not Create Their Own Dependencies

In a well-designed application, a class should focus on its own responsibility.

For example, `OrderService` should focus on placing an order. It should not be responsible for creating the object of `PaymentService`.

```
public class OrderService {

    private PaymentService paymentService = new PaymentServic
e();

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

The problem here is that `OrderService` is tightly coupled with `PaymentService`.

If tomorrow we want to replace `PaymentService` with another implementation, we will have to change the code inside `OrderService`.

A better design is:

```
public class OrderService {

    private final PaymentService paymentService;
```

```
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

Now `OrderService` does not create its dependency. It receives the dependency from outside.

This is Dependency Injection.

---

2. First-Principles Understanding

At the most basic level, every application needs objects.

Those objects may also need other objects.

So the real questions are:

```
Who will create these objects?
Who will connect them together?
Who will manage their lifecycle?
```

Without Spring, we usually do this manually.

```
main() creates objects.
main() connects objects.
main() behaves like a small manual container.
```

With Spring, this responsibility is shifted to the Spring IoC container.

```
Spring creates objects.
Spring connects objects.
```

Coder Army

```
Spring manages their lifecycle.
```

This is the basic idea of IoC: Inversion of Control.

Instead of our code controlling object creation and dependency wiring, Spring controls it for us.

---

3. Basic Project Setup

To work with Spring Core using annotation-based configuration:

1. Create a Maven project.
2. Add the `spring-context` dependency.

`spring-context` gives us important container features such as:

- `ApplicationContext`
- Annotation-based configuration
- Component scanning
- Bean creation and dependency injection

---

4. What Is a Spring Bean?

A Spring Bean is an object managed by the Spring IoC container.

That means Spring is responsible for:

- Creating the object
- Wiring its dependencies
- Managing its lifecycle
- Giving it to us when we ask for it

Simple definition:

> A Spring Bean is an object whose creation, dependency wiring, and lifecycle are managed by the Spring IoC container.

Coder Army

Coder Army

5. How Can Spring Manage Our Objects?

Spring can manage objects mainly through two configuration styles:

1. Annotation-based configuration
2. XML-based configuration

Annotation-Based Configuration

This is the modern and commonly used approach.

We use annotations such as:

- `@Component`
- `@Configuration`
- `@ComponentScan`
- `@Autowired`
- `@Bean`

XML-Based Configuration

This was widely used in older Spring applications.

Today, XML configuration is not preferred for new projects, but it is still useful to understand because many legacy projects may still contain XML-based bean configuration.

---

6. Reflection: Why `Student.class` Matters

When we write something like:

```
Student.class
```

we are not creating a `Student` object.

Instead, we are referring to a special object of type `Class`.

Example:

Coder Army

```
Class<Student> c = Student.class;
```

Here, `c` does not contain a `Student` object.

It contains metadata about the `Student` class, such as:

```
Class name     -> Student
Fields         -> name, age
Methods        -> study()
Constructors   -> Student()
Annotations    -> @Component, @Service, etc.
```

Spring uses this kind of class metadata internally.

When we pass a class like this:

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

we are giving Spring metadata about the `AppConfig` class so that Spring can read configuration instructions from it.

---

7. Telling Spring Which Classes to Manage

Spring does not automatically manage every class in the project.

We need to tell Spring which classes are eligible to become beans.

One common way is by using `@Component`.

```
@Component
public class PaymentService {

    public void pay() {
        System.out.println("Payment done");
    }
}
```

`@Component` tells Spring:

Coder Army

```
This class is eligible to become a Spring bean.
```

But just writing `@Component` is not enough.

Spring also needs to know where it should search for such classes. That is where `@ComponentScan` comes in.

---

8. `ApplicationContext`: The Spring IoC Container

In normal Spring applications, we commonly work with `ApplicationContext`.

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.clas
s);
```

`ApplicationContext` represents the Spring IoC container.

It is responsible for:

- Reading configuration
- Creating beans
- Resolving dependencies
- Managing bean lifecycle
- Providing beans when requested

Important point:

```
ApplicationContext context = new ApplicationContext();
```

This is not possible because `ApplicationContext` is an interface.

So we use one of its implementations.

For annotation-based configuration, we commonly use:

```
AnnotationConfigApplicationContext
```

Coder Army

9. What Is `AnnotationConfigApplicationContext`?

`AnnotationConfigApplicationContext` is an implementation of `ApplicationContext`.

It starts a Spring container using Java annotation-based configuration.

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.clas
s);
```

This means:

```
Start the Spring container.
Read instructions from AppConfig.class.
Use annotation-based configuration.
Create and manage beans accordingly.
```

---

10. What Is `AppConfig.class`?

`AppConfig` is a configuration class.

Example:

```
@Configuration
@ComponentScan("in.coderarmy")
public class AppConfig {
}
```

This class tells Spring:

```
This is a configuration class.
Scan the package in.coderarmy.
Find classes marked with annotations like @Component.
Create their beans.
Wire their dependencies.
```

So when we write:

Coder Army

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

we are giving Spring its startup instruction file.

---

11. What Is `@Configuration`?

`@Configuration` tells Spring that a class contains Spring configuration instructions.

```
@Configuration
public class AppConfig {
}
```

When Spring sees this, it understands:

```
This is not just a normal class.
This class may contain Spring setup instructions.
This class can be a source of bean definitions.
```

A configuration class may contain:

- `@ComponentScan`
- `@Bean` methods
- Other configuration-related instructions

Simple definition:

> `@Configuration` marks a class as a source of bean definitions.

---

12. What Is `@ComponentScan`?

When Spring starts, it needs to know where to search for classes marked with annotations like `@Component`.

```
@Configuration
@ComponentScan("com.coderarmy")
```

Coder Army

```
public class AppConfig {
}
```

This tells Spring:

```
Start scanning from com.coderarmy.
Also scan its sub-packages.
Find classes marked with @Component, @Service, @Repository, @Controller, etc.
Register them as beans.
```

`@Component` vs `@ComponentScan`

Annotation	Meaning
`@Component`	Marks a class as eligible to become a Spring bean
`@ComponentScan`	Tells Spring where to search for such classes

Can We Write `@ComponentScan` Without a Package Name?

Yes.

```
@Configuration
@ComponentScan
public class AppConfig {
}
```

In this case, Spring scans the package where `AppConfig` is present and its sub-packages.

For example, if `AppConfig` is present here:

```
in.coderarmy.AppConfig
```

then Spring scans:

```
in.coderarmy
```

and its sub-packages.

13. What Does `getBean()` Mean?

After Spring creates and stores beans inside the container, we can ask the container for a bean.

```
OrderService orderService =
        context.getBean(OrderService.class);
```

This means:

```
Spring, give me the OrderService bean from your container.
```

If Spring has already created the bean, it returns that object.

If Spring does not know about `OrderService`, we will get an error.

Common reasons:

```
OrderService is not annotated with @Component.
The package of OrderService is not included in @ComponentScan.
The bean is not registered using @Bean or XML.
```

Example:

```
public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
                new AnnotationConfigApplicationContext(AppCon
fig.class);

        OrderService orderService =
                context.getBean(OrderService.class);

        orderService.placeOrder();
```

Coder Army

```
    }
}
```

14. Types of Dependency Injection in Spring

Spring commonly supports three types of dependency injection:

1. Constructor injection
2. Field injection
3. Setter injection

15. Constructor Injection

In constructor injection, dependencies are provided through the constructor.

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

In modern Spring, if a bean has only one constructor, writing `@Autowired` on that constructor is optional.

Spring can automatically use that constructor for dependency injection.

This is also valid:

Coder Army

Coder Army

```java
@Component
public class OrderService {

    private final PaymentService paymentService;

    @Autowired
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

16. Why Constructor Injection Is Preferred

A constructor is called when an object is created.

So if `OrderService` needs `PaymentService`, the cleanest time to provide `PaymentService` is while creating `OrderService`.

That means the object is created in a complete and usable state.

Benefit 1: Dependency Is Mandatory

If `OrderService` cannot work without `PaymentService`, constructor injection makes that requirement clear.

```java
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

The object cannot be created without its required dependency.

Benefit 2: We Can Use `final`

```java
private final PaymentService paymentService;
```

This means once the dependency is assigned, it cannot be changed accidentally.

Coder Army

This makes the class safer.

Benefit 3: Easy to Test Without Spring

With constructor injection, we can manually create and test the class.

```
PaymentService paymentService = new PaymentService();
OrderService orderService = new OrderService(paymentService);

orderService.placeOrder();
```

We do not need Spring just to create the object.

This keeps our class clean, simple, and testable.

Spring helps us wire the object, but our Java class does not become tightly dependent on the Spring container.

---

17. Field Injection

In field injection, Spring directly injects the dependency into a field.

```
@Component
public class OrderService {

    @Autowired
    private PaymentService paymentService;

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

This works because Spring can use reflection to set the field value.

However, field injection is generally not preferred.

Reasons:

Coder Army

- The dependency is hidden.
- The class cannot be easily tested without Spring.
- The field cannot be marked as `final`.
- The object can exist in an incomplete state before Spring injects the field.

Constructor injection is usually the better choice for required dependencies.

---

18. Setter Injection

In setter injection, Spring creates the object first and then calls a setter method to provide the dependency.

```
@Component
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentServi
ce) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

Here, Spring follows this flow:

```
Create OrderService object using no-argument constructor.
Call setPaymentService().
Pass PaymentService into the setter.
```

Coder Army

Setter injection is useful when a dependency is optional or can be changed after object creation.

For mandatory dependencies, constructor injection is usually preferred.

---

19. What Happens Internally When Spring Starts?

When we write:

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

Spring starts the container and performs several steps.

Step 1: Spring Starts the Container

Spring creates an `ApplicationContext`.

This becomes the IoC container for our application.

Step 2: Spring Reads `AppConfig.class`

Spring looks at the class passed to it.

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

It understands:

```
This class contains configuration instructions.
I need to read annotations present on this class.
```

Step 3: Spring Processes `@ComponentScan`

If `AppConfig` contains:

Coder Army

```
@ComponentScan("com.coderarmy")
```

Spring understands:

```
Search inside com.coderarmy and its sub-packages.
```

Step 4: Spring Finds Component Classes

Spring searches the given package and finds classes marked with annotations such as:

- `@Component`
- `@Service`
- `@Repository`
- `@Controller`

Example:

```
@Component
public class PaymentService {
}
```

```
@Component
public class OrderService {
}
```

Step 5: Spring Creates Bean Definitions

Before creating actual objects, Spring first stores information about those objects.

For example, for `PaymentService`, Spring may store metadata like:

Coder Army

```
Bean name        -> paymentService
Bean class       -> com.coderarmy.service.PaymentService
Scope            -> singleton
Dependencies     -> none
Creation strategy -> constructor
```

This metadata is called a `BeanDefinition`.

A `BeanDefinition` is not the actual object.

It is information about how the object should be created and managed.

---

20. Why Does Spring Create Bean Definitions First?

Spring is not just creating one object.

It is managing a complete application.

Before creating objects, Spring needs to know:

- Which beans exist
- What their classes are
- What their scopes are
- What dependencies they need
- How they should be created
- What lifecycle methods should be applied

That is why Spring first builds bean definitions.

Conceptually:

```
BeanDefinition list
|
├── paymentService
└── orderService
```

After this, Spring starts creating actual bean objects.

Coder Army

Step 6: Spring Creates Bean Objects

Now Spring has a list of bean definitions.

For example:

```
paymentService
orderService
```

If `PaymentService` has no dependency, Spring can create it easily.

Conceptually:

```
PaymentService paymentService = new PaymentService();
```

But this object is created internally by Spring.

Now the container has:

```
ApplicationContext
|
└── paymentService bean
```

Step 7: Spring Creates `OrderService`

Now Spring tries to create `OrderService`.

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Coder Army

Spring checks the constructor:

```
public OrderService(PaymentService paymentService)
```

Spring understands:

```
To create OrderService, I need PaymentService.
```

This is called dependency resolution.

Spring then checks:

```
Do I have a PaymentService bean in the container?
```

If yes, Spring passes that bean into the constructor.

Conceptually:

```
OrderService orderService = new OrderService(paymentService);
```

Now the container has:

```
ApplicationContext
|
├── paymentService bean
└── orderService bean
```

Step 8: Spring Injects Dependencies

In constructor injection, dependency injection happens at object creation time.

When Spring calls:

```
new OrderService(paymentService)
```

`PaymentService` is injected into `OrderService`.

Coder Army

So after creation, `OrderService` is already ready to use.

Step 9: Our Application Uses the Bean

Now we can use the bean:

```
orderService.placeOrder();
```

At this point, `OrderService` already has `PaymentService` inside it.

```
public void placeOrder() {
    paymentService.pay();
    System.out.println("Order placed");
}
```

Output:

```
Payment done
Order placed
```

21. What If No Matching Bean Exists?

Suppose `OrderService` needs `PaymentService`, but Spring does not have any `PaymentService` bean.

Then Spring cannot create `OrderService`.

We may get an error like:

```
No qualifying bean of type 'PaymentService' available
```

Common reasons:

- `PaymentService` is not annotated with `@Component`.
- Its package is not included in `@ComponentScan`.

Coder Army

- No `@Bean` method is creating it.
- The required dependency is not registered in Spring.

---

22. What If Multiple Matching Beans Exist?

Now consider this case:

```
public interface PaymentService {
    void pay();
}
```

```
@Component
public class UPIPaymentService implements PaymentService {

    public void pay() {
        System.out.println("UPI payment done");
    }
}
```

```
@Component
public class CardPaymentService implements PaymentService {

    public void pay() {
        System.out.println("Card payment done");
    }
}
```

Now `OrderService` depends on the interface:

```
@Component
public class OrderService {

    private final PaymentService paymentService;
```

```
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring sees:

```
OrderService needs PaymentService.
```

But inside the container, Spring has two matching beans:

```
upiPaymentService  -> UPIPaymentService  -> PaymentService
cardPaymentService -> CardPaymentService -> PaymentService
```

Both are valid implementations of `PaymentService`.

Now Spring is confused:

```
Should I inject UPIPaymentService?
Should I inject CardPaymentService?
```

This is a bean ambiguity problem.

We can solve it using:

- `@Primary`
- `@Qualifier`

---

23. Using `@Primary`

`@Primary` is used when one implementation should be the default choice.

```
@Primary
@Component
public class UPIPaymentService implements PaymentService {
```

Coder Army

Coder Army

```
    public void pay() {
        System.out.println("UPI payment done");
    }
}
```

This tells Spring:

```
If multiple PaymentService beans are available,
prefer this one by default.
```

Now if `OrderService` asks for `PaymentService`, Spring will inject `UPIPaymentService`.

---

24. Using `@Qualifier`

`@Qualifier` is used when we want to explicitly choose a specific bean.

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(
            @Qualifier("cardPaymentService") PaymentService p
aymentService
    ) {
        this.paymentService = paymentService;
    }
}
```

This tells Spring:

```
Inject the bean named cardPaymentService here.
```

How Did Spring Get the Name `cardPaymentService`?

Coder Army

By default, Spring creates the bean name from the class name.

```
CardPaymentService
```

becomes:

```
cardPaymentService
```

Similarly:

```
UPIPaymentService
```

becomes:

```
UPIPaymentService
```

In many cases, the first letter is lowercased. However, acronyms can behave differently depending on Java bean naming rules.

To avoid confusion, we can give a custom bean name.

---

25. Custom Bean Names with `@Component`

We can give a custom name to a component.

```
@Component("upi")
public class UPIPaymentService implements PaymentService {
}
```

```
@Component("card")
public class CardPaymentService implements PaymentService {
}
```

Now we can use:

```
public OrderService(@Qualifier("card") PaymentService payment
Service) {
    this.paymentService = paymentService;
}
```

This is clearer and easier to control.

26. `@Qualifier` with Field Injection

`@Qualifier` can also be used with field injection.

```
@Component
public class OrderService {

    @Autowired
    @Qualifier("upiPaymentService")
    private PaymentService paymentService;

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

This works, but constructor injection is still usually preferred.

27. `@Qualifier` with Setter Injection

`@Qualifier` can also be used with setter injection.

```
@Component
public class OrderService {

    private PaymentService paymentService;
```

Coder Army

Coder Army

```
    @Autowired
    public void setPaymentService(
            @Qualifier("cardPaymentService") PaymentService p
aymentService
    ) {
        this.paymentService = paymentService;
    }
}
```

28. What If Both `@Primary` and `@Qualifier` Are Used?

If both `@Primary` and `@Qualifier` are present, `@Qualifier` gets priority.

Reason:

- `@Primary` gives a default choice.
- `@Qualifier` gives a specific choice.

Specific choice wins over default choice.

29. Why Do We Need `@Bean` ?

So far, we used `@Component` to register our own classes as Spring beans.

But sometimes we cannot use `@Component`.

For example, suppose a class comes from an external library.

```
public class EmailClient {

    private final String apiKey;

    public EmailClient(String apiKey) {
        this.apiKey = apiKey;
    }

    public void sendEmail() {
```

```
        System.out.println("Email sent using API key: " + apiKey);
    }
}
```

We cannot open this library class and add:

```
@Component
```

But we may still want Spring to manage its object.

In such cases, we use `@Bean`.

---

30. Another Case: Custom Object Creation

Sometimes object creation is not simple.

Example:

```
public class User {

    private String name;
    private String email;

    public User(String name, String email) {
        this.name = name;
        this.email = email;
    }
}
```

If we simply ask Spring to create this object, Spring may not know what values to pass for `name` and `email`.

So we can tell Spring exactly how to create it.

That is what `@Bean` helps us do.

---

31. What Exactly Does `@Bean` Mean?

Coder Army

Coder Army

`@Bean` is used on a method inside a configuration class.

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

This means:

```
Spring, call this method.
Take the returned object.
Register it inside your container.
Manage it as a Spring bean.
```

For singleton beans, Spring usually creates and stores the object when the context starts.

---

32. Custom Bean Name with `@Bean`

By default, the bean name is the method name.

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

Bean name:

```
paymentService
```

We can also give a custom bean name.

Coder Army

```
@Bean("myPaymentService")
public PaymentService paymentService() {
    return new PaymentService();
}
```

Now we can fetch it by name:

```
PaymentService paymentService =
        (PaymentService) context.getBean("myPaymentService");
```

---

33. `@Bean` with Dependencies

Suppose `OrderService` needs `PaymentService`.

```
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

No `@Component` is used here.

We can create both beans using `@Bean`.

```
@Configuration
public class AppConfig {
```

Coder Army

```
    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }

    @Bean
    public OrderService orderService(PaymentService paymentSe
rvice) {
        return new OrderService(paymentService);
    }
}
```

Here, Spring first creates `PaymentService`.

Then it passes `PaymentService` into the `orderService()` method.

This is clean because Spring is still resolving the dependency.

Another possible style is:

```
@Bean
public OrderService orderService() {
    return new OrderService(paymentService());
}
```

In a full `@Configuration` class, Spring ensures singleton behavior for such method calls. Still, using method parameters is usually cleaner and easier to understand.

---

34. `@Component` vs `@Bean`

Point	`@Component`	`@Bean`
Where used?	On a class	On a method
Style	Automatic detection	Manual registration
Best for	Our own classes	External library classes or custom object creation
Requires component scanning?	Yes	No, but configuration class must be loaded

Coder Army

Point	`@Component`	`@Bean`
Bean name	Class name based by default	Method name by default

Simple comparison:

```
@Component = Spring finds the class automatically.
@Bean      = We manually tell Spring how to create the object.
```

---

35. Important Note: Avoid Creating the Same Bean Twice

Do not register the same object through both `@Component` and `@Bean` unless you intentionally want multiple beans.

Example:

```java
@Component
public class PaymentService {
}
```

and also:

```java
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

This can create confusion.

Possible outcomes:

- Spring may treat them as two different beans if their names are different.
- Dependency injection by type may become ambiguous.
- If the names are same, bean overriding rules may apply depending on configuration.

Coder Army

So the clean rule is:

> Register a bean using either `@Component` or `@Bean`, not both, unless you have a clear reason.

---

36. Dependency Resolution with `@Bean`

If multiple beans of the same type exist, the same rules apply.

We can use `@Primary`.

```
@Bean
@Primary
public PaymentService upiPaymentService() {
    return new UPIPaymentService();
}
```

We can use `@Qualifier`.

```
@Bean
public OrderService orderService(
        @Qualifier("cardPaymentService") PaymentService payme
ntService
) {
    return new OrderService(paymentService);
}
```

This tells Spring exactly which bean to inject.

---

37. Why Not Put Everything in `main()`?

A common question is:

> Why do we need `AppConfig`? Why not create everything in `main()`?

Because `main()` should only start the application.

Coder Army

If we put all object creation and wiring logic inside `main()`, then `main()` again becomes responsible for configuration and object management.

That brings us back to manual object creation.

Instead, we keep `main()` simple:

```
public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
                new AnnotationConfigApplicationContext(AppCon
fig.class);

        OrderService orderService =
                context.getBean(OrderService.class);

        orderService.placeOrder();
    }
}
```

Now object creation, dependency wiring, and lifecycle management are handled by Spring.

---

38. BeanFactory vs ApplicationContext

Internally, Spring has something called `BeanFactory`.

`BeanFactory` is the basic container interface.

`ApplicationContext` builds on top of it and provides more complete features for real applications.

In normal Spring applications, we commonly use `ApplicationContext`.

Simple understanding:

```
BeanFactory        -> Basic container
ApplicationContext -> Advanced container used commonly in applications
```

Coder Army

`ApplicationContext` gives us features like:

- Bean creation and dependency injection
- Lifecycle management
- Event publishing
- Internationalization support
- Integration with other Spring features

---

39. Final Flow Summary

When we start Spring using:

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.clas
s);
```

Spring performs this flow:

```
1. Start the IoC container.
2. Read AppConfig.class.
3. Process @Configuration.
4. Process @ComponentScan.
5. Scan the given package and sub-packages.
6. Find classes marked with @Component and related annotations.
7. Create BeanDefinitions.
8. Resolve dependencies.
9. Create bean objects.
10. Inject dependencies.
11. Store beans inside the container.
12. Return beans when getBean() is called.
```

---

40. Key Takeaways

- A Spring Bean is an object managed by the Spring IoC container.
- `ApplicationContext` represents the Spring IoC container in normal applications.

Coder Army

- `AnnotationConfigApplicationContext` starts a container using annotation-based configuration.
- `@Configuration` marks a class as a source of Spring configuration.
- `@ComponentScan` tells Spring where to search for components.
- `@Component` marks a class as eligible to become a bean.
- `getBean()` asks Spring for an object from the container.
- Constructor injection is generally preferred for mandatory dependencies.
- Field injection works, but it is not recommended for clean design.
- Setter injection is useful for optional or changeable dependencies.
- Spring first creates `BeanDefinition` metadata, then creates actual bean objects.
- If no matching bean exists, Spring throws an error.
- If multiple matching beans exist, Spring needs help through `@Primary` or `@Qualifier`.
- `@Bean` is useful when we want to manually tell Spring how to create an object.
- `@Component` is class-level and automatic.
- `@Bean` is method-level and manual.
- Avoid registering the same bean through both `@Component` and `@Bean` unless you intentionally want multiple beans.
- `main()` should start the application, not manage all object creation.
- `ApplicationContext` is commonly used because it provides complete container features.

---

41. One-Line Revision

> Spring IoC Container creates objects, wires dependencies, manages lifecycle, and gives us ready-to-use beans when required.

Coder Army

Lecture 6 : Circular Dependency, Bean Scope & Bean Initialization in Spring

1. Quick Recap: DI, IoC and Beans

Before understanding circular dependency, bean scopes, and initialization, we should already be clear about three basic Spring concepts:

Concept	Meaning
DI	Dependency Injection. Spring provides the required dependency to a class instead of the class creating it manually.
IoC	Inversion of Control. The control of object creation and dependency management is given to the Spring container.
Bean	An object created, managed, and maintained by the Spring IoC container.

In Spring, classes annotated with annotations like `@Component`, `@Service`, `@Repository`, `@Controller`, or methods annotated with `@Bean` can become Spring-managed beans.

---

2. `@Configuration` Is Also a Spring Component

In Spring, `@Configuration` is used to define configuration classes.

Example:

```
@Configuration
public class AppConfig {
}
```

A configuration class can contain `@Bean` methods:

Coder Army

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

An important point is that `@Configuration` itself internally contains `@Component`.

That means a class annotated with `@Configuration` is also detected by component scanning and registered as a Spring bean.

So we can understand it like this:

```
@Configuration
        ↓
@Component
        ↓
Spring can detect and manage this class
```

Part 1: Why Bean Creation Order Matters

3. Can Spring Create Beans in Any Random Order?

No.

Spring does not create beans blindly in random order. It creates beans based on their dependencies.

Suppose we have this dependency chain:

```
OrderController depends on OrderService
OrderService depends on PaymentService
PaymentService depends on PaymentGateway
```

Before Spring can create `OrderService`, it needs `PaymentService`.

Before it can create `PaymentService`, it needs `PaymentGateway`.

Coder Army

So the creation order becomes:

```
PaymentGateway
      ↓
PaymentService
      ↓
OrderService
      ↓
OrderController
```

Spring tries to create the dependency first, and then the class that depends on it.

---

4. Why This Matters for Circular Dependency

Now imagine this situation:

```
A needs B
B needs A
```

Spring starts creating `A`.

But to create `A`, it needs `B`.

Then Spring starts creating `B`.

But to create `B`, it needs `A`.

So the container gets stuck in a loop:

```
To create A, I need B.
To create B, I need A.
To create A, I need B.
...
```

This is the circular dependency problem.

Spring creates beans based on dependency order. But when the dependency order becomes circular, Spring cannot find a clear starting point.

Coder Army

Part 2: Circular Dependency

5. What Is Circular Dependency?

Circular dependency means two or more classes depend on each other directly or indirectly.

Example:

```
OrderService depends on PaymentService
PaymentService depends on OrderService
```

Example Code

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

```
@Service
public class PaymentService {

    private final OrderService orderService;

    public PaymentService(OrderService orderService) {
        this.orderService = orderService;
    }
}
```

Here, Spring cannot decide which object should be created first.

To create `OrderService`, Spring needs `PaymentService`.

To create `PaymentService`, Spring needs `OrderService`.

This creates a deadlock-like situation during bean creation.

Coder Army

6. Circular Dependency Is Not Only a Spring Problem

Circular dependency is not only a Spring-specific issue. It can happen in normal Java as well.

Simple Java Example

```
public class A {

    private B b;

    public A() {
        this.b = new B();
    }
}
```

```
public class B {

    private A a;

    public B() {
        this.a = new A();
    }
}
```

Now suppose we write:

```
public class Main {
    public static void main(String[] args) {
        A a = new A();
    }
}
```

The flow becomes:

```
new A()
  → A constructor runs
  → new B()
      → B constructor runs
      → new A()
          → A constructor runs
```

Coder Army

```
            → new B()
                → new A()
                    → ...
```

This never ends.

Eventually, Java throws:

```
StackOverflowError
```

This happens because constructor calls keep getting stacked again and again.

So circular dependency is a design problem. Spring simply exposes this problem during bean creation.

Part 3: Constructor-Based Circular Dependency

7. Why Constructor Injection Fails in Circular Dependency

Constructor injection has one strict rule:

> An object cannot be created until all constructor arguments are available.

For example:

```
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

To create `OrderService`, Spring must already have a `PaymentService` object.

But if `PaymentService` also needs `OrderService` in its constructor, Spring cannot create either of them first.

The loop becomes:

Coder Army

```
Create OrderService → needs PaymentService
Create PaymentService → needs OrderService
Create OrderService → needs PaymentService
...
```

There is no starting point.

That is why constructor-based circular dependency fails.

Spring detects this situation during startup and throws an exception such as:

```
BeanCurrentlyInCreationException
```

The application fails to start.

---

8. Why Constructor Injection Is Still Recommended

Even though constructor injection fails in circular dependency, constructor injection is still the preferred style in most cases.

Why?

Because it makes dependencies clear and mandatory.

If a class cannot work without a dependency, constructor injection clearly communicates that.

Example:

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

This tells us:

Coder Army

```
OrderService cannot exist without PaymentService.
```

So constructor injection is not the problem. The real problem is the circular design.

Part 4: Setter and Field Injection in Circular Dependency

9. Circular Dependency Using Setter Injection

Now let us rewrite the same example using setter injection.

```
@Service
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

```
@Service
public class PaymentService {

    private OrderService orderService;

    @Autowired
    public void setOrderService(OrderService orderService) {
        this.orderService = orderService;
    }
}
```

In setter injection, object creation and dependency injection are separate steps.

Spring can conceptually do this:

Coder Army

```
Step 1: Create empty OrderService object
Step 2: Create empty PaymentService object
Step 3: Inject PaymentService into OrderService
Step 4: Inject OrderService into PaymentService
```

This is different from constructor injection.

In constructor injection:

```
Dependency must exist before object creation.
```

In setter injection:

```
Object can be created first.
Dependency can be injected later.
```

Because of this difference, setter injection may sometimes allow Spring to resolve circular dependency.

---

10. Circular Dependency Using Field Injection

The same thing can happen with field injection.

```
@Service
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

```
@Service
public class PaymentService {

    @Autowired
    private OrderService orderService;
}
```

Coder Army

Here also, Spring can create the object first and inject the dependency later.

But this does not mean circular dependency is a good practice.

It only means Spring may be able to handle some circular dependencies when object creation and dependency injection are separated.

---

11. Why Setter/Field Circular Dependency May Work

In setter or field injection, the constructor does not require the dependency immediately.

Example:

```
@Service
public class OrderService {

    @Autowired
    private PaymentService paymentService;

    public OrderService() {
        System.out.println("OrderService object created");
    }
}
```

Here, Spring can create the `OrderService` object without immediately needing `PaymentService`.

Similarly:

```
@Service
public class PaymentService {

    @Autowired
    private OrderService orderService;

    public PaymentService() {
        System.out.println("PaymentService object created");
    }
}
```

Spring gets some flexibility.

Coder Army

Conceptually, Spring can say:

```
I have created OrderService object,
but it is not fully initialized yet.

Still, I can keep an early reference to it.

Now I create PaymentService.
PaymentService needs OrderService.

I can give it the early reference of OrderService.

Then I come back and inject PaymentService into OrderService.
```

This is the basic idea behind Spring's early reference mechanism.

Part 5: Spring's Early Reference Concept

12. Normal Dependency Flow

In a normal dependency situation:

```
A needs B
```

Spring creates `B` first.

Then it injects `B` into `A`.

```
Create B
Inject dependencies into B
Initialize B
B is ready

Create A
Inject B into A
Initialize A
A is ready
```

The dependent bean receives a fully created and initialized dependency.

Coder Army

13. Circular Dependency Flow with Early Reference

In a circular dependency:

```
A needs B
B needs A
```

Spring may try to create one object first and expose an early reference to it.

Conceptually:

```
A a = new A();  // A exists, but dependencies are not injected yet

B b = new B();  // B exists

b.setA(a);      // B receives early reference of A
a.setB(b);      // A receives B
```

An early reference means:

> Spring exposes a reference of a bean before the bean is fully initialized, so another bean can temporarily use that reference during circular dependency resolution.

This can help Spring resolve some setter/field-based circular dependencies.

---

14. Important Spring Boot Point

Spring Framework has internal mechanisms to resolve some setter or field-based circular dependencies.

However, Spring Boot discourages circular references.

From Spring Boot 2.6 onward, circular references are disabled by default.

The related property is:

```
spring.main.allow-circular-references=false
```

Its default value is:

Coder Army

```
false
```

You can technically enable it:

```
spring.main.allow-circular-references=true
```

But this should not be treated as the solution.

The better solution is to improve the design.

Part 6: Why Circular Dependency Is a Design Smell

15. Circular Dependency Usually Indicates Mixed Responsibilities

Circular dependency is usually not only a technical issue.

It often shows that responsibilities are not clearly separated.

Ask these questions:

```
Why does A need B?
Why does B also need A?
Are both classes doing too much?
Can we move common logic to a third class?
Can one dependency direction be removed?
```

Example:

```
OrderService depends on PaymentService
PaymentService depends on OrderService
```

This may indicate that order-related logic and payment-related logic are mixed.

A better design may be:

Coder Army

```
OrderService → PaymentService

or

OrderService → PaymentService
PaymentService → PaymentStatusRepository

or

OrderService → PaymentProcessor
PaymentService → PaymentProcessor
```

Instead of both depending on each other.

---

16. Best Way to Solve Circular Dependency

The best solution is refactoring.

Possible ways:

Problem	Better Design
Two services call each other	Move shared logic to a third service
One service has too many responsibilities	Split responsibilities
Both classes depend on each other's internal behavior	Introduce an interface or event-based communication
Service is doing orchestration and business logic together	Create a separate coordinator/orchestrator class

The first principle is simple:

> Avoid circular dependency by designing clear responsibility boundaries.

Part 7: Bean Scope

17. What Is Bean Scope?

Bean scope decides:

Coder Army

```
How many objects Spring will create for a bean
and how long those objects will live.
```

The important question is:

```
How many objects will Spring create for one bean definition?
```

Spring provides different scopes for different use cases.

The two most important core scopes are:

```
singleton
prototype
```

In web applications, Spring also provides:

```
request
session
application
websocket
```

Part 8: Singleton Scope

18. What Is Singleton Scope?

Singleton scope means:

> Spring creates exactly one object for a bean definition inside the Spring container, stores it, and returns the same object whenever needed.

Singleton is the default scope in Spring.

So this:

Coder Army

```
@Component
public class PaymentService {
}
```

is the same as:

```
@Component
@Scope("singleton")
public class PaymentService {
}
```

Usually, we do not need to write `@Scope("singleton")` explicitly.

19. Singleton Scope Demo

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.class);

PaymentService p1 = context.getBean(PaymentService.class);
PaymentService p2 = context.getBean(PaymentService.class);

System.out.println(p1 == p2);
```

Output:

```
true
```

Both references point to the same Spring-managed singleton bean.

20. Important Clarification About Singleton

Spring singleton does not mean:

```
Only one object of this class can ever exist in the JVM.
```

Coder Army

We can still manually create an object:

```
PaymentService p = new PaymentService();
```

But this object is not managed by Spring.

Spring singleton means:

> Inside this Spring container, for this bean definition, Spring keeps one shared object.

---

21. Singleton Is Per Bean Definition, Not Per Class

Consider this example:

```
@Bean
public User createUser() {
    return new User();
}

@Bean
public User createUser2() {
    return new User();
}
```

Spring creates two different bean definitions:

```
createUser
createUser2
```

Both are singleton by default.

So Spring creates:

```
createUser  → one User object
createUser2 → one User object
```

Total:

Coder Army

```
2 bean definitions
2 singleton objects
```

Therefore:

> In Spring, singleton means one object per bean name or bean definition inside the Spring container.

It does not mean one object per class.

This is different from the traditional Singleton Design Pattern.

Part 9: Prototype Scope

22. What Is Prototype Scope?

Prototype scope means:

> Spring creates a new object every time that bean is requested from the container.

Example:

```
@Component
@Scope("prototype")
public class OrderRequest {

    public OrderRequest() {
        System.out.println("OrderRequest object created");
    }
}
```

Now if we do:

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.class);

OrderRequest r1 = context.getBean(OrderRequest.class);
OrderRequest r2 = context.getBean(OrderRequest.class);
```

Coder Army

```
System.out.println(r1 == r2);
```

Output:

```
false
```

Why?

Because each request gives a new object.

---

23. Prototype Feels Closer to `new`

Prototype scope feels closer to manual object creation using `new`.

Every time we ask Spring for the bean, it gives a fresh object.

```
context.getBean(OrderRequest.class) → new object
context.getBean(OrderRequest.class) → new object
context.getBean(OrderRequest.class) → new object
```

But the object is still created by Spring, not manually by us.

---

24. When to Use Singleton and Prototype

A simple rule:

Scope	Best For
Singleton	Stateless beans that provide behavior
Prototype	Stateful beans that store changing data

Singleton Example

Services are usually singleton:

```
@Service
public class PaymentService {
```

```
}
```

Why?

Because services usually contain behavior, not user-specific changing data.

Example:

```
processPayment()
validateOrder()
sendEmail()
calculateDiscount()
```

These methods can be reused safely.

Prototype Example

Prototype can be used when each request needs a fresh object with its own state.

Example:

```
@Component
@Scope("prototype")
public class OrderRequest {
}
```

Each time we need a new `OrderRequest`, Spring can create a fresh object.

---

25. Prototype Bean Injected Into Singleton Bean

Important question:

> If we inject a prototype bean into a singleton bean, will the singleton get a new prototype object every time?

Answer:

```
No.
```

Coder Army

Coder Army

Why?

Because the singleton bean is created only once.

At the time of singleton creation, Spring injects one prototype object into it.

After that, the same singleton object keeps using the same injected prototype reference.

Example:

```
@Service
public class OrderService {

    private final OrderRequest orderRequest;

    public OrderService(OrderRequest orderRequest) {
        this.orderRequest = orderRequest;
    }
}
```

If `OrderService` is singleton and `OrderRequest` is prototype, then Spring injects one `OrderRequest` object when `OrderService` is created.

After that, the same `OrderRequest` reference remains inside the singleton.

So prototype does not automatically mean:

```
new object every time a singleton method runs
```

It means:

```
new object every time the prototype bean is requested from the container
```

Part 10: Web Scopes

26. Web-Specific Bean Scopes

Apart from `singleton` and `prototype`, Spring also provides web-specific scopes.

Coder Army

The important web scopes are:

Scope	One Object For
`request`	One HTTP request
`session`	One user session
`application`	One ServletContext / whole web application
`websocket`	One WebSocket session

These scopes are available only in a web-aware Spring `ApplicationContext`.

For beginner-level Spring MVC or Spring Boot Web understanding, the most important scopes are:

```
request
session
application
```

---

27. Request Scope

Request scope means:

> Spring creates one bean object for one HTTP request.

Example:

```
User sends request 1 → new request-scoped bean
User sends request 2 → new request-scoped bean
```

This is useful when data should live only during one HTTP request.

---

28. Session Scope

Session scope means:

> Spring creates one bean object for one user session.

Example:

Coder Army

```
User A session → one bean object
User B session → another bean object
```

This can be useful for user-specific session data.

---

29. Application Scope

Application scope means:

> Spring creates one bean object for the entire web application.

This bean is tied to the `ServletContext`.

It is shared across the web application.

Part 11: Bean Initialization

30. When Are Beans Actually Created?

Spring has to answer one important question:

```
When should I create this bean?
```

There are two main answers:

Initialization Type	Meaning
Eager initialization	Create the bean during application startup
Lazy initialization	Create the bean only when it is actually needed

By default:

```
Singleton beans are eagerly initialized.
Prototype beans are created lazily, when requested.
```

Lazy initialization can also be enabled using `@Lazy`.

Coder Army

Part 12: Eager Initialization

31. What Is Eager Initialization?

Eager initialization means:

> Spring creates the bean as soon as the application context starts.

For singleton beans, this is the default behavior.

Example:

```
@Component
public class PaymentService {

    public PaymentService() {
        System.out.println("PaymentService object created");
    }
}
```

When the application starts, Spring creates the singleton bean immediately.

---

32. Why Eager Initialization Is the Default

At first, eager initialization may look wasteful.

We may think:

```
Why create everything at startup if the user may not use it immediately?
```

But eager initialization has a big advantage.

It helps the application fail fast.

Suppose a bean has a configuration issue, missing dependency, or invalid setup.

With eager initialization, the application fails during startup.

That is useful because we find the problem early.

This is better than starting the application successfully and then failing only when the first user hits a feature.

Coder Army

Eager initialization gives:

```
Fast failure
Early error detection
Predictable startup validation
```

That is why Spring generally prefers eager singleton creation.

Part 13: Lazy Initialization

33. What Is Lazy Initialization?

Lazy initialization means:

> Do not create the bean during startup. Create it only when someone actually asks for it.

Example:

```
@Component
@Lazy
public class ReportService {

    public ReportService() {
        System.out.println("ReportService object created");
    }

    public void generateReport() {
        System.out.println("Report generated");
    }
}
```

Here, Spring knows about `ReportService`, but it does not immediately create the object.

Spring only keeps the bean definition.

A bean definition contains information like:

Coder Army

```
class name
scope
dependencies
lifecycle details
bean name
```

The actual object is created only when someone asks for it:

```
ReportService reportService = context.getBean(ReportService.class);
reportService.generateReport();
```

Output:

```
Application started
ReportService object created
Report generated
```

---

34. Important Point: Lazy Bean Needed by Eager Bean

If a lazy bean is required by an eager singleton bean, then the lazy bean may still be created at startup.

Why?

Because Spring must satisfy the dependencies of the eager singleton.

Example:

```
@Component
public class UserService {

    private final ReportService reportService;

    public UserService(ReportService reportService) {
        this.reportService = reportService;
    }
}
```

Coder Army

If `UserService` is eager and needs `ReportService`, Spring has to provide `ReportService` while creating `UserService`.

So even if `ReportService` is marked lazy, it may still be created during startup because another eager bean needs it.

Part 14: `@Lazy` on Injection Point

35. Lazy Dependency Injection Using Proxy

There is another interesting use of `@Lazy`.

Instead of making the whole bean lazy, we can mark the injection point as lazy.

Example:

```
@Component
public class UserService {

    private final EmailService emailService;

    public UserService(@Lazy EmailService emailService) {
        this.emailService = emailService;
        System.out.println("UserService created");
    }

    public void registerUser() {
        System.out.println("User registered");
        emailService.sendEmail();
    }
}
```

Here, Spring does not inject the real `EmailService` object immediately.

Instead, Spring injects a proxy object.

A proxy is like a placeholder.

Conceptually:

```
UserService does not receive the real EmailService immediately.

UserService receives an object that looks like EmailService.
```

Coder Army

```
When someone actually calls a method on it,
Spring creates or fetches the real EmailService.
```

Startup flow:

```
UserService created
Application started
```

Later, when this method runs:

```
emailService.sendEmail();
```

Spring resolves the real `EmailService`.

Output:

```
User registered
EmailService created
Email sent
```

---

36. Bean-Level `@Lazy` vs Injection-Point `@Lazy`

Usage	Meaning
`@Lazy` on class	Do not create this bean until it is requested
`@Lazy` on injection point	Inject a proxy and resolve the real dependency only when it is used

Example of bean-level lazy:

```
@Component
@Lazy
public class ReportService {
}
```

Example of injection-point lazy:

Coder Army

```
public UserService(@Lazy EmailService emailService) {
    this.emailService = emailService;
}
```

Part 15: Global Lazy Initialization in Spring Boot

37. Enabling Global Lazy Initialization

Spring Boot can make the whole application lazy using this property:

```
spring.main.lazy-initialization=true
```

In YAML:

```
spring:
  main:
    lazy-initialization: true
```

This means beans are created only when needed instead of being created during startup.

This can reduce startup time.

But there is a trade-off.

Misconfiguration errors may be discovered later, only when the lazy bean is actually initialized.

---

38. Default Value

By default:

```
spring.main.lazy-initialization=false
```

Coder Army

So Spring Boot does not make all beans lazy by default.

39. Opting Out of Global Lazy Initialization

If global lazy initialization is enabled, we can still force a particular bean to initialize eagerly.

Example:

```
@Component
@Lazy(false)
public class ImportantStartupBean {
}
```

This means:

> Even if global lazy initialization is enabled, create this bean eagerly.

Part 16: Common Lazy Initialization Scenarios

40. Eager Bean Depends on Lazy Bean

```
Eager Bean → Lazy Dependency
```

If an eager singleton directly needs a lazy dependency, Spring may still create the lazy dependency at startup to satisfy the eager bean.

Unless we use `@Lazy` at the injection point, where Spring can inject a proxy instead.

41. Lazy Bean Depends on Eager Bean

Coder Army

```
Lazy Bean → Eager Dependency
```

The eager dependency is usually created during startup.

The lazy bean is created only when it is needed.

When the lazy bean is finally created, Spring injects the already-created eager dependency into it.

---

42. Lazy Bean Depends on Lazy Bean

```
Lazy Bean → Lazy Dependency
```

Both beans are not created at startup.

When the first lazy bean is requested, Spring creates it.

If it directly needs the second lazy bean, Spring may create the second bean at that time as well.

If `@Lazy` is used on the injection point, Spring may inject a proxy and delay the actual dependency creation until a method is called.

Part 17: How `@Lazy` Can Help in Circular Dependency

43. Using `@Lazy` to Break Constructor Circular Dependency

Sometimes `@Lazy` can help break a circular dependency by injecting a proxy instead of the real object immediately.

Example:

```
@Service
public class OrderService {
```

Coder Army

```
    private final PaymentService paymentService;

    public OrderService(@Lazy PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

```
@Service
public class PaymentService {

    private final OrderService orderService;

    public PaymentService(OrderService orderService) {
        this.orderService = orderService;
    }
}
```

Here, Spring does not need to create the real `PaymentService` immediately while creating `OrderService`.

It can inject a proxy.

The real `PaymentService` is resolved later when it is actually used.

---

44. Important Warning

Using `@Lazy` to solve circular dependency should be treated as a workaround, not the best design.

It may help in some situations, but the better solution is still refactoring.

Use `@Lazy` only when:

```
You understand the dependency flow clearly
Refactoring is not immediately possible
You are dealing with legacy code
You want a temporary solution
```

The best long-term solution is:

```
Fix the design
Separate responsibilities
```

```
Remove circular dependency
```

Coder Army

Part 18: Bean Lifecycle

45. What Is Bean Lifecycle?

Bean lifecycle means:

> The complete journey of a Spring bean from the moment Spring discovers it, creates it, injects dependencies into it, initializes it, uses it, and finally destroys it.

At a high level, the lifecycle looks like this:

```
Bean Definition
      ↓
Object Creation
      ↓
Dependency Injection
      ↓
Initialization
      ↓
Ready to Use
      ↓
Destruction
```

46. Complete Journey of a Spring Bean

Step 1: Bean Definition Is Created

Spring first discovers the bean.

This can happen through:

```
@Component
@Service
@Repository
@Controller
```

Coder Army

```
@Bean method
XML configuration
```

At this stage, Spring has not necessarily created the actual object.

It has only stored the bean definition.

The bean definition contains information like:

```
bean name
class name
scope
dependencies
lazy/eager behavior
lifecycle methods
```

Step 2: Object Is Created

Spring creates the actual object using the constructor.

Example:

```
PaymentService paymentService = new PaymentService();
```

But in real Spring code, we do not write this manually.

The Spring container does it for us.

Step 3: Dependencies Are Injected

After object creation, Spring injects the required dependencies.

Example:

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
```

```
    }
}
```

Spring provides `PaymentService` to `OrderService`.

Step 4: Bean Is Initialized

After dependency injection, Spring runs initialization logic.

This is the stage where we can perform setup tasks such as:

```
validating configuration
opening resources
loading required data
checking required fields
```

Initialization can be done using different mechanisms such as:

```
@PostConstruct
InitializingBean
custom init method
```

Step 5: Bean Is Ready to Use

After initialization, the bean is ready.

Other beans can use it.

The application can now call its methods.

Step 6: Bean Is Destroyed

When the Spring container shuts down, singleton beans are destroyed.

This is where cleanup logic can run, such as:

```
closing resources
releasing connections
stopping background tasks
```

Coder Army

Coder Army

Destruction can be handled using:

```
@PreDestroy
DisposableBean
custom destroy method
```

For prototype beans, Spring creates the object and injects dependencies, but it does not fully manage the destruction lifecycle in the same way as singleton beans.

Part 19: Final Summary

47. Circular Dependency Summary

Circular dependency happens when two or more beans depend on each other.

Example:

```
A needs B
B needs A
```

Constructor-based circular dependency usually fails because constructor injection requires all dependencies before object creation.

Setter and field injection may sometimes work because object creation and dependency injection are separate steps.

But circular dependency is usually a design smell.

The best solution is not to depend on hacks, but to refactor the code.

---

48. Scope Summary

Scope	Meaning
`singleton`	One object per bean definition inside the Spring container
`prototype`	New object every time the bean is requested from the container

Coder Army

`request`	One object per HTTP request
`session`	One object per user session
`application`	One object per web application / ServletContext

Important point:

> Spring singleton means one object per bean definition inside the container, not one object per class in the whole JVM.

---

49. Initialization Summary

Type	Meaning
Eager initialization	Bean is created during application startup
Lazy initialization	Bean is created only when needed

By default:

```
Singleton beans → eagerly initialized
Prototype beans → created when requested
```

`@Lazy` can be used to delay bean creation.

`@Lazy` on an injection point creates a proxy and resolves the real dependency later.

Spring Boot also supports global lazy initialization:

```
spring.main.lazy-initialization=true
```

But lazy initialization can delay errors until runtime.

---

50. Key Takeaways

1. Spring creates beans based on dependency order.
2. Circular dependency means the dependency order becomes a loop.
3. Constructor-based circular dependency usually fails.

Coder Army

4. Setter or field injection may allow Spring to resolve some circular dependencies.
5. Spring Boot discourages circular references and disables them by default.
6. Circular dependency is usually a design smell and should be solved by refactoring.
7. Singleton scope means one object per bean definition inside the Spring container.
8. Prototype scope means a new object is created whenever the bean is requested from the container.
9. Web scopes like request, session, and application are mainly used in web applications.
10. Eager initialization helps catch errors early during startup.
11. Lazy initialization delays bean creation until needed.
12. `@Lazy` on an injection point injects a proxy and resolves the real bean later.
13. Bean lifecycle covers the complete journey from bean definition to destruction

Coder Army

Spring Bean Lifecycle

1. What is Bean Lifecycle?

In normal Java, when we create an object manually, we are responsible for everything.

```
PaymentService paymentService = new PaymentService();
```

That means we handle the complete object journey ourselves:

```
Create object
→ set required values
→ call required methods
→ clean up resources if needed
```

But in Spring, we usually do not create important application objects manually.

Instead, we tell Spring which classes or methods should be managed by the container.

Example using `@Component`:

```
@Component
public class PaymentService {
}
```

Example using `@Bean`:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
```

Coder Army

```
    }
}
```

Now this object becomes Spring's responsibility.

Spring will:

```
discover the bean
→ create the object
→ inject dependencies
→ call lifecycle methods
→ keep the bean ready
→ destroy it when required
```

---

2. Simple Definition

Bean lifecycle means the complete journey of a Spring-managed object from the moment Spring discovers its definition until the moment Spring destroys it.

A Spring-managed object is called a **bean**.

So whenever we say "bean lifecycle," we are talking about:

```
How Spring creates, prepares, manages, and destroys a bean.
```

---

3. Complete Bean Lifecycle Flow

```
Spring container starts
        ↓
Reads configuration / annotations
        ↓
Creates BeanDefinition
        ↓
Instantiates bean object
        ↓
Injects dependencies
        ↓
Calls Aware interfaces
        ↓        ↓
```

Coder Army

```
Runs initialization callbacks
        ↓        ↓
Bean is ready to use
        ↓
Application uses the bean
        ↓        ↓
Runs destruction callbacks
        ↓
Bean is removed
```

---

4. Step 0: Spring Container Starts

When we write:

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.clas
s);
```

we are not passing an object of `AppConfig`.

We are passing the **class metadata** of `AppConfig`.

Spring uses reflection to inspect that class.

It checks things like:

```
Does this class have @Configuration?
Does it have @ComponentScan?
Does it have @Bean methods?
Which package should be scanned?
Which bean definitions should be registered?
```

This is where the Spring container starts understanding the application structure.

---

5. Step 1: Bean Definition is Read

Before creating objects, Spring first collects metadata about beans.

This metadata is called a **BeanDefinition**.

Coder Army

A bean definition contains information like:

```
Bean name: paymentService
Bean class: PaymentService
Scope: singleton
Lazy: false
Dependencies: check later
Init method: check later
Destroy method: check later
```

At this stage, the object is not necessarily created.

Spring is only registering information about the bean.

Why does Spring need BeanDefinition first?

Because Spring is not creating just one object.

It is building a complete object network.

For example:

```
OrderService depends on PaymentService
PaymentService depends on PaymentGateway
PaymentGateway depends on ApiClient
```

Spring first needs to know:

```
Which classes should I manage?
What is the bean name?
What is the scope?
What dependencies does it need?
Is it lazy or eager?
Does it have init or destroy methods?
```

Only after understanding this structure does Spring start creating objects.

Where does Spring read bean definitions from?

Spring can read bean definitions from different sources.

Coder Army

Common sources are:

1. **Annotation-based configuration**

```
@Configuration
@ComponentScan
public class AppConfig {
}
```

```
@Component
public class PaymentService {
}
```

1. **Java-based `@Bean` configuration**

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

1. **XML-based configuration**

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

---

6. Step 2: Bean Object is Instantiated

After reading the bean definition, Spring creates the actual object.

This step is called **instantiation**.

Example:

```
@Component
public class PaymentService {
```

```
    public PaymentService() {
        System.out.println("PaymentService constructor calle
d");
    }
}
```

When Spring creates this bean, the constructor is called.

```
Instantiation = object is created
Initialization = object is prepared after creation
```

These two are not the same.

A common beginner mistake is thinking that object creation and initialization are the same thing.

They are different phases in the Spring lifecycle.

---

7. Step 3: Dependencies are Injected

After or during object creation, Spring supplies the required dependencies.

This is called **dependency injection**.

The exact timing depends on the type of injection.

Constructor Injection

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
```

Coder Army

```
    }
}
```

In constructor injection, the dependency is supplied while creating the object.

Flow:

```
Spring needs OrderService
        ↓
OrderService needs PaymentService
        ↓
Spring creates or finds PaymentService
        ↓
Spring passes PaymentService into OrderService constructor
```

Setter Injection

```
@Component
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentServi
ce) {
        this.paymentService = paymentService;
    }
}
```

In setter injection:

```
Object is created first
        ↓
Spring calls the setter method
        ↓
Dependency is assigned
```

Coder Army

Coder Army

Field Injection

```
@Component
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

In field injection:

```
Object is created first
        ↓
Spring uses reflection
        ↓
Dependency is injected directly into the field
```

Field injection is common in examples, but constructor injection is generally preferred for clean and testable code.

---

8. Step 4: Aware Interfaces are Called

Sometimes a bean wants to know something about the Spring container.

For example:

```
What is my bean name?
Which BeanFactory created me?
Which ApplicationContext am I running inside?
```

Spring provides this information using **Aware interfaces**.

Aware interfaces are callback interfaces.

A callback means:

```
Spring automatically calls a method at the right time.
```

Coder Army

Common Aware Interfaces

Some common Aware interfaces are:

```
BeanNameAware
ApplicationContextAware
```

Example:

```
@Component("myCustomBeanName")
public class MyBean implements BeanNameAware, ApplicationCont
extAware {

    public MyBean() {
        System.out.println("1. Constructor called");
    }

    @Override
    public void setBeanName(String name) {
        System.out.println("2. BeanNameAware called");
        System.out.println("Bean name: " + name);
    }

    @Override
    public void setApplicationContext(ApplicationContext appl
icationContext) {
        System.out.println("3. ApplicationContextAware calle
d");
        System.out.println("ApplicationContext: " + applicati
onContext.getClass().getSimpleName());
    }
}
```

Output idea:

Coder Army

```
1. Constructor called
2. BeanNameAware called
Bean name: myCustomBeanName
3. ApplicationContextAware called
ApplicationContext: AnnotationConfigApplicationContext
```

Why is the method called `setBeanName()` and not `getBeanName()`?

Because Spring is giving the bean some information.

The direction is:

```
Spring container → bean
```

So Spring calls the method and sets the bean name into that bean.

If we call `setBeanName()` manually, it will behave like a normal Java method.

It will not change the actual name registered inside the Spring container.

Do we use Aware interfaces in normal business logic?

Usually, no.

In normal service classes, we rarely need them.

They are more useful when we are building:

```
framework-level code
logging utilities
custom libraries
infrastructure components
```

For learning Spring lifecycle, they are useful because they show how Spring communicates container information to beans.

Coder Army

9. Step 5: Initialization Callbacks

At this point:

```
Bean object is created
Dependencies are injected
Aware callbacks are completed
```

Now Spring gives the bean a chance to run some startup logic before it is used.

This is called the **initialization phase**.

Initialization logic can include:

```
validate configuration
initialize internal cache
check required setup
prepare resources
fill a HashMap with calculated values
log startup state
```

Common Ways to Write Initialization Logic

There are three common ways.

Option 1: `@PostConstruct`

```java
import jakarta.annotation.PostConstruct;

@Component
public class PaymentService {

    @PostConstruct
    public void init() {
        System.out.println("Bean initialized");
```

```
    }
}
```

`@PostConstruct` tells Spring:

```
After dependencies are injected, call this method once.
```

This is clean and commonly used.

In Spring 6 and Spring 7, use:

```
import jakarta.annotation.PostConstruct;
```

If you are using only Spring Core/Spring Context and not Spring Boot, you may need to add the Jakarta annotation dependency separately.

Maven dependency:

```
<dependency>
    <groupId>jakarta.annotation</groupId>
    <artifactId>jakarta.annotation-api</artifactId>
    <version>3.0.0</version>
</dependency>
```

Option 2: `InitializingBean`

```
import org.springframework.beans.factory.InitializingBean;

@Component
public class PaymentService implements InitializingBean {

    @Override
    public void afterPropertiesSet() {
        System.out.println("Bean initialized");
```

Coder Army

```
    }
}
```

Here the bean implements a Spring interface.

Spring calls `afterPropertiesSet()` during the initialization phase.

This works, but it tightly couples our class with Spring.

Option 3: Custom `initMethod`

```
@Configuration
public class AppConfig {

    @Bean(initMethod = "start")
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

```
public class PaymentService {

    public void start() {
        System.out.println("Bean initialized");
    }
}
```

Here we define our own method name and tell Spring to call it after dependency injection.

This keeps the class free from Spring-specific interfaces.

Initialization Callback Order

If all three are used together, the usual order is:

Coder Army

Coder Army

```
@PostConstruct
        ↓
InitializingBean.afterPropertiesSet()
        ↓
custom initMethod
```

In real projects, we usually do not use all three together.

Pick one clear approach.

---

10. Step 6: Bean is Ready to Use

After initialization is complete, the bean is fully ready.

Now other beans can use it.

For singleton beans, Spring stores the object and reuses the same instance again and again.

```
Bean created
→ dependencies injected
→ initialization completed
→ bean ready
```

At this stage, the bean can safely handle business logic.

---

11. Step 7: Destruction Callbacks

When the Spring container is shutting down, Spring gives beans a chance to clean up resources.

This is called the **destruction phase**.

Cleanup logic can include:

```
close database connection
stop background thread
release file handle
```

Coder Army

```
clear resource
flush pending data
```

Common Ways to Write Destruction Logic

There are three common ways.

Option 1: `@PreDestroy`

```java
import jakarta.annotation.PreDestroy;

@Component
public class PaymentService {

    @PreDestroy
    public void cleanup() {
        System.out.println("Cleaning up bean");
    }
}
```

Spring calls this method before destroying the bean.

Option 2: `DisposableBean`

```java
import org.springframework.beans.factory.DisposableBean;

@Component
public class PaymentService implements DisposableBean {

    @Override
    public void destroy() {
        System.out.println("Cleaning up bean");
```

```
    }
}
```

This works, but it couples our class with Spring.

Option 3: Custom `destroyMethod`

```
@Configuration
public class AppConfig {

    @Bean(destroyMethod = "stop")
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

```
public class PaymentService {

    public void stop() {
        System.out.println("Cleaning up bean");
    }
}
```

Destruction Callback Order

If all three are used together, the usual order is:

```
@PreDestroy
        ↓
DisposableBean.destroy()
        ↓
custom destroyMethod
```

Again, in real projects, we usually choose one approach.

Coder Army

Coder Army

12. Full Lifecycle Summary

```
1. Spring container starts

2. Spring reads configuration and annotations

3. Spring creates BeanDefinition

4. Spring instantiates the bean object

5. Spring injects dependencies

6. Spring calls Aware interfaces

7. Spring runs initialization callbacks
   - @PostConstruct
   - InitializingBean
   - custom initMethod

8. Bean is ready to use

9. Application uses the bean

10. Spring runs destruction callbacks
   - @PreDestroy
   - DisposableBean
   - custom destroyMethod

11. Bean is removed
```

---

13. Singleton Bean Lifecycle

By default, Spring beans are singleton scoped.

Singleton means:

```
Spring creates only one object for that bean.
```

For singleton beans, Spring manages the complete lifecycle.

```
creation
→ dependency injection
```

```
→ initialization
→ usage
→ destruction
```

For example:

```java
@Component
public class PaymentService {

    public PaymentService() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("@PostConstruct called");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("@PreDestroy called");
    }
}
```

When the context starts:

```
Constructor called
@PostConstruct called
```

When the context closes:

```
@PreDestroy called
```

So for singleton beans:

Coder Army

Coder Army

```
Spring creates it.
Spring stores it.
Spring reuses it.
Spring destroys it.
```

---

14. Prototype Bean Lifecycle

Prototype means:

```
A new object is created every time the bean is requested.
```

Example:

```java
@Component
@Scope("prototype")
public class PaymentService {

    public PaymentService() {
        System.out.println("Constructor called");
    }
}
```

Whenever we request this bean:

```java
PaymentService p1 = context.getBean(PaymentService.class);
PaymentService p2 = context.getBean(PaymentService.class);
```

Spring creates two different objects.

```
p1 and p2 are different instances.
```

Prototype Lifecycle Flow

Coder Army

For prototype beans:

```
Container starts
        ↓
BeanDefinition is read
        ↓
No object is usually created at startup
        ↓
Client requests prototype bean
        ↓
Spring creates a new object
        ↓
Dependencies are injected
        ↓
Aware callbacks are called
        ↓
Initialization callbacks are called
        ↓
Spring hands object to the client
        ↓
Spring does not track it for normal destruction
```

Important Difference Between Singleton and Prototype

For singleton:

```
Spring creates it.
Spring stores it.
Spring reuses it.
Spring destroys it.
```

For prototype:

```
Spring creates it.
Spring prepares it.
Spring gives it to the caller.
After that, the caller is responsible for it.
```

Spring manages the beginning of the prototype lifecycle, but not the complete ending.

Coder Army

15. Prototype Destruction Does Not Happen Automatically

Consider this example:

```
@Component
@Scope("prototype")
public class PaymentService {

    public PaymentService() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("@PostConstruct called");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("@PreDestroy called");
    }
}
```

Main code:

```
AnnotationConfigApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.clas
s);

PaymentService p1 = context.getBean(PaymentService.class);

context.close();
```

Expected output idea:

Coder Army

```
Constructor called
@PostConstruct called
```

Usually, we will not see:

```
@PreDestroy called
```

Why does `@PreDestroy` not run for prototype beans?

Because after Spring creates a prototype bean and gives it to the caller, Spring does not keep tracking that specific object.

For singleton beans, Spring knows:

```
I created this one object.
It is stored in my singleton cache.
When the context closes, I can destroy it.
```

For prototype beans, Spring says:

```
I created this object.
I prepared it.
I gave it to the caller.
Now the caller may use it, store it, discard it, or create many more copies.
I will not track all prototype instances for automatic destruction.
```

Why does Spring not keep a list of every prototype object?

Technically, Spring could keep references to every prototype object.

But that would create a serious problem.

Prototype beans can be created many times.

If Spring stored every prototype object forever, those objects could not be garbage collected.

That could cause memory leaks.

So Spring does not fully manage prototype destruction.

Does `context.close()` destroy prototype beans?

Usually, no.

`context.close()` destroys singleton beans because Spring has them registered and tracked.

But prototype instances that were already handed out are not automatically destroyed.

So if a prototype bean holds expensive resources, we should clean them manually.

For example:

```
close the resource manually
use try-with-resources where possible
avoid putting heavy resource cleanup responsibility inside prototype beans
```

Java can still garbage collect the prototype object when there are no references to it.

But garbage collection and Spring destruction callbacks are not the same thing.

---

16. Important Case: Singleton Depends on Prototype

Now consider this case:

```
OrderService is singleton.
PaymentSession is prototype.
OrderService depends on PaymentSession.
```

Example:

Coder Army

Coder Army

```java
@Component
public class OrderService {

    private final PaymentSession paymentSession;

    public OrderService(PaymentSession paymentSession) {
        this.paymentSession = paymentSession;
    }
}
```

```java
@Component
@Scope("prototype")
public class PaymentSession {

    public PaymentSession() {
        System.out.println("PaymentSession created");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("PaymentSession destroyed");
    }
}
```

Since `OrderService` is singleton, it is created only once.

At the time of creating `OrderService`, Spring creates one `PaymentSession` and injects it.

After that, the same `PaymentSession` instance remains inside `OrderService`.

So even though `PaymentSession` is prototype, it does not automatically mean a new object will be used for every method call inside the singleton.

Question

Coder Army

```
When the context closes, will Spring call @PreDestroy on PaymentSession?
```

Answer:

```
No, usually Spring will not automatically call @PreDestroy on that prototype instance.
```

Reason:

```
PaymentSession is prototype scoped.
Spring created it, initialized it, and handed it to OrderService.
After that, Spring does not manage its full destruction lifecycle.
```

Key Learning

Prototype scope means:

```
new object per request from Spring
```

It does not mean:

```
new object every time a singleton method runs
```

If a singleton needs a fresh prototype instance every time, we need other approaches like:

```
ObjectProvider
Provider
lookup method injection
applicationContext.getBean()
```

Example using `ObjectProvider`:

Coder Army

```
@Component
public class OrderService {

    private final ObjectProvider<PaymentSession> paymentSessi
onProvider;

    public OrderService(ObjectProvider<PaymentSession> paymen
tSessionProvider) {
        this.paymentSessionProvider = paymentSessionProvider;
    }

    public void createOrder() {
        PaymentSession session = paymentSessionProvider.getOb
ject();
        System.out.println(session);
    }
}
```

Now a new prototype object can be requested whenever needed.

---

17. `@Lazy` and Bean Lifecycle

By default, singleton beans are eagerly created.

That means:

```
When the Spring container starts, singleton beans are usually created immediately.
```

But if we mark a singleton bean as `@Lazy`, Spring reads its bean definition but does not create the object immediately.

Example:

```
@Component
@Lazy
```

Coder Army

```java
public class PaymentService {

    public PaymentService() {
        System.out.println("PaymentService created");
    }
}
```

Flow:

```
Container starts
        ↓
BeanDefinition is registered
        ↓
Object is not created yet
        ↓
Bean is created only when someone asks for it
```

So for a lazy singleton:

```
BeanDefinition is read at startup.
Actual object creation is delayed.
Dependency injection and initialization also happen later.
```

---

18. `@PostConstruct` and Circular Dependency

```java
@Component
public class A {

        B b;

    public A(B b) {
    }
}
```

Coder Army

```
@Component
public class B {

        A a;

    public B(A a) {
    }
}
```

Here Spring cannot create `A` without `B`, and it cannot create `B` without `A`.

So constructor circular dependency fails.

Where can `@PostConstruct` help?

```
@Component
public class A {
    B b;

    public A(B b) {
        this.b = b;
    }

    @PostConstruct
    public void setB() {
        b.setA(this);
    }
}


@Component
public class B {
    A a;

    public void setA(A a) {
```

```
        this.a = a;
    }
}
```

Coder Army

20. Final Revision Summary

Bean lifecycle is the complete journey of a Spring bean.

The main phases are:

```
BeanDefinition reading
→ object creation
→ dependency injection
→ Aware callbacks
→ BeanPostProcessor before initialization
→ initialization callbacks
→ BeanPostProcessor after initialization
→ bean ready
→ destruction callbacks
```

For singleton beans:

```
Spring manages the full lifecycle from creation to destruction.
```

For prototype beans:

```
Spring creates, injects dependencies, initializes, and gives the object to the caller.
After that, Spring does not automatically destroy it.
```

For `@Lazy` singleton beans:

```
BeanDefinition is read at startup.
Object creation is delayed until the bean is actually needed.
```

For `@PostConstruct`:

```
It is used to run logic after dependency injection.
It is part of initialization.
```

---

21. One-Line Takeaway

Spring Bean Lifecycle is not just object creation. It is the complete process through which Spring discovers, creates, prepares, manages, and destroys a bean.

Coder Army

Coder Army

Spring XML-Based Configuration

1. Why XML Configuration Exists

In Spring, annotations are not the only way to tell the IoC container which objects it should manage.

Before annotation-based configuration became common, Spring applications were mostly configured using XML files.

XML configuration is another way of telling Spring:

> These are my classes.
>
> These are the objects you need to create.
>
> These are their dependencies.
>
> Please manage them inside the IoC container.

---

2. What Does Spring Actually Need?

The main job of the Spring IoC container is to:

> Create objects, connect their dependencies, manage their lifecycle, and destroy them when required.

To do this, Spring needs configuration information such as:

1. Which classes should become beans?
2. What should be the bean name?
3. Should the bean be `singleton` or `prototype`?
4. Which dependency should be injected?
5. Which bean should be preferred if multiple beans of the same type exist?
6. Which method should run after bean creation?

7. Which method should run before bean destruction?

This information is called **configuration metadata**.

3. Configuration Metadata in Spring

Spring can receive configuration metadata in different ways:

Configuration Style	Where Configuration Is Written
XML-based configuration	Inside an XML file
Annotation-based configuration	Inside Java classes
Java-based configuration	Inside Java config classes using `@Configuration` and `@Bean`

No matter which style we use, Spring converts this configuration metadata into internal objects called **BeanDefinition** objects.

```
Configuration Metadata
        |
        |---- XML
        |---- Annotations
        |
   BeanDefinition
        |
   IoC Container
        |
Creates + Wires + Manages Beans
```

The important idea is:

> XML, annotations, and Java config are only different ways of giving instructions to Spring.
>
> Internally, Spring uses those instructions to create bean definitions.

4. Why Should We Learn XML Configuration?

Even though modern Spring projects mostly use annotations and Java-based configuration, XML is still useful to understand.

Coder Army

Coder Army

Reason 1: Legacy Projects

Many old Spring applications still use XML configuration.

If you work on an older enterprise project, you may find files like:

```
beans.xml
applicationContext.xml
spring-context.xml
```

Understanding XML configuration helps you read and maintain such projects.

Reason 2: Better Understanding of Spring Internals

Annotations sometimes hide what Spring is doing internally.

XML makes the wiring more visible because we manually write:

- Which bean should be created
- What its name should be
- Which dependency should be injected
- Which lifecycle method should be called

This makes XML a good way to understand the foundation of Spring.

---

5. Recommended Project Structure

A simple XML-based Spring project can follow this structure:

```
src
 └── main
     ├── java
     │   └── in
     │       └── strikes
     │           ├── Main.java
     │           ├── PaymentService.java
     │           └── OrderService.java
     │
     └── resources
         └── beans.xml
```

Coder Army

The XML file should be placed inside:

```
src/main/resources
```

This is important because files inside `resources` become available on the **classpath**.

So when we write:

```
new ClassPathXmlApplicationContext("beans.xml");
```

Spring searches for `beans.xml` from the classpath.

Part 1: Creating the First XML Bean

6. Create a Simple Java Class

```
package in.strikes;

public class PaymentService {

    public PaymentService() {
        System.out.println("PaymentService object created");
    }

    public void pay() {
        System.out.println("Payment successful");
    }
}
```

This is a normal Java class.

There is no annotation like `@Component`.

So Spring will not automatically detect it.

We will register this class as a bean manually using XML.

Coder Army

7. Create `beans.xml`

Inside `src/main/resources`, create a file named:

```
beans.xml
```

Add the basic XML structure:

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

</beans>
```

For now, this XML file is empty.

The root `<beans>` tag is the place where we write all bean definitions.

---

8. Register the First Bean

Now add this inside the `<beans>` tag:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

Complete `beans.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="paymentService" class="in.strikes.PaymentService"/>
```

Coder Army

```
</beans>
```

This line means:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

> Spring, create and manage one object of `PaymentService`.
>
> Keep its bean name as `paymentService`.

Here:

XML Attribute	Meaning
`id`	Name of the bean inside the Spring container
`class`	Fully qualified class name whose object Spring should create

9. XML Bean Compared with `@Component`

In annotation-based configuration, we write:

```
@Component
public class PaymentService {
}
```

Spring usually gives this bean the default name:

```
paymentService
```

In XML configuration, we manually write:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

So this XML configuration is conceptually similar to:

Coder Army

```
@Component("paymentService")
public class PaymentService {
}
```

The difference is:

Annotation Config	XML Config
Configuration is written inside Java class	Configuration is written inside XML file
Spring detects class using component scanning	Spring reads bean definition from XML
Bean name may be generated automatically	Bean name is usually written manually

10. Load XML Configuration in Java

Create `Main.java`:

```
package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {

    public static void main(String[] args) {

        ApplicationContext context =
                new ClassPathXmlApplicationContext("beans.xml");

        PaymentService paymentService =
                context.getBean("paymentService", PaymentService.class);

        paymentService.pay();
    }
}
```

Output:

```
PaymentService object created
Payment successful
```

Coder Army

11. Why Did the Constructor Run Immediately?

When we write:

```
ApplicationContext context =
        new ClassPathXmlApplicationContext("beans.xml");
```

Spring loads the XML file and creates the beans.

By default, Spring beans are:

```
singleton + eager
```

This means Spring creates the bean immediately when the container starts.

That is why this message prints before calling `getBean()`:

```
PaymentService object created
```

Part 2: Getting Beans from the Container

12. Get Bean by Name

```
PaymentService paymentService =
        (PaymentService) context.getBean("paymentService");
```

This works, but we need manual type casting.

---

13. Get Bean by Name and Type

Better approach:

```
PaymentService paymentService =
```

```
        context.getBean("paymentService", PaymentService.class);
```

This is safer and cleaner because it avoids manual casting.

14. Get Bean by Type

If there is only one bean of that type, we can write:

```
PaymentService paymentService =
        context.getBean(PaymentService.class);
```

This works only when Spring has exactly one matching bean of that type.

Example:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

This is fine because only one `PaymentService` bean exists.

But if we have two beans of the same type:

```
<bean id="paymentService1" class="in.strikes.PaymentService"/>
<bean id="paymentService2" class="in.strikes.PaymentService"/>
```

and we call:

```
PaymentService paymentService =
        context.getBean(PaymentService.class);
```

Spring will not randomly choose one.

It will throw:

```
NoUniqueBeanDefinitionException
```

because multiple beans match the same type.

Coder Army

Coder Army

Part 3: Bean Naming in XML

15. Bean Name Convention

Bean names usually follow Java variable naming style.

Example:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

Here, the bean name is:

```
paymentService
```

---

16. What If We Do Not Give an `id`?

We can write:

```
<bean class="in.strikes.PaymentService"/>
```

In this case, we have not manually given the bean a clear name.

So this will not work:

```
context.getBean("paymentService", PaymentService.class);
```

because we did not define any bean named `paymentService`.

However, this may still work:

```
PaymentService paymentService =
        context.getBean(PaymentService.class);
```

because Spring can search by type.

But again, this only works when there is exactly one matching bean of that type.

For beginner-level XML configuration, it is better to give beans a clear `id`.

---

17. What If Two Beans Have the Same `id`?

This is invalid:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

In the same XML configuration file, bean IDs should be unique.

Otherwise, Spring cannot clearly identify which bean is meant by:

```
context.getBean("paymentService");
```

---

18. Create Two Different Beans

Create another class:

```
package in.strikes;

public class OrderService {

    public OrderService() {
        System.out.println("OrderService object created");
    }

    public void placeOrder() {
        System.out.println("Order placed");
    }
}
```

Now register both beans in XML:

Coder Army

Coder Army

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
<bean id="orderService" class="in.strikes.OrderService"/>
```

Use them in Java:

```
PaymentService paymentService =
        context.getBean("paymentService", PaymentService.class);

OrderService orderService =
        context.getBean("orderService", OrderService.class);

paymentService.pay();
orderService.placeOrder();
```

Output:

```
PaymentService object created
OrderService object created
Payment successful
Order placed
```

Part 4: `id`, `name`, and Alias

19. `id` Attribute

The `id` attribute gives one unique name to a bean.

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

Now we can access it using:

```
PaymentService paymentService =
        context.getBean("paymentService", PaymentService.class);
```

Coder Army

20. `name` Attribute

XML also supports the `name` attribute.

```
<bean name="paymentService" class="in.strikes.PaymentService"/>
```

This also creates a bean that can be accessed as:

```
PaymentService paymentService =
        context.getBean("paymentService", PaymentService.class);
```

At a beginner level, both of these look similar:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

```
<bean name="paymentService" class="in.strikes.PaymentService"/>
```

Both allow us to access the bean using the name `paymentService`.

---

21. Multiple Names Using `name`

The `id` attribute gives one main name.

The `name` attribute can be used for additional aliases.

Example:

```
<bean id="paymentService"
      name="payService, gatewayService, transactionService"
      class="in.strikes.PaymentService"/>
```

Now the same bean can be accessed using different names:

```
context.getBean("paymentService", PaymentService.class);
context.getBean("payService", PaymentService.class);
```

Coder Army

```
context.getBean("gatewayService", PaymentService.class);
context.getBean("transactionService", PaymentService.class);
```

All of these refer to the same bean.

---

22. External Alias Using `<alias>`

Sometimes a bean is already defined:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

Later, we may want to give it another name without changing the original bean definition.

We can use:

```
<alias name="paymentService" alias="payService"/>
```

Now this also works:

```
PaymentService paymentService =
        context.getBean("payService", PaymentService.class);
```

Part 5: Dependency Injection in XML

23. Two Common Ways of Dependency Injection in XML

In XML configuration, dependency injection is mainly done in two ways:

1. Constructor Injection
2. Setter Injection

Coder Army

XML does not directly perform field injection in the common style because XML works by calling constructors or setter methods.

> In XML configuration, Spring injects dependencies mainly through constructors or setters.

Part 6: Constructor Injection

24. Constructor Injection with Bean Reference

Suppose `OrderService` depends on `PaymentService`.

```
package in.strikes;

public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        System.out.println("OrderService constructor called");
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

XML configuration:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>

<bean id="orderService" class="in.strikes.OrderService">
    <constructor-arg ref="paymentService"/>
</bean>
```

This means:

Coder Army

```
Create PaymentService bean.
Create OrderService bean.
While creating OrderService, pass paymentService into its constructor.
```

The important line is:

```
<constructor-arg ref="paymentService"/>
```

Here:

Attribute	Meaning
`constructor-arg`	Pass value/dependency through constructor
`ref`	Reference to another bean in the Spring container

---

25. Constructor Injection with Simple Value

Suppose `PaymentService` has a constructor that takes a simple value:

```
package in.strikes;

public class PaymentService {

    private final String providerName;

    public PaymentService(String providerName) {
        this.providerName = providerName;
    }

    public void pay() {
        System.out.println("Payment successful using " + providerName);
    }
}
```

XML:

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg value="Razorpay"/>
</bean>
```

Coder Army

Here:

```
value="Razorpay"
```

means Spring will pass `"Razorpay"` as a normal value, not as a bean reference.

---

26. Constructor Injection with Multiple Arguments

```
package in.strikes;

public class PaymentService {

    private final String providerName;
    private final int retryCount;

    public PaymentService(String providerName, int retryCount) {
        this.providerName = providerName;
        this.retryCount = retryCount;
    }

    public void pay() {
        System.out.println("Payment successful using " + providerName);
        System.out.println("Retry count: " + retryCount);
    }
}
```

XML:

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg value="Razorpay"/>
    <constructor-arg value="3"/>
</bean>
```

Spring tries to match constructor arguments by order and type.

---

27. Constructor Injection by Index

To make the order clear, we can use `index`.

Coder Army

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg index="0" value="Razorpay"/>
    <constructor-arg index="1" value="3"/>
</bean>
```

Here:

```
index="0" → first constructor parameter
index="1" → second constructor parameter
```

This is useful when a constructor has multiple parameters.

---

28. Constructor Injection by Name

If constructor parameter names are available, we can write:

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg name="providerName" value="Razorpay"/>
    <constructor-arg name="retryCount" value="3"/>
</bean>
```

This is more readable.

However, constructor injection by name depends on whether Spring can discover parameter names.

For teaching examples, `index` is usually easier to demonstrate reliably.

---

29. Constructor Injection by Type

We can also specify the type:

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg type="java.lang.String" value="Razorpay"/>
    <constructor-arg type="int" value="3"/>
</bean>
```

Coder Army

This helps Spring choose the correct constructor argument when matching becomes confusing.

30. Summary of Constructor Injection Options

Method	Example	Use Case
By order	`<constructor-arg value="Razorpay"/>`	Simple cases
By index	`<constructor-arg index="0" value="Razorpay"/>`	When order must be clear
By name	`<constructor-arg name="providerName" value="Razorpay"/>`	When parameter names are available
By type	`<constructor-arg type="java.lang.String" value="Razorpay"/>`	When constructor matching needs help

Part 7: Setter Injection

31. Create `OrderService` with Setter

```
package in.strikes;

public class OrderService {

    private PaymentService paymentService;

    public OrderService() {
        System.out.println("OrderService object created");
    }

    public void setPaymentService(PaymentService paymentService) {
        System.out.println("Setter called for paymentService");
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

Coder Army

The setter method is important:

```
public void setPaymentService(PaymentService paymentService)
```

XML setter injection works by calling this setter method.

---

32. XML Configuration for Setter Injection

```
<bean id="paymentService" class="in.strikes.PaymentService"/>

<bean id="orderService" class="in.strikes.OrderService">
    <property name="paymentService" ref="paymentService"/>
</bean>
```

The important line is:

```
<property name="paymentService" ref="paymentService"/>
```

It has two parts:

```
name="paymentService"
```

This refers to the property inside `OrderService`.

Spring looks for a setter method like:

```
setPaymentService(...)
```

Then:

```
ref="paymentService"
```

This refers to another bean in the Spring container.

Coder Army

So the XML means:

```
Create OrderService.
Call setPaymentService(...).
Pass the bean named paymentService into it.
```

---

33. `value` vs `ref`

In XML configuration, `value` and `ref` are different.

Attribute	Meaning
`value`	Injects a simple value like String, int, boolean
`ref`	Injects another bean from the Spring container

Example using `value`:

```
<constructor-arg value="Razorpay"/>
```

Example using `ref`:

```
<constructor-arg ref="paymentService"/>
```

Part 8: Duplicate Beans During Dependency Injection

34. Problem: Multiple Beans of Same Type

Suppose `OrderService` needs one `PaymentService`.

But the XML has two payment service beans:

```
<bean id="razorpayPaymentService" class="in.strikes.RazorpayPaymentService"/>
<bean id="stripePaymentService" class="in.strikes.StripePaymentService"/>
```

Coder Army

Now Spring needs to know which one should be injected into `OrderService`.

Spring should not guess randomly.

---

35. Solution 1: Explicit Wiring Using `ref`

The cleanest solution in XML is explicit wiring.

```
<bean id="razorpayPaymentService" class="in.strikes.RazorpayPaymentService"/>
<bean id="stripePaymentService" class="in.strikes.StripePaymentService"/>

<bean id="orderService" class="in.strikes.OrderService">
    <constructor-arg ref="razorpayPaymentService"/>
</bean>
```

Now there is no confusion.

This line:

```
<constructor-arg ref="razorpayPaymentService"/>
```

means:

> When creating `OrderService`, inject the bean named `razorpayPaymentService`.

Even if multiple beans are available, Spring knows exactly which one to use.

---

36. Solution 2: Using `primary="true"`

We can mark one bean as primary:

```
<bean id="razorpayPaymentService"
      class="in.strikes.RazorpayPaymentService"
      primary="true"/>

<bean id="stripePaymentService"
      class="in.strikes.StripePaymentService"/>
```

Coder Army

Now, when Spring needs a bean by type and multiple options exist, it will prefer the primary bean.

However, in XML projects, explicit wiring using `ref` is usually clearer than depending on `primary`.

---

37. Solution 3: Using `autowire-candidate="false"`

This is a more advanced XML option.

```
<bean id="razorpayPaymentService"
      class="in.strikes.RazorpayPaymentService"/>

<bean id="stripePaymentService"
      class="in.strikes.StripePaymentService"
      autowire-candidate="false"/>
```

This tells Spring:

> Do not consider `stripePaymentService` as a candidate for autowiring.

This can be useful in legacy XML projects, but beginners should first understand explicit wiring with `ref`.

Part 9: XML Autowiring

38. What Is XML Autowiring?

XML autowiring means:

> Instead of writing every dependency manually using `ref`, we ask Spring to find the dependency automatically.

Without autowiring:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>

<bean id="orderService" class="in.strikes.OrderService">
```

```
    <property name="paymentService" ref="paymentService"/>
</bean>
```

With XML autowiring:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>

<bean id="orderService"
      class="in.strikes.OrderService"
      autowire="byName"/>
```

Spring tries to figure out the dependency automatically.

39. Common XML Autowiring Modes

Mode	Meaning
`byName`	Matches dependency using property name and bean name
`byType`	Matches dependency using type
`constructor`	Matches constructor parameters using available beans

40. `autowire="byName"`

```
<bean id="paymentService" class="in.strikes.PaymentService"/>

<bean id="orderService"
      class="in.strikes.OrderService"
      autowire="byName"/>
```

This means:

> Spring matches dependency using the property name and bean name.

If `OrderService` has this setter:

```
public void setPaymentService(PaymentService paymentService) {
    this.paymentService = paymentService;
```

Coder Army

```
}
```

Spring sees the property name:

```
paymentService
```

Then it searches for a bean named:

```
paymentService
```

If found, Spring injects it automatically.

---

41. `autowire="byType"`

```
<bean id="abc" class="in.strikes.PaymentService"/>

<bean id="orderService"
      class="in.strikes.OrderService"
      autowire="byType"/>
```

This means:

> Spring matches dependency using type, not name.

If `OrderService` needs a `PaymentService`, Spring searches for a bean of type `PaymentService`.

This works only when there is one clear matching bean.

If multiple beans of the same type exist, Spring cannot decide and may throw an exception.

---

42. `autowire="constructor"`

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

Coder Army

Coder Army

```
<bean id="orderService"
      class="in.strikes.OrderService"
      autowire="constructor"/>
```

This means:

> Spring uses constructor injection automatically by matching constructor parameters with available beans.

If `OrderService` has a constructor that needs `PaymentService`, Spring tries to find a matching `PaymentService` bean and pass it into the constructor.

---

43. Comparison of XML Autowiring Modes

XML Autowiring Mode	How Spring Matches Dependency	Works Best When
`byName`	Property name matches bean name	Bean names are predictable
`byType`	Property type matches bean type	Only one bean of that type exists
`constructor`	Constructor parameter type matches bean type	Constructor dependencies have clear matching beans

For beginners, it is better to first understand manual dependency injection using `ref`.

Autowiring should be treated as a convenience feature.

Part 10: Bean Scopes in XML

44. Two Important Scopes

For this video, focus mainly on two scopes:

1. `singleton`
2. `prototype`

Coder Army

45. Singleton Scope

By default, Spring beans are singleton.

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

This means Spring creates only one object of this bean inside the container.

If we call `getBean()` multiple times:

```
PaymentService p1 =
        context.getBean("paymentService", PaymentService.class);

PaymentService p2 =
        context.getBean("paymentService", PaymentService.class);

System.out.println(p1 == p2);
```

Output:

```
true
```

Both references point to the same object.

---

46. Prototype Scope

Prototype means Spring creates a new object every time we request the bean.

```
<bean id="paymentService"
      class="in.strikes.PaymentService"
      scope="prototype"/>
```

Now:

```
PaymentService p1 =
        context.getBean("paymentService", PaymentService.class);

PaymentService p2 =
```

```
        context.getBean("paymentService", PaymentService.class);

System.out.println(p1 == p2);
```

Output:

```
PaymentService object created
PaymentService object created
false
```

Each `getBean()` call creates a new object.

---

47. Singleton Bean Depending on Prototype Bean

Suppose:

```
OrderService → singleton
PaymentService → prototype
```

XML:

```
<bean id="paymentService"
      class="in.strikes.PaymentService"
      scope="prototype"/>

<bean id="orderService"
      class="in.strikes.OrderService"
      scope="singleton">
    <constructor-arg ref="paymentService"/>
</bean>
```

Important point:

> The prototype bean is injected when the singleton bean is created.

So if `OrderService` is singleton, it receives one `PaymentService` object at creation time.

It does not automatically get a new `PaymentService` every time we call a method on `OrderService`.

Coder Army

This is an important concept while working with mixed scopes.

Part 11: Bean Lifecycle Methods in XML

48. Custom Init and Destroy Methods

In XML configuration, we can tell Spring which method should run:

- After bean creation
- Before bean destruction

Create `PaymentService.java`:

```
package in.strikes;

public class PaymentService {

    public PaymentService() {
        System.out.println("1. PaymentService constructor called");
    }

    public void init() {
        System.out.println("2. PaymentService init method called");
    }

    public void pay() {
        System.out.println("3. Payment successful");
    }

    public void cleanup() {
        System.out.println("4. PaymentService cleanup method called");
    }
}
```

These are normal Java methods.

They do not need annotations.

They do not need to implement any Spring interface.

The XML file tells Spring when to call them.

Coder Army

Coder Army

49. XML Configuration with `init-method` and `destroy-method`

```
<bean id="paymentService"
      class="in.strikes.PaymentService"
      init-method="init"
      destroy-method="cleanup"/>
```

Here:

Attribute	Meaning
`init-method`	Method to call after bean creation and dependency injection
`destroy-method`	Method to call before bean destruction

50. Closing the Context

To see the destroy method run, the context must be closed.

Use `ClassPathXmlApplicationContext`:

```
ClassPathXmlApplicationContext context =
        new ClassPathXmlApplicationContext("beans.xml");

PaymentService paymentService =
        context.getBean("paymentService", PaymentService.class);

paymentService.pay();

context.close();
```

Better approach using try-with-resources:

```
try (ClassPathXmlApplicationContext context =
             new ClassPathXmlApplicationContext("beans.xml")) {

    PaymentService paymentService =
            context.getBean("paymentService", PaymentService.class);
```

```
    paymentService.pay();
}
```

When the context closes, Spring calls the destroy method for singleton beans.

51. Prototype Scope with Init and Destroy

```
<bean id="paymentService"
      class="in.strikes.PaymentService"
      scope="prototype"
      init-method="init"
      destroy-method="cleanup"/>
```

For prototype beans:

- Spring creates the object.
- Spring injects dependencies.
- Spring calls the init method.
- After that, Spring hands the object to the application.

Important:

> Spring does not automatically call the destroy method for prototype beans when the context closes.

This is because Spring does not manage the complete lifecycle of prototype beans after creation.

Part 12: Collection Injection in XML

52. Injecting a List

XML can also inject collections like `List`, `Set`, and `Map`.

Example:

Coder Army

Coder Army

```
package in.strikes;

import java.util.List;

public class UserService {

    private final List<String> userNames;

    public UserService(List<String> userNames) {
        this.userNames = userNames;
    }

    public void printUsers() {
        System.out.println(userNames);
    }
}
```

XML:

```
<bean id="userService" class="in.strikes.UserService">
    <constructor-arg name="userNames">
        <list>
            <value>Aditya</value>
            <value>Rohit</value>
            <value>Rohan</value>
        </list>
    </constructor-arg>
</bean>
```

---

53. Injecting a Map

```
package in.strikes;

import java.util.Map;

public class PaymentService {

    private final Map<String, Double> userBalances;

    public PaymentService(Map<String, Double> userBalances) {
        this.userBalances = userBalances;
    }

    public void printBalances() {
```

```
        System.out.println(userBalances);
    }
}
```

XML:

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg name="userBalances">
        <map>
            <entry key="Aditya" value="2.0"/>
            <entry key="Rohit" value="2.5"/>
            <entry key="Rohan" value="3.0"/>
        </map>
    </constructor-arg>
</bean>
```

This shows that XML can handle not only simple beans, but also more complex values and collections.

Part 13: Splitting XML Files

54. Why Split XML Files?

In real legacy Spring projects, one XML file can become very large.

Instead of keeping everything inside one `beans.xml`, we can split configuration into multiple XML files.

Example:

```
src/main/resources
 ├── applicationContext.xml
 ├── orderContext.xml
 └── userContext.xml
```

---

55. Example: `orderContext.xml`

Coder Army

Coder Army

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="paymentService" class="in.strikes.PaymentService"/>

    <bean id="orderService" class="in.strikes.OrderService">
        <constructor-arg ref="paymentService"/>
    </bean>

</beans>
```

56. Example: `userContext.xml`

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userService" class="in.strikes.UserService"/>

</beans>
```

57. Main File: `applicationContext.xml`

Now create a main XML file that imports other XML files.

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
    <import resource="orderContext.xml"/>
    <import resource="userContext.xml"/>

</beans>
```

Now in Java, we only load the main file:

```
ClassPathXmlApplicationContext context =
        new ClassPathXmlApplicationContext("applicationContext.xml");
```

Spring will load `applicationContext.xml`, and that file will import the other XML files.

Part 14: XML and Annotation Configuration Together

58. Can XML and Annotations Work Together?

Yes.

XML and annotations can work together because both are just different ways of giving configuration metadata to Spring.

For example, an old project may have:

- Some beans defined in XML
- Some beans detected using annotations
- Some beans created using Java config

Ultimately, Spring converts all of them into bean definitions.

---

59. Component Scanning from XML

In XML, we can enable annotation scanning using:

```
<context:component-scan base-package="in.strikes"/>
```

Coder Army

Coder Army

For this, the XML file needs the context namespace:

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           http://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="in.strikes"/>

</beans>
```

This tells Spring:

> Scan the package `in.strikes` and detect annotation-based components like `@Component`, `@Service`, and `@Repository`.

---

60. Should Beginners Mix XML and Annotations?

For learning, it is better to understand both separately first.

Mixing XML and annotations too early can become confusing because:

- XML beans can depend on annotation-based beans.
- Annotation-based beans can depend on XML beans.
- Component scanning and manual bean registration can create duplicate beans.
- Debugging becomes harder for beginners.

So the best teaching approach is:

1. First understand pure annotation-based configuration.
2. Then understand pure XML-based configuration.
3. Then briefly explain that both can work together in legacy projects.

Coder Army

Final Summary

Key Takeaways

- XML configuration is an older but important way of configuring Spring applications.
- XML files provide configuration metadata to Spring.
- Spring converts XML configuration into `BeanDefinition` objects.
- `<bean>` tells Spring to create and manage an object.
- `id` gives the bean a name inside the container.
- `class` tells Spring which class object should be created.
- Dependencies can be injected using constructor injection or setter injection.
- `value` is used for simple values.
- `ref` is used for another Spring bean.
- If multiple beans of the same type exist, explicit wiring using `ref` is the clearest solution.
- XML also supports autowiring using `byName`, `byType`, and `constructor`.
- The default bean scope is singleton.
- Prototype scope creates a new object every time `getBean()` is called.
- `init-method` and `destroy-method` allow lifecycle callbacks in XML.
- Spring does not automatically destroy prototype beans.
- Large XML configurations can be split into multiple files using `<import>`.
- XML and annotations can work together, but beginners should first understand them separately.

Coder Army

Spring Boot Annotations and Configuration

1. Why Spring Boot After Spring Core?

Till now, we learned how Spring Core manages objects using the IoC container.

In a pure Spring Core project, we usually start the container manually:

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.clas
s);
```

Then we provide a configuration class:

```
@Configuration
@ComponentScan(basePackages = "in.strikes")
public class AppConfig {
}
```

Then we fetch a bean manually and call a method:

```
OrderService orderService = context.getBean(OrderService.clas
s);
orderService.placeOrder();
```

This is useful for learning because it clearly shows how Spring works internally. But in real applications, this setup becomes repetitive.

Every Spring application needs common startup work:

```
Start application
Create Spring container
Read configuration
Scan components
Create beans
```

Coder Army

```
Inject dependencies
Load properties
Prepare application context
```

Spring Boot solves this repetitive startup problem.

> Spring Boot does not replace Spring Core.
>
> Spring Boot uses Spring Core and gives us a simpler way to start, configure, and run the Spring container.

A common beginner misunderstanding is:

```
Spring Boot = Web API
```

That is not correct.

Spring Boot can be used for:

- Console applications
- Web applications
- Database applications
- Batch applications
- Microservices

Spring Boot itself is not only about web development. Web behavior comes later when we add web-related dependencies.

---

2. Setting Up a Spring Boot Project

There are two common ways to create a Spring Boot project:

1. Create a Maven project manually and add Spring Boot dependencies.
2. Use Spring Initializr to generate a ready-made project structure.

In real projects, Spring Initializr is usually preferred because it creates the correct project structure, parent configuration, dependencies, and plugin setup.

Coder Army

3. Why Do We Use `spring-boot-starter-parent`?

In Spring Boot projects, we commonly use:

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.x.x</version>
    <relativePath/>
</parent>
```

The parent helps Maven by providing common Spring Boot project configuration.

It manages things like:

- Dependency versions
- Plugin versions
- Java version configuration
- Build defaults
- Encoding settings

The main benefit is version compatibility.

For example, if we later add dependencies like Spring JDBC, Hibernate, Validation, Jackson, or Spring Web, we do not have to manually decide every version.

Maven can ask the Spring Boot parent:

```
Which version of this dependency should I use?
```

The Spring Boot parent provides compatible versions.

This reduces version mismatch problems.

---

4. What Is a Spring Boot Starter?

A starter is a dependency shortcut.

Coder Army

Instead of adding many related dependencies one by one, we add one starter, and Maven brings the required dependencies transitively.

Example:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
</dependency>
```

`spring-boot-starter` is the basic starter for a non-web Spring Boot application.

It gives us the basic Spring Boot setup, logging, and core Spring Boot infrastructure.

Important point:

```
Starter does not mean one single library.
Starter means a collection of commonly needed dependencies.
```

---

5. Spring Core Startup vs Spring Boot Startup

In Spring Core

We manually created the Spring container:

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.clas
s);
```

We manually provided the configuration class:

```
@Configuration
@ComponentScan(basePackages = "in.strikes")
public class AppConfig {
}
```

Coder Army

We manually fetched the bean:

```
OrderService orderService = context.getBean(OrderService.clas
s);
orderService.placeOrder();
```

So in Spring Core, we usually handle these steps ourselves:

```
1. Create ApplicationContext manually
2. Provide configuration class manually
3. Fetch bean manually
4. Trigger application logic manually
```

In Spring Boot

Spring Boot gives us a standard startup mechanism:

```
SpringApplication.run(MyApplication.class, args);
```

This line starts and prepares the Spring application context for us.

---

6. Same Application in Spring Boot

A basic Spring Boot application starts like this:

```
package in.strikes;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootAppli
cation;

@SpringBootApplication
public class SpringBootCoreDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringBootCoreDemoApplication.c
```

Coder Army

```
lass, args);
    }
}
```

Here, we did not manually write:

```
new AnnotationConfigApplicationContext(...)
context.getBean(...)
@Configuration
@ComponentScan
```

The reason is that `@SpringBootApplication` already combines the common setup required for a Spring Boot application.

---

7. Adding Simple Components

We can create normal Spring components just like we did in Spring Core.

```
package in.strikes.service;

import org.springframework.stereotype.Component;

@Component
public class PaymentService {

    public void pay() {
        System.out.println("Payment completed");
    }
}
```

```
package in.strikes.service;

import org.springframework.stereotype.Component;

@Component
```

Coder Army

```
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

This is still Spring Core behavior.

Spring sees `@Component` and creates beans.

Spring sees constructor dependency and injects `PaymentService` into `OrderService`.

Spring Boot does not change dependency injection. It simply starts and configures the Spring container in a more convenient way.

---

8. How Will the Method Run?

In Spring Core, we manually fetched the bean and called the method:

```
OrderService orderService = context.getBean(OrderService.clas
s);
orderService.placeOrder();
```

In Spring Boot, we usually do not fetch beans manually from the context.

To run code after application startup, we can use `CommandLineRunner`.

```
package in.strikes.runner;

import in.strikes.service.OrderService;
```

Coder Army

```
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class AppRunner implements CommandLineRunner {

    private final OrderService orderService;

    public AppRunner(OrderService orderService) {
        this.orderService = orderService;
    }

    @Override
    public void run(String... args) {
        orderService.placeOrder();
    }
}
```

`CommandLineRunner` is a Spring Boot interface.

Its `run()` method executes after the Spring application context is created and ready.

This is useful for console-based Spring Boot applications, testing startup logic, or running some code immediately after the application starts.

---

9. Understanding `SpringApplication.run()`

The most important line in a Spring Boot application is:

```
SpringApplication.run(SpringBootCoreDemoApplication.class, ar
gs);
```

This line:

- Starts the Spring Boot application
- Creates the application context

Coder Army

- Reads configuration and properties
- Performs component scanning
- Creates beans
- Applies auto-configuration
- Injects dependencies
- Runs startup hooks like `CommandLineRunner`

`run()` also returns the application context:

```
ConfigurableApplicationContext context =
        SpringApplication.run(SpringBootCoreDemoApplication.c
lass, args);
```

Technically, we can still fetch beans manually:

```
OrderService orderService = context.getBean(OrderService.clas
s);
orderService.placeOrder();
```

But in real Spring Boot applications, this is not the usual approach.

The preferred approach is:

```
Let Spring inject dependencies wherever they are needed.
```

---

10. Understanding `@SpringBootApplication`

`@SpringBootApplication` is not a small annotation. It is a combination annotation.

It roughly combines:

```
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
```

Coder Army

So this:

```
@SpringBootApplication
public class SpringBootCoreDemoApplication {
}
```

is conceptually similar to:

```
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
public class SpringBootCoreDemoApplication {
}
```

Meaning:

Annotation	Purpose
`@SpringBootConfiguration`	Marks the main configuration class of the Spring Boot application
`@EnableAutoConfiguration`	Enables Spring Boot's automatic configuration mechanism
`@ComponentScan`	Scans the current package and subpackages for Spring components

We already studied `@Configuration` and `@ComponentScan` in Spring Core.

The major new concept here is:

```
@EnableAutoConfiguration
```

---

11. `@SpringBootConfiguration`

`@SpringBootConfiguration` is Spring Boot's version of a configuration marker.

In Spring Core, we used:

```
@Configuration
public class AppConfig {
```

Coder Army

```
    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

This class tells Spring:

```
This class contains bean definitions.
```

In Spring Boot, the main class indirectly gets `@SpringBootConfiguration` because of `@SpringBootApplication`.

That means the main class can also define beans:

```
@SpringBootApplication
public class SpringBootCoreDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringBootCoreDemoApplication.c
lass, args);
    }

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

This works because the main class is also treated as a configuration class.

However, in real projects, we usually keep the main class clean and create separate configuration classes when needed.

Example:

Coder Army

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Simple difference:

```
@Configuration = Regular Spring configuration class
@SpringBootConfiguration = Main Spring Boot application configuration marker
```

`@SpringBootConfiguration` also helps Spring Boot tests locate the main application configuration automatically.

---

12. `@ComponentScan` in Spring Boot

In Spring Core, we manually wrote:

```
@Configuration
@ComponentScan(basePackages = "in.strikes")
public class AppConfig {
}
```

This told Spring:

```
Go inside the in.strikes package and find classes marked with @Component, @Service, @Repo
sitory, or @Controller.
```

In Spring Boot, we usually do not write `@ComponentScan` manually because it is already included inside `@SpringBootApplication`.

Coder Army

By default, Spring Boot scans the package where the main application class is present and all its subpackages.

Example package structure:

```
in.strikes
├── SpringBootCoreDemoApplication.java
├── service
│   ├── PaymentService.java
│   └── OrderService.java
└── runner
    └── AppRunner.java
```

Here, the main class is inside:

```
in.strikes
```

So Spring Boot scans:

```
in.strikes and all packages under it
```

This is why Spring Boot applications usually keep the main application class in the root package.

Correct structure:

```
in.strikes
├── SpringBootCoreDemoApplication.java
├── controller
├── service
├── repository
└── config
```

Less ideal structure:

```
in.strikes.app
└── SpringBootCoreDemoApplication.java
```

Coder Army

```
in.strikes.service
└── PaymentService.java
```

In the second structure, `PaymentService` may not be scanned automatically because it is outside the main class package hierarchy.

---

13. Can We Manually Change Component Scanning?

Yes.

If required, we can manually provide base packages:

```
@SpringBootApplication(scanBasePackages = "in.strikes")
public class SpringBootCoreDemoApplication {
}
```

This tells Spring Boot to scan from `in.strikes` even if the main class is placed somewhere else.

However, the better practice is to place the main class in the root package and let Spring Boot scan naturally.

---

14. `@EnableAutoConfiguration`

`@EnableAutoConfiguration` tells Spring Boot:

```
Look at my project setup and automatically configure useful things for me.
```

Spring Boot checks:

- Which dependencies are present
- Which classes are available on the classpath
- Which beans already exist
- Which properties are configured

Then it creates useful default beans only when needed.

Coder Army

For example, Spring Boot may check:

```
Is a web dependency present?
Is a database dependency present?
Is Spring Security present?
Has the developer already created a custom bean?
```

Based on these checks, Spring Boot applies its pre-written configuration classes.

Examples of common auto-configuration classes:

```
DataSourceAutoConfiguration
WebMvcAutoConfiguration
JacksonAutoConfiguration
TaskExecutionAutoConfiguration
```

Simple meaning:

```
@EnableAutoConfiguration = Apply Spring Boot's ready-made configurations when conditions
match.
```

---

15. How Auto-Configuration Works Internally

When the application starts:

```
SpringApplication.run(SpringBootCoreDemoApplication.class, ar
gs);
```

Spring Boot sees:

```
@SpringBootApplication
```

Inside it, auto-configuration is enabled:

```
@EnableAutoConfiguration
```

CoderArmy

Then Spring Boot considers auto-configuration classes.

These classes are not blindly applied. They are applied only when their conditions match.

Conceptually, an auto-configuration class looks like this:

```
@AutoConfiguration
@ConditionalOnClass(SomeLibrary.class)
public class SomeLibraryAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public SomeService someService() {
        return new SomeService();
    }
}
```

Meaning:

```
If SomeLibrary is present in the project
AND the developer has not already created SomeService bean
THEN create a default SomeService bean.
```

This is the core idea of Spring Boot auto-configuration.

---

16. Manual Configuration vs Auto-Configuration

Manual Configuration

In Spring Core, the developer writes the configuration manually:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
```

Coder Army

```
        return new PaymentService();
    }
}
```

Here, the developer directly tells Spring:

```
Create this bean.
```

Auto-Configuration

In Spring Boot, many configuration classes are already written for us.

They are provided by Spring Boot or sometimes by third-party starters.

Spring Boot applies them only when the right conditions are satisfied.

So auto-configuration does not mean random configuration.

It means:

```
Pre-written configuration + condition-based activation
```

---

17. Classpath-Based Decision Making

Classpath means the set of classes and libraries available to the application at runtime.

In Maven terms:

```
Dependency added in pom.xml
        ↓
Maven downloads JAR files
        ↓
Those JARs become available to the application
        ↓
Spring Boot checks the available classes
        ↓
Auto-configuration decisions are made
```

When we add a dependency, we indirectly give Spring Boot a signal.

Coder Army

Example:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
</dependency>
```

This tells Spring Boot:

```
This is a basic Spring Boot application.
```

But if we do not add:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

then Spring Boot does not treat the application as a web application.

It checks:

```
Do I see web server classes? No.
Do I see Spring MVC web classes? No.
Should I start Tomcat? No.
```

So the application starts like a normal console application.

You will not see:

```
Tomcat started on port 8080
```

This is because no web-related classes are present on the classpath.

---

18. What Happens When We Add Web Dependency?

Coder Army

Later, when we add:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Maven brings web-related libraries.

Then Spring Boot sees:

```
Web classes are present.
Embedded server support is present.
Spring MVC infrastructure is present.
```

Now Spring Boot understands:

```
This looks like a web application.
```

So it configures web-related beans and starts an embedded server such as Tomcat.

That is why a Spring Boot web application keeps running and listens for HTTP requests.

---

19. Three Main Checks in Auto-Configuration

Spring Boot commonly checks three things:

```
1. Is the required class present?
2. Is the required bean missing?
3. Is the required property enabled?
```

Conceptual example:

```
@AutoConfiguration
@ConditionalOnClass(PaymentGateway.class)
```

Coder Army

```java
public class PaymentAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public PaymentGateway paymentGateway() {
        return new DefaultPaymentGateway();
    }
}
```

Meaning:

```
If PaymentGateway class exists
AND the developer has not created a PaymentGateway bean
THEN create a default PaymentGateway bean.
```

This is why Spring Boot feels automatic but still remains controlled.

---

20. `@ConditionalOnClass`

`@ConditionalOnClass` is a condition that matches only when a particular class is present on the classpath.

Simple meaning:

```
Apply this configuration only if a specific class or library is available in the project.
```

Example idea:

```java
@ConditionalOnClass(SomeLibrary.class)
```

This means:

```
Only activate this configuration if SomeLibrary is present.
```

This is how Spring Boot detects what kind of application we are building.

CoderArmy

21. `@ConditionalOnMissingBean`

`@ConditionalOnMissingBean` is a condition that matches only when a required bean is not already present in the Spring container.

Simple meaning:

```
Create this bean only if the developer has not already created one.
```

This is very important because Spring Boot does not want to override our custom configuration unnecessarily.

Example:

```
@Bean
@ConditionalOnMissingBean
public SomeService someService() {
    return new DefaultSomeService();
}
```

Meaning:

```
If the developer has not created SomeService bean, then create the default one.
```

This is why our own beans usually get preference over Boot's default beans.

---

22. Our Beans vs Auto-Configured Beans

In a Spring Boot application, beans can come from two main sources.

1. Beans Created from Our Code

These are developer-defined beans.

Example using `@Component`:

```
@Component
public class PaymentService {
```

Coder Army

```
    public void pay() {
        System.out.println("Payment done");
    }
}
```

Example using `@Bean`:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

These beans are usually discovered through component scanning or explicit configuration.

2. Beans Created by Spring Boot Auto-Configuration

These beans come from Spring Boot's pre-written configuration classes.

Conceptual example:

```
@AutoConfiguration
@ConditionalOnClass(SomeLibrary.class)
public class SomeLibraryAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public SomeService someService() {
        return new DefaultSomeService();
    }
}
```

Coder Army

These are not classes we write in our application.

They are provided by Spring Boot or by a third-party starter.

---

23. Complete Spring Boot Startup Flow

A simplified Spring Boot startup flow looks like this:

```
Application starts
        ↓
SpringApplication.run() is called
        ↓
Application context is created
        ↓
User configuration is processed
        ↓
Component scanning finds our classes
        ↓
User-defined beans are registered
        ↓
Auto-configuration classes are considered
        ↓
Conditions are checked
        ↓
Missing default beans are created
        ↓
Dependencies are injected
        ↓
Application context becomes ready
        ↓
CommandLineRunner/ApplicationRunner runs, if present
```

---

24. Important Difference: `@ComponentScan` vs `@EnableAutoConfiguration`

These two annotations solve different problems.

Annotation	What It Does
`@ComponentScan`	Finds our application classes marked with `@Component`, `@Service`, `@Repository`, `@Controller`, etc.
`@EnableAutoConfiguration`	Applies Spring Boot's ready-made configurations based on classpath, beans, and properties.

Coder Army

Simple comparison:

```
@ComponentScan finds our code.
@EnableAutoConfiguration applies Boot's default setup.
```

Both are included inside `@SpringBootApplication`.

---

25. Final Takeaways

- Spring Boot does not replace Spring Core.
- Spring Boot uses Spring Core internally.
- Spring Core gives us IoC and dependency injection.
- Spring Boot gives us a standard way to start and configure the application.
- `SpringApplication.run()` creates and prepares the application context.
- `@SpringBootApplication` combines `@SpringBootConfiguration`, `@EnableAutoConfiguration`, and `@ComponentScan`.
- `@ComponentScan` finds our classes.
- `@EnableAutoConfiguration` applies Spring Boot's ready-made configurations.
- Auto-configuration works through conditions.
- `@ConditionalOnClass` checks whether a class or library is present.
- `@ConditionalOnMissingBean` creates a bean only when the developer has not already created one.
- Adding dependencies changes the classpath, and Spring Boot uses that classpath to decide what to configure.
- A basic Spring Boot starter does not start a web server.
- A web starter adds web-related classes, so Spring Boot starts an embedded server.

---

26. One-Line Summary

Spring Boot is not magic. It is Spring Core plus a smart startup and configuration system that uses dependencies, properties, and conditions to prepare the application automatically.

Coder Army

Coder Army

Spring Boot Configuration: `application.properties`, `@Value`, `@ConfigurationProperties`, and Runner Interfaces

1. Why Do We Need Configuration?

In real applications, not every value should be hardcoded inside Java classes.

For example:

```
@Component
public class PaymentService {

    private String providerName = "Razorpay";
    private int retryCount = 3;

    public void pay() {
        System.out.println("Payment done using " + providerNa
me);
        System.out.println("Retry count: " + retryCount);
    }
}
```

This code works, but the values are fixed inside the Java source code.

Now imagine that tomorrow we want to change the payment provider from:

```
Razorpay
```

Coder Army

to:

```
Stripe
```

If the value is hardcoded, we would need to:

1. Change the Java code
2. Recompile the project
3. Rebuild the application
4. Redeploy the application

For values that may change across environments or over time, this is not a clean approach.

Examples of such values:

```
payment provider name
retry count
timeout value
feature enabled/disabled flag
database URL
API key
server port
external service URL
```

These values are better kept outside the main business logic.

This is where configuration files come in.

---

2. What Is `application.properties`?

`application.properties` is a key-value configuration file that Spring Boot automatically reads by convention.

The default location is:

```
src/main/resources/application.properties
```

Coder Army

Example:

```
payment.provider=Razorpay
payment.retry-count=3
payment.enabled=true
payment.timeout=5000
```

Here, the values are no longer written directly inside the Java class.

They are now configuration values.

Spring Boot automatically loads this file when the application starts.

Spring Boot also supports YAML configuration using:

```
application.yml
```

Example:

```yaml
payment:
  provider: Razorpay
  retry-count: 3
  enabled: true
  timeout: 5000
```

Both `application.properties` and `application.yml` are used for configuration. The difference is mainly in the writing style.

---

3. Important Point: Is `application.properties` Really External?

When `application.properties` is placed inside:

```
src/main/resources
```

it becomes part of the application build.

That means it is packaged inside the final JAR file.

So technically, if we only change this internal file in the source code, we still need to rebuild the application.

But Spring Boot provides externalized configuration, which means these values can also be supplied or overridden from outside the packaged application.

For example, Spring Boot can read configuration from:

```
application.properties
application.yml
environment variables
command-line arguments
system properties
external config files
```

So the main idea is:

> Keep changeable values outside Java business logic so the code remains clean and flexible.

---

4. What Is Externalized Configuration?

Externalized configuration means keeping configuration values outside the Java code.

Instead of writing this:

```java
private String providerName = "Razorpay";
```

we write the value in a configuration file:

```
payment.provider=Razorpay
```

Then we inject that value into the Java class.

This makes the application easier to manage across different environments.

For example:

CoderArmy

Coder Army

Development Environment

```
payment.provider=TestProvider
payment.retry-count=1
```

Production Environment

```
payment.provider=Razorpay
payment.retry-count=3
```

The Java code can remain the same.

Only the configuration changes.

---

5. Using Configuration Values with `@Value`

Once values are present in `application.properties`, we need a way to use them inside Java classes.

One simple way is to use:

```
@Value
```

Example:

```
@Component
public class PaymentService {

    private final String providerName;
    private final int retryCount;

    public PaymentService(
            @Value("${payment.provider}") String providerNam
e,
            @Value("${payment.retry-count}") int retryCount)
{
```

Coder Army

```
        this.providerName = providerName;
        this.retryCount = retryCount;
    }

    public void pay() {
        System.out.println("Payment done using " + providerNa
me);
        System.out.println("Retry count: " + retryCount);
    }
}
```

Here:

```
@Value("${payment.provider}")
```

means:

> Go to the Spring Environment, find the property named `payment.provider`, and inject its value here.

Similarly:

```
@Value("${payment.retry-count}")
```

means:

> Find the property named `payment.retry-count` and inject it into this parameter.

---

6. Default Value with `@Value`

Sometimes a property may be missing from `application.properties`.

If the property is missing and no default value is provided, the application may fail during startup.

To avoid this, we can provide a default value.

Coder Army

Example:

```
@Value("${payment.provider:DefaultProvider}")
private String providerName;
```

Here:

```
${payment.provider:DefaultProvider}
```

means:

> Use `payment.provider` if it is available.
>
> If it is not available, use `DefaultProvider`.

Constructor example:

```
@Component
public class PaymentService {

    private final String providerName;

    public PaymentService(
            @Value("${payment.provider:DefaultProvider}") Str
ing providerName) {
        this.providerName = providerName;
    }
}
```

This is useful when a property is optional.

---

7. Problem with Too Many `@Value` Annotations

`@Value` is simple and useful for small cases.

But imagine we have many related properties:

Coder Army

```
payment.provider=Razorpay
payment.retry-count=3
payment.enabled=true
payment.timeout=5000
```

If we use `@Value`, we need to inject every value separately:

```java
@Value("${payment.provider}")
private String provider;

@Value("${payment.retry-count}")
private int retryCount;

@Value("${payment.enabled}")
private boolean enabled;

@Value("${payment.timeout}")
private int timeout;
```

This works, but it becomes messy when the number of properties increases.

For grouped configuration, Spring Boot provides a cleaner option:

```java
@ConfigurationProperties
```

---

8. What Is `@ConfigurationProperties`?

`@ConfigurationProperties` is used to bind a group of related configuration properties to a Java object.

Instead of injecting every property separately, we create one configuration class.

Example `application.properties`:

```
payment.provider=Razorpay
payment.retry-count=3
```

```
payment.enabled=true
payment.timeout=5000
```

Now create a Java class:

```java
package in.strikes.config;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

@Component
@ConfigurationProperties(prefix = "payment")
public class PaymentProperties {

    private String provider;
    private int retryCount;
    private boolean enabled;
    private int timeout;

    public String getProvider() {
        return provider;
    }

    public void setProvider(String provider) {
        this.provider = provider;
    }

    public int getRetryCount() {
        return retryCount;
    }

    public void setRetryCount(int retryCount) {
        this.retryCount = retryCount;
    }
```

Coder Army

Coder Army

```
    public boolean isEnabled() {
        return enabled;
    }

    public void setEnabled(boolean enabled) {
        this.enabled = enabled;
    }

    public int getTimeout() {
        return timeout;
    }

    public void setTimeout(int timeout) {
        this.timeout = timeout;
    }
}
```

The prefix is:

```
@ConfigurationProperties(prefix = "payment")
```

This means Spring Boot will look for all properties starting with:

```
payment
```

Then it will bind them to this class.

Mapping:

```
payment.provider      → provider
payment.retry-count   → retryCount
payment.enabled       → enabled
payment.timeout       → timeout
```

Now this configuration class can be injected wherever needed.

Example:

Coder Army

```
@Component
public class PaymentService {

    private final PaymentProperties paymentProperties;

    public PaymentService(PaymentProperties paymentPropertie
s) {
        this.paymentProperties = paymentProperties;
    }

    public void pay() {
        System.out.println("Payment done using " + paymentPro
perties.getProvider());
        System.out.println("Retry count: " + paymentPropertie
s.getRetryCount());
        System.out.println("Enabled: " + paymentProperties.is
Enabled());
        System.out.println("Timeout: " + paymentProperties.ge
tTimeout());
    }
}
```

This is cleaner because all payment-related configuration is stored in one object.

---

9. Why Does `payment.retry-count` Map to `retryCount`?

This happens because of relaxed binding.

Spring Boot is flexible with property names.

So this property:

```
payment.retry-count=3
```

can bind to this Java field:

Coder Army

```
private int retryCount;
```

Spring Boot understands different naming formats such as:

```
retry-count
retryCount
retry_count
RETRY_COUNT
```

For beginners, the simplest way to explain it is:

> Spring Boot can automatically convert common property naming styles into Java field naming style.

In Java, we usually write field names in camelCase:

```
retryCount
```

In properties files, we usually write names in kebab-case:

```
retry-count
```

Spring Boot can connect both of them.

---

10. `@Value` vs `@ConfigurationProperties`

`@Value`

Use `@Value` when you need one or two simple values.

Example:

```
@Value("${payment.provider}")
private String provider;
```

Good for:

Coder Army

```
single value
small demo
simple property injection
```

But it becomes repetitive when many related properties are needed.

`@ConfigurationProperties`

Use `@ConfigurationProperties` when many related values belong to the same group.

Example:

```
payment.provider=Razorpay
payment.retry-count=3
payment.enabled=true
payment.timeout=5000
```

Good for:

```
grouped configuration
cleaner code
large applications
better readability
```

Simple rule:

> For one or two values, `@Value` is fine.
>
> For a group of related values, prefer `@ConfigurationProperties`.

---

11. Why Do We Need Runner Interfaces?

In a web application, code usually runs when an HTTP request comes.

For example:

```
Browser/Postman sends request
↓
```

Coder Army

```
Controller receives request
↓
Service method executes
```

But in this lecture, we are not using:

```
spring-boot-starter-web
controller
endpoint
browser request
Postman request
```

So we need a way to run some code after the Spring Boot application starts.

Spring Boot gives us two common interfaces for this:

```
CommandLineRunner
ApplicationRunner
```

These interfaces allow us to run code after the Spring container is ready.

---

12. Why Not Call the Bean Manually from `main()`?

Technically, we can do this:

```
public static void main(String[] args) {
    ConfigurableApplicationContext context =
            SpringApplication.run(SpringBootCoreDemoApplicati
on.class, args);

    PaymentService paymentService = context.getBean(PaymentSe
rvice.class);
    paymentService.pay();
}
```

Coder Army

This works.

But this should not be the main approach in a Spring Boot application.

Why?

Because it brings us back to a manual style:

```
start context manually
fetch bean manually
call method manually
```

A better Spring Boot style is:

> Let Spring create a bean that runs automatically after startup.

That is where runner interfaces help.

---

13. `CommandLineRunner`

`CommandLineRunner` is a Spring Boot interface used to run code after the Spring application has started.

Example:

```java
package in.strikes.runner;

import in.strikes.service.PaymentService;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class AppRunner implements CommandLineRunner {

    private final PaymentService paymentService;

    public AppRunner(PaymentService paymentService) {
        this.paymentService = paymentService;
```

Coder Army

```
    }

    @Override
    public void run(String... args) {
        paymentService.pay();
    }
}
```

What happens here?

```
Spring creates AppRunner as a bean
↓
Spring injects PaymentService into AppRunner
↓
Spring Boot sees that AppRunner implements CommandLineRunner
↓
Spring Boot calls the run() method after startup
↓
paymentService.pay() executes
```

The method signature is:

```
public void run(String... args)
```

Here, `String... args` means the method can receive multiple command-line arguments as strings.

---

14. Passing Arguments to `CommandLineRunner`

Suppose we run the application from terminal like this:

```
java -jar app.jar hello world
```

Then inside `CommandLineRunner`:

Coder Army

```
@Override
public void run(String... args) {
    for (String arg : args) {
        System.out.println(arg);
    }
}
```

Output:

```
hello
world
```

`CommandLineRunner` receives arguments as a simple string array.

---

15. `ApplicationRunner`

`ApplicationRunner` is similar to `CommandLineRunner`.

The difference is that it provides arguments in a more structured form using:

```
ApplicationArguments
```

Example:

```
package in.strikes.runner;

import in.strikes.service.PaymentService;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

@Component
public class AppStartupRunner implements ApplicationRunner {

    private final PaymentService paymentService;
```

Coder Army

```
    public AppStartupRunner(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    @Override
    public void run(ApplicationArguments args) {
        paymentService.pay();
    }
}
```

Here, the method signature is:

```
public void run(ApplicationArguments args)
```

Instead of raw string arguments, Spring Boot gives us an `ApplicationArguments` object.

This object can understand option arguments more clearly.

For example, if we run:

```
java -jar app.jar --provider=Razorpay --retry=3
```

`ApplicationRunner` can read these as named options.

Example:

```
@Override
public void run(ApplicationArguments args) {
    System.out.println(args.getOptionValues("provider"));
    System.out.println(args.getOptionValues("retry"));
}
```

So the simple difference is:

```
CommandLineRunner  → gives raw String arguments
```

```
ApplicationRunner  → gives structured ApplicationArguments
```

---

16. Complete Startup Flow

A simplified Spring Boot startup flow looks like this:

```
main() method starts
↓
SpringApplication.run() executes
↓
Spring Boot prepares the Environment
↓
Configuration values are loaded
(application.properties, application.yml, environment variables, command-line arguments, etc.)
↓
ApplicationContext is created
↓
@SpringBootApplication is processed
↓
@ComponentScan scans the package
↓
@Component / @Service / @Repository / @Controller classes are discovered
↓
@EnableAutoConfiguration checks dependencies on the classpath
↓
Matching auto-configurations are applied
↓
Beans are created
↓
Dependencies are injected
↓
Configuration values are injected or bound
```

Coder Army

Coder Army

```
(@Value / @ConfigurationProperties)
↓
CommandLineRunner / ApplicationRunner executes
↓
Application either exits or keeps running depending on the ty
pe of application
```

In simple words:

> Spring Boot starts the container, loads configuration, scans classes, creates beans, injects dependencies, applies auto-configuration, and finally runs startup logic.

---

17. Final Summary

`application.properties`

`application.properties` is used to keep configuration values in key-value format.

Example:

```
payment.provider=Razorpay
payment.retry-count=3
```

Externalized Configuration

Externalized configuration means keeping changeable values outside Java business logic.

This helps us avoid hardcoding values inside classes.

`@Value`

`@Value` is used to inject a single configuration value.

Example:

Coder Army

```
@Value("${payment.provider}")
private String provider;
```

It is simple and useful for small cases.

`@ConfigurationProperties`

`@ConfigurationProperties` is used to bind a group of related properties to a Java object.

Example:

```
@ConfigurationProperties(prefix = "payment")
```

It is cleaner when many related configuration values are present.

`CommandLineRunner`

`CommandLineRunner` runs code after the Spring Boot application has started.

It receives command-line arguments as raw strings.

```
run(String... args)
```

`ApplicationRunner`

`ApplicationRunner` also runs code after startup, but it receives arguments in a structured form.

```
run(ApplicationArguments args)
```

Spring Boot does not force us to hardcode everything inside Java classes.

We can keep changeable values in configuration files, inject them into beans, and run startup logic using runner interfaces when there is no web request involved.

Coder Army

Spring Boot CRUD Operations

What Exactly Is a CRUD Application?

Let’s understand CRUD from first principles

Almost every application does four basic things with data:

```
Create data
Read data
Update data
Delete data
```

CRUD is not just beginner content

Many students think CRUD is basic.

CRUD is the foundation of most backend systems.

Example:

```
YouTube video system
Create video
Read video
Update title/description
Delete video
```

```
Instagram post system
Create post
Read post
Update caption
Delete post
```

CRUD is the basic skeleton of backend development.

Student Management API

Coder Army

CRUD Mapping with HTTP Methods

Now connect CRUD with REST API style.

CRUD Operation	HTTP Method	Example URL	Meaning
Create	POST	`/api/students/`	Add new student
Read All	GET	`/api/students/`	Get all students
Read One	GET	`/api/students/{id}`	Get one student
Update	PUT	`/api/students/{id}`	Replace/update student details
Hard Delete	DELETE	`/api/students/{id}`	Remove permanently

What features will we build?

We will build six main APIs.

1. Create Student

Admin can add a new student.

```
POST /api/students/
```

Example request body:

```
{
  "name":"Rahul Sharma",
  "email":"rahul@example.com",
  "course":"Spring Boot",
  "age":22,
  "rollNo" : 101
}
```

This will insert a row into the database.

Coder Army

2. Get All Students

Admin can see all active students.

```
GET /api/students/
```

---

3. Get Student by ID

Admin can fetch one student.

```
GET /api/students/1
```

Here, `1` is the student ID.

This will help you explain:

```
@PathVariable
```

Meaning:

> Value comes from URL

---

4. Update Student

Admin can update student details.

```
PUT /api/students/1
```

Example:

```
{
  "name":"Rahul Sharma",
  "email":"rahul.updated@example.com",
  "course":"Spring Data JPA",
```

Coder Army

```
  "age": 23,
  "rollNo" : 101
}
```

Logic:

```
First find student by ID
Then update fields
Then save updated student
```

This is a very important backend pattern.

---

5. Delete Student

Admin can permanently delete a student.

```
DELETE /api/students/1
```

This will actually remove the row from the database.

So we will show both delete styles:

```
Soft delete → row remains, isDeleted becomes true
Hard delete → row is permanently removed
```

This makes the project more real-world.

What will our database table look like?

The table name can be:

```
students
```

Columns:

Coder Army

Column	Type	Meaning
id	BIGINT	Unique student ID
name	VARCHAR	Student name
email	VARCHAR	Student email
course	VARCHAR	Student course
age	INT	Student age
roll_no	INT	Rolll Number

In Java, this table will be represented by a class:

```
public class Student {
    private String name;
    private String email;
    private int rollNo;
    private int age;
    private String subject;
    }
```

This is where students will see:

> Java object database table se connect ho sakta hai.

That is the foundation of JPA.

Why do we need architecture?

Imagine we write the full CRUD code inside one class:

```
@RestController
public class StudentController {

    // receive request
    // validate data
    // write business logic
```

Coder Army

```
    // talk to database
    // update student
    // delete student
}
```

Technically, for a very small demo, this may work.

But this is not a good backend design.

Why?

Because one class is doing everything:

```
Receiving HTTP request
Processing business logic
Talking to database
Deciding response
Managing update/delete rules
```

This becomes messy very quickly.

So in real applications, we divide responsibility.

That is the main reason we create layers.

The final architecture

Our application will follow this flow:

```
Postman
   ↓
Controller
   ↓
Service
   ↓
Repository
   ↓
Database
```

This is the core structure of the project.

Coder Army

Step 1: Postman sends request

Postman is acting like a client.

In real life, this request may come from:

```
React frontend
Angular frontend
Mobile app
Another backend service
Postman
```

Postman sends data to our Spring Boot application.

Step 2: Controller receives request

The first Java layer that handles this request is:

```
Controller
```

Controller's job is not to write all logic.

Controller's job is mainly:

```
Receive request
Read input data
Call service layer
Return response
```

So the controller is like the entry gate of the application.

Step 3: Service handles business logic

Controller will call the service:

```
studentService.createStudent(student);
```

The service layer contains the actual business rules.

For example:

Coder Army

```
Should this student be saved?
Should deleted students be hidden?
How should update happen?
What should happen if student ID does not exist?
Soft delete or hard delete?
```

In our first CRUD app, business logic will be simple.

Step 4: Repository talks to database

The service layer will call repository:

```
studentRepository.save(student);
```

Repository is responsible for database operations.

Important thing:

We will not write SQL manually for basic operations.

Spring Data JPA gives us ready-made methods like:

```
save()
findAll()
findById()
deleteById()
```

So repository is the layer that connects Java code with database operations.

Step 5: Database stores the data

Finally, the data reaches MySQL.

A new row is inserted into the `students` table.

Then response travels back:

Coder Army

```
Database
   ↓
Repository
   ↓
Service
   ↓
Controller
   ↓
Postman
```

Layer-by-layer responsibility

This table should definitely be shown in your video:

Layer	Main Responsibility	Example
Postman	Sends HTTP request	`POST /api/students`
Controller	Handles API request	`StudentController`
Service	Contains business logic	`StudentService`
Repository	Performs database operations	`StudentRepository`
Database	Stores actual data	MySQL `students` table
Entity/Model	Represents table as Java object	`Student` class

Where does Entity/Model fit?

In the diagram, we usually write:

```
Controller → Service → Repository → Database
```

But there is one more important part:

```
Entity / Model
```

The entity is not exactly a “flow layer” like controller or service.

Coder Army

Instead, it is the object that travels between layers.

Example:

```
Student student
```

This `Student` object contains data like:

```
name
email
course
age
```

And JPA maps this object to the database table.

```
Controller receives Student data.
Service works with Student object.
Repository saves Student object.
JPA converts Student object into students table row
```

Till now, we learned DI using console examples. Today, you will see the same DI in a real Spring Boot backend application.

Setting up the Project :

Dependencies We Need

For this CRUD project, add these dependencies:

```
Spring Web
Spring Data JPA
MySQL Driver
```

Coder Army

Why Spring Web?

Spring Web is needed because we want to build APIs

Without Spring Web, our application will be like a normal backend app, but it will not expose HTTP endpoints like:

```
GET /api/students
POST /api/students
PUT /api/students/1
DELETE /api/students/1
```

Spring Web gives us annotations like:

```
@RestController
@RequestMapping
@GetMapping
@PostMapping
@PutMapping
@DeleteMapping
@PatchMapping
```

Why Spring Data JPA?

Spring Data JPA is needed because we want to perform database operations without writing basic SQL manually.

So Spring Data JPA reduces boilerplate database code.

Why MySQL Driver?

Our Spring Boot app is Java code.

MySQL is a separate database.

So Java needs a driver to communicate with MySQL.

That is why we add:

Coder Army

```
MySQL Driver
```

RUN THE EMPTY APP WITH HELLO WORLD :

```
The app failed because you added Spring Data JPA + MySQL Driver,
so Spring Boot assumed:

“This project wants to connect to a database.”

Then during startup, Spring Boot tried to auto-create a DataSource for MySQL.
But your project does not yet have database details like URL, username,
and password, so it failed with:

Failed to configure a DataSource: 'url' attribute is not specified

Reason: Failed to determine a suitable driver class
```

Spring Boot does this automatically:

```
Spring Web present       → start embedded Tomcat on port 8080
Spring Data JPA present  → configure JPA, Hibernate, EntityManager
MySQL Driver present     → configure MySQL DataSource
```

But it cannot connect because you have not provided:

```
spring.datasource.url
spring.datasource.username
spring.datasource.password
```

So the app crashes before fully starting.

Coder Army

Spring Boot auto-configuration works based on the JAR dependencies present in your project. For example, if a database library is present and you have not manually configured a database bean, Spring Boot tries to configure one automatically

So in your project:

```
spring-boot-starter-web
```

tells Spring Boot:

```
This looks like a web application.

I should configure Tomcat, DispatcherServlet, web MVC setup,
JSON support, etc.
```

And:

```
spring-boot-starter-data-jpa
mysql-connector-j
```

tells Spring Boot:

```
This looks like a database/JPA application.
I should configure DataSource, Hibernate, EntityManagerFactory, TransactionManager, Repos
itory support, etc.
```

It showed failure while creating `entityManagerFactory` because the `DataSource` could not be created without DB URL details.

Why Do We Need Folder Structure?

Imagine we write everything inside the main package directly:

Coder Army

```
com.coderarmy.studentcrud
│
├── StudentcrudApplication
├── Student
├── StudentController
├── StudentService
├── StudentRepository
```

This will still work.

But as soon as the project grows, it becomes messy.

Suppose later we add:

```
Teacher
Course
Batch
Payment
Attendance
Enrollment
```

Then the package becomes crowded.

So instead of keeping all classes together, we divide them based on responsibility.

That is why we create packages like:

```
controller
service
repository
entity
```

Each package has a clear purpose.

Main Application Class

This class is the starting point of the Spring Boot application.

It should stay in the root package:

```
com.coderarmy.studentcrud
```

Coder Army

Why?

Because `@SpringBootApplication` includes component scanning.

By default, Spring scans the package where this class is present and its sub-packages.

Entity Package

Package:

```
com.coderarmy.studentcrud.entity
```

Class:

```java
Student.java

@Entity
public class Student {
    private String name;
    private String email;
    private int rollNo;
    private int age;
    private String subject;
    private boolean isDeleted;
}
```

The entity package contains classes that represent database tables.

Why Are We Using MySQL?

For this project, we will use MySQL because:

```
It is popular.
It is beginner-friendly.
```

```
It is commonly used with Spring Boot.
It is easy to inspect data using MySQL Workbench or terminal.
```

Coder Army

What Exactly Are We Creating?

We are creating a database named:

```
student_crud_db
```

Inside this database, later JPA will create a table named:

```
students
```

So the structure will become:

```
MySQL Server
   ↓
student_crud_db
   ↓
students table
   ↓
student rows
```

Important point:

At this stage, we will only create the database

We will not manually create the table.

The table can be created automatically by JPA when we run the application with proper configuration.

Setting up MySQL :

- MySQL Server = actual database engine
- MySQL GUI Application = GUI application to manage that database

Coder Army

GUI application for MySQL:

- Work Bench (Official MySQL GUI)
- Sequel Ace (For mac)
- DBeaver (Light cross platform)

Install both on your mac/PC by following instructions.
If i teach the install instructions here, this will be unnecesssary.
I will include this in notes or follow chatGPT / google instructions.

Installing MySQL Server, MySQL Client, and MySQL Workbench

1. First understand the difference

Before using MySQL with a Spring Boot CRUD application, we need to understand three things:

```
MySQL Server  = actual database engine
MySQL Client  = terminal command used to connect to MySQL Ser
ver
MySQL Workbench = GUI application used to manage MySQL visual
ly
```

Part A: Install MySQL on macOS

Install MySQL using Homebrew (Install homebrew if not already)

Open Terminal and run:

```
brew install mysql
```

Coder Army

This installs:

```
MySQL Server
MySQL command-line client
```

Now start MySQL Server:

```
brew services start mysql
```

Check whether MySQL is running:

```
brew services list
```

You should see MySQL with status:

```
started
```

You can also check the installed MySQL version:

```
mysql --version
```

Set root password on macOS

Run:

```
mysql_secure_installation
```

It may ask for the current root password.

If you have not set any password yet, just press:

```
Enter
```

Coder Army

Then follow the prompts.

Recommended beginner-friendly answers:

```
Set root password? → Yes
New password → your password
Remove anonymous users? → Yes
Disallow root login remotely? → Yes
Remove test database? → Yes
Reload privilege tables? → Yes
```

Now test login:

```
mysql -u root -p
```

Enter your password.

If login is successful, you will see:

```
mysql>
```

That means your MySQL Server is working and your terminal client is connected.

Useful macOS MySQL commands

```
Start MySQL Server: brew services start mysql
Stop MySQL Server: brew services stop mysql
Restart MySQL Server: brew services restart mysql
Check service status: brew services list
Login : mysql -u root -p
Exit : exit;
```

Part B: Install MySQL on Windows

Coder Army

Step 1: Download MySQL Installer

Download **MySQL Installer for Windows** from the official MySQL website.

Choose:

```
MySQL Installer
```

During installation, select:

```
Developer Default
```

This usually installs:

```
MySQL Server
MySQL Workbench
MySQL Shell
MySQL command-line client
```

Step 2: Configure MySQL Server on Windows

During setup, keep these settings:

```
Config Type: Development Computer
Port: 3306
Authentication Method: Strong Password Encryption
Username: root
Password: your password
Run MySQL as Windows Service: Yes
Start MySQL Server at System Startup: Yes
```

The installer requires assigning a root password during server configuration.

Step 3: Start or stop MySQL Server on Windows

Open **Command Prompt** or **PowerShell** as Administrator.

Check MySQL service name:

Coder Army

```
Get-Service *mysql*
```

You may see a service name like:

```
MySQL80
```

Start MySQL Server:

```
net start MySQL80
```

Stop MySQL Server:

```
net stop MySQL80
```

If your service name is different, replace `MySQL80` with the name shown on your system.

Example:

```
net start MySQL
```

or:

```
net start MySQL84
```

MySQL's Windows documentation explains that MySQL Server can run as a Windows Service and can be started from the Services utility or command line.

Step 4: Open MySQL from Windows terminal

Open Command Prompt or PowerShell.

Run:

Coder Army

```
mysql -u root -p
```

Enter the root password you created during installation.

If login is successful, you will see:

```
mysql>
```

The default MySQL port is `3306`, and Workbench's Standard TCP/IP connection uses host, port, username, and password to connect to MySQL Server.

```
mysql --version
```

Coder Army

Spring Boot CRUD Project — Database, Entity, Repository, Service, and Controller

Why Are We Not Creating the Table Manually?

Normally, when working directly with SQL, we manually create tables using queries such as:

```
CREATE TABLE students (...);
```

But in this project, we are using **JPA with Hibernate**.

JPA allows us to work with Java classes and objects, while Hibernate can map those classes to database tables.

So instead of manually creating the `students` table, Hibernate can look at our entity class and create or update the table structure based on our configuration.

```
spring.jpa.hibernate.ddl-auto=update
```

This means Hibernate will try to update the database schema based on the entity class.

For example:

```
@Entity
@Table(name = "students")
public class Student {
    // fields
}
```

Hibernate can use this class to create or update a `students` table.

Coder Army

Important Point

Hibernate can create or update tables, but the database itself should already exist.

So we still need to create the database once:

```
CREATE DATABASE student_crud_db;
```

After that, Hibernate can create/update tables inside this database.

Basic MySQL Configuration

Inside `application.properties`, add the following configuration:

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_crud_db
spring.datasource.username=root
spring.datasource.password=your_password

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Understanding Each Property

1. `spring.datasource.url`

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_crud_db
```

This tells Spring Boot where the database is located.

URL Breakdown

Coder Army

Part	Meaning
`jdbc`	Java Database Connectivity URL format
`mysql`	Database type is MySQL
`localhost`	MySQL is running on the same system
`3306`	Default MySQL port
`student_crud_db`	Database name

So this URL means:

> Connect this Spring Boot application to the MySQL database named `student_crud_db` running on the local machine at port `3306`.

---

2. `spring.datasource.username`

```
spring.datasource.username=root
```

This is the MySQL username.

In local development, we commonly use the `root` user.

---

3. `spring.datasource.password`

```
spring.datasource.password=your_password
```

This is the password of the MySQL user.

For example, if the MySQL password is `123456`, then write:

```
spring.datasource.password=123456
```

---

4. `spring.jpa.hibernate.ddl-auto`

Coder Army

```
spring.jpa.hibernate.ddl-auto=update
```

This property controls how Hibernate handles database table structure.

With `update`, Hibernate checks the entity class and tries to update the table structure accordingly.

For example:

- If the table does not exist, Hibernate can create it.
- If a new field is added in the entity class, Hibernate can try to add a new column.
- Existing data is usually not deleted with `update`.

Other `ddl-auto` Values

Value	Meaning
`none`	Hibernate will not manage the table structure
`validate`	Hibernate only checks whether the table matches the entity
`update`	Hibernate updates the table structure based on the entity
`create`	Drops old tables and creates fresh tables every time the app starts
`create-drop`	Creates tables when the app starts and drops them when the app stops

---

Production Note

In real production projects, database changes are usually not handled using:

```
spring.jpa.hibernate.ddl-auto=update
```

Instead, teams use controlled migration tools such as:

```
Flyway
Liquibase
```

Coder Army

```
SQL migration scripts
```

This gives better control over database changes.

For learning and local development, `ddl-auto=update` is useful and convenient.

---

5. `spring.jpa.show-sql`

```
spring.jpa.show-sql=true
```

This tells Hibernate to print SQL queries in the console.

For example, when we save a student, Hibernate may print an `insert` query.

This helps us understand what SQL Hibernate is generating internally.

---

6. `spring.jpa.properties.hibernate.format_sql`

```
spring.jpa.properties.hibernate.format_sql=true
```

This formats SQL queries properly in the console.

Without formatting, the SQL query may appear in one long line.

With formatting, it becomes more readable.

---

7. Driver Class Configuration

In most Spring Boot projects, we do not need to manually write the driver class.

Spring Boot can detect the driver from:

```
1. MySQL dependency
2. JDBC URL
```

But if someone faces driver-related issues, they can add:

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Coder Army

Java Objects and Database Tables

In Java, we work with classes and objects.

In SQL databases, we work with tables and rows.

This creates a natural mismatch:

```
Java world      → classes and objects
Database world  → tables and rows
```

Now the question is:

> How will a Java object become a database row?

This is where JPA helps.

JPA provides **Object-Relational Mapping**, commonly called ORM.

ORM means mapping Java classes to relational database tables.

Example:

```
Student class   → students table
Student object  → one row inside students table
Student fields  → columns of the table
```

So when we create:

```java
Student student = new Student();
```

we are creating a Java object.

When JPA saves this object, it can become a row inside the `students` table.

Coder Army

Entity Class

An entity is a Java class that represents a database table.

Example:

```
@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private Integer age;
    private String mobile;
    private Boolean isDeleted = false;
}
```

Important Entity Annotations

1. `@Entity`

```
@Entity
```

`@Entity` tells JPA that this class should be treated as a database entity.

Without `@Entity`, JPA treats the class as a normal Java class.

With `@Entity`, JPA understands:

```
This class can be mapped to a database table.
Objects of this class can be saved as database rows.
```

Coder Army

2. `@Table`

```
@Table(name = "students")
```

`@Table` tells JPA the name of the database table.

If we do not use `@Table`, JPA usually uses the class name as the table name.

Example:

```
public class Student
```

may be mapped to a table named:

```
student
```

By using `@Table`, we can control the table name explicitly.

3. `@Id`

```
@Id
private Long id;
```

Every table should have a primary key.

A primary key uniquely identifies each row.

In our `Student` entity, `id` is the primary key.

4. `@GeneratedValue`

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

We do not want to manually provide the ID every time we create a student.

Coder Army

`@GeneratedValue` tells JPA that the ID should be generated automatically.

`GenerationType.IDENTITY` means the database will handle ID generation.

In MySQL, this usually means an auto-increment column.

What Is Repository?

Our Spring Boot application works with Java objects.

Example:

```
Student student = new Student();
student.setName("Rahul Sharma");
student.setEmail("rahul@example.com");
student.setAge(22);
```

But the actual data should be stored in MySQL.

So we need a layer that can perform database operations such as:

```
Save student into database
Fetch students from database
Find student by ID
Update student
Delete student
```

This responsibility belongs to the **Repository layer**.

Creating Repository

```
public interface StudentRepository extends JpaRepository<Stud
ent, Long> {
}
```

Here, we are creating an interface, not a class.

Coder Army

Normally, when we create an interface, we need to manually create its implementation class.

But with Spring Data JPA, we do not manually create the implementation.

Spring Data JPA creates the implementation automatically at runtime.

Understanding `JpaRepository<Student, Long>`

`JpaRepository` needs two things:

```
1. Entity type
2. Primary key type
```

In our case:

```
Student → Entity class
Long    → Type of primary key/id
```

So this means:

```
JpaRepository<Student, Long>
```

will provide database operations for the `Student` entity, whose ID type is `Long`.

Do We Need `@Repository`?

For Spring Data JPA repositories, we usually do not need to write:

```
@Repository
```

Spring Data JPA automatically detects interfaces that extend `JpaRepository`.

So this is enough:

Coder Army

```
public interface StudentRepository extends JpaRepository<Stud
ent, Long> {
}
```

Important Repository Methods

`JpaRepository` gives us many ready-made methods.

For this CRUD project, these methods are important:

```
save()
findAll()
findById()
deleteById()
existsById()
```

1. `save()`

```
studentRepository.save(student);
```

`save()` is used to save data in the database.

It can do two things:

```
If the student is new       → insert new row
If the student already exists → update existing row
```

Example:

```
public Student createStudent(Student student) {
    return studentRepository.save(student);
}
```

If the student does not have an ID, JPA treats it as a new record and inserts it.

Coder Army

2. `findAll()`

```
studentRepository.findAll();
```

This fetches all rows from the `students` table.

But in our project, we are using soft delete.

So if we directly use:

```
studentRepository.findAll();
```

it may return both:

```
Active students
Soft-deleted students
```

Usually, when the user calls:

```
GET /api/students
```

we want only active students.

So instead of `findAll()`, we can create a custom method:

```
List<Student> findByIsDeletedFalse();
```

Then we can fetch only those students whose `isDeleted` value is `false`.

Example:

```
public interface StudentRepository extends JpaRepository<Stud
ent, Long> {
    List<Student> findByIsDeletedFalse();
}
```

Coder Army

3. `findById()`

```
studentRepository.findById(id);
```

This fetches one student by ID.

Return type:

```
Optional<Student>
```

Example:

```
Optional<Student> student = studentRepository.findById(1L);
```

Why does it return `Optional`?

Because the student may or may not exist.

If the student exists, we get the student object.

If the student does not exist, the `Optional` is empty.

So `findById()` forces us to handle this case properly:

```
What if the data is not found?
```

This helps us write safer code.

---

4. `deleteById()`

```
studentRepository.deleteById(id);
```

This performs hard delete.

It removes the row permanently from the database.

Example:

Coder Army

```
studentRepository.deleteById(1L);
```

After this, student ID `1` is removed from the table.

This can be used for an API like:

```
DELETE /api/students/{id}
```

---

5. `existsById()`

```
studentRepository.existsById(id);
```

This checks whether a student exists with the given ID.

It returns a boolean value:

```
boolean exists = studentRepository.existsById(1L);
```

This is useful before hard delete.

Example:

```
public boolean deleteStudent(Long id) {
    if (!studentRepository.existsById(id)) {
        return false;
    }

    studentRepository.deleteById(id);
    return true;
}
```

This avoids trying to delete a record that does not exist.

Service Layer

Coder Army

The service layer contains the business logic of the application.

The controller should not directly contain all business logic.

The controller should mainly handle HTTP requests and responses.

The service layer should handle application logic such as:

```
Create student
Update student
Fetch active students
Soft delete student
Hard delete student
Check whether student exists
```

`@Service`

```java
@Service
public class StudentService {
}
```

`@Service` tells Spring that this class belongs to the service layer and Spring should create its object as a bean.

`@Service` is similar to `@Component`, but it is more specific.

```
@Component → generic Spring-managed class
@Service   → Spring-managed class that contains business logic
```

Using `@Service` makes the code more readable and clearly shows the responsibility of the class.

What Is Controller?

The repository talks to the database.

The service contains business logic.

Coder Army

But an external client, like Postman, browser, frontend application, or mobile app, cannot directly call the repository or service.

So we need a controller.

The controller is the layer that receives HTTP requests and sends HTTP responses.

Example:

```
GET /api/students
POST /api/students
PUT /api/students/{id}
DELETE /api/students/{id}
```

These requests are handled by the controller.

@RestController

```java
@RestController
public class StudentController {
}
```

`@RestController` tells Spring that this class will handle REST API requests.

It is used when we want to return data directly as the response, usually in JSON format.

For example, if our method returns a `Student` object:

```java
return student;
```

Spring Boot can automatically convert it into JSON:

```
{
  "id": 1,
  "name": "Rahul Sharma",
  "email": "rahul@example.com",
```

Coder Army

```
  "age": 22,
  "mobile": "9876543210",
  "isDeleted": false
}
```

This conversion is usually done using Jackson, which comes automatically with Spring Boot web dependency.

`@RequestMapping`

```
@RequestMapping("/api/students")
```

`@RequestMapping` defines the base URL for all APIs inside the controller.

Example:

```
@RestController
@RequestMapping("/api/students")
public class StudentController {
}
```

Now all APIs inside this controller will start with:

```
/api/students
```

So instead of writing `/api/students` again and again on every method, we define it once at the class level.

Why Use `ResponseEntity` ?

In simple tutorials, we may see code like this:

```
public Student getStudentById(@PathVariable Long id) {
    return studentService.getStudentById(id).get();
```

Coder Army

```
}
```

This works only when the student exists.

But what if the student does not exist?

Then our API should return a proper HTTP status code.

Example:

```
Student found      → 200 OK
Student not found  → 404 Not Found
```

`ResponseEntity` helps us control:

```
Response body
HTTP status code
Headers, if needed
```

Example: Returning 200 OK

```
return ResponseEntity.ok(student);
```

Meaning:

```
Status: 200 OK
Body: student data
```

Example: Returning 404 Not Found

```
return ResponseEntity.notFound().build();
```

Meaning:

Coder Army

```
Status: 404 Not Found
Body: empty
```

Example: Returning 201 Created

When a new student is created, we can return `201 Created`.

```
return ResponseEntity.status(HttpStatus.CREATED).body(created
Student);
```

Meaning:

```
Status: 201 Created
Body: created student data
```

This is more meaningful than always returning `200 OK`.

Complete Flow of Request

When a client sends a request:

```
POST /api/students
```

the flow is:

```
Client/Postman
      ↓
Controller
      ↓
Service
      ↓
Repository
      ↓
Database
```

Coder Army

After saving the data, the response goes back:

```
Database
      ↓
Repository
      ↓
Service
      ↓
Controller
      ↓
Client/Postman
```

Layer Responsibilities

Layer	Responsibility
Controller	Handles HTTP requests and responses
Service	Contains business logic
Repository	Talks to the database
Entity	Represents database table structure
Database	Stores actual data

Summary

In this CRUD project:

- MySQL stores the actual student data.
- `application.properties` connects Spring Boot with MySQL.
- JPA maps Java classes to database tables.
- Hibernate can create/update tables using `ddl-auto=update`.
- `@Entity` marks a class as a database entity.
- `@Table` defines the table name.
- `@Id` marks the primary key.

Coder Army

- `@GeneratedValue` allows automatic ID generation.
- Repository layer performs database operations.
- Service layer contains business logic.
- Controller layer handles API requests and responses.
- `ResponseEntity` helps return proper HTTP status codes.

The main idea is:

```
Java object gets converted into database row using JPA.
```

And the complete Spring Boot CRUD flow becomes:

```
Controller → Service → Repository → Database
```

Coder Army

Soft Delete in Spring Boot CRUD API

Why Use PATCH for Soft Delete?

In hard delete, we permanently remove the row from the database.

Example:

```
DELETE /api/students/1
```

But in soft delete, we do not actually delete the row.

Instead, we only update one field:

```
isDeleted = true
```

So the student record still remains in the database, but it is marked as deleted.

Because we are only changing one specific field, soft delete is technically a **partial update**.

That is why using `PATCH` makes sense for soft delete.

```
PATCH /api/students/1/soft-delete
```

PATCH vs DELETE

Operation	What happens?	HTTP Method
Hard Delete	Row is permanently removed from the database	`DELETE`
Soft Delete	Row remains in the database, but `isDeleted` becomes `true`	`PATCH`

So for hard delete:

Coder Army

```
DELETE /api/students/1
```

For soft delete:

```
PATCH /api/students/1/soft-delete
```

Creating a Custom Finder Method for Soft Delete

When we use soft delete, our table contains a column like this:

```
is_deleted
```

In the Java entity class, we can represent it like this:

```
@Column(name = "is_deleted")
private Boolean isDeleted = false;
```

Here, the default value is `false`, which means the student is active and not deleted.

When a student is soft deleted, we update this value to:

```
isDeleted = true;
```

Problem with Normal Fetch API

Normally, when we fetch all students, we may use:

```
studentRepository.findAll();
```

But there is a problem.

Coder Army

`findAll()` fetches every row from the database, including soft-deleted records.

For example:

id	name	is_deleted
1	Rahul	false
2	Priya	true
3	Aman	false

If we use `findAll()`, it may return all three students.

But in normal application behavior, soft-deleted students should not be shown.

So we only want records where:

```
isDeleted = false
```

In SQL, the query would look like this:

```sql
SELECT * FROM students
WHERE is_deleted = false;
```

Using Spring Data JPA Derived Query Method

Spring Data JPA allows us to create queries by simply writing method names.

Instead of writing SQL manually, we can create this method inside the repository:

```
List<Student> findByIsDeletedFalse();
```

Spring Data JPA reads the method name and automatically generates the required query.

This method means:

```
Find all students where isDeleted is false.
```

Coder Army

Equivalent SQL idea:

```
SELECT * FROM students
WHERE is_deleted = false;
```

This type of method is called a **derived query method**.

How `findByIsDeletedFalse()` Works

Method:

```
List<Student> findByIsDeletedFalse();
```

Breakdown:

Part	Meaning
`findBy`	Query starts here
`IsDeleted`	Entity field name
`False`	Condition should be false

So Spring understands the method as:

```
Find students where isDeleted is false.
```

This works because Spring Data JPA follows naming conventions. It reads the method name, identifies the field, understands the condition, and creates the query automatically.

Why Add `findByIdAndIsDeletedFalse()`?

For getting a student by ID, we may normally write:

```
studentRepository.findById(id);
```

But this can create an issue when soft delete is used.

Coder Army

Suppose a student exists in the database but is already soft deleted:

id	name	is_deleted
1	Rahul	true

If we use:

```
findById(1L)
```

it will still return Rahul because the row still exists in the database.

But from the application's point of view, a soft-deleted student should behave like an inactive or hidden record.

So for normal get-by-id APIs, we should fetch only students that are not deleted.

For that, we can create this repository method:

```
Optional<Student> findByIdAndIsDeletedFalse(Long id);
```

This means:

```
Find student where id = given id AND isDeleted = false.
```

Equivalent SQL idea:

```
SELECT * FROM students
WHERE id = 1
AND is_deleted = false;
```

Breakdown of `findByIdAndIsDeletedFalse()`

Method:

```
Optional<Student> findByIdAndIsDeletedFalse(Long id);
```

Coder Army

Breakdown:

Part	Meaning
`findBy`	Query starts here
`Id`	Match the student ID
`And`	Add another condition
`IsDeleted`	Check the `isDeleted` field
`False`	Value should be false

So Spring Data JPA understands:

```
Find a student where id matches and isDeleted is false.
```

Final Repository Methods

Inside the repository interface, we can add:

```
public interface StudentRepository extends JpaRepository<Stud
ent, Long> {

    List<Student> findByIsDeletedFalse();

    Optional<Student> findByIdAndIsDeletedFalse(Long id);
}
```

Now we can use:

```
findByIsDeletedFalse()
```

to fetch all active students.

And:

```
findByIdAndIsDeletedFalse(id)
```

to fetch one active student by ID.

Coder Army

Final Flow

For Soft Delete

Instead of deleting the row permanently:

```
studentRepository.deleteById(id);
```

we first fetch the student, then update the field:

```
student.setIsDeleted(true);
studentRepository.save(student);
```

So the row remains in the database, but it becomes hidden from normal APIs.

Key Takeaways

- Soft delete means marking a record as deleted instead of removing it permanently.
- In soft delete, we usually update a field like `isDeleted`.
- Since only one field is updated, `PATCH` is a suitable HTTP method.
- `DELETE` is suitable when the row is actually removed from the database.
- `findAll()` may return soft-deleted records also.
- To fetch only active records, we can use custom finder methods.
- Spring Data JPA can generate queries automatically from method names.
- Methods like `findByIsDeletedFalse()` and `findByIdAndIsDeletedFalse()` are called derived query methods.
- Soft-deleted records should usually be hidden from normal get APIs.

Coder Army

Servlets: Complete Masterclass

Note : For command on running tomcat server on windows/mac refer to the last.

1. Why Do We Need Servlets?

Before understanding Servlets, first understand what happens when a browser sends a request.

When we open:

```
http://localhost:8080/hello
```

the browser is not directly saying:

```
Please call HelloServlet.java
```

A browser does not understand Java classes, Java methods, or Java objects.

It only understands **HTTP**.

So the browser sends an HTTP request like this:

```
GET /hello HTTP/1.1
Host: localhost:8080
```

This is just text written according to HTTP rules.

The important question is:

> Who receives this HTTP request?

A Java class does not automatically receive browser requests.

There must be a server process running on a port that can receive the request.

---

2. Java Can Listen on a Port

Coder Army

Java can do networking through the `java.net` package.

Using classes like `ServerSocket`, Java can listen on a port:

```
ServerSocket serverSocket = new ServerSocket(8080);
```

This means Java can open port `8080` and wait for incoming connections.

So technically, we can write a Java program that receives browser requests.

But the browser sends HTTP text, and if we use raw Java networking, we must handle everything manually.

For example, we would need to manually:

```
1. Read raw HTTP text
2. Parse the URL
3. Parse headers
4. Parse query parameters
5. Parse request body
6. Identify GET, POST, PUT, DELETE
7. Create response text
8. Add status code
9. Add response headers
10. Send response back to browser
11. Manage multiple users at the same time
12. Manage threads
13. Keep the server running continuously
```

Java can do networking, but building a full web application directly with low-level sockets is difficult, repetitive, and error-prone.

This is the first major problem Servlets help solve.

---

3. Normal Java Program vs Web Application

In a normal Java program, the entry point is:

```
public static void main(String[] args)
```

The program starts, executes some code, prints output, and usually finishes.

Coder Army

A web application is different.

A web application should not start once and stop immediately.

It should keep running continuously and wait for HTTP requests.

For example:

```
User 1 sends request
User 2 sends request
User 3 sends request
```

The application should keep responding to all these requests.

So a web application needs a **long-running server environment**.

A normal Java class alone is not enough for that.

---

4. Static Website vs Dynamic WebsiteStatic Website

Suppose we have a simple HTML file:

```
<h1>Welcome to my website</h1>
```

If the browser asks for:

```
GET /index.html
```

the server simply returns that file.

No Java logic is required.

The same request gives the same pre-written response.

This is called a **static website**.

Simple meaning:

> A static website returns already existing files.

Examples of static files:

Coder Army

```
HTML
CSS
JavaScript
Images
Videos
```

Dynamic Website

Now imagine this request:

```
GET /user?id=101
```

Here, the response depends on the user id.

For `id=101`, the response may be:

```
Name: Rahul
Email: rahul@test.com
```

For `id=102`, the response may be:

```
Name: Priya
Email: priya@test.com
```

Now the server needs logic.

It may need to:

```
1. Read the user id
2. Connect to a database
3. Fetch user details
4. Prepare the response
5. Send the response back
```

This is where Java becomes useful.

Java is good for writing business logic.

Coder Army

But there is still one important question:

> How will the HTTP request enter Java code?

This is the gap Servlets solve.

---

5. The Main Problem Before Servlets

Before Servlets, if we wanted to build a Java web application, every developer would have to write a lot of low-level networking and HTTP handling code manually.

The flow would look like this:

```
Browser
   ↓
Raw HTTP request
   ↓
Java socket code
   ↓
Manual HTTP parsing
   ↓
Manual routing
   ↓
Manual response creation
```

This is not practical for every web application.

Developers should focus on business logic, not on manually parsing HTTP again and again.

So Java needed a standard way where:

```
Low-level HTTP work is handled by a server/container
Business logic is written by the developer
```

That standard idea is called a **Servlet**.

---

6. What Servlet Did Conceptually

CoderArmy

Servlet introduced a simple idea:

> Instead of every developer manually handling raw HTTP, let a container handle HTTP and call our Java class.

So the flow became:

```
Browser
   ↓
HTTP Request
   ↓
Tomcat
   ↓
HttpServletRequest object
HttpServletResponse object
   ↓
Our Servlet class
```

Tomcat handles the low-level HTTP work.

Our Servlet handles the application logic.

So instead of reading raw HTTP text manually, we get ready-made Java objects:

```
HttpServletRequest request
HttpServletResponse response
```

The request object gives us details like:

```
URL
HTTP method
Headers
Query parameters
Request body
Cookies
```

The response object helps us send data back to the browser.

---

7. Simplest Meaning of Servlet

A Servlet is:

Coder Army

> A Java class that Tomcat can call when an HTTP request comes.

In practical words:

> A Servlet is a Java class used to handle web requests and generate web responses.

Example:

```
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) {
        // Logic for GET request
    }
}
```

The browser does not directly call `doGet()`.

Tomcat calls `doGet()` when a matching GET request comes.

---

8. Servlet Does Not Directly Listen on Port 8080

A common beginner mistake is thinking:

> Servlet is listening on port 8080.

That is not technically correct.

The correct mental model is:

```
Tomcat listens on the port.
Servlet lives inside Tomcat.
When a matching request comes, Tomcat calls the Servlet.
```

Flow:

Coder Army

```
Port 8080
   ↓
Tomcat is listening
   ↓
Request comes: /hello
   ↓
Tomcat checks URL mapping
   ↓
Tomcat calls HelloServlet
```

So the Servlet is not directly connected to the network.

Tomcat is the long-running server process.

Servlet is a Java object managed by Tomcat.

---

9. Browser Is the Client

When we open:

```
http://localhost:8080/hello
```

the browser acts as the client.

Its job is simple:

```
Send request → receive response
```

The browser does not care whether the backend is written in:

```
Java
Python
Node.js
PHP
Go
```

The browser only cares about the HTTP response.

From the browser's perspective:

Coder Army

```
I sent an HTTP request.
I received an HTTP response.
```

---

10. Server Is the Receiver

The request reaches a server process that is already running.

In the Servlet world, that server process is usually:

```
Tomcat
```

Tomcat usually runs on port:

```
8080
```

So this URL:

```
http://localhost:8080/hello
```

means:

Part	Meaning
`localhost`	My own machine
`8080`	Port where Tomcat is listening
`/hello`	Path requested by the browser

The request first reaches Tomcat.

Then Tomcat decides which Java code should handle the request.

---

11. What Is Tomcat?

Tomcat is a Java-based server that can run Java web applications.

Coder Army

More specifically:

> Tomcat is a Servlet Container.

This means Tomcat provides the environment where Servlets can live, run, and handle HTTP requests.

---

12. What Is a Servlet Container?

The word **container** is very important.

A container means:

> Something that creates, manages, and controls objects.

In Spring Core, we say:

```
Spring Container manages beans.
```

Similarly, in the Servlet world:

```
Servlet Container manages Servlets.
```

Tomcat is called a Servlet Container because it manages Servlet objects.

Tomcat controls:

```
1. Servlet object creation
2. init() method call
3. service() method call
4. doGet() / doPost() / doPut() / doDelete() method call
5. destroy() method call
6. Request object creation
7. Response object creation
8. Thread management
9. URL mapping
10. Sending response back to browser
```

So we do not manually create Servlet objects.

Tomcat creates and manages them.

Coder Army

13. Control Is Inverted

This connects with the idea of Inversion of Control.

In normal Java:

```
We create object.
We call method.
```

Example:

```
HelloServlet servlet = new HelloServlet();
servlet.doGet(request, response);
```

But in Servlet-based applications:

```
Tomcat creates the object.
Tomcat calls the method.
```

So the control is not with our code.

The control is with the container.

This is similar to the idea of **Inversion of Control**.

World	Object Managed By
Servlet world	Tomcat / Servlet Container
Spring world	Spring Container

14. What Happens When Tomcat Starts?

When Tomcat starts, it becomes a long-running server process.

It does not start, run one method, and stop.

It keeps waiting for HTTP requests.

Coder Army

Basic flow:

```
Tomcat starts
   ↓
Tomcat opens port 8080
   ↓
Tomcat waits for HTTP requests
   ↓
Browser sends request
   ↓
Tomcat receives request
   ↓
Tomcat finds matching application
   ↓
Tomcat finds matching Servlet
   ↓
Tomcat calls Servlet method
   ↓
Servlet prepares response
   ↓
Tomcat sends response back to browser
```

---

15. What Tomcat Does When a Request Comes

Suppose a request comes like this:

```
GET /myapp/hello
```

Tomcat checks:

```
Which application?  → myapp
Which URL?          → /hello
Which Servlet?      → HelloServlet
Which method?       → doGet()
```

Then Tomcat calls something conceptually similar to:

```
helloServlet.doGet(request, response);
```

Tomcat acts as the bridge between:

Coder Army

```
HTTP world and Java world
```

The browser sends HTTP.

Tomcat converts that HTTP request into Java-friendly objects.

Then our Servlet uses those objects.

---

16. Does Tomcat Create Servlet Object on Startup?

Usually, Tomcat creates the Servlet object when the first matching request comes.

Example:

```
First request to /hello
   ↓
Tomcat creates HelloServlet object
   ↓
Tomcat calls init()
   ↓
Tomcat calls doGet()
```

After that, the same Servlet object can handle future requests.

But we can also configure Tomcat to load a Servlet during application startup.

So there are two possibilities:

Behavior	Meaning
Default behavior	Servlet loads on first request
Startup loading	Servlet loads when the application starts

External Tomcat and WAR Deployment

17. What Is External Tomcat?

In traditional Java web development, Tomcat is installed separately.

Coder Army

The flow is:

```
1. Install Tomcat on your machine/server
2. Build the Java web application as a WAR file
3. Put the WAR file inside Tomcat
4. Start Tomcat
5. Tomcat runs the application
```

This is called using **external Tomcat**.

Mental model:

```
Application goes inside Tomcat.
```

---

18. What Is a WAR File?

WAR means:

```
Web Application Archive
```

A WAR file is a packaged Java web application.

Just like a normal Java project can be packaged as:

```
myapp.jar
```

a traditional Java web application is packaged as:

```
myapp.war
```

A WAR file can contain:

```
Compiled Java classes
Servlet classes
HTML/CSS/JS files
```

```
web.xml
Libraries
Configuration files
```

Simple mental model:

```
WAR = complete web application package
```

External Tomcat knows how to run WAR files.

---

19. One Tomcat Can Run Multiple Web Applications

External Tomcat can host multiple Java web applications at the same time.

Example:

```
External Tomcat
 ├── app1
 │    └── HelloServlet
 │
 ├── app2
 │    └── LoginServlet
 │
 └── app3
      └── ProductServlet
```

Each application can have its own Servlets.

If a request comes like:

```
/app1/hello
```

Tomcat sends it to the Servlet inside `app1`.

If a request comes like:

```
/app2/login
```

Coder Army

Coder Army

Tomcat sends it to the Servlet inside `app2`.

This is another reason Tomcat is called a container.

It can contain and manage multiple web applications.

---

20. Starting External Tomcat

Go inside your external Tomcat folder.

Example on Mac/Linux:

```
cd ~/Downloads/apache-tomcat-10.1.x
```

Inside this folder, you should see folders like:

```
bin
conf
logs
webapps
```

Check Tomcat Port

Open this file:

```
conf/server.xml
```

Find the connector configuration:

```
<Connector port="8080" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443" />
```

This means Tomcat will listen on port `8080`.

If it is using another port, change it to:

Coder Army

```
port="8080"
```

Then save the file.

Give Permission on Mac/Linux

On Mac/Linux, give permission to run Tomcat scripts:

```
chmod +x bin/*.sh
```

This step is not required on Windows.

Start Tomcat in Foreground

Foreground mode is useful while teaching because logs are visible directly.

On Mac/Linux:

```
./bin/catalina.sh run
```

If everything is fine, you should see something like:

```
Server startup in ... milliseconds
```

Now open:

```
http://localhost:8080
```

You should see the Tomcat welcome page.

Start and Stop Tomcat in Background

On Mac/Linux:

Coder Army

```
./bin/startup.sh
```

To stop Tomcat:

```
./bin/shutdown.sh
```

On Windows:

```
bin\startup.bat
```

To stop Tomcat:

```
bin\shutdown.bat
```

If Port 8080 Is Already Busy

On Mac/Linux, check what is using port `8080`:

```
lsof -i :8080
```

You may see a Java process using the port.

Kill that process:

```
kill -9 <process-id>
```

Then start Tomcat again.

---

21. Deploying WAR in External Tomcat

After building the project, copy the WAR file:

```
target/servlet-crud.war
```

into Tomcat's `webapps` folder.

Coder Army

Mac/Linux

```
cp target/servlet-crud.war ~/tools/apache-tomcat-10/webapps/
```

Windows

Copy:

```
target\servlet-crud.war
```

to:

```
C:\tools\apache-tomcat-10\webapps
```

Now start Tomcat.

Tomcat will automatically extract:

```
servlet-crud.war
```

into:

```
webapps/servlet-crud
```

Now open:

```
http://localhost:8080/servlet-crud/
```

If the app is configured correctly, you should see:

Coder Army

```
Servlet CRUD App is running
```

Servlet Dependency Scope

22. Why Is Servlet Dependency Scope `provided`?

In a Servlet project, we use Servlet classes like:

```
HttpServlet
HttpServletRequest
HttpServletResponse
```

These classes are needed to compile our code.

But at runtime, these Servlet classes are already provided by Tomcat.

External Tomcat has its own `lib` folder:

```
apache-tomcat-10
 ├── bin
 ├── conf
 ├── lib
 ├── logs
 ├── webapps
```

Inside Tomcat's `lib`, Tomcat already has the Servlet-related classes needed to run web applications.

So Maven needs the Servlet dependency only during compilation.

At runtime, Tomcat provides it.

That is exactly what `provided` means.

Simple meaning:

```
I need this dependency while compiling my code,
but do not package it inside my final WAR,
```

```
because the runtime environment will provide it.
```

In this case:

```
Compile time → Maven needs Servlet API
Runtime      → Tomcat provides Servlet API
```

So the dependency should not be bundled inside:

```
servlet-crud.war
```

---

23. What Happens If We Do Not Use `provided`?

If we remove:

```
<scope>provided</scope>
```

then Maven uses the default scope:

```
compile
```

That means Maven may package the Servlet API inside the WAR file.

But Tomcat already has the Servlet API.

Now two copies may exist:

```
One copy inside Tomcat
One copy inside the application WAR
```

This can create classloading conflicts or confusing runtime issues.

In container-based applications, container APIs like Servlet API should usually be provided by the container, not packaged by the application.

Coder Army

CoderArmy

Query Parameters in First Servlet Example

24. Why Use Query Parameters Instead of POST Body?

In the first Servlet example, query parameters are easier to use.

For example:

```
/users?name=Rahul&email=rahul@test.com
```

Using a JSON request body in Servlet requires extra work.

We would need to understand:

```
1. How to read the request body manually
2. How to parse JSON manually
3. How to add Jackson or Gson dependency
4. How to map JSON to a Java object
5. How to handle invalid JSON
```

So for the first Servlet example, query parameters keep the focus on the main concept:

```
How Tomcat receives a request and calls our Servlet.
```

Later, JSON request body handling can be introduced separately.

Servlet Lifecycle

25. What Is Servlet Lifecycle?

Servlet lifecycle means the complete journey of a Servlet object.

The basic lifecycle is:

```
Servlet object creation
   ↓
init()
```

Coder Army

```
   ↓
service()
   ↓
doGet() / doPost() / doPut() / doDelete()
   ↓
destroy()
```

Tomcat controls this lifecycle.

We do not manually create or destroy Servlet objects.

---

26. `init()` Method

`init()` is called when the Servlet object is initialized.

Usually:

```
First request comes
   ↓
Tomcat creates Servlet object
   ↓
Tomcat calls init()
```

By default, Tomcat usually creates the Servlet object on the first matching request.

Example:

```
First GET /lifecycle

Constructor runs
   ↓
init() runs
   ↓
service() runs
   ↓
doGet() runs
```

After that, if another request comes:

```
Second GET /lifecycle
   ↓
```

Coder Army

```
service() runs
   ↓
doGet() runs
```

The constructor and `init()` do not run again for every request.

---

27. Use of `init()`

`init()` is used for one-time setup work.

Examples:

```
Load configuration
Create expensive resources
Initialize helper objects
Open connection pool
Prepare cache
```

---

28. `service()` Method

`service()` is called for every request.

For example:

```
GET /lifecycle      → service()
POST /lifecycle     → service()
PUT /lifecycle      → service()
DELETE /lifecycle   → service()
```

But when we extend `HttpServlet`, we usually do not write all logic inside `service()`.

Why?

Because `HttpServlet` already has a `service()` method that checks the HTTP method and forwards the request to the correct method.

Conceptually:

Coder Army

```
if (requestMethod is GET) {
    call doGet();
}

if (requestMethod is POST) {
    call doPost();
}

if (requestMethod is PUT) {
    call doPut();
}

if (requestMethod is DELETE) {
    call doDelete();
}
```

So the internal flow is:

```
service()
   ↓
Checks HTTP method
   ↓
Calls doGet() / doPost() / doPut() / doDelete()
```

In real Servlet code, we usually override:

```
doGet()
doPost()
doPut()
doDelete()
```

We usually do not override `service()` unless we have a specific reason.

Coder Army

29. `doGet()`, `doPost()`, `doPut()`, and `doDelete()`

`doGet()`

`doGet()` handles HTTP GET requests.

Usually used for reading/fetching data.

`doPost()`

`doPost()` handles HTTP POST requests.

Usually used for creating new data.

`doPut()`

`doPut()` handles HTTP PUT requests.

Usually used for updating existing data.

`doDelete()`

`doDelete()` handles HTTP DELETE requests.

Usually used for deleting data.

---

30. `destroy()` Method

`destroy()` is called when the Servlet is being removed from service.

This can happen when:

```
Tomcat stops
Application is undeployed
Application is reloaded
New WAR replaces old WAR
```

Use of `destroy()`:

Coder Army

```
Close resources
Release memory-heavy objects
Stop background tasks
Flush pending data
```

Important point:

```
destroy() is called once before the Servlet is removed.
```

After `destroy()`, the Servlet object becomes eligible for garbage collection.

---

31. Most Important Lifecycle Point

Usually, Tomcat creates one Servlet object and reuses it for multiple requests.

So the flow is not:

```
Every request → new Servlet object
```

The better mental model is:

```
One Servlet object → many requests
```

That is why we should be careful with instance variables inside Servlets, especially when multiple users are sending requests at the same time.

---

32. How to See `destroy()` Output

To see `destroy()` output, stop Tomcat.

On Mac/Linux:

```
./bin/shutdown.sh
```

On Windows:

Coder Army

```
shutdown.bat
```

When Tomcat stops, it removes the Servlet from service and calls `destroy()`.

Embedded Tomcat in Spring Boot

33. What Is Embedded Tomcat?

In traditional Servlet applications:

```
Application goes inside Tomcat.
```

But in Spring Boot, we usually do not install Tomcat separately.

Spring Boot brings Tomcat inside the application through:

```
spring-boot-starter-web
```

When we run:

```
public static void main(String[] args) {
    SpringApplication.run(MyApp.class, args);
}
```

Spring Boot starts the application and also starts embedded Tomcat internally.

So in Spring Boot:

```
Tomcat is inside the application.
```

Coder Army

34. External Tomcat vs Embedded Tomcat

Traditional Java Web App

```
Install Tomcat
   ↓
Build WAR
   ↓
Deploy WAR inside Tomcat
   ↓
Start Tomcat
   ↓
Application runs
```

Mental model:

```
Application is deployed into the server.
```

Spring Boot Web App

```
Run main()
   ↓
Spring Boot starts
   ↓
Embedded Tomcat starts
   ↓
Application is ready
```

Mental model:

```
Server starts inside the application.
```

---

35. Why Spring Boot Uses Embedded Tomcat

Spring Boot makes Java web applications easier to run.

Coder Army

Without embedded Tomcat:

```
Install Tomcat
Configure Tomcat
Build WAR
Deploy WAR
Start server
Debug deployment issues
```

With embedded Tomcat:

```
Run main()
Application starts
Server starts
API is ready
```

Spring Boot did not remove the Servlet Container concept.

It simply made it easier to use.

Internally, Tomcat is still there.

From Servlets to Spring MVC

36. Traditional Servlet Style

In a traditional Servlet application, we may create different Servlets for different features.

Example:

```
/users       → UserServlet
/products    → ProductServlet
/orders      → OrderServlet
```

Flow:

Coder Army

```
Browser/Postman
   ↓
Tomcat
   ↓
Specific Servlet
   ↓
Business logic
```

Example:

```java
@WebServlet("/users")
public class UserServlet extends HttpServlet {
}
```

Here, Tomcat checks the URL and directly calls the matching Servlet.

This works, but as the application grows, we may end up creating many Servlets.

Each Servlet may handle:

```
Request reading
Response writing
Validation
Routing
Error handling
Business flow
```

Slowly, the code becomes difficult to manage.

---

37. The Spring MVC Idea

Spring MVC also uses the Servlet concept.

But instead of asking us to manually create many Servlets, Spring MVC gives us one central Servlet.

That central Servlet is called:

```
DispatcherServlet
```

Coder Army

This is the heart of Spring MVC.

The idea is simple:

> Let all incoming requests first come to one central Servlet, and then that Servlet will decide which controller method should handle the request.

So in Spring MVC, the flow becomes:

```
Browser/Postman
   ↓
Tomcat
   ↓
DispatcherServlet
   ↓
Controller
   ↓
Service
   ↓
Response
```

---

38. Traditional Servlet vs Spring MVC

Traditional Servlet Style

```
/users       → UserServlet
/products    → ProductServlet
/orders      → OrderServlet
```

Here, we manually create multiple Servlets.

Spring MVC Style

```
/users       ┐
/products    ├── DispatcherServlet ──→ Correct Controller Method
/orders      ┘
```

Here, requests first come to `DispatcherServlet`.

Coder Army

Then `DispatcherServlet` finds the correct controller method.

Example:

```
@RestController
public class UserController {

    @GetMapping("/users")
    public String getUsers() {
        return "All users";
    }
}
```

The browser is still sending an HTTP request.

Tomcat is still receiving that request.

But instead of directly calling our custom `UserServlet`, Tomcat calls Spring's `DispatcherServlet`.

Then `DispatcherServlet` calls the correct controller method.

---

39. Important Mental Model for Spring MVC

A Spring MVC controller is not directly called by the browser.

The browser does not know about:

```
UserController
getUsers()
@GetMapping
```

The browser only sends:

```
GET /users
```

The actual flow is:

Coder Army

```
Browser sends HTTP request
   ↓
Tomcat receives request
   ↓
Tomcat calls DispatcherServlet
   ↓
DispatcherServlet checks Spring MVC mappings
   ↓
Correct controller method is called
   ↓
Response goes back
```

---

40. Why Spring MVC Is Powerful

With traditional Servlets, we manually handle many things.

For example:

```
Read request parameters
Call service class
Prepare response
Convert object to JSON
Handle errors
Manage dependencies
```

Spring MVC simplifies this by giving us:

```
@Controller / @RestController
@GetMapping / @PostMapping
@RequestParam
@RequestBody
@ResponseBody
Automatic JSON conversion
Cleaner Controller-Service-Repository separation
```

But internally, the foundation is still Servlet.

Spring MVC did not remove Servlet.

Spring MVC built a better framework on top of Servlet.

Coder Army

Final Mental Model

The complete Servlet request flow is:

```
Browser sends HTTP request
   ↓
Tomcat receives request
   ↓
Tomcat identifies application
   ↓
Tomcat identifies Servlet
   ↓
Tomcat creates request and response objects
   ↓
Tomcat calls Servlet method
   ↓
Servlet writes response
   ↓
Tomcat sends HTTP response back to browser
```

Remember this:

```
Browser speaks HTTP.
Java writes business logic.
Tomcat connects the HTTP world with the Java world.
Servlet is the Java class Tomcat calls.
```

Do not think your Java class is directly listening to the browser.

The correct understanding is:

```
Tomcat listens.
Tomcat receives the HTTP request.
Tomcat converts it into Java-friendly objects.
Tomcat calls your Servlet.
Your Servlet writes the response.
Tomcat sends it back to the browser.
```

This is the foundation of Java web development.

Once this is clear, Spring MVC and Spring Boot controllers become much easier to understand.

Coder Army

Running Tomcat 10macOS / Linux

Go inside Tomcat folder:

```
cd ~/Downloads/apache-tomcat-10.1.x
```

Give permission to scripts:

```
chmod +x bin/*.sh
```

Run Tomcat in foreground:

```
./bin/catalina.sh run
```

Open in browser:

```
http://localhost:8080
```

Stop Tomcat:

```
Control + C
```

Run Tomcat in background:

```
./bin/startup.sh
```

Stop background Tomcat:

```
./bin/shutdown.sh
```

Coder Army

Windows

Go inside Tomcat folder:

```
cd C:\apache-tomcat-10.1.x
```

Run Tomcat in foreground:

```
bin\catalina.bat run
```

Open in browser:

```
http://localhost:8080
```

Stop Tomcat:

```
Control + C
```

Run Tomcat in background:

```
bin\startup.bat
```

Stop background Tomcat:

```
bin\shutdown.bat
```

If Port 8080 Is BusymacOS / Linux

```
lsof -i :8080
kill -9 PROCESS_ID
```

Coder Army

Windows

```
netstat -ano | findstr :8080
taskkill /PID PROCESS_ID /F
```

Coder Army

Spring MVC Architecture

1. What is Spring Web MVC?

Spring Web MVC, commonly called **Spring MVC**, is Spring's web framework used to build:

- Web applications
- REST APIs
- Backend services that handle HTTP requests

Officially, its module name is:

```
spring-webmvc
```

Spring MVC is built on top of the **Servlet API**. That means, even though we do not directly write `doGet()` or `doPost()` in Spring MVC, the application still works through servlet-based request handling internally.

---

2. What is Spring Boot's Role?

Spring Boot does **not** replace Spring MVC.

Spring Boot simply makes Spring MVC easier to use.

In a traditional Spring MVC application, we manually configure many things:

- Tomcat setup
- Spring application context
- DispatcherServlet registration
- DispatcherServlet mapping
- Component scanning
- JSON conversion

Coder Army

- Spring MVC configuration

Spring Boot does most of this automatically.

In a Spring Boot CRUD project, we usually write methods like:

```
@GetMapping("/{id}")
public Student getStudent(@PathVariable Integer id) {
    return studentService.getStudent(id);
}
```

We do not write:

```
doGet()
doPost()
doPut()
doDelete()
```

But internally, someone still has to receive the HTTP request from Tomcat, understand the URL and HTTP method, and then call the correct controller method.

That internal web system is **Spring MVC**.

The central servlet inside Spring MVC is called:

```
DispatcherServlet
```

---

3. DispatcherServletWhat is DispatcherServlet?

`DispatcherServlet` is the **front controller** of Spring MVC.

It receives incoming HTTP requests and coordinates the complete request-processing flow.

A simple mental model:

Coder Army

```
Client
  ↓
Tomcat
  ↓
DispatcherServlet
  ↓
Controller Method
  ↓
Response
```

Why Do We Need DispatcherServlet?

In a pure Servlet application, we may create different servlets for different URLs.

For example:

```
/student/create  → CreateStudentServlet
/student/get     → GetStudentServlet
/student/update  → UpdateStudentServlet
/student/delete  → DeleteStudentServlet
```

Each servlet may handle its own:

- URL matching
- Request parsing
- Response writing
- JSON conversion
- Error handling

This becomes repetitive as the project grows.

Spring MVC solves this by using one central servlet:

```
All requests first come to DispatcherServlet.
DispatcherServlet then forwards the request to the correct controller method.
```

That is why it is called **DispatcherServlet**.

It dispatches the request to the correct handler.

DispatcherServlet is Still a Servlet

DispatcherServlet is not magic.

It is also a servlet.

Tomcat knows how to call servlets. So when a request comes, Tomcat sends the request to DispatcherServlet.

Then DispatcherServlet takes help from other Spring MVC components to process the request.

4. High-Level Spring MVC Flow

```
Client sends HTTP request
        ↓
Tomcat receives request
        ↓
Tomcat sends request to DispatcherServlet
        ↓
DispatcherServlet asks HandlerMapping:
"Which controller method should handle this request?"
        ↓
HandlerMapping returns the matching controller method
        ↓
DispatcherServlet uses HandlerAdapter to call that method
        ↓
Request data is assigned to method parameters
        ↓
Controller calls Service
        ↓
Service calls Repository
        ↓
Response is returned
        ↓
Spring converts Java object to JSON
        ↓
Client receives response
```

DispatcherServlet is the coordinator.

Coder Army

Coder Army

It does not do everything by itself. It takes help from multiple Spring MVC components.

Important components:

Component	Responsibility
`DispatcherServlet`	Central servlet that receives and coordinates requests
`HandlerMapping`	Finds which controller method should handle the request
`HandlerAdapter`	Actually invokes the selected controller method
`HandlerMethodArgumentResolver`	Resolves method parameters like path variables, query parameters, request body
`HttpMessageConverter`	Converts JSON to Java object and Java object to JSON
`Jackson`	Common JSON library used for conversion

5. HandlerMapping

Why HandlerMapping is Needed

DispatcherServlet receives the request, but it does not directly know which controller method to call.

For example, if this request comes:

```
GET /students/1
```

DispatcherServlet needs to know that this should call:

```
@GetMapping("/{id}")
public Student getStudent(@PathVariable Integer id) {
    return studentService.getStudent(id);
}
```

This matching is done by **HandlerMapping**.

Coder Army

What is HandlerMapping?

HandlerMapping is the Spring MVC component that maps an HTTP request to the correct controller method.

It checks things like:

- URL path
- HTTP method
- Controller annotations
- Method-level mappings

Example:

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping("/{id}")
    public Student getStudent(@PathVariable Integer id) {
        return studentService.getStudent(id);
    }

    @PostMapping
    public Student createStudent(@RequestBody Student student) {
        return studentService.createStudent(student);
    }
}
```

Spring internally understands:

```
GET /students/1       → getStudent()
POST /students        → createStudent()
```

Even if two URLs look similar, the HTTP method can change the meaning.

For example:

```
GET /students     → fetch students
```

Coder Army

```
POST /students    → create student
```

When Does HandlerMapping Build This Mapping?

When the Spring MVC application starts:

1. `@ComponentScan` finds controller classes.
2. Spring creates controller objects as beans.
3. HandlerMapping reads annotations like:
    - `@RequestMapping`
    - `@GetMapping`
    - `@PostMapping`
    - `@PutMapping`
    - `@DeleteMapping`
4. It builds an internal mapping table.

Example internal mapping:

```
GET /students/{id}     → StudentController.getStudent()
POST /students         → StudentController.createStudent()
PUT /students/{id}     → StudentController.updateStudent()
DELETE /students/{id}  → StudentController.deleteStudent()
```

At runtime, HandlerMapping uses this prepared mapping table to quickly find the correct controller method.

6. HandlerAdapter

HandlerMapping only finds the correct method.

It does not execute the method.

The actual method invocation is done by another Spring MVC component called:

Coder Army

```
HandlerAdapter
```

Simple flow:

```
DispatcherServlet asks HandlerMapping:
"Which method should handle this request?"

HandlerMapping replies:
"StudentController.getStudent()"

DispatcherServlet then asks HandlerAdapter:
"Please call this method properly."
```

HandlerAdapter then prepares method arguments and invokes the controller method.

---

7. How Request Data Reaches the Controller Method

In a web API, request data usually comes from three places:

1. Path variable
2. Query parameter
3. Request body

Spring MVC uses argument resolvers to extract this data and assign it to controller method parameters.

7.1 Path Variable

Example request:

```
GET /students/10
```

Controller method:

Coder Army

```
@GetMapping("/{id}")
public Student getStudent(@PathVariable Integer id) {
    return studentService.getStudent(id);
}
```

Spring compares:

```
Request URL: /students/10
Endpoint:    /students/{id}
```

Then it assigns:

```
id = 10
```

This is handled by Spring MVC's argument resolution mechanism.

7.2 Query Parameter

Example request:

```
GET /students?name=Rohit
```

Controller method:

```
@GetMapping
public List<Student> getStudents(@RequestParam String name) {
    return studentService.getStudentsByName(name);
}
```

Here Spring extracts:

```
name = Rohit
```

Tomcat already parses the HTTP request and makes query parameters available through the servlet request object.

Spring MVC then reads those values and assigns them to controller method parameters.

7.3 Request Body

Example POST request:

```
POST /students
Content-Type: application/json
```

Request body:

```
{
  "name": "Rohit",
  "email": "rohit@example.com",
  "mobile": "9999999999"
}
```

Controller method:

```
@PostMapping
public Student createStudent(@RequestBody Student student) {
    return studentService.createStudent(student);
}
```

Here Spring MVC reads the JSON body and converts it into a Java object:

```
Student student
```

This is mainly done using:

```
HttpMessageConverter + Jackson
```

Coder Army

Coder Army

8. Who Converts JSON to Java Object?

In most Spring MVC and Spring Boot applications, JSON conversion is handled by **Jackson**.

Spring MVC uses `HttpMessageConverter` to convert:

```
JSON → Java Object
Java Object → JSON
```

Example:

```
{
  "name": "Rohit",
  "email": "rohit@example.com"
}
```

gets converted into:

```
Student student
```

And when a controller returns:

```
return student;
```

Spring converts that Java object back into JSON response.

---

9. Application Goal

We will create a small Student REST API.

The API will support:

Coder Army

```
POST   /students        → create student
GET    /students/{id}   → get student by id
GET    /students        → get all students
PUT    /students/{id}   → update student
DELETE /students/{id}   → delete student
```

Data will be stored in memory using:

```
Map<Integer, Student>
```

Layered flow:

```
Controller → Service → Repository
```

---

10. Project Structure

```
src
 └── main
     └── java
         └── in
             └── strikes
                 ├── App.java
                 ├── config
                 │   └── WebConfig.java
                 ├── controller
                 │   └── StudentController.java
                 ├── model
                 │   └── Student.java
                 ├── repository
                 │   └── StudentRepository.java
                 └── service
                     └── StudentService.java
```

---

11. pom.xml

Coder Army

Because we are not using Spring Boot, we manually add the required dependencies.

```
<dependencies>

    <!-- Spring MVC -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-webmvc</artifactId>
        <version>6.1.6</version>
    </dependency>

    <!-- Embedded Tomcat -->
    <dependency>
        <groupId>org.apache.tomcat.embed</groupId>
        <artifactId>tomcat-embed-core</artifactId>
        <version>10.1.20</version>
    </dependency>

    <!-- JSON conversion using Jackson -->
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
        <version>2.17.0</version>
    </dependency>

</dependencies>
```

Explanation of Dependencies1. `spring-webmvc`

This gives Spring MVC features such as:

- `DispatcherServlet`
- `@RestController`
- `@Controller`
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`
- HandlerMapping

Coder Army

- HandlerAdapter
- ViewResolver support

2. `tomcat-embed-core`

This allows us to start Tomcat directly from the Java `main()` method.

So instead of installing external Tomcat and deploying a WAR file, we can run:

```
tomcat.start();
```

This means Tomcat is embedded inside our Java application.

---

3. `jackson-databind`

This dependency helps convert:

```
JSON → Java Object
Java Object → JSON
```

Spring MVC uses `HttpMessageConverter`, and Jackson is commonly used behind the scenes for JSON conversion.

---

12. Student.java

```
package in.strikes.model;

public class Student {

    private Integer id;
    private String name;
    private String email;
    private String mobile;

    public Student() {
    }

    public Student(Integer id, String name, String email, String mobile) {
```

Coder Army

```
        this.id = id;
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getMobile() {
        return mobile;
    }

    public void setMobile(String mobile) {
        this.mobile = mobile;
    }
}
```

Explanation

This is a simple model class.

It represents one student.

A student has:

- `id`

Coder Army

- `name`
- `email`
- `mobile`

The no-argument constructor is important because Jackson needs it while converting JSON into a Java object.

---

13. StudentRepository.java

```
package in.strikes.repository;

import in.strikes.model.Student;
import org.springframework.stereotype.Repository;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

@Repository
public class StudentRepository {

    private final Map<Integer, Student> students = new HashMap<>();
    private Integer currentId = 1;

    public Student save(Student student) {
        student.setId(currentId);
        students.put(currentId, student);
        currentId++;
        return student;
    }

    public Student findById(Integer id) {
        return students.get(id);
    }

    public Collection<Student> findAll() {
        return students.values();
    }

    public Student update(Integer id, Student student) {
        Student existingStudent = students.get(id);

        if (existingStudent == null) {
            return null;
        }
```

Coder Army

```
        existingStudent.setName(student.getName());
        existingStudent.setEmail(student.getEmail());
        existingStudent.setMobile(student.getMobile());

        return existingStudent;
    }

    public Student delete(Integer id) {
        return students.remove(id);
    }
}
```

Explanation of `@Repository`

```
@Repository
```

This tells Spring:

```
This class belongs to the repository layer.
Please create and manage its object inside the IoC container.
```

Conceptually, this is the database layer.

Here, we are not using MySQL.

We are using:

```
Map<Integer, Student>
```

as an in-memory database.

In a real project, this repository would talk to the database using:

- JDBC
- JPA
- Hibernate
- Spring Data JPA

Coder Army

14. StudentService.java

```
package in.strikes.service;

import in.strikes.model.Student;
import in.strikes.repository.StudentRepository;
import org.springframework.stereotype.Service;

import java.util.Collection;

@Service
public class StudentService {

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    public Student createStudent(Student student) {
        return studentRepository.save(student);
    }

    public Student getStudent(Integer id) {
        return studentRepository.findById(id);
    }

    public Collection<Student> getAllStudents() {
        return studentRepository.findAll();
    }

    public Student updateStudent(Integer id, Student student) {
        return studentRepository.update(id, student);
    }

    public Student deleteStudent(Integer id) {
        return studentRepository.delete(id);
    }
}
```

Explanation of `@Service`

Coder Army

```
@Service
```

This tells Spring:

```
This class belongs to the service layer.
Please create and manage its object inside the IoC container.
```

The service layer contains business logic.

Right now, the logic is simple, but in real applications this layer may contain:

- Validation
- Business rules
- Calculations
- Calls to multiple repositories
- Transaction handling

Constructor Injection

```
public StudentService(StudentRepository studentRepository) {
    this.studentRepository = studentRepository;
}
```

This is dependency injection.

The service needs the repository, but it does not create the repository object manually.

We do not write:

```
StudentRepository studentRepository = new StudentRepository();
```

Instead, we ask Spring to provide the object.

This keeps the code loosely coupled and easier to test.

Coder Army

15. StudentController.java

```
package in.strikes.controller;

import in.strikes.model.Student;
import in.strikes.service.StudentService;
import org.springframework.web.bind.annotation.*;

import java.util.Collection;

@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentService studentService;

    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @PostMapping
    public Student createStudent(@RequestBody Student student) {
        return studentService.createStudent(student);
    }

    @GetMapping("/{id}")
    public Student getStudent(@PathVariable Integer id) {
        return studentService.getStudent(id);
    }

    @GetMapping
    public Collection<Student> getAllStudents() {
        return studentService.getAllStudents();
    }

    @PutMapping("/{id}")
    public Student updateStudent(@PathVariable Integer id,
                                 @RequestBody Student student) {
        return studentService.updateStudent(id, student);
    }

    @DeleteMapping("/{id}")
    public Student deleteStudent(@PathVariable Integer id) {
        return studentService.deleteStudent(id);
    }
}
```

Coder Army

Explanation of `@RestController`

```
@RestController
```

This tells Spring:

```
This class will handle REST API requests.
The return value of its methods should be written directly in the HTTP response body.
```

So when this method returns a `Student` object:

```
return student;
```

Spring converts it into JSON.

`@RestController` is basically a shortcut for:

```
@Controller + @ResponseBody
```

Explanation of `@RequestMapping`

```
@RequestMapping("/students")
```

This defines the base URL for this controller.

So every API inside this class starts with:

```
/students
```

Coder Army

Explanation of `@PostMapping`

```
@PostMapping
public Student createStudent(@RequestBody Student student) {
    return studentService.createStudent(student);
}
```

This handles:

```
POST /students
```

It is used to create a new student.

Explanation of `@RequestBody`

```
@RequestBody Student student
```

This means:

```
Read the JSON request body and convert it into a Student object.
```

Example request body:

```
{
  "name": "Rohit",
  "email": "rohit@example.com",
  "mobile": "9999999999"
}
```

Spring MVC uses Jackson through `HttpMessageConverter` to convert this JSON into:

```
Student student
```

Coder Army

Explanation of `@GetMapping("/{id}")`

```
@GetMapping("/{id}")
public Student getStudent(@PathVariable Integer id) {
    return studentService.getStudent(id);
}
```

This handles:

```
GET /students/1
```

Explanation of `@PathVariable`

```
@PathVariable Integer id
```

This means:

```
Take the value from the URL path and assign it to this variable.
```

Example:

```
/students/1
```

Here:

```
id = 1
```

16. WebConfig.java

Coder Army

```
package in.strikes.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;

@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "in.strikes")
public class WebConfig {
}
```

This file is very important because we are not using Spring Boot.

So we must manually tell Spring MVC what to do.

Explanation of `@Configuration`

```
@Configuration
```

This tells Spring:

```
This class contains Spring configuration.
```

Explanation of `@EnableWebMvc`

```
@EnableWebMvc
```

This enables Spring MVC features.

Without this, Spring MVC annotations may not work fully, such as:

- `@RestController`
- `@Controller`
- `@RequestMapping`

Coder Army

- `@GetMapping`
- `@PostMapping`
- `@RequestBody`
- `@PathVariable`

It also enables Spring MVC infrastructure such as:

- HandlerMapping
- HandlerAdapter
- Message converters
- Controller handling

Explanation of `@ComponentScan`

```
@ComponentScan(basePackages = "in.strikes")
```

This tells Spring:

```
Scan this package and find Spring-managed classes.
```

So Spring finds:

- `StudentController`
- `StudentService`
- `StudentRepository`

and manages them inside the IoC container.

---

17. App.java

This is the main class where we manually start embedded Tomcat and register Spring MVC.

Coder Army

```
package in.strikes;

import in.strikes.config.WebConfig;
import org.apache.catalina.Context;
import org.apache.catalina.startup.Tomcat;
import org.springframework.web.context.support.AnnotationConfigWebApplicationContext;
import org.springframework.web.servlet.DispatcherServlet;

import java.io.File;

public class App {

    public static void main(String[] args) throws Exception {

        Tomcat tomcat = new Tomcat();

        tomcat.setPort(8080);
        tomcat.getConnector();

        String contextPath = "";
        String docBase = new File("src/main/webapp").getAbsolutePath();

        Context context = tomcat.addContext(contextPath, docBase);

        AnnotationConfigWebApplicationContext springContext =
                new AnnotationConfigWebApplicationContext();

        springContext.register(WebConfig.class);

        DispatcherServlet dispatcherServlet =
                new DispatcherServlet(springContext);

        Tomcat.addServlet(context, "dispatcherServlet", dispatcherServlet);

        context.addServletMappingDecoded("/", "dispatcherServlet");

        tomcat.start();
        tomcat.getServer().await();
    }
}
```

18. App.java Step-by-Step Explanation

Step 1: Create Tomcat

Coder Army

```
Tomcat tomcat = new Tomcat();
```

We are manually creating an embedded Tomcat server.

In external Tomcat, Tomcat already exists outside the application.

Here, Tomcat is created inside our Java application.

Step 2: Set Port

```
tomcat.setPort(8080);
```

This means our server will run on:

```
http://localhost:8080
```

Step 3: Create Connector

```
tomcat.getConnector();
```

This initializes the connector that listens for HTTP requests.

Step 4: Create Web Context

```
String contextPath = "";
String docBase = new File("src/main/webapp").getAbsolutePath();

Context context = tomcat.addContext(contextPath, docBase);
```

This creates a web application context.

`contextPath` means the base path of the application.

Coder Army

For example, in external Tomcat, if we deploy:

```
student-api.war
```

then the context path usually becomes:

```
/student-api
```

So the URL becomes:

```
http://localhost:8080/student-api/students
```

But here:

```
String contextPath = "";
```

So our API runs directly at:

```
http://localhost:8080/students
```

not:

```
http://localhost:8080/app-name/students
```

What is `docBase`?

`docBase` is the physical folder representing the web application.

Tomcat expects a folder for the web application context.

In our REST API, we may not need static files, but Tomcat still expects a valid folder.

Coder Army

So we pass:

```
src/main/webapp
```

as the `docBase`.

Step 5: Create Spring Web Context

```
AnnotationConfigWebApplicationContext springContext =
        new AnnotationConfigWebApplicationContext();
```

This creates a Spring application context for a web application.

This is where Spring manages:

- Controllers
- Services
- Repositories
- Configuration classes

Step 6: Register Config Class

```
springContext.register(WebConfig.class);
```

This tells Spring:

```
Use WebConfig as the configuration class.
```

From `WebConfig`, Spring knows:

- Spring MVC should be enabled
- Which package should be scanned

- Which classes should become beans

Step 7: Create DispatcherServlet

```
DispatcherServlet dispatcherServlet =
        new DispatcherServlet(springContext);
```

This creates the central servlet of Spring MVC.

We pass `springContext` to DispatcherServlet because DispatcherServlet needs access to the Spring IoC container.

Why DispatcherServlet Needs Spring Context

DispatcherServlet receives the HTTP request from Tomcat.

But after receiving the request, it needs to know:

- Which controllers exist
- Which URL maps to which method
- Which services are available
- Which repositories are available
- Which message converters are available

All this information is inside the Spring IoC container.

That is why DispatcherServlet needs the Spring context.

Step 8: Register DispatcherServlet with Tomcat

```
Tomcat.addServlet(context, "dispatcherServlet", dispatcherServlet);
```

This tells Tomcat:

Coder Army

Coder Army

```
This web application has a servlet named dispatcherServlet.
```

Step 9: Map DispatcherServlet

```
context.addServletMappingDecoded("/", "dispatcherServlet");
```

This tells Tomcat:

```
Send all requests to DispatcherServlet.
```

So when a request comes:

```
GET /students/1
```

Tomcat sends it to DispatcherServlet.

Then DispatcherServlet finds and calls the correct controller method.

Step 10: Start Server

```
tomcat.start();
tomcat.getServer().await();
```

`tomcat.start()` starts the server.

`await()` keeps the server running.

Without `await()`, the `main()` method may finish and the application may stop immediately.

19. How to Run

Coder Army

Run the `App.java` file from the IDE.

Then test APIs using Postman or browser.

Example URLs:

```
POST http://localhost:8080/students
GET  http://localhost:8080/students
GET  http://localhost:8080/students/1
PUT  http://localhost:8080/students/1
DELETE http://localhost:8080/students/1
```

---

20. What Spring Boot Auto-Configures

In the manual Spring MVC application, we configured many things ourselves:

- Added Spring MVC dependency
- Added embedded Tomcat dependency
- Added Jackson dependency
- Created Tomcat manually
- Created Spring web context manually
- Registered WebConfig manually
- Created DispatcherServlet manually
- Registered DispatcherServlet with Tomcat
- Mapped DispatcherServlet to `/`
- Started Tomcat manually

In Spring Boot, most of this is automatic.

Spring Boot web applications include an embedded web server by default when we use:

```
spring-boot-starter-web
```

For servlet-based applications, Spring Boot brings embedded Tomcat by default.

Coder Army

21. Same App in Spring Boot

With Spring Boot, the `pom.xml` mainly needs:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

And the main class becomes:

```
@SpringBootApplication
public class StudentApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentApplication.class, args);
    }
}
```

That is it.

The controller, service, repository, and model can remain almost the same.

22. Manual Spring MVC vs Spring Boot

Work	Manual Spring MVC Embedded Tomcat	Spring Boot
Add Spring MVC dependency	Manually add `spring-webmvc`	Included in starter
Add embedded Tomcat	Manually add `tomcat-embed-core`	Included in starter
Add Jackson	Manually add `jackson-databind`	Included in starter
Start Tomcat	Manually in `App.java`	Automatic

Coder Army

Create Spring web context	Manual	Automatic
Register DispatcherServlet	Manual	Automatic
Map DispatcherServlet	Manual	Automatic
Enable Spring MVC	`@EnableWebMvc`	Auto-configured
Component scanning	`@ComponentScan`	Via `@SpringBootApplication`
Run application	Custom Tomcat code	`SpringApplication.run()`

23. `@Controller` vs `@RestController`

`@Controller`

`@Controller` is traditionally used when we want to return web pages.

Example:

```
@Controller
public class HomeController {

    @GetMapping("/home")
    public String home() {
        return "home";
    }
}
```

Here:

```
return "home";
```

does not mean response body `"home"`.

It means:

Coder Army

```
Find and return the home view page.
```

That view page could be:

```
home.jsp
```

`@RestController`

`@RestController` is used for REST APIs.

Example:

```
@RestController
public class StudentController {

    @GetMapping("/student")
    public Student getStudent() {
        return new Student(1, "Rohit", "rohit@example.com", "9999999999");
    }
}
```

Here the return value means:

```
Convert this Java object into JSON and send it in the HTTP response body.
```

So:

```
return student;
```

becomes:

```
{
  "id": 1,
  "name": "Rohit",
  "email": "rohit@example.com",
```

Coder Army

```
  "mobile": "9999999999"
}
```

Simple difference:

```
@Controller      → usually returns views/pages
@RestController  → usually returns data/JSON
```

---

24. Why ViewResolver Exists

Spring MVC was not created only for REST APIs.

Originally, Spring MVC was also heavily used to create full web applications where the backend returned HTML pages.

In that style, the controller does not return JSON.

The controller returns the name of a view page.

Example:

```
@GetMapping("/home")
public String home() {
    return "home";
}
```

Here:

```
return "home";
```

means:

```
Find the home page and render it.
```

That page could be:

```
/WEB-INF/views/home.jsp
```

This job is done by **ViewResolver**.

---

25. ViewResolver Example

If the controller returns:

```
return "home";
```

and ViewResolver is configured like this:

```
resolver.setPrefix("/WEB-INF/views/");
resolver.setSuffix(".jsp");
```

then Spring converts:

```
home
```

into:

```
/WEB-INF/views/home.jsp
```

So ViewResolver maps logical view names to actual view files.

---

26. JSP MVC Project Structure

```
src
 └── main
     ├── java
     │   └── in
     │       └── strikes
```

Coder Army

```
│           ├── App.java
│           ├── config
│           │   └── WebConfig.java
│           └── controller
│               └── HomeController.java
│
└── webapp
    ├── WEB-INF
    │   └── views
    │       └── home.jsp
    └── resources
        └── css
            └── style.css
```

Important:

```
JSP files are kept inside WEB-INF so users cannot directly access them from the browser.
```

The request should go through the controller first.

---

27. Extra Dependency for JSP

For JSP support, we need Tomcat's JSP engine.

Add:

```
<dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
    <version>10.1.20</version>
</dependency>
```

`tomcat-embed-jasper` is needed because JSP pages require Tomcat's JSP engine, called Jasper.

---

28. WebConfig.java for JSP

Coder Army

Coder Army

```
package in.strikes.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.ViewResolver;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.web.servlet.view.InternalResourceViewResolver;

@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "in.strikes")
public class WebConfig implements WebMvcConfigurer {

    @Bean
    public ViewResolver viewResolver() {
        InternalResourceViewResolver resolver =
                new InternalResourceViewResolver();

        resolver.setPrefix("/WEB-INF/views/");
        resolver.setSuffix(".jsp");

        return resolver;
    }

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/resources/**")
                .addResourceLocations("/resources/");
    }
}
```

Explanation of ViewResolver

```
resolver.setPrefix("/WEB-INF/views/");
resolver.setSuffix(".jsp");
```

When controller returns:

```
return "home";
```

Coder Army

ViewResolver converts it into:

```
/WEB-INF/views/home.jsp
```

So:

```
home → /WEB-INF/views/home.jsp
```

Explanation of `addResourceHandlers`

```
registry.addResourceHandler("/resources/**")
        .addResourceLocations("/resources/");
```

This allows the browser to access static files like CSS.

For example:

```
/resources/css/style.css
```

will load:

```
src/main/webapp/resources/css/style.css
```

Without this configuration, CSS may not load properly because with `@EnableWebMvc`, we are manually controlling Spring MVC configuration.

---

29. HomeController.java

```
package in.strikes.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
```

Coder Army

```
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/home")
    public String home(Model model) {

        model.addAttribute("title", "Spring MVC");
        model.addAttribute("message", "Welcome to Spring MVC with JSP");

        return "home";
    }
}
```

Explanation of `@Controller`

```
@Controller
```

This is used when we want to return a view page like JSP.

Explanation of Model

```
Model model
```

Model is used to send data from the controller to the JSP page.

Simple MVC meaning:

```
Model      → data
View       → JSP page
Controller → handles request and prepares model
```

Example:

```
model.addAttribute("title", "Spring MVC");
```

```
model.addAttribute("message", "Welcome to Spring MVC with JSP");
```

These values can be used inside JSP.

30. home.jsp

Create this file:

```
src/main/webapp/WEB-INF/views/home.jsp
```

Example:

```
<!DOCTYPE html>
<html>
<head>
    <title>${title}</title>
    <link rel="stylesheet" href="/resources/css/style.css">
</head>
<body>
    <h1>${title}</h1>
    <p>${message}</p>
</body>
</html>
```

Explanation

JSP is the View.

It receives data from the Model.

These values:

```
${title}
${message}
```

come from the controller:

Coder Army

Coder Army

```
model.addAttribute("title", "Spring MVC");
model.addAttribute("message", "Welcome to Spring MVC with JSP");
```

---

31. style.css

Create this file:

```
src/main/webapp/resources/css/style.css
```

Example:

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f4;
    padding: 40px;
}

h1 {
    color: #222;
}

p {
    font-size: 18px;
}
```

---

32. Important Difference Between REST API and JSP Example

In the REST API example, we mainly returned JSON.

So the focus was:

```
Controller → Service → Repository → JSON Response
```

Coder Army

But in the JSP example, we return a view page.

So the flow becomes:

```
Controller → Model → ViewResolver → JSP Page
```

Earlier, the REST API did not really need JSP files or static web pages.

But in the JSP example, we need:

```
src/main/webapp
```

because it contains:

- JSP files
- CSS files
- Static resources

Here, `docBase` becomes more important because Tomcat needs to find the actual web application files.

---

33. Final MVC Meaning

Spring MVC stands for:

```
Model - View - Controller
```

In a web-page-based MVC application:

Part	Meaning
Model	Data sent from controller to view
View	JSP page or UI page
Controller	Handles request and decides response/view

Example:

Coder Army

```
User opens /home
        ↓
DispatcherServlet receives request
        ↓
HandlerMapping finds HomeController.home()
        ↓
Controller adds data to Model
        ↓
Controller returns "home"
        ↓
ViewResolver converts "home" to /WEB-INF/views/home.jsp
        ↓
JSP is rendered
        ↓
HTML response is sent to browser
```

---

34. Final Summary

Spring MVC is the web framework that handles HTTP requests in Spring applications.

The most important component is:

```
DispatcherServlet
```

It receives every request and coordinates with other Spring MVC components.

The main request flow is:

```
Tomcat
  ↓
DispatcherServlet
  ↓
HandlerMapping
  ↓
HandlerAdapter
  ↓
Controller
  ↓
Service
  ↓
Repository
```

```
  ↓
Response
```

For REST APIs:

```
Java Object → JSON
```

is handled using:

```
HttpMessageConverter + Jackson
```

For web pages:

```
View name → JSP page
```

is handled using:

```
ViewResolver
```

Spring Boot makes all of this easier by auto-configuring most of the setup.

But understanding manual Spring MVC helps us understand what Spring Boot is doing behind the scenes.

Coder Army

Coders Army

DTOs and Validations in Spring Boot

1. The Real Problem

Suppose we are building a simple **Student Management Application**.

The application should support basic CRUD operations:

```
Create student
Get all students
Get student by ID
Update student
Delete student
```

For example, while creating a student, the client may send this request body:

```
{
  "name": "Amit",
  "email": "amit@gmail.com",
  "age": 21
}
```

Now the important backend question is:

> When this request reaches our Spring Boot application, which layer receives it, which layer processes it, which layer talks to the database, and which layer returns the response?

This is where **layered architecture** becomes important.

---

2. Basic Layers in Our Current Project

In a simple Spring Boot CRUD project, we commonly work with these four parts:

Coder Army

```
Controller
Service
Repository
Entity
```

Layer	Main Responsibility
Controller	Handles HTTP requests and sends HTTP responses
Service	Contains business logic
Repository	Communicates with the database
Entity	Represents a database table

A clean backend application should keep these responsibilities separate.

---

3. Entity: The Class Connected to the Database

An **Entity** is a Java class mapped to a database table.

Example:

```java
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    private Integer age;

    // getters and setters
}
```

Coder Army

Because of the `@Entity` annotation, JPA/Hibernate understands that this class should be mapped to a database table.

The class:

```
public class Student
```

can become a table like:

```
student
```

And its fields:

```
private Long id;
private String name;
private String email;
private Integer age;
```

can become columns like:

```
id
name
email
age
```

At the database level, the table may look like this:

id	name	email	age
1	Amit	amit@gmail.com	21
2	Neha	neha@gmail.com	22

The first important point is:

> Entity is mainly designed to represent the database structure.

---

4. Complete Create API Flow

Coder Army

In a normal create API, the flow looks like this:

```
Client
   ↓
Controller
   ↓
Service
   ↓
Repository
   ↓
Database
```

Example flow:

```
1. Client sends POST request with JSON data.
2. Controller receives the request.
3. Spring/Jackson converts JSON into a Java object.
4. Controller calls the Service layer.
5. Service applies business logic.
6. Service calls Repository.
7. Repository saves data into the database.
8. Database returns saved data.
9. Service prepares the response.
10. Controller sends the HTTP response back to the client.
```

This flow is correct.

The problem starts when we use the **same Entity class everywhere**.

Part 1: Problem With Passing Entity Everywhere

5. Current Problem

In a beginner CRUD project, we often pass the `Student` entity everywhere:

```
Controller → Service → Repository → Database
```

Coder Army

The same `Student` entity may be used for:

```
1. Representing the database table
2. Receiving request data
3. Moving data through the service layer
4. Getting saved by the repository
5. Returning API response
```

For a small demo project, this works.

But in real-world applications, this becomes a bad design.

---

6. Two Worlds in a Backend Application

A backend application mainly deals with two worlds:

```
Outside World  → API Request / API Response
Inside World   → Database / Business Logic
```

The **outside world** can be:

```
Frontend
Mobile app
Postman
Third-party client
Another backend service
```

The **inside world** contains:

```
Database tables
Entity classes
Repository layer
Business logic
Internal fields
```

The key principle is:

Coder Army

> The outside world should not directly control or depend on the internal database structure.

But when we accept and return entities directly, we start exposing the internal structure of our application.

---

7. Entity Has a Specific Purpose

An Entity is created mainly for database mapping.

It belongs to the internal side of the application because it is closely connected to:

```
Database table
Columns
Primary key
Relationships
Hibernate/JPA
```

On the other hand, request and response objects belong to the external API side.

So a clean separation should be:

```
Entity            → Internal database model
Request/Response  → External API model
```

But when we pass Entity everywhere, we are doing this:

```
Entity = Database model
Entity = Request model
Entity = Response model
```

This is the root problem.

---

8. Problem 1: Client Can Send Fields They Should Not Send

Suppose our `Student` entity has internal fields:

Coder Army

```java
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    private Integer age;

    private Boolean isDeleted = false;

    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;
}
```

Now the controller accepts the entity directly:

```java
@PostMapping
public Student createStudent(@RequestBody Student student) {
    return studentService.createStudent(student);
}
```

The client is supposed to send only this:

```json
{
  "name": "Amit",
  "email": "amit@gmail.com",
  "age": 21
}
```

But because the controller directly accepts `Student`, the client may send extra fields:

```
{
  "id": 100,
  "name": "Amit",
  "email": "amit@gmail.com",
  "age": 21,
  "isDeleted": true,
  "createdAt": "2026-07-04T10:00:00",
  "updatedAt": "2026-07-04T10:00:00"
}
```

This is a design problem.

The client should not decide values like:

```
id
isDeleted
createdAt
updatedAt
internal status fields
```

These fields should be controlled by the backend.

So the first problem is:

> Entity exposes internal fields to the outside world.

---

9. Problem 2: Client Can Modify Sensitive/Internal Data

Later, suppose the entity grows and contains more fields:

```
@Entity
public class Student {

    @Id
```

Coder Army

Coder Army

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    private String password;

    private Boolean isDeleted;

    private Boolean isVerified;

    private String role;

    private String internalRemark;

    private LocalDateTime createdAt;
}
```

Now imagine the client sends this during update:

```
{
  "name": "Amit",
  "email": "amit@gmail.com",
  "age": 21,
  "role": "ADMIN",
  "isVerified": true,
  "isDeleted": false
}
```

This is dangerous.

The user should not be able to make themselves:

Coder Army

```
ADMIN
verified
not deleted
approved
premium
```

These values should be controlled by business logic, not by the client.

This problem is commonly called the **mass assignment problem**.

In simple words:

> The client sends more data than required, and the backend accidentally accepts it.

---

10. Problem 3: Response May Leak Unwanted Data

Suppose the entity contains fields like:

```java
private String password;
private Boolean isDeleted;
private String internalRemark;
private LocalDateTime createdAt;
```

And the controller returns the entity directly:

```java
@GetMapping("/{id}")
public Student getStudentById(@PathVariable Long id) {
    return studentService.getStudentById(id);
}
```

The API response may accidentally expose internal or sensitive fields.

The client usually does not need fields like:

```
password
isDeleted
internalRemark
```

```
internal flags
backend-only metadata
```

So another problem is:

> Returning Entity directly can leak unwanted internal data.

---

11. Problem 4: Request and Response Usually Need Different Structures

In real projects, the data received from the client and the data returned to the client are often different.

For creating a student, the client may send:

```
{
  "name": "Amit",
  "email": "amit@gmail.com",
  "age": 21
}
```

But after creating the student, the API may return:

```
{
  "id": 1,
  "name": "Amit",
  "email": "amit@gmail.com",
  "message": "Student created successfully"
}
```

The request does not need `id` or `message`.

The response may need `id` and `message`.

So request and response should not always use the same class.

Coder Army

Coder Army

12. Problem 5: Database Changes Can Break the API Contract

Suppose the current entity is:

```
@Entity
public class Student {

    private Long id;

    private String name;

    private String email;

    private Integer age;
}
```

The frontend may be expecting this response:

```
{
  "id": 1,
  "name": "Amit Sharma",
  "email": "amit@gmail.com",
  "age": 21
}
```

Later, for database design reasons, we split `name` into two fields:

```
private String firstName;
private String lastName;
```

If we return Entity directly, the API response may also change:

```
{
  "id": 1,
```

Coder Army

```
  "firstName": "Amit",
  "lastName": "Sharma",
  "email": "amit@gmail.com",
  "age": 21
}
```

But the frontend may still be expecting:

```
{
  "id": 1,
  "name": "Amit Sharma",
  "email": "amit@gmail.com",
  "age": 21
}
```

This means one internal database change can break the external API contract.

A good API should not break just because the database structure changed internally.

---

13. Main Design Issue

A class should ideally have one clear responsibility.

But when we pass Entity everywhere, the Entity becomes responsible for too many things:

```
Database mapping
Request handling
Response handling
Business data transfer
External API contract
```

This is why we introduce DTOs.

Part 2: DTOs14. What is a DTO?

Coder Army

DTO stands for:

```
Data Transfer Object
```

A DTO is a simple object used to transfer data between layers or between client and server.

Simple meaning:

> DTO is used to carry only the data required for a particular request or response.

DTO is not created for database mapping.

Class Type	Main Purpose
Entity	Represents database table
DTO	Transfers data in request/response or between layers

---

15. Types of DTOs

In a normal CRUD project, we commonly create two main types of DTOs:

```
Request DTO
Response DTO
```

In real projects, we may create more specific DTOs:

```
CreateStudentRequestDto
UpdateStudentRequestDto
StudentResponseDto
StudentSummaryResponseDto
StudentDetailResponseDto
```

For a beginner project, we can start with:

```
StudentRequestDto
```

```
StudentResponseDto
```

Coder Army

16. Where Should DTOs Be Created?

A common package structure is:

```
src/main/java/com/example/studentapp
    ├── controller
    ├── service
    ├── repository
    ├── entity
    └── dto
```

The `dto` package contains request and response DTO classes.

17. Request DTO

A **Request DTO** represents the data that the client is allowed to send to the API.

It is part of the API contract.

Example:

```java
public class StudentRequestDto {

    private String name;

    private String email;

    private Integer age;

    // getters and setters
}
```

Now the controller can receive this DTO instead of the Entity:

CoderArmy

```
@PostMapping
public StudentResponseDto createStudent(@RequestBody StudentR
equestDto requestDto) {
    return studentService.createStudent(requestDto);
}
```

This clearly says:

```
For creating a student, the client can send only:
name
email
age
```

The Request DTO protects the API boundary.

It prevents the client from directly controlling internal database fields.

---

18. Response DTO

A **Response DTO** represents the data that the API wants to send back to the client.

Example:

```
public class StudentResponseDto {

    private Long id;

    private String name;

    private String email;

    private Integer age;

    private String message;
```

```
    // getters and setters
}
```

This DTO allows us to decide exactly what should be visible in the API response.

19. Better Response Structure

Instead of mixing student data and message in the same level, we can also create a cleaner response structure.

Example:

```
{
  "studentInfo": {
    "id": 1,
    "name": "Amit",
    "email": "amit@gmail.com",
    "age": 21
  },
  "message": "Student created successfully"
}
```

For this, we can create separate response classes.

```
public class CreateStudentResponseDto {

    private StudentInfoDto studentInfo;

    private String message;

    // getters and setters
}
```

```
public class StudentInfoDto {
```

Coder Army

Coder Army

```
    private Long id;

    private String name;

    private String email;

    private Integer age;

    // getters and setters
}
```

This gives a more professional and structured API response.

---

20. DTO Naming Convention

For beginners, simple names are fine:

```
StudentRequestDto
StudentResponseDto
```

In real projects, more specific names are better:

```
CreateStudentRequestDto
UpdateStudentRequestDto
StudentResponseDto
StudentDetailsResponseDto
StudentListResponseDto
```

Reason:

> Different APIs may require different request and response structures.

For example, create request may allow email:

```
public class CreateStudentRequestDto {
```

Coder Army

```
    private String name;

    private String email;

    private Integer age;
}
```

But update request may not allow changing email:

```
public class UpdateStudentRequestDto {

    private String name;

    private Integer age;
}
```

This is the power of DTOs.

They allow each API to define its own contract clearly.

Part 3: Manual DTO Mapping

21. Why Mapping Is Required

DTO and Entity are two different classes.

So we need to convert one into another.

There are two common mappings:

```
StudentRequestDto → Student Entity
Student Entity    → StudentResponseDto
```

---

22. Mapping 1: Request DTO to Entity

Coder Army

When the client sends data, the controller receives `StudentRequestDto`.

But the repository cannot save `StudentRequestDto`.

The repository saves the Entity.

So we convert:

```
StudentRequestDto → Student
```

Example:

```
private Student mapToEntity(StudentRequestDto requestDto) {
    Student student = new Student();

    student.setName(requestDto.getName());
    student.setEmail(requestDto.getEmail());
    student.setAge(requestDto.getAge());

    return student;
}
```

23. Mapping 2: Entity to Response DTO

After saving or fetching data from the database, we get a `Student` entity.

But we do not want to return the Entity directly.

So we convert:

```
Student → StudentResponseDto
```

Example:

```
private StudentResponseDto mapToResponseDto(Student student)
{
    StudentResponseDto responseDto = new StudentResponseDto
```

```
();

    responseDto.setId(student.getId());
    responseDto.setName(student.getName());
    responseDto.setEmail(student.getEmail());
    responseDto.setAge(student.getAge());

    return responseDto;
}
```

Coder Army

24. Where Should Mapping Happen?

For a beginner project, mapping can be done inside the **Service layer**.

Reason:

```
Controller side  → DTO
Repository side  → Entity
```

The Service layer sits between the Controller and Repository.

So it can work as the bridge:

```
Controller
   ↓ DTO
Service
   ↓ Entity
Repository
```

The Controller should not know too much about Entity.

The Repository should not know anything about DTO.

That is why the Service layer is a clean place for mapping in a beginner project.

In larger projects, we can create a separate mapper class or use mapping libraries like MapStruct.

Coder Army

Part 4: Updating CRUD APIs With DTOs

25. Create API With DTO

Controller:

```
@PostMapping
public StudentResponseDto createStudent(@RequestBody StudentR
equestDto requestDto) {
    return studentService.createStudent(requestDto);
}
```

Service:

```
public StudentResponseDto createStudent(StudentRequestDto req
uestDto) {
    Student student = mapToEntity(requestDto);

    Student savedStudent = studentRepository.save(student);

    return mapToResponseDto(savedStudent);
}
```

Flow:

```
Client JSON
   ↓
StudentRequestDto
   ↓
Student Entity
   ↓
Database
   ↓
Student Entity
   ↓
StudentResponseDto
```

```
   ↓
Client Response
```

Coder Army

26. Get All Students API With DTO

Controller:

```java
@GetMapping
public List<StudentResponseDto> getAllStudents() {
    return studentService.getAllStudents();
}
```

Service:

```java
public List<StudentResponseDto> getAllStudents() {
    List<Student> students = studentRepository.findAll();

    return students.stream()
            .map(this::mapToResponseDto)
            .toList();
}
```

Here, for every `Student` entity, we call:

```java
mapToResponseDto(student)
```

So the API returns a list of response DTOs instead of a list of entities.

27. Get Student By ID API With DTO

Controller:

```java
@GetMapping("/{id}")
public StudentResponseDto getStudentById(@PathVariable Long i
d) {
```

```
    return studentService.getStudentById(id);
}
```

Service:

```
public StudentResponseDto getStudentById(Long id) {
    Student student = studentRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Student
not found"));

    return mapToResponseDto(student);
}
```

The Repository returns Entity.

The Service converts Entity into Response DTO.

The Controller returns only the DTO.

---

28. Update API With DTO

Controller:

```
@PutMapping("/{id}")
public StudentResponseDto updateStudent(
        @PathVariable Long id,
        @RequestBody StudentRequestDto requestDto) {

    return studentService.updateStudent(id, requestDto);
}
```

Service:

```
public StudentResponseDto updateStudent(Long id, StudentReque
stDto requestDto) {
    Student existingStudent = studentRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Student
```

Coder Army

Coder Army

```
not found"));

    existingStudent.setName(requestDto.getName());
    existingStudent.setEmail(requestDto.getEmail());
    existingStudent.setAge(requestDto.getAge());

    Student updatedStudent = studentRepository.save(existingS
tudent);

    return mapToResponseDto(updatedStudent);
}
```

Important point:

> We do not create a completely new entity during update. We first fetch the existing entity, update allowed fields, save it again, and return a response DTO.

---

29. Delete API and DTO

For delete APIs, many projects return only a message.

Example:

```
@DeleteMapping("/{id}")
public String deleteStudent(@PathVariable Long id) {
    studentService.deleteStudent(id);
    return "Student deleted successfully";
}
```

A more professional response can be:

```
{
  "message": "Student deleted successfully"
}
```

For that, we can create a simple response DTO:

Coder Army

```
public class ApiResponseDto {

    private String message;

    // getters and setters
}
```

Part 5: Validations30. Why Validation Is Needed

Whenever a backend receives data from outside, the data is not guaranteed to be correct.

The client may send invalid data:

```
{
  "name": "",
  "email": "wrong-email",
  "age": -5
}
```

Or:

```
{
  "name": "A",
  "email": "",
  "age": null
}
```

This data should not directly reach the service layer or database.

Without validation, bad data may get stored:

id	name	email	age
1		wrong-email	-5

Coder Army

This is not meaningful student data.

Validation means:

> Before accepting request data, check whether the data follows the rules of the application.

---

31. Without Validation

Controller:

```
@PostMapping
public StudentResponseDto createStudent(@RequestBody StudentR
equestDto requestDto) {
    return studentService.createStudent(requestDto);
}
```

Without validation, invalid data can reach the service layer.

Then the service maps it to Entity.

Then the repository saves it.

That means bad input can enter the system.

---

32. With Validation

We want rules like:

```
name should not be blank
name should have at least 2 characters
email should not be blank
email should be valid
age should not be null
age should be at least 18
```

Correct flow:

Coder Army

```
Client sends JSON
   ↓
Controller receives request
   ↓
Validation happens
   ↓
If valid   → Service method runs
If invalid → Error response goes back
```

Validation protects the service layer and database from invalid input.

---

33. Adding Validation Dependency

In Spring Boot, validation support usually comes from this dependency:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

If we are using Spring Initializr, we can add this by selecting:

```
Validation
```

---

34. Common Validation Annotations

Validation annotations are placed on fields.

Example:

```java
@NotBlank
private String name;
```

This means:

Coder Army

```
name cannot be null, empty, or only spaces
```

Common validation annotations:

Annotation	Used For	Meaning
`@NotNull`	Any object	Value cannot be `null`
`@NotBlank`	String	Value cannot be `null`, empty, or only spaces
`@NotEmpty`	String, Collection, Array	Value cannot be `null` or empty
`@Email`	String	Value should be a valid email format
`@Min(value)`	Number	Minimum allowed value
`@Max(value)`	Number	Maximum allowed value
`@Size(min, max)`	String, Collection	Length or size should be within range
`@Pattern(regexp)`	String	Value should match a regex pattern
`@Positive`	Number	Value should be positive
`@Past`	Date/time	Date should be in the past
`@Future`	Date/time	Date should be in the future

35. `@NotNull` vs `@NotBlank` vs `@NotEmpty` `@NotNull`

Example:

```java
@NotNull
private Integer age;
```

This checks only one thing:

```
age should not be null
```

Invalid:

Coder Army

```
{
  "age": null
}
```

For numbers like `Integer`, `Long`, `Double`, etc., `@NotNull` is useful.

But for strings, `@NotNull` is often not enough because this is not null:

```
{
  "name": ""
}
```

`@NotBlank`

Example:

```
@NotBlank
private String name;
```

This rejects:

```
{
  "name": null
}
```

```
{
  "name": ""
}
```

```
{
  "name": "      "
}
```

For fields like name, email, title, city, etc., `@NotBlank` is usually better.

Coder Army

`@NotEmpty`

Example:

```
@NotEmpty
private List<String> subjects;
```

This checks that the value is not null and not empty.

Invalid:

```
{
  "subjects": []
}
```

For normal strings, prefer `@NotBlank`.

For lists, arrays, and collections, `@NotEmpty` is useful.

36. Where Should We Add Validations?

A common question is:

```
Should validations be added in Entity or Request DTO?
```

For request validation, validations should be added in the **Request DTO**.

Reason:

> Validation rules are about what the client is allowed to send.

The client sends request data, so request validation naturally belongs to the Request DTO.

37. Updated Student Request DTO With Validations

Coder Army

```java
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

public class StudentRequestDto {

    @NotBlank(message = "Student name is required")
    @Size(min = 2, max = 50, message = "Student name must be between 2 and 50 characters")
    private String name;

    @NotBlank(message = "Student email is required")
    @Email(message = "Student email should be valid")
    private String email;

    @NotNull(message = "Student age is required")
    @Min(value = 18, message = "Student age must be at least 18")
    @Max(value = 60, message = "Student age must not be greater than 60")
    private Integer age;

    // getters and setters
}
```

This DTO clearly defines the rules:

```
name cannot be blank
name length should be between 2 and 50 characters
email cannot be blank
email should be valid
```

```
age cannot be null
age should be between 18 and 60
```

Coder Army

38. Using `@Valid` in Controller

Adding validation annotations in the DTO is not enough.

To trigger validation for request body data, we use `@Valid` in the controller.

```
import jakarta.validation.Valid;

@PostMapping
public StudentResponseDto createStudent(
        @Valid @RequestBody StudentRequestDto requestDto) {

    return studentService.createStudent(requestDto);
}
```

Now Spring validates `StudentRequestDto` before calling the service method.

Same thing can be done for update API:

```
@PutMapping("/{id}")
public StudentResponseDto updateStudent(
        @PathVariable Long id,
        @Valid @RequestBody StudentRequestDto requestDto) {

    return studentService.updateStudent(id, requestDto);
}
```

Important line:

```
@Valid @RequestBody StudentRequestDto requestDto
```

Meaning:

Coder Army

```
@RequestBody → Convert JSON into Java object
@Valid       → Apply validation rules on that Java object
```

39. What Happens Internally During Validation?

Suppose the client sends:

```
POST /api/students
```

Request body:

```
{
  "name": "",
  "email": "wrong-email",
  "age": 15
}
```

Spring performs these steps:

```
1. JSON is converted into StudentRequestDto.
2. Because @Valid is present, Spring checks validation annotations.
3. It checks name.
   name is blank, so validation fails.
4. It checks email.
   email format is invalid, so validation fails.
5. It checks age.
   age is less than 18, so validation fails.
6. Service method is not called.
7. Error response goes back to the client.
```

Key point:

> If validation fails, the request does not reach the service layer.

So validation protects both business logic and database from bad input.

Coder Army

40. Validation Annotations vs `@Valid`

Both parts are important.

```
@NotBlank
@Email
@Min
@Max
@Size
```

These annotations define the rules.

```
@Valid
```

This tells Spring to apply those rules.

So:

```
Validation annotations define the rules.
@Valid triggers the validation process.
```

Part 6: Default Validation Error Response

41. What Status Code Is Returned When Validation Fails?

When validation fails, Spring Boot returns:

```
HTTP 400 Bad Request
```

Why `400`?

Because the client sent invalid request data.

Coder Army

The server is working fine.

The problem is with the request body.

So validation failure is not:

```
500 Internal Server Error
```

It is:

```
400 Bad Request
```

---

42. Why Default Error Response Is Not Good Enough

The default error response is technically correct, but it is not always clean or frontend-friendly.

Frontend usually wants a simple response like this:

```
{
  "name": "Student name is required",
  "email": "Student email should be valid",
  "age": "Student age must be at least 18"
}
```

Or a more structured response like this:

```
{
  "success": false,
  "message": "Validation failed",
  "errors": {
    "name": "Student name is required",
    "email": "Student email should be valid",
    "age": "Student age must be at least 18"
```

```
    }
}
```

Spring Boot may not give this exact clean format automatically.

That is why we later use:

```
Global Exception Handling
```

Global Exception Handling allows us to control the exact error response format.

---

43. Custom Validation Messages

Without a custom message:

```
@NotBlank
private String name;
```

Spring may return a default message like:

```
must not be blank
```

This works, but it is generic.

A better approach is to write a clear custom message:

```
@NotBlank(message = "Student name is required")
private String name;
```

Another example:

```
@Email(message = "Student email should be valid")
private String email;
```

Custom messages make validation errors easier to understand for frontend developers and API users.

Coder Army

Coder Army

Part 7: Final Clean Flow

44. Final Create API Flow With DTO and Validation

```
Client sends JSON
   ↓
Controller receives StudentRequestDto
   ↓
@Valid checks request data
   ↓
If invalid:
   return 400 Bad Request
   ↓
If valid:
   Service method runs
   ↓
Service maps StudentRequestDto to Student Entity
   ↓
Repository saves Student Entity
   ↓
Service maps saved Student Entity to StudentResponseDto
   ↓
Controller returns response DTO
```

This is a cleaner and more professional API design.

45. Entity vs DTO Summary

Point	Entity	DTO
Main purpose	Database mapping	Data transfer
Belongs to	Internal application side	API/request-response side
Connected with	JPA, Hibernate, database table	Controller, request body, response body
Should client control it?	No	Yes, in a limited and controlled way
Can contain internal fields?	Yes	Should contain only required fields

Coder Army

Point	Entity	DTO
Example	`Student`	`StudentRequestDto`, `StudentResponseDto`

46. Request DTO vs Response DTO Summary

Point	Request DTO	Response DTO
Direction	Client to server	Server to client
Represents	What client can send	What API returns
Example	`StudentRequestDto`	`StudentResponseDto`
Usually contains	Input fields	Output fields
Validation	Usually added here	Usually not needed for normal response

47. Important Takeaways

- Entity is mainly for database mapping.
- DTO is mainly for data transfer.
- Request DTO defines what the client is allowed to send.
- Response DTO defines what the API will return.
- Passing Entity everywhere exposes internal database structure.
- DTOs help protect the API boundary.
- DTOs prevent accidental exposure of sensitive/internal fields.
- DTOs help maintain a stable API contract even if the database changes.
- Validation should be added on Request DTOs.
- `@Valid` is required in the controller to trigger validation.
- Invalid request data should return `400 Bad Request`, not `500 Internal Server Error`.
- Custom validation messages make APIs easier to use.
- For a beginner project, manual mapping inside the Service layer is acceptable.

- In larger projects, mapping can be moved to a separate mapper class.

Coder Army

48. Suggested Teaching Ending

DTOs and validations make a CRUD API more professional.

Without DTOs, the API directly exposes the database model.

Without validations, bad data can enter the system.

With DTOs and validations, the application becomes cleaner:

```
Controlled request data
Clean response data
Protected internal model
Better API contract
Better error handling
```

The next natural topic after this is:

```
Global Exception Handling
```

That is where we customize validation errors and other exceptions into a clean, consistent response format.

Coder Army

Exception Handling in Spring Boot

Overview

Exception handling is one of the most important parts of building professional REST APIs. A good API should not only return data; it should also clearly communicate what happened during the request.

A professional API response answers questions like:

- Was the request successful?
- Did the client send something wrong?
- Was the requested resource missing?
- Did something break inside the server?
- Can the frontend understand the response and show the right message?

This is where **HTTP status codes**, **ResponseEntity**, **custom exceptions**, and **global exception handling** become important.

---

1. API Response: Communication Between Two Systems

An API response usually has three main parts:

```
1. Status Code
2. Headers
3. Body
```

Status Code

The status code tells the meaning of the response at the HTTP level.

Example:

Coder Army

```
200 OK
404 Not Found
500 Internal Server Error
```

The frontend, mobile app, or another backend service can read this status code and understand what happened.

Headers

Headers contain metadata about the response.

Example:

```
Location: /api/students/10
Content-Type: application/json
```

Headers are not the main data, but they give extra information about the response.

Body

The body contains the actual data or error message.

Example:

```
{
  "id": 10,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

---

2. Beginner API vs Professional API

A beginner API usually focuses only on this question:

```
Did I get data or not?
```

A professional API focuses on a better question:

Coder Army

```
Did the server clearly tell the client what happened?
```

For example, suppose the frontend calls:

```
GET /api/students/10
```

Many things can happen:

- The student exists.
- The student does not exist.
- The ID format is wrong.
- The database is down.
- There is a bug in backend code.

A professional API should communicate each case properly.

---

3. Example: Professional API Responses

Case 1: Student Exists

```
200 OK
```

```
{
  "id": 10,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

Meaning:

```
The request was successful, and the student data is returned.
```

Coder Army

Case 2: Student Does Not Exist

```
404 Not Found
```

```
{
  "message": "Student with id 10 not found"
}
```

Meaning:

```
The request was valid, but the requested resource does not exist.
```

Case 3: Invalid Request

```
400 Bad Request
```

```
{
  "message": "Student id must be a number"
}
```

Meaning:

```
The client sent an invalid request.
```

Case 4: Server Error

```
500 Internal Server Error
```

Coder Army

```
{
  "message": "Something went wrong on the server"
}
```

Meaning:

```
The client may have sent a valid request, but something failed inside the server.
```

---

4. Common Problems in Beginner CRUD APIs

Problem 1: Everything Returns `200 OK`

Many beginner APIs return `200 OK` for almost every operation:

Operation	Beginner Response
Create student	`200 OK`
Fetch student	`200 OK`
Update student	`200 OK`
Delete student	`200 OK`
Student not found	`200 OK` with `null`
Validation failed	Random error response

This is not expressive.

Better responses would be:

Situation	Better Status Code
New resource created	`201 Created`
Resource not found	`404 Not Found`
Validation failed	`400 Bad Request`
Duplicate data	`409 Conflict`
Unexpected backend issue	`500 Internal Server Error`

Coder Army

Problem 2: Repeated Error Logic in Controllers

A beginner controller may look like this:

```
@GetMapping("/{id}")
public ResponseEntity<?> getStudentById(@PathVariable Long i
d) {
    Student student = studentService.getStudentById(id);

    if (student == null) {
        return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body("Student not found");
    }

    return ResponseEntity.ok(student);
}
```

This is better than returning `null`, but it creates repeated logic.

The same check may appear in many methods:

```
getStudentById
updateStudent
deleteStudent
getCourseById
getTeacherById
getBatchById
```

Everywhere we write:

```
if (data == null) {
    return 404;
}
```

This creates duplicate code and makes controllers bulky.

Problem 3: Inconsistent Error Responses

Coder Army

One API may return:

```
{
  "message": "Student not found"
}
```

Another API may return:

```
"Course not found"
```

Another API may return:

```
{
  "error": "Something went wrong",
  "status": 500
}
```

Another API may return Spring Boot's default error response.

This inconsistency makes the frontend harder to build.

A professional backend should return a consistent structure for errors.

---

5. What Happens When an Exception Occurs in Spring Boot?

Think about the normal request flow:

```
Client
  ↓
Tomcat / Embedded Server
  ↓
DispatcherServlet
  ↓
Controller
  ↓
Service
  ↓
Repository
```

```
  ↓
Database
```

Now suppose the service layer throws an exception:

```java
public Student getStudentById(Long id) {
    return studentRepository.findById(id).get();
}
```

If the student does not exist, `findById(id)` returns an empty `Optional`.

Calling `.get()` on an empty `Optional` throws an exception.

The exception then travels backward:

```
Repository / Service exception
  ↓
Controller does not handle it
  ↓
Spring MVC tries to resolve it
  ↓
If no custom handler exists, Spring Boot default error handling takes over
```

Spring Boot does have default error handling, but the default response may not match the meaning of your business case.

---

6. Spring Boot Default Error Handling

Spring Boot provides a default `/error` mapping.

When an unhandled error occurs, Spring Boot internally forwards the request to `/error`.

For browser clients, Spring Boot may show the Whitelabel Error Page.

For API clients such as Postman, it usually returns a JSON error response.

Test Example

Create a test endpoint:

Coder Army

Coder Army

```
@GetMapping("/test/500")
public String test500Error() {
    throw new RuntimeException("Something went wrong inside s
erver");
}
```

Open this URL in the browser:

```
http://localhost:8080/test/500
```

You may see an HTML page similar to:

```
Whitelabel Error Page
This application has no explicit mapping for /error, so you are seeing this as a fallbac
k.
```

In Postman, the response may look like this:

```
{
  "timestamp": "2026-07-05T11:45:06.511Z",
  "status": 500,
  "error": "Internal Server Error",
  "path": "/test/500"
}
```

By default, Spring Boot hides detailed exception messages for safety.

For learning or local testing, you can temporarily add this in `application.properties`:

```
spring.web.error.include-message=always
```

Then the response may include the exception message:

```
{
  "timestamp": "2026-07-05T10:15:30.123+00:00",
```

Coder Army

```
  "status": 500,
  "error": "Internal Server Error",
  "message": "Something went wrong inside server",
  "path": "/test/500"
}
```

> Do not expose internal exception messages in production. They may reveal implementation details.

---

7. Problem with Default Error Response

Default error handling is useful, but it is not always domain-specific.

Suppose the real issue is:

```
Student with id 2 does not exist.
```

But the response says:

```
{
  "status": 500,
  "error": "Internal Server Error"
}
```

This is misleading.

The server did not fail because of an unknown internal crash. The requested student simply does not exist.

The correct response should be:

```
404 Not Found
```

This is one of the main reasons we need custom exception handling.

Coder Army

8. HTTP Status Codes for REST APIs

The response body is for data and human-readable messages.

The status code is for protocol-level meaning.

Frontend code can use status codes like this:

```
if (response.status === 404) {
    showNotFoundMessage();
}
```

HTTP status codes are grouped into categories:

Category	Meaning
`1xx`	Informational
`2xx`	Success
`3xx`	Redirection
`4xx`	Client error
`5xx`	Server error

For a beginner-to-intermediate Spring Boot CRUD API, these status codes are enough:

```
200 OK
201 Created
204 No Content
400 Bad Request
404 Not Found
409 Conflict
500 Internal Server Error
```

---

9. Common Status Codes in CRUD APIs

Situation	Status Code
Data fetched successfully	`200 OK`
Data updated successfully	`200 OK`

Coder Army

Situation	Status Code
New resource created	`201 Created`
Delete successful with no body	`204 No Content`
Invalid request body or validation failure	`400 Bad Request`
Resource does not exist	`404 Not Found`
Duplicate email, username, or roll number	`409 Conflict`
Unexpected backend failure	`500 Internal Server Error`

10. `201 Created`

Use `201 Created` when a new resource is successfully created.

Example:

```
POST /api/students
```

Response:

```
201 Created
```

```
{
  "id": 10,
  "name": "Riya",
  "email": "riya@gmail.com"
}
```

This tells the client:

```
A new student resource has been created successfully.
```

11. `204 No Content`

Coder Army

`204 No Content` means the request was successful, but there is no response body.

This is commonly used for delete APIs.

Example:

```
DELETE /api/students/1
```

Response:

```
204 No Content
```

No body is returned.

For delete operations, both approaches are common:

```
204 No Content without body
200 OK with message/body
```

For clean REST APIs, `204 No Content` is often preferred.

---

12. `409 Conflict`

`409 Conflict` means the request could not be completed because it conflicts with the current state of the resource.

Example:

A student already exists with this email:

```
{
  "id": 1,
  "email": "amit@gmail.com"
}
```

Now the client sends:

Coder Army

```
POST /api/students
```

```
{
  "name": "Another Amit",
  "email": "amit@gmail.com",
  "age": 23
}
```

The JSON is valid.

The email format is valid.

But the email already exists.

So the correct response is:

```
409 Conflict
```

```
{
  "message": "Student with email amit@gmail.com already exist
s"
}
```

Use `409 Conflict` for cases like:

- Duplicate email
- Duplicate username
- Duplicate roll number
- Trying to create something that already exists
- Version conflict during update
- Business rule conflict

---

13. `500 Internal Server Error`

Coder Army

`500 Internal Server Error` means something unexpected failed inside the backend.

Examples:

- Database connection failed
- NullPointerException due to a bug
- Unexpected runtime exception
- Third-party service failure
- File system error
- Server configuration issue

Use `500` for unexpected server-side failures, not for normal business cases like "student not found".

ResponseEntity in Spring Boot

14. Why Use `ResponseEntity`?

In a basic controller, we may write:

```
@GetMapping("/{id}")
public Student getStudentById(@PathVariable Long id) {
    return studentService.getStudentById(id);
}
```

This returns only the body.

Spring decides the status code automatically.

With `ResponseEntity`, we control the response explicitly:

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudentById(@PathVariable L
ong id) {
    Student student = studentService.getStudentById(id);
```

```
    return ResponseEntity.ok(student);
}
```

This means:

```
Response body: student
Status code: 200 OK
```

The important difference is control.

Without `ResponseEntity`:

```
Spring decides the response status.
```

With `ResponseEntity`:

```
We explicitly decide the response status, headers, and body.
```

---

15. Mental Model of ResponseEntity

`ResponseEntity` can represent:

```
Body + Status
Body + Headers + Status
Status only
```

Examples:

```
ResponseEntity.ok(student);
```

```
200 OK + student body
```

Coder Army

```
ResponseEntity.status(HttpStatus.CREATED).body(savedStudent);
```

```
201 Created + saved student body
```

```
ResponseEntity.noContent().build();
```

```
204 No Content + no body
```

---

16. Returning Headers with ResponseEntity

Headers contain extra metadata.

For a create API, the `Location` header is useful.

Example:

```
Location: /api/students/10
```

This tells the client where the newly created student can be found.

```
@PostMapping
public ResponseEntity<Student> createStudent(@RequestBody Stu
dent student) {
    Student savedStudent = studentService.createStudent(stude
nt);

    URI location = URI.create("/api/students/" + savedStuden
t.getId());

    return ResponseEntity
            .created(location)
```

Coder Army

```
                .body(savedStudent);
}
```

Response:

```
201 Created
Location: /api/students/10
```

```
{
  "id": 10,
  "name": "Riya",
  "email": "riya@gmail.com"
}
```

Here:

```
Status  → 201 Created
Header  → Location: /api/students/10
Body    → saved student data
```

`ResponseEntity.created(location)` automatically sets:

```
Status: 201 Created
Header: Location
```

---

17. Common ResponseEntity Methods

`ResponseEntity.ok()`

```
return ResponseEntity.ok(student);
```

Meaning:

Coder Army

Coder Army

```
Status code: 200 OK
Body: student
```

It is a shortcut for:

```
return ResponseEntity
        .status(HttpStatus.OK)
        .body(student);
```

`ResponseEntity.noContent()`

Use this when the operation is successful but there is no body to return.

Common use case:

```
DELETE API
```

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteStudent(@PathVariable Long
id) {
    studentService.deleteStudent(id);

    return ResponseEntity.noContent().build();
}
```

Meaning:

```
Status code: 204 No Content
Body: nothing
```

Why Error Handling Should Not Live Inside Controllers

Coder Army

18. Problem with Handling Errors in Controller

Once beginners learn `ResponseEntity`, they often start writing all error handling inside the controller.

```
@GetMapping("/{id}")
public ResponseEntity<?> getStudentById(@PathVariable Long i
d) {
    Student student = studentService.getStudentById(id);

    if (student == null) {
        return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body("Student not found");
    }

    return ResponseEntity.ok(student);
}
```

This looks fine for one method.

But in a real project, the controller starts doing too many things:

```
1. Receive request
2. Call service
3. Check null
4. Decide error status
5. Decide error message
6. Build error response
7. Build success response
```

A controller should mainly do this:

```
1. Receive request
2. Call service
3. Return response
```

Coder Army

Business checks should stay in the service layer, and common error response logic should stay in one central place.

19. Example: Duplicate Email Check in Controller

A controller may become like this:

```
@PostMapping
public ResponseEntity<?> createStudent(@RequestBody Student s
tudent) {
    boolean emailExists = studentService.existsByEmail(studen
t.getEmail());

    if (emailExists) {
        return ResponseEntity
                .status(HttpStatus.CONFLICT)
                .body("Email already exists");
    }

    Student savedStudent = studentService.createStudent(stude
nt);

    return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(savedStudent);
}
```

Now the controller has too much `if-else` logic.

This same pattern will repeat across many APIs.

That is not clean design.

20. Problems Caused by Repeated Error Logic

1. Repetition

Coder Army

The same error handling logic appears again and again.

2. Less Readable Controllers

Controllers become full of `if-else` blocks.

3. Weak Service Layer Signals

If the service returns `null`, the controller has to guess what happened.

4. Inconsistent Error Responses

Different developers may return different error structures.

5. Poor Separation of Concerns

The controller should not contain repeated business error response logic.

---

21. Better Separation of Responsibility

A cleaner design looks like this:

Layer	Responsibility
Controller	Receive request and return success response
Service	Run business logic and throw meaningful exceptions
Global Exception Handler	Convert exceptions into proper HTTP responses

This gives a clean flow:

```
Request comes
  ↓
Controller calls service
  ↓
Service either returns data or throws exception
  ↓
GlobalExceptionHandler catches exception
  ↓
Proper error response goes back to client
```

Coder Army

Global Exception Handling in Spring Boot

22. What Is Global Exception Handling?

When an API fails, two things happen:

```
1. Some exception occurs inside backend code.
2. That exception must be converted into a proper HTTP response.
```

Global exception handling means:

```
Instead of handling errors separately inside every controller,
we create one central place that catches exceptions and converts them into proper API res
ponses.
```

Clean flow:

```
Client
  ↓
Controller
  ↓
Service
  ↓
Exception thrown
  ↓
GlobalExceptionHandler catches exception
  ↓
HTTP response returned
```

---

23. What Is `@ControllerAdvice`?

`@ControllerAdvice` is a Spring annotation used to define common behavior for multiple controllers.

In simple words:

```
@ControllerAdvice allows one class to provide common support to controllers.
```

Coder Army

That support can include:

- Exception handling
- Model attributes
- Data binding configuration

Example:

```
@ControllerAdvice
public class GlobalExceptionHandler {

}
```

By itself, `@ControllerAdvice` does not handle exceptions.

To actually handle exceptions, we need methods annotated with:

```
@ExceptionHandler
```

So the combination is:

```
@ControllerAdvice + @ExceptionHandler
```

---

24. What Is `@RestControllerAdvice`?

In REST APIs, we usually return JSON responses.

For that, Spring provides:

```
@RestControllerAdvice
```

`@RestControllerAdvice` is a shortcut for:

```
@ControllerAdvice
@ResponseBody
```

That means exception handler methods return response content directly as JSON instead of resolving a view or JSP page.

For REST API projects, prefer:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

}
```

25. What Is `@ExceptionHandler`?

`@ExceptionHandler` is used on a method to handle a specific exception type.

Example:

```
@ExceptionHandler(RuntimeException.class)
public ResponseEntity<String> handleRuntimeException(RuntimeE
xception ex) {
    return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(ex.getMessage());
}
```

Meaning:

```
If RuntimeException occurs, call this method and return its response.
```

The exception class decides which handler method will run.

Spring tries to use the most specific matching exception handler.

For example, if you have handlers for both `ResourceNotFoundException` and `Exception`, then `ResourceNotFoundException` will be handled by its specific handler, not the generic `Exception` handler.

Coder Army

Coder Army

26. Creating a Global Exception Handler Class

Recommended package:

```
in.strikes.dtoDemo.exception
```

Example:

```
package in.strikes.dtoDemo.exception;

import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

}
```

This class becomes the central place for API error handling.

---

27. Handling Generic Exceptions

Generic exception means:

```
Any unexpected error that we did not specifically handle.
```

Example:

```
@ExceptionHandler(Exception.class)
public ResponseEntity<String> handleGenericException(Exception ex) {
    return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Something went wrong. Please try again lat
```

```
er.");
}
```

This generic handler is a safety net.

It should not be used for normal business cases like:

```
Student not found
Duplicate email
Validation failed
```

Those should have specific exception handlers.

The generic handler is for unexpected errors.

Custom Exceptions

28. Why Create Custom Exceptions?

Java already has generic exceptions like:

```
RuntimeException
IllegalArgumentException
NoSuchElementException
```

But these do not clearly describe the business meaning.

For example:

```
RuntimeException
```

does not tell much.

But this:

```
ResourceNotFoundException
```

clearly says:

Coder Army

Coder Army

```
The requested resource does not exist.
```

Custom exceptions make service-layer code more meaningful.

29. Creating `ResourceNotFoundException`

```
package in.strikes.dtoDemo.exception;

public class ResourceNotFoundException extends RuntimeExcepti
on {

    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

Why extend `RuntimeException`?

In Spring Boot service-layer business errors, unchecked exceptions are commonly used.

That means you do not need to write `throws` in every method signature.

For beginner Spring Boot projects, extending `RuntimeException` is simple and practical.

30. Throwing Custom Exception from Service Layer

Instead of returning `null`, the service should clearly say what happened.

Bad approach:

```
public Student getStudentById(Long id) {
    return studentRepository.findById(id).orElse(null);
}
```

Coder Army

Better approach:

```
public Student getStudentById(Long id) {
    return studentRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException(
                    "Student with id " + id + " not found"
            ));
}
```

Now the service behavior is clear:

```
If student exists, return the student.
If student does not exist, throw ResourceNotFoundException.
```

This is much better than returning `null`.

---

31. Clean Controller After Using Custom Exception

Once the service throws a meaningful exception, the controller becomes clean:

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudentById(@PathVariable L
ong id) {
    Student student = studentService.getStudentById(id);

    return ResponseEntity.ok(student);
}
```

There is no `if`.

There is no `null` check.

There is no `404` logic.

The success response stays in the controller.

The error response goes to the global exception handler.

Coder Army

32. Update API Example

Service

```
public Student updateStudent(Long id, Student updatedStudent)
{
    Student existingStudent = studentRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException(
                    "Student with id " + id + " not found"
            ));

    existingStudent.setName(updatedStudent.getName());
    existingStudent.setEmail(updatedStudent.getEmail());
    existingStudent.setAge(updatedStudent.getAge());

    return studentRepository.save(existingStudent);
}
```

Controller

```
@PutMapping("/{id}")
public ResponseEntity<Student> updateStudent(
        @PathVariable Long id,
        @RequestBody Student student
) {
    Student updatedStudent = studentService.updateStudent(id,
student);

    return ResponseEntity.ok(updatedStudent);
}
```

33. Delete API Example

Service

Coder Army

```
public void deleteStudent(Long id) {
    Student existingStudent = studentRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException(
                    "Student with id " + id + " not found"
            ));

    studentRepository.delete(existingStudent);
}
```

Controller

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteStudent(@PathVariable Long
id) {
    studentService.deleteStudent(id);

    return ResponseEntity.noContent().build();
}
```

Flow:

```
If deletion succeeds → controller returns 204 No Content
If student does not exist → service throws ResourceNotFoundException
GlobalExceptionHandler returns 404 Not Found
```

---

34. Handling Custom Exception Globally

Now handle `ResourceNotFoundException` in `GlobalExceptionHandler`.

```
package in.strikes.dtoDemo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandl
```

Coder Army

```
er;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleResourceNotFoundException(
            ResourceNotFoundException ex
    ) {
        return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body(ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGenericException(Exception ex) {
        return ResponseEntity
                .status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body("Something went wrong. Please try again later.");
    }
}
```

This works, but returning plain strings is not the most professional structure.

Next, we improve it using an error response DTO.

Standard Error Response DTO

35. Why Use an Error Response DTO?

An exception is a Java-side problem.

Coder Army

Example:

```
throw new ResourceNotFoundException("Student with id 100 not found");
```

The client does not understand Java exceptions.

The client understands HTTP responses.

So our job is:

```
Java exception
  ↓
GlobalExceptionHandler
  ↓
Standard error response JSON
```

Instead of returning only a string, we can return a structured error response.

36. ErrorResponseDto Structure

```
package in.strikes.dtoDemo.dto;

import java.time.LocalDateTime;

public class ErrorResponseDto {

    private LocalDateTime timestamp;
    private int statusCode;
    private String error;
    private String message;
    private String path;

    public ErrorResponseDto(
            LocalDateTime timestamp,
            int statusCode,
            String error,
```

Coder Army

```
            String message,
            String path
    ) {
        this.timestamp = timestamp;
        this.statusCode = statusCode;
        this.error = error;
        this.message = message;
        this.path = path;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public int getStatusCode() {
        return statusCode;
    }

    public String getError() {
        return error;
    }

    public String getMessage() {
        return message;
    }

    public String getPath() {
        return path;
    }
}
```

Example response:

```
{
  "timestamp": "2026-07-07T10:45:30",
  "statusCode": 404,
```

Coder Army

```
  "error": "Not Found",
  "message": "Student with id 100 not found",
  "path": "/api/students/100"
}
```

37. Meaning of Each Error Field

Field	Meaning
`timestamp`	When the error happened
`statusCode`	HTTP status code returned
`error`	Short HTTP error name
`message`	Specific explanation of the failure
`path`	API endpoint where the error occurred

Why `timestamp` is useful

It helps backend developers match the API error with logs.

Why `statusCode` is useful

Frontend code can use it programmatically.

Example:

```
if (error.statusCode === 404) {
    showNotFoundPage();
}
```

Why `path` is useful

If one page calls multiple APIs, the error response itself tells which endpoint failed.

Example:

```
/api/students/100
/api/courses/5
```

```
/api/teachers/8
```

38. Getting Request Path Using `HttpServletRequest`

In Spring Boot, we can get the current request path using:

```
request.getRequestURI()
```

Example handler:

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponseDto> handleResourceNotFoun
dException(
        ResourceNotFoundException ex,
        HttpServletRequest request
) {
    ErrorResponseDto errorResponse = new ErrorResponseDto(
            LocalDateTime.now(),
            HttpStatus.NOT_FOUND.value(),
            HttpStatus.NOT_FOUND.getReasonPhrase(),
            ex.getMessage(),
            request.getRequestURI()
    );

    return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(errorResponse);
}
```

Required import:

```
import jakarta.servlet.http.HttpServletRequest;
```

Coder Army

Coder Army

Validation Errors

39. Handling Validation Errors Globally

Suppose we have this request DTO:

```
public class CreateStudentRequestDto {

    @NotBlank(message = "Name cannot be blank")
    @Size(min = 2, max = 50, message = "Name must be between
2 and 50 characters")
    private String name;

    @NotBlank(message = "Email cannot be blank")
    @Email(message = "Email must be valid")
    private String email;

    @NotNull(message = "Age is required")
    @Min(value = 18, message = "Student must be at least 18 y
ears old")
    private Integer age;

    // getters and setters
}
```

If the client sends invalid data:

```
{
  "name": "",
  "email": "wrong-email",
  "age": 15
}
```

validation should fail and the API should return a clean `400 Bad Request` response.

Coder Army

40. Validation Flow in Spring Boot

The flow is:

```
Request comes
  ↓
Spring binds JSON to DTO
  ↓
Validation runs because of @Valid
  ↓
Validation fails
  ↓
Spring throws MethodArgumentNotValidException
  ↓
GlobalExceptionHandler catches it
  ↓
Field-level validation response is returned
```

Important point:

Validation runs only when we use `@Valid` in the controller.

Example:

```
@PostMapping
public ResponseEntity<StudentResponseDto> createStudent(
        @Valid @RequestBody CreateStudentRequestDto requestDt
o
) {
    StudentResponseDto savedStudent = studentService.createSt
udent(requestDto);

    return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(savedStudent);
}
```

---

41. What Is `MethodArgumentNotValidException`?

Coder Army

`MethodArgumentNotValidException` occurs when validation fails on a controller method argument.

Most common case:

```
@Valid @RequestBody CreateStudentRequestDto requestDto
```

If the request body violates validation rules, Spring throws `MethodArgumentNotValidException`.

---

42. Field-Level Validation Error Response

A professional validation error response should tell exactly which field failed and why.

Example:

```
{
  "timestamp": "2026-07-07T11:20:30",
  "statusCode": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "path": "/api/students",
  "fieldErrors": {
    "name": "Name cannot be blank",
    "email": "Email must be valid",
    "age": "Student must be at least 18 years old"
  }
}
```

---

43. ValidationErrorResponseDto

```
package in.strikes.dtoDemo.dto;

import java.time.LocalDateTime;
import java.util.Map;
```

Coder Army

```
public class ValidationErrorResponseDto {

    private LocalDateTime timestamp;
    private int statusCode;
    private String error;
    private String message;
    private String path;
    private Map<String, String> fieldErrors;

    public ValidationErrorResponseDto(
            LocalDateTime timestamp,
            int statusCode,
            String error,
            String message,
            String path,
            Map<String, String> fieldErrors
    ) {
        this.timestamp = timestamp;
        this.statusCode = statusCode;
        this.error = error;
        this.message = message;
        this.path = path;
        this.fieldErrors = fieldErrors;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public int getStatusCode() {
        return statusCode;
    }

    public String getError() {
        return error;
```

```
    }

    public String getMessage() {
        return message;
    }

    public String getPath() {
        return path;
    }

    public Map<String, String> getFieldErrors() {
        return fieldErrors;
    }
}
```

44. Extracting Field Errors

Use `getBindingResult().getFieldErrors()`:

```
Map<String, String> fieldErrors = new HashMap<>();

ex.getBindingResult().getFieldErrors().forEach(error ->
        fieldErrors.put(error.getField(), error.getDefaultMes
sage())
);
```

This creates a map like:

```
{
  "name": "Name cannot be blank",
  "email": "Email must be valid",
  "age": "Student must be at least 18 years old"
}
```

Coder Army

Coder Army

45. Handler for Validation Errors

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ValidationErrorResponseDto> handleValid
ationException(
        MethodArgumentNotValidException ex,
        HttpServletRequest request
) {
    Map<String, String> fieldErrors = new HashMap<>();

    ex.getBindingResult().getFieldErrors().forEach(error ->
            fieldErrors.put(error.getField(), error.getDefaul
tMessage())
    );

    ValidationErrorResponseDto errorResponse = new Validation
ErrorResponseDto(
            LocalDateTime.now(),
            HttpStatus.BAD_REQUEST.value(),
            HttpStatus.BAD_REQUEST.getReasonPhrase(),
            "Validation failed",
            request.getRequestURI(),
            fieldErrors
    );

    return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(errorResponse);
}
```

Required imports:

```
import org.springframework.web.bind.MethodArgumentNotValidExc
eption;
```

```java
import java.util.HashMap;
import java.util.Map;
```

Coder Army

Bad Request Scenarios

46. What Is `400 Bad Request`?

`400 Bad Request` means the client sent something wrong.

Validation failure is one type of bad request.

Other examples:

- Invalid JSON
- Wrong data type in JSON
- Malformed request body
- Invalid request format
- Invalid path variable type
- Missing required query parameter

47. Invalid JSON or Wrong Data Type

Suppose the DTO expects:

```java
private Integer age;
```

But the client sends:

```json
{
  "name": "Amit",
  "email": "amit@gmail.com",
  "age": "twenty"
}
```

Coder Army

Spring tries to convert `"twenty"` into `Integer`.

It cannot, so the API should return:

```
400 Bad Request
```

For unreadable or malformed request bodies, Spring throws:

```
HttpMessageNotReadableException
```

48. Handler for Invalid Request Body

```
@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ErrorResponseDto> handleHttpMessageNotR
eadableException(
        HttpMessageNotReadableException ex,
        HttpServletRequest request
) {
    ErrorResponseDto errorResponse = new ErrorResponseDto(
            LocalDateTime.now(),
            HttpStatus.BAD_REQUEST.value(),
            HttpStatus.BAD_REQUEST.getReasonPhrase(),
            "Invalid request body. Please check your JSON for
mat and data types.",
            request.getRequestURI()
    );

    return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(errorResponse);
}
```

Required import:

Coder Army

```
import org.springframework.http.converter.HttpMessageNotReadableException;
```

49. Invalid Path Variable Type

Suppose the controller expects:

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
    Student student = studentService.getStudentById(id);
    return ResponseEntity.ok(student);
}
```

But the client calls:

```
GET /api/students/abc
```

Spring cannot convert `"abc"` into `Long`.

In this case, Spring may throw:

```
MethodArgumentTypeMismatchException
```

A clean API should return `400 Bad Request`.

Example handler:

```
@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<ErrorResponseDto> handleMethodArgumentTypeMismatchException(
        MethodArgumentTypeMismatchException ex,
        HttpServletRequest request
) {
    ErrorResponseDto errorResponse = new ErrorResponseDto(
```

Coder Army

```
            LocalDateTime.now(),
            HttpStatus.BAD_REQUEST.value(),
            HttpStatus.BAD_REQUEST.getReasonPhrase(),
            "Invalid value for parameter '" + ex.getName() +
"'. Please check the required data type.",
            request.getRequestURI()
    );

    return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(errorResponse);
}
```

Required import:

```
import org.springframework.web.method.annotation.MethodArgume
ntTypeMismatchException;
```

Duplicate Resource / Conflict Handling

50. Difference Between `400 Bad Request` and `409 Conflict`

First principle:

```
400 means the request itself is invalid.
409 means the request may be valid, but it conflicts with existing server data.
```

Example:

```
Invalid email format       → 400 Bad Request
Email already exists       → 409 Conflict
Age sent as "twenty"       → 400 Bad Request
Roll number already exists → 409 Conflict
```

Coder Army

51. Creating `DuplicateResourceException`

```
package in.strikes.dtoDemo.exception;

public class DuplicateResourceException extends RuntimeExcept
ion {

    public DuplicateResourceException(String message) {
        super(message);
    }
}
```

52. Throwing DuplicateResourceException from Service

```
public StudentResponseDto createStudent(CreateStudentRequestD
to requestDto) {
    boolean emailExists = studentRepository.existsByEmail(req
uestDto.getEmail());

    if (emailExists) {
        throw new DuplicateResourceException(
                "Student with email " + requestDto.getEmail()
+ " already exists"
        );
    }

    Student student = new Student();
    student.setName(requestDto.getName());
    student.setEmail(requestDto.getEmail());
    student.setAge(requestDto.getAge());

    Student savedStudent = studentRepository.save(student);
```

```
    return mapToResponseDto(savedStudent);
}
```

Coder Army

53. Handler for Duplicate Resource

```
@ExceptionHandler(DuplicateResourceException.class)
public ResponseEntity<ErrorResponseDto> handleDuplicateResour
ceException(
        DuplicateResourceException ex,
        HttpServletRequest request
) {
    ErrorResponseDto errorResponse = new ErrorResponseDto(
            LocalDateTime.now(),
            HttpStatus.CONFLICT.value(),
            HttpStatus.CONFLICT.getReasonPhrase(),
            ex.getMessage(),
            request.getRequestURI()
    );

    return ResponseEntity
            .status(HttpStatus.CONFLICT)
            .body(errorResponse);
}
```

Final GlobalExceptionHandler Example

54. Complete Handler Class

```
package in.strikes.dtoDemo.exception;

import in.strikes.dtoDemo.dto.ErrorResponseDto;
```

Coder Army

```
import in.strikes.dtoDemo.dto.ValidationErrorResponseDto;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReada
bleException;
import org.springframework.web.bind.MethodArgumentNotValidExc
eption;
import org.springframework.web.bind.annotation.ExceptionHandl
er;
import org.springframework.web.bind.annotation.RestController
Advice;
import org.springframework.web.method.annotation.MethodArgume
ntTypeMismatchException;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponseDto> handleResourceNot
FoundException(
            ResourceNotFoundException ex,
            HttpServletRequest request
    ) {
        ErrorResponseDto errorResponse = new ErrorResponseDto
(
                LocalDateTime.now(),
                HttpStatus.NOT_FOUND.value(),
                HttpStatus.NOT_FOUND.getReasonPhrase(),
                ex.getMessage(),
                request.getRequestURI()
        );
```

Coder Army

```
        return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body(errorResponse);
    }

    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ErrorResponseDto> handleDuplicateRe
sourceException(
            DuplicateResourceException ex,
            HttpServletRequest request
    ) {
        ErrorResponseDto errorResponse = new ErrorResponseDto
(
                LocalDateTime.now(),
                HttpStatus.CONFLICT.value(),
                HttpStatus.CONFLICT.getReasonPhrase(),
                ex.getMessage(),
                request.getRequestURI()
        );

        return ResponseEntity
                .status(HttpStatus.CONFLICT)
                .body(errorResponse);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ValidationErrorResponseDto> handleV
alidationException(
            MethodArgumentNotValidException ex,
            HttpServletRequest request
    ) {
        Map<String, String> fieldErrors = new HashMap<>();

        ex.getBindingResult().getFieldErrors().forEach(error
->
```

```
                fieldErrors.put(error.getField(), error.getDe
faultMessage())
        );

        ValidationErrorResponseDto errorResponse = new Valida
tionErrorResponseDto(
                LocalDateTime.now(),
                HttpStatus.BAD_REQUEST.value(),
                HttpStatus.BAD_REQUEST.getReasonPhrase(),
                "Validation failed",
                request.getRequestURI(),
                fieldErrors
        );

        return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .body(errorResponse);
    }

    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<ErrorResponseDto> handleHttpMessage
NotReadableException(
            HttpMessageNotReadableException ex,
            HttpServletRequest request
    ) {
        ErrorResponseDto errorResponse = new ErrorResponseDto
(
                LocalDateTime.now(),
                HttpStatus.BAD_REQUEST.value(),
                HttpStatus.BAD_REQUEST.getReasonPhrase(),
                "Invalid request body. Please check your JSON
format and data types.",
                request.getRequestURI()
        );

        return ResponseEntity
```

Coder Army

```
                .status(HttpStatus.BAD_REQUEST)
                .body(errorResponse);
    }

    @ExceptionHandler(MethodArgumentTypeMismatchException.cla
ss)
    public ResponseEntity<ErrorResponseDto> handleMethodArgum
entTypeMismatchException(
            MethodArgumentTypeMismatchException ex,
            HttpServletRequest request
    ) {
        ErrorResponseDto errorResponse = new ErrorResponseDto
(
                LocalDateTime.now(),
                HttpStatus.BAD_REQUEST.value(),
                HttpStatus.BAD_REQUEST.getReasonPhrase(),
                "Invalid value for parameter '" + ex.getName
() + "'. Please check the required data type.",
                request.getRequestURI()
        );

        return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .body(errorResponse);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponseDto> handleGenericExce
ption(
            Exception ex,
            HttpServletRequest request
    ) {
        ErrorResponseDto errorResponse = new ErrorResponseDto
(
                LocalDateTime.now(),
                HttpStatus.INTERNAL_SERVER_ERROR.value(),
```

Coder Army

```
                HttpStatus.INTERNAL_SERVER_ERROR.getReasonPhr
ase(),
                "Something went wrong. Please try again late
r.",
                request.getRequestURI()
        );

        return ResponseEntity
                .status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body(errorResponse);
    }
}
```

Coder Army

Final Mental Model

55. Categories of Exceptions

In this project, we handle two major categories:

1. Business Exceptions

These represent meaningful business problems.

Exception	Meaning	Status Code
`ResourceNotFoundException`	Requested resource does not exist	`404 Not Found`
`DuplicateResourceException`	Resource already exists or conflicts with data	`409 Conflict`

2. Request/Input Exceptions

These happen when the client sends incorrect data.

Exception	Meaning	Status Code
`MethodArgumentNotValidException`	Validation failed on DTO	`400 Bad Request`

Coder Army

Exception	Meaning	Status Code
`HttpMessageNotReadableException`	Invalid JSON or wrong data type in request body	`400 Bad Request`
`MethodArgumentTypeMismatchException`	Wrong type in path variable or request parameter	`400 Bad Request`

3. Unexpected Exceptions

These are fallback server-side errors.

Exception	Meaning	Status Code
`Exception`	Any unhandled unexpected error	`500 Internal Server Error`

56. Clean API Flow After Global Exception Handling

```
Client sends request
  ↓
Controller receives request
  ↓
Controller calls service
  ↓
Service performs business logic
  ↓
If success:
    Controller returns success response
  ↓
If failure:
    Service throws meaningful exception
  ↓
GlobalExceptionHandler catches exception
  ↓
ErrorResponseDto / ValidationErrorResponseDto is returned
```

57. Why This Makes APIs Professional

Global exception handling improves the API because:

- Controllers stay clean.
- Repeated error logic is removed.

Coder Army

- Service layer communicates failures clearly.
- Frontend receives consistent error responses.
- Status codes match the real meaning of the failure.
- Debugging becomes easier because error responses contain timestamp and path.
- Unexpected errors are handled safely through a generic fallback.

---

58. Best Practices

Do not return `null` for missing resources

Avoid this:

```
return studentRepository.findById(id).orElse(null);
```

Prefer this:

```
return studentRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException(
                "Student with id " + id + " not found"
        ));
```

Keep success logic in controller

Controller should mainly handle successful responses.

```
return ResponseEntity.ok(student);
```

Keep business checks in service

Examples:

- Student exists or not
- Email already exists

Coder Army

- Roll number already exists
- User is allowed to perform action or not

Keep error response logic in GlobalExceptionHandler

The global handler should decide:

- Status code
- Error message
- Error response structure

Use specific handlers before relying on generic fallback

Specific handlers give meaningful responses.

Generic handler should only be a final safety net.

Do not expose internal exception messages in production

Avoid sending raw exception messages for unexpected errors.

Bad:

```
.body(ex.getMessage())
```

Better:

```
.body("Something went wrong. Please try again later.")
```

This prevents leaking internal implementation details.

Final Summary

A professional Spring Boot API should clearly communicate both success and failure.

For success responses, use `ResponseEntity` to control status codes, headers, and body.

For failure responses, avoid writing repeated `if-else` logic inside controllers.

The cleaner approach is:

```
Service throws meaningful custom exceptions.
GlobalExceptionHandler catches them.
API returns consistent HTTP error responses.
```

This makes the backend cleaner, easier to maintain, and easier for frontend developers to consume.

Coder Army

Coder Army

Spring Boot Configurations, `application.properties`, YAML, and Profiles

1. Why Configuration Is Needed

A real-world application is not just code. It is a combination of two things:

```
Application = Code + Configuration
```

Code contains the actual business logic.

Examples:

```
Create student
Fetch student
Update student
Delete student
Connect to database
Send email
Run server
```

But to run this code properly, the application also needs some values.

Examples:

```
Which port should the application run on?
Which database should it connect to?
What is the database username?
What is the database password?
What should be the application name?
Which environment is currently active?
What message should be shown to the user?
```

These values are called **configuration values**.

Coder Army

In real projects, the same Spring Boot application may run in different environments:

```
Local machine
Testing server
Staging server
Production server
```

The Java code usually remains the same, but configuration values change from one environment to another.

---

2. The Problem With Hardcoding Values

Hardcoding means writing fixed values directly inside Java classes.

Example:

```java
String dbUrl = "jdbc:mysql://localhost:3306/student_db";
String username = "root";
String password = "root";
```

At first, this may look simple. But it creates serious problems in real-world applications.

Problem 1: Code Becomes Difficult to Change

Suppose the database URL changes.

If the value is hardcoded inside Java code, we must open the Java file, update the value, rebuild the application, and redeploy it.

This is unnecessary because database URL is not business logic. It is an environment-specific configuration value.

Problem 2: Same Code Cannot Easily Run in Multiple Environments

Your local machine may use:

Coder Army

```
username = root
password = root
```

Production may use:

```
username = prod_user
password = strong_secure_password
```

If these values are hardcoded, the same code becomes tightly attached to one environment.

A professional application should follow this idea:

```
Same code
Different configuration
```

Problem 3: Sensitive Information Can Leak

If you write passwords directly inside Java code:

```java
String password = "admin123";
```

and then push the code to GitHub, the password may get exposed.

This is why sensitive values like passwords, API keys, and tokens should not be hardcoded inside Java classes.

---

3. What Is `application.properties`?

Spring Boot provides a special configuration file:

```
application.properties
```

This file is used to write application configuration values in one central place.

Example:

Coder Army

```
server.port=8081
spring.application.name=student-management-app
app.welcome.message=Welcome to Student Management App
```

Instead of writing these values inside Java code, we write them inside `application.properties`.

---

4. Why Is It Named `application.properties`?

Spring Boot automatically looks for a file named:

```
application.properties
```

during application startup.

When the application starts, Spring Boot checks whether this file exists. If it exists, Spring Boot loads the values from it.

So `application.properties` becomes the central place for application configuration.

---

5. Basic Key-Value Format in `application.properties`

The format is simple:

```
key=value
```

Example:

```
server.port=8081
spring.application.name=student-management-app
app.owner=Coder Army
```

Here:

```
server.port                → key
8081                       → value
```

Coder Army

```
spring.application.name      → key
student-management-app       → value
```

6. Reading Configuration Values Using `@Value`

Spring provides the `@Value` annotation to inject configuration values into Java variables.

Example:

```
app.welcome.message=Welcome to Student Management App
```

Java code:

```java
@RestController
public class WelcomeController {

    @Value("${app.welcome.message}")
    private String welcomeMessage;

    @GetMapping("/welcome")
    public String getWelcomeMessage() {
        return welcomeMessage;
    }
}
```

Here:

```java
@Value("${app.welcome.message}")
```

means:

```
Read the value of app.welcome.message from the configuration file
and inject it into this variable.
```

Coder Army

This is called **property injection**.

7. Limitations of `application.properties`

For small projects, `application.properties` is simple and readable.

Example:

```
server.port=8081
spring.application.name=student-management-app
app.welcome.message=Welcome to Student Management App
```

But as the project grows, the file can become large and repetitive.

7.1 Flat Structure

`application.properties` stores values in a flat key-value format.

Example:

```
app.name=Student Management App
app.version=1.0
app.owner=Coder Army
app.support.email=support@studentapp.com
app.support.phone=9999999999
```

The values are related, but the file does not visually show a proper hierarchy.

7.2 Repeated Prefixes

For grouped configuration, prefixes are repeated again and again.

Example:

```
notification.email.enabled=true
notification.email.subject=Welcome
notification.email.from=no-reply@studentapp.com
notification.email.retry.count=3

notification.sms.enabled=false
```

```
notification.sms.provider=twilio
notification.sms.retry.count=2
```

This works, but it becomes less readable when configuration grows.

7.3 Lists Are Not Very Clean

Example:

```
app.supported-cities=Delhi,Mumbai,Bangalore,Pune
```

This is fine for a small list, but for bigger lists or nested objects, `.properties` becomes difficult to read.

8. Introduction to YAML

YAML solves the readability problem of large configuration files.

The problem is not that `application.properties` is wrong. The problem is that `.properties` becomes flat and repetitive when configuration grows.

Example in `.properties`:

```
app.name=Student Management App
app.version=1.0
app.owner=Coder Army
app.support.email=support@studentapp.com
app.support.phone=9999999999
```

Actually, this data has a natural hierarchy:

```
app
 ├── name
 ├── version
 ├── owner
 └── support
      ├── email
      └── phone
```

Coder Army

Coder Army

YAML allows us to write this in a structured way.

Same configuration in YAML:

```yaml
app:
name: Student Management App
version:1.0
owner: Coder Army
support:
email: support@studentapp.com
phone:9999999999
```

YAML converts flat configuration into structured configuration.

```
.properties → flat key-value format
.yml        → structured, nested format
```

---

9. How Spring Boot Understands YAML

YAML looks nested, but internally Spring Boot understands it as normal property keys.

Example YAML:

```yaml
server:
port:8081
```

Spring Boot understands it like:

```
server.port=8081
```

Another example:

```yaml
spring:
application:
```

Coder Army

```yaml
name: student-management-app
```

Spring Boot understands it like:

```
spring.application.name=student-management-app
```

So YAML is not a completely different configuration system. It is a cleaner way to write the same configuration.

---

10. `application.properties` vs `application.yml`

Spring Boot supports both formats:

```
application.properties
application.yml
application.yaml
```

But for a project, it is better to stick to one format.

Avoid keeping both `application.properties` and `application.yml` for the same purpose in the same location, because it can create confusion about which value is actually being used.

A clean project should usually follow one consistent style.

---

11. YAML Syntax and Indentation Rules

YAML is simple, but it is strict about indentation.

In `.properties`:

```
server.port=8081
```

In YAML:

```yaml
server:
```

Coder Army

```
port:8081
```

Here:

```
server → parent
port   → child
```

The spaces before `port` show that `port` belongs inside `server`.

Rule 1: Use Colon `:`

Every key is followed by a colon.

```
app:
name: Student Management App
```

Rule 2: Put a Space After the Colon

Correct:

```
server:
port:8081
```

Avoid:

```
server:
  port:8081
```

Use:

```
key: value
```

not:

```
key:value
```

Coder Army

Rule 3: Indentation Defines Relationship

Correct:

```yaml
app:
support:
email: support@studentapp.com
```

This means:

```
app.support.email=support@studentapp.com
```

Wrong:

```yaml
app:
support:
email: support@studentapp.com
```

This destroys the hierarchy.

YAML does not use dots like `.properties`. YAML uses spaces to show structure.

Rule 4: Use Spaces, Not Tabs

YAML is indentation-sensitive.

Use spaces only. Usually, two spaces are used.

Correct:

```yaml
spring:
application:
name: student-management-app
```

Avoid tabs.

Rule 5: Same-Level Keys Must Have Same Indentation

Coder Army

Correct:

```yaml
app:
name: Student Management App
version:1.0
owner: Coder Army
```

Here, `name`, `version`, and `owner` are at the same level.

Rule 6: Comments Use `#`

```yaml
# Server configuration
server:
port:8081
```

Comments are useful for dividing configuration sections.

Rule 7: Strings Can Be Written With or Without Quotes

Both are usually valid:

```yaml
app:
name: Student Management App
```

and:

```yaml
app:
name:"Student Management App"
```

Quotes are useful when the value contains special characters, placeholders, or symbols.

---

12. Converting `application.properties` to `application.yml`

Example `application.properties`:

Coder Army

```
server.port=8081

spring.application.name=student-management-app

app.name=Student Management App
app.version=1.0
app.owner=Coder Army

app.welcome.message=Welcome to Student Management App

app.support.email=support@studentapp.com
app.support.phone=9999999999
```

Equivalent `application.yml`:

```yaml
server:
port:8081

spring:
application:
name: student-management-app

app:
name: Student Management App
version:1.0
owner: Coder Army
welcome:
message: Welcome to Student Management App
support:
email: support@studentapp.com
phone:9999999999
```

---

13. Real-World Example: Database Configuration

In `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_db
spring.datasource.username=root
```

```
spring.datasource.password=root
```

In `application.yml`:

```yaml
spring:
datasource:
url: jdbc:mysql://localhost:3306/student_db
username: root
password: root
```

Now all database-related configuration is grouped under:

```
spring.datasource
```

---

14. Real-World Example: JPA Configuration

In `application.properties`:

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

In `application.yml`:

```yaml
spring:
jpa:
hibernate:
ddl-auto: update
show-sql:true
```

This is more readable because related configuration is grouped together.

---

15. Lists and Multiple Values in YAML

Suppose one key has multiple values.

Coder Army

Coder Army

Example:

```
Supported cities:
Delhi
Mumbai
Bangalore
Pune
```

In `.properties`, we may write:

```
app.supported-cities=Delhi,Mumbai,Bangalore,Pune
```

This works, but it looks like one long string.

In YAML, we can write it as a proper list:

```yaml
app:
supported-cities:
- Delhi
- Mumbai
- Bangalore
- Pune
```

The `-` symbol represents one item in the list.

This:

```yaml
supported-cities:
- Delhi
- Mumbai
- Bangalore
```

means:

```
supported-cities has multiple values:
1. Delhi
```

```
2. Mumbai
3. Bangalore
```

Coder Army

16. List of Objects in YAML

YAML can also represent a list of objects.

Example:

```yaml
app:
team-members:
-name: Rohit
role: Backend Mentor
-name: Mohit
role: DSA Mentor
-name: Aditya
role: Spring Mentor
```

This type of structure becomes difficult to manage in `.properties`.

YAML makes it much more readable.

For complex nested configuration, Spring Boot also provides:

```
@ConfigurationProperties
```

`@ConfigurationProperties` is better when we need to bind multiple related values, lists, or nested objects into a Java class.

17. Reading YAML Values Using Dot Notation

Even though YAML is nested, we still read values using dot notation.

YAML:

```yaml
app:
name: Student Management App
```

Coder Army

Java:

```
@Value("${app.name}")
private String appName;
```

Spring internally treats it like:

```
app.name=Student Management App
```

---

18. Reading List Values

For beginner-level teaching, list values can be shown in two simple ways.

Way 1: Reading Individual List Items

`application.yml`:

```
app:
supported-cities:
- Delhi
- Mumbai
- Bangalore
```

Java:

```
@Value("${app.supported-cities[0]}")
private String firstCity;

@Value("${app.supported-cities[1]}")
private String secondCity;
```

This reads:

```
Delhi
```

```
Mumbai
```

This shows that YAML lists are index-based internally.

Way 2: Reading Comma-Separated Values as a List

For `@Value`, this style is easier:

```yaml
app:
supported-cities: Delhi,Mumbai,Bangalore,Pune
```

Java:

```java
@Value("#{'${app.supported-cities}'.split(',')}")
private List<String> supportedCities;
```

This converts the comma-separated string into a Java `List<String>`.

For professional projects, avoid using too many `@Value` annotations for complex configuration. Prefer `@ConfigurationProperties`.

19. Why Spring Profiles Are Needed

Till now, we solved one problem:

```
Configuration values should not be hardcoded inside Java code.
```

So we moved values into:

```
application.properties
application.yml
```

Now the next real-world problem is:

```
The same application needs different values in different environments.
```

Coder Army

That is where **Spring Profiles** come in.

Spring Boot allows the same application code to work with different configuration sources such as:

```
Properties files
YAML files
Environment variables
Command-line arguments
```

---

20. Problem With Multiple Environments

In real projects, a Spring Boot application is not used only on a developer laptop.

It may run in:

```
Development environment
Testing environment
Staging environment
Production environment
```

The Java logic usually remains the same:

```
Create student
Fetch student
Update student
Delete student
Validate request
Handle exception
Connect to database
```

But the surrounding values change.

Example:

```
Local machine:
Database URL = jdbc:mysql://localhost:3306/student_db
```

CoderArmy

Coder Army

```
Production server:
Database URL = jdbc:mysql://prod-db-server:3306/student_db
```

The application logic is the same, but the database location is different.

This is the main idea:

```
Same code
Different configuration
```

---

21. Example Without Profiles

Suppose this is your local configuration:

```yaml
server:
port:8080

spring:
datasource:
url: jdbc:mysql://localhost:3306/student_db
username: root
password: root
```

For production, you may need:

```yaml
server:
port:8080

spring:
datasource:
url: jdbc:mysql://prod-server:3306/student_db
username: prod_user
password: prod_password
```

A bad approach is:

Coder Army

```
Comment local configuration
Uncomment production configuration
Run application
Again comment production configuration
Uncomment local configuration
```

This is risky.

One wrong uncommented line can connect your local application to the production database.

So we need a clean mechanism to separate environment-specific configuration.

---

22. Introduction to Spring Profiles

A **profile** is a named environment or mode in which the application runs.

Examples:

```
dev
test
staging
prod
```

Simple meaning:

```
Profile = Which environment configuration should Spring Boot use right now?
```

Examples:

```
If dev profile is active:
Use development configuration.

If prod profile is active:
Use production configuration.
```

Instead of changing code again and again, we tell Spring Boot:

Coder Army

```
Run this application with the dev profile.
```

or:

```
Run this application with the prod profile.
```

Then Spring Boot loads the matching configuration.

---

23. Creating Profile-Specific Files

Spring Boot supports profile-specific configuration files using this naming pattern:

```
application-{profile}.properties
```

Examples:

```
application-dev.properties
application-test.properties
application-prod.properties
```

Project structure:

```
src/main/resources
 ├── application.properties
 ├── application-dev.properties
 ├── application-test.properties
 └── application-prod.properties
```

---

24. Meaning of Each File

`application.properties`

This is the common/default file.

Put values here that are common for all environments.

Coder Army

```
spring.application.name=student-management-app
```

`application-dev.properties`

This is for development.

```
server.port=8081
app.message=Application is running in DEV mode
```

`application-test.properties`

This is for testing.

```
server.port=8082
app.message=Application is running in TEST mode
```

`application-prod.properties`

This is for production.

```
server.port=8080
app.message=Application is running in PROD mode
```

25. Common File + Profile-Specific File

Spring Boot does not simply ignore the common file.

If the `dev` profile is active, Spring Boot loads:

```
application.properties
        +
application-dev.properties
```

Example:

Coder Army

`application.properties`

```
spring.application.name=student-management-app
```

`application-dev.properties`

```
server.port=8081
```

Final configuration:

```
spring.application.name=student-management-app
server.port=8081
```

So the common file provides shared values, and the profile-specific file provides environment-specific values.

---

26. What If the Same Key Exists in Both Files?

Example:

`application.properties`

```
server.port=8080
app.message=Common message
```

`application-dev.properties`

```
server.port=8081
app.message=DEV message
```

If the `dev` profile is active, final values become:

```
server.port=8081
app.message=DEV message
```

Profile-specific configuration overrides common configuration.

Coder Army

This is an important point for students:

```
Common file gives default values.
Profile-specific file can override those values.
```

---

27. Profile-Specific YAML Files

The same concept works with YAML.

Instead of:

```
application-dev.properties
application-prod.properties
```

we can use:

```
application-dev.yml
application-prod.yml
```

or:

```
application-dev.yaml
application-prod.yaml
```

Project structure:

```
src/main/resources
 ├── application.yml
 ├── application-dev.yml
 ├── application-test.yml
 └── application-prod.yml
```

---

28. Common YAML File: `application.yml`

Coder Army

```yaml
spring:
application:
name: student-management-app
```

This value is common for all profiles.

29. Dev YAML File: `application-dev.yml`

```yaml
server:
port:8081

app:
message: Application is running in DEV mode

spring:
datasource:
url: jdbc:mysql://localhost:3306/student_dev
username: root
password: root
```

30. Test YAML File: `application-test.yml`

```yaml
server:
port:8082

app:
message: Application is running in TEST mode

spring:
datasource:
url: jdbc:mysql://localhost:3306/student_test
```

```
username: test_user
password: test_password
```

Coder Army

31. Prod YAML File: `application-prod.yml`

```
server:
port:8080

app:
message: Application is running in PROD mode

spring:
datasource:
url: jdbc:mysql://prod-server:3306/student_prod
username: prod_user
password: prod_password
```

32. Activating a Profile

Creating profile-specific files is not enough.

We also need to tell Spring Boot which profile should be active.

Example:

```
Use dev profile
Use prod profile
Use test profile
```

There are multiple ways to activate a profile.

33. Activating Profile Using `application.properties`

Inside `application.properties`:

Coder Army

```
spring.application.name=student-management-app
spring.profiles.active=dev
```

Project structure:

```
src/main/resources
 ├── application.properties
 ├── application-dev.properties
 └── application-prod.properties
```

`application-dev.properties`:

```
server.port=8081
app.message=Application is running in DEV mode
```

`application-prod.properties`:

```
server.port=8080
app.message=Application is running in PROD mode
```

When the application starts, Spring Boot sees:

```
spring.profiles.active=dev
```

So it loads the dev-specific configuration.

---

34. Controller Demo for Profile Message

```java
@RestController
public class ProfileDemoController {

    @Value("${app.message}")
    private String message;
```

Coder Army

```
    @GetMapping("/profile-message")
    public String getProfileMessage() {
        return message;
    }
}
```

If `dev` profile is active, response:

```
Application is running in DEV mode
```

If `prod` profile is active, response:

```
Application is running in PROD mode
```

Same controller. Same Java code. Different output.

The output changes because the active profile changes.

---

35. Important Production Note

For beginner demos, activating the profile inside `application.properties` or `application.yml` is okay.

But in real production deployment, we usually avoid hardcoding this inside the packaged application:

```
spring:
profiles:
active: prod
```

Reason:

```
The same JAR should be able to run in different environments.
```

Better production-style approach:

Coder Army

```
Pass the active profile from outside.
```

Examples:

```
Command line
Environment variable
Docker configuration
Kubernetes configuration
Server configuration
CI/CD pipeline
```

This keeps the application artifact reusable.

---

36. Activating Profile From Command Line

When we build a Spring Boot JAR, we can run it like this:

```
java -jar student-management-app.jar --spring.profiles.active
=dev
```

For production:

```
java -jar student-management-app.jar --spring.profiles.active
=prod
```

This method is powerful because the same JAR can run differently.

```
Same JAR + dev profile  = Development behavior
Same JAR + prod profile = Production behavior
```

This is professional because:

```
Code is same.
Build is same.
Environment decides configuration.
```

Coder Army

37. Activating Profile While Running With Maven

During development, if you are running the application with Maven:

```
mvn spring-boot:run -Dspring-boot.run.profiles=dev
```

This starts the application with the `dev` profile active.

---

38. Activating Profile Using Environment Variables

Another professional way is using environment variables.

For Spring profiles, the environment variable name is:

```
SPRING_PROFILES_ACTIVE
```

macOS/Linux:

```
export SPRING_PROFILES_ACTIVE=dev
java -jar student-management-app.jar
```

Windows PowerShell:

```
$env:SPRING_PROFILES_ACTIVE="dev"
java -jar student-management-app.jar
```

---

39. Why Environment Variables Are Used

In production, values often come from the server or deployment platform.

Examples:

```
Docker container environment variable
Kubernetes environment variable
Cloud platform environment variable
CI/CD pipeline variable
Server-level configuration
```

Coder Army

Instead of putting this inside the project:

```yaml
spring:
profiles:
active: prod
```

production can provide:

```
SPRING_PROFILES_ACTIVE=prod
```

This is cleaner and safer because the environment decides which profile should run.

---

40. Practical Demo: Different Server Ports for Different Profiles

Project structure:

```
src/main/resources
 ├── application.yml
 ├── application-dev.yml
 └── application-prod.yml
```

`application.yml`:

```yaml
spring:
application:
name: student-management-app
```

`application-dev.yml`:

```yaml
server:
port:8081
```

Coder Army

```
app:
message: Running in DEV profile
```

`application-prod.yml`:

```
server:
port:9090

app:
message: Running in PROD profile
```

Run with dev:

```
java -jar student-management-app.jar --spring.profiles.active
=dev
```

Application runs on:

```
http://localhost:8081
```

Run with prod:

```
java -jar student-management-app.jar --spring.profiles.active
=prod
```

Application runs on:

```
http://localhost:9090
```

Same code. Same JAR. Different ports.

This is a simple demo that helps students clearly see the use of profiles.

Coder Army

41. Practical Demo: Different Custom Messages for Different Profiles

`application-dev.yml`:

```yaml
app:
message: Hello from DEV environment
```

`application-prod.yml`:

```yaml
app:
message: Hello from PROD environment
```

Controller:

```java
@RestController
@RequestMapping("/api/profile")
public class ProfileController {

    @Value("${app.message}")
    private String message;

    @GetMapping("/message")
    public String getMessage() {
        return message;
    }
}
```

Run with dev:

```
java -jar student-management-app.jar --spring.profiles.active
=dev
```

Hit:

Coder Army

```
GET /api/profile/message
```

Response:

```
Hello from DEV environment
```

Run with prod:

```
java -jar student-management-app.jar --spring.profiles.active
=prod
```

Response:

```
Hello from PROD environment
```

Important observation:

```
The controller did not change.
The endpoint did not change.
Only the active profile changed.
```

---

42. `@Profile` Annotation and Conditional Beans

Till now, profiles were mainly used for changing configuration values.

Examples:

```
dev profile  → different port, different message, different database
prod profile → different port, different message, different database
```

Now we move one step deeper.

Sometimes we do not only want different values. Sometimes we want different Spring beans or classes to be created based on the active profile.

That is where the `@Profile` annotation is used.

43. What Problem Does `@Profile` Solve?

Suppose we have a notification feature.

In development, we do not want to send real emails.

In production, we want to send real emails.

So we may create two implementations:

```
DevNotificationService  → fake notification or console log
ProdNotificationService → real notification or email service
```

Both classes can implement the same interface:

```
public interface NotificationService {
    void sendNotification(String message);
}
```

Development implementation:

```
@Service
public class DevNotificationService implements NotificationSe
rvice {

    @Override
    public void sendNotification(String message) {
        System.out.println("DEV MODE: Notification not actual
ly sent. Message: " + message);
    }
}
```

Production implementation:

Coder Army

CoderArmy

```
@Service
public class ProdNotificationService implements NotificationS
ervice {

    @Override
    public void sendNotification(String message) {
        System.out.println("PROD MODE: Real notification sen
t. Message: " + message);
    }
}
```

Now both classes are marked as `@Service`.

So Spring creates both beans.

If we inject the interface:

```
@Autowired
private NotificationService notificationService;
```

Spring may get confused:

```
Which NotificationService bean should be injected?
DevNotificationService?
ProdNotificationService?
```

This can cause an error because multiple beans of the same type are available.

We need a way to say:

```
Create DevNotificationService only when dev profile is active.
Create ProdNotificationService only when prod profile is active.
```

That is exactly what `@Profile` does.

---

44. Basic Syntax of `@Profile`

Coder Army

```
@Profile("dev")
```

Meaning:

```
Create this bean only when dev profile is active.
```

Example:

```
@Service
@Profile("dev")
public class DevNotificationService implements NotificationSe
rvice {

    @Override
    public void sendNotification(String message) {
        System.out.println("DEV MODE: Notification not actual
ly sent. Message: " + message);
    }
}
```

Production version:

```
@Service
@Profile("prod")
public class ProdNotificationService implements NotificationS
ervice {

    @Override
    public void sendNotification(String message) {
        System.out.println("PROD MODE: Real notification sen
t. Message: " + message);
    }
}
```

Coder Army

Now run with dev:

```
java -jar app.jar --spring.profiles.active=dev
```

Result:

```
DevNotificationService bean is created.
ProdNotificationService bean is not created.
```

Run with prod:

```
java -jar app.jar --spring.profiles.active=prod
```

Result:

```
ProdNotificationService bean is created.
DevNotificationService bean is not created.
```

Simple understanding:

```
@Service  → Spring, please create an object of this class.
@Profile  → But create it only when this profile is active.
```

---

45. Complete `@Profile` Demo

Interface:

```
public interface NotificationService {
    void sendNotification(String message);
}
```

Dev implementation:

Coder Army

```
@Service
@Profile("dev")
public class DevNotificationService implements NotificationSe
rvice {

    @Override
    public void sendNotification(String message) {
        System.out.println("DEV MODE: Fake notification. Mess
age: " + message);
    }
}
```

Prod implementation:

```
@Service
@Profile("prod")
public class ProdNotificationService implements NotificationS
ervice {

    @Override
    public void sendNotification(String message) {
        System.out.println("PROD MODE: Real notification sen
t. Message: " + message);
    }
}
```

Controller:

```
@RestController
@RequestMapping("/api/notification")
public class NotificationController {

    private final NotificationService notificationService;

    public NotificationController(NotificationService notific
```

Coder Army

```
ationService) {
        this.notificationService = notificationService;
    }

    @GetMapping("/send")
    public String sendNotification() {
        notificationService.sendNotification("Welcome to the
application");
        return "Notification processed";
    }
}
```

If `dev` is active, Spring injects `DevNotificationService`.

If `prod` is active, Spring injects `ProdNotificationService`.

The controller remains the same. The implementation changes based on the active profile.

---

46. When to Use Profile-Specific Files vs `@Profile`

This distinction is very important.

Use Profile-Specific Files When Values Change

Use profile-specific files when the class or logic remains the same, but configuration values change.

Examples:

```
Server port
Database URL
Database username
Database password
Logging level
Custom messages
Feature flags
JPA show-sql
Third-party API URLs
```

Coder Army

Example:

`application-dev.yml`:

```yaml
server:
port:8081

app:
message: Running in development
```

`application-prod.yml`:

```yaml
server:
port:8080

app:
message: Running in production
```

Here, the Java class does not change. Only the values change.

Use profile-specific files when your problem is:

```
I need different configuration values for different environments.
```

Use `@Profile` When Bean/Class Implementation Changes

Use `@Profile` when different environments need different implementations.

Examples:

```
DevEmailService vs ProdEmailService
DummyPaymentService vs RealPaymentService
LocalStorageService vs CloudStorageService
MockSmsService vs RealSmsService
FakeNotificationService vs RealNotificationService
```

Here, the difference is not just a value. The actual implementation is different.

Coder Army

Use `@Profile` when your problem is:

```
I need different beans/classes to be created for different environments.
```

47. Quick Comparison

Topic	Use Case	Example
`application.properties`	Simple key-value configuration	`server.port=8081`
`application.yml`	Structured and nested configuration	`spring.datasource.url` grouped under `spring.datasource`
Profile-specific files	Different values for different environments	`application-dev.yml`, `application-prod.yml`
`spring.profiles.active`	Tells Spring Boot which profile to use	`dev`, `test`, `prod`
Command-line profile	Run same JAR with different profiles	`--spring.profiles.active=prod`
Environment variable profile	Activate profile from OS/server/deployment platform	`SPRING_PROFILES_ACTIVE=prod`
`@Value`	Read a single configuration value	`@Value("${app.message}")`
`@Profile`	Create bean only for a specific profile	`@Profile("dev")`

48. Common Mistakes to Avoid

Mistake 1: Hardcoding Environment Values in Java Code

Avoid:

```
String dbUrl = "jdbc:mysql://prod-server:3306/student_db";
```

Prefer configuration files or environment variables.

Coder Army

Mistake 2: Keeping Both `.properties` and `.yml` for the Same Purpose

Avoid confusing project structure.

Prefer one consistent format.

Mistake 3: Hardcoding `prod` Profile Inside the JAR

Avoid:

```
spring:
profiles:
active: prod
```

inside the packaged application for production use.

Prefer passing the profile from outside using command line, environment variables, or deployment configuration.

Mistake 4: Using Too Many `@Value` Annotations for Complex Config

For a few values, `@Value` is fine.

For complex nested configuration, lists, or grouped values, prefer:

```
@ConfigurationProperties
```

Mistake 5: Confusing Profile Files With `@Profile`

Remember:

```
Profile-specific files → change values
@Profile               → change beans/classes
```

49. Final Mental Model

Coder Army

Use this simple mental model:

```
Configuration means values that can change without changing Java code.
```

```
application.properties gives key-value configuration.
```

```
application.yml gives the same configuration in a cleaner nested structure.
```

```
Profiles allow the same application to run with different environment-specific values.
```

```
@Profile allows Spring to create different beans/classes based on the active environment.
```

The final goal is:

```
Same code
Same build
Different environment
Different configuration
```

---

50. Video Ending Summary

In this video, we learned:

```
Why configuration is needed
Why hardcoding is a bad practice
What application.properties is
How to read configuration using @Value
Why YAML is useful
How YAML indentation works
How to convert properties to YAML
How to represent lists in YAML
Why profiles are needed
How profile-specific files work
How to activate profiles
```

```
How to use command-line and environment variables
How @Profile controls bean creation
When to use profile files and when to use @Profile
```

The main takeaway is simple:

```
Spring Boot configuration helps us write flexible applications.
Profiles help the same application behave differently in different environments.
@Profile helps us create environment-specific beans only when needed.
```

Coder Army

Coder Army

Filters in Spring Boot :

> Filters allow us to execute common HTTP-level logic before a request reaches the Spring MVC controller and after the request has been processed.

---

1. Why Do We Need Filters?

A typical request in a Spring Boot application may appear to follow this flow:

```
Client / Browser / Postman
        ↓
Spring Boot Application
        ↓
Controller Method
        ↓
Response
```

In a real application, the request does not directly reach the controller. Before executing the controller, the application may need to perform common tasks such as:

- Logging the request URL
- Checking an authentication token
- Blocking suspicious requests
- Measuring request execution time
- Adding common response headers
- Applying character encoding
- Creating a request or trace ID

Writing this logic inside every controller method leads to repeated code.

```java
@GetMapping("/students")
public List<Student> getStudents(HttpServletRequest request)
{
```

Coder Army

```
    System.out.println(
            "Request URL: " + request.getRequestURI()
    );

    // Authentication check
    // Logging
    // Timing logic

    return studentService.getAllStudents();
}
```

Although this code works, it creates several design problems:

1. The same infrastructure logic is repeated across multiple APIs.
2. Controller methods become difficult to read.
3. Developers may forget to add the logic to newly created APIs.
4. Controllers stop focusing on their primary responsibility: handling application use cases.

A Filter provides a common place for request-level logic that must apply to multiple endpoints.

---

2. Cross-Cutting Concerns

Tasks that apply across many parts of an application are known as **cross-cutting concerns**.

Common examples include:

- Request logging
- Authentication and authorization checks
- Security checks
- Character encoding
- Request timing
- Audit tracking

Coder Army

- Rate limiting
- Correlation or trace IDs

These concerns are not part of the business logic of a particular controller, service, or repository.

They operate across multiple requests, making Filters suitable when the work belongs to the HTTP or Servlet layer.

---

3. Request–Response Flow in Spring Boot

A simplified request flow in a Spring Boot web application is:

```
Client
  ↓
Servlet Container, such as Tomcat
  ↓
Filters
  ↓
DispatcherServlet
  ↓
Interceptors
  ↓
Controller
  ↓
Service
  ↓
Repository
```

The response returns through the same components in reverse order:

```
Repository
  ↓
Service
  ↓
Controller
  ↓
Interceptors
  ↓
DispatcherServlet
  ↓
Filters
```

Coder Army

```
  ↓
Client
```

Where does a Filter execute?

Filters belong to the **Servlet layer**. They execute before the request reaches Spring MVC's `DispatcherServlet`.

```
Client
  ↓
Tomcat
  ↓
Filter 1
  ↓
Filter 2
  ↓
DispatcherServlet
  ↓
Controller
```

Because a Filter executes before `DispatcherServlet`, it can reject a request before Spring MVC performs controller mapping.

> A Filter operates at a lower web level than a Spring MVC Interceptor.

---

4. What Is a Filter?

A Filter acts like a checkpoint through which an HTTP request and response pass.

It can perform logic:

- Before the request continues
- After downstream request processing returns

Conceptually, a Filter works like this:

```
doSomethingBefore();

chain.doFilter(request, response);
```

Coder Army

```
doSomethingAfter();
```

The call to `chain.doFilter(...)` passes control to the next component in the request-processing chain.

A Filter can:

- Inspect request headers, cookies, query parameters, and other metadata
- Add response headers
- Log request and response details
- Measure total request duration
- Validate authentication information
- Wrap the request or response
- Allow the request to continue
- Stop the request immediately

Because Filters operate at the Servlet layer, they are not limited to controller endpoints. Depending on their registration, they may also apply to paths such as:

```
/error
/favicon.ico
/static/logo.png
Servlet endpoints
Spring MVC endpoints
```

---

5. Filters Come from the Servlet API

Filters are not originally a Spring feature. They are defined by the Servlet specification.

In Spring Boot 3 and modern Jakarta-based applications, the Filter interface is available from:

```
jakarta.servlet.Filter
```

Coder Army

A simplified version of the interface looks like this:

```
public interface Filter {

    default void init(FilterConfig filterConfig)
            throws ServletException {
    }

    void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException;

    default void destroy() {
    }
}
```

The Filter lifecycle contains three methods:

Method	Purpose
`init()`	Called when the Filter is initialized
`doFilter()`	Called when a matching request passes through the Filter
`destroy()`	Called when the Filter is removed or the application shuts down

In most Spring Boot applications, custom Filter logic is mainly written inside `doFilter()`.

---

6. Spring Boot 2 vs Spring Boot 3 Imports

Spring Boot 2 applications generally used the `javax.servlet` package:

```
import javax.servlet.Filter;
import javax.servlet.FilterChain;
```

Coder Army

```
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
```

Spring Boot 3 uses the `jakarta.servlet` package:

```
import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
```

For Spring Boot 3, use:

```
import jakarta.servlet.Filter;
```

Do not use:

```
import javax.servlet.Filter;
```

---

7. Why Does `Filter` Use `ServletRequest` and `ServletResponse`?

The standard Filter interface uses generic Servlet abstractions:

```
ServletRequest
ServletResponse
```

These types are not limited to HTTP-specific operations.

In a normal Spring Boot web application, requests are HTTP requests. Therefore, they can be cast to HTTP-specific types.

```
HttpServletRequest httpRequest =
        (HttpServletRequest) request;
```

Coder Army

```
HttpServletResponse httpResponse =
        (HttpServletResponse) response;
```

After casting, we can access HTTP-specific information.

```
httpRequest.getMethod();
httpRequest.getRequestURI();
httpRequest.getHeader("Authorization");

httpResponse.setStatus(401);
httpResponse.setHeader(
        "X-App-Name",
        "Student API"
);
```

8. Role of `DispatcherServlet`

`DispatcherServlet` is the central Servlet of Spring MVC. It coordinates the MVC request-processing flow.

Its responsibilities include:

- Finding the correct controller method
- Resolving controller method arguments
- Invoking the controller
- Handling controller return values
- Converting Java objects into JSON
- Coordinating exception handling
- Producing the HTTP response

Filters execute before this Spring MVC processing begins.

```
Tomcat
  ↓
Registered Filters
```

```
  ↓
DispatcherServlet
  ↓
Controller Mapping and Execution
```

Does a Filter know which controller method will execute?

At the beginning of Filter execution, Spring MVC has usually not yet selected the controller method.

A Filter can directly access information such as:

- HTTP method
- Request URI
- Query parameters
- Headers
- Cookies
- Remote address
- Request body stream

However, it does not naturally work with controller metadata such as:

```
StudentController
getStudentById()
@GetMapping("/api/students/{id}")
```

Logic that depends on the selected controller or handler is generally better suited to a Spring MVC Interceptor.

---

9. Creating a Custom Filter

A basic custom Filter can be created by implementing the `Filter` interface and registering the class as a Spring bean.

```
package in.strikes.filterdemo.filter;
```

Coder Army

Coder Army

```java
import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import org.springframework.stereotype.Component;

import java.io.IOException;

@Component
public class LoggingFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        System.out.println(
                "Request entered the Filter"
        );

        chain.doFilter(request, response);

        System.out.println(
                "Response passed through the Filter"
        );
    }
}
```

What does the Filter receive?

Parameter	Meaning
`request`	The incoming request

Coder Army

Parameter	Meaning
`response`	The outgoing response
`chain`	The remaining Filters and final target Servlet

Before-chain logic

```
System.out.println(
        "Request entered the Filter"
);
```

This statement runs before the request reaches the next Filter or `DispatcherServlet`.

Continuing the request

```
chain.doFilter(request, response);
```

This passes the request to:

- The next Filter, or
- `DispatcherServlet` when the current Filter is the final Filter

After-chain logic

```
System.out.println(
        "Response passed through the Filter"
);
```

This statement runs after downstream processing returns.

---

10. Understanding `doFilter()`

The main method of a Filter is:

```
void doFilter(
        ServletRequest request,
        ServletResponse response,
```

```
        FilterChain chain
) throws IOException, ServletException;
```

`ServletRequest request`

The request object provides general request information.

```
request.getParameter("name");
request.getRemoteAddr();
request.getContentType();
```

For HTTP-specific information:

```
HttpServletRequest httpRequest =
        (HttpServletRequest) request;
```

Useful HTTP request methods include:

```
httpRequest.getMethod();
httpRequest.getRequestURI();
httpRequest.getRequestURL();
httpRequest.getHeader("Authorization");
httpRequest.getCookies();
httpRequest.getQueryString();
httpRequest.getSession();
```

`ServletResponse response`

The response object represents the outgoing response.

```
HttpServletResponse httpResponse =
        (HttpServletResponse) response;
```

It can be used to:

Coder Army

Coder Army

```
httpResponse.setStatus(401);
httpResponse.setHeader("X-App", "Demo");
httpResponse.setContentType("application/json");
httpResponse.getWriter().write("...");
```

`FilterChain chain`

`FilterChain` represents everything that remains after the current Filter.

This may include:

- Other registered Filters
- `DispatcherServlet`
- Controller execution
- Service and repository calls
- Response conversion

The current Filter does not need to know which exact component comes next. It only calls:

```
chain.doFilter(request, response);
```

---

11. Role of `FilterChain`

Suppose an application contains three Filters:

```
AuthenticationFilter
LoggingFilter
TimingFilter
```

The request-processing chain may look like this:

```
AuthenticationFilter
        ↓
LoggingFilter
        ↓
```

Coder Army

```
TimingFilter
        ↓
DispatcherServlet
```

Every Filter has two choices.

Continue the request

```
chain.doFilter(request, response);
```

Stop the request

```
response.setStatus(
        HttpServletResponse.SC_UNAUTHORIZED
);

return;
```

> If a Filter does not call `chain.doFilter(...)`, the request does not continue to the remaining Filters or controller.

---

12. Request and Response Execution Order

Filters surround the remaining request-processing chain.

Suppose two Filters are registered:

```
AuthenticationFilter → LoggingFilter → Controller
```

The incoming request executes in normal order, while the response-side logic executes in reverse order.

Authentication Filter

Coder Army

```java
@Component
@Order(1)
public class AuthenticationFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        System.out.println(
                "Authentication before"
        );

        chain.doFilter(request, response);

        System.out.println(
                "Authentication after"
        );
    }
}
```

Logging Filter

```java
@Component
@Order(2)
public class LoggingFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
```

```
    ) throws IOException, ServletException {

        System.out.println("Logging before");

        chain.doFilter(request, response);

        System.out.println("Logging after");
    }
}
```

Coder Army

Controller

```
@GetMapping("/hello")
public String hello() {
    System.out.println("Controller");

    return "Hello";
}
```

Output:

```
Authentication before
Logging before
Controller
Logging after
Authentication after
```

The execution can be visualized as:

```
AuthenticationFilter before
        ↓
LoggingFilter before
        ↓
Controller
        ↓
LoggingFilter after
```

```
        ↓
AuthenticationFilter after
```

13. Reading HTTP Request and Response Information

A practical Filter usually casts the generic Servlet objects to HTTP-specific objects.

```
@Component
public class RequestLoggingFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;

        HttpServletResponse httpResponse =
                (HttpServletResponse) response;

        System.out.println(
                "Incoming request: "
                        + httpRequest.getMethod()
                        + " "
                        + httpRequest.getRequestURI()
        );

        chain.doFilter(
                httpRequest,
                httpResponse
        );
```

Coder Army

Coder Army

```
        System.out.println(
                "Response status: "
                        + httpResponse.getStatus()
        );
    }
}
```

Example output:

```
Incoming request: GET /api/students
Response status: 200
```

---

14. Common Use Cases of Filters

14.1 Request Logging

A Filter can log common request information for every API.

```
String method = httpRequest.getMethod();

String uri = httpRequest.getRequestURI();

String remoteAddress =
        httpRequest.getRemoteAddr();

String userAgent =
        httpRequest.getHeader("User-Agent");
```

Example log:

```
Method: GET
URI: /api/students
Remote IP: 192.168.1.10
User-Agent: PostmanRuntime
```

Coder Army

14.2 Authentication Token Inspection

A Filter can inspect an authentication header.

```
String token =
        httpRequest.getHeader("Authorization");
```

It may then:

- Check whether the token exists
- Validate the token
- Reject invalid requests
- Extract user information
- Populate security-related context

Spring Security itself relies heavily on a chain of Filters.

In a real secured application, authentication should normally be integrated with Spring Security rather than implemented through an unrelated custom Filter.

14.3 Character Encoding

A Filter can enforce consistent request and response encoding.

```
request.setCharacterEncoding("UTF-8");
response.setCharacterEncoding("UTF-8");
```

Spring also provides built-in support such as `CharacterEncodingFilter`.

14.4 Common Response Headers

A Filter can add headers that should appear in multiple responses.

```
httpResponse.setHeader(
        "X-Application-Name",
        "Student Service"
```

```
);

httpResponse.setHeader(
        "X-Content-Type-Options",
        "nosniff"
);
```

Coder Army

Common purposes include:

- Security headers
- Request tracking
- Application metadata
- Cache control

14.5 Measuring Request Duration

A Filter can measure the time spent in the downstream request-processing chain.

```
long startTime =
        System.currentTimeMillis();

chain.doFilter(request, response);

long duration =
        System.currentTimeMillis() - startTime;

System.out.println(
        "Duration: " + duration + " ms"
);
```

This generally includes processing performed by:

- Remaining Filters
- `DispatcherServlet`
- Interceptors

Coder Army

- Controller
- Service
- Repository
- Response conversion

The exact measurement depends on the position of the Filter and how the response is handled.

14.6 Blocking Unwanted Requests

A Filter can reject a request before it reaches Spring MVC.

```
String apiKey =
        httpRequest.getHeader("X-API-KEY");

if (apiKey == null) {
    httpResponse.setStatus(
            HttpServletResponse.SC_UNAUTHORIZED
    );

    return;
}

chain.doFilter(request, response);
```

The `return` statement stops the current Filter method.

Because `chain.doFilter(...)` was not called, the controller does not execute.

14.7 Correlation or Trace IDs

A Filter can generate a unique ID for every request.

```
String requestId =
        UUID.randomUUID().toString();
```

Coder Army

```
httpResponse.setHeader(
        "X-Request-Id",
        requestId
);
```

The same ID can be included in:

- Application logs
- Response headers
- Downstream service calls
- Error reports

This makes it easier to trace one request across multiple log entries or services.

---

15. Blocking a Request and Returning JSON

A Filter can directly create an error response without allowing the request to reach the controller.

```
package in.strikes.filterdemo.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;

import java.io.IOException;

@Component
public class ApiKeyFilter implements Filter {

    @Override
```

Coder Army

```
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;

        HttpServletResponse httpResponse =
                (HttpServletResponse) response;

        String apiKey =
                httpRequest.getHeader("X-API-KEY");

        if (apiKey == null
                || !apiKey.equals("secret123")) {

            httpResponse.setStatus(
                    HttpServletResponse.SC_UNAUTHORIZED
            );

            httpResponse.setContentType(
                    "application/json"
            );

            httpResponse.setCharacterEncoding(
                    "UTF-8"
            );

            httpResponse.getWriter().write(
                    """
                    {
                      "message": "Invalid or missing API key"
                    }
                    """
```

Coder Army

```
            );

            return;
        }

        chain.doFilter(request, response);
    }
}
```

Request flow when the API key is valid

```
API key is valid
      ↓
chain.doFilter(...)
      ↓
Request continues
```

Request flow when the API key is invalid

```
API key is missing or invalid
      ↓
Write a 401 response
      ↓
Return without calling chain.doFilter(...)
      ↓
Request stops
```

> The hard-coded key in this example is only for understanding Filter behaviour. Real authentication should use a proper security mechanism.

---

16. Using `try-finally` for Timing and Cleanup

Consider this code:

```
chain.doFilter(request, response);
```

Coder Army

```
System.out.println("Request completed");
```

If downstream processing throws an exception, the statement after `chain.doFilter(...)` may not execute normally.

A safer pattern for timing, cleanup, or final logging is:

```
long startTime =
        System.currentTimeMillis();

try {
    chain.doFilter(request, response);
} finally {
    long duration =
            System.currentTimeMillis() - startTime;

    System.out.println(
            "Duration: " + duration + " ms"
    );
}
```

The `finally` block executes whether downstream processing completes normally or throws an exception.

---

17. Complete Request Logging Filter

The following Filter logs:

- HTTP method
- Request path
- Query string
- Remote IP address
- Response status
- Total duration

Coder Army

```java
package in.strikes.filterdemo.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

import java.io.IOException;

@Component
public class RequestLoggingFilter implements Filter {

    private static final Logger logger =
            LoggerFactory.getLogger(
                    RequestLoggingFilter.class
            );

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;

        HttpServletResponse httpResponse =
                (HttpServletResponse) response;
```

Coder Army

```
        long startTime =
                System.currentTimeMillis();

        String method =
                httpRequest.getMethod();

        String requestUri =
                httpRequest.getRequestURI();

        String queryString =
                httpRequest.getQueryString();

        String remoteAddress =
                httpRequest.getRemoteAddr();

        String completePath =
                queryString == null
                        ? requestUri
                        : requestUri
                        + "?"
                        + queryString;

        logger.info(
                "Incoming request: "
                        + "method={}, path={}, ip={}",
                method,
                completePath,
                remoteAddress
        );

        try {
            chain.doFilter(request, response);
        } finally {
            long duration =
                    System.currentTimeMillis()
```

Coder Army

```
                                - startTime;

            logger.info(
                    "Request completed: "
                            + "method={}, path={}, "
                            + "status={}, duration={}ms",
                    method,
                    completePath,
                    httpResponse.getStatus(),
                    duration
            );
        }
    }
}
```

Using a logging framework such as SLF4J is preferred over `System.out.println()` in real applications because logs can be configured, filtered, stored, and integrated with monitoring tools.

---

18. Filter Ordering with `@Order`

When multiple Filters are registered, their order affects request and response processing.

```
@Component
@Order(1)
public class FirstFilter implements Filter {
    // Filter logic
}
```

```
@Component
@Order(2)
public class SecondFilter implements Filter {
```

```
    // Filter logic
}
```

For an incoming request:

```
FirstFilter
    ↓
SecondFilter
    ↓
DispatcherServlet
```

For the returning response:

```
DispatcherServlet
    ↓
SecondFilter
    ↓
FirstFilter
```

What happens without `@Order`?

A Filter registered as a Spring bean still executes.

However, when several Filters depend on a particular sequence, the application should define their order explicitly instead of relying on an observed default order.

---

19. Where Should Response Logic Be Placed?

Consider adding a response header after downstream processing:

```
chain.doFilter(request, response);

httpResponse.setHeader(
        "X-Test",
        "value"
);
```

This may work when the response has not yet been committed.

Coder Army

Coder Army

However, once the response body or headers have been flushed, changing the status or headers may be too late.

Headers that must definitely be present should generally be added before continuing the chain.

```
httpResponse.setHeader(
        "X-Test",
        "value"
);

chain.doFilter(request, response);
```

Good candidates for pre-processing

- Request validation
- Authentication checks
- Required response headers
- Character encoding
- Request setup

Good candidates for post-processing

- Logging the final response status
- Measuring duration
- Metrics collection
- Cleanup operations

---

20. Logging Safety

Filters can access sensitive request information, but not every value should be written to logs.

Avoid logging:

- Authorization tokens

Coder Army

- Session cookies
- API keys
- Passwords
- Credit card information
- Sensitive personal information

Logging entire request headers or bodies without filtering can create serious security and privacy problems.

> Log only the information required for debugging, auditing, or monitoring, and mask sensitive values wherever necessary.

---

21. Filter Responsibilities at a Glance

Requirement	Is a Filter suitable?
Log all incoming HTTP requests	Yes
Read or validate request headers	Yes
Add a common response header	Yes
Block a request before Spring MVC	Yes
Apply character encoding	Yes
Measure overall request duration	Yes
Work with the selected controller method	Usually no; prefer an Interceptor
Implement full application security manually	No; prefer Spring Security
Execute business logic for one API	No; keep it in controller or service layers

---

22. Key Takeaways

- Filters belong to the Servlet layer.
- They execute before Spring MVC's `DispatcherServlet`.
- A Filter can run logic before and after downstream request processing.
- `chain.doFilter(request, response)` continues the request.

Coder Army

- Returning without calling `chain.doFilter(...)` stops the request.
- Incoming Filter logic executes in order.
- Response-side Filter logic executes in reverse order.
- `HttpServletRequest` and `HttpServletResponse` provide HTTP-specific operations.
- Filters are suitable for logging, headers, encoding, request timing, trace IDs, and early request validation.
- Filter order should be explicitly defined when the sequence matters.
- Sensitive headers, tokens, and personal data should not be logged.
- Authentication in production applications should normally be handled through Spring Security.

Final Mental Model

```
Request enters the server
        ↓
Filter performs pre-processing
        ↓
Filter either blocks the request
or calls chain.doFilter(...)
        ↓
Remaining Filters and DispatcherServlet execute
        ↓
Controller, Service, and Repository execute
        ↓
Response returns through the Filter chain
        ↓
Filter performs post-processing
        ↓
Response reaches the client
```

A Filter is a reusable HTTP-level checkpoint placed around the remaining request-processing flow.

Coder Army

Spring Boot Filters - 2

Modifying Requests and Responses, Filter Registration, and `OncePerRequestFilter`

A servlet Filter can do more than inspect an HTTP request and response. It can also change what is sent to the client or what downstream components receive.

However, request and response modification do not work in the same way:

- Response headers and status codes are relatively easy to change.
- Response bodies usually require a response wrapper.
- Request headers and parameters are read-only from the original request object.
- Request-body handling is more sensitive because the body arrives as a stream.

> Request and response mutation is possible, but it should be used only when the Filter layer is genuinely the correct place for that responsibility.

---

1. Modifying the HTTP Response

1.1 Adding a response header

A Filter can add or replace response headers before passing the request further through the chain.

```
package in.strikes.crudDemo.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
```

Coder Army

```java
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;

import java.io.IOException;

@Component
public class ResponseHeaderFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletResponse httpResponse =
                (HttpServletResponse) response;

        httpResponse.setHeader(
                "X-Application-Name",
                "Student Management API"
        );

        chain.doFilter(request, response);
    }
}
```

The client now receives a header similar to:

```
X-Application-Name: Student Management API
```

`setHeader()` vs `addHeader()`

```java
httpResponse.setHeader("X-Application-Name", "Student API");
```

Coder Army

`setHeader()` replaces the existing value of that header.

```
httpResponse.addHeader("X-Feature", "Students");
httpResponse.addHeader("X-Feature", "Courses");
```

`addHeader()` can add another value without replacing the previous one.

1.2 Why headers are usually added before `chain.doFilter()`

```
httpResponse.setHeader(
        "X-Application-Name",
        "Student Management API"
);

chain.doFilter(request, response);
```

The downstream code may write and flush the response. Once that happens, the response can become **committed**.

A committed response means that its status and headers may already have been sent to the client. After that point, changing them may have no effect or may cause an error.

The response state can be checked using:

```
boolean committed = httpResponse.isCommitted();
```

> Add headers early when they should be present on every response. Logic that depends on the final result may need to run after `chain.doFilter()` and must account for the response possibly being committed.

---

2. Setting the Response Status

A Filter can assign an HTTP status code.

Coder Army

```
httpResponse.setStatus(
        HttpServletResponse.SC_UNAUTHORIZED
);
```

The constant above represents HTTP status `401 Unauthorized`.

A Filter can also stop request processing and return a complete response directly.

```
httpResponse.setStatus(
        HttpServletResponse.SC_UNAUTHORIZED
);

httpResponse.setContentType("application/json");
httpResponse.setCharacterEncoding("UTF-8");

httpResponse.getWriter().write(
        """
        {
          "message": "Authentication required"
        }
        """
);

return;
```

Because the method returns without calling `chain.doFilter()`, the request does not reach the next Filter or the controller.

Important rule

```
chain.doFilter(request, response);
```

means:

> Continue processing the request.

Coder Army

```
return;
```

without calling the chain means:

> Stop processing the request here.

Practical note

For production authentication and authorization, Spring Security is generally the correct solution. For example, authentication failures are commonly handled through components such as `AuthenticationEntryPoint`, rather than a general homemade Filter.

---

3. Modifying the Response Body

Suppose a controller returns:

```
{
  "name": "Aditya",
  "age": 25
}
```

The desired response is:

```
{
  "data": {
    "name": "Aditya",
    "age": 25
  },
  "application": "Student Management API"
}
```

A normal `HttpServletResponse` does not provide a method such as:

```
String body = response.getBody(); // Does not exist
```

Coder Army

Why is there no `getBody()` method?

The controller or Spring MVC writes response data to:

```
response.getWriter();
```

or:

```
response.getOutputStream();
```

The data may then move progressively toward the client. The ordinary response acts mainly as a destination into which data is written. It is not designed as a Java object from which the complete body can later be retrieved.

```
Controller
    ↓ writes
HttpServletResponse output stream
    ↓
Client
```

To examine or replace the body after downstream processing, the response must first be wrapped.

---

4. Using `ContentCachingResponseWrapper`

Spring provides:

```
ContentCachingResponseWrapper
```

This wrapper stores a copy of the response body so that it can be accessed after the controller has executed.

Conceptual flow

```
Client request
      ↓
ResponseBodyFilter
```

Coder Army

```
      ↓ creates wrapper
ContentCachingResponseWrapper
      ↓ passed through chain
Controller writes response into wrapper
      ↓
Filter reads cached response bytes
      ↓
Filter optionally replaces the body
      ↓
Final body is copied to real response
      ↓
Client receives response
```

Example

```
package in.strikes.crudDemo.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.util.ContentCachingResponseWra
pper;

import java.io.IOException;
import java.nio.charset.StandardCharsets;

@Component
public class ResponseBodyFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
```

Coder Army

```
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;

        HttpServletResponse httpResponse =
                (HttpServletResponse) response;

        ContentCachingResponseWrapper wrappedResponse =
                new ContentCachingResponseWrapper(httpRespons
e);


        // Downstream code writes into the wrapper.
        chain.doFilter(httpRequest, wrappedResponse);

        byte[] originalBodyBytes =
                wrappedResponse.getContentAsByteArray();

        String originalBody = new String(
                originalBodyBytes,
                StandardCharsets.UTF_8
        );

        String modifiedBody =
                """
                {
                  "originalResponse": %s,
                  "processedBy": "ResponseBodyFilter"
                }
                """.formatted(originalBody);

        byte[] modifiedBodyBytes =
                modifiedBody.getBytes(StandardCharsets.UTF_
8);
```

```
        // Clears the currently cached body, but keeps status
and headers.
        wrappedResponse.resetBuffer();

        wrappedResponse.setContentType("application/json");
        wrappedResponse.setCharacterEncoding(
                StandardCharsets.UTF_8.name()
        );

        wrappedResponse.getOutputStream()
                .write(modifiedBodyBytes);

        // Copies the cached final body into the actual respo
nse.
        wrappedResponse.copyBodyToResponse();
    }
}
```

Coder Army

Important steps

Step 1: Wrap the real response

```
ContentCachingResponseWrapper wrappedResponse =
        new ContentCachingResponseWrapper(httpResponse);
```

Step 2: Pass the wrapper through the chain

```
chain.doFilter(httpRequest, wrappedResponse);
```

The controller now writes into `wrappedResponse`, not directly into the original response.

Step 3: Read the cached body

Coder Army

```
byte[] originalBodyBytes =
        wrappedResponse.getContentAsByteArray();
```

Step 4: Clear the cached body before replacing it

```
wrappedResponse.resetBuffer();
```

`resetBuffer()` clears the body buffer while retaining the existing status code and headers.

Step 5: Write the replacement body

```
wrappedResponse.getOutputStream()
        .write(modifiedBodyBytes);
```

Step 6: Copy the body to the real response

```
wrappedResponse.copyBodyToResponse();
```

This step is essential. Without it, the client may receive an empty response because the data remains inside the wrapper.

4.1 Limitations of manually modifying a response body

The example is useful for understanding wrappers, but manually rebuilding JSON inside a Filter is uncommon in normal application code.

Potential problems include:

- The original body may be empty.
- The body may not contain JSON.
- The original JSON may be malformed.
- Binary responses such as files or images must not be treated as text.
- Buffering large responses consumes memory.

Coder Army

- Existing headers such as `Content-Length`, `ETag`, or compression-related headers may become invalid after modification.
- Streaming responses are not suitable for complete-body caching.
- Exceptions and error dispatches require careful handling.

This expression:

```
"""
{
  "originalResponse": %s
}
""".formatted(originalBody);
```

assumes that `originalBody` is already valid JSON.

For production JSON transformation, use Jackson rather than constructing JSON manually.

For globally changing controller response bodies in Spring MVC, `ResponseBodyAdvice` is often a cleaner choice than a servlet Filter.

---

5. Modifying Request Data

Request modification is generally more complex than response-header modification.

Suppose the client sends:

```
GET /api/students
Authorization: Bearer abc123
```

The Filter can read the header:

```
String token =
        httpRequest.getHeader("Authorization");
```

However, the original request does not provide a normal method such as:

Coder Army

```
httpRequest.setHeader(
        "Authorization",
        "Bearer new-token"
); // Does not exist
```

`HttpServletRequest` is primarily a read-oriented interface. To change what downstream code sees, the normal technique is to pass a wrapper instead of the original request.

---

6. The Request Wrapper Concept

```
Original HttpServletRequest
            ↓ wrapped by
Custom HttpServletRequestWrapper
            ↓ passed to
Next Filter or DispatcherServlet
```

A custom wrapper can override methods such as:

```
getHeader(String name)
getHeaders(String name)
getHeaderNames()
getParameter(String name)
getParameterMap()
getParameterValues(String name)
getInputStream()
getReader()
```

Downstream components receive the wrapper and therefore observe the values returned by its overridden methods.

Base class

```
HttpServletRequestWrapper
```

Coder Army

A custom request wrapper normally extends this class:

```
public class CustomRequestWrapper
        extends HttpServletRequestWrapper {

    public CustomRequestWrapper(
            HttpServletRequest request
    ) {
        super(request);
    }
}
```

The wrapper delegates unchanged operations to the original request and overrides only the behaviour that needs to change.

> Request mutation through wrappers is valid, but it should be used carefully. Business-level transformations are often better placed in Spring MVC, validation, Jackson configuration, or the service layer.

---

7. Modifying Query Parameters

Query parameters are also read-only from the perspective of the original request.

Suppose the request is:

```
GET /products?page=1
```

This method does not exist:

```
request.setParameter("page", "2");
```

To change how downstream code reads parameters, a wrapper must consistently override methods such as:

Coder Army

```
getParameter(String name)
getParameterMap()
getParameterValues(String name)
getParameterNames()
```

Overriding only `getParameter()` may be insufficient because Spring MVC or another library may read the complete parameter map instead.

Use this technique carefully because the same parameters may be consumed by:

- `@RequestParam`
- Data binding
- Validation
- Other Filters
- Spring MVC infrastructure

---

8. Understanding the Request Body

Request-body handling is more sensitive than reading headers or query parameters.

Common request information is exposed through methods such as:

```
httpRequest.getHeader("Authorization");
httpRequest.getHeaderNames();
httpRequest.getParameter("page");
httpRequest.getParameterMap();
httpRequest.getMethod();
httpRequest.getRequestURI();
```

However, this method does not exist:

```
httpRequest.getBody(); // Does not exist
```

The request body is accessed as a stream:

Coder Army

```
request.getInputStream();
```

or as a character reader:

```
request.getReader();
```

8.1 Why is the request body exposed as a stream?

The body can contain many different kinds of data:

- JSON
- XML
- Plain text
- PDF documents
- Images
- Videos
- Multipart form data
- Other binary content

The servlet container cannot assume:

- That the body contains text
- That it contains JSON
- Which Java class it should become
- How large it is
- Which parser should process it
- Which character encoding should be used

At the HTTP level, the body is fundamentally a sequence of bytes arriving from the client.

Coder Army

```
Client sends bytes
        ↓
ServletInputStream
        ↓
Spring MVC selects an HttpMessageConverter
        ↓
Jackson handles JSON when applicable
        ↓
Java object
```

For a JSON request handled with `@RequestBody`, Jackson commonly converts those bytes into the required Java object.

```
@PostMapping
public Student createStudent(
        @RequestBody Student student
) {
    return student;
}
```

---

9. Why the Request Body Normally Cannot Be Read Repeatedly

An input stream has a current position. Each read moves that position forward.

Suppose the client sends:

```
{
  "name": "Aditya"
}
```

The server receives a sequence of bytes:

```
{ " n a m e " : ...
```

The read position moves through the stream:

Coder Army

```
Start
  ↓
{ → " → n → a → m → e → " → : → ...
                                  ↓
                              End of stream
```

After the bytes have been consumed, the stream is at its end. The original network bytes are not automatically placed back at position zero.

9.1 Incorrectly reading the body inside a Filter

Controller:

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    @PostMapping
    public Student createStudent(
            @RequestBody Student student
    ) {

        System.out.println(
                "Controller received: " + student
        );

        return student;
    }
}
```

Filter:

```
@Component
public class RequestBodyReadingFilter
        implements Filter {
```

Coder Army

```
    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;

        BufferedReader reader =
                httpRequest.getReader();

        StringBuilder body =
                new StringBuilder();

        String line;

        while ((line = reader.readLine()) != null) {
            body.append(line);
        }

        System.out.println(
                "Filter read body: " + body
        );

        chain.doFilter(request, response);
    }
}
```

The Filter may print the body successfully. However, when Spring MVC later tries to create the `Student` object, the stream may already be empty.

The controller can then fail with an error similar to:

Coder Army

```
Required request body is missing
```

9.2 Reading through `InputStream`

The same issue occurs when using `getInputStream()`:

```
InputStream inputStream =
        request.getInputStream();

String body = new String(
        inputStream.readAllBytes(),
        StandardCharsets.UTF_8
);

System.out.println(body);
```

`getReader()` and `getInputStream()` are two different views of the same request body. Reading the body through either one consumes it.

The Servlet API also does not allow an application to freely mix both methods on the same request.

10. Why Java and Tomcat Do Not Automatically Rewind Every Body

Automatic repeated reading would require the server to buffer the complete body somewhere.

Consider a request containing a 2 GB video:

```
Receive 2 GB
    ↓
Store 2 GB in memory or on disk
    ↓
Keep it available for possible repeated reads
```

Coder Army

Doing this automatically for every request would:

- Consume large amounts of memory or disk space
- Increase latency
- Reduce throughput
- Create denial-of-service risks
- Be unnecessary for most requests

Streaming therefore allows the server to process data progressively without storing every request body in full.

---

11. Re-reading or Replacing a Request Body

A Filter that must read and still preserve the body needs a request wrapper.

Typical use cases include:

- Logging selected request data
- Masking sensitive fields before logging
- Decrypting an encrypted payload
- Sanitising input
- Replacing or transforming request content

The general strategy is:

```
Read the original request stream once
            ↓
Store the bytes in a byte array
            ↓
Pass a custom HttpServletRequestWrapper
            ↓
Wrapper returns a fresh stream over cached bytes
            ↓
Spring MVC can read the body normally
```

A wrapper commonly overrides both:

Coder Army

```
getInputStream()
getReader()
```

Whenever downstream code calls `getInputStream()`, the wrapper can create a new stream over the stored bytes.

```
new ByteArrayInputStream(cachedBody);
```

A new `ByteArrayInputStream` begins at position zero, allowing the cached body to be read again.

Memory warning

Caching a request body means storing it in memory unless a more advanced disk-backed strategy is used.

Before buffering, consider:

- Maximum request size
- File uploads
- Multipart requests
- Sensitive information
- Passwords and tokens
- Performance impact

> Never log complete request bodies blindly. Logs can expose credentials, personal data, access tokens, and financial information.

---

12. Choosing the Correct Layer

Before modifying data inside a Filter, consider whether another Spring mechanism is more suitable.

Coder Army

Requirement	Usually better suited mechanism
Authentication and authorization	Spring Security
Standard controller response envelope	`ResponseBodyAdvice`
Exception response formatting	`@RestControllerAdvice`
JSON field conversion	Jackson configuration or custom deserializer
Controller argument creation	`HandlerMethodArgumentResolver`
Input validation	Bean Validation
Business-data transformation	Service layer
Low-level HTTP concern	Filter

Filters are most appropriate for concerns that truly belong at the servlet or HTTP boundary.

---

13. Registering Filters in Spring Boot

There are multiple ways to register and scope a Filter.

Approach 1: Register with `@Component`

```
@Component
public class RequestLoggingFilter implements Filter {
    // Filter logic
}
```

Spring Boot detects the bean and registers it with the servlet container.

This approach is simple, but the Filter is generally mapped broadly, commonly to `/*`.

Use it when:

- The Filter should apply broadly
- Advanced registration options are unnecessary
- The registration should remain simple

Coder Army

14. Registering with `FilterRegistrationBean`

`FilterRegistrationBean` provides explicit control over Filter registration.

It can configure:

- URL patterns
- Filter order
- Filter name
- Dispatcher types
- Whether registration is enabled
- Async support

Why use it?

Suppose a Filter should apply only to:

```
/api/*
```

and not to:

```
/actuator/*
/swagger-ui/*
/static/*
```

`FilterRegistrationBean` can restrict the Filter at registration time.

14.1 Step 1: Create a normal Filter class

Do not annotate it with `@Component` when the same instance will be registered manually through `FilterRegistrationBean`.

```
package in.strikes.crudDemo.filter;
```

Coder Army

```
import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;

import java.io.IOException;

public class RequestLoggingFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;

        System.out.println(
                "Filtered request: "
                        + httpRequest.getRequestURI()
        );

        chain.doFilter(request, response);
    }
}
```

14.2 Step 2: Register it in a configuration class

```
package in.strikes.crudDemo.config;
```

Coder Army

```java
import in.strikes.crudDemo.filter.RequestLoggingFilter;
import org.springframework.boot.web.servlet.FilterRegistratio
nBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class FilterConfig {

    @Bean
    public FilterRegistrationBean<RequestLoggingFilter>
            requestLoggingFilterRegistration() {

        FilterRegistrationBean<RequestLoggingFilter>
                registrationBean =
                new FilterRegistrationBean<>();

        registrationBean.setFilter(
                new RequestLoggingFilter()
        );

        registrationBean.addUrlPatterns("/api/*");
        registrationBean.setOrder(1);
        registrationBean.setName(
                "requestLoggingFilter"
        );

        return registrationBean;
    }
}
```

The Filter now applies to paths such as:

```
/api/students
/api/products
```

```
/api/orders/10
```

It does not apply to paths such as:

```
/hello
/actuator/health
/swagger-ui/index.html
```

unless another configured pattern matches them.

14.3 Registering multiple URL patterns

```
registrationBean.addUrlPatterns(
        "/api/*",
        "/admin/*"
);
```

The Filter will apply to both groups:

```
/api/students
/admin/users
```

14.4 Avoid accidental duplicate registration

Do not normally use both of these for the same Filter:

```
@Component
public class RequestLoggingFilter implements Filter {
}
```

and:

```
registrationBean.setFilter(
        new RequestLoggingFilter()
```

Coder Army

```
);
```

Doing both can create two Filter registrations and cause the logic to execute more than once.

Choose one registration strategy unless duplicate registrations are intentional.

---

15. Controlling Filter Order

When multiple Filters are registered, their order determines how the request and response pass through them.

```
registrationBean.setOrder(1);
```

A lower order value runs earlier while processing the request.

Suppose:

```
LoggingFilter order = 1
AuthenticationFilter order = 2
```

Request flow:

```
Client
  ↓
LoggingFilter
  ↓
AuthenticationFilter
  ↓
DispatcherServlet
  ↓
Controller
```

Response flow returns in reverse:

```
Controller
  ↓
AuthenticationFilter
  ↓
```

Coder Army

```
LoggingFilter
  ↓
Client
```

This nested behaviour occurs because each Filter executes code before and after `chain.doFilter()`.

16. Filtering Specific URLs Manually

A Filter can inspect the current URI and decide whether to run its custom logic.

```java
@Component
public class ApiFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain
    ) throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;

        String uri =
                httpRequest.getRequestURI();

        if (!uri.startsWith("/api/")) {
            chain.doFilter(request, response);
            return;
        }

        System.out.println(
                "API-specific filter logic"
        );
```

Coder Army

```
            chain.doFilter(request, response);
        }
    }
```

This works, but the Filter is still invoked for every URL included in its servlet registration. It then decides internally whether to perform the custom logic.

Use this approach when:

- The condition is small and easy to understand
- The decision depends on more than a simple URL pattern
- The Filter is already registered broadly

For clean URL-level registration, `FilterRegistrationBean` is generally clearer.

---

17. Using `OncePerRequestFilter`

Implementing the servlet `Filter` interface directly is completely valid.

For Spring web applications, Spring also provides:

```
OncePerRequestFilter
```

It is an abstract Spring class designed specifically for HTTP request filtering.

Example

```
package in.strikes.crudDemo.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
```

Coder Army

Coder Army

```java
import java.io.IOException;

@Component
public class RequestLoggingFilter
        extends OncePerRequestFilter {

    private static final Logger logger =
            LoggerFactory.getLogger(
                    RequestLoggingFilter.class
            );

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain
    ) throws ServletException, IOException {

        logger.info(
                "{} {}",
                request.getMethod(),
                request.getRequestURI()
        );

        filterChain.doFilter(request, response);
    }
}
```

Advantages

- Direct access to `HttpServletRequest`
- Direct access to `HttpServletResponse`
- No manual casting from `ServletRequest` and `ServletResponse`

Coder Army

- Designed for Spring-based web applications
- Prevents duplicate execution within the same dispatch using an internal request attribute
- Provides extension methods for controlling filtering behaviour

Important nuance

The name does not mean that the Filter can never run again for every possible dispatch associated with a request.

Servlet processing can involve dispatch types such as:

- `REQUEST`
- `ASYNC`
- `ERROR`
- `FORWARD`
- `INCLUDE`

`OncePerRequestFilter` provides once-per-dispatch protection and offers hooks for controlling async and error dispatch behaviour.

For most normal Spring MVC requests, it behaves as developers expect: the Filter logic runs once for the main request processing.

---

18. Excluding URLs with `shouldNotFilter()`

When extending `OncePerRequestFilter`, URL-exclusion logic can be placed in:

```
shouldNotFilter()
```

Example

```
package in.strikes.crudDemo.filter;

import jakarta.servlet.FilterChain;
```

Coder Army

```java
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class ApiLoggingFilter
        extends OncePerRequestFilter {

    @Override
    protected boolean shouldNotFilter(
            HttpServletRequest request
    ) {

        String path = request.getRequestURI();

        return !path.startsWith("/api/");
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain
    ) throws ServletException, IOException {

        System.out.println(
                "Filtering: "
                        + request.getRequestURI()
        );

        filterChain.doFilter(request, response);
```

```
    }
}
```

Understanding the return value

```
return true;
```

means:

> Do not run `doFilterInternal()` for this request.

```
return false;
```

means:

> Run `doFilterInternal()` for this request.

In the example:

```
return !path.startsWith("/api/");
```

- `/api/students` produces `false`, so the Filter runs.
- `/hello` produces `true`, so the Filter is skipped.

---

19. Comparing URL-Scoping Approaches

Approach	Where the decision occurs	Best suited for
`FilterRegistrationBean` URL patterns	Servlet registration	Clean, broad URL-pattern control
Manual URI check inside `doFilter()`	Inside Filter execution	Small or custom runtime conditions
`OncePerRequestFilter.shouldNotFilter()`	Spring Filter hook	Clean exclusions in a Spring-specific Filter

Coder Army

Coder Army

General guidance

Use `FilterRegistrationBean` when the Filter should not be registered for unrelated paths at all.

Use `shouldNotFilter()` when using `OncePerRequestFilter` and the inclusion or exclusion rule naturally belongs inside that Filter.

Use manual checks only when they remain simple or depend on custom request information.

---

20. `Filter` vs `OncePerRequestFilter`

Point	`Filter`	`OncePerRequestFilter`
API	Jakarta Servlet API	Spring Framework
Request type	`ServletRequest`	`HttpServletRequest`
Response type	`ServletResponse`	`HttpServletResponse`
Casting required	Usually yes	No
Main method	`doFilter()`	`doFilterInternal()`
Built-in skip hook	No dedicated Spring hook	`shouldNotFilter()`
Dispatch protection	Must manage manually if needed	Built-in once-per-dispatch mechanism
Portability outside Spring	Greater	Spring-specific

Both are valid. In a Spring Boot HTTP application, `OncePerRequestFilter` often produces cleaner code.

---

21. Practical Guidelines

Prefer a Filter for

- Low-level request and response logging
- Correlation or trace IDs

Coder Army

- Cross-origin handling when not using existing framework support
- Compression or encoding concerns
- Security-chain integration
- Request timing across the complete servlet flow
- HTTP-boundary concerns that apply before Spring MVC

Avoid using a Filter for

- Core business logic
- Database operations unrelated to the HTTP boundary
- Normal DTO mapping
- Controller-specific behaviour
- Complex response restructuring when `ResponseBodyAdvice` is sufficient
- Authentication systems that should use Spring Security
- Blind logging of passwords, tokens, or complete request bodies

---

22. Final Mental Model

```
Headers
  → Easy to read
  → Response headers are easy to add
  → Request headers require a wrapper to appear modified

Query parameters
  → Easy to read
  → Require a wrapper to appear modified

Request body
  → Arrives as a stream
  → Normally consumed once
  → Requires caching and a wrapper for repeated reading

Response body
  → Written to an output stream
  → Requires a response wrapper to inspect or replace later
```

Coder Army

```
Filter registration
  → @Component for simple broad registration
  → FilterRegistrationBean for explicit registration control
  → OncePerRequestFilter for a Spring-oriented Filter base class
```

23. Key Takeaways

1. A Filter can stop a request by returning without calling `chain.doFilter()`.
2. Add important response headers before the response becomes committed.
3. A normal `HttpServletResponse` does not expose the completed body through `getBody()`.
4. `ContentCachingResponseWrapper` allows the response body to be inspected after downstream processing.
5. Always call `copyBodyToResponse()` when using `ContentCachingResponseWrapper` to preserve the final response.
6. The original `HttpServletRequest` does not provide setters for headers or query parameters.
7. Request wrappers change what downstream components see without changing the original request object.
8. Reading a request body in a Filter can consume the stream before the controller receives it.
9. Re-readable request bodies require caching and an overridden `getInputStream()` and `getReader()`.
10. Buffering request or response bodies has memory, performance, privacy, and security costs.
11. `FilterRegistrationBean` provides URL-pattern and ordering control.
12. Avoid combining `@Component` registration and manual registration for the same Filter unless duplicate execution is intentional.
13. `OncePerRequestFilter` is often more convenient for Spring Boot applications.
14. Use `shouldNotFilter()` for clean request exclusions in a `OncePerRequestFilter`.

15. Use the narrowest and most appropriate Spring layer for each concern.

Coder Army

Coder Army

Spring Boot Interceptors

Spring MVC interceptors allow us to execute common logic around controller execution without placing the same code inside every controller method.

They are useful when the logic depends on the **Spring MVC handler** that has been selected, such as the controller class, controller method, or annotations present on that method.

---

1. The Problem Interceptors Solve

Consider a controller with multiple endpoints:

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    @GetMapping
    public String getAllStudents() {
        return "All students";
    }

    @PostMapping
    public String createStudent() {
        return "Student created";
    }

    @DeleteMapping("/{id}")
    public String deleteStudent(@PathVariable Long id) {
        return "Student deleted";
    }
}
```

Suppose every request needs some common operations:

- Log the requested URL

Coder Army

- Record execution time
- Check authentication or authorization
- Read common request information
- Identify the controller method being executed
- Add request attributes

Adding this logic directly to every controller method creates several problems:

1. Controller methods become cluttered.
2. The same code is duplicated across endpoints.
3. A small change must be repeated in several places.
4. Business logic becomes mixed with infrastructure logic.

An interceptor provides one central place for such cross-cutting MVC logic.

> **Definition:** A Spring MVC interceptor is a component that executes before a handler, after a handler, and after the complete MVC request-processing cycle.

```
Request
   ↓
Interceptor
   ↓
Controller
   ↓
Interceptor
   ↓
Response
```

An interceptor does not normally replace the controller. It surrounds its execution.

---

2. Where an Interceptor Fits in Spring MVC

The `DispatcherServlet` is the central servlet of Spring MVC.

When a request reaches a Spring MVC application:

1. The `DispatcherServlet` receives the request.
2. A `HandlerMapping` identifies the matching handler.

Coder Army

3. Spring creates a handler execution chain.
4. Registered interceptors execute around the handler.
5. A `HandlerAdapter` invokes the controller method.
6. Spring processes the controller's return value.
7. The response is sent to the client.

```
Client
   ↓
Servlet Container
   ↓
DispatcherServlet
   ↓
HandlerMapping finds the handler
   ↓
Interceptor
   ↓
HandlerAdapter invokes the controller
   ↓
Response processing
   ↓
Client
```

Why is it called `HandlerInterceptor`?

Spring uses the general term **handler** because the selected handler is not always a controller method. It may be:

- A controller method
- A static-resource handler
- Another handler type supported by Spring MVC

For normal controller requests, the handler is commonly represented by a `HandlerMethod`.

`HandlerExecutionChain`

When Spring finds a handler, it conceptually creates a chain containing the handler and its interceptors:

Coder Army

```
HandlerExecutionChain
├── Interceptor 1
├── Interceptor 2
├── Interceptor 3
└── Controller handler
```

The `DispatcherServlet` executes this chain. An interceptor can therefore allow or stop the request before the controller runs.

3. The Three Interceptor Lifecycle Methods

A `HandlerInterceptor` provides three main callback methods:

```
preHandle()
postHandle()
afterCompletion()
```

Their overall flow is:

```
Request
   ↓
preHandle()
   ↓
Controller method
   ↓
postHandle()
   ↓
Response processing or view rendering
   ↓
afterCompletion()
   ↓
Response
```

Method	Execution point	Main purpose
`preHandle()`	Before the selected handler executes	Validation, logging, authentication checks, starting timers
`postHandle()`	After successful handler execution, before traditional view rendering	Model or view-related post-processing

Method	Execution point	Main purpose
`afterCompletion()`	After MVC request processing finishes	Cleanup, final logging, timing, status inspection

Coder Army

4. Creating a Custom Interceptor

Create a Spring component that implements `HandlerInterceptor`:

```
package in.strikes.crudDemo.interceptor;

import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class LoggingInterceptor implements HandlerInterceptor
{
}
```

Modern versions of `HandlerInterceptor` provide default method implementations. Therefore, an interceptor only needs to override the callbacks it actually uses.

A complete basic interceptor looks like this:

```
package in.strikes.crudDemo.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.lang.Nullable;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.ModelAndView;

@Component
public class LoggingInterceptor implements HandlerInterceptor
{
```

Coder Army

```
    @Override
    public boolean preHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler
    ) throws Exception {

        System.out.println("preHandle executed");
        return true;
    }

    @Override
    public void postHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            @Nullable ModelAndView modelAndView
    ) throws Exception {

        System.out.println("postHandle executed");
    }

    @Override
    public void afterCompletion(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            @Nullable Exception ex
    ) throws Exception {

        System.out.println("afterCompletion executed");
    }
}
```

For an interceptor that only checks something before the controller, overriding only `preHandle()` is sufficient:

Coder Army

```
@Component
public class AuthenticationInterceptor
        implements HandlerInterceptor {

    @Override
    public boolean preHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler
    ) throws Exception {

        return true;
    }
}
```

5. Understanding the Method Parameters

`HttpServletRequest request`

Represents the incoming HTTP request.

Commonly used methods include:

```
request.getMethod();
request.getRequestURI();
request.getQueryString();
request.getHeader("Authorization");
request.getRemoteAddr();
request.getParameter("name");
request.getSession();
```

Request-specific values can also be stored as attributes:

```
request.setAttribute("requestStartTime", System.nanoTime());
```

Coder Army

They can later be retrieved using:

```
Long startTime =
        (Long) request.getAttribute("requestStartTime");
```

`HttpServletResponse response`

Represents the outgoing HTTP response.

It can be used to set the status, headers, content type, or response content:

```
response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
response.setHeader("X-App-Name", "Student Management Syste
m");
response.setContentType("application/json");
response.getWriter().write("""
        {
          "message": "Unauthorized"
        }
        """);
```

Once a response is committed, some changes may be too late. This is especially important when working with REST response bodies.

`Object handler`

Represents the handler selected by Spring.

For a normal controller request, it is commonly a `HandlerMethod`:

```
if (handler instanceof HandlerMethod handlerMethod) {

    String controllerName =
            handlerMethod.getBeanType().getSimpleName();

    String methodName =
            handlerMethod.getMethod().getName();
```

Coder Army

```
    System.out.println(controllerName);
    System.out.println(methodName);
}
```

Possible output:

```
StudentController
getAllStudents
```

Do not cast the handler blindly:

```
HandlerMethod handlerMethod = (HandlerMethod) handler;
```

Some requests may be handled by something other than a controller method, such as a static-resource handler. Use pattern matching with `instanceof` instead:

```
if (handler instanceof HandlerMethod handlerMethod) {
    // Safe to use handlerMethod
}
```

This ability to inspect the selected controller and method is one of the major differences between an interceptor and a servlet filter.

---

6. Understanding `preHandle()`

Signature:

```
boolean preHandle(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler
) throws Exception
```

It executes after Spring has selected the handler but before the handler is invoked.

Why does it return `boolean`?

Coder Army

The return value tells Spring whether request processing should continue.

Returning `true`

```
return true;
```

Spring continues to the next interceptor or the controller.

Returning `false`

```
return false;
```

Spring stops the execution chain, and the controller is not called.

When returning `false`, the interceptor should usually prepare an appropriate response. Otherwise, the client may receive an empty or confusing response.

```
response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
response.setContentType("application/json");
response.getWriter().write("""
        {
          "message": "Authentication required"
        }
        """);

return false;
```

Common uses of `preHandle()`

- Logging request arrival
- Starting an execution timer
- Reading a tenant or organization ID
- Creating a correlation ID
- Adding request attributes

Coder Army

- Validating custom headers
- Checking custom annotations
- Rejecting a request before the controller

> For production-grade authentication and authorization, prefer **Spring Security** instead of building a complete security system through a custom interceptor.

---

7. Understanding `postHandle()`

Signature:

```
void postHandle(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler,
        ModelAndView modelAndView
) throws Exception
```

It executes after the handler completes successfully but before traditional MVC view rendering.

```
preHandle()
   ↓
Controller
   ↓
postHandle()
   ↓
View rendering
```

Why does it receive `ModelAndView`?

Traditional Spring MVC controllers may return a view. Spring stores the selected view and model data inside a `ModelAndView`.

An interceptor can add common model information:

Coder Army

```
@Override
public void postHandle(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler,
        ModelAndView modelAndView
) {

    if (modelAndView != null) {
        modelAndView.addObject(
                "applicationName",
                "Student Management System"
        );
    }
}
```

What happens with REST controllers?

A `@RestController` normally returns data that Spring serializes directly into the response body:

```
@RestController
public class StudentController {

    @GetMapping("/students")
    public List<Student> getStudents() {
        return studentService.getAllStudents();
    }
}
```

In such cases:

```
modelAndView == null
```

Coder Army

For methods using `@ResponseBody` or `ResponseEntity`, Spring may already have written the response before `postHandle()` executes. Therefore, `postHandle()` is not a reliable place to redesign or wrap a REST response body.

Use `ResponseBodyAdvice` when a common REST response transformation is required:

```
@ControllerAdvice
public class GlobalResponseAdvice
        implements ResponseBodyAdvice<Object> {
}
```

Good uses of `postHandle()`

- Updating a traditional MVC model
- Adding common view attributes
- Logging successful controller completion
- Inspecting the selected handler
- Running logic before view rendering

For REST APIs, `afterCompletion()` is often more useful for final logging and timing.

---

8. Understanding `afterCompletion()`

Signature:

```
void afterCompletion(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler,
        Exception ex
) throws Exception
```

It executes after the complete Spring MVC request-processing cycle finishes.

```
preHandle()
   ↓
```

Coder Army

```
Controller
   ↓
postHandle()
   ↓
Response processing or view rendering
   ↓
afterCompletion()
```

It is similar to a `finally` block and is commonly used for:

- Cleanup
- Final request logging
- Execution-time calculation
- Releasing request-related resources
- Recording the final response status
- Inspecting an unhandled exception

Example:

```java
@Override
public void afterCompletion(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler,
        Exception ex
) {

    System.out.println(
            "Request completed with status: "
                    + response.getStatus()
    );
}
```

Inspecting exceptions

Coder Army

```
@Override
public void afterCompletion(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler,
        Exception ex
) {

    if (ex != null) {
        System.out.println(
                "Request failed: " + ex.getMessage()
        );
    } else {
        System.out.println("Request completed");
    }
}
```

An important limitation is that `ex == null` does not always prove that no exception occurred.

An exception may have been handled by Spring through:

- `@ExceptionHandler`
- `@ControllerAdvice`
- `@RestControllerAdvice`
- Another exception resolver

The exception may already have been converted into a valid HTTP response. Therefore, final monitoring should also inspect:

```
response.getStatus();
```

9. Registering an Interceptor with `WebMvcConfigurer`

Coder Army

Creating an interceptor bean is not enough. It must also be added to Spring MVC's interceptor registry.

```
package in.strikes.crudDemo.config;

import in.strikes.crudDemo.interceptor.LoggingInterceptor;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.Inte
rceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebM
vcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    private final LoggingInterceptor loggingInterceptor;

    public WebConfig(LoggingInterceptor loggingInterceptor) {
        this.loggingInterceptor = loggingInterceptor;
    }

    @Override
    public void addInterceptors(
            InterceptorRegistry registry
    ) {
        registry.addInterceptor(loggingInterceptor);
    }
}
```

Role of each part

- `@Configuration` marks the class as an application configuration class.
- `WebMvcConfigurer` provides callback methods for customizing Spring MVC.
- `addInterceptors()` is used to register MVC interceptors.

CoderArmy

- `registry.addInterceptor()` adds an interceptor to the MVC handler chain.
- Constructor injection provides the interceptor bean to the configuration class.

The `addInterceptor()` method returns an `InterceptorRegistration`, which can be used to configure:

- Included URL patterns
- Excluded URL patterns
- Execution order

---

10. Logging Request Details

A practical logging interceptor can access both HTTP information and the selected controller method.

```
package in.strikes.crudDemo.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.method.HandlerMethod;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class RequestLoggingInterceptor
        implements HandlerInterceptor {

    @Override
    public boolean preHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler
    ) {

        System.out.println("----- Incoming Request -----");
        System.out.println("HTTP Method: " + request.getMetho
```

```
d());
        System.out.println("Request URI: " + request.getReque
stURI());
        System.out.println("Query String: " + request.getQuer
yString());
        System.out.println("Client IP: " + request.getRemoteA
ddr());

        if (handler instanceof HandlerMethod handlerMethod) {

            String controllerName =
                    handlerMethod
                            .getBeanType()
                            .getSimpleName();

            String methodName =
                    handlerMethod
                            .getMethod()
                            .getName();

            System.out.println("Controller: " + controllerNam
e);
            System.out.println("Controller Method: " + method
Name);
        }

        return true;
    }
}
```

`System.out.println()` is acceptable for learning examples. In a real application, use a logging framework such as SLF4J:

```
private static final Logger log =
        LoggerFactory.getLogger(RequestLoggingInterceptor.cla
```

Coder Army

```
ss);
```

Coder Army

11. Authentication Demonstration with an API Key

The following example demonstrates how an interceptor can stop a request. It is useful for understanding the flow but should not replace Spring Security in a production application.

Requirement

The client sends:

```
X-API-KEY: student-secret-key
```

- Correct key: continue to the controller
- Missing or incorrect key: return `401 Unauthorized`
- Controller must not execute

Interceptor

```
package in.strikes.crudDemo.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.HandlerInterceptor;

@Component
public class ApiKeyAuthenticationInterceptor
        implements HandlerInterceptor {

    private static final String VALID_API_KEY =
            "student-secret-key";
```

Coder Army

```
    @Override
    public boolean preHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler
    ) throws Exception {

        String apiKey =
                request.getHeader("X-API-KEY");

        if (apiKey == null) {
            writeUnauthorizedResponse(
                    response,
                    "API key is missing"
            );
            return false;
        }

        if (!VALID_API_KEY.equals(apiKey)) {
            writeUnauthorizedResponse(
                    response,
                    "Invalid API key"
            );
            return false;
        }

        return true;
    }

    private void writeUnauthorizedResponse(
            HttpServletResponse response,
            String message
    ) throws Exception {

        response.setStatus(
                HttpServletResponse.SC_UNAUTHORIZED
```

Coder Army

```
        );

        response.setContentType(
                MediaType.APPLICATION_JSON_VALUE
        );

        response.getWriter().write(
                """
                {
                  "message": "%s"
                }
                """.formatted(message)
        );
    }
}
```

The response is written before returning `false` because the controller will not execute and will not create a response.

Registering it for selected paths

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    private final ApiKeyAuthenticationInterceptor
            authenticationInterceptor;

    public WebConfig(
            ApiKeyAuthenticationInterceptor authenticationInt
erceptor
    ) {
        this.authenticationInterceptor =
                authenticationInterceptor;
    }

    @Override
```

```
    public void addInterceptors(
            InterceptorRegistry registry
    ) {
        registry.addInterceptor(authenticationInterceptor)
                .addPathPatterns("/api/**")
                .excludePathPatterns(
                        "/api/auth/login",
                        "/api/public/**"
                );
    }
}
```

12. Authorization Demonstration

Suppose only users with the `ADMIN` role can access a delete operation.

The client sends:

```
X-USER-ROLE: ADMIN
```

A simplified interceptor check could be:

```
@Override
public boolean preHandle(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler
) throws Exception {

    if ("DELETE".equalsIgnoreCase(request.getMethod())) {

        String role =
                request.getHeader("X-USER-ROLE");

        if (!"ADMIN".equals(role)) {
```

Coder Army

Coder Army

```
            response.setStatus(
                    HttpServletResponse.SC_FORBIDDEN
            );

            response.setContentType(
                    MediaType.APPLICATION_JSON_VALUE
            );

            response.getWriter().write(
                    """
                    {
                      "message": "Admin access required"
                    }
                    """
            );

            return false;
        }
    }

    return true;
}
```

This is a learning example. Real authorization rules should normally be implemented through Spring Security.

`401` vs `403`

Status	Meaning	Example
`401 Unauthorized`	The client has not been successfully authenticated	Missing, invalid, or expired token
`403 Forbidden`	The client is authenticated but lacks permission	A normal user attempts an admin-only operation

Coder Army

```
HttpServletResponse.SC_UNAUTHORIZED // 401
HttpServletResponse.SC_FORBIDDEN    // 403
```

13. Measuring Request Execution Time

The basic requirement is:

```
Record start time
      ↓
Execute the controller
      ↓
Calculate total duration
```

Incorrect approach: using an instance field

```
@Component
public class TimingInterceptor
        implements HandlerInterceptor {

    private long startTime;

    @Override
    public boolean preHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler
    ) {

        startTime = System.currentTimeMillis();
        return true;
    }
}
```

This is unsafe because Spring components are singleton-scoped by default. Multiple requests may use the same interceptor instance concurrently.

Coder Army

```
Request A starts → startTime = 1000
Request B starts → startTime = 1100
Request A ends   → incorrectly uses 1100
```

Request-specific mutable data should not be stored in a normal interceptor instance field.

Correct approach: use request attributes

Each request has its own `HttpServletRequest` object. Store the starting time in the request:

```
request.setAttribute(
        "requestStartTime",
        System.nanoTime()
);
```

Retrieve it in `afterCompletion()`:

```
Long startTime =
        (Long) request.getAttribute("requestStartTime");
```

Complete timing interceptor

```
package in.strikes.crudDemo.interceptor;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.method.HandlerMethod;
import org.springframework.web.servlet.HandlerInterceptor;

import java.util.concurrent.TimeUnit;

@Component
```

Coder Army

```
public class RequestTimingInterceptor
        implements HandlerInterceptor {

    private static final String START_TIME_ATTRIBUTE =
            "requestStartTime";

    @Override
    public boolean preHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler
    ) {

        request.setAttribute(
                START_TIME_ATTRIBUTE,
                System.nanoTime()
        );

        return true;
    }

    @Override
    public void afterCompletion(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            Exception ex
    ) {

        Long startTime =
                (Long) request.getAttribute(
                        START_TIME_ATTRIBUTE
                );

        if (startTime == null) {
            return;
```

```
        }

        long durationNanos =
                System.nanoTime() - startTime;

        long durationMillis =
                TimeUnit.NANOSECONDS.toMillis(
                        durationNanos
                );

        String handlerName = "UnknownHandler";

        if (handler instanceof HandlerMethod handlerMethod) {
            handlerName =
                    handlerMethod
                            .getBeanType()
                            .getSimpleName()
                            + "."
                            + handlerMethod
                            .getMethod()
                            .getName();
        }

        System.out.println(
                "Request: "
                        + request.getMethod()
                        + " "
                        + request.getRequestURI()
        );

        System.out.println(
                "Handler: " + handlerName
        );

        System.out.println(
                "Status: " + response.getStatus()
```

Coder Army

```
        );

        System.out.println(
                "Execution time: "
                        + durationMillis
                        + " ms"
        );
    }
}
```

Coder Army

Why use `afterCompletion()`?

`postHandle()` runs after successful handler execution.

`afterCompletion()` measures closer to the end of the complete MVC lifecycle, including later processing such as exception resolution and traditional view handling.

Why use `System.nanoTime()`?

- `System.currentTimeMillis()` represents wall-clock time.
- The system clock can be adjusted.
- `System.nanoTime()` is designed for measuring elapsed time.

---

14. Applying an Interceptor to URL Patterns

Without path configuration:

```
registry.addInterceptor(loggingInterceptor);
```

the interceptor may apply broadly to requests handled by Spring MVC's configured handler mappings.

One URL pattern

Coder Army

```
@Override
public void addInterceptors(
        InterceptorRegistry registry
) {
    registry.addInterceptor(loggingInterceptor)
            .addPathPatterns("/api/students/**");
}
```

Multiple URL patterns

```
registry.addInterceptor(loggingInterceptor)
        .addPathPatterns(
                "/api/students/**",
                "/api/teachers/**",
                "/api/courses/**"
        );
```

Common patterns

Pattern	Meaning
`/api/students`	Exact path
`/api/students/*`	One path segment after `/students`
`/api/students/**`	Any nested path under `/students`
`/api/**`	All API paths
`/**`	All matching MVC paths

15. Excluding URL Patterns

Suppose authentication should apply to all API routes except login and registration:

```
@Override
public void addInterceptors(
```

Coder Army

```
        InterceptorRegistry registry
) {
    registry.addInterceptor(authenticationInterceptor)
            .addPathPatterns("/api/**")
            .excludePathPatterns(
                    "/api/auth/login",
                    "/api/auth/register"
            );
}
```

Included and excluded patterns can be combined on the same interceptor registration.

---

16. Multiple Interceptors and Execution Order

Suppose two interceptors, `A` and `B`, are registered in that order.

Their successful execution flow is:

```
A.preHandle()
   ↓
B.preHandle()
   ↓
Controller
   ↓
B.postHandle()
   ↓
A.postHandle()
   ↓
Response processing or view rendering
   ↓
B.afterCompletion()
   ↓
A.afterCompletion()
```

The pattern is:

- `preHandle()` executes in forward order.
- `postHandle()` executes in reverse order.
- `afterCompletion()` executes in reverse order.

Coder Army

This resembles nested method calls:

```
A opens
    B opens
        Controller executes
    B closes
A closes
```

Configuring the order

```
@Override
public void addInterceptors(
        InterceptorRegistry registry
) {

    registry.addInterceptor(loggingInterceptor)
            .order(1);

    registry.addInterceptor(authInterceptor)
            .order(2);

    registry.addInterceptor(timingInterceptor)
            .order(3);
}
```

Lower order values have higher priority and execute earlier during `preHandle()`.

> Only interceptors whose `preHandle()` successfully participated in the chain are eligible for the corresponding completion callbacks.

---

17. Filters and Interceptors Together

A filter surrounds servlet processing. An interceptor surrounds a Spring MVC handler.

Coder Army

```
HTTP Request
     ↓
Servlet Container
     ↓
Filter 1: before chain.doFilter()
     ↓
Filter 2: before chain.doFilter()
     ↓
DispatcherServlet
     ↓
HandlerMapping finds controller
     ↓
Interceptor 1: preHandle()
     ↓
Interceptor 2: preHandle()
     ↓
Controller
     ↓
Interceptor 2: postHandle()
     ↓
Interceptor 1: postHandle()
     ↓
Response processing or view rendering
     ↓
Interceptor 2: afterCompletion()
     ↓
Interceptor 1: afterCompletion()
     ↓
Filter 2: after chain.doFilter()
     ↓
Filter 1: after chain.doFilter()
     ↓
HTTP Response
```

Core difference

- A **filter** works at the Servlet layer.
- An **interceptor** works inside Spring MVC after a handler has been selected.

Request and response wrapping

A filter can wrap or replace the request and response objects passed further into the chain:

```
chain.doFilter(wrappedRequest, wrappedResponse);
```

Coder Army

This makes filters suitable for:

- Caching a request body
- Reading a request body multiple times
- Capturing a response body
- Transforming request or response data
- Compression and encoding concerns

An interceptor receives the request and response after processing has already entered Spring MVC. It cannot replace the objects used by the entire servlet chain in the same way.

18. Filter vs Interceptor

Point	Filter	Interceptor
Layer	Servlet layer	Spring MVC layer
Main target	Servlet request and response	Selected MVC handler
Knows controller method	Normally no	Yes, through `HandlerMethod`
Main before callback	Code before `chain.doFilter()`	`preHandle()`
Main after callback	Code after `chain.doFilter()`	`postHandle()` and `afterCompletion()`
Can wrap request/response	Yes	Not in the same chain-wide manner
Can block processing	Yes	Yes, by returning `false` from `preHandle()`
Suitable for body wrapping	Yes	No
Suitable for controller annotations	No	Yes
Common security mechanism	Spring Security filter chain	Not a replacement for Spring Security

Coder Army

19. When to Use a Filter

Use a filter when the concern applies to the complete HTTP or servlet request rather than specifically to a controller method.

Common examples:

1. Request or response body wrapping
2. Request-body caching
3. Response-body capture or transformation
4. Character encoding
5. Compression
6. CORS-related processing
7. Logging all servlet traffic
8. Security processing before Spring MVC

Spring Security uses servlet filters because authentication and authorization generally need to happen before a request reaches the controller layer.

---

20. When to Use an Interceptor

Use an interceptor when the requirement depends on Spring MVC or the selected handler.

Common examples:

1. Logging controller and method names
2. Reading custom annotations from controller methods
3. Starting and completing controller-related timing
4. Adding common request attributes
5. Adding common model attributes in traditional MVC
6. Collecting MVC-specific metrics
7. Applying tenant or organization context based on handler rules
8. Running logic before and after a particular group of controller endpoints

Coder Army

21. Common Mistakes

Mistake 1: Creating an interceptor but not registering it

`@Component` creates the bean, but `addInterceptors()` places it in the Spring MVC chain.

Mistake 2: Returning `false` without writing a response

When `preHandle()` returns `false`, the controller does not execute. Prepare an appropriate status and body before stopping the chain.

Mistake 3: Storing request data in an interceptor field

Interceptors are normally singleton beans. Store request-specific values in request attributes instead.

Mistake 4: Blindly casting `handler`

Not every handler is a `HandlerMethod`. Always check with `instanceof`.

Mistake 5: Modifying a REST body inside `postHandle()`

The response may already have been written. Use `ResponseBodyAdvice` or a filter-based wrapper, depending on the requirement.

Mistake 6: Building production security manually

A custom API-key interceptor is useful for learning request flow, but production authentication and authorization should normally use Spring Security.

Mistake 7: Assuming `ex == null` means no error occurred

A global exception handler may already have handled the exception. Also inspect the final HTTP status.

22. Final Mental Model

A servlet filter asks:

> What should happen around the complete HTTP request before or after it reaches a servlet?

A Spring MVC interceptor asks:

> What should happen around the selected Spring MVC handler before and after the controller executes?

```
Filter → surrounds DispatcherServlet and servlet processing

Interceptor → surrounds the selected Spring MVC handler
```

The most important lifecycle rule is:

```
preHandle()       → before the controller
postHandle()      → after successful controller execution
afterCompletion() → after MVC request processing finishes
```

And with multiple interceptors:

```
Forward order before the controller
Reverse order after the controller
```

Coder Army

Coder Army

Introduction to Spring AOP

> **Core idea:** Spring AOP allows us to apply common behaviour—such as logging, security, auditing, and performance measurement—across multiple classes without mixing that behaviour into every business method.

---

1. A Simple Business Method

Consider a student-management application:

```
@Service
public class StudentService {

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepositor
y) {
        this.studentRepository = studentRepository;
    }

    public Student createStudent(Student student) {
        return studentRepository.save(student);
    }
}
```

The responsibility of `createStudent()` is easy to understand:

```
Receive student
      ↓
Save student
      ↓
Return saved student
```

This is the **business logic** of the method.

Coder Army

A business method answers a domain-related question, such as:

- How is a student created?
- How is an order placed?
- How is money transferred?
- How is inventory updated?
- How is a payment processed?

The method is currently focused on one responsibility, so its intent is immediately visible.

---

2. Real Applications Need More Than Business Logic

A production application may also need to:

- Log when a method begins
- Log whether it succeeds or fails
- Measure execution time
- Check whether the caller has permission
- Record an audit event
- Start, commit, or roll back a transaction
- Collect application metrics
- Retry temporary failures
- Cache a result
- Mask sensitive information

These requirements are important, but they are not part of the core process of creating a student.

If all of them are added directly to `createStudent()`, the method becomes difficult to read because most of its code no longer represents student-related business logic.

Coder Army

3. Business Logic vs Supporting Logic

A typical application contains two broad categories of responsibility.

Business Logic

Business logic represents the rules and operations of the application's domain.

Examples:

- Creating a student
- Placing an order
- Processing a payment
- Updating inventory
- Approving a loan

Business logic differs from one feature or domain to another.

Supporting or Infrastructure Logic

Supporting logic helps the application operate safely, reliably, securely, and observably.

Examples:

- Logging
- Security checks
- Transaction management
- Auditing
- Performance monitoring
- Caching

This logic is usually not specific to students, orders, payments, or inventory. The same supporting behaviour may be required across many unrelated features.

---

4. What Is a Concern?

A **concern** is a responsibility or area of interest within an application.

Coder Army

Examples:

- Student management is one concern.
- Payment processing is another concern.
- Logging is another concern.
- Security is another concern.
- Transaction management is another concern.

Core Business Concerns

Core business concerns normally belong to a particular feature or module.

```
Student concern  → StudentService
Order concern    → OrderService
Payment concern  → PaymentService
```

These concerns are organized **vertically**. Each feature has its own classes and methods.

Cross-Cutting Concerns

Some concerns are required across many business modules.

```
                    Logging
                       ↓
StudentService ← Security → OrderService
                       ↓
                PaymentService
                       ↓
                    Auditing
```

These concerns cut horizontally across otherwise unrelated features. They are therefore called **cross-cutting concerns**.

> A concern becomes cross-cutting when the same kind of behaviour must be applied in many different parts of the application.

Coder Army

5. Common Examples of Cross-Cutting Concerns

Logging

The application may need to record:

- Which method was called
- When it started
- Whether it completed successfully
- Whether it failed
- Which arguments were supplied

The same logging requirement may apply to student, order, payment, inventory, and user services.

Security

Suppose only administrators are allowed to delete students.

A permission check must run before the delete operation. Similar checks may also be needed before refunding payments, changing user roles, or approving requests.

Performance Measurement

Suppose we want to identify slow methods.

A timer must run before the method, and the elapsed time must be calculated after the method finishes. Without a reusable mechanism, this timer would have to be added manually to every method that needs monitoring.

Auditing

Certain business operations may need a permanent record, including:

- Who performed the operation
- What operation was performed
- When it happened
- Whether it succeeded or failed

Coder Army

Transaction Management

Database operations may require a transaction to begin before the business method and then commit or roll back after the method completes.

---

6. The Two Major Problems: Scattering and Tangling

Cross-cutting concerns create two major architectural problems when they are implemented directly inside business classes.

6.1 Scattering

Scattering means that the implementation of one concern is spread across many locations.

For example, logging code may appear in:

```
StudentService
OrderService
PaymentService
InventoryService
UserService
ReportService
```

The logging concern is now scattered throughout the application.

Imagine an application containing 20 service classes with five methods each. Logging, auditing, or performance code may have to be added and maintained in approximately 100 different methods.

If the logging format or auditing policy changes, many classes may need to be modified.

Problems Caused by Scattering

- The same concern is repeated in many places.
- Developers may implement it differently.
- A required method may be missed.
- A small policy change may require edits across the entire codebase.

- Maintenance becomes expensive.

6.2 Tangling

Tangling means that a single method contains code belonging to multiple unrelated concerns.

```
public Payment processPayment(PaymentRequest request) {

    // Logging concern
    log.info("Processing payment");

    // Security concern
    securityService.checkPermission("PROCESS_PAYMENT");

    // Performance concern
    long start = System.currentTimeMillis();

    try {
        // Business concern
        Payment payment = gateway.charge(request);

        // Auditing concern
        auditService.record("PAYMENT_COMPLETED");

        return payment;

    } catch (Exception exception) {
        // Logging concern
        log.error("Payment failed", exception);

        // Auditing concern
        auditService.record("PAYMENT_FAILED");

        throw exception;
```

Coder Army

```
    } finally {
        // Performance concern
        metrics.record(
                System.currentTimeMillis() - start
        );
    }
}
```

Coder Army

This one method is responsible for:

- Business logic
- Logging
- Security
- Auditing
- Performance measurement
- Exception-related behaviour

The problem is not that these responsibilities are unnecessary. The problem is that they are implemented together in the same method.

Problems Caused by Tangling

1. The business intent becomes difficult to see.
2. Supporting behaviour may be applied inconsistently.
3. Changes become more expensive.
4. Developers can forget mandatory behaviour.
5. Testing becomes more complicated.
6. The method becomes harder to maintain.

Compare the two versions below.

Tangled Version

Coder Army

```java
public Student createStudent(Student student) {

    log.info("Method started");
    securityService.checkPermission("CREATE_STUDENT");

    long start = System.currentTimeMillis();

    try {
        Student result = studentRepository.save(student);

        auditService.record("STUDENT_CREATED");

        return result;

    } catch (Exception exception) {
        log.error("Method failed", exception);
        throw exception;

    } finally {
        metrics.record(
                System.currentTimeMillis() - start
        );
    }
}
```

Focused Version

```java
public Student createStudent(Student student) {
    return studentRepository.save(student);
}
```

The second version communicates the business operation immediately. The challenge is to keep the method this clean while still applying the required supporting behaviour.

Coder Army

7. First Attempt: Utility Methods

A common first solution is to centralize repeated code inside a utility class.

```
public class LoggingUtil {

    public static void logStart(String methodName) {
        System.out.println(methodName + " started");
    }

    public static void logEnd(String methodName) {
        System.out.println(methodName + " completed");
    }
}
```

The service can then call the utility methods:

```
public Student createStudent(Student student) {

    LoggingUtil.logStart("createStudent");

    Student result = studentRepository.save(student);

    LoggingUtil.logEnd("createStudent");

    return result;
}
```

This is better than duplicating the complete logging implementation, but it does not solve the fundamental problem.

The Invocation Is Still Scattered

Every business method must still contain calls such as:

```
LoggingUtil.logStart(...);
```

```
LoggingUtil.logEnd(...);
```

The logging implementation is centralized, but the decision about **where and when to invoke it** remains scattered across business methods.

Developers Must Remember to Call It

Nothing automatically guarantees that every required method invokes the utility.

A developer may:

- Forget to call `logStart()`
- Call `logStart()` but forget `logEnd()`
- Use inconsistent method names or messages
- Handle failure differently in different methods

Success and Failure Paths Still Require Boilerplate

```
LoggingUtil.logStart("createStudent");

try {
    Student result = studentRepository.save(student);

    LoggingUtil.logSuccess("createStudent");

    return result;

} catch (Exception exception) {

    LoggingUtil.logFailure(
            "createStudent",
            exception
    );

    throw exception;
}
```

Coder Army

Coder Army

The utility avoids repeating the internal logging implementation, but every business method still contains control-flow boilerplate.

> Utility methods centralize **what the concern does**, but not **where and when the concern is applied**.

---

8. Second Attempt: Inheritance

Another possibility is to place common behaviour inside a base class.

```
public abstract class BaseService {

    protected void logStart(String methodName) {
        System.out.println(methodName + " started");
    }

    protected void logEnd(String methodName) {
        System.out.println(methodName + " completed");
    }
}
```

A service can extend this class:

```
@Service
public class StudentService extends BaseService {

    public Student createStudent(Student student) {

        logStart("createStudent");

        Student result = studentRepository.save(student);

        logEnd("createStudent");

        return result;
```

```
    }
}
```

Why Inheritance Does Not Solve the Problem

Inheritance shares the implementation of `logStart()` and `logEnd()`, but every method must still invoke them manually.

It also creates additional design problems:

- Java supports only single class inheritance.
- A service may already need to extend another class.
- Unrelated classes are forced into an artificial inheritance hierarchy.
- Logging, security, auditing, and other unrelated concerns may get mixed into one large base class.
- The parent-child relationship does not naturally represent these concerns.

> Inheritance provides access to shared behaviour; it does not automatically execute that behaviour around selected methods.

---

9. Third Attempt: Manual Wrappers

A stronger solution is the **wrapper** or **decorator** approach.

Start with an interface:

```
public interface StudentOperations {

    Student createStudent(Student student);
}
```

The real service implements the interface:

```
public class StudentService implements StudentOperations {

    private final StudentRepository studentRepository;
```

Coder Army

Coder Army

```
    public StudentService(StudentRepository studentRepositor
y) {
        this.studentRepository = studentRepository;
    }

    @Override
    public Student createStudent(Student student) {
        return studentRepository.save(student);
    }
}
```

Now create a wrapper that implements the same interface:

```
public class LoggingStudentService
        implements StudentOperations {

    private final StudentOperations target;

    public LoggingStudentService(StudentOperations target) {
        this.target = target;
    }

    @Override
    public Student createStudent(Student student) {

        long start = System.currentTimeMillis();

        System.out.println("Method started");

        try {
            return target.createStudent(student);
        } finally {
            System.out.println(
                    "Execution time: "
                            + (System.currentTimeMillis() - s
```

```
tart)
                );
            }
        }
}
```

Coder Army

The caller receives the wrapper instead of directly receiving the real service:

```
Controller
    ↓
LoggingStudentService
    ↓
StudentService
```

The original business class remains clean because the wrapper surrounds the real method invocation.

The wrapper performs three fundamental actions:

```
Run supporting behaviour before
              ↓
Call the real business object
              ↓
Run supporting behaviour after
```

This is close to the basic execution model used by Spring AOP.

> AOP did not invent the idea of wrapping behaviour. It provides infrastructure for creating and applying such wrappers systematically.

---

10. Limitations of Manual Wrappers

Manual wrappers can work in a small application, but they become expensive at scale.

A Wrapper Is Needed for Every Service Type

A wrapper written for `StudentOperations` cannot automatically wrap unrelated service interfaces such as `OrderOperations` or `PaymentOperations`.

Coder Army

Separate wrappers may be required for:

- Student services
- Order services
- Payment services
- Inventory services
- User services

Multiple Concerns Require Multiple Layers

Suppose a service needs:

- Security
- Logging
- Performance measurement
- Auditing

The object chain may become:

```
Controller
    ↓
SecurityStudentService
    ↓
LoggingStudentService
    ↓
PerformanceStudentService
    ↓
AuditingStudentService
    ↓
StudentService
```

Manual Object Construction Becomes Complicated

Someone must create all wrappers in the correct order and inject the final outer wrapper wherever the service is needed.

The Same Boilerplate Appears Again

Each wrapper contains similar delegation and exception-handling code. The concern has moved out of the business class, but wrapper creation and maintenance remain manual.

---

11. What a Better Solution Must Provide

A scalable solution should satisfy four requirements.

Requirement 1: Keep Common Behaviour in One Place

Logging, security, auditing, or performance logic should have a dedicated module rather than being repeated in business classes.

Requirement 2: Select Where the Behaviour Should Apply

We need a way to express rules such as:

```
Apply to all service methods
```

```
Apply only to methods annotated with @TrackExecutionTime
```

```
Apply to every method whose name starts with save
```

Requirement 3: Control When the Behaviour Should Run

Supporting logic may need to run:

- Before the method
- After the method
- Only after successful completion
- Only after an exception is thrown
- Around the complete method execution

Requirement 4: Avoid Modifying Every Business Method

Coder Army

Coder Army

We do not want every method to contain infrastructure calls:

```
public Student createStudent(Student student) {
    startLogging();
    startTimer();
    checkSecurity();

    // Business logic
}
```

The application should apply cross-cutting behaviour without requiring the caller or business method to manage it manually.

> The mechanism should remain largely transparent to the business code and its caller.

---

12. Introduction to Aspect-Oriented Programming

This is the problem that **Aspect-Oriented Programming**, or **AOP**, is designed to solve.

AOP provides a way to modularize cross-cutting concerns instead of mixing them into business classes.

The essential idea is:

```
Keep business logic in business classes
                +
Keep cross-cutting logic in a separate module
                +
Define where and when that logic should apply
                =
Combined behaviour at runtime
```

With AOP, the business method can remain focused:

```
public Student createStudent(Student student) {
    return studentRepository.save(student);
```

```
}
```

Logging, security, performance measurement, or auditing can be applied around it through separate AOP configuration.

---

13. OOP and AOP Solve Different Dimensions

Object-Oriented Programming and Aspect-Oriented Programming are not competitors. They organize different dimensions of an application.

OOP Organizes Business Capabilities Vertically

```
StudentService
├── createStudent
├── updateStudent
└── deleteStudent

OrderService
├── placeOrder
├── cancelOrder
└── getOrder

PaymentService
├── makePayment
├── refundPayment
└── getPayment
```

Each class groups behaviour related to a particular business capability.

AOP Organizes Behaviour Horizontally

```
                      Logging
                         ↓
StudentService ← Performance → OrderService
                         ↓
                   PaymentService
                         ↓
                      Auditing
```

AOP captures behaviour that spans several business capabilities.

Coder Army

Coder Army

```
OOP organizes the core business capabilities.
AOP organizes behaviour that cuts across those capabilities.
```

> AOP complements OOP by modularizing concerns that do not fit neatly inside one business class.

---

14. The Three Fundamental Questions in AOP

AOP can be understood through three questions.

Question 1: What Should Happen?

Examples:

```
Log method execution
Measure execution time
Check permission
Create an audit record
```

The code that performs this additional action is called **advice**.

Question 2: Where Should It Happen?

Examples:

```
All methods in StudentService
All service-layer methods
Methods whose names begin with save
Methods annotated with @TrackExecutionTime
```

The rule that selects the relevant method executions is called a **pointcut**.

Question 3: When Should It Happen?

Examples:

```
Before the method
After the method
```

Coder Army

```
After successful completion
After an exception
Around the complete method execution
```

The timing determines the type of advice.

A Simple Mental Model

```
What should happen?   → Advice
Where should it apply? → Pointcut
When should it run?    → Advice type
```

---

15. What Is an Aspect?

An **aspect** is a module that contains a cross-cutting policy.

It normally brings together:

- The cross-cutting action that must run
- The rule that decides where it should run
- The timing of that action

For example:

```
PerformanceAspect
├── Which methods should be measured?
└── How should their execution time be measured?
```

Another example:

```
AuditAspect
├── Which operations require auditing?
├── What information should be recorded?
└── Should both success and failure be recorded?
```

An aspect therefore represents a complete cross-cutting concern as a separate module.

Coder Army

16. How Spring AOP Applies an Aspect

Spring AOP normally creates a **proxy** around a Spring-managed target object.

Target Object

The **target** is the real object containing the business logic.

```
StudentService target object
```

Proxy Object

When AOP is applied, the caller normally receives a proxy instead of directly receiving the target.

```
StudentService proxy
```

The proxy looks compatible with the target from the caller's perspective, but it can intercept method calls and apply additional behaviour.

Runtime Flow

```
Controller
    ↓
AOP proxy
    ↓
Cross-cutting logic before the method
    ↓
Actual StudentService method
    ↓
Cross-cutting logic after the method
    ↓
Controller receives the result
```

Conceptually, the proxy behaves like this:

```
public Student createStudent(Student student) {

    logBefore();
```

Coder Army

```
    try {
        Student result =
                target.createStudent(student);

        logSuccess(result);

        return result;

    } catch (Exception exception) {

        logFailure(exception);

        throw exception;
    }
}
```

The actual business method remains unchanged. Spring creates and manages the interception layer instead of requiring us to write a separate wrapper for every service.

17. Manual Wrapper vs Spring AOP Proxy

Manual Wrapper	Spring AOP
Written manually for a specific service type	Proxy infrastructure is created and managed by Spring
Delegation code must be repeated	Interception logic is reusable
Wrappers must be assembled manually	Spring integrates proxies with the IoC container
Selection logic is hard-coded into wrapper creation	Pointcuts define where advice applies
Multiple concerns create nested wrapper classes	Multiple advices can be applied through AOP configuration
Becomes difficult to scale	Designed for systematic cross-cutting behaviour

Coder Army

18. Important Spring AOP Terms at a Glance

Term	Meaning
Concern	A responsibility or area of interest in the application
Cross-cutting concern	A concern needed across multiple unrelated modules
Aspect	A module that represents a cross-cutting concern
Advice	The action that runs, such as logging or timing
Pointcut	The rule that selects where advice should run
Target	The real Spring bean containing the business logic
Proxy	The wrapper object created by Spring to intercept calls to the target

19. Final Mental Model

Without AOP:

```
Business method
├── Logging
├── Security
├── Performance measurement
├── Auditing
└── Actual business logic
```

With AOP:

```
Business class
└── Actual business logic

Separate aspect
├── Logging/security/auditing logic
├── Rule defining where it applies
└── Rule defining when it runs

Spring proxy
└── Combines both behaviours at runtime
```

The complete idea can be summarized as:

Coder Army

```
Caller
  ↓
Spring AOP proxy
  ↓
Advice runs
  ↓
Target business method runs
  ↓
Remaining advice runs
  ↓
Result returns to caller
```

20. Key Takeaways

- Business methods should primarily express business intent.
- Logging, security, auditing, transactions, and monitoring are common cross-cutting concerns.
- Implementing these concerns directly creates scattering and tangling.
- Utility methods centralize implementation but still require manual invocation.
- Inheritance shares code but does not automatically apply behaviour.
- Manual wrappers demonstrate the right basic idea but do not scale well.
- AOP keeps cross-cutting logic in separate modules.
- Advice defines what happens.
- A pointcut defines where it happens.
- The advice type determines when it happens.
- An aspect combines the cross-cutting policy.
- Spring AOP commonly uses proxies to intercept calls to Spring-managed objects.
- OOP organizes business capabilities vertically, while AOP organizes shared behaviour horizontally.

Coder Army

Spring AOP: Implementing Advice Types

Spring AOP allows us to execute additional logic before, after, or around selected Spring bean methods without mixing that logic into the business code.

Typical cross-cutting concerns include:

- Logging
- Authorization
- Auditing
- Performance measurement
- Metrics
- Technical validation
- Context management

In this chapter, we will create a basic aspect and implement the five principal advice types provided by Spring AOP:

1. `@Before`
2. `@AfterReturning`
3. `@AfterThrowing`
4. `@After`
5. `@Around`

---

1. Add the AOP Dependency

The dependency name differs between Spring Boot 3 and Spring Boot 4.

Spring Boot 3.x

Coder Army

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Spring Boot 4.x

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

Once the appropriate starter is present, Spring Boot configures the required AOP infrastructure automatically. In a normal Spring Boot application, we do not need to manually add `@EnableAspectJAutoProxy`.

---

2. Recommended Package Structure

```
com.coderarmy.studentmanagement
│
├── StudentManagementApplication
├── controller
│   └── StudentController
├── service
│   └── StudentService
├── repository
│   └── StudentRepository
└── aspect
    └── LoggingAspect
```

The aspect class should normally be placed inside the main application package or one of its subpackages. This allows Spring Boot's component scanning to discover it automatically.

Creating the First Aspect

Coder Army

3. Create the Target Bean

A **target bean** is the ordinary Spring bean whose method we want to intercept.

```
package com.coderarmy.studentmanagement.service;

import org.springframework.stereotype.Service;

@Service
public class StudentService {

    public String createStudent() {
        System.out.println("Actual createStudent() method exe
cuted");
        return "Student created successfully";
    }

    public String getStudent() {
        System.out.println("Actual getStudent() method execut
ed");
        return "Student details";
    }
}
```

Here:

- `StudentService` is the target class.
- The Spring-managed `StudentService` instance is the target object.
- `createStudent()` and `getStudent()` are possible target methods.

---

4. Create the Aspect Class

```
package com.coderarmy.studentmanagement.aspect;

import org.aspectj.lang.annotation.Aspect;
```

Coder Army

```
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before(
        "execution(String " +
        "com.coderarmy.studentmanagement.service.StudentServi
ce" +
        ".createStudent())"
    )
    public void logBeforeMethodExecution() {
        System.out.println(
            "A StudentService method is about to execute"
        );
    }
}
```

This class contains three important annotations.

`@Aspect`

Declares that the class contains AOP configuration, such as pointcuts and advice.

It tells Spring:

> This class describes cross-cutting behaviour that must be applied to selected method executions.

`@Component`

Registers the aspect as a Spring bean.

`@Aspect` describes the role of the class, but it does not by itself register an object in the Spring container. `@Component` makes the class discoverable through component scanning.

CoderArmy

`@Before`

Declares that the advice method must execute before every method matched by its pointcut expression.

---

5. Understanding the Pointcut Expression

```
execution(
    String
    com.coderarmy.studentmanagement.service.StudentService
    .createStudent
    ()
)
```

The expression can be read from left to right:

Part	Meaning
`execution`	Match a method execution
`String`	The method must return `String`
`StudentService`	The method must belong to this class
`createStudent`	The method name must be `createStudent`
`()`	The method must take no arguments

The complete string inside `@Before` is a **pointcut expression**.

```
"execution(String com.coderarmy.studentmanagement.service.StudentService.createStudent())"
```

The method containing the additional behaviour is the **advice**.

```
@Before("...")
public void logBeforeMethodExecution() {
    System.out.println(
        "A StudentService method is about to execute"
```

```
    );
}
```

The matched business method is the **target method**.

```
public String createStudent() {
    return "Student created successfully";
}
```

Target Object and Proxy Object

6. How Does the Advice Execute Automatically?

The `StudentService` class does not explicitly call `LoggingAspect`. Still, the advice executes before the service method.

Spring makes this possible by placing a **proxy object** in front of the target object.

Conceptually, the proxy behaves like a wrapper:

```
public String createStudent() {
    loggingAspect.logBeforeMethodExecution();
    return target.createStudent();
}
```

The real proxy created by Spring is generic and considerably more sophisticated, but this is the correct mental model.

---

7. Target Object

The target object contains the actual business implementation.

```
public class StudentService {

    public String createStudent() {
        return "Student created";
```

Coder Army

```
    }
}
```

Conceptually:

```
Target object = actual StudentService instance
```

8. Proxy Object

The proxy is the Spring-generated object that stands in front of the target object and intercepts matching method calls.

```
Caller → Proxy → Advice chain → Target object
```

The proxy can run additional behaviour before or after delegating the call to the target.

Are the target and proxy two different objects?

Conceptually, yes. Spring may maintain:

- A proxy object
- A target object behind that proxy

However, they should not be imagined as two ordinary, independently injectable application beans.

The normal bean name exposes the proxy:

```
studentService → proxy object
```

The target is held behind the proxy. The Spring container manages the advised service as one logical bean, while callers normally interact with the proxy.

Advice Types in Spring AOP

Coder Army

Coder Army

9. What Is an Advice?

An advice answers one fundamental question:

> What additional logic should run before, after, or around a matched method execution?

The general flow can be represented as:

```
Caller
  ↓
Advice logic
  ↓
Target method
  ↓
Advice logic
  ↓
Result or exception
```

Spring provides five principal advice types:

- `@Before`
- `@AfterReturning`
- `@AfterThrowing`
- `@After`
- `@Around`

---

10. The `try-catch-finally` Mental Model

The advice types can be understood by comparing them with ordinary Java control flow.

```
beforeLogic();

try {
    Object result = targetMethod();

    afterReturningLogic(result);
```

Coder Army

```
        return result;

    } catch (Throwable exception) {

        afterThrowingLogic(exception);
        throw exception;

    } finally {

        afterLogic();
    }
```

Approximate mapping:

Java control-flow position	Spring advice
Before entering `try`	`@Before`
After a successful return	`@AfterReturning`
Inside `catch`	`@AfterThrowing`
Inside `finally`	`@After`
The entire surrounding block	`@Around`

`@Around` is more powerful than simply combining the other four. It controls whether, when, and how the remaining invocation chain proceeds.

`@Before` Advice

11. What Does `@Before` Do?

`@Before` executes before the matched target method begins.

```
@Before(
        "execution(* " +
        "com.coderarmy.studentmanagement.service.PaymentService.*
(..))"
)
```

Coder Army

```
public void logMethodEntry(JoinPoint joinPoint) {

    System.out.println(
        "Entering method: " +
        joinPoint.getSignature().getName()
    );

    System.out.println(
        "Arguments: " +
        Arrays.toString(joinPoint.getArgs())
    );
}
```

Common uses include:

- Logging method entry
- Reading method arguments
- Checking authorization
- Checking technical preconditions
- Populating request or tracing context
- Validating a common technical policy

---

12. Using `JoinPoint`

`JoinPoint` represents the current intercepted method execution.

It allows the advice to inspect contextual information such as:

```
joinPoint.getSignature();
joinPoint.getSignature().getName();
joinPoint.getArgs();
joinPoint.getTarget();
joinPoint.getThis();
```

Coder Army

Common methods

Method	Information returned
`getSignature()`	Method signature information
`getArgs()`	Arguments passed to the target method
`getTarget()`	The underlying target object
`getThis()`	The proxy object currently handling the call

`JoinPoint` allows inspection, but it does not allow the advice to continue, skip, or repeat the target invocation.

---

13. Can `@Before` Prevent the Target Method?

`@Before` does not have a `doNotProceed()` operation because it does not directly control the invocation.

However, the advice can throw an exception:

```
@Before(
    "execution(* " +
    "com.coderarmy.studentmanagement.service.PaymentService"
+
    ".makePayment(..))"
)
public void checkAccess() {

    boolean allowed = false;

    if (!allowed) {
        throw new SecurityException(
            "You cannot make this payment"
        );
    }
}
```

Coder Army

If the advice throws an exception, execution stops before the target method is reached.

Therefore:

- `@Before` cannot directly skip the method and return an alternative result.
- It can stop execution by throwing an exception.
- Use `@Around` when controlled short-circuiting is required.

---

14. Can `@Before` Modify Method Arguments?

It can read the arguments:

```
Object[] arguments = joinPoint.getArgs();
```

However, `@Before` does not provide a controlled mechanism for replacing the arguments that will be used for the target invocation.

For argument replacement, use `@Around`:

```
joinPoint.proceed(modifiedArguments);
```

`@AfterReturning` Advice

15. When Does `@AfterReturning` Execute?

`@AfterReturning` executes only when the matched method completes normally.

```
@AfterReturning(
    "execution(* " +
    "com.coderarmy.studentmanagement.service.PaymentService.*
(..))"
)
public void logSuccessfulExecution() {
    System.out.println(
```

```
            "PaymentService method completed successfully"
        );
    }
```

Successful execution

```
Target method
      ↓
Returns normally
      ↓
@AfterReturning executes
```

Exceptional execution

```
Target method
      ↓
Throws an exception
      ↓
@AfterReturning does not execute
```

---

16. Accessing the Returned Value

Use the `returning` attribute to bind the returned value to an advice parameter.

```
@AfterReturning(
    pointcut =
        "execution(* " +
        "com.coderarmy.studentmanagement.service.PaymentServi
ce.*(..))",
    returning = "result"
)
public void logReturnedValue(String result) {
    System.out.println(
        "Method returned: " + result
```

Coder Army

```
    );
}
```

The value of `returning` must match the parameter name:

```
returning = "result"
```

```
public void logReturnedValue(String result)
```

The parameter type also affects matching.

```
public void logReturnedValue(String result)
```

This applies only when the returned value is compatible with `String`.

For a generic advice, use `Object`:

```
public void logReturnedValue(Object result)
```

---

17. Can `@AfterReturning` Replace the Returned Value?

No.

```
@AfterReturning(
    pointcut = "...",
    returning = "result"
)
public Object replaceResult(Object result) {
    return "Different result";
}
```

The value returned by the advice method is ignored. It does not replace the value returned by the target method.

Therefore, an `@AfterReturning` method should normally return `void`:

Coder Army

Coder Army

```
public void logResult(Object result)
```

Use `@Around` when the caller must receive a changed, wrapped, or completely different result.

---

18. What About Mutating the Returned Object?

Suppose the target returns a mutable object:

```
PaymentReceipt receipt
```

The advice receives the same object reference and could technically modify it:

```
receipt.setStatus("CHANGED BY ASPECT");
```

This is normally poor design because it hides meaningful data transformation outside the visible business flow.

Use `@AfterReturning` mainly for observation:

- Logging a returned value
- Recording a metric
- Writing an audit record
- Publishing a success-related event

Use `@Around` when deliberate result transformation is genuinely required.

---

19. Normal Completion Means How the Method Exits

Consider a target method that catches an internal exception:

```
public String makePayment(double amount) {

    try {
        throw new RuntimeException("Gateway failed");
    } catch (RuntimeException exception) {
```

Coder Army

```
            return "Fallback response";
        }
    }
```

An exception occurred inside the method, but it did not escape. The method ultimately returned a value.

Therefore:

```
@AfterReturning executes
@AfterThrowing does not execute
```

Advice reacts to how the target method exits, not to every internal event that occurs inside it.

`@AfterThrowing` Advice

20. When Does `@AfterThrowing` Execute?

`@AfterThrowing` runs when the matched target method exits by throwing an exception.

```
@AfterThrowing(
    "execution(* " +
    "com.coderarmy.studentmanagement.service.PaymentService.*
(..))"
)
public void logFailure() {
    System.out.println(
        "PaymentService method failed"
    );
}
```

Execution flow:

Coder Army

```
Target method
      ↓
Exception escapes the method
      ↓
@AfterThrowing executes
      ↓
Exception continues toward the caller
```

---

21. Accessing the Exception

Use the `throwing` attribute to bind the exception to an advice parameter.

```
@AfterThrowing(
    pointcut =
        "execution(* " +
        "com.coderarmy.studentmanagement.service.PaymentServi
ce.*(..))",
    throwing = "exception"
)
public void logException(Throwable exception) {

    System.out.println(
        "Exception type: " +
        exception.getClass().getSimpleName()
    );

    System.out.println(
        "Exception message: " +
        exception.getMessage()
    );
}
```

The `throwing` value must match the parameter name:

```
throwing = "exception"
```

Coder Army

```
public void logException(Throwable exception)
```

22. Restricting Advice to a Specific Exception Type

The parameter type can restrict when the advice executes.

```
@AfterThrowing(
    pointcut =
        "execution(* " +
        "com.coderarmy.studentmanagement.service.PaymentServi
ce.*(..))",
    throwing = "exception"
)
public void handleInvalidAmount(
    IllegalArgumentException exception
) {
    System.out.println(
        "Invalid payment request: " +
        exception.getMessage()
    );
}
```

This advice executes for:

- `IllegalArgumentException`
- Compatible subclasses

It does not execute for unrelated exception types.

23. Does `@AfterThrowing` Handle the Exception?

Not automatically.

It observes the failed exit, but the original exception still propagates.

Coder Army

```
Target throws exception
        ↓
@AfterThrowing logs it
        ↓
Original exception continues
        ↓
Controller or caller receives it
```

The advice may itself throw another exception, but merely returning from an `@AfterThrowing` method does not consume the original failure.

---

24. Can `@AfterThrowing` Return a Fallback Value?

No.

```java
@AfterThrowing("...")
public String returnFallback(Exception exception) {
    return "Fallback";
}
```

The target method has already completed exceptionally. The advice method's return value is not used as a recovery result.

Use `@Around` when the invocation must be caught and converted into a fallback value. This must be done carefully because silently swallowing failures can create incorrect application behaviour.

---

25. Suitable Uses of `@AfterThrowing`

Use it for behaviour that applies specifically to failed method execution:

- Error logging
- Failure metrics
- Failed-operation audit records
- Alert creation
- Recording exception type and message

Coder Army

- Technical cleanup that applies only to failure

`@After` Advice

26. `@After` Means After-Finally

The name can be misleading. `@After` does not mean “after successful completion.”

It behaves like a `finally` block:

```
try {
    targetMethod();
} finally {
    afterAdvice();
}
```

It executes whether the target method:

- Returns successfully
- Throws an exception

Spring therefore describes it as **after-finally advice**.

---

27. `@After` Example

```
@After(
    "execution(* " +
    "com.coderarmy.studentmanagement.service.PaymentService.*
(..))"
)
public void clearContext() {
    System.out.println("Clearing method context");
}
```

Execution on success:

Coder Army

```
Target returns normally
        ↓
@After executes
```

Execution on failure:

```
Target throws an exception
        ↓
@After executes
        ↓
Exception continues
```

---

28. Comparing the Three After Advices

Advice	Success	Failure	Direct access to result	Direct access to exception
`@AfterReturning`	Yes	No	Yes	No
`@AfterThrowing`	No	Yes	No	Yes
`@After`	Yes	Yes	No	No

`@After` vs `@AfterReturning`

```
@After          → success or failure
@AfterReturning → success only
```

`@After` vs `@AfterThrowing`

```
@After         → every outcome
@AfterThrowing → failure only
```

---

29. Suitable Uses of `@After`

Use `@After` for logic that must happen regardless of outcome:

Coder Army

- Releasing a lock
- Clearing contextual data
- Removing a `ThreadLocal` value
- Releasing a technical resource
- Generic method-exit logging

Use `@AfterReturning` when the returned value is needed.

Use `@AfterThrowing` when the thrown exception is needed.

`@After` is intentionally outcome-neutral.

`@Around` Advice

30. Why Is `@Around` the Most Powerful Advice?

`@Around` surrounds the complete invocation.

```
Code before
    ↓
Target method
    ↓
Code after
```

Unlike the other advice types, it controls the target method's execution.

It can:

- Run logic before the target method
- Run logic after the target method
- Inspect and replace arguments
- Inspect and replace the returned value
- Catch, rethrow, wrap, or replace exceptions
- Skip the target method entirely
- Invoke the method more than once

- Measure execution duration

Because it has this level of control, it should be used only when that control is actually required.

31. Basic `@Around` Syntax

```
@Around("...")
public Object logAround(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    System.out.println("Before target method");

    Object result = joinPoint.proceed();

    System.out.println("After target method");

    return result;
}
```

Execution flow:

```
Enter @Around advice
        ↓
Execute code before proceed()
        ↓
Call proceed()
        ↓
Remaining advice and target execute
        ↓
Target result returns to the advice
        ↓
Execute code after proceed()
        ↓
Return result to caller
```

Coder Army

CoderArmy

32. What Is `ProceedingJoinPoint`?

`ProceedingJoinPoint` represents the current intercepted invocation and provides the ability to continue that invocation.

It is used specifically with `@Around` advice.

`ProceedingJoinPoint` extends `JoinPoint`, so it provides the normal contextual information plus control methods such as:

```
proceed();
proceed(Object[] arguments);
```

The core distinction is:

```
JoinPoint
    → inspect the current invocation

ProceedingJoinPoint
    → inspect and control the current invocation
```

---

33. What Does `proceed()` Actually Do?

```
Object result = joinPoint.proceed();
```

This means:

> Continue with the remaining interceptor chain and eventually invoke the actual target method.

It is not always equivalent to directly calling the business method because other interceptors or advice may still remain inside the chain.

```
First around advice
        ↓
proceed()
        ↓
Second advice
        ↓
```

Coder Army

```
Transaction interceptor
        ↓
Security interceptor
        ↓
Actual target method
```

Therefore, `proceed()` means:

> Continue to the next step in the current invocation chain.

---

34. What Happens When `proceed()` Is Not Called?

```java
@Around("...")
public Object blockExecution(
    ProceedingJoinPoint joinPoint
) {
    System.out.println("Method blocked");
    return "Blocked response";
}
```

The target method never executes.

```
Caller
  ↓
@Around advice
  ↓
No proceed()
  ↓
Target method skipped
  ↓
Alternative value returned
```

This ability can support patterns such as:

- Caching
- Feature flags
- Access rejection

Coder Army

- Short-circuiting
- Idempotency
- Circuit-breaking

It can also make behaviour difficult to understand because the source code appears to call a method that an aspect silently prevents from running.

---

35. Why Must We Return the Result of `proceed()`?

Consider this incorrect advice:

```
@Around("...")
public Object incorrectAdvice(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    joinPoint.proceed();
    return null;
}
```

Even when the target returns:

```
Payment successful
```

the caller receives:

```
null
```

The value returned by the `@Around` advice becomes the value observed by the caller.

Therefore, the normal pattern is:

```
Object result = joinPoint.proceed();
```

Coder Army

```
return result;
```

Return something else only when deliberately replacing the target's result.

36. Why Does Around Advice Usually Return `Object`?

A generic aspect may intercept methods with different return types:

```
String
Student
PaymentReceipt
List<Order>
int
boolean
void
```

Therefore, a reusable around advice normally uses:

```
public Object advice(...) throws Throwable
```

Primitive values are boxed:

```
int     → Integer
boolean → Boolean
```

For a `void` target method, the result is effectively `null`.

Declaring a generic around advice as `void` would discard any value returned by `proceed()` and cause the caller to observe `null` where a value was expected.

Practical Uses of `@Around`

37. Measuring Execution Time

Execution-time measurement requires state that exists both before and after the same method invocation:

Coder Army

```
Start time before method
End time after method
```

This makes `@Around` a natural choice.

```
@Around(
    "execution(* " +
    "com.coderarmy.studentmanagement.service.PaymentService.*
(..))"
)
public Object measureExecutionTime(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    long startTime = System.nanoTime();

    try {
        return joinPoint.proceed();
    } finally {

        long executionTime =
            System.nanoTime() - startTime;

        System.out.println(
            joinPoint.getSignature().getName() +
            " took " +
            executionTime +
            " nanoseconds"
        );
    }
}
```

Why use `finally`?

This version is incomplete:

Coder Army

```
long startTime = System.nanoTime();

Object result = joinPoint.proceed();

long executionTime =
    System.nanoTime() - startTime;
```

If `proceed()` throws an exception, execution never reaches the duration calculation.

Using `finally` ensures that timing is recorded on both success and failure.

---

38. Exception Behaviour Inside `@Around`

```
@Around("...")
public Object observeExecution(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    System.out.println("Before");

    Object result = joinPoint.proceed();

    System.out.println("After");

    return result;
}
```

When the target throws an exception:

```
"Before" is printed
proceed() throws
"After" is not printed
Exception propagates
```

`proceed()` behaves like an ordinary Java method call. If it throws, statements after it do not run unless the exception is caught or a `finally` block is used.

39. Observing Success and Failure in One Advice

```
@Around("...")
public Object observeExecution(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    System.out.println(
        "Starting: " +
        joinPoint.getSignature().getName()
    );

    try {
        Object result = joinPoint.proceed();

        System.out.println("Execution successful");
        return result;

    } catch (Throwable exception) {

        System.out.println(
            "Execution failed: " +
            exception.getMessage()
        );

        throw exception;

    } finally {
        System.out.println("Execution completed");
    }
}
```

Coder Army

Coder Army

This single advice can reproduce the major lifecycle positions represented by:

- `@Before`
- `@AfterReturning`
- `@AfterThrowing`
- `@After`

That does not mean `@Around` should always be used. When only one lifecycle position is required, a narrower advice communicates the intention more clearly.

---

40. Converting an Exception into a Fallback Value

```
@Around("...")
public Object provideFallback(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    try {
        return joinPoint.proceed();

    } catch (PaymentGatewayException exception) {

        return new PaymentReceipt(
            "PENDING",
            "Payment will be retried"
        );
    }
}
```

Here, the original exception does not reach the caller. The aspect converts a failed invocation into a successful return value.

This is powerful but potentially dangerous:

- Monitoring may no longer detect the original failure.
- Transaction behaviour may change.

Coder Army

- Callers may believe the operation succeeded.
- Business failure rules become hidden inside infrastructure code.

Fallback, retry, and circuit-breaker policies are usually safer when implemented through established resilience infrastructure rather than an informal generic aspect.

---

41. Modifying Method Arguments

Suppose the target method is:

```
public String greetStudent(String name) {
    return "Hello " + name;
}
```

Around advice can replace the argument before continuing the invocation:

```
@Around(
    "execution(* " +
    "com.coderarmy.studentmanagement.service.StudentService"
+
    ".greetStudent(..))"
)
public Object normalizeName(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    Object[] arguments = joinPoint.getArgs();

    String originalName =
        (String) arguments[0];

    String normalizedName =
        originalName.trim().toUpperCase();

    Object[] modifiedArguments = {
```

Coder Army

```
            normalizedName
        };

        return joinPoint.proceed(modifiedArguments);
    }
```

Calling:

```
studentService.greetStudent("  aditya  ");
```

causes the target method to receive:

```
ADITYA
```

The new argument array must remain compatible with the target method's parameter types and order.

---

42. Modifying the Returned Value

```
@Around(
    "execution(* " +
    "com.coderarmy.studentmanagement.service.PaymentService"
+
    ".makePayment(..))"
)
public Object wrapResponse(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    Object originalResult =
        joinPoint.proceed();

    return new ApiResponse<>(
        true,
```

```
            "Operation completed",
            originalResult
        );
    }
```

The caller receives the wrapper instead of the target method's direct result.

The replacement value must be compatible with the target method's declared return type. Otherwise, the caller may receive a runtime type error.

For example, a method declared to return `String` cannot safely be replaced with an unrelated `ApiResponse` object unless the original method signature is designed to return that type.

---

43. Calling `proceed()` More Than Once

Around advice may call `proceed()`:

- Once
- More than once
- Not at all

Example retry:

```
@Around("...")
public Object retry(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    try {
        return joinPoint.proceed();

    } catch (TemporaryServiceException exception) {
        return joinPoint.proceed();
    }
}
```

Coder Army

Coder Army

This may invoke the target method twice.

That is dangerous for operations with side effects:

```
transferMoney()
sendEmail()
createOrder()
chargeCard()
```

Calling such methods twice may create duplicate operations.

Multiple calls to `proceed()` should be considered only when:

- The operation is safe to retry.
- Idempotency is guaranteed.
- The retry policy is clearly defined.
- Failure categories are carefully selected.

For production applications, specialized retry infrastructure is generally safer than a simplistic custom retry aspect.

Choosing the Correct Advice Type

44. Advice Comparison

Requirement	Best advice type
Run logic before a method	`@Before`
Observe a successful result	`@AfterReturning`
Observe a failure and access the exception	`@AfterThrowing`
Run cleanup regardless of outcome	`@After`
Measure complete execution time	`@Around`
Replace method arguments	`@Around`
Replace or wrap the return value	`@Around`
Skip the method and return another value	`@Around`

Coder Army

Requirement	Best advice type
Catch an exception and return a fallback	`@Around`
Retry the invocation	`@Around`

45. Prefer the Narrowest Suitable Advice

The most powerful tool is not always the clearest tool.

Use:

- `@Before` when only pre-execution logic is required.
- `@AfterReturning` when only successful completion matters.
- `@AfterThrowing` when only failed completion matters.
- `@After` when cleanup must happen for every outcome.
- `@Around` only when the invocation itself must be controlled.

A narrow advice makes the lifecycle position obvious and reduces the chance of accidentally changing business behaviour.

Final Mental Model

46. Complete Invocation Flow

```
Caller
  ↓
Spring proxy
  ↓
@Around: code before proceed()
  ↓
@Before
  ↓
Target method
  ├── Returns normally
  │       ↓
  │   @AfterReturning
  │
  └── Throws exception
          ↓
```

Coder Army

```
      @AfterThrowing

  ↓
@After executes in either case
  ↓
Control returns to @Around
  ↓
@Around returns a result or propagates an exception
  ↓
Caller
```

The exact ordering becomes more nuanced when several aspects and interceptors are involved, but this flow provides the correct conceptual foundation for understanding the role of each advice type.

Key Takeaways

1. Advice is the additional behaviour applied to matched method executions.
2. A pointcut expression decides which methods are matched.
3. Spring normally exposes a proxy in place of the original target object.
4. `JoinPoint` allows an advice to inspect the current invocation.
5. `ProceedingJoinPoint` allows `@Around` advice to inspect and control the invocation.
6. `@Before` runs before the target but cannot directly replace arguments or return an alternate result.
7. `@AfterReturning` runs only after normal completion and cannot replace the returned value.
8. `@AfterThrowing` runs only when an exception escapes the target and does not automatically consume it.
9. `@After` behaves like `finally` and runs on both success and failure.
10. `@Around` can modify, skip, repeat, or replace the invocation, so it must be used carefully.
11. `proceed()` continues the remaining interceptor chain and eventually reaches the target method.

12. Prefer the narrowest advice type that correctly expresses the requirement.

Coder Army

Coder Army

Spring AOP Pointcut Expressions

1. What Is a Pointcut Expression?

An advice defines **what additional behaviour should run**, such as logging, security, auditing or performance tracking.

A pointcut expression defines **where that advice should run**.

```
@Before("execution(* getStudent(..))")
public void logBeforeMethod() {
    System.out.println("Method is about to execute");
}
```

Here:

- `@Before` defines when the advice runs.
- `execution(* getStudent(..))` selects the target methods.
- `logBeforeMethod()` contains the additional behaviour.

> A pointcut expression does not execute a method. It decides whether a method execution should be intercepted.

---

2. The `execution()` Designator

`execution()` is the most commonly used pointcut designator in Spring AOP. It selects methods from their signatures.

It can match a method using its:

- Access modifier
- Return type
- Declaring class

Coder Army

- Method name
- Parameters
- Declared exceptions

Complete syntax:

```
execution(
    modifiers-pattern?
    return-type-pattern
    declaring-type-pattern?
    method-name-pattern(parameter-pattern)
    throws-pattern?
)
```

The parts marked with `?` are optional. The essential parts are:

```
Return type + method name + parameters
```

The broadest commonly used expression is:

```
execution(* *(..))
```

```
*     → any return type
*     → any method name
(..)  → zero or more parameters
```

2.1 Exact Method Matching

Consider:

```
package com.coderarmy.student.service;

public class StudentService {
```

Coder Army

```
    public Student getStudent(Long id) {
        return null;
    }
}
```

An exact pointcut is:

```
execution(public com.coderarmy.student.model.Student com.coderarmy.student.service.StudentService.getStudent(java.lang.Long))
```

Part	Meaning
`public`	The method must be public
`com.coderarmy.student.model.Student`	The return type must be `Student`
`com.coderarmy.student.service.StudentService`	The method must belong to `StudentService`
`getStudent`	The method name must be `getStudent`
`java.lang.Long`	It must accept exactly one `Long` argument

This expression matches only that exact signature. Real applications normally use broader expressions so that one pointcut can cover multiple related methods.

---

3. Wildcards: `*` and `..`

The meaning of a wildcard depends on where it appears.

3.1 The `*` Wildcard

- matches anything in one position or one naming segment.

Any return type

```
execution(* StudentService.getStudent(..))
```

Coder Army

The method may return `Student`, `String`, `void`, `int` or any other type.

Any method name

```
execution(* *(..))
```

Method names beginning with `get`

```
execution(* get*(..))
```

Matches:

```
getStudent()
getAllStudents()
getStudentById()
getStudentCount()
```

Method names ending with `Student`

```
execution(* *Student(..))
```

Matches:

```
getStudent()
createStudent()
deleteStudent()
```

Method names containing `Student`

```
execution(* *Student*(..))
```

Matches:

Coder Army

```
getStudent()
getStudentById()
createStudentAccount()
updateStudentDetails()
```

Exactly one parameter of any type

```
execution(* *(*))
```

Matches:

```
getStudent(Long id)
saveStudent(Student student)
deleteStudent(String email)
```

It does not match methods with zero or multiple parameters.

```
(*)  → exactly one parameter of any type
(..) → zero or more parameters of any type
```

3.2 The `..` Wildcard

`..` represents zero or more elements.

Inside a parameter list

```
execution(* *(..))
```

It matches methods with any number of parameters, including none:

```
getAllStudents()
getStudent(Long id)
```

Coder Army

```
updateStudent(Long id, Student student)
search(String name, Integer age, String city)
```

Inside a package pattern

```
com.coderarmy.student.service..*
```

It matches the package and all its subpackages:

```
com.coderarmy.student.service.StudentService
com.coderarmy.student.service.impl.StudentServiceImpl
com.coderarmy.student.service.internal.StudentValidator
```

By comparison:

```
com.coderarmy.student.service.*
```

matches only classes placed directly inside `service`. It does not include `service.impl` or other subpackages.

```
*   → one package or type-name segment
..  → zero or more package segments
```

---

4. Common `execution()` Patterns

Consider the following service:

```
public class StudentService {

    public Student getStudent(Long id) {
        return null;
    }

    public List<Student> getAllStudents() {
```

Coder Army

```
        return null;
    }

    public Student createStudent(Student student) {
        return null;
    }

    public Student updateStudent(Long id, Student student) {
        return null;
    }

    public void deleteStudent(Long id) {
    }
}
```

Match a method name everywhere

```
execution(* getStudent(..))
```

This can match any interceptable method named `getStudent`, including methods in controllers, services or other classes.

A safer expression is:

```
execution(* com.coderarmy.student.service.StudentService.getStudent(..))
```

Match every method

```
execution(* *(..))
```

This is extremely broad and should normally be restricted by package, class, bean name or annotation.

Coder Army

Match public methods

```
execution(public * *(..))
```

The modifier is optional. Public methods are the safest and most common targets in proxy-based Spring AOP. Private methods cannot be intercepted through normal proxy overriding.

Match all public methods in the service layer

```
execution(public * com.coderarmy.student.service..*.*(..))
```

Part	Meaning
`public`	Public methods only
first `*`	Any return type
`com.coderarmy.student.service..*`	Any class in the package or its subpackages
second `*`	Any method name
`(..)`	Any number and type of arguments

Quick examples

Requirement	Expression
Any method	`execution(* *(..))`
Any method named `getStudent`	`execution(* getStudent(..))`
Methods beginning with `get`	`execution(* get*(..))`
Any method in `StudentService`	`execution(* com.coderarmy.student.service.StudentService.*(..))`
Methods in the service package only	`execution(* com.coderarmy.student.service.*.*(..))`
Methods in the service package and subpackages	`execution(* com.coderarmy.student.service..*.*(..))`
Methods with no parameters	`execution(* *())`

Coder Army

Requirement	Expression
Methods with exactly one parameter	`execution(* *(*))`
Methods accepting exactly one `Long`	`execution(* *(java.lang.Long))`
Methods whose first parameter is `Long`	`execution(* *(java.lang.Long, ..))`

5. The `within()` Designator

`execution()` focuses on a method signature. `within()` focuses on the class or package where the method is declared.

```
execution() → Which method signature should match?
within()    → Inside which class or package should methods match?
```

Exact class

```
within(com.coderarmy.student.service.StudentService)
```

Classes directly inside a package

```
within(com.coderarmy.student.service.*)
```

A package and all its subpackages

```
within(com.coderarmy.student.service..*)
```

Practical example:

```
@Before("within(com.coderarmy.student.service..*)")
public void logServiceMethod() {
```

```
    System.out.println("Service method called");
}
```

Coder Army

6. `execution()` vs `within()`

These expressions may produce similar results for a basic service-layer requirement:

```
execution(* com.coderarmy.student.service..*.*(..))
```

```
within(com.coderarmy.student.service..*)
```

The difference is expressive power.

`execution()` can filter by modifier, return type, method name and parameters. `within()` only restricts the declaring class or package.

They can also be combined:

```
within(com.coderarmy.student.service..*)
&& execution(public * get*(..))
```

Meaning:

> Match public methods beginning with `get`, but only inside the service layer.

7. The `@annotation()` Designator

`@annotation()` matches methods that carry a specified annotation.

```
@Transactional
public void transferMoney() {
}
```

Pointcut:

Coder Army

```
@annotation(org.springframework.transaction.annotation.Transa
ctional)
```

It means:

> Match a method execution when the method has the specified annotation.

`@annotation()` is used for method-level annotations. Class-level annotation matching is handled by `@within()` and `@target()`.

Why annotation-based matching is useful

A package-based pointcut applies behaviour throughout an architectural region. An annotation-based pointcut applies behaviour only to methods that explicitly opt in.

Common use cases include:

- Auditing selected operations
- Measuring execution time
- Recording sensitive actions
- Custom authorization
- Idempotency protection

Example with a custom annotation:

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface TrackExecutionTime {
}
```

```
@TrackExecutionTime
public Student createStudent(Student student) {
    return student;
}
```

Coder Army

```
@Around("@annotation(com.coderarmy.aop.TrackExecutionTime)")
public Object measureExecutionTime(ProceedingJoinPoint joinPo
int)
        throws Throwable {

    long start = System.currentTimeMillis();
    Object result = joinPoint.proceed();
    long end = System.currentTimeMillis();

    System.out.println("Execution time: " + (end - start) + "
ms");
    return result;
}
```

---

8. The `bean()` Designator

`bean()` matches methods using the Spring bean name rather than the Java class name.

```
@Service
public class StudentService {
}
```

The default bean name is normally:

```
studentService
```

Pointcut:

```
bean(studentService)
```

Example:

Coder Army

```
@Before("bean(studentService)")
public void beforeStudentService() {
    System.out.println("Student service method called");
}
```

Wildcard matching is also supported:

```
bean(*Service)
```

This can match bean names such as:

```
studentService
teacherService
paymentService
emailService
```

> `bean()` is specific to Spring AOP and works with Spring-managed bean names.

---

9. Combining Pointcut Expressions

Pointcut expressions can be combined using boolean operators:

```
&&  // AND
||  // OR
!   // NOT
```

AND: `&&`

Both expressions must match.

```
within(com.coderarmy.student.service..*)
&& execution(public * *(..))
```

Matches public methods inside the service layer.

Coder Army

OR: `||`

At least one expression must match.

```
within(com.coderarmy.student.service..*)
|| within(com.coderarmy.student.controller..*)
```

Matches methods in either the service layer or the controller layer.

NOT: `!`

The negated expression must not match.

```
within(com.coderarmy.student.service..*)
&& !execution(* getHealth(..))
```

Matches service methods except methods named `getHealth`.

Another example:

```
within(com.coderarmy.student..*)
&& !within(com.coderarmy.student.repository..*)
```

Matches application methods except repository methods.

Use parentheses to keep complex expressions readable:

```
(
    within(com.coderarmy.student.service..*)
    || within(com.coderarmy.student.controller..*)
)
&& execution(public * *(..))
```

---

10. Named Pointcuts with `@Pointcut`

Long expressions are often reused across multiple advice methods. Repeating them creates duplication and makes maintenance harder.

Coder Army

A named pointcut assigns a reusable name to an expression.

```
@Pointcut("within(com.coderarmy.student.service..*)")
public void serviceLayer() {
}
```

The method body remains empty. Its name represents the pointcut expression.

Reuse it in an advice:

```
@Before("serviceLayer()")
public void beforeServiceMethod() {
    System.out.println("Before service method");
}
```

`serviceLayer()` is not called as a normal Java method. Spring resolves it as a pointcut declaration.

---

11. Centralizing Application Pointcuts

When several aspects share the same layer definitions, keep them in one class.

```
package com.coderarmy.student.aop;

import org.aspectj.lang.annotation.Pointcut;

public class ApplicationPointcuts {

    @Pointcut("within(com.coderarmy.student.controller..*)")
    public void controllerLayer() {
    }

    @Pointcut("within(com.coderarmy.student.service..*)")
    public void serviceLayer() {
    }
```

Coder Army

```
    @Pointcut("within(com.coderarmy.student.repository..*)")
    public void repositoryLayer() {
    }

    @Pointcut("execution(public * *(..))")
    public void publicMethod() {
    }

    @Pointcut("serviceLayer() && publicMethod()")
    public void publicServiceMethod() {
    }
}
```

Reference it from another aspect using its fully qualified name:

```
@Aspect
@Component
public class LoggingAspect {

    @Before(
        "com.coderarmy.student.aop.ApplicationPointcuts.serviceLayer()"
    )
    public void logBeforeServiceMethod() {
        System.out.println("Service method called");
    }
}
```

This creates one architectural definition of each layer. If a package changes, the expression needs to be updated in only one place.

---

12. Advanced Pointcut Designators

12.1 `@within()`

Coder Army

```
@within(com.coderarmy.aop.Audited)
```

Matches methods declared inside types carrying the specified annotation.

```
@Audited
@Service
public class PaymentService {

    public void processPayment() {
    }
}
```

12.2 `@target()`

```
@target(com.coderarmy.aop.Audited)
```

Matches when the runtime target object's class carries the specified annotation.

In an ordinary class, `@within()` and `@target()` may appear to behave the same:

```
@TrackExecution
@Service
public class StudentService {

    public void addStudent() {
    }
}
```

The difference becomes clearer with inheritance.

```
public class BaseService {

    public void save() {
        System.out.println("Saving");
```

Coder Army

```
    }
}
```

```
@TrackExecution
@Service
public class StudentService extends BaseService {
}
```

When `studentService.save()` executes:

- `@target(TrackExecution)` matches because the runtime target object is a `StudentService`.
- `@within(TrackExecution)` does not match because `save()` was declared inside the unannotated `BaseService`.

```
@within → checks the type where the method is declared
@target → checks the runtime target object's class
```

---

13. `args()` and `@args()`

`args()`

`args()` checks runtime argument types.

```
@Before("args(com.coderarmy.student.model.Student)")
public void beforeStudentArgument() {
    System.out.println("Student argument received");
}
```

It matches when the runtime argument is compatible with `Student`.

`@args()`

`@args()` checks annotations on the runtime classes of the arguments.

Coder Army

```java
@TrackData
public class Student {

    private String name;
}
```

```java
public void saveStudent(Student student) {
}
```

```java
@Before("@args(com.coderarmy.annotation.TrackData)")
public void beforeMethod() {
    System.out.println("Annotated argument received");
}
```

When `new Student()` is passed, Spring checks the runtime class of that object. Because `Student` carries `@TrackData`, the pointcut matches.

```
args(Type)             → runtime argument type
@args(AnnotationType)  → annotation on the runtime argument type
```

`@args()` does not check whether the method parameter itself is annotated.

---

14. `this()` and `target()`

Spring AOP normally places a proxy in front of the actual Spring bean.

```
Caller → Spring proxy → Actual target bean
```

In pointcut terminology:

```
Proxy object       → this
Actual bean object → target
```

Coder Army

Therefore:

```
this(Type)   → checks the Spring proxy type
target(Type) → checks the actual target bean type
```

Consider:

```
public interface StudentOperations {

    void addStudent();
}
```

```
@Service
public class StudentService implements StudentOperations {

    @Override
    public void addStudent() {
        System.out.println("Student added");
    }
}
```

target(StudentService)

```
@Before("target(com.coderarmy.student.service.StudentServic
e)")
```

Matches because the actual bean is a `StudentService`.

this(StudentOperations)

```
@Before("this(com.coderarmy.student.service.StudentOperation
s)")
```

Coder Army

With a JDK dynamic proxy, this matches because the proxy implements `StudentOperations`.

15. JDK Proxy vs CGLIB Proxy

JDK dynamic proxy

A JDK proxy exposes the implemented interfaces.

```
Proxy type  → StudentOperations
Target type → StudentService
```

Therefore:

```
this(StudentOperations)  → matches
target(StudentService)   → matches
this(StudentService)     → normally does not match
```

The proxy implements `StudentOperations`, but it is not a subclass of the concrete `StudentService` class.

CGLIB proxy

A CGLIB proxy subclasses the target class.

```
StudentService
      ↑
StudentService$$SpringCGLIB$$...
```

Therefore, both can match:

```
this(com.coderarmy.student.service.StudentService)
```

```
target(com.coderarmy.student.service.StudentService)
```

Coder Army

The difference between `this()` and `target()` is most visible with JDK interface-based proxies.

---

16. `target()` vs `@target()`

These designators check different things.

```
target(com.coderarmy.student.service.StudentService)
```

Checks the Java type of the actual target object.

```
@target(com.coderarmy.aop.TrackExecution)
```

Checks whether the actual target object's class carries `@TrackExecution`.

```
this(Type)          → proxy type
target(Type)        → real bean type
@target(Annotation) → annotation on the real bean's class
```

Memory diagram:

```
Caller → [proxy] → [actual bean]
           this       target
```

---

17. Configuring the Proxy Type

Spring Boot

Spring Boot normally uses class-based proxying for AOP auto-configuration.

```
spring.aop.proxy-target-class=true
```

To prefer JDK interface-based proxies:

Coder Army

```
spring.aop.proxy-target-class=false
```

JDK proxies require interfaces. If a class does not implement an interface, Spring uses a class-based proxy instead.

Plain Spring Framework

Class-based proxies:

```
@Configuration
@EnableAspectJAutoProxy(proxyTargetClass = true)
public class AopConfig {
}
```

Interface-based proxies:

```
@Configuration
@EnableAspectJAutoProxy(proxyTargetClass = false)
public class AopConfig {
}
```

The default value of `proxyTargetClass` in `@EnableAspectJAutoProxy` is `false`.

In a normal Spring Boot application, the application property is generally preferred because Boot already provides AOP auto-configuration.

---

18. Quick Reference

Designator	What it checks	Example
`execution()`	Method signature	`execution(* service..*.*(..))`
`within()`	Declaring class or package	`within(service..*)`
`@annotation()`	Annotation on the method	`@annotation(TrackExecutionTime)`
`bean()`	Spring bean name	`bean(*Service)`
`@within()`	Annotation on the declaring type	`@within(Audited)`

Coder Army

Designator	What it checks	Example
`@target()`	Annotation on the runtime target class	`@target(Audited)`
`args()`	Runtime argument types	`args(Student)`
`@args()`	Annotations on runtime argument types	`@args(TrackData)`
`this()`	Spring proxy type	`this(StudentOperations)`
`target()`	Actual target bean type	`target(StudentService)`

19. Practical Guidelines

1. Prefer pointcuts that represent clear architectural boundaries.

```
within(com.coderarmy.student.service..*)
```

is safer than:

```
execution(* *(..))
```

1. Use `execution()` when matching depends on method name, return type, modifier or parameters.
2. Use `within()` when the requirement applies to a complete class, package or application layer.
3. Use `@annotation()` when selected methods should explicitly opt into the aspect behaviour.
4. Use named pointcuts instead of repeating long expressions.
5. Keep pointcuts narrow enough to understand and test. Overly broad expressions can produce excessive logs, unnecessary overhead and unexpected behaviour.
6. Remember that Spring AOP works through Spring-managed proxies:

Coder Army

```
External caller → Spring proxy → target method
```

A direct internal call from one method to another method in the same object normally bypasses the proxy and does not trigger the advice.

---

20. Final Mental Model

A pointcut answers:

> Which method executions should receive the aspect behaviour?

```
Method signature      → execution()
Class or package      → within()
Method annotation     → @annotation()
Spring bean name      → bean()
Runtime argument type → args()
Proxy type            → this()
Target bean type      → target()
```

For most Spring applications, the most important designators are:

```
execution()
within()
@annotation()
bean()
```

The advanced designators become useful when inheritance, runtime argument types or proxy implementation details matter.

Coder Army

Custom Annotations and AOP Binding in Spring

Spring AOP becomes especially useful when a method can declare **what behaviour it needs**, while an aspect contains **how that behaviour is implemented**.

A custom annotation gives us a clean, declarative way to connect business methods with cross-cutting concerns such as:

- Execution-time tracking
- Logging
- Auditing
- Authorization
- Rate limiting
- Retry handling
- Validation

---

1. Separating “What” from “How”

Consider a service method that measures its own execution time:

```
public Report generateAnnualReport() {

    long startTime = System.nanoTime();

    try {
        return reportGenerator.generate();
    } finally {
        long duration = System.nanoTime() - startTime;
        System.out.println("Duration: " + duration);
```

```
    }
}
```

The method now has two responsibilities:

1. Generating the report
2. Measuring execution time

Execution-time measurement is not part of the core business logic. It is a **cross-cutting concern** that may be required by many methods.

We can move that behaviour into an aspect:

```
@Around("@annotation(TrackExecutionTime)")
public Object measureExecutionTime(
        ProceedingJoinPoint joinPoint
) throws Throwable {
    // Execution-time logic
}
```

The business method only declares its intention:

```
@TrackExecutionTime
public Report generateAnnualReport() {
    return reportGenerator.generate();
}
```

The responsibilities are now separated:

```
Annotation
    → Declares what behaviour is required

Aspect
    → Defines how that behaviour is implemented

Business method
    → Contains the actual business logic
```

Coder Army

This is an example of **declarative programming**. The method declares the behaviour it needs without implementing that behaviour itself.

Coder Army

2. A Custom Annotation Does Nothing by Itself

Suppose we create an annotation:

```
public @interface TrackExecutionTime {
}
```

and apply it to a method:

```
@TrackExecutionTime
public void generateReport() {
}
```

Nothing happens automatically.

An annotation is only **metadata** attached to a program element.

```
Method
  +
Metadata: “Track the execution time of this method”
```

Some other component must:

1. Detect the annotation
2. Read its information
3. Perform the required behaviour

In this case, the Spring AOP aspect performs that work:

```
@Around("@annotation(TrackExecutionTime)")
public Object trackTime(
        ProceedingJoinPoint joinPoint
```

```
) throws Throwable {
    // Timing implementation
}
```

> The annotation declares the intention. The aspect gives that annotation its behaviour.

---

3. Creating a Custom Annotation

A custom annotation is declared using `@interface`:

```
public @interface TrackExecutionTime {
}
```

It is not declared using the normal `interface` keyword.

For Spring AOP, the basic declaration is usually not enough. Java must also know:

- Where the annotation can be used
- How long the annotation remains available

These rules are defined through **meta-annotations**.

A meta-annotation is an annotation applied to another annotation declaration.

The most important meta-annotations for custom AOP annotations are:

```
@Target
@Retention
```

`@Documented` is also commonly added.

---

4. Understanding `@Target`

`@Target` defines where an annotation is legally allowed to appear.

For an annotation that should be placed on methods:

Coder Army

Coder Army

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Target;

@Target(ElementType.METHOD)
public @interface TrackExecutionTime {
}
```

The compiler now permits this:

```
@TrackExecutionTime
public void generateReport() {
}
```

but does not permit the annotation on a class:

```
@TrackExecutionTime // Compilation error with METHOD-only tar
get
public class ReportService {
}
```

Common `ElementType` values

`ElementType.METHOD`

The annotation can be applied to methods.

```
@Target(ElementType.METHOD)
```

`ElementType.TYPE`

The annotation can be applied to classes, interfaces, enums, records and annotation interfaces.

```
@Target(ElementType.TYPE)
```

Coder Army

`ElementType.FIELD`

The annotation can be applied to fields.

```
@Target(ElementType.FIELD)
```

`ElementType.PARAMETER`

The annotation can be applied to method or constructor parameters.

```
@Target(ElementType.PARAMETER)
```

`ElementType.CONSTRUCTOR`

The annotation can be applied to constructors.

```
@Target(ElementType.CONSTRUCTOR)
```

`ElementType.ANNOTATION_TYPE`

The annotation can be applied to another annotation declaration.

```
@Target(ElementType.ANNOTATION_TYPE)
```

For method-level Spring AOP, the most common choice is:

```
@Target(ElementType.METHOD)
```

Spring AOP is proxy-based and primarily intercepts method executions on Spring-managed beans.

Supporting classes and methods

An annotation may support more than one target:

```
@Target({
    ElementType.TYPE,
```

```
    ElementType.METHOD
})
```

This allows the annotation to be placed on either a class or an individual method.

---

5. Understanding `@Retention`

`@Retention` defines how long annotation information is preserved.

Java provides three retention policies:

```
RetentionPolicy.SOURCE
RetentionPolicy.CLASS
RetentionPolicy.RUNTIME
```

`RetentionPolicy.SOURCE`

```
@Retention(RetentionPolicy.SOURCE)
```

The annotation exists only in the source code and is discarded during compilation.

It is useful for compiler-related tools but cannot be inspected by Spring while the application is running.

`RetentionPolicy.CLASS`

```
@Retention(RetentionPolicy.CLASS)
```

The annotation is stored in the compiled `.class` file but is not normally available through runtime reflection.

`CLASS` is Java's default retention policy when `@Retention` is not specified.

`RetentionPolicy.RUNTIME`

```
@Retention(RetentionPolicy.RUNTIME)
```

Coder Army

Coder Army

The annotation remains available while the application is running.

Spring and Java reflection can therefore inspect it at runtime.

Custom annotations used for Spring AOP should normally use:

```
@Retention(RetentionPolicy.RUNTIME)
```

Without runtime retention, Spring cannot reliably detect and bind the annotation while executing the application.

---

6. Understanding `@Documented`

`@Documented` tells Java documentation tools to include the annotation in generated Javadoc.

```
@Documented
public @interface TrackExecutionTime {
}
```

It improves documentation but does not affect AOP matching or proxy creation.

A typical method-level AOP annotation therefore looks like this:

```
import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface TrackExecutionTime {
}
```

Coder Army

7. Marker Annotations and Configured Annotations

A custom annotation can either act as a simple marker or carry additional configuration.

Marker annotation

A marker annotation has no properties:

```
public @interface TrackExecutionTime {
}
```

It communicates only one thing:

```
Apply execution-time tracking to this method.
```

Usage:

```
@TrackExecutionTime
public Report generateReport() {
    return reportGenerator.generate();
}
```

Annotation with properties

An annotation can also carry values:

```
public @interface TrackExecutionTime {

    String operation() default "";

    long warnAfterMillis() default 1000;
}
```

Coder Army

Usage:

```
@TrackExecutionTime(
    operation = "Generate annual report",
    warnAfterMillis = 500
)
public Report generateAnnualReport() {
    return reportGenerator.generate();
}
```

The method now declares:

- That execution time should be measured
- The name of the operation
- The threshold after which the operation should be treated as slow

The aspect can read these values:

```
trackExecutionTime.operation()
```

```
trackExecutionTime.warnAfterMillis()
```

This makes the annotation both a marker and a small configuration object.

---

8. The `@annotation()` Pointcut Designator

The `@annotation()` pointcut designator matches methods based on an annotation present on the method being executed.

It can be used in two important ways:

1. Type matching
2. Annotation binding

8.1 Type-Matching Form

Coder Army

```
@annotation(com.coderarmy.student.aop.annotation.TrackExecuti
onTime)
```

This asks:

> Is the executed method annotated with `TrackExecutionTime`?

Use this form when the advice only needs to know whether the annotation exists.

```
@Before(
    "@annotation(com.coderarmy.student.aop.annotation.TrackEx
ecutionTime)"
)
public void beforeTrackedMethod() {
    System.out.println("Tracked method called");
}
```

The fully qualified annotation name is written directly inside the pointcut expression.

8.2 Binding Form

```
@annotation(trackExecutionTime)
```

This form performs two jobs:

1. It matches methods containing the annotation
2. It passes the actual annotation instance to the advice method

```
@Before("@annotation(trackExecutionTime)")
public void beforeTrackedMethod(
        TrackExecutionTime trackExecutionTime
) {
    System.out.println(
        trackExecutionTime.operation()
```

```
    );
}
```

The name used in the pointcut expression corresponds to the advice parameter:

```
@annotation(trackExecutionTime)
            └─────────────────┘

TrackExecutionTime trackExecutionTime
                   └─────────────────┘
```

Binding is required when the aspect needs to read annotation properties such as:

```
operation()
warnAfterMillis()
```

> Type matching checks whether the annotation is present. Binding also gives the advice access to the annotation object.

---

9. Complete Example: Measuring Execution Time

Step 1: Create the annotation

```
package com.coderarmy.student.aop.annotation;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
```

Coder Army

Coder Army

```
@Documented
public @interface TrackExecutionTime {

    String operation() default "";

    long warnAfterMillis() default 1000;
}
```

Step 2: Apply it to a service method

```
package com.coderarmy.student.service;

import com.coderarmy.student.aop.annotation.TrackExecutionTim
e;
import org.springframework.stereotype.Service;

@Service
public class ReportService {

    @TrackExecutionTime(
        operation = "Generate annual student report",
        warnAfterMillis = 500
    )
    public StudentReport generateAnnualReport() {
        return createReport();
    }

    private StudentReport createReport() {
        return new StudentReport();
    }
}
```

The service method contains only business logic. It does not contain timing, logging or threshold-handling code.

Coder Army

Step 3: Create the aspect

```
package com.coderarmy.student.aop.aspect;

import com.coderarmy.student.aop.annotation.TrackExecutionTim
e;
import java.util.concurrent.TimeUnit;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class PerformanceAspect {

    @Around("@annotation(trackExecutionTime)")
    public Object measureExecutionTime(
            ProceedingJoinPoint joinPoint,
            TrackExecutionTime trackExecutionTime
    ) throws Throwable {

        long startTime = System.nanoTime();

        try {
            return joinPoint.proceed();
        } finally {
            long durationNanos =
                    System.nanoTime() - startTime;

            long durationMillis =
                    TimeUnit.NANOSECONDS.toMillis(
                            durationNanos
                    );
```

Coder Army

```
            String operation =
                    trackExecutionTime.operation();

            if (operation.isBlank()) {
                operation =
                        joinPoint.getSignature()
                                .toShortString();
            }

            long warningThreshold =
                    trackExecutionTime
                            .warnAfterMillis();

            if (durationMillis >= warningThreshold) {
                System.out.println(
                        "SLOW OPERATION: "
                        + operation
                        + " took "
                        + durationMillis
                        + " ms"
                );
            } else {
                System.out.println(
                        operation
                        + " took "
                        + durationMillis
                        + " ms"
                );
            }
        }
    }
}
```

Execution flow

Coder Army

When another Spring bean calls `generateAnnualReport()`:

```
Caller
   ↓
AOP proxy
   ↓
PerformanceAspect.measureExecutionTime()
   ↓
joinPoint.proceed()
   ↓
ReportService.generateAnnualReport()
   ↓
Result returns to the aspect
   ↓
Aspect calculates and logs the duration
   ↓
Result returns to the caller
```

The `finally` block ensures that duration is calculated whether the target method:

- Returns successfully
- Throws an exception

The original exception is not swallowed because the advice does not catch and replace it.

---

10. Where Spring AOP Enters the Bean Lifecycle

At a simplified level, Spring creates a bean through the following stages:

```
1. Read the bean definition
2. Instantiate the bean
3. Inject dependencies
4. Run Aware callbacks
5. Run BeanPostProcessors before initialization
6. Run initialization callbacks
7. Run BeanPostProcessors after initialization
8. Store and expose the final bean reference
```

Coder Army

Spring AOP's automatic proxy creation is built on Spring's bean post-processing infrastructure.

An AOP auto-proxy creator examines beans during their creation and determines whether any configured advice applies to them.

Conceptually:

```
Spring creates the StudentService target
                    ↓
Injects its dependencies
                    ↓
Runs initialization callbacks
                    ↓
AOP infrastructure examines the bean
                    ↓
Does any advisor match this bean?
            ↙                    ↘
          No                      Yes
          ↓                        ↓
Return original bean        Create AOP proxy
                                   ↓
                            Return the proxy
```

For an eligible bean, the final reference exposed by the container is normally:

```
Proxy → Target
```

rather than only:

```
Target
```

> AOP advice does not run while the bean is merely being created. Spring first creates the proxy. Advice runs later when a method is invoked through that proxy.

---

11. BeanPostProcessor from First Principles

Coder Army

`BeanPostProcessor` is a Spring extension point that allows infrastructure code to inspect or modify beans created by the container.

Its basic idea is:

> Whenever Spring creates a bean, infrastructure components get an opportunity to process it before the bean is finally exposed.

A simplified form of the interface is:

```
public interface BeanPostProcessor {

    Object postProcessBeforeInitialization(
            Object bean,
            String beanName
    );

    Object postProcessAfterInitialization(
            Object bean,
            String beanName
    );
}
```

A post-processor may return the original bean:

```
return bean;
```

or return a wrapped object:

```
return proxy;
```

`postProcessBeforeInitialization()`

This method runs before initialization callbacks such as:

- `@PostConstruct`
- `InitializingBean.afterPropertiesSet()`

Coder Army

- A custom `initMethod`

It may be used to:

- Inspect the bean
- Perform additional configuration
- Process certain annotations
- Validate or prepare the bean before initialization

`postProcessAfterInitialization()`

This method runs after initialization callbacks have completed.

Spring's AOP auto-proxy creation normally wraps eligible beans around this stage and returns the proxy as the final bean reference.

```
Original bean created
        ↓
Initialization completed
        ↓
AOP post-processing
        ↓
Original bean or proxy returned
```

The exact internal implementation is more advanced, but the important conceptual point is:

> Spring AOP uses bean post-processing to replace an eligible bean reference with a proxy reference.

---

12. How Spring Decides Which Beans Need Proxies

Consider the following aspect:

```
@Aspect
@Component
```

Coder Army

```java
public class LoggingAspect {

    @Before(
        "within(com.coderarmy.student.service..*)"
    )
    public void log() {
        System.out.println("Service method called");
    }
}
```

Spring processes the aspect and creates an internal representation of:

```
Advice
    → Run log()

Pointcut
    → Match classes inside the service package
```

The combination of a pointcut and advice is commonly represented internally as an **advisor**.

As Spring creates beans, the AOP infrastructure checks whether the advisors can apply to each bean.

For `StudentController`:

```
Is StudentController inside the service package?
No
```

No proxy is required for this aspect.

For `StudentService`:

```
Is StudentService inside the service package?
Yes
```

Spring creates an AOP proxy for the service bean.

Coder Army

The same idea applies to an annotation-based pointcut:

```
@Around("@annotation(trackExecutionTime)")
```

Spring checks whether the bean contains methods that can match the annotation-based advisor. Eligible calls are intercepted by the proxy at runtime.

---

13. The Container Stores the Final Reference

Suppose Spring first creates the target object:

```
Target object: T100
```

It then creates a proxy:

```
Proxy object: P200
```

The relationship becomes:

```
P200: Proxy
   ↓
T100: Target
```

When another bean asks Spring for `StudentService`, the container supplies:

```
P200
```

not the raw target reference:

```
T100
```

For example:

Coder Army

```
public StudentController(
        StudentService studentService
) {
    this.studentService = studentService;
}
```

Conceptually, the field stores the proxy:

```
StudentController
studentService field
       |
       ▼
    AOP proxy
       |
       ▼
  StudentService target
```

This is why dependency injection and AOP work together.

Spring controls:

- Object creation
- Dependency injection
- Bean post-processing
- The final reference supplied to other beans

Therefore, Spring can inject the proxy instead of the raw object.

---

14. The Method Interceptor Chain

A single method may match multiple cross-cutting concerns.

For example:

```
Logging
Security
Performance measurement
```

Coder Army

The runtime chain may look like this:

```
Caller
   ↓
Proxy
   ↓
Logging interceptor
   ↓
Security interceptor
   ↓
Performance interceptor
   ↓
Target method
```

When the target method completes, execution unwinds in reverse order:

```
Target method returns
   ↓
Performance interceptor completes
   ↓
Security interceptor completes
   ↓
Logging interceptor completes
   ↓
Proxy returns the result
   ↓
Caller receives the result
```

This behaves like nested method calls:

```
Logging(
    Security(
        Performance(
            Target method
        )
    )
)
```

Each interceptor gets an opportunity to run code:

- Before continuing the chain
- After the chain returns

- When the chain throws an exception

15. What `proceed()` Actually Means

A common beginner interpretation is:

```
proceed() directly calls the target method
```

That is not always precise.

`proceed()` means:

> Continue to the next interceptor in the current chain.

If another interceptor still remains, that interceptor runs next.

Only the final `proceed()` in the chain reaches the target method.

```
First around advice
    ↓ proceed()
Second around advice
    ↓ proceed()
Third around advice
    ↓ proceed()
Target method
```

An `@Around` advice controls whether the chain continues.

If it does not call `proceed()`:

```
@Around("execution(* com.coderarmy..*(..))")
public Object intercept(
        ProceedingJoinPoint joinPoint
) {
    return "Blocked";
}
```

then:

Coder Army

Coder Army

- Remaining interceptors are not invoked
- The target method is not executed
- The advice directly returns its own value

Therefore, `@Around` is the most powerful advice type.

---

16. Controlling Aspect Order with `@Order`

When multiple aspects match the same method, `@Order` controls their precedence.

```
@Aspect
@Component
@Order(1)
public class SecurityAspect {
}
```

```
@Aspect
@Component
@Order(2)
public class LoggingAspect {
}
```

```
@Aspect
@Component
@Order(3)
public class PerformanceAspect {
}
```

The fundamental rule is:

> A lower order value has higher precedence.

For the example above, execution enters the aspects in this order:

Coder Army

```
SecurityAspect      @Order(1)
        ↓
LoggingAspect       @Order(2)
        ↓
PerformanceAspect   @Order(3)
        ↓
Target method
```

After the target method completes, execution returns in reverse order:

```
Target method
        ↓
PerformanceAspect completes
        ↓
LoggingAspect completes
        ↓
SecurityAspect completes
```

Another way to visualize it is:

```
Security(
    Logging(
        Performance(
            Target method
        )
    )
)
```

The lower order value forms the outer layer of the interceptor chain.

---

17. Important Proxy-Based AOP Boundaries

Custom annotation-based AOP works only when the method call passes through the Spring AOP proxy.

The object must be managed by Spring

A manually created object is not automatically proxied:

Coder Army

```
ReportService service = new ReportService();
```

Calling an annotated method on this object bypasses Spring AOP.

The object should be obtained from the Spring container through dependency injection.

Self-invocation bypasses the proxy

Consider:

```
@Service
public class ReportService {

    public void generateAllReports() {
        generateAnnualReport();
    }

    @TrackExecutionTime
    public void generateAnnualReport() {
    }
}
```

When `generateAllReports()` calls `generateAnnualReport()` using `this`, the call stays inside the target object:

```
Target method
   ↓
this.generateAnnualReport()
```

It does not return to the proxy, so the annotation-based advice is not applied to that internal call.

Private methods are not suitable join points for Spring proxy-based AOP

Spring AOP intercepts method calls through proxies. Private methods cannot be overridden or exposed through a proxy in the required way.

Custom AOP annotations should therefore normally be applied to externally invoked, non-private methods on Spring-managed beans.

Coder Army

18. Complete Mental Model

During application startup

```
Spring reads bean definitions
        ↓
Creates target objects
        ↓
Injects dependencies
        ↓
Runs initialization callbacks
        ↓
AOP infrastructure checks available advisors
        ↓
Creates proxies for eligible beans
        ↓
Stores and injects the final proxy references
```

During method execution

```
Caller invokes a method
        ↓
Call reaches the proxy
        ↓
Proxy finds matching interceptors
        ↓
Interceptor chain begins
        ↓
Around advice calls proceed()
        ↓
Target method executes
        ↓
Chain unwinds in reverse order
        ↓
Result or exception returns to the caller
```

Coder Army

Responsibility of each part

```
Custom annotation
    → Declares the required behaviour

Pointcut expression
    → Selects methods carrying that annotation

Advice
    → Implements the cross-cutting behaviour

AOP proxy
    → Intercepts method calls

BeanPostProcessor infrastructure
    → Creates and exposes the proxy during bean creation

ProceedingJoinPoint.proceed()
    → Continues the interceptor chain
```

---

19. Key Takeaways

- A custom annotation is metadata; it has no behaviour by itself.
- `@Target` controls where an annotation may be used.
- `@Retention(RUNTIME)` keeps the annotation available for Spring at runtime.
- `@Documented` affects generated documentation, not AOP behaviour.
- A marker annotation only declares that behaviour is required.
- Annotation properties can provide configuration to an aspect.
- `@annotation(TypeName)` matches an annotation by type.
- `@annotation(variableName)` binds the annotation instance to an advice parameter.
- Spring AOP uses bean post-processing infrastructure to create proxies for eligible beans.
- The Spring container stores and injects the final proxy reference.
- A method call must pass through the proxy for advice to execute.

- `proceed()` continues the interceptor chain; it does not always call the target immediately.
- Multiple interceptors execute as a nested chain and unwind in reverse order.
- A lower `@Order` value gives an aspect higher precedence.
- Self-invocation and private methods are important limitations of proxy-based Spring AOP.

Coder Army

Coder Army

JDBC Fundamentals and CRUD Operations

> A first-principles guide to connecting a Java application to MySQL and performing CRUD operations using JDBC.

Table of Contents

1. How Java Communicates with a Database
2. JDBC API and JDBC Driver
3. Installing MySQL Server and DBeaver
4. Creating the JDBC Project
5. Creating the Database and Table
6. Establishing a Database Connection
7. Executing SQL with JDBC
8. Understanding `ResultSet`
9. Why `PreparedStatement` Is Preferred
10. Complete CRUD Operations
11. Closing JDBC Resources
12. Common Setup Problems
13. Key Takeaways

1. How Java Communicates with a Database

Java and MySQL are separate programs

Coder Army

A Java application normally runs inside the JVM:

```
Java Application
       ↓
      JVM
```

MySQL runs as a separate server process:

```
MySQL Server
       ↓
Stores and manages data
```

The two programs may run:

- On the same computer
- On different computers in the same network
- On remote servers in different locations

Calling a database is therefore different from calling a normal Java method.

Consider the following code:

```java
String sql = "SELECT * FROM students";
```

This only creates a Java `String` in JVM memory. The SQL has not yet reached MySQL.

The application still needs a mechanism that can:

1. Establish a connection with MySQL
2. Send SQL to the server
3. Wait for the server response
4. Convert the response into objects Java can work with

JDBC provides the standard Java-side API for this communication.

Why a database driver is required

Coder Army

Every database server follows its own communication protocol.

For example:

- MySQL uses the MySQL client-server protocol
- PostgreSQL uses the PostgreSQL protocol
- Oracle Database uses its own protocol

These protocols define:

- How a client opens a connection
- How authentication data is transmitted
- How SQL commands are packaged
- How SQL data types are represented
- How rows and metadata are returned
- How errors are reported
- How transactions are controlled

Java does not contain the implementation of every database protocol. It uses a database-specific JDBC driver as the translator.

```
Java Application
       ↓
    JDBC API
       ↓
MySQL JDBC Driver
       ↓
  MySQL Server
```

---

2. JDBC API and JDBC Driver

What is JDBC?

JDBC stands for **Java Database Connectivity**.

It is a standard Java API used to communicate with relational databases.

Coder Army

The main JDBC types are available in the `java.sql` package:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
```

JDBC gives Java developers a common programming model for databases such as MySQL, PostgreSQL, Oracle and Microsoft SQL Server.

The database URL and driver may change, but the JDBC interfaces remain largely the same.

MySQL connection

```
Connection connection = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/student_db",
        "root",
        "password"
);
```

PostgreSQL connection

```
Connection connection = DriverManager.getConnection(
        "jdbc:postgresql://localhost:5432/student_db",
        "postgres",
        "password"
);
```

This provides portability at the API level. It does not mean that every database supports identical SQL syntax or behaviour.

JDBC API versus JDBC driver

Coder Army

JDBC API

The JDBC API defines the contracts used by Java code.

Important interfaces include:

```
Connection
Statement
PreparedStatement
ResultSet
```

For example, `Connection` defines operations such as creating statements, managing transactions and closing a database session.

Your application normally programs against these interfaces rather than database-specific implementation classes.

JDBC driver

The JDBC driver provides the database-specific implementation of the JDBC contracts.

For MySQL, the official driver is **MySQL Connector/J**.

Database	Common JDBC driver
MySQL	MySQL Connector/J
PostgreSQL	PostgreSQL JDBC Driver
Oracle Database	Oracle JDBC Driver
Microsoft SQL Server	Microsoft JDBC Driver for SQL Server
H2	H2 JDBC Driver

What happens internally?

When Java executes a query through JDBC, the flow is approximately:

1. The Java application calls a JDBC method.
2. The JDBC driver receives the SQL command and parameter values.
3. The driver converts them into a database-specific network request.

Coder Army

4. The request is sent to the database server.
5. The server parses and validates the SQL.
6. The server creates an execution plan.
7. The server executes the operation.
8. The server returns rows, metadata, an update count or an error.
9. The driver decodes the database response.
10. JDBC exposes the response through types such as `ResultSet`.
11. The Java application reads and maps the returned data.

---

3. Installing MySQL Server and DBeaver

MySQL Server, DBeaver and the JDBC driver are different

Component	Purpose
MySQL Server	Stores data and executes SQL
DBeaver	Desktop client used to view databases and run SQL manually
MySQL Connector/J	Java library that allows the application to communicate with MySQL

Installing DBeaver does **not** install or start MySQL Server. The server must be installed and running separately.

3.1 Install MySQL Server on macOS

Option A: Official MySQL package installer

1. Open the official MySQL Community Server download page.
2. Select **macOS**.
3. Download the DMG package that matches your Mac:
    - Apple Silicon for M-series Macs

Coder Army

- x86-64 for Intel Macs

4. Open the downloaded `.dmg` file.
5. Run the MySQL `.pkg` installer.
6. Complete the installation wizard.
7. Set and safely store the MySQL `root` password when prompted.

The official package normally installs MySQL under:

```
/usr/local/mysql
```

Start MySQL from macOS settings

Depending on the installer and macOS version:

1. Open **System Settings**.
2. Search for **MySQL**.
3. Open the MySQL settings panel.
4. Select **Start MySQL Server**.

Start MySQL from Terminal

```
sudo /usr/local/mysql/support-files/mysql.server start
```

Check its status:

```
sudo /usr/local/mysql/support-files/mysql.server status
```

Stop the server:

```
sudo /usr/local/mysql/support-files/mysql.server stop
```

Restart the server:

Coder Army

```
sudo /usr/local/mysql/support-files/mysql.server restart
```

Connect through the MySQL command-line client

```
/usr/local/mysql/bin/mysql -u root -p
```

To make the `mysql` command available without its full path, add MySQL to the shell path:

```
echo 'export PATH="/usr/local/mysql/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

You can then run:

```
mysql -u root -p
```

Option B: Install MySQL with Homebrew

This option is convenient for developers who already use Homebrew.

```
brew install mysql
```

Start MySQL and configure it to launch automatically:

```
brew services start mysql
```

Check the service:

```
brew services list
```

Stop it:

```
brew services stop mysql
```

Coder Army

Restart it:

```
brew services restart mysql
```

Run the security setup:

```
mysql_secure_installation
```

Connect to the server:

```
mysql -u root -p
```

> Use either the official package installation or Homebrew. Avoid maintaining two separate MySQL installations unless you understand how their service paths and ports differ.

3.2 Install MySQL Server on Windows

1. Open the official MySQL Community Server download page.
2. Download the Windows MSI package.
3. Run the installer as an administrator.
4. Open the MySQL Configurator when prompted.
5. Configure MySQL as a standalone server.
6. Keep port `3306` unless another program already uses it.
7. Set and safely store the `root` password.
8. Configure MySQL as a **Windows service**.
9. Keep **Start the MySQL Server at System Startup** selected for a beginner setup.
10. Apply the configuration.

Start MySQL using Windows Services

Coder Army

1. Press `Win + R`.
2. Enter:

```
services.msc
```

1. Find the MySQL service. Its name may be similar to `MySQL80` or `MySQL84`.
2. Right-click it and select **Start**, **Stop** or **Restart**.

Start MySQL from Command Prompt

Open Command Prompt as administrator.

First, identify the actual service name:

```
sc query type= service state= all | findstr /I mysql
```

Start the service:

```
net start <service-name>
```

Example:

```
net start MySQL84
```

Stop the service:

```
net stop <service-name>
```

You may also use:

```
sc start <service-name>
sc stop <service-name>
```

PowerShell alternative

Open PowerShell as administrator:

Coder Army

```
Get-Service *mysql*
Start-Service -Name <service-name>
Stop-Service -Name <service-name>
Restart-Service -Name <service-name>
```

Connect through the command-line client

If the MySQL `bin` directory is available in `PATH`:

```
mysql -u root -p
```

Otherwise, run it from the installation directory, which is commonly similar to:

```
C:\Program Files\MySQL\MySQL Server 8.4\bin
```

The exact directory depends on the installed version.

3.3 Verify that MySQL is running

Log in:

```
mysql -u root -p
```

Then run:

```
SELECT VERSION();
SHOW DATABASES;
```

Exit the MySQL client:

```
EXIT;
```

If the command-line login works, the server is running and accepting connections.

Coder Army

3.4 Install DBeaver Community

Download DBeaver Community from the official DBeaver download page.

macOS

1. Download the correct DMG for Apple Silicon or Intel.
2. Open the DMG.
3. Drag DBeaver into the **Applications** folder.
4. Open DBeaver from Applications.

Homebrew alternative:

```
brew install --cask dbeaver-community
```

Windows

1. Download the Windows installer.
2. Run the installer executable.
3. Follow the installation wizard.
4. Launch DBeaver.

3.5 Connect DBeaver to MySQL

1. Make sure MySQL Server is running.
2. Open DBeaver.
3. Select **New Database Connection**.
4. Choose **MySQL**.
5. Enter the connection details:

Field	Value
Host	`localhost`
Port	`3306`

Coder Army

Field	Value
Database	`student_db` or leave empty initially
Username	`root`
Password	Your MySQL root password

1. Allow DBeaver to download the MySQL driver if it asks.
2. Select **Test Connection**.
3. Select **Finish** after the test succeeds.

---

4. Creating the JDBC Project

Recommended project structure

```
jdbc-crud-demo
│
├── pom.xml
└── src
    └── main
        └── java
            └── com
                └── coderarmy
                    └── jdbc
                        ├── Main.java
                        ├── model
                        │   └── Student.java
                        └── repository
                            └── StudentRepository.java
```

This is a plain Maven project. Spring Boot is not required to learn raw JDBC.

Add MySQL Connector/J

Add the MySQL JDBC driver to `pom.xml`:

```xml
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
```

```
    <version>9.7.0</version>
    <scope>runtime</scope>
</dependency>
```

> Dependency versions change over time. Use the current stable Connector/J version when creating a new project.

A minimal `pom.xml` can look like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instanc
e"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.
0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.coderarmy</groupId>
    <artifactId>jdbc-crud-demo</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.so
urceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>9.7.0</version>
            <scope>runtime</scope>
        </dependency>
```

Coder Army

```
    </dependencies>

</project>
```

Coder Army

5. Creating the Database and Table

Open an SQL editor in DBeaver or use the MySQL command-line client.

Create the database

```sql
CREATE DATABASE student_db;
```

Select it:

```sql
USE student_db;
```

Create the `students` table

```sql
CREATE TABLE students (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    age INT
);
```

Verify the table:

```sql
DESCRIBE students;
```

Create the `Student` model

Coder Army

```
package com.coderarmy.jdbc.model;

public class Student {

    private Long id;
    private String name;
    private String email;
    private Integer age;

    public Student() {
    }

    public Student(String name, String email, Integer age) {
        this.name = name;
        this.email = email;
        this.age = age;
    }

    public Student(Long id, String name, String email, Intege
r age) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.age = age;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
```

Coder Army

```
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public Integer getAge() {
        return age;
    }

    public void setAge(Integer age) {
        this.age = age;
    }

    @Override
    public String toString() {
        return "Student{" +
                "id=" + id +
                ", name='" + name + '\'' +
                ", email='" + email + '\'' +
                ", age=" + age +
                '}';
    }
}
```

Coder Army

6. Establishing a Database Connection

What is a database connection?

A database connection represents an active communication session between the Java application and the database server.

In JDBC, it is represented by the `Connection` interface:

```
Connection connection;
```

A connection allows Java to:

- Send SQL commands
- Receive query results
- Create statements
- Start and commit transactions
- Roll back transactions
- Read database metadata

Opening a database connection involves network communication and authentication. It is more expensive than creating an ordinary Java object.

JDBC connection details

```
String url = "jdbc:mysql://localhost:3306/student_db";
String username = "root";
String password = "your_password";
```

URL breakdown

```
jdbc:mysql://localhost:3306/student_db
|    |      |         |      |
|    |      |         |      └── Database name
|    |      |         └──────── Port
```

Coder Army

```
|     |       └──────────────────── Host
|     └──────────────────────────── Database type
└────────────────────────────────── JDBC URL prefix
```

For learning, these values may be kept as constants.

In production code, credentials should not be committed to source control. Use environment variables, configuration files excluded from Git or a secret-management system.

> `DriverManager` does not automatically read Spring Boot's `application.properties`. Values must be passed directly unless Spring is creating and configuring a `DataSource`.

Is `Class.forName()` still required?

Older JDBC applications explicitly loaded the MySQL driver:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

This forced the driver class to load and register itself with `DriverManager`.

Since JDBC 4.0, compatible drivers can be discovered and registered automatically when the driver JAR is present on the classpath.

Therefore, modern JDBC applications generally do not need `Class.forName()`.

How `DriverManager` creates a connection

```
Connection connection = DriverManager.getConnection(
        url,
        username,
        password
);
```

Conceptually, `DriverManager`:

1. Looks at the JDBC URL.

Coder Army

2. Checks the registered JDBC drivers.
3. Finds the driver that supports `jdbc:mysql:`.
4. Asks that driver to connect to MySQL.
5. Sends the authentication details.
6. Returns a MySQL-specific implementation of `Connection` through the JDBC interface.

Your code depends on:

```
Connection
```

rather than a MySQL implementation class. This is interface-based abstraction in practice.

Basic connection program

```
package com.coderarmy.jdbc;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Main {

    private static final String URL =
            "jdbc:mysql://localhost:3306/student_db";

    private static final String USERNAME = "root";
    private static final String PASSWORD = "your_password";

    public static void main(String[] args) {

        try (Connection connection = DriverManager.getConnect
ion(
```

Coder Army

```
                URL,
                USERNAME,
                PASSWORD
        )) {
            System.out.println("Database connected successful
ly");

        } catch (SQLException exception) {
            System.out.println("Database connection failed");
            System.out.println("Message: " + exception.getMes
sage());
            System.out.println("SQL state: " + exception.getS
QLState());
            System.out.println("Error code: " + exception.get
ErrorCode());
        }
    }
}
```

Useful information available from `SQLException`:

```
exception.getMessage();
exception.getSQLState();
exception.getErrorCode();
```

7. Executing SQL with JDBC

From a connection to a statement

A `Connection` represents the database session. It does not represent one specific SQL command.

A statement object is created to execute SQL through that connection.

Coder Army

```
Connection connection = DriverManager.getConnection(
        URL,
        USERNAME,
        PASSWORD
);

Statement statement = connection.createStatement();
```

```
Connection
    └── Statement
```

The statement uses its parent connection to communicate with MySQL.

Main JDBC execution methods

Method	Typical use	Return value
`executeQuery()`	`SELECT`	`ResultSet`
`executeUpdate()`	`INSERT`, `UPDATE`, `DELETE`, some DDL	Affected-row count
`execute()`	Unknown or mixed result type	`boolean` indicating the first result type

`executeUpdate()`

Despite its name, `executeUpdate()` is used for more than SQL `UPDATE`.

Typical uses:

```
INSERT
UPDATE
DELETE
```

Example:

Coder Army

```
int rowsAffected = statement.executeUpdate(
        """
        INSERT INTO students(name, email, age)
        VALUES ('Rahul', 'rahul@gmail.com', 24)
        """
);
```

The returned integer represents the number of affected rows.

```
Successful single-row insert  → 1
No row matched an update      → 0
Multiple rows affected        → more than 1
```

It can also execute DDL such as:

```
CREATE TABLE
ALTER TABLE
DROP TABLE
```

DDL commonly returns `0` because it does not modify table rows in the same way as DML.

`executeQuery()`

`executeQuery()` is used when SQL returns tabular data, normally with `SELECT`.

```
ResultSet resultSet = statement.executeQuery(
        "SELECT id, name, email, age FROM students"
);
```

The returned `ResultSet` gives Java access to the rows returned by MySQL.

JDBC does not automatically convert these rows into `Student` objects. The application must map them manually.

Coder Army

`execute()`

`execute()` is a general-purpose method:

```
boolean hasResultSet = statement.execute(sql);
```

The boolean does **not** mean success or failure.

```
true  → The first result is a ResultSet
false → The first result is an update count or there is no result
```

Example:

```
boolean hasResultSet = statement.execute(sql);

if (hasResultSet) {
    ResultSet resultSet = statement.getResultSet();
} else {
    int rowsAffected = statement.getUpdateCount();
}
```

Use `execute()` when:

- The SQL type is not known in advance
- A stored procedure can return different result types
- Multiple results may be returned
- Infrastructure code needs a generic execution method

For normal CRUD code, prefer the more specific methods:

```
SELECT                → executeQuery()
INSERT/UPDATE/DELETE  → executeUpdate()
```

---

8. Understanding `ResultSet`

Coder Army

`ResultSet` is not a `List<Student>`

Suppose MySQL returns:

id	name	email	age
1	Rahul	rahul@gmail.com	24
2	Priya	priya@gmail.com	22

JDBC exposes these rows through `ResultSet`.

The developer must:

1. Move the cursor to a row.
2. Read each required column.
3. Create a Java object.
4. Assign the values.
5. Add the object to a collection when reading multiple rows.

The `ResultSet` cursor

When a `ResultSet` is created, its cursor is positioned **before the first row**.

```
Cursor → before first row

Row 1
Row 2
Row 3
```

Calling:

```
resultSet.next();
```

moves the cursor to the next row and returns:

- `true` when a row is available
- `false` when no more rows remain

For one expected row:

Coder Army

```
if (resultSet.next()) {
    // Read the current row
}
```

For multiple rows:

```
while (resultSet.next()) {
    // Read each row
}
```

Reading values

By column label

```
long id = resultSet.getLong("id");
String name = resultSet.getString("name");
String email = resultSet.getString("email");
int age = resultSet.getInt("age");
```

By column position

```
long id = resultSet.getLong(1);
String name = resultSet.getString(2);
String email = resultSet.getString(3);
int age = resultSet.getInt(4);
```

Column labels are usually clearer because the code states which value it is reading.

Handling nullable numeric columns

Calling:

Coder Army

```
resultSet.getInt("age");
```

returns primitive `int`. If the SQL value is `NULL`, JDBC returns `0`, and the code must call `wasNull()` to distinguish SQL `NULL` from a real zero.

A clearer alternative for a nullable `Integer` field is:

```
Integer age = resultSet.getObject("age", Integer.class);
```

Prefer explicit columns over `SELECT *`

Instead of:

```
SELECT *
FROM students;
```

prefer:

```
SELECT id, name, email, age
FROM students;
```

Benefits:

- The required columns are visible
- Unnecessary columns are not retrieved
- Mapping is more predictable
- Schema changes are less likely to surprise the application
- The repository clearly communicates the data it needs

`SELECT *` is convenient during exploration, but explicit columns are better in application code.

Mapping one row to a `Student`

Coder Army

```
private Student mapRow(ResultSet resultSet) throws SQLExcepti
on {

    Student student = new Student();

    student.setId(resultSet.getLong("id"));
    student.setName(resultSet.getString("name"));
    student.setEmail(resultSet.getString("email"));
    student.setAge(resultSet.getObject("age", Integer.clas
s));

    return student;
}
```

This method removes duplicate mapping code from repository methods.

The mapping is still manual. JDBC does not know that a row from `students` should become a `Student` object.

---

9. Why `PreparedStatement` Is Preferred

The problem with plain `Statement`

A hardcoded statement is simple:

```
String sql = """
        INSERT INTO students(name, email, age)
        VALUES ('Rahul', 'rahul@gmail.com', 24)
        """;
```

Real applications receive dynamic values:

```
String name = student.getName();
String email = student.getEmail();
```

Coder Army

```
Integer age = student.getAge();
```

Building SQL through concatenation is unsafe and error-prone:

```
String sql =
        "INSERT INTO students(name, email, age) VALUES ('"
                + name + "', '"
                + email + "', "
                + age + ")";
```

Formatting a text block does not solve the underlying problem:

```
String sql = """
        INSERT INTO students(name, email, age)
        VALUES ('%s', '%s', %d)
        """.formatted(name, email, age);
```

Problems include:

- SQL injection
- Broken quotation when text contains apostrophes
- Incorrect handling of `NULL`
- Manual type formatting
- Date and time conversion issues
- Binary-data handling issues
- Poor readability and maintainability

Parameterized SQL

`PreparedStatement` separates the SQL structure from its values.

```
String sql = """
        INSERT INTO students(name, email, age)
        VALUES (?, ?, ?)
```

Coder Army

```
        """;

PreparedStatement preparedStatement =
        connection.prepareStatement(sql);

preparedStatement.setString(1, student.getName());
preparedStatement.setString(2, student.getEmail());
preparedStatement.setObject(3, student.getAge());

int rowsAffected = preparedStatement.executeUpdate();
```

The SQL is sent as a parameterized command, while the values are supplied separately.

Positional parameters

In this query:

```
INSERT INTO students(name, email, age)
VALUES (?, ?, ?)
```

the placeholders mean:

```
First ?   → name
Second ?  → email
Third ?   → age
```

JDBC parameter indexes begin at `1`, not `0`.

Common setter methods

```
setString()
setInt()
setLong()
setDouble()
```

Coder Army

```
setBoolean()
setBigDecimal()
setDate()
setTimestamp()
setObject()
setNull()
```

For a nullable value:

```
if (student.getAge() == null) {
    preparedStatement.setNull(3, java.sql.Types.INTEGER);
} else {
    preparedStatement.setInt(3, student.getAge());
}
```

A simpler alternative is often:

```
preparedStatement.setObject(3, student.getAge());
```

Does `PreparedStatement` completely prevent SQL injection?

It safely handles values supplied through placeholders:

```
String sql = "SELECT id, name, email, age FROM students WHERE
email = ?";
preparedStatement.setString(1, email);
```

However, placeholders cannot normally represent SQL identifiers or keywords.

This remains unsafe:

```
String sql = "SELECT * FROM students ORDER BY " + requestedCo
lumn;
```

For dynamic table names, column names or sort directions, validate the input against a strict allow-list.

> Use placeholders for values. Use allow-list validation for unavoidable dynamic SQL structure.

---

10. Complete CRUD Operations

Repository configuration

```
package com.coderarmy.jdbc.repository;

import com.coderarmy.jdbc.model.Student;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class StudentRepository {

    private static final String URL =
            "jdbc:mysql://localhost:3306/student_db";

    private static final String USERNAME = "root";
    private static final String PASSWORD = "your_password";

    // CRUD methods
}
```

Coder Army

For a real application, move credentials outside the source code and use a connection pool. They remain inline here only to keep the first JDBC example focused.

10.1 Create: insert a student

```
public int save(Student student) {

    String sql = """
            INSERT INTO students(name, email, age)
            VALUES (?, ?, ?)
            """;

    try (
        Connection connection = DriverManager.getConnection(
                URL,
                USERNAME,
                PASSWORD
        );

        PreparedStatement preparedStatement =
                connection.prepareStatement(sql)
    ) {
        preparedStatement.setString(1, student.getName());
        preparedStatement.setString(2, student.getEmail());
        preparedStatement.setObject(3, student.getAge());

        return preparedStatement.executeUpdate();

    } catch (SQLException exception) {
        throw new RuntimeException(
                "Failed to save student",
                exception
        );
```

Coder Army

```
    }
}
```

Usage:

```
Student student = new Student(
        "Rahul",
        "rahul@gmail.com",
        24
);

int rowsAffected = repository.save(student);

if (rowsAffected == 1) {
    System.out.println("Student saved successfully");
}
```

Returning the generated ID

To retrieve an auto-generated key:

```
public long saveAndReturnId(Student student) {

    String sql = """
            INSERT INTO students(name, email, age)
            VALUES (?, ?, ?)
            """;

    try (
        Connection connection = DriverManager.getConnection(
                URL,
                USERNAME,
                PASSWORD
        );
```

Coder Army

Coder Army

```
        PreparedStatement preparedStatement =
                connection.prepareStatement(
                        sql,
                        Statement.RETURN_GENERATED_KEYS
                )
    ) {
        preparedStatement.setString(1, student.getName());
        preparedStatement.setString(2, student.getEmail());
        preparedStatement.setObject(3, student.getAge());

        int rowsAffected = preparedStatement.executeUpdate();

        if (rowsAffected != 1) {
            throw new RuntimeException("Student was not inser
ted");
        }

        try (ResultSet generatedKeys = preparedStatement.getG
eneratedKeys()) {
            if (generatedKeys.next()) {
                return generatedKeys.getLong(1);
            }
        }

        throw new RuntimeException("Generated ID was not retu
rned");

    } catch (SQLException exception) {
        throw new RuntimeException(
                "Failed to save student",
                exception
        );
    }
}
```

Coder Army

10.2 Read one: find a student by ID

```
public Optional<Student> findById(Long id) {

    String sql = """
            SELECT id, name, email, age
            FROM students
            WHERE id = ?
            """;

    try (
        Connection connection = DriverManager.getConnection(
                URL,
                USERNAME,
                PASSWORD
        );

        PreparedStatement preparedStatement =
                connection.prepareStatement(sql)
    ) {
        preparedStatement.setLong(1, id);

        try (ResultSet resultSet = preparedStatement.executeQ
uery()) {
            if (resultSet.next()) {
                return Optional.of(mapRow(resultSet));
            }

            return Optional.empty();
        }

    } catch (SQLException exception) {
        throw new RuntimeException(
                "Failed to fetch student with ID " + id,
                exception
```

```
        );
    }
}
```

Usage:

```
Optional<Student> optionalStudent = repository.findById(1L);

if (optionalStudent.isPresent()) {
    System.out.println(optionalStudent.get());
} else {
    System.out.println("Student not found");
}
```

A more concise usage:

```
repository.findById(1L)
        .ifPresentOrElse(
                System.out::println,
                () -> System.out.println("Student not found")
        );
```

`Optional.empty()` represents a normal “student not found” result. A database failure is different and should not be silently converted into `Optional.empty()`.

10.3 Read all: fetch every student

```
public List<Student> findAll() {

    String sql = """
            SELECT id, name, email, age
            FROM students
            """;

    List<Student> students = new ArrayList<>();
```

Coder Army

Coder Army

```
    try (
        Connection connection = DriverManager.getConnection(
                URL,
                USERNAME,
                PASSWORD
        );

        PreparedStatement preparedStatement =
                connection.prepareStatement(sql);

        ResultSet resultSet =
                preparedStatement.executeQuery()
    ) {
        while (resultSet.next()) {
            students.add(mapRow(resultSet));
        }

        return students;

    } catch (SQLException exception) {
        throw new RuntimeException(
                "Failed to fetch students",
                exception
        );
    }
}
```

Flow:

```
Move to the next row
        ↓
Read the columns
        ↓
Create a Student
        ↓
Add it to the List
```

```
        ↓
Repeat until next() returns false
```

10.4 Update a student

```
public int update(Long id, Student student) {

    String sql = """
            UPDATE students
            SET name = ?,
                email = ?,
                age = ?
            WHERE id = ?
            """;

    try (
        Connection connection = DriverManager.getConnection(
                URL,
                USERNAME,
                PASSWORD
        );

        PreparedStatement preparedStatement =
                connection.prepareStatement(sql)
    ) {
        preparedStatement.setString(1, student.getName());
        preparedStatement.setString(2, student.getEmail());
        preparedStatement.setObject(3, student.getAge());
        preparedStatement.setLong(4, id);

        return preparedStatement.executeUpdate();

    } catch (SQLException exception) {
        throw new RuntimeException(
```

Coder Army

```
                "Failed to update student with ID " + id,
                exception
        );
    }
}
```

Usage:

```
Student updatedStudent = new Student(
        "Rahul Sharma",
        "rahul.sharma@gmail.com",
        25
);

int rowsAffected = repository.update(1L, updatedStudent);

if (rowsAffected == 1) {
    System.out.println("Student updated successfully");
} else {
    System.out.println("Student not found");
}
```

Understanding the update count

For a simple beginner-level example:

```
1 → one row was matched or changed
0 → no row was matched, or the driver reported no changed row
```

Exact update-count semantics can vary by database and driver configuration. MySQL Connector/J settings such as `useAffectedRows` can influence whether the count reflects matched rows or changed rows.

Do not treat update count as the only source of truth for critical business rules without understanding the driver behaviour.

Coder Army

Coder Army

Should `findById()` be called before `update()`?

This approach performs two SQL operations:

```
SELECT student
       ↓
UPDATE student
```

Problems:

- An extra database round trip
- Additional latency
- A possible race condition

The row can be deleted or changed by another transaction after the `SELECT` and before the `UPDATE`.

For a basic CRUD operation, execute the `UPDATE` directly and inspect its update count.

Perform an additional query only when the application genuinely needs more information, such as distinguishing between:

- Row not found
- Row found but values unchanged
- Optimistic-locking conflict

10.5 Delete a student

```
public int deleteById(Long id) {

    String sql = """
            DELETE FROM students
            WHERE id = ?
            """;

    try (
```

Coder Army

```
        Connection connection = DriverManager.getConnection(
                URL,
                USERNAME,
                PASSWORD
        );

        PreparedStatement preparedStatement =
                connection.prepareStatement(sql)
    ) {
        preparedStatement.setLong(1, id);

        return preparedStatement.executeUpdate();

    } catch (SQLException exception) {
        throw new RuntimeException(
                "Failed to delete student with ID " + id,
                exception
        );
    }
}
```

Usage:

```
int rowsAffected = repository.deleteById(1L);

if (rowsAffected == 1) {
    System.out.println("Student deleted successfully");
} else {
    System.out.println("Student not found");
}
```

For a delete operation:

```
1 → one row deleted
0 → no matching row found
```

Coder Army

Complete `mapRow()` method

```
private Student mapRow(ResultSet resultSet) throws SQLExcepti
on {

    return new Student(
            resultSet.getLong("id"),
            resultSet.getString("name"),
            resultSet.getString("email"),
            resultSet.getObject("age", Integer.class)
    );
}
```

11. Closing JDBC Resources

Resource dependency

JDBC resources have a parent-child relationship:

```
Connection
    ↓ creates
PreparedStatement
    ↓ creates
ResultSet
```

A `ResultSet` depends on the statement that produced it. The statement depends on the connection used to execute it.

Resources should therefore be closed in reverse order:

```
ResultSet
    ↓
PreparedStatement
    ↓
Connection
```

Coder Army

Manual closing with `finally`

Before try-with-resources, JDBC code commonly followed this structure:

```
Connection connection = null;
PreparedStatement preparedStatement = null;
ResultSet resultSet = null;

try {
    connection = DriverManager.getConnection(
            URL,
            USERNAME,
            PASSWORD
    );

    preparedStatement = connection.prepareStatement(sql);
    resultSet = preparedStatement.executeQuery();

    // Process data

} catch (SQLException exception) {
    throw new RuntimeException(exception);

} finally {
    try {
        if (resultSet != null) {
            resultSet.close();
        }
    } catch (SQLException exception) {
        exception.printStackTrace();
    }

    try {
        if (preparedStatement != null) {
            preparedStatement.close();
```

Coder Army

```
        }
    } catch (SQLException exception) {
        exception.printStackTrace();
    }

    try {
        if (connection != null) {
            connection.close();
        }
    } catch (SQLException exception) {
        exception.printStackTrace();
    }
}
```

This works, but it is verbose and easy to get wrong.

Try-with-resources

`Connection`, `Statement`, `PreparedStatement` and `ResultSet` implement `AutoCloseable`.

They can be declared inside try-with-resources:

```
try (
    Connection connection = DriverManager.getConnection(
            URL,
            USERNAME,
            PASSWORD
    );

    PreparedStatement preparedStatement =
            connection.prepareStatement(sql);

    ResultSet resultSet =
            preparedStatement.executeQuery()
) {
```

```
    // Use the resources
}
```

Java closes them automatically in reverse declaration order:

```
ResultSet → PreparedStatement → Connection
```

Try-with-resources is the recommended approach for normal JDBC code.

Why closing the connection matters

A database connection holds resources in both the application and the database server.

Failing to close connections can cause:

- Resource leaks
- Too many open database sessions
- Application slowdown
- “Too many connections” errors
- Uncommitted transactions remaining open

In this basic project, `connection.close()` closes the physical connection.

In applications that use a connection pool, `close()` usually returns the connection to the pool instead of shutting down the underlying network connection.

---

12. Common Setup Problems

Connection refused

Typical error:

```
Communications link failure
Connection refused
```

Coder Army

Coder Army

Check:

1. Is MySQL Server running?
2. Is the host correct?
3. Is the port `3306` correct?
4. Is a firewall blocking the connection?
5. Is another MySQL installation using a different port?

Access denied

Typical error:

```
Access denied for user 'root'@'localhost'
```

Check:

- Username
- Password
- MySQL user permissions
- Host from which the user is allowed to connect

Unknown database

Typical error:

```
Unknown database 'student_db'
```

Create the database first:

```
CREATE DATABASE student_db;
```

MySQL command not found on macOS

Coder Army

For the official package installation:

```
/usr/local/mysql/bin/mysql -u root -p
```

Or add it to the shell path:

```
echo 'export PATH="/usr/local/mysql/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

MySQL service name not found on Windows

List services containing “mysql”:

```
sc query type= service state= all | findstr /I mysql
```

Then use the exact returned service name:

```
net start <service-name>
```

DBeaver: “Public Key Retrieval is not allowed”

This error can occur with some local MySQL authentication configurations.

For a local development connection in DBeaver:

1. Edit the connection.
2. Open **Driver Properties**.
3. Set:

```
allowPublicKeyRetrieval = true
```

If the local server is not configured for TLS, some learners also use:

Coder Army

```
useSSL = false
```

Use `useSSL=false` only for a trusted local development environment. Remote and production database connections should use properly configured TLS.

Duplicate email error

The table defines:

```
email VARCHAR(150) NOT NULL UNIQUE
```

Inserting an existing email produces a constraint violation. This is expected database behaviour and should eventually be handled with application-specific exception handling.

13. Key Takeaways

1. A Java application and a database server are separate programs.
2. JDBC is the standard Java API for relational database communication.
3. A JDBC driver implements the database-specific protocol.
4. `Connection` represents an active database session.
5. `Statement` executes plain SQL, while `PreparedStatement` executes parameterized SQL.
6. Use `executeQuery()` for queries that return rows.
7. Use `executeUpdate()` for insert, update and delete operations.
8. The boolean returned by `execute()` identifies the first result type, not success or failure.
9. A `ResultSet` cursor starts before the first row.
10. JDBC rows must be mapped manually into Java objects.
11. Prefer explicit column lists over `SELECT *`.

12. Use `PreparedStatement` for dynamic values.
13. Placeholders protect values, not dynamic SQL identifiers.
14. Close `ResultSet`, statement and connection resources.
15. Prefer try-with-resources over manual cleanup in `finally`.
16. MySQL Server must be running before Java or DBeaver can connect to it.
17. DBeaver is a client; it does not replace the MySQL server.
18. Raw JDBC is repetitive by design, which helps explain the value later provided by connection pools, `DataSource`, Spring JDBC, JPA and Hibernate.

Coder Army

Complete communication flow

```
Java Application
       ↓
JDBC Interfaces
       ↓
MySQL Connector/J
       ↓
Network Connection
       ↓
MySQL Server
       ↓
SQL Execution
       ↓
Rows / Update Count / Error
       ↓
MySQL Connector/J
       ↓
ResultSet or affected-row count
       ↓
Java object mapping
```

Coder Army

Spring JDBC: From Raw JDBC to `JdbcTemplate`

Spring JDBC reduces the repetitive infrastructure code required by raw JDBC while keeping SQL under the developer's control.

With Spring JDBC, we still write:

- SQL queries
- Query parameters
- Row-to-object mapping
- Application-specific result handling

Spring takes care of:

- Acquiring database connections
- Creating and executing statements
- Closing JDBC resources
- Translating SQL exceptions

---

1. The Problem with Raw JDBC

Consider a simple requirement: insert a student into the database.

```
INSERT INTO students(name, email, age)
VALUES (?, ?, ?);
```

Using raw JDBC, the repository method may look like this:

```
public boolean save(Student student) {

    String sql = """
            INSERT INTO students(name, email, age)
```

Coder Army

```
            VALUES (?, ?, ?)
            """;

    try (
        Connection connection =
                DriverManager.getConnection(URL, USERNAME, PA
SSWORD);

        PreparedStatement preparedStatement =
                connection.prepareStatement(sql)
    ) {
        preparedStatement.setString(1, student.getName());
        preparedStatement.setString(2, student.getEmail());
        preparedStatement.setInt(3, student.getAge());

        int rowsAffected = preparedStatement.executeUpdate();

        return rowsAffected == 1;
    }
    catch (SQLException exception) {
        exception.printStackTrace();
        return false;
    }
}
```

The actual database-specific work is limited to:

```
String sql = "...";

preparedStatement.setString(1, student.getName());
preparedStatement.setString(2, student.getEmail());
preparedStatement.setInt(3, student.getAge());

preparedStatement.executeUpdate();
```

The remaining code manages JDBC infrastructure:

Coder Army

- Opening a connection
- Creating a `PreparedStatement`
- Closing the statement
- Closing the connection
- Catching `SQLException`
- Repeating the same workflow in every repository method

The same structure appears in insert, update, delete and select operations.

> Spring JDBC removes this repeated workflow without removing our control over SQL.

---

2. The Template Idea Behind Spring JDBC

Most JDBC operations follow the same general algorithm:

```
1. Obtain a database connection
2. Create a statement
3. Bind parameter values
4. Execute the statement
5. Process the result
6. Handle SQL exceptions
7. Close JDBC resources
```

Some parts remain almost identical for every operation:

```
Obtain connection
Create statement
Execute statement
Handle exceptions
Close resources
```

Other parts change for each query:

```
SQL statement
Parameter values
```

Coder Army

```
Result mapping
```

Spring separates these responsibilities.

Spring handles	Developer handles
Connection management	SQL query
Statement execution workflow	SQL parameter values
Resource cleanup	Row-to-object mapping
Exception translation	Business interpretation of the result

This follows the **Template Method idea**: Spring provides the fixed JDBC workflow, while the developer supplies the operation-specific parts.

The central class used for this purpose is:

```
JdbcTemplate
```

Spring JDBC still uses standard JDBC objects internally:

```
Connection
PreparedStatement
ResultSet
SQLException
```

The difference is that Spring manages their lifecycle for us.

---

3. Understanding `DataSource`

Before using `JdbcTemplate`, we need to understand where database connections come from.

That responsibility belongs to `DataSource`.

3.1 What Is a `DataSource`?

`DataSource` is an interface provided by Java:

Coder Army

```
javax.sql.DataSource
```

Its main responsibility is to provide database connections.

Conceptually, its important method is:

```
Connection getConnection();
```

A `DataSource` is therefore a **connection provider**.

```
DataSource
    ↓
Provides Connection objects
```

A `DataSource` is not itself a database connection.

```
Connection connection = dataSource.getConnection();
```

3.2 `DriverManager` vs `DataSource`

With raw JDBC, a connection is commonly created using `DriverManager`:

```
Connection connection =
        DriverManager.getConnection(
                URL,
                USERNAME,
                PASSWORD
        );
```

This makes the repository responsible for knowing:

- The database URL
- The username
- The password

CoderArmy

- How connections are created

A repository should mainly be concerned with database operations related to its domain, such as student queries.

With `DataSource`, a connection provider can be injected:

```
public class StudentRepository {

    private final DataSource dataSource;

    public StudentRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }
}
```

The repository can then request a connection without knowing how it was created:

```
Connection connection = dataSource.getConnection();
```

`DriverManager`	`DataSource`
Static connection-acquisition mechanism	Injectable connection-provider abstraction
Repository knows connection details	Connection configuration remains outside the repository
Common in small raw JDBC programs	Preferred in managed applications
Does not itself represent pooling	Can be implemented by a connection pool

4. Connection Pooling

A connection pool maintains a controlled collection of database connections that can be reused by the application.

> Instead of creating a new physical connection for every query, the application borrows an existing connection and returns it after use.

Coder Army

4.1 Connection Creation Without a Pool

Creating a database connection may involve several steps:

```
Java application
      ↓
Network connection established
      ↓
Database server contacted
      ↓
Credentials verified
      ↓
Database session created
      ↓
Connection returned to the application
```

Without pooling, every operation may follow this lifecycle:

```
Create connection
      ↓
Execute SQL
      ↓
Physically close connection
```

For a small console application, this may be acceptable.

For a web application handling many concurrent requests, repeatedly creating physical connections becomes expensive.

```
Request 1 → Create connection → Query → Close
Request 2 → Create connection → Query → Close
Request 3 → Create connection → Query → Close
```

4.2 How a Connection Pool Works

A pool creates and manages a limited number of database connections.

```
Connection Pool
├── Connection 1
├── Connection 2
```

Coder Army

```
├── Connection 3
├── Connection 4
└── Connection 5
```

When repository code requests a connection:

```
Connection connection = dataSource.getConnection();
```

the pool returns an available connection.

When the code calls:

```
connection.close();
```

the connection is generally not physically destroyed. The pool intercepts the call and returns the connection for reuse.

```
Borrow connection
      ↓
Execute database operation
      ↓
Call close()
      ↓
Connection returned to pool
```

4.3 What Happens When Every Connection Is Busy?

Assume the maximum pool size is three:

```
Connection A → In use
Connection B → In use
Connection C → In use
```

A fourth request asks for a connection:

```
dataSource.getConnection();
```

Because no connection is currently available, the request waits.

Coder Army

```
Request 4
    ↓
Wait for an available connection
```

When another request finishes, its connection returns to the pool and becomes available.

If no connection becomes available within the configured timeout, the pool throws an exception.

This protects the database from an unlimited number of simultaneous connections.

4.4 Why Connections Must Be Limited

Each database connection consumes resources such as:

- Memory
- Socket resources
- Session state
- Transaction-related resources
- Database processing capacity

Allowing every incoming request to create its own connection can overwhelm the database.

A connection pool therefore improves:

- Performance through connection reuse
- Stability through controlled concurrency
- Resource management on both the application and database sides

---

5. `DataSource` Does Not Always Mean Connection Pooling

`DataSource` is only an interface.

Coder Army

Different implementations can behave differently:

- One implementation may create a new physical connection for every call.
- Another implementation may return an existing connection from a pool.

Therefore:

```
DataSource ≠ Connection Pool
```

Instead:

```
A connection pool can implement DataSource
```

For example:

```
HikariDataSource
```

is a `DataSource` implementation backed by the HikariCP connection pool.

5.1 Why `DataSource` Is a Better Abstraction

Repository code depends on the `DataSource` interface:

```
Connection connection = dataSource.getConnection();
```

The implementation can change without changing repository logic.

```
Development
DataSource
    ↓
Simple connection provider
```

```
Production
DataSource
```

```
    ↓
HikariCP connection pool
```

The repository depends on the abstraction:

```
DataSource
```

rather than a specific implementation:

```
HikariDataSource
```

This is a practical example of dependency inversion.

---

6. HikariCP

HikariCP is a JDBC connection-pool implementation.

Spring Boot commonly uses HikariCP when it is available on the classpath. It is included by default with starters such as:

```
spring-boot-starter-jdbc
spring-boot-starter-data-jpa
```

HikariCP manages concerns such as:

- Maximum pool size
- Minimum idle connections
- Connection timeout
- Idle timeout
- Maximum connection lifetime
- Borrowing and returning connections
- Connection validation

Common configuration properties include:

Coder Army

Coder Army

```
maximumPoolSize
minimumIdle
connectionTimeout
idleTimeout
maxLifetime
```

7. Creating a Spring JDBC Project

7.1 Required Dependencies

Add the Spring JDBC starter:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

Add the MySQL JDBC driver:

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

For a REST application, also include:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

The JDBC starter provides Spring JDBC infrastructure.

The MySQL dependency provides the database-specific JDBC driver.

Coder Army

7.2 Database Configuration

Configure the database in `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_db
spring.datasource.username=root
spring.datasource.password=your_password
```

Spring Boot reads datasource settings from:

```
spring.datasource.*
```

Spring Boot can normally determine the JDBC driver from the connection URL, so explicitly configuring the driver class is usually unnecessary.

When required, it can be set using:

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

7.3 HikariCP Configuration

Pool-specific settings can be placed under:

```
spring.datasource.hikari.*
```

Example:

```
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.connection-timeout=30000
spring.datasource.hikari.idle-timeout=600000
spring.datasource.hikari.max-lifetime=1800000
```

Meaning of the Main Properties

Coder Army

Property	Purpose
`maximum-pool-size`	Maximum number of connections managed by the pool
`minimum-idle`	Minimum number of idle connections the pool attempts to maintain
`connection-timeout`	Maximum waiting time for a connection before failure
`idle-timeout`	Time an idle connection may remain in the pool before removal
`max-lifetime`	Maximum lifetime of a pooled connection

> A larger pool does not automatically improve performance.

Pool size should be selected using:

- Expected concurrency
- Average query duration
- Database capacity
- Number of application instances
- Load-test results

Every connection consumes resources in both the application and the database.

---

8. How Spring Boot Creates the `DataSource` Bean

Spring Boot performs a process conceptually similar to the following:

```
1. Detect Spring JDBC on the classpath
2. Detect the JDBC driver
3. Read spring.datasource.* properties
4. Detect an available connection-pool implementation
5. Prefer HikariCP when available
6. Create and configure a DataSource
7. Register the DataSource as a Spring bean
```

Conceptually, Spring Boot creates something similar to:

Coder Army

```
@Bean
public DataSource dataSource() {

    HikariDataSource dataSource = new HikariDataSource();

    dataSource.setJdbcUrl(
            "jdbc:mysql://localhost:3306/student_db"
    );

    dataSource.setUsername("root");
    dataSource.setPassword("password");

    return dataSource;
}
```

This bean normally does not need to be created manually because Spring Boot auto-configuration creates it.

When an application defines its own `DataSource` bean, Spring Boot generally backs away from its default datasource auto-configuration.

8.1 Verifying the `DataSource` Implementation

The configured implementation can be verified by injecting the bean:

```
@Component
public class DataSourceChecker {

    private final DataSource dataSource;

    public DataSourceChecker(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @PostConstruct
    public void checkDataSource() {
```

```
        System.out.println(dataSource.getClass());
    }
}
```

A typical output is:

```
class com.zaxxer.hikari.HikariDataSource
```

---

9. Introducing `JdbcTemplate`

`JdbcTemplate` is Spring's main abstraction for performing JDBC operations.

```
org.springframework.jdbc.core.JdbcTemplate
```

It provides operations for:

- Insert
- Select
- Update
- Delete
- Batch operations
- Stored procedures
- Generated-key retrieval

The most commonly used methods are:

```
update()
query()
queryForObject()
```

Coder Army

Coder Army

9.1 Relationship Between `JdbcTemplate` and `DataSource`

`JdbcTemplate` does not create database connections independently. It requires a `DataSource`.

```
JdbcTemplate
    ↓ requests a connection from
DataSource
    ↓ provides
Connection
```

Conceptually, the bean can be created as follows:

```java
@Bean
public JdbcTemplate jdbcTemplate(DataSource dataSource) {
    return new JdbcTemplate(dataSource);
}
```

Spring Boot normally configures this bean automatically when Spring JDBC and a valid `DataSource` are available.

The configured template can then be injected:

```java
@Repository
public class StudentRepository {

    private final JdbcTemplate jdbcTemplate;

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }
}
```

There is usually no need to create an unconfigured instance manually:

```java
new JdbcTemplate();
```

Coder Army

9.2 Responsibilities Handled by `JdbcTemplate`

For an update operation:

```
jdbcTemplate.update(sql, parameters);
```

Spring conceptually performs:

```
1. Ask DataSource for a connection
2. Create a PreparedStatement
3. Bind parameter values
4. Execute the statement
5. Read the affected-row count
6. Close the statement
7. Release the connection
8. Translate SQLException when necessary
```

For a select operation:

```
1. Obtain a connection
2. Create a PreparedStatement
3. Bind parameter values
4. Execute the query
5. Obtain the ResultSet
6. Iterate through the rows
7. Call the RowMapper for each row
8. Collect the mapped objects
9. Close the ResultSet
10. Close the statement
11. Release the connection
12. Translate SQL exceptions
```

10. Project Structure

A simple project structure can be:

```
src/main/java
└── in/coderarmy/springjdbc
    ├── SpringJdbcApplication.java
```

Coder Army

```
    ├── controller
    │   └── StudentController.java
    ├── service
    │   └── StudentService.java
    ├── repository
    │   ├── StudentRepository.java
    │   └── StudentRowMapper.java
    └── model
        └── Student.java
```

10.1 Database and Table

```sql
CREATE DATABASE student_db;

USE student_db;

CREATE TABLE students (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    age INT NOT NULL
);
```

10.2 `Student` Model

```java
package in.coderarmy.springjdbc.model;

public class Student {

    private Long id;
    private String name;
    private String email;
    private Integer age;

    // No-argument constructor, parameterized constructor,
```

```
    // getters and setters
}
```

Coder Army

11. Create Operation with `JdbcTemplate`

The `update()` method is used for SQL statements that change database state:

```
INSERT
UPDATE
DELETE
```

11.1 Saving a Student

```
@Repository
public class StudentRepository {

    private final JdbcTemplate jdbcTemplate;

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public boolean save(Student student) {

        String sql = """
                INSERT INTO students(name, email, age)
                VALUES (?, ?, ?)
                """;

        int rowsAffected = jdbcTemplate.update(
                sql,
                student.getName(),
                student.getEmail(),
                student.getAge()
```

```
        );

        return rowsAffected == 1;
    }
}
```

The values are bound according to their position:

```
First ?  → student.getName()
Second ? → student.getEmail()
Third ?  → student.getAge()
```

Parameter order must match placeholder order.

The following call is incorrect:

```
jdbcTemplate.update(
        sql,
        student.getAge(),
        student.getName(),
        student.getEmail()
);
```

The first placeholder represents `name`, not `age`.

Spring uses prepared-statement-based execution internally when parameter values are supplied to the appropriate `update()` overload.

11.2 Affected-Row Count

`update()` returns an `int` representing the number of affected rows.

```
int rowsAffected = jdbcTemplate.update(...);
```

For an insert that should create one row:

Coder Army

Coder Army

```
return rowsAffected == 1;
```

Spring removes the need for:

- Explicit `Connection` handling
- Explicit `PreparedStatement` creation
- Individual parameter-setter calls
- `try-with-resources`
- Manual resource cleanup
- Repeated `SQLException` catch blocks

Spring does not remove responsibility for:

- Writing correct SQL
- Supplying parameters in the correct order
- Interpreting the affected-row count

---

12. Read Operations with `JdbcTemplate`

A write operation usually returns an affected-row count.

A select operation returns rows and columns that must be converted into Java objects.

Spring uses `RowMapper` for this conversion.

12.1 What Is `RowMapper`?

`RowMapper<T>` is an interface whose responsibility is:

```
Convert one ResultSet row into one Java object
```

Its central method is:

Coder Army

```
T mapRow(
        ResultSet resultSet,
        int rowNumber
) throws SQLException;
```

For a student query:

```
One database row
        ↓
StudentRowMapper
        ↓
One Student object
```

If a query returns ten rows, Spring calls the mapper ten times.

```
Row 1  → Student 1
Row 2  → Student 2
Row 3  → Student 3
...
Row 10 → Student 10
```

Spring handles result-set iteration. The mapper handles conversion of one row.

12.2 Creating `StudentRowMapper`

```
package in.coderarmy.springjdbc.repository;

import in.coderarmy.springjdbc.model.Student;
import org.springframework.jdbc.core.RowMapper;

import java.sql.ResultSet;
import java.sql.SQLException;

public class StudentRowMapper implements RowMapper<Student> {

    @Override
```

Coder Army

```
    public Student mapRow(
            ResultSet resultSet,
            int rowNumber
    ) throws SQLException {

        Student student = new Student();

        student.setId(
                resultSet.getLong("id")
        );

        student.setName(
                resultSet.getString("name")
        );

        student.setEmail(
                resultSet.getString("email")
        );

        student.setAge(
                resultSet.getInt("age")
        );

        return student;
    }
}
```

12.3 Fetching All Students with `query()`

Use `query()` when zero, one or multiple rows may be returned.

```
private final RowMapper<Student> studentRowMapper =
        new StudentRowMapper();

public List<Student> findAll() {
```

Coder Army

```
    String sql = """
            SELECT id, name, email, age
            FROM students
            """;

    return jdbcTemplate.query(
            sql,
            studentRowMapper
    );
}
```

The return type is:

```
List<Student>
```

Possible results:

```
No rows:
[]

Three rows:
[Student1, Student2, Student3]
```

`query()` normally returns an empty list when no rows match.

This makes it suitable for collection-based queries.

Internally, the workflow is conceptually similar to:

```
while (resultSet.next()) {
    Student student =
            rowMapper.mapRow(resultSet, rowNumber);

    results.add(student);
}
```

Coder Army

12.4 Reusing a Mapper

A stateless mapper can be reused instead of creating a new instance for every query.

```
@Repository
public class StudentRepository {

    private final JdbcTemplate jdbcTemplate;

    private final RowMapper<Student> studentRowMapper =
            new StudentRowMapper();

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }
}
```

The mapper only reads the current `ResultSet` row and creates a new object, so it does not need mutable shared state.

12.5 Lambda-Based `RowMapper`

Because `RowMapper` is a functional interface, it can also be implemented with a lambda:

```
private final RowMapper<Student> studentRowMapper =
        (resultSet, rowNumber) ->
                new Student(
                        resultSet.getLong("id"),
                        resultSet.getString("name"),
                        resultSet.getString("email"),
                        resultSet.getInt("age")
                );
```

Coder Army

A separate mapper class makes the responsibility easier to understand during teaching.

A lambda is convenient for short and simple mappings.

---

13. Fetching One Student with `queryForObject()`

Use `queryForObject()` when the query is expected to return exactly one row.

```
public Student findById(Long id) {

    String sql = """
            SELECT id, name, email, age
            FROM students
            WHERE id = ?
            """;

    return jdbcTemplate.queryForObject(
            sql,
            studentRowMapper,
            id
    );
}
```

`queryForObject()` communicates the following expectation:

> This query must return exactly one result.

It does not mean that Spring will return `null` when no row exists.

13.1 Behaviour of `queryForObject()`

Exactly One Row

Coder Army

```
Returns the mapped Student object
```

Zero Rows

```
Throws EmptyResultDataAccessException
```

More Than One Row

```
Throws IncorrectResultSizeDataAccessException
```

For a primary-key query such as:

```
WHERE id = ?
```

more than one row should be impossible because the primary key is unique.

The common missing-data case is zero rows.

13.2 Returning `Optional<Student>`

A missing row can be represented using `Optional.empty()`:

```
public Optional<Student> findById(Long id) {

    String sql = """
            SELECT id, name, email, age
            FROM students
            WHERE id = ?
            """;

    try {
        Student student =
                jdbcTemplate.queryForObject(
                        sql,
```

Coder Army

```
                        studentRowMapper,
                        id
                );

            return Optional.of(student);
        }
        catch (EmptyResultDataAccessException exception) {
            return Optional.empty();
        }
    }
```

Only `EmptyResultDataAccessException` is converted into an empty optional.

Do not catch every `DataAccessException` and return `Optional.empty()`.

These situations are different:

```
Student does not exist
```

```
Database server is unavailable
```

Only the first case represents an absent student.

Database failures should continue to propagate as errors.

13.3 Alternative Using `query()`

The same lookup can be implemented without exception-based no-result handling:

```
public Optional<Student> findById(Long id) {

    String sql = """
            SELECT id, name, email, age
            FROM students
            WHERE id = ?
```

Coder Army

```
                """;

        List<Student> students =
                jdbcTemplate.query(
                        sql,
                        studentRowMapper,
                        id
                );

        return students.stream().findFirst();
    }
```

`query()` returns an empty list when no student exists, which is converted into `Optional.empty()`.

The trade-off is semantic clarity:

```
queryForObject()
```

clearly communicates that exactly one row is expected.

For primary-key lookups, either approach can be used when its behaviour is understood.

---

14. Update Operation

```
public boolean update(
        Long id,
        Student student
) {

    String sql = """
            UPDATE students
            SET name = ?,
                email = ?,
                age = ?
```

Coder Army

```
            WHERE id = ?
            """;

    int rowsAffected = jdbcTemplate.update(
            sql,
            student.getName(),
            student.getEmail(),
            student.getAge(),
            id
    );

    return rowsAffected == 1;
}
```

Parameter order must match placeholder order:

```
First ?  → name
Second ? → email
Third ?  → age
Fourth ? → id
```

Possible affected-row counts:

```
1 → One matching student was updated
0 → No matching student was found
```

For a primary-key condition, more than one affected row should not be possible.

---

15. Delete Operation

```
public boolean deleteById(Long id) {

    String sql = """
            DELETE FROM students
            WHERE id = ?
```

```
            """;

    int rowsAffected = jdbcTemplate.update(
            sql,
            id
    );

    return rowsAffected == 1;
}
```

`jdbcTemplate.update()` is used for insert, update and delete operations because each changes database state and returns an affected-row count.

---

16. `BeanPropertyRowMapper`

A custom mapper provides explicit control:

```
private final RowMapper<Student> studentRowMapper =
        (resultSet, rowNumber) ->
                new Student(
                        resultSet.getLong("id"),
                        resultSet.getString("name"),
                        resultSet.getString("email"),
                        resultSet.getInt("age")
                );
```

Spring also provides:

```
BeanPropertyRowMapper
```

It can be configured as follows:

```
private final RowMapper<Student> studentRowMapper =
        new BeanPropertyRowMapper<>(Student.class);
```

Coder Army

Coder Army

The query remains unchanged:

```
public List<Student> findAll() {

    String sql = """
            SELECT id, name, email, age
            FROM students
            """;

    return jdbcTemplate.query(
            sql,
            studentRowMapper
    );
}
```

`BeanPropertyRowMapper` attempts to match database columns with JavaBean properties.

```
Database column → Java property

id              → id
name            → name
email           → email
age             → age
```

It can also map underscore-separated column names to camel-case properties:

```
first_name     → firstName
date_of_birth  → dateOfBirth
```

The target class generally needs:

- A no-argument constructor
- Public setter methods

Advantages

Coder Army

- Reduces mapper code
- Convenient for simple tables
- Useful for prototypes and straightforward mappings

Limitations

- Depends on naming conventions
- Provides less control over conversions
- Mapping problems may be less obvious
- Not ideal for complex joins
- Uses reflection and metadata-based mapping

A custom `RowMapper` is usually better when mapping logic is complex or explicit control is important.

---

17. Spring JDBC Exception Translation

17.1 Problems with `SQLException`

Raw JDBC operations throw:

```
SQLException
```

`SQLException` is a checked exception.

The compiler therefore requires either:

```
try {
    // JDBC operation
}
catch (SQLException exception) {
    // handle exception
}
```

Coder Army

or:

```
throws SQLException
```

`SQLException` is also very broad. The same exception type may represent:

- Invalid SQL
- Duplicate key
- Constraint violation
- Connection failure
- Authentication failure
- Deadlock
- Timeout
- Invalid column name
- SQL syntax error

Developers may need to inspect:

```
exception.getSQLState();
exception.getErrorCode();
exception.getMessage();
```

Vendor-specific error codes may differ between MySQL, PostgreSQL, Oracle and other databases.

17.2 `DataAccessException`

Spring catches `SQLException` and translates it into Spring's data-access exception hierarchy.

The root exception is:

```
DataAccessException
```

Coder Army

The flow is:

```
Database error
    ↓
JDBC driver throws SQLException
    ↓
JdbcTemplate catches SQLException
    ↓
Spring examines the exception
    ↓
A suitable DataAccessException subtype is thrown
```

This exception hierarchy is not tied to one particular database vendor.

17.3 Checked vs Unchecked Exceptions

`SQLException` is checked:

```
public class SQLException extends Exception
```

`DataAccessException` is unchecked:

```
public abstract class DataAccessException
        extends NestedRuntimeException
```

Repository methods therefore do not need to catch or declare every database exception.

```
public boolean save(Student student) {
    return jdbcTemplate.update(...) == 1;
}
```

When the operation fails, the relevant `DataAccessException` subtype propagates.

Many database failures cannot be meaningfully resolved inside the repository itself, for example:

- Database server is unavailable

Coder Army

- SQL is invalid
- Credentials are incorrect
- Required table does not exist

Such failures can propagate to centralized exception-handling code.

---

18. Common Spring Database Exceptions

18.1 `DuplicateKeyException`

Thrown when an insert or update violates a unique or primary-key constraint.

Suppose `email` is unique:

```
email VARCHAR(150) UNIQUE
```

Inserting the same email again may produce:

```
DuplicateKeyException
```

Application-specific handling may convert it into a domain exception:

```
try {
    studentRepository.save(student);
}
catch (DuplicateKeyException exception) {
    throw new EmailAlreadyExistsException(
            student.getEmail()
    );
}
```

18.2 `DataIntegrityViolationException`

Represents a broader data-integrity or constraint violation.

Coder Army

Examples include:

- Foreign-key violation
- Not-null violation
- Check-constraint violation
- Certain unique-constraint violations

`DuplicateKeyException` represents a more specific category of integrity failure.

18.3 `BadSqlGrammarException`

May be thrown when SQL is invalid.

Examples:

```
SELEKT id FROM students;
```

```
SELECT unknown_column FROM students;
```

18.4 `CannotGetJdbcConnectionException`

Represents a failure to obtain a database connection.

Possible causes include:

- Database server is down
- Incorrect hostname
- Incorrect port
- Network failure
- Invalid credentials
- Pool cannot provide a connection within the timeout

18.5 `EmptyResultDataAccessException`

Coder Army

Thrown when an operation expects at least one result but receives none.

A common example is:

```
queryForObject()
```

when no row matches the query.

18.6 `IncorrectResultSizeDataAccessException`

Thrown when the number of returned rows does not match the expected result size.

Example:

```
Expected exactly one row
Received three rows
```

---

19. Why Exception Translation Improves Portability

Suppose an application moves from MySQL to PostgreSQL.

With raw JDBC, exception-handling logic may depend on MySQL-specific error codes.

With Spring JDBC, application code can handle a common abstraction such as:

```
DuplicateKeyException
```

Spring translates the database-specific `SQLException` into a consistent exception hierarchy.

This reduces the amount of vendor-specific exception-handling code in the application.

Coder Army

20. Repository Abstraction

The controller and service layers do not need to know whether the repository uses:

- Raw JDBC
- Spring JDBC
- JPA
- Hibernate

That implementation detail remains inside the repository layer.

```
Controller
    ↓
Service
    ↓
Repository interface or repository class
    ↓
Spring JDBC / JPA / another persistence mechanism
```

This separation makes the application easier to maintain and allows persistence details to change with limited impact on higher layers.

---

21. Raw JDBC vs Spring JDBC

Area	Raw JDBC	Spring JDBC
SQL control	Full control	Full control
Connection handling	Manual	Managed through `DataSource`
Statement creation	Manual	Managed by `JdbcTemplate`
Parameter binding	Manual setter calls	Positional method arguments
Result-set iteration	Manual	Managed by `JdbcTemplate`
Row mapping	Manual	Provided through `RowMapper`
Resource cleanup	Manual	Automatic

Area	Raw JDBC	Spring JDBC
Exception model	Checked `SQLException`	Unchecked `DataAccessException` hierarchy
Connection pooling	Must be configured separately	Commonly provided through Boot and HikariCP
Boilerplate	High	Significantly reduced

Coder Army

22. Complete Execution Flow

For an insert operation:

```
StudentRepository.save(student)
        ↓
JdbcTemplate.update(sql, parameters)
        ↓
JdbcTemplate asks DataSource for a connection
        ↓
HikariCP provides a pooled connection
        ↓
JdbcTemplate creates PreparedStatement
        ↓
JdbcTemplate binds parameter values
        ↓
PreparedStatement executes SQL
        ↓
Database returns affected-row count
        ↓
JdbcTemplate closes the statement
        ↓
Connection is returned to the pool
        ↓
Repository receives the affected-row count
```

For a select operation:

```
StudentRepository.findAll()
        ↓
JdbcTemplate.query(sql, rowMapper)
        ↓
Connection obtained from DataSource
        ↓
PreparedStatement created and executed
```

Coder Army

```
        ↓
Database returns ResultSet
        ↓
JdbcTemplate iterates through each row
        ↓
RowMapper converts each row into Student
        ↓
Mapped students are collected into a List
        ↓
ResultSet and statement are closed
        ↓
Connection is returned to the pool
        ↓
List<Student> is returned
```

---

23. Key Takeaways

1. Spring JDBC still uses JDBC internally.
2. It does not remove SQL from the application.
3. `DataSource` is an abstraction that provides database connections.
4. A `DataSource` implementation may or may not use connection pooling.
5. HikariCP is the connection pool commonly used by Spring Boot.
6. `JdbcTemplate` manages the repetitive JDBC workflow.
7. `update()` is used for insert, update and delete operations.
8. `query()` is suitable when multiple rows may be returned.
9. `queryForObject()` expects exactly one result.
10. `RowMapper` converts one database row into one Java object.
11. `BeanPropertyRowMapper` reduces mapping code but provides less explicit control.
12. Spring translates `SQLException` into the `DataAccessException` hierarchy.
13. Database-specific failures should not be confused with a valid “record not found” result.
14. Spring JDBC reduces boilerplate while preserving direct control over SQL.

Coder Army

Introduction to Hibernate

Java objects and relational tables

Consider this Java class:

```
public class Student {

    private Long id;
    private String name;
    private String email;
    private Integer age;
}
```

The equivalent relational representation might be:

```
CREATE TABLE students (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    age INT
);
```

The object and row contain similar information, but they are fundamentally different structures.

Java understands

- Classes
- Objects
- References
- Inheritance
- Encapsulation

Coder Army

- Collections
- Object identity
- Method behaviour

A relational database understands

- Tables
- Rows
- Columns
- Primary keys
- Foreign keys
- Constraints
- Joins
- SQL data types

Therefore, Java cannot directly save an object into MySQL.

The object must first be translated into SQL-compatible values.

Persistence using Spring JDBC

With Spring JDBC, an insertion may look like this:

```
public int save(Student student) {

    String sql = """
            INSERT INTO students(name, email, age)
            VALUES (?, ?, ?)
            """;

    return jdbcTemplate.update(
            sql,
            student.getName(),
            student.getEmail(),
```

```
                student.getAge()
        );
    }
```

This is much cleaner than raw JDBC, but the developer still performs the translation manually.

Spring JDBC manages much of the JDBC infrastructure, but it does not understand the semantic relationship between the `Student` class and the `students` table.

Manual SQL writing

For every persistence operation, SQL must be written manually.

Insert

```
INSERT INTO students(name, email, age)
VALUES (?, ?, ?);
```

Select

```
SELECT id, name, email, age
FROM students
WHERE id = ?;
```

Update

```
UPDATE students
SET name = ?, email = ?, age = ?
WHERE id = ?;
```

Delete

Coder Army

Coder Army

```
DELETE FROM students
WHERE id = ?;
```

This is not necessarily bad. SQL gives precise control.

The problem appears when an application has:

- Hundreds of entities
- Many relationships
- Repeated CRUD operations

The amount of repetitive translation code grows rapidly.

Manual parameter binding

Spring JDBC reduces direct `PreparedStatement` handling, but values must still be passed in the correct order.

```
jdbcTemplate.update(
        sql,
        student.getName(),   // first ?
        student.getEmail(),  // second ?
        student.getAge(),    // third ?
        student.getId()      // fourth ?
);
```

The programmer must ensure that:

```
First ?  = name
Second ? = email
Third ?  = age
Fourth ? = id
```

A wrong order may not cause a compilation error.

Manual row-to-object mapping

Coder Army

When reading data, the opposite translation is needed.

```
private final RowMapper<Student> studentRowMapper =
        (resultSet, rowNumber) -> {

            Student student = new Student();

            student.setId(resultSet.getLong("id"));
            student.setName(resultSet.getString("name"));
            student.setEmail(resultSet.getString("email"));
            student.setAge(resultSet.getInt("age"));

            return student;
        };
```

Object-relational impedance mismatch

The differences between object models and relational models are collectively called **object-relational impedance mismatch**.

Introduction to Object-Relational Mapping

Object-Relational Mapping, or ORM, is the process of defining how:

```
Java classes    map to database tables
Java objects    map to database rows
Java fields     map to database columns
Java references map to foreign-key relationships
Java collections map to related rows
```

For Example :

```
public class Student {
```

Coder Army

```
    private Long id;

    private String name;
}
```

This metadata tells the ORM framework:

```
Student class → students table
id field      → id column
name field    → student_name column
```

The mapping is declared once and reused during `inserts`, `selects`, `updates` and `deletes`.

Responsibilities of an ORM framework

An ORM framework normally handles several responsibilities.

Mapping metadata

It understands how classes and attributes correspond to tables and columns.

SQL generation

It can generate SQL based on entity operations.

Parameter binding

It extracts values from the entity and binds them to JDBC parameters.

Result-set mapping

It converts database rows back into entity objects.

Relationship management

It converts object references and collections into foreign-key and join-table operations.

Coder Army

Caching

It may avoid unnecessary database access through first-level and optional second-level caches.

Transaction integration

It coordinates entity operations with database transactions.

Introduction to Hibernate

Hibernate is an ORM framework for Java and JVM applications.

Hibernate operates above JDBC.

```
Application
    ↓
Hibernate
    ↓
JDBC
    ↓
JDBC Driver
    ↓
Database
```

JPA as a specification

JPA originally meant **Java Persistence API**.

The current standard is named **Jakarta Persistence**, though developers still commonly use the term JPA.

It defines contracts such as:

```
jakarta.persistence.Entity
jakarta.persistence.Id
jakarta.persistence.EntityManager
```

Coder Army

```
jakarta.persistence.EntityManagerFactory
jakarta.persistence.PersistenceContext
```

But a specification does not perform the work by itself.

An implementation is required.

Hibernate as a JPA provider

Hibernate implements the Jakarta Persistence specification.

Therefore, when application code uses:

```
import jakarta.persistence.EntityManager;
```

the interface is defined by Jakarta Persistence, but the runtime object may be implemented by Hibernate.

This resembles other interface-and-implementation relationships:

```
List interface
    ↓
ArrayList implementation
```

Jakarta Persistence defines the API. Hibernate supplies the implementation.

Other JPA providers have historically included EclipseLink and OpenJPA, but Spring Boot commonly uses Hibernate when the JPA starter is present. Spring Boot's JPA starter includes Hibernate, Spring Data JPA and Spring ORM.

Setting Up Hibernate with Spring Boot

1. Required dependencies

Create a Spring Boot project with:

Coder Army

- **Spring Data JPA** (We are including `spring-boot-starter-data-jpa` for configuration and dependencies, but we are not initially using `JpaRepository`. Spring Data repositories would otherwise hide many of these operations.)
- **MySQL Driver**

Creating and using Student database

Hibernate can manage tables, but for a basic MySQL setup, create the database first:

```
CREATE DATABASE student_db;

USE student_db;
```

DataSource configuration

In `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/hibernate_demo
spring.datasource.username=root
spring.datasource.password=your_password
```

Spring Boot uses these properties to configure a `DataSource`.

The `DataSource` supplies JDBC connections to Hibernate.

Schema generation with `ddl-auto`

For learning:

```
spring.jpa.hibernate.ddl-auto=update
```

Common values are:

`none`

Coder Army

Hibernate performs no schema action.

```
spring.jpa.hibernate.ddl-auto=none
```

`validate`

Hibernate checks whether the existing schema matches the entity mappings.

```
spring.jpa.hibernate.ddl-auto=validate
```

It does not create or modify tables.

`update`

Hibernate attempts to update the schema to match the mappings.

```
spring.jpa.hibernate.ddl-auto=update
```

Convenient for demonstrations and early development, but it should not be treated as a production migration system.

`create`

Hibernate drops and recreates the mapped schema when the application starts.

```
spring.jpa.hibernate.ddl-auto=create
```

Existing table data may be lost.

`create-drop`

Hibernate creates the schema at startup and drops it when the persistence factory closes.

```
spring.jpa.hibernate.ddl-auto=create-drop
```

Spring Boot exposes `spring.jpa.hibernate.ddl-auto`, and current documentation lists `none`, `validate`, `update`, `create` and `create-drop` as its supported modes.

Coder Army

In production systems, schema changes are usually managed using tools such as Flyway or Liquibase rather than depending on `update`.

SQL logging

For learning, enable generated SQL:

```
spring.jpa.show-sql=true
```

Format the SQL:

```
spring.jpa.properties.hibernate.format_sql=true
```

Spring Boot Project Structure

Use this structure:

```
hibernate-demo
├── pom.xml
└── src
    └── main
        ├── java
        │   └── com.coderarmy.hibernate
        │       ├── HibernateDemoApplication.java
        │       ├── entity
        │       │   ├── Student.java
        │       │   └── Address.java
        │       ├── repository
        │       │   └── StudentPersistenceRepository.java
        │       ├── service
        │       │   └── StudentService.java
        │       └── controller
        │           └── StudentController.java
        └── resources
            └── application.properties
```

Coder Army

Initial entity

```java
package com.coderarmy.hibernate.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    private Integer age;

    public Student() {
    }

    public Student(
            String name,
            String email,
            Integer age
    ) {
        this.name = name;
        this.email = email;
        this.age = age;
    }

   // getter & setter
}
```

Coder Army

Repository

```
@Repository
public class StudentPersistenceRepository {

    @PersistenceContext
    private EntityManager entityManager;

    public void persist(Student student) {
        entityManager.persist(student);
    }

    public Student findById(Long id) {
        return entityManager.find(Student.class, id);
    }

    public void remove(Student student) {
        entityManager.remove(student);
    }
}
```

> The standard JPA interface through which application code asks the persistence provider to manage and retrieve entities.

5.2 Create with `persist()`

```
@Transactional
public Long createStudent(
        String name,
        String email,
        Integer age
) {
    Student student =
            new Student(name, email, age);
```

```
    repository.persist(student);

    return student.getId();
}
```

Coder Army

5.3 Read with `find()`

```
@Transactional(readOnly = true)
public Student getStudent(Long id) {
    return repository.findById(id);
}
```

5.4 Update through a managed entity

```
@Transactional
public void updateStudent(Long id, String newName) {

    Student student = repository.findById(id);

    if (student == null) {
        throw new IllegalArgumentException(
                "Student not found: " + id
        );
    }

    student.setName(newName);
}
```

Pause on what is missing:

```
entityManager.update(student);
repository.save(student);
```

Neither is required for a managed entity.

Coder Army

Do not fully explain dirty checking here. Create curiosity:

> Hibernate somehow knows that the loaded object changed and generates an update before the transaction commits. The persistence-context section will show exactly how.

5.5 Delete with `remove()`

```
@Transactional
public void deleteStudent(Long id) {

    Student student = repository.findById(id);

    if (student == null) {
        return;
    }

    repository.remove(student);
}
```

Explain:

- `remove()` operates on a managed entity.
- Hibernate marks that entity for deletion.
- The delete is synchronized with the database during flushing.

Initial entity

```
package com.coderarmy.hibernate.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
```

Coder Army

```java
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(
            name = "student_name",
            nullable = false,
            length = 100
    )
    private String name;

    @Column(
            nullable = false,
            unique = true,
            length = 150
    )
    private String email;

    private Integer age;

    protected Student() {
    }

    public Student(
            String name,
            String email,
            Integer age
    ) {
        this.name = name;
        this.email = email;
        this.age = age;
    }
```

```
    // getter & setter
}
```

Coder Army

Important options include:

```
@Column(
        name = "student_name",
        nullable = false,
        unique = false,
        length = 100,
        insertable = true,
        updatable = true
)
private String name;

//name
Controls the mapped column name.

//nullable
Used in schema generation to indicate whether the column may
be null.

//unique
Declares a single-column uniqueness constraint during schema
generation.

//length
Provides the expected string-column length for generated sche
ma.

//insertable
Controls whether the column appears in generated insert state
ments.

//updatable
```

```
Controls whether the column appears in generated update statements.
```

Use `BigDecimal` when decimal precision matters.

For example:

```
@Column(precision = 5, scale = 2)
private BigDecimal percentage;
```

Conceptually:

```
precision = total number of digits
scale     = digits after decimal point
```

For:

```
98.75
```

The precision is 4 and the scale is 2.

Date and time mapping

Modern Java date/time classes should normally be preferred.

```
private LocalDate dateOfBirth;
```

Maps conceptually to:

```
DATE

2000-08-21
```

```
private LocalTime classTime;
```

Maps conceptually to:

Coder Army

Coder Army

```
TIME
```

```
private LocalDateTime createdAt;
```

Maps conceptually to:

```
TIMESTAMP


2026-07-25T10:30:00
```

Enum mapping

Consider:

```
public enum StudentStatus {
    ACTIVE,
    INACTIVE,
    SUSPENDED
}
```

Entity field:

```
@Enumerated(EnumType.STRING)
private StudentStatus status;
```

Database values:

```
ACTIVE
INACTIVE
SUSPENDED
```

Large objects

Coder Army

Large text

```
@Lob
private String profileDescription;
```

Binary data

```
@Lob
private byte[] profileImage;
```

`@Lob` instructs the provider to use a database-native large-object mapping.

For real production systems, storing large files directly inside the database should be an intentional architectural decision. Often, the database stores file metadata and an object-storage URL instead.

`@Transient`

Sometimes an entity needs a field that should not be persisted.

```
@Transient
private String displayName;
```

Hibernate ignores it.

Attribute converters

Suppose the database stores:

```
Y
N
```

But Java should use:

```
Boolean
```

Coder Army

Create a converter:

```
package com.coderarmy.hibernate.converter;

import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;

@Converter
public class BooleanToYesNoConverter
        implements AttributeConverter<Boolean, String> {

    @Override
    public String convertToDatabaseColumn(
            Boolean attribute
    ) {
        if (attribute == null) {
            return null;
        }

        return attribute ? "Y" : "N";
    }

    @Override
    public Boolean convertToEntityAttribute(
            String databaseValue
    ) {
        if (databaseValue == null) {
            return null;
        }

        return "Y".equals(databaseValue);
    }
}
```

Use it:

```
@Convert(converter = BooleanToYesNoConverter.class)
private Boolean active;
```

Coder Army

Embeddable Objects and Value Types

1. Entity types versus value types

Consider a student and an address.

Student

A student has independent identity.

The student continues to exist even if the address changes.

Address

An address often has no independent identity in the domain.

```
House number
Street
City
State
Postal code
```

It exists as part of another object.

```
Student owns Address
```

That makes `Address` a good candidate for a value type.

Hibernate classifies value types as values whose lifecycle is owned by an entity, whereas entities have independent identity.

2. `@Embeddable`

Define the value type:

Coder Army

```java
package com.coderarmy.hibernate.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Embeddable;

@Embeddable
public class Address {

    @Column(name = "address_line")
    private String addressLine;

    private String city;

    private String state;

    @Column(name = "postal_code")
    private String postalCode;

    protected Address() {
    }

    public Address(
            String addressLine,
            String city,
            String state,
            String postalCode
    ) {
        this.addressLine = addressLine;
        this.city = city;
        this.state = state;
        this.postalCode = postalCode;
    }
}
```

`@Embeddable` means:

Coder Army

> This class does not represent an independent entity. Its state may be embedded inside an owning entity.

An embeddable does not have its own `@Id`. Its state is stored as an intrinsic part of its owner.

`@Embedded`

Use the value inside `Student`:

```
@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Embedded
    private Address address;
}
```

The table may become:

```
students
├── id
├── name
├── address_line
├── city
├── state
└── postal_code
```

There is no separate address table by default.

The object structure:

Coder Army

```
Student
└── Address
```

Is flattened into:

```
students row
├── student fields
└── address fields
```

`@Embedded` declares an entity attribute whose value is an embeddable object.

Why use an embeddable instead of separate fields?

Without an embeddable:

```java
private String addressLine;
private String city;
private String state;
private String postalCode;
```

This works, but the entity begins to contain unrelated groups of primitive fields.

With an embeddable The Java model becomes more expressive.

Multiple embedded objects

Suppose a student has:

```java
@Embedded
private Address currentAddress;

@Embedded
private Address permanentAddress;
```

Both embeddables contain fields named:

Coder Army

```
address_line
city
state
postal_code
```

This would create conflicting column mappings.

We must override them.

@AttributeOverride

```
@Embedded
@AttributeOverrides({
        @AttributeOverride(
                name = "addressLine",
                column = @Column(
                        name = "current_address_line"
                )
        ),
        @AttributeOverride(
                name = "city",
                column = @Column(
                        name = "current_city"
                )
        ),
        @AttributeOverride(
                name = "state",
                column = @Column(
                        name = "current_state"
                )
        ),
        @AttributeOverride(
                name = "postalCode",
                column = @Column(
                        name = "current_postal_code"
```

```
                )
        )
})
private Address currentAddress;
```

same for permanent address.

The resulting table contains:

```
students
├── id
├── current_address_line
├── current_city
├── current_state
├── current_postal_code
├── permanent_address_line
├── permanent_city
├── permanent_state
└── permanent_postal_code
```

`@AttributeOverride` replaces a basic mapping originally declared by the embeddable for a particular embedding location

`@ElementCollection`

An entity may own a collection of value types.

For example:

```
private Set<String> skills;
```

A collection cannot normally fit into one relational column, so it is stored in a separate table.

```
@ElementCollection
@CollectionTable(
        name = "student_skills",
```

Coder Army

```
        joinColumns = @JoinColumn(name = "student_id")
)
@Column(name = "skill")
private Set<String> skills =
        new HashSet<>();
```

Possible database structure:

```
students
├── id
└── name
```

```
student_skills
├── student_id
└── skill
```

The values in `student_skills` are not independent entities.

There is no:

```
Skill skill = entityManager.find(Skill.class, id);
```

Each string belongs to the owning `Student`.

Element collection of embeddables

You may also store a collection of value objects.

```
@Embeddable
public class PhoneNumber {

    private String countryCode;
    private String number;

    protected PhoneNumber() {
    }
```

Coder Army

Coder Army

```
    public PhoneNumber(
            String countryCode,
            String number
    ) {
        this.countryCode = countryCode;
        this.number = number;
    }
}
```

Inside `Student`:

```
@ElementCollection
@CollectionTable(
        name = "student_phone_numbers",
        joinColumns = @JoinColumn(name = "student_id")
)
private Set<PhoneNumber> phoneNumbers =
        new HashSet<>();
```

Table:

```
student_phone_numbers
├── student_id
├── country_code
└── number
```

`@ElementCollection` maps collections of basic or embeddable values, while `@CollectionTable` controls the separate table used to store them.

Entity association versus embedded value

This is an important modelling decision.

Use an entity when the object has independent identity

Example:

Coder Army

```
Student
Department
Course
Teacher
```

A department exists independently of one particular student.

It may be:

- Shared by many students
- Queried independently
- Updated independently
- Referenced by an identifier
- Managed through its own lifecycle

Hibernate Internals

Hibernate becomes much easier to understand once we stop seeing it as a tool that only generates SQL. Internally, Hibernate creates and manages a working area in memory, tracks entity objects inside that area, detects changes, and synchronizes those changes with the database at the correct time.

This chapter covers:

- `EntityManagerFactory` and `EntityManager`
- Persistence Context
- `@PersistenceContext`
- Transactions and `@Transactional`
- Entity lifecycle states
- Dirty checking
- Flushing and write-behind
- First-level cache

---

1. The Core Hibernate Components

Component	Responsibility	Typical Lifetime
`EntityManagerFactory`	Creates `EntityManager` instances and stores shared ORM metadata	Entire application
`EntityManager`	Provides operations for interacting with entities and the persistence context	One unit of work or transaction
Persistence Context	Stores and tracks managed entity instances	Usually one transaction
Database Transaction	Ensures a group of database operations succeeds or fails as one unit	One business operation

The basic relationship is:

```
EntityManagerFactory
        |
        | creates
        ▼
EntityManager
        |
        | interacts with
        ▼
Persistence Context
        |
        | synchronizes through Hibernate
        ▼
Database
```

The factory is expensive to create and is normally shared for the lifetime of the application. An actual `EntityManager` is short-lived and must not be shared concurrently between threads.

---

2. JPA, Hibernate, and Spring Boot

JPA: The Specification

Jakarta Persistence, commonly called JPA, defines a standard contract for object-relational mapping in Java. It provides interfaces, annotations, and rules such as:

- `EntityManagerFactory`
- `EntityManager`
- `@Entity`
- `@Id`
- `@GeneratedValue`

JPA defines what persistence operations should do. It does not contain the complete ORM engine that performs those operations.

Hibernate: The Provider

Hibernate is a JPA provider. It implements the JPA contract and performs the actual work:

- mapping entity objects to tables,

- generating SQL,
- interacting with JDBC,
- tracking entity state,
- performing dirty checking,
- managing the persistence context,
- coordinating synchronization with the database.

Hibernate also offers its own native API:

JPA API	Hibernate Native API
`EntityManagerFactory`	`SessionFactory`
`EntityManager`	`Session`

In Hibernate, `SessionFactory` extends `EntityManagerFactory`, and `Session` extends `EntityManager`. This allows Hibernate to support the standard JPA API while also providing Hibernate-specific features.

Spring Boot's Role

In a Spring Boot application, our code normally uses JPA types while Hibernate works underneath as the provider.

```
import jakarta.persistence.Entity;
import jakarta.persistence.EntityManager;
import jakarta.persistence.Id;
```

Programming against JPA keeps most persistence code provider-independent. In principle, Hibernate could be replaced by another JPA provider with fewer changes than if the application directly depended on Hibernate-specific APIs.

> Spring Boot configures the persistence infrastructure, JPA provides the standard API, and Hibernate implements the ORM behavior.

---

3. The Persistence Context

A persistence context is Hibernate's in-memory workspace for managed entities.

```
Persistence Context
┌──────────────────────────────────┐
│ Student#1 → Student object       │
│ Student#2 → Student object       │
│ Student#7 → Student object       │
│                                  │
│ Tracks identity and state        │
└──────────────────────────────────┘
```

Suppose we execute:

```java
Student student =
        entityManager.find(Student.class, 1L);
```

If the entity is not already available in the current persistence context, Hibernate may:

1. execute a `SELECT`,
2. create a `Student` object from the returned row,
3. place that object in the persistence context,
4. begin managing its state.

Conceptually:

```
(Student.class, 1L)
          |
          ▼
Student {
    id = 1,
    name = "Rahul",
    email = "rahul@gmail.com"
}
```

The persistence context is responsible for much more than storing objects. It supports:

- entity identity,

- lifecycle state management,
- automatic change detection,
- first-level caching,
- transactional write-behind,
- synchronization with the database.

The Real Purpose of `EntityManager`

`EntityManager` is not merely a class that runs database queries. It is the main JPA interface through which we interact with a persistence context.

```
entityManager.persist(student);
entityManager.find(Student.class, 1L);
entityManager.remove(student);
entityManager.flush();
entityManager.detach(student);
entityManager.clear();
entityManager.refresh(student);
```

Each operation affects an entity, the persistence context, or the synchronization between the persistence context and the database.

---

4. `@PersistenceContext` in Spring

An `EntityManager` can be injected into a Spring repository:

```
@Repository
public class StudentRepository {

    @PersistenceContext
    private EntityManager entityManager;
}
```

The name `@PersistenceContext` is meaningful. We are asking Spring for an `EntityManager` connected to the persistence context that is appropriate for the

current operation.

Why Spring Does Not Inject One Permanent `EntityManager`

Spring repositories are singleton beans by default. The same repository object can therefore be used by many request threads.

An actual JPA `EntityManager`, like a Hibernate `Session`, is not thread-safe. Sharing one physical instance across concurrent requests would be unsafe.

Spring normally injects a shared proxy instead:

```
Singleton Repository
        |
        ▼
Shared EntityManager Proxy
        |
        | routes the call
        ▼
Transaction-associated EntityManager
        |
        ▼
Persistence Context
```

Two concurrent transactions can therefore use the same repository while working with different actual persistence contexts:

```
Request A → Transaction A → EntityManager A → Persistence Context A

Request B → Transaction B → EntityManager B → Persistence Context B
```

The injected proxy is stable, but the actual `EntityManager` handling a call depends on the current transactional context.

---

5. Transactions from First Principles

Consider a bank transfer:

```sql
UPDATE accounts
SET balance = balance - 100
```

```
WHERE id = 1;

UPDATE accounts
SET balance = balance + 100
WHERE id = 2;
```

If the first statement succeeds and the second fails, money is deducted from one account but never added to the other.

A transaction treats both statements as one logical operation:

```
BEGIN
  UPDATE account A
  UPDATE account B
COMMIT
```

If any operation fails:

```
BEGIN
  UPDATE account A
  UPDATE account B  ← error
ROLLBACK
```

The region between beginning and completing the transaction is called the **transaction boundary**.

Unit of Work

A unit of work is the complete set of persistence actions required to perform one business operation.

For example, changing a student's email may involve:

1. retrieving the student,
2. checking business rules,
3. modifying the entity,
4. detecting the change,

5. generating SQL,
6. executing SQL,
7. committing the transaction.

All these steps belong to one logical unit of work.

The transaction defines whether the database changes succeed together. The persistence context tracks the entities participating in that work.

Database, Hibernate, and Spring Transactions

These are different layers around the same transactional work.

Database Transaction

The database understands commands such as:

```
BEGIN;
UPDATE students SET email = 'new@email.com' WHERE id = 1;
COMMIT;
```

Hibernate Transaction

Using Hibernate's native API, transaction management can be written manually:

```
Session session = sessionFactory.openSession();
Transaction transaction = session.beginTransaction();

try {
    session.persist(student);
    transaction.commit();
} catch (Exception exception) {
    transaction.rollback();
    throw exception;
} finally {
```

```
    session.close();
}
```

Spring Transaction

Spring removes the repetitive transaction-management code:

```
@Transactional
public void createStudent(Student student) {
    entityManager.persist(student);
}
```

Conceptually, Spring performs:

```
Begin transaction
        ↓
Execute method
        ↓
Flush pending changes
        ↓
Commit
```

If the method fails with an exception that triggers rollback:

```
Begin transaction
        ↓
Execute method
        ↓
Exception
        ↓
Rollback
```

By default, Spring rolls back for unchecked exceptions such as `RuntimeException` and `Error`. Rollback behavior for checked exceptions can be configured using attributes such as `rollbackFor`.

---

6. How `@Transactional` Works

Spring normally applies `@Transactional` through an AOP proxy.

```
@Service
public class StudentService {

    @Transactional
    public void updateStudent(Long id) {
        // business operation
    }
}
```

Conceptual call flow:

```
Controller
    |
    ▼
StudentService Proxy
    |
    | TransactionInterceptor begins transaction
    ▼
StudentService.updateStudent()
    |
    | method completes or throws an exception
    ▼
Commit or rollback
```

Because the proxy controls the transaction boundary, a call made from one method to another method on the same object may bypass the proxy. This is the usual self-invocation limitation of Spring proxy-based AOP.

---

7. Complete Transaction and Persistence Context Flow

Consider:

```
@Transactional
public void updateStudent(Long id) {

    Student student =
```

```
            entityManager.find(Student.class, id);

    student.setName("Rohan");
}
```

Step 1: The Call Reaches the Service Proxy

```
Controller → StudentService Proxy
```

Step 2: Spring Starts the Transaction

The transaction interceptor creates or joins a transaction according to the propagation rules.

Step 3: An `EntityManager` Is Associated with the Transaction

```
Transaction
    |
    ▼
EntityManager
    |
    ▼
Persistence Context
```

Step 4: Hibernate Executes `find()`

Hibernate first checks the persistence context for `Student#id`.

- If present, it returns the managed instance.
- If absent, it queries the database, creates the entity, and manages it.

Step 5: The Managed Entity Is Modified

```
student.setName("Rohan");
```

No explicit update method is required. Hibernate tracks the managed object.

Step 6: Hibernate Flushes

Before commit, Hibernate normally synchronizes pending persistence-context changes with the database.

```
UPDATE students
SET name = 'Rohan'
WHERE id = ?;
```

Step 7: The Database Transaction Commits

After a successful commit, the changes become durable. When the transaction-scoped persistence context ends, its entities are no longer managed by that context.

---

8. Why Transactions Usually Belong in the Service Layer

A repository method often represents one database action. A business operation may involve several repository calls.

```
@Transactional
public void enrollStudent(Long studentId, Long courseId) {

    Student student = studentRepository.findById(studentId);
    Course course = courseRepository.findById(courseId);

    enrollmentRepository.create(student, course);
    paymentRepository.recordPayment(student, course);
}
```

If each repository method had an independent transaction, part of the business operation could commit while a later part failed.

The service layer is normally the better transaction boundary because it represents the complete business use case.

> Repository methods perform persistence operations. Service methods define which operations must succeed or fail together.

Repository-level `@Transactional` can still be useful, but it should not accidentally divide one business operation into several unrelated transaction boundaries.

9. Which Operations Require a Transaction?

For a normal transaction-scoped JPA persistence context:

Operation	Transaction Required?	Purpose
`find()`	Not normally required without locking	Retrieve an entity by primary key
`persist()`	Yes	Make a new entity managed and schedule its insertion
`merge()`	Yes	Copy state into a managed instance
`remove()`	Yes	Mark a managed entity for deletion
`flush()`	Yes	Synchronize pending changes with the database
`refresh()`	Yes in normal application usage	Reload an entity's state from the database

A basic `find()` may work without an explicit `@Transactional`, but modifying data should be performed inside a clearly defined transaction.

10. Entity Lifecycle States

An entity instance can move through four main lifecycle states:

1. Transient
2. Managed
3. Detached
4. Removed

The managed state is central to Hibernate because dirty checking, automatic updates, first-level caching, relationship synchronization, and flushing depend on

it.

10.1 Transient State

```
Student student = new Student();
student.setName("Rahul");
student.setEmail("rahul@gmail.com");
```

This object:

- exists in Java memory,
- is not associated with a persistence context,
- is not tracked by Hibernate,
- normally does not yet represent a stored database row.

```
new Student() → TRANSIENT
```

`@Entity` makes instances of the class eligible for persistence. It does not cause every object created using `new` to be managed automatically.

10.2 Managed State

An entity is managed when it is associated with the current persistence context.

A new entity becomes managed through `persist()`:

```
entityManager.persist(student);
```

An existing entity returned by `find()` is also managed:

```
Student student =
        entityManager.find(Student.class, 1L);
```

```
Transient entity ──persist()──→ Managed entity
```

```
Database row ──find()──→ Managed entity
```

Hibernate does not create a second copy when `persist()` is called. The same Java object becomes managed.

Managed Does Not Mean "Already Written to the Database"

Calling:

```
entityManager.persist(student);
```

makes the entity managed, but the `INSERT` may be delayed until a flush. The exact timing can depend on identifier generation and other Hibernate behavior.

The term **managed** describes the object's relationship with the persistence context, not whether SQL has already executed.

10.3 Detached State

A detached entity:

- was previously managed,
- still exists as a Java object,
- may still represent an existing row,
- is no longer associated with its original persistence context.

Common ways an entity becomes detached:

The Persistence Context Ends

```
Transaction-scoped context ends
            ↓
Previously managed entities become detached
```

`detach()`

```
entityManager.detach(student);
```

This detaches one entity.

`clear()`

```
entityManager.clear();
```

This detaches every managed entity and empties the persistence context.

Changes made after detachment are not automatically detected by the original persistence context.

10.4 Removed State

A managed entity becomes removed when:

```
entityManager.remove(student);
```

Hibernate now knows that the corresponding row should be deleted when changes are flushed.

`remove()` is primarily a lifecycle transition. It does not necessarily mean that the `DELETE` statement executes at that exact line.

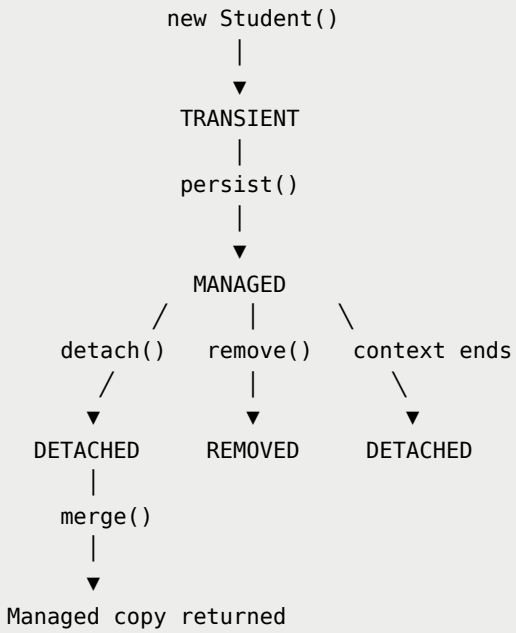
Removed vs. Detached

Detached Entity	Removed Entity
Hibernate has stopped managing it	Hibernate is managing its deletion
Its database row should remain	Its database row is scheduled for deletion
Later field changes are not dirty-checked	Deletion is synchronized during flush

`detach(student)` does not delete the student. `remove(student)` does.

---

11. Entity Lifecycle Diagram



Important Rule About `merge()`

`merge()` does not reattach the same detached Java object.

```
Student managedStudent =
        entityManager.merge(detachedStudent);
```

Hibernate copies the detached object's state into a managed instance and returns that managed instance.

```
detachedStudent  → remains detached
managedStudent   → managed return value
```

Always use the object returned by `merge()` when further managed operations are required.

---

12. Checking Whether an Entity Is Managed

Use `contains()`:

```
Student student =
        entityManager.find(Student.class, 1L);

boolean managed =
        entityManager.contains(student);
```

Inside the same persistence context, the result is normally:

```
true
```

After:

```
entityManager.detach(student);
```

the result becomes:

```
false
```

---

13. Managed Entity Identity

Within one persistence context, Hibernate maintains one managed object for a particular entity type and primary key.

```
Student first =
        entityManager.find(Student.class, 1L);

Student second =
        entityManager.find(Student.class, 1L);
```

Within the same persistence context:

```
first == second   // normally true
```

Both references point to the same managed Java object.

This is called the **identity map** principle:

> Within one persistence context, one database identity is represented by one managed entity instance.

Without this rule, two objects representing the same row could contain conflicting values inside the same unit of work.

---

14. Dirty Checking

In Hibernate, **dirty** does not mean invalid or corrupted. An entity is dirty when its current managed state differs from the state Hibernate previously knew.

```
Original state: name = "Rahul"
Current state:  name = "Rohan"
```

The Problem Dirty Checking Solves

```
@Transactional
public void updateStudent(Long id) {

    Student student =
            entityManager.find(Student.class, id);

    student.setName("Rohan");
}
```

There is no call to:

```
entityManager.update(student);
```

JPA does not even define an `EntityManager.update()` method. The object returned by `find()` is managed, so Hibernate can automatically detect its changed state and generate the required `UPDATE`.

```
find()
   ↓
Entity becomes managed
   ↓
Field value changes
   ↓
Flush occurs
   ↓
Dirty checking detects the change
   ↓
UPDATE is generated
```

When Dirty Checking Happens

Calling a setter does not normally execute SQL immediately.

```
Java object changes
        ↓
Entity remains managed in memory
        ↓
Flush occurs
        ↓
Hibernate detects relevant changes
        ↓
SQL executes
```

Hibernate can compare the entity's current state with its earlier snapshot during flushing. Depending on configuration and enhancement, Hibernate may also use more optimized change-tracking techniques.

Dirty Checking Applies Only to Managed Entities

```java
Student student =
        entityManager.find(Student.class, 1L);

entityManager.detach(student);
student.setName("Rohan");
```

The name change is not automatically written to the database because `student` is detached.

Setting the Same Value

```
student.setName("Rahul");
```

If the original value was already `"Rahul"`, Hibernate normally detects no actual state change and does not need to issue an update for that field change.

---

15. Flushing

Flush synchronizes the changes represented by the persistence context with the database by executing the required SQL statements.

```
Persistence Context
        |
        | flush
        ▼
INSERT / UPDATE / DELETE
        |
        ▼
Database
```

Flush Is Not Commit

This distinction is essential:

`flush()`	`commit()`
Sends pending SQL to the database	Successfully completes the transaction
Changes are still part of the current transaction	Changes become durable
Can be followed by rollback	Ends the transaction successfully

Example:

```
BEGIN
  Modify managed entity
  FLUSH       → UPDATE executes
  Error occurs
ROLLBACK      → update is undone
```

SQL execution does not automatically mean that a transaction has committed.

Automatic Flush

With the common `AUTO` flush mode, Hibernate normally flushes:

- before transaction commit,
- before certain queries whose results could be affected by pending changes.

Example:

```
@Transactional
public void updateStudent() {

    Student student =
            entityManager.find(Student.class, 1L);

    student.setName("Rohan");
}
```

Typical flow:

```
Transaction begins
        ↓
SELECT
        ↓
Student becomes managed
        ↓
setName("Rohan")
        ↓
Method returns
        ↓
Flush
        ↓
UPDATE
        ↓
Commit
```

Manual Flush

```
entityManager.flush();
```

Manual flushing is useful when SQL must be executed before the end of the method, for example:

- to detect a database constraint violation earlier,
- to synchronize pending changes before later database work,
- to control memory during batch processing when combined with `clear()`.

---

16. Transactional Write-Behind

Hibernate describes the persistence context as a transactional write-behind cache.

In a write-through approach, each Java change would immediately cause SQL:

```
Java change → SQL
Java change → SQL
Java change → SQL
```

With write-behind:

```
Java change ─┐
Java change ─┼─→ Persistence Context ─→ Flush ─→ SQL
Java change ─┘
```

Delaying synchronization gives Hibernate an opportunity to:

- avoid unnecessary work,
- order SQL statements,
- group operations,
- support JDBC batching,
- synchronize at a suitable point in the transaction.

Write-behind does not remove the need for transactions. It works inside the transaction so that the database can still commit or roll back the resulting SQL as one unit.

---

17. `flush()` and `clear()` in Batch Processing

Suppose we persist 100,000 entities in one loop:

```
for (int i = 0; i < 100_000; i++) {
    Student student = createStudent(i);
    entityManager.persist(student);
}
```

If every entity remains managed until the end, the persistence context can grow very large. That increases memory usage and the amount of entity-management work.

A common batching pattern is:

```
for (int i = 0; i < 100_000; i++) {

    Student student = createStudent(i);
    entityManager.persist(student);

    if ((i + 1) % 100 == 0) {
        entityManager.flush();
        entityManager.clear();
    }
}
```

The two operations serve different purposes:

Operation	Effect
`flush()`	Executes pending SQL; entities remain managed
`clear()`	Detaches all managed entities and empties the persistence context

Conceptually:

```
100 managed entities
        ↓
flush() → SQL is synchronized
        ↓
```

```
clear() → entities become detached
        ↓
Process the next batch
```

This controls persistence-context memory. Actual JDBC batching also depends on Hibernate's batch configuration.

---

18. First-Level Cache

The persistence context provides Hibernate's first-level cache behavior.

```
Persistence Context
┌──────────────────────────────────────┐
│ (Student, 1) → Student Object A      │
│ (Student, 2) → Student Object B      │
└──────────────────────────────────────┘
```

The cache key is conceptually made from:

```
Entity type + primary key
```

The first-level cache:

- is enabled by default,
- cannot be disabled like an optional external cache,
- belongs to one persistence context,
- is normally transaction-scoped in a Spring application,
- also guarantees managed entity identity.

First Lookup: Cache Miss

```java
Student first =
        entityManager.find(Student.class, 1L);
```

If `Student#1` is not in the persistence context:

```
Persistence Context miss
        ↓
SELECT from database
        ↓
Create Student Object A
        ↓
Store and manage Object A
```

Second Lookup: Cache Hit

```java
Student second =
        entityManager.find(Student.class, 1L);
```

If this happens in the same persistence context:

```
Persistence Context hit
        ↓
Return Student Object A
        ↓
No second SELECT required
```

Therefore:

```java
first == second   // normally true
```

Scope of the First-Level Cache

```java
@Transactional
public void methodA() {
    entityManager.find(Student.class, 1L); // SELECT
    entityManager.find(Student.class, 1L); // first-level cac
he hit
}
```

In a later independent transaction:

```
@Transactional
public void methodB() {
    entityManager.find(Student.class, 1L); // SELECT again
}
```

This is expected because the second transaction normally uses a different persistence context.

> First-level cache is not an application-wide cache. It is local to one persistence context.

Important Query Limitation

Primary-key lookups such as `find()` can be satisfied directly from the persistence context.

A JPQL or Criteria query may still execute SQL even if matching entities are already managed. Hibernate will preserve identity by returning the existing managed instances where applicable, but the query itself is not automatically skipped in every case.

---

19. How `clear()` and `detach()` Affect the Cache

`clear()`

```
Student first =
        entityManager.find(Student.class, 1L);

entityManager.clear();

Student second =
        entityManager.find(Student.class, 1L);
```

`clear()` removes all managed entities from the persistence context. The second `find()` may therefore execute another `SELECT`.

```
first == second   // normally false
```

`first` is the old detached object, while `second` is a new managed object.

`detach()`

```
Student student =
        entityManager.find(Student.class, 1L);

entityManager.detach(student);
```

Only that entity is detached. A later lookup may need to load and manage the entity again.

Method	Scope
`detach(entity)`	One entity
`clear()`	Every managed entity

20. Refreshing an Entity

The first-level cache can contain values that are older than the current database values if another transaction or external process changes the row.

Suppose the managed entity contains:

```
name = "Rahul"
```

Another process changes the database value to:

```
name = "Aman"
```

The managed Java object does not update automatically. Its value can be reloaded using:

```
entityManager.refresh(student);
```

Conceptually:

```
Current managed state
        ↓ replaced
Latest database state
        ↓
Managed entity now contains fresh values
```

`refresh()` discards unsynchronized in-memory changes for the refreshed state and replaces them with database values, so it should be used deliberately.

---

21. Quick Reference: Important `EntityManager` Methods

Method	Main Effect
`persist(entity)`	Makes a new entity managed and schedules insertion
`find(Type.class, id)`	Finds by primary key and returns a managed entity
`merge(entity)`	Copies state into a managed instance and returns it
`remove(entity)`	Marks a managed entity for deletion
`contains(entity)`	Checks whether the instance is managed
`detach(entity)`	Detaches one entity
`clear()`	Detaches all entities
`flush()`	Synchronizes pending changes with the database
`refresh(entity)`	Replaces current entity state with database state

---

22. Common Misconceptions

“An `@Entity` object is automatically managed.”

No. `new Student()` is transient until it becomes associated with a persistence context.

“Calling a setter immediately executes an `UPDATE`.”

No. A setter changes the Java object. Hibernate normally detects the change during flush.

“Managed means the SQL has already executed.”

No. Managed describes the entity’s relationship with the persistence context.

“`flush()` and `commit()` are the same.”

No. Flush executes pending SQL inside the current transaction. Commit completes the transaction.

“`detach()` deletes the database row.”

No. It only stops the current persistence context from managing that object.

“`merge()` makes the same detached object managed.”

No. It returns a managed instance containing copied state. The supplied object remains detached.

“The first-level cache is shared across the application.”

No. It belongs to a single persistence context.

“A singleton repository shares one physical `EntityManager` across all requests.”

Normally no. Spring injects a proxy that routes calls to the transaction-associated `EntityManager`.

---

23. Complete Mental Model

```
@Transactional service method begins
                ↓
Spring starts or joins a transaction
                ↓
EntityManager is associated with the transaction
                ↓
Persistence Context manages loaded/new entities
                ↓
```

```
Business code changes managed objects
                ↓
Dirty checking detects state changes
                ↓
Flush converts changes into SQL
                ↓
Database transaction commits or rolls back
                ↓
Transaction-scoped Persistence Context ends
                ↓
Previously managed objects become detached
```

The central idea is:

> Hibernate does not simply translate individual method calls into SQL. It manages entity state inside a persistence context and synchronizes that state with the database within a transaction.

Coder Army

JPA Relationships: Mapping, Ownership and Cardinality

JPA relationships allow Java entities to reference one another while Hibernate translates those object references into foreign keys and join tables in a relational database.

These notes cover:

- Why relationship mapping is required
- Relationship cardinality
- Unidirectional and bidirectional relationships
- Owning and inverse sides
- `@ManyToOne` and `@OneToMany`
- `@OneToOne`
- `@ManyToMany`
- `@JoinColumn`, `mappedBy` and `@JoinTable`
- Maintaining both sides of a relationship
- Relationship insert, update and dirty-checking behaviour

---

1. The Fundamental Problem

Consider the following Java model:

```
class Student {
    Long id;
    String name;
    Department department;
}
```

In Java, the `department` field stores a reference to a `Department` object:

Coder Army

```
student.getDepartment();
```

A relational database cannot place a Java object inside a column. It represents the same relationship using a foreign key:

```
students
--------------------------------
id | name   | department_id
--------------------------------
1  | Rahul  | 10
2  | Aman   | 10
3  | Priya  | 20
```

```
departments
-----------------
id | name
-----------------
10 | Engineering
20 | HR
```

The Java relationship:

```
student.department
```

must be translated into:

```
students.department_id → departments.id
```

> Relationship mapping is the configuration that tells JPA how an object reference should be represented in the database.

Without ORM, an entity might store only the foreign-key value:

```
@Entity
class Student {
```

Coder Army

```
    @Id
    private Long id;

    private String name;
    private Long departmentId;
}
```

With JPA, the object model can contain an actual entity reference:

```
@Entity
class Student {

    @Id
    private Long id;

    private String name;

    @ManyToOne
    private Department department;
}
```

Hibernate then converts the entity reference into the required SQL and foreign-key value.

2. Entity Relationships

Simple entities contain only values that belong directly to that entity:

```
@Entity
class Student {

    @Id
    private Long id;

    private String name;
```

```
    private Integer age;
}
```

Real applications, however, contain connected entities:

```
class Student {
    Department department;
}
```

```
class Order {
    List<OrderItem> items;
}
```

```
class Employee {
    Employee manager;
}
```

```
class User {
    Profile profile;
}
```

The application therefore contains a graph of connected objects rather than a set of isolated objects. These connections are called **relationships** or **associations**.

---

3. Relationship Cardinality

Cardinality answers one question:

> How many entities on one side can be associated with how many entities on the other side?

JPA supports four basic cardinalities:

Coder Army

Coder Army

Relationship	Meaning	Common example
`@OneToOne`	One entity is associated with one other entity	User and Profile
`@OneToMany`	One entity is associated with many entities	Department and Students
`@ManyToOne`	Many entities are associated with one entity	Students and Department
`@ManyToMany`	Many entities on both sides can be associated	Students and Courses

One-to-many and many-to-one are two views of the same relationship

Suppose the Engineering department contains three students:

```
Engineering
├── Rahul
├── Aman
└── Priya
```

From the department's perspective:

```
One Department → Many Students
```

This is `@OneToMany`.

From a student's perspective:

```
Many Students → One Department
```

This is `@ManyToOne`.

The database does not create two separate relationships. Both directions normally describe the same foreign key:

```
students.department_id
```

CoderArmy

4. Unidirectional and Bidirectional Relationships

These terms describe navigation in the Java object model. They do not describe how many directions SQL can query.

Unidirectional relationship

Only one entity contains a reference to the other:

```
class Student {
    Department department;
}
```

```
class Department {
}
```

The application can navigate from `Student` to `Department`:

```
student.getDepartment();
```

It cannot navigate directly from `Department` to its students:

```
department.getStudents(); // No such field
```

The direction is:

```
Student → Department
```

Bidirectional relationship

Both entities contain a reference to the other:

```
class Student {
    Department department;
}
```

Coder Army

```
class Department {
    List<Student> students;
}
```

The application can navigate in both directions:

```
student.getDepartment();
department.getStudents();
```

The direction is:

```
Student ↔ Department
```

A bidirectional mapping still represents one database relationship. In this example, there is still only one foreign key:

```
students.department_id
```

Bidirectional fields do not synchronize themselves

These two Java fields are independent:

```
student.setDepartment(engineering);
```

```
engineering.getStudents().add(student);
```

Calling the first statement does not automatically execute the second. It is therefore possible to produce an inconsistent in-memory object graph:

```
student.department = Engineering
engineering.students = []
```

The application must keep both sides synchronized.

CoderArmy

5. Owning Side and Inverse Side

When the same database relationship is mapped on two entities, JPA needs one authoritative side.

Owning side

The owning side is the side JPA uses to determine the database value of the relationship.

For a department–student relationship:

```
@ManyToOne
@JoinColumn(name = "department_id")
private Department department;
```

`Student` is the owning side because changing:

```
student.setDepartment(department);
```

changes:

```
students.department_id
```

Owning side does **not** mean:

- The parent entity
- The more important entity
- The entity that conceptually contains the other entity

It means only that this side controls the database representation of the relationship.

Inverse side

The inverse side provides the opposite navigation path but does not independently control the same relationship:

Coder Army

```java
@OneToMany(mappedBy = "department")
private List<Student> students = new ArrayList<>();
```

Here, `Department.students` is the inverse side.

> For database relationship updates, the owning side is authoritative.

---

6. Many-to-One Mapping

Many students can belong to one department:

```
Student N → 1 Department
```

Database design

```
departments
-------------------
id | name
-------------------
1  | Engineering
2  | HR
```

```
students
--------------------------------
id  | name   | department_id
--------------------------------
101 | Rahul  | 1
102 | Priya  | 1
103 | Aman   | 2
```

`students.department_id` is a foreign key that references `departments.id`.

Multiple student rows can contain the same department ID, which creates the many-to-one relationship.

Mapping with `@ManyToOne`

Coder Army

```java
@Entity
public class Student {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
}
```

`@ManyToOne` tells JPA that:

- The field references another entity.
- Many `Student` entities may reference the same `Department`.

`@JoinColumn` specifies the foreign-key column used to store the relationship:

```
Student.department
        ↓
students.department_id
```

By default, the join column references the primary key of the target entity.

If:

```java
engineering.getId() == 5L;
student.setDepartment(engineering);
```

then the database relationship becomes:

```
students.department_id = 5
```

Coder Army

Nullable and mandatory relationships

If a student may exist without a department, the foreign-key value may be `NULL`:

```
id | name    | department_id
-----------------------------
1  | Harshit | NULL
```

The default mapping permits this:

```
@ManyToOne
@JoinColumn(name = "department_id")
private Department department;
```

If every student must belong to a department:

```
@ManyToOne(optional = false)
@JoinColumn(
    name = "department_id",
    nullable = false
)
private Department department;
```

The two settings express the same rule at different layers:

Setting	Layer	Meaning
`optional = false`	JPA object/persistence model	The association is required
`nullable = false`	Generated database schema	The foreign-key column must be `NOT NULL`

Insert behaviour

If the department already exists:

```
Department department =
        entityManager.find(Department.class, 1L);
```

Coder Army

```
Student student = new Student();
student.setName("Rahul");
student.setDepartment(department);

entityManager.persist(student);
```

Hibernate logically executes:

```
INSERT INTO students (name, department_id)
VALUES ('Rahul', 1);
```

The existing department is not inserted again. Hibernate stores its identifier as the foreign key.

If both entities are new:

```
Department department = new Department();
department.setName("Engineering");

Student student = new Student();
student.setName("Rahul");
student.setDepartment(department);
```

Without cascading, persist them explicitly in the correct lifecycle:

```
entityManager.persist(department);
entityManager.persist(student);
```

With cascade persist:

```
@ManyToOne(cascade = CascadeType.PERSIST)
private Department department;
```

persisting the student can also persist the new department.

Coder Army

> Setting an entity reference and managing the lifecycle of the referenced entity are separate concerns.

Update behaviour and dirty checking

Suppose Rahul currently belongs to Engineering:

```
student_id = 1
department_id = 10
```

Changing the department of a managed student:

```
Department hr =
        entityManager.find(Department.class, 20L);

student.setDepartment(hr);
```

changes the relationship from department `10` to `20`. During flush, Hibernate can issue:

```
UPDATE students
SET department_id = 20
WHERE id = 1;
```

This is relationship dirty checking.

---

7. One-to-Many Mapping

`@OneToMany` represents the same department–student relationship from the department side:

```
Department 1 → N Students
```

Database design

No additional foreign key is required:

Coder Army

```
students
---------------------------------
id | name   | department_id
---------------------------------
1  | Rahul  | 10
2  | Aman   | 10
3  | Priya  | 10
```

All three students belong to department `10`. The collection in `Department` is an object-side view of these rows.

Mapping with `@OneToMany`

```java
@Entity
class Department {

    @Id
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department")
    private List<Student> students =
            new ArrayList<>();
}
```

`mappedBy = "department"` refers to the name of the Java field in `Student`:

```java
class Student {
    private Department department;
}
```

It does **not** refer to the database column name `department_id`.

The meaning is:

> `Department.students` is the inverse view of the relationship owned by `Student.department`.

Coder Army

Complete bidirectional mapping

```
@Entity
class Student {

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
}
```

```
@Entity
class Department {

    @OneToMany(mappedBy = "department")
    private List<Student> students =
            new ArrayList<>();
}
```

This creates two Java navigation paths:

```
student.getDepartment();
department.getStudents();
```

Both paths describe the same foreign key:

```
students.department_id
```

Maintaining both sides

The following statement updates only the `Student` object:

```
student.setDepartment(engineering);
```

The department collection must also be updated:

Coder Army

```
engineering.getStudents().add(student);
```

For removal:

```
engineering.getStudents().remove(student);
student.setDepartment(null);
```

Relationship helper methods

Instead of repeating synchronization logic throughout the application, place it inside the entity:

```
@Entity
public class Department {

    @OneToMany(mappedBy = "department")
    private List<Student> students =
            new ArrayList<>();

    public void addStudent(Student student) {
        students.add(student);
        student.setDepartment(this);
    }

    public void removeStudent(Student student) {
        students.remove(student);
        student.setDepartment(null);
    }
}
```

The application can now use:

```
engineering.addStudent(rahul);
engineering.removeStudent(rahul);
```

Coder Army

This keeps the object graph consistent and ensures the owning side is updated.

Collection dirty checking

Hibernate can detect changes made to a managed relationship collection:

```
Department department =
        entityManager.find(Department.class, 10L);

department.getStudents().remove(student);
```

However, detecting a collection change is not the same as deciding which side controls the foreign key.

Because `Department.students` uses `mappedBy`, it is the inverse side. The owning side must also be changed:

```
student.setDepartment(null);
```

This is why helper methods are important.

---

8. One-to-One Mapping

A one-to-one relationship means that each entity can be associated with at most one entity on the other side:

```
User 1 ↔ 1 Profile
```

Database design

```
users
----------------
id | username
----------------
1  | rahul
```

Coder Army

```
profiles
-------------------------------
id | bio              | user_id
-------------------------------
10 | Software dev     | 1
```

A foreign key alone does not guarantee one-to-one. Without a uniqueness constraint, the database could allow:

```
id | user_id
-------------
10 | 1
11 | 1
12 | 1
```

That would mean one user has many profiles.

The usual relational representation of one-to-one is:

```
Foreign key + unique constraint
```

Mapping with `@OneToOne`

```java
@Entity
class Profile {

    @Id
    @GeneratedValue
    private Long id;

    private String bio;

    @OneToOne
    @JoinColumn(
        name = "user_id",
        unique = true
```

```
    )
    private User user;
}
```

The essential database difference is:

Mapping	Foreign key	Uniqueness
Many-to-one	Present	Repeated values allowed
One-to-one	Present	Foreign-key value must be unique

Owning and inverse sides

`Profile` is the owning side because it contains the join column:

```
@OneToOne
@JoinColumn(name = "user_id")
private User user;
```

The inverse side in `User` is:

```
@Entity
class User {

    @OneToOne(mappedBy = "user")
    private Profile profile;
}
```

Here, `"user"` refers to `Profile.user`.

```
Profile.user  → Owning side
User.profile  → Inverse side
```

Mandatory one-to-one relationship

If a profile must always belong to a user:

Coder Army

Coder Army

```
@OneToOne(optional = false)
@JoinColumn(
    name = "user_id",
    nullable = false,
    unique = true
)
private User user;
```

Shared primary-key mapping

Some one-to-one entities cannot exist independently. A profile, for example, may use the same identifier as its user:

```
users
----------------
id | username
----------------
10 | rahul
```

```
profiles
----------------
id | bio
----------------
10 | ...
```

In `profiles`, the `id` column acts as both:

- The primary key of `Profile`
- A foreign key referencing `users.id`

JPA can model this with `@MapsId`:

```
@Entity
class Profile {

    @Id
    private Long id;
```

Coder Army

```
    private String bio;

    @OneToOne
    @MapsId
    @JoinColumn(name = "id")
    private User user;
}
```

`@MapsId` means that the associated user's ID is also used as the profile's ID.

This mapping is useful when the dependent entity has no meaningful identity or lifecycle without its parent.

---

9. Many-to-Many Mapping

A many-to-many relationship allows multiple entities on each side to be connected:

```
Student N ↔ N Course
```

A student can enroll in many courses, and a course can contain many students.

Database design

```
students
------------
1 | Rahul
2 | Priya
```

```
courses
--------------
101 | Java
102 | Spring
103 | SQL
```

Coder Army

Neither table can represent the relationship using one foreign-key column. A third table is required:

```
student_course
------------------------
student_id | course_id
------------------------
1          | 101
1          | 102
2          | 101
2          | 103
```

Each row represents one association between one student and one course.

This middle table is called a:

- Join table
- Association table
- Junction table

It contains two foreign keys:

```
student_id → students.id
course_id  → courses.id
```

The pair should normally be unique:

```
(student_id, course_id)
```

This prevents the same enrollment from being inserted more than once.

Owning side with `@JoinTable`

```java
@Entity
public class Student {

    @Id
    private Long id;
```

Coder Army

```
    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns =
            @JoinColumn(name = "student_id"),
        inverseJoinColumns =
            @JoinColumn(name = "course_id"),
        uniqueConstraints =
            @UniqueConstraint(
                columnNames = {
                    "student_id",
                    "course_id"
                }
            )
    )
    private Set<Course> courses =
            new HashSet<>();
}
```

`@JoinTable` describes the middle table:

Attribute	Meaning
`name`	Name of the join table
`joinColumns`	Foreign key pointing to the owning entity
`inverseJoinColumns`	Foreign key pointing to the other entity

`Student.courses` is the owning side. Changes to this collection control rows in `student_course`:

```
student.getCourses().add(javaCourse);
```

Hibernate can then insert:

```
INSERT INTO student_course (student_id, course_id)
```

Coder Army

```
VALUES (1, 101);
```

Inverse side

```
@Entity
public class Course {

    @Id
    private Long id;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students =
            new HashSet<>();
}
```

`mappedBy = "courses"` points to the `courses` field in `Student`.

```
Student.courses → Owning side
Course.students → Inverse side
```

The mapping creates two Java navigation paths:

```
student.getCourses();
course.getStudents();
```

Both represent rows in the same `student_course` table.

Maintaining both sides

Adding a course to only one collection leaves the in-memory model inconsistent:

```
student.getCourses().add(course);
```

Use helper methods:

Coder Army

```
public void addCourse(Course course) {
    courses.add(course);
    course.getStudents().add(this);
}

public void removeCourse(Course course) {
    courses.remove(course);
    course.getStudents().remove(this);
}
```

Now:

```
student.addCourse(course);
student.removeCourse(course);
```

keeps both sides synchronized.

---

10. Lombok with JPA Entities

Common Lombok annotations include:

Annotation	Generated code
`@Getter`	Getter methods
`@Setter`	Setter methods
`@NoArgsConstructor`	Constructor without parameters
`@AllArgsConstructor`	Constructor containing all fields
`@RequiredArgsConstructor`	Constructor for `final` and `@NonNull` fields
`@Builder`	Builder-pattern API

Be careful with `@Data` on JPA entities. In addition to getters and setters, it generates `toString()`, `equals()` and `hashCode()`.

When relationships are bidirectional, generated methods may:

- Recursively move from one entity to the other

Coder Army

- Cause a `StackOverflowError`
- Trigger lazy relationship loading unexpectedly
- Compare large relationship collections
- Produce unstable equality or hash-code behaviour for entities

For JPA entities, explicit Lombok annotations are usually safer:

```
@Getter
@Setter
@NoArgsConstructor
@Entity
public class Student {
    // fields
}
```

If `toString()`, `equals()` or `hashCode()` is generated, relationship fields should generally be excluded and entity identity should be designed deliberately.

---

11. Quick Reference

Relationship annotations

Annotation	Typical database representation
`@ManyToOne`	Foreign key on the many side
`@OneToMany(mappedBy = "...")`	Inverse collection over an existing foreign key
`@OneToOne`	Foreign key with a uniqueness constraint
`@ManyToMany`	Join table containing two foreign keys

Ownership rules

Mapping	Typical owning side
Bidirectional one-to-many/many-to-one	The `@ManyToOne` side containing `@JoinColumn`
Bidirectional one-to-one	The side containing `@JoinColumn`

Mapping	Typical owning side
Bidirectional many-to-many	The side containing `@JoinTable`

Coder Army

Core principles

1. Java stores entity references; relational databases store foreign keys.
2. Cardinality describes how many entities can be associated on each side.
3. Unidirectional and bidirectional describe Java navigation.
4. A bidirectional mapping still represents one database relationship.
5. The owning side determines what relationship value JPA writes.
6. `mappedBy` points to a Java field on the owning entity, not a database column.
7. Both sides of a bidirectional relationship must be synchronized in Java.
8. Helper methods are the safest way to preserve relationship consistency.
9. Setting an entity reference does not automatically persist the referenced entity.
10. Cascading controls lifecycle propagation; it does not define relationship ownership.

Coder Army

JPA Cascading, Lazy Loading and the N+1 Problem

JPA relationships do more than connect two entity classes. They also raise three practical questions:

1. What should happen to related entities when one entity is saved or deleted?
2. When should related data be loaded from the database?
3. How can we avoid executing a large number of hidden SQL queries?

These questions are handled through cascading, fetch strategies and query-level fetch plans.

---

1. Cascading

What Is a Cascade?

A cascade propagates a persistence operation from one entity to another through a mapped relationship.

Suppose an `Order` contains multiple `OrderItem` objects:

```
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(
        mappedBy = "order",
        cascade = CascadeType.PERSIST
    )
```

```
    private List<OrderItem> items = new ArrayList<>();
}
```

```
@Entity
public class OrderItem {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "order_id")
    private Order order;
}
```

Now consider:

```
entityManager.persist(order);
```

Because the relationship uses `CascadeType.PERSIST`, Hibernate also persists the items contained in `order.getItems()`.

Conceptually, the single call behaves like this:

```
persist(order);
persist(item1);
persist(item2);
persist(item3);
```

The application does not need to call `persist()` separately for every `OrderItem`.

Cascade Does Not Mean Relationship Ownership

Cascading and relationship ownership solve different problems:

- **Relationship ownership** decides which side controls the foreign-key value.

Coder Army

Coder Army

- **Cascading** decides whether a persistence operation should propagate to a related entity.

In the example above:

```
@ManyToOne
@JoinColumn(name = "order_id")
private Order order;
```

`OrderItem` is the owning side because it contains the `order_id` foreign key.

The `Order.items` collection is the inverse side because it uses:

```
mappedBy = "order"
```

Adding an item only to the inverse collection is not enough to set the foreign key correctly. Both sides of the object relationship should be maintained:

```
public void addItem(OrderItem item) {
    items.add(item);
    item.setOrder(this);
}
```

Usage:

```
Order order = new Order();

OrderItem item1 = new OrderItem();
OrderItem item2 = new OrderItem();

order.addItem(item1);
order.addItem(item2);

entityManager.persist(order);
```

The helper method keeps the Java object graph consistent, while `CascadeType.PERSIST` allows the items to be persisted through the order.

Coder Army

Cascade Is Not Enabled by Default

JPA does not automatically cascade every operation across relationships.

If no cascade is configured:

```
@OneToMany(mappedBy = "order")
private List<OrderItem> items;
```

then:

```
entityManager.persist(order);
```

does not automatically persist new `OrderItem` objects.

This is intentional. Related entities do not always share the same lifecycle.

---

2. `CascadeType.PERSIST`

`CascadeType.PERSIST` propagates the persist operation.

```
@OneToMany(
    mappedBy = "order",
    cascade = CascadeType.PERSIST
)
private List<OrderItem> items;
```

When a new order is persisted:

```
entityManager.persist(order);
```

new items reachable through the relationship are persisted as well.

This cascade is useful when:

- the parent and children are created together;
- the child belongs to the parent's lifecycle;

Coder Army

- saving the parent should also save its newly created children.

It should not be added merely for convenience. The correct question is:

> If this parent is persisted, should these related objects always be persisted with it?

---

3. `CascadeType.REMOVE`

`CascadeType.REMOVE` propagates deletion from one entity to related entities.

```
@OneToMany(
    mappedBy = "order",
    cascade = CascadeType.REMOVE
)
private List<OrderItem> items;
```

Now:

```
entityManager.remove(order);
```

conceptually becomes:

```
remove(Order)
    ├── remove(Item A)
    ├── remove(Item B)
    └── remove(Item C)
```

Possible SQL:

```
DELETE FROM order_item
WHERE id = 1;

DELETE FROM order_item
WHERE id = 2;
```

Coder Army

```
DELETE FROM order_item
WHERE id = 3;

DELETE FROM orders
WHERE id = 100;
```

The child rows are normally deleted before the parent row so that the foreign-key constraint is not violated.

When Does `REMOVE` Make Sense?

`CascadeType.REMOVE` makes sense when the child belongs exclusively to the parent.

An `OrderItem` normally has no independent meaning without its `Order`. If the order is deleted, deleting its items is usually reasonable.

It can be dangerous when the related entity is shared.

For example:

```
@ManyToMany
private Set<Course> courses;
```

If many students share the same course, deleting one student must not delete the course itself. Therefore, `CascadeType.REMOVE` is usually inappropriate on shared many-to-many relationships.

---

4. Other Cascade Types

JPA provides the following cascade types:

```
CascadeType.PERSIST
CascadeType.MERGE
CascadeType.REMOVE
CascadeType.REFRESH
CascadeType.DETACH
CascadeType.ALL
```

Coder Army

Their meaning can be summarized as follows:

Cascade type	Propagated operation
`PERSIST`	Persist new related entities
`MERGE`	Merge the state of related detached entities
`REMOVE`	Remove related entities
`REFRESH`	Reload related entities from the database
`DETACH`	Detach related entities from the persistence context
`ALL`	Apply all the cascade types above

Multiple cascade types can be selected:

```
@OneToMany(
    mappedBy = "order",
    cascade = {
        CascadeType.PERSIST,
        CascadeType.REMOVE
    }
)
private List<OrderItem> items;
```

`CascadeType.ALL` should be used only when the related entity genuinely shares the complete lifecycle of the parent.

---

5. Relationship Loading

Fetching an entity by its primary key is straightforward:

```
Student student =
        entityManager.find(
            Student.class,
            1L
        );
```

However, a `Student` may be related to many other entities:

Coder Army

```
Student
├── Department
├── Address
├── Profile
├── Courses
├── Projects
├── Certificates
└── Comments
```

Those entities may contain further relationships of their own.

If Hibernate loaded every reachable entity whenever one `Student` was requested, a small operation could load a huge portion of the database.

Hibernate therefore needs a fetch plan:

> Which relationships are required now, and which relationships can wait until they are actually used?

---

6. Static Fetch Strategy

A fetch strategy can be configured directly in the entity mapping:

```
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

This is called a **static fetch strategy** because it becomes part of the entity's mapping metadata.

It acts as the default whenever the entity is loaded:

```
Student student =
        entityManager.find(
            Student.class,
            1L
        );
```

Coder Army

The default can still be overridden for a particular use case through a fetch join, an entity graph or another query-specific fetch plan.

---

7. `FetchType.LAZY` and `FetchType.EAGER`

JPA defines two fetch types:

```
FetchType.LAZY
FetchType.EAGER
```

Conceptually:

```
LAZY
Load the relationship when it is first needed.

EAGER
Load the relationship as part of loading the entity.
```

There is also an important specification-level distinction:

- `EAGER` is a requirement. The persistence provider must make the relationship available eagerly.
- `LAZY` is a hint. The provider is asked to delay loading, but the specification does not absolutely require it to do so.

Hibernate supports lazy loading extensively:

- to-many relationships are represented using lazy persistent collections;
- to-one relationships can be loaded lazily through proxies or bytecode enhancement.

---

8. Default Fetch Types

JPA defines the following defaults:

Relationship	Default fetch type

Coder Army

`@OneToMany`	`LAZY`
`@ManyToMany`	`LAZY`
`@ManyToOne`	`EAGER`
`@OneToOne`	`EAGER`

Therefore, to-many relationships are lazy by default, while to-one relationships are eager by default.

These defaults should not automatically be treated as the best choice for every application. In practice, many teams explicitly configure to-one relationships as lazy:

```
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

This keeps the default object graph small and allows each query to request the relationships required by its use case.

---

9. How Lazy Loading Works

Consider:

```
@Entity
public class Student {

    @Id
    private Long id;

    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}
```

When the student is loaded:

Coder Army

```
Student student =
        entityManager.find(
            Student.class,
            1L
        );
```

Hibernate can initially execute:

```
SELECT
    id,
    name,
    department_id
FROM student
WHERE id = 1;
```

The `Student` is loaded, but its `Department` may be represented by an uninitialized proxy.

Conceptually:

```
Student
├── id = 1
├── name = Rahul
└── department = not initialized yet
```

Accessing only the department reference does not necessarily initialize it:

```
Department department =
        student.getDepartment();
```

Accessing data that requires the real department state can trigger initialization:

```
String departmentName =
        student.getDepartment().getName();
```

Hibernate may then execute:

Coder Army

```sql
SELECT
    id,
    name
FROM department
WHERE id = 10;
```

Lazy loading therefore defers the cost of fetching an association until that association is actually used.

---

10. Why Lazy Loading Is Useful

Lazy loading is useful because different use cases need different parts of the same entity graph.

For example:

- a student list may need only `id` and `name`;
- a student details page may also need `department` and `profile`;
- an administration report may need `department`, `courses` and `projects`.

Using one large eager graph for all three cases wastes database work and memory.

Lazy mapping provides a small default graph. Each use case can then explicitly fetch the associations it needs.

However, lazy loading creates another risk: database queries can be triggered implicitly while Java code navigates relationships. This is how the N+1 problem commonly appears.

---

11. The N+1 Query Problem

The N+1 problem occurs when:

1. one query loads a collection of parent entities; and
2. additional queries are executed while accessing a relationship for each parent.

Coder Army

Suppose `Student.department` is lazy:

```
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

Load all students:

```
List<Student> students =
        studentRepository.findAll();
```

Hibernate first executes one query:

```
SELECT
    id,
    name,
    department_id
FROM student;
```

Now iterate through the results:

```
for (Student student : students) {
    System.out.println(
        student.getDepartment().getName()
    );
}
```

Each department access may initialize a lazy proxy:

```
SELECT * FROM department WHERE id = 1;
SELECT * FROM department WHERE id = 2;
SELECT * FROM department WHERE id = 3;
-- additional queries
```

The total pattern is:

Coder Army

```
1 query for the students
+
N additional relationship queries
```

The Java loop looks harmless, but every relationship access may involve a database round trip.

---

12. First-Level Cache and the Exact Query Count

The term N+1 describes the query pattern. It does not guarantee that exactly N additional SQL statements will always be executed.

Suppose 100 students belong to only three departments.

The initial query loads all students:

```
1 student query
```

As departments are accessed, Hibernate may need only:

```
3 department queries
```

Once a department has been loaded into the current persistence context, Hibernate can reuse that managed entity for other students referencing the same department.

The result may therefore be:

```
1 + 3 queries
```

rather than:

```
1 + 100 queries
```

Coder Army

The first-level cache can reduce duplicate primary-key lookups within one persistence context, but it does not solve the underlying fetch-plan problem.

The important issue remains:

> Relationship access is causing additional SQL statements that were not visible in the original repository call.

---

13. N+1 During Collection Iteration

The classic to-many example begins by loading departments:

```
List<Department> departments =
        departmentRepository.findAll();
```

Suppose `Department.students` is lazy:

```
@OneToMany(
    mappedBy = "department",
    fetch = FetchType.LAZY
)
private List<Student> students;
```

Now iterate over every collection:

```
for (Department department : departments) {

    for (Student student :
            department.getStudents()) {

        System.out.println(student.getName());
    }
}
```

Hibernate may execute:

Coder Army

```
SELECT * FROM department;

SELECT * FROM student
WHERE department_id = 1;

SELECT * FROM student
WHERE department_id = 2;

SELECT * FROM student
WHERE department_id = 3;
```

There is one query for all departments and one additional query for the student collection of each department.

This is a true N+1 pattern:

```
1 parent query + N collection queries
```

---

14. Does N+1 Mean Lazy Loading Is Bad?

No.

The N+1 problem can make it appear that lazy loading is bad and eager loading is the solution. That conclusion is too broad.

Lazy loading is useful when an association is not required. The problem arises when the application knows that it will access an association for every result but does not include that association in the query's fetch plan.

The correct principle is:

> Keep relationships lazy by default and fetch the required graph explicitly for each use case.

Changing the mapping to `EAGER` is not a reliable N+1 solution.

For example:

```
@ManyToOne(fetch = FetchType.EAGER)
```

Coder Army

```
private Department department;
```

JPA requires the departments to be available eagerly, but it does not require Hibernate to obtain everything using one join query. Depending on the query and provider behavior, Hibernate may still execute secondary selects.

`EAGER` also applies globally. It loads the association even in use cases that do not need it.

Therefore:

- `LAZY` controls the default timing;
- a query-level fetch plan controls what a specific operation needs;
- neither setting should be chosen without considering the use case and generated SQL.

---

15. Detecting N+1 with SQL Logging

One of the simplest ways to detect N+1 during development is to inspect the SQL generated by Hibernate.

Enable SQL logging:

```
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

For parameter values in modern Hibernate versions, logging can also be configured through the application's logging settings:

```
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.orm.jdbc.bind=TRACE
```

Now run the use case and inspect:

- how many SQL queries are executed;
- whether queries repeat inside a loop;
- which Java relationship access triggers each query;

Coder Army

- whether the same association is fetched even when it is not required.

SQL logging is especially useful while learning because it makes the hidden database work visible.

---

16. Preventing N+1 with an Entity Graph

An entity graph defines which associations should be fetched for a particular operation.

Suppose the default mapping remains lazy:

```
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

In Spring Data JPA:

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {

    @EntityGraph(attributePaths = "department")
    List<Student> findAll();
}
```

This method requests `department` as part of its fetch plan.

The important design is:

```
Default entity mapping
Student.department = LAZY

Specific repository method
Fetch Student + Department
```

The mapping remains small for other operations, while this method fetches the association it knows it will need.

Coder Army

Hibernate commonly satisfies this graph using a join, although the exact SQL strategy is controlled by the persistence provider. The entity graph defines what must be fetched, not one universally guaranteed SQL statement.

Fetching Multiple Attributes

Multiple relationships can be included:

```
@EntityGraph(
    attributePaths = {
        "department",
        "profile"
    }
)
List<Student> findAll();
```

Conceptually, this repository method requests:

```
Student
├── Department
└── Profile
```

Both attributes become part of that method's fetch plan.

Nested Attribute Paths

Spring Data JPA also supports nested paths:

```
@EntityGraph(
    attributePaths = {
        "department.manager"
    }
)
List<Student> findAll();
```

This asks for:

Coder Army

```
Student
└── Department
    └── Manager
```

Nested graphs should be selected carefully. Fetching several large collections in one operation can produce a very large joined result and duplicate rows at the SQL level.

Coder Army

Spring Data JPA

Repository Abstraction, Query Methods, Sorting and Pagination

Spring Data JPA simplifies the repository layer of a JPA application. It does not replace JPA or Hibernate. Instead, it builds a higher-level repository abstraction on top of them.

```
Application service
        ↓
Spring Data JPA repository
        ↓
JPA EntityManager
        ↓
Hibernate
        ↓
JDBC driver
        ↓
Relational database
```

---

1. The Persistence Layer after Hibernate and JPA

Before JPA and Hibernate, persistence code commonly used JDBC or Spring JDBC:

```
public Student findById(Long id) {
    String sql = """
            SELECT id, name, email
            FROM students
            WHERE id = ?
            """;

    return jdbcTemplate.queryForObject(
```

```
            sql,
            studentRowMapper,
            id
    );
}
```

This code requires the developer to handle:

- SQL statements
- Parameter binding
- Result-set mapping
- Object creation
- Database-specific details

JPA and Hibernate remove much of this work:

```
public Student findById(Long id) {
    return entityManager.find(Student.class, id);
}
```

JPA and Hibernate provide:

- Object-relational mapping
- Entity state management
- Persistence context
- Dirty checking
- Relationship mapping
- SQL generation
- First-level caching
- Transaction synchronization

However, an application still needs a dedicated persistence layer. Business services should focus on use cases, while repositories should contain data-access logic.

Coder Army

Coder Army

```java
@Repository
public class StudentRepository {

    @PersistenceContext
    private EntityManager entityManager;

    public void save(Student student) {
        entityManager.persist(student);
    }

    public Student findById(Long id) {
        return entityManager.find(Student.class, id);
    }

    public void delete(Student student) {
        entityManager.remove(student);
    }
}
```

> JPA and Hibernate do not eliminate the repository layer. They simplify the persistence logic written inside it.

---

2. The Repository Boilerplate Problem

Suppose an application contains three entities:

```
Student
Department
Course
```

Without Spring Data JPA, each entity may need its own repository implementation:

```
StudentRepository
DepartmentRepository
```

```
CourseRepository
```

These repositories repeatedly implement the same basic operations:

- Save an entity
- Find an entity by ID
- Find all entities
- Check whether an entity exists
- Count entities
- Delete an entity

Only two pieces of information usually change:

1. The entity type
2. The identifier type

This common structure can be represented generically:

```
Repository<EntityType, IdentifierType>
```

Examples:

```
Repository<Student, Long>
Repository<Department, Long>
Repository<Course, Integer>
```

This generic repository idea is the foundation of Spring Data.

---

3. Spring Data and Spring Data JPA

Spring Data

Spring Data is an umbrella project that provides a consistent repository-based programming model for different persistence technologies.

Coder Army

Coder Army

Examples include:

- Spring Data JPA
- Spring Data JDBC
- Spring Data MongoDB
- Spring Data Redis
- Spring Data Elasticsearch
- Spring Data Cassandra
- Spring Data Neo4j

The central idea is simple:

> Define a repository interface for a domain type, and let the appropriate Spring Data module provide the supporting implementation and infrastructure.

The implementation underneath depends on the selected module.

```
Spring Data JPA repository
        ↓
EntityManager
        ↓
JPA provider such as Hibernate
        ↓
Relational database
```

```
Spring Data MongoDB repository
        ↓
MongoDB infrastructure
        ↓
MongoDB
```

Spring Data JPA

Spring Data JPA is the Spring Data module designed for applications that use the Jakarta Persistence API.

Instead of manually implementing a repository class:

Coder Army

```
@Repository
public class StudentRepository {

    @PersistenceContext
    private EntityManager entityManager;

    public Student findById(Long id) {
        return entityManager.find(Student.class, id);
    }
}
```

the developer can declare an interface:

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

The repository can then be injected and used like a normal Spring bean:

```
@Service
public class StudentService {

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepositor
y) {
        this.studentRepository = studentRepository;
    }
}
```

Common operations become immediately available:

```
studentRepository.save(student);
studentRepository.findById(id);
```

Coder Army

```
studentRepository.findAll();
studentRepository.deleteById(id);
```

At startup, Spring Data detects the interface and creates a repository bean for it.

4. JPA versus Spring Data JPA

JPA	Spring Data JPA
Jakarta specification	Spring framework module
Defines persistence concepts and APIs	Builds a repository abstraction on top of JPA
Provides `EntityManager` and JPQL	Provides repository interfaces and query-method support
Requires a persistence provider	Uses the configured JPA provider underneath
Does not contain `JpaRepository`	Provides `JpaRepository`

JPA provides

```
@Entity
@Id
@OneToMany
@ManyToOne
EntityManager
EntityManagerFactory
TypedQuery
```

Typical JPA operations are:

```
entityManager.persist(student);
entityManager.find(Student.class, 1L);
entityManager.remove(student);
```

Spring Data JPA provides

Coder Army

```
Repository
CrudRepository
ListCrudRepository
PagingAndSortingRepository
JpaRepository
@Query
@NativeQuery
@Modifying
@EntityGraph
JpaSpecificationExecutor
```

> `JpaRepository` is part of Spring Data JPA, not part of the JPA specification.

Main responsibilities of Spring Data JPA

Common CRUD operations

```
save()
findById()
findAll()
existsById()
count()
deleteById()
```

Runtime repository implementation

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

Spring Data JPA creates the repository bean; the developer does not write the implementation class.

Query derivation from method names

```
List<Student> findByDepartmentName(String departmentName);
```

Custom JPQL and native SQL

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.email = :email
       """)
Optional<Student> findStudentByEmail(
        @Param("email") String email
);
```

Sorting and pagination

```
Page<Student> findAll(Pageable pageable);
```

What Spring Data JPA does not do

Spring Data JPA does not remove the need to understand:

- Entity mapping
- Transactions
- Managed and detached entity states
- Lazy loading and fetching
- SQL performance
- Database indexes and constraints
- The N+1 query problem

It reduces repository boilerplate; it does not replace sound persistence design.

Coder Army

Coder Army

5. Repository Interface Hierarchy

The important repository abstractions are:

```
Repository<T, ID>
├── CrudRepository<T, ID>
│   └── ListCrudRepository<T, ID>
└── PagingAndSortingRepository<T, ID>
    └── ListPagingAndSortingRepository<T, ID>

JpaRepository<T, ID>
├── ListCrudRepository<T, ID>
├── ListPagingAndSortingRepository<T, ID>
└── QueryByExampleExecutor<T>
```

`Repository<T, ID>`

```java
public interface Repository<T, ID> {
}
```

`Repository` is primarily a marker interface. It identifies:

- The domain type: `T`
- The identifier type: `ID`
- The interface as a Spring Data repository

Extending it does not automatically expose the full CRUD API. This allows a restricted repository contract:

```java
public interface StudentRepository
        extends Repository<Student, Long> {

    Student save(Student student);

    Optional<Student> findById(Long id);
```

```
    List<Student> findAll();
}
```

Because no delete method is declared, callers cannot delete students through this repository interface.

`CrudRepository<T, ID>`

`CrudRepository` extends `Repository` and provides generic CRUD operations:

```
public interface CrudRepository<T, ID>
        extends Repository<T, ID> {

    <S extends T> S save(S entity);

    Optional<T> findById(ID id);

    Iterable<T> findAll();

    boolean existsById(ID id);

    long count();

    void deleteById(ID id);

    void delete(T entity);
}
```

`ListCrudRepository<T, ID>`

`ListCrudRepository` offers CRUD functionality similar to `CrudRepository`, but methods returning multiple entities return a `List` instead of an `Iterable`.

```
public interface StudentRepository
        extends ListCrudRepository<Student, Long> {
}
```

Coder Army

Coder Army

```
List<Student> students = studentRepository.findAll();
```

`PagingAndSortingRepository<T, ID>`

This interface adds sorted and paginated retrieval:

```
Iterable<T> findAll(Sort sort);

Page<T> findAll(Pageable pageable);
```

Since Spring Data 3, the sorting repository hierarchy is separate from the CRUD hierarchy. Extending only `PagingAndSortingRepository` does not guarantee methods such as `save()` and `deleteById()`.

To combine the lower-level contracts explicitly:

```
public interface StudentRepository
        extends ListCrudRepository<Student, Long>,
                ListPagingAndSortingRepository<Student, Long>
{
}
```

In a JPA application, extending `JpaRepository` is normally more convenient because it already combines these capabilities.

`JpaRepository<T, ID>`

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

It provides three broad groups of operations.

CRUD operations

Coder Army

```
save()
saveAll()
findById()
findAll()
existsById()
count()
delete()
deleteById()
deleteAll()
```

Sorting and pagination

```
findAll(Sort sort)
findAll(Pageable pageable)
```

JPA-specific operations

```
flush()
saveAndFlush()
saveAllAndFlush()
getReferenceById()
deleteAllInBatch()
deleteAllByIdInBatch()
```

> Batch delete methods can behave differently from deleting managed entities one by one. They should be used deliberately, especially when cascades, lifecycle callbacks, or the current persistence context matter.

---

6. Declaring a Repository Interface

A typical repository declaration is:

Coder Army

```java
package com.coderarmy.jpa.repository;

import com.coderarmy.jpa.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

Normally:

- No implementation class is required.
- The interface does not need `@Repository`.
- Spring Boot scans repository interfaces under the application's base package.
- `@EnableJpaRepositories` is generally unnecessary in a standard Spring Boot project, but it can be used for explicit or custom repository scanning.

The interface can declare additional query methods:

```java
public interface StudentRepository
        extends JpaRepository<Student, Long> {

    Optional<Student> findByEmail(String email);

    List<Student> findByAgeGreaterThan(Integer age);
}
```

Repository methods fall into two main categories:

Category	Examples	How they are handled
Inherited methods	`save()`, `findById()`, `findAll()`	Implemented by the repository base class
Declared query methods	`findByEmail()`, `findByAgeGreaterThan()`	Converted into or associated with queries

Coder Army

7. Repository Proxy Internals

Why an interface can be injected

The following interface cannot be instantiated directly:

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

```
new StudentRepository(); // compilation error
```

Spring Data creates a runtime object that implements the repository interface. This runtime-generated object is called a repository proxy.

```
StudentService
      ↓
StudentRepository proxy
      ↓
Classifies the invoked method
      ↓
Chooses the correct execution path
```

For an inherited method:

```
studentRepository.findById(1L)
      ↓
Repository proxy
      ↓
SimpleJpaRepository.findById(1L)
      ↓
EntityManager.find(...)
```

For a derived method:

```
studentRepository.findByEmail(email)
      ↓
Repository proxy
```

Coder Army

```
      ↓
Prepared query-method execution
      ↓
Bind parameter
      ↓
EntityManager executes query
```

`SimpleJpaRepository`

`SimpleJpaRepository<T, ID>` is the standard base implementation behind most `JpaRepository` operations.

Conceptually, it contains:

```
public class SimpleJpaRepository<T, ID>
        implements JpaRepository<T, ID> {

    private final EntityManager entityManager;
    private final JpaEntityInformation<T, ?> entityInformatio
n;
}
```

It implements methods such as:

```
save()
findById()
findAll()
existsById()
count()
delete()
flush()
saveAndFlush()
getReferenceById()
```

Why the proxy is still necessary

`SimpleJpaRepository` knows how to execute standard repository operations, but it cannot contain every application-specific method:

Coder Army

```
public interface StudentRepository
        extends JpaRepository<Student, Long> {

    Optional<Student> findByEmail(String email);

    List<Student> findByDepartmentName(String name);
}
```

The proxy combines multiple sources of behavior:

Method type	Execution source
Standard CRUD method	`SimpleJpaRepository`
Derived query method	Query created from the method name
`@Query` method	Declared JPQL or SQL query
Default interface method	Java interface default method
Custom repository fragment	Developer-provided implementation

Spring Data JPA remains a layer above JPA. Its base implementation ultimately delegates persistence work to `EntityManager`, whose implementation is commonly provided by Hibernate.

---

8. Important `JpaRepository` Methods

`save()`

```
Student savedStudent = studentRepository.save(student);
```

`save()` does not simply mean “execute an `INSERT`.” It means that the entity state should become persistent.

Conceptually:

```
Is the entity new?
      ↓
```

```
Yes → EntityManager.persist(entity)
No  → EntityManager.merge(entity)
```

Spring Data generally determines whether an entity is new by inspecting its version property or identifier. An entity can also implement `Persistable` to control this decision.

The return value matters when `merge()` is used:

```
Student managedStudent = studentRepository.save(detachedStude
nt);
```

`merge()` returns a managed instance. The detached object passed as the argument does not automatically become managed.

saveAll()

```
List<Student> savedStudents = studentRepository.saveAll(
        List.of(student1, student2, student3)
);
```

Conceptually, the standard implementation calls `save()` for each entity:

```
for (Student student : students) {
    result.add(save(student));
}
```

`saveAll()` does not itself guarantee one multi-row SQL statement. Hibernate/JDBC batching is a separate optimization that requires suitable configuration and an identifier strategy that supports it.

findById()

```
Optional<Student> student = studentRepository.findById(1L);
```

The core lookup delegates to `EntityManager.find()`.

Coder Army

Coder Army

```
findById(1L)
      ↓
Check persistence context
      ↓
Already managed?
├── Yes → return the managed instance
└── No  → query the database if necessary
```

It returns `Optional<Student>` because a row with the requested ID may not exist.

`findAll()`

```java
List<Student> students = studentRepository.findAll();
```

The method means “load every `Student` matched by the repository operation.” It is suitable for small datasets but dangerous for large tables. Spring Data cannot infer that pagination was intended.

`existsById()`

```java
boolean exists = studentRepository.existsById(1L);
```

This expresses an existence question rather than a request to load the complete entity. Spring Data generates an appropriate existence query, commonly using a count or equivalent mechanism.

`count()`

```java
long numberOfStudents = studentRepository.count();
```

Conceptually:

```sql
SELECT COUNT(*)
FROM students;
```

`delete()`

Coder Army

```
studentRepository.delete(student);
```

Deletion must ultimately operate on a managed entity. If the supplied entity is detached, the repository implementation may obtain or merge the corresponding managed state before removing it.

`deleteById()`

```
studentRepository.deleteById(id);
```

Use this when the identifier is available and the repository should remove the corresponding entity.

`deleteAll()`

```
studentRepository.deleteAll();
```

This is a broad and potentially expensive operation. It may process entities individually. For bulk deletion, JPA-specific batch methods or an explicit bulk query may be more suitable, but their persistence-context and lifecycle behavior must be understood.

`flush()`

```
studentRepository.flush();
```

`flush()` synchronizes pending changes in the persistence context with the database. It does not commit the transaction.

```
flush  = send pending SQL to the database
commit = permanently complete the transaction
```

The transaction can still roll back after a flush.

`saveAndFlush()`

Coder Army

```
Student savedStudent = studentRepository.saveAndFlush(studen
t);
```

Conceptually:

```
Student result = save(student);
flush();
return result;
```

Use it when the database must be synchronized immediately—for example, to detect a constraint violation before the transaction reaches its normal commit point. It should not be used automatically for every save.

---

9. `save()` versus Dirty Checking

Consider a managed entity loaded inside a transaction:

```
@Transactional
public void renameStudent(Long id, String newName) {
    Student student = studentRepository.findById(id)
            .orElseThrow();

    student.setName(newName);
}
```

No call to `save()` is required for JPA to update the row.

The process is:

```
Repository loads Student
        ↓
Student becomes managed
        ↓
Application changes the managed object
        ↓
Dirty checking detects the change
```

```
        ↓
UPDATE is generated during flush/commit
```

Calling `save()` for the managed entity is usually redundant from a pure JPA perspective. It may still be used when a team deliberately follows the repository abstraction consistently, but it is not what causes dirty checking.

`save()` is important when:

- Persisting a new entity
- Reattaching state from a detached entity through `merge()`
- Returning the managed instance produced by `merge()`

---

10. Derived Query Methods

Query derivation

Without query derivation:

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.email = :email
       """)
Optional<Student> findByEmail(@Param("email") String email);
```

With query derivation:

```
Optional<Student> findByEmail(String email);
```

Spring Data parses the method name and creates a query from it.

```
find  → selection operation
By    → beginning of the predicate
Email → Student.email property
```

Coder Army

Coder Army

Conceptual JPQL:

```
SELECT s
FROM Student s
WHERE s.email = ?1
```

The default lookup strategy first checks for a declared query and derives a query from the method name if none is found.

Subject and predicate

```
findByEmail
```

contains two logical sections:

```
findBy  → subject
Email   → predicate
```

The subject describes the operation:

```
findByEmail(...)
existsByEmail(...)
countByActiveTrue()
deleteByActiveFalse()
findTop5ByActiveTrueOrderByAgeDesc()
```

The predicate describes the matching condition:

```
findByAgeGreaterThanAndActiveTrue(Integer age)
```

Conceptual JPQL:

```
SELECT s
FROM Student s
```

```
WHERE s.age > ?1
  AND s.active = true
```

Common query keywords

Keyword	Repository method	Meaning
Equality	`findByEmail(String email)`	`email = ?`
`And`	`findByNameAndAge(String name, Integer age)`	Both conditions must match
`Or`	`findByNameOrEmail(String name, String email)`	Either condition may match
`GreaterThan`	`findByAgeGreaterThan(Integer age)`	`age > ?`
`LessThan`	`findByAgeLessThan(Integer age)`	`age < ?`
`Between`	`findByAgeBetween(Integer min, Integer max)`	Inclusive range
`Like`	`findByNameLike(String pattern)`	Caller supplies the pattern
`Containing`	`findByNameContaining(String value)`	Contains the supplied value
`StartingWith`	`findByNameStartingWith(String prefix)`	Starts with the value
`EndingWith`	`findByNameEndingWith(String suffix)`	Ends with the value
`In`	`findByAgeIn(Collection<Integer> ages)`	Value belongs to the collection
`NotIn`	`findByAgeNotIn(Collection<Integer> ages)`	Value does not belong to the collection
`IsNull`	`findByDepartmentIsNull()`	Property is `null`
`IsNotNull`	`findByDepartmentIsNotNull()`	Property is not `null`
`True`	`findByActiveTrue()`	Boolean property is true
`False`	`findByActiveFalse()`	Boolean property is false
`IgnoreCase`	`findByNameIgnoreCase(String name)`	Case-insensitive comparison
`OrderBy`	`findByActiveTrueOrderByNameAsc()`	Static ordering
`Top` / `First`	`findTop5ByOrderByAgeDesc()`	Limit the result

Coder Army

Coder Army

`Like` versus `Containing`

With `Like`, the caller supplies the wildcard characters:

```
List<Student> findByNameLike(String pattern);

studentRepository.findByNameLike("%har%");
```

With `Containing`, Spring Data adds the contains pattern:

```
List<Student> findByNameContaining(String value);

studentRepository.findByNameContaining("har");
```

Conceptually, the second method binds a pattern such as `%har%`.

Limiting results

```
List<Student> findTop5ByActiveTrueOrderByAgeDesc();
```

This means:

1. Select active students.
2. Sort them by age in descending order.
3. Return at most five.

An explicit order is important. Without it, “top five” has no reliable business meaning because a relational database does not guarantee a natural row order.

```
Optional<Student> findFirstByDepartmentNameOrderByAgeDesc(
        String departmentName
);
```

Limitations of derived query methods

Derived queries are ideal when the method name remains short and readable:

Coder Army

```
findByEmail(...)
findByActiveTrueOrderByNameAsc()
```

They become difficult to maintain when they grow into long query-language sentences:

```
findTop10ByNameContainingIgnoreCaseAndAgeGreaterThanAndActive
TrueAndDepartmentNameOrderByAdmissionDateDesc(...)
```

Main limitations:

- Long names are difficult to read.
- Complex `And` / `Or` grouping is hard to express clearly.
- The method name cannot show complicated joins and projections well.
- Database-specific behavior may require native SQL.

For such cases, use `@Query`, specifications, Query by Example, Querydsl, or a custom repository implementation depending on the problem.

---

11. Custom JPQL Queries

`@Query`

`@Query` keeps a custom query close to the repository method that executes it.

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.email = :email
       """)
Optional<Student> findStudentByEmail(
        @Param("email") String email
);
```

Coder Army

JPQL queries:

- Entity names
- Entity fields
- Entity relationships

JPQL does not directly use table names and database column names.

```
@Entity
public class Student {
}
```

```
SELECT s
FROM Student s
```

`Student` is the entity name; `s` is its query alias.

Named parameters

```
@Query("""
        SELECT s
        FROM Student s
        WHERE s.age >= :minimumAge
          AND s.age <= :maximumAge
        """)
List<Student> findStudentsInAgeRange(
        @Param("minimumAge") Integer minimumAge,
        @Param("maximumAge") Integer maximumAge
);
```

Named parameters are usually easier to read and maintain.

Positional parameters

Coder Army

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.age >= ?1
         AND s.active = ?2
       """)
List<Student> findStudents(
        Integer minimumAge,
        Boolean active
);
```

The indexes are one-based: `?1`, `?2`, and so on.

Relationship joins

Suppose `Student` contains:

```
@ManyToOne
private Department department;
```

An explicit JPQL join follows the entity relationship:

```
@Query("""
       SELECT s
       FROM Student s
       JOIN s.department d
       WHERE d.active = true
       """)
List<Student> findStudentsInActiveDepartments();
```

JPQL uses:

```
s.department
```

not a database expression such as:

```
s.department_id = d.id
```

Hibernate already knows the join-column mapping from the entity relationship.

Implicit versus explicit joins

Explicit join:

```
@Query("""
       SELECT s
       FROM Student s
       JOIN s.department d
       WHERE d.name = :departmentName
       """)
List<Student> findByDepartmentName(
        @Param("departmentName") String departmentName
);
```

Path navigation with an implicit join:

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.department.name = :departmentName
       """)
List<Student> findByDepartmentName(
        @Param("departmentName") String departmentName
);
```

Both can express the same filtering requirement. An explicit join is useful when:

- The joined entity needs an alias.
- Several conditions refer to the joined entity.
- The join type must be controlled.
- The query selects data from both entities.

Coder Army

Coder Army

Inner join

```
JOIN s.department d
```

An inner join includes a student only when a matching department exists. A student whose `department` is `null` is excluded.

Left join

```
@Query("""
       SELECT s.name, d.name
       FROM Student s
       LEFT JOIN s.department d
       """)
List<Object[]> findStudentAndDepartmentNames();
```

A left join preserves students without a department.

```
Aditya | CSE
Rohit  | IT
Harsh  | null
```

For production code, a DTO or interface projection is usually clearer than exposing `Object[]` to higher layers.

Static ordering in JPQL

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.active = true
       ORDER BY s.name ASC
       """)
List<Student> findActiveStudentsOrderedByName();
```

Coder Army

Multiple sort fields:

```
@Query("""
       SELECT s
       FROM Student s
       JOIN s.department d
       WHERE s.active = true
       ORDER BY d.name ASC, s.name ASC
       """)
List<Student> findActiveStudentsOrdered();
```

Static ordering is appropriate when the business operation always requires the same order.

Dynamic ordering with `Sort`

```
@Query("""
       SELECT s
       FROM Student s
       WHERE s.active = true
       """)
List<Student> findActiveStudents(Sort sort);
```

```
Sort sort = Sort.by("name").ascending();

List<Student> students =
        studentRepository.findActiveStudents(sort);
```

---

12. Native Queries

JPQL operates on the entity model. Native SQL is useful when a query needs:

- Database-specific functions
- Vendor-specific syntax

Coder Army

- Common table expressions or advanced SQL features
- Existing SQL that cannot be expressed cleanly in JPQL
- Precise control over the SQL statement

Using `@Query(nativeQuery = true)`

```
@Query(
    value = """
            SELECT *
            FROM students
            WHERE email = :email
            """,
    nativeQuery = true
)
Optional<Student> findByEmailUsingNativeQuery(
        @Param("email") String email
);
```

`nativeQuery = true` tells Spring Data to interpret the string as database SQL instead of JPQL.

Recent Spring Data JPA versions also provide `@NativeQuery`, a composed form designed specifically for native queries:

```
@NativeQuery("""
        SELECT *
        FROM students
        WHERE email = :email
        """)
Optional<Student> findByEmailUsingNativeQuery(
        @Param("email") String email
);
```

Native named parameters

Coder Army

```
@Query(
    value = """
            SELECT *
            FROM students
            WHERE age >= :minimumAge
              AND active = :active
            """,
    nativeQuery = true
)
List<Student> findStudentsUsingNativeQuery(
        @Param("minimumAge") Integer minimumAge,
        @Param("active") Boolean active
);
```

Native positional parameters

```
@Query(
    value = """
            SELECT *
            FROM students
            WHERE age >= ?1
              AND active = ?2
            """,
    nativeQuery = true
)
List<Student> findStudentsUsingNativeQuery(
        Integer minimumAge,
        Boolean active
);
```

> A native query is coupled to the database schema and possibly to the selected database vendor. Use it when the benefit outweighs the loss of portability.

Coder Army

13. Sorting

Without `ORDER BY`, a relational database does not guarantee a meaningful result order. Sorting is therefore part of query design, not just presentation.

Creating a `Sort`

```
Sort sort = Sort.by("name");
```

Ascending order is the default. An explicit form is:

```
Sort sort = Sort.by(
        Sort.Direction.ASC,
        "name"
);
```

Use it with a repository method:

```
List<Student> students = studentRepository.findAll(sort);
```

The `Sort` argument must not be `null`. Use `Sort.unsorted()` when no ordering is required.

Ascending and descending order

```
Sort byName = Sort.by("name").ascending();

Sort byNewest =
        Sort.by("admissionDate").descending();
```

Multiple properties

```
Sort sort = Sort.by("department.name")
        .ascending()
```

```
        .and(Sort.by("name").ascending());
```

Conceptually:

```
ORDER BY department.name ASC,
         student.name ASC
```

Sort properties refer to entity properties, not database column names:

```
Sort.by("admissionDate")
```

Dynamic sorting with a query method

```
List<Student> findByActiveTrue(Sort sort);
```

```
Sort sort = Sort.by(
        Sort.Order.desc("age"),
        Sort.Order.asc("name")
);

List<Student> students =
        studentRepository.findByActiveTrue(sort);
```

The predicate remains fixed while the caller controls the order.

Validating user-controlled sorting

Suppose an API accepts:

```
GET /students?sortBy=name&direction=asc
```

Do not pass arbitrary client input directly into `Sort`. Validate it against an allowlist:

Coder Army

Coder Army

```
private static final Set<String> ALLOWED_SORT_FIELDS =
        Set.of("name", "age", "admissionDate");

private Sort createSort(
        String property,
        String direction
) {
    if (!ALLOWED_SORT_FIELDS.contains(property)) {
        throw new IllegalArgumentException(
                "Unsupported sort property: " + property
        );
    }

    Sort.Direction sortDirection =
            Sort.Direction.fromString(direction);

    return Sort.by(sortDirection, property);
}
```

Validation prevents invalid property paths and keeps the API contract intentional.

14. Pagination

Why pagination exists

Suppose a table contains two million students. Calling:

```
studentRepository.findAll();
```

can cause problems throughout the stack:

- The database reads too many rows.
- The network transfers too much data.
- Hibernate creates too many entity objects.

Coder Army

- The persistence context consumes excessive memory.
- The JSON response becomes enormous.
- The client cannot use the entire result effectively.

Pagination changes the request from:

```
Give me every student.
```

to:

```
Give me a controlled window of students.
```

Page number and page size

With zero-based page indexing and a page size of 10:

```
Page 0 → records 1–10
Page 1 → records 11–20
Page 2 → records 21–30
```

The conceptual offset is:

```
offset = page number × page size
```

For page `2` with size `10`:

```
offset = 2 × 10 = 20
```

Conceptual SQL:

```sql
SELECT *
FROM students
```

```
ORDER BY id
LIMIT 10 OFFSET 20;
```

The actual pagination syntax is database-dependent and generated by Hibernate.

Pageable

`Pageable` describes:

- Requested page number
- Requested page size
- Sort information
- Whether the request is paged or unpaged

```
Page<Student> findByActiveTrue(Pageable pageable);
```

`Pageable` is an interface. `PageRequest` is its commonly used implementation.

PageRequest

Unsorted request:

```
Pageable pageable = PageRequest.of(0, 10);
```

Sorted request:

```
Pageable pageable = PageRequest.of(
        0,
        10,
        Sort.by("name").ascending()
);
```

Convenience form:

```
Pageable pageable = PageRequest.of(
        0,
```

Coder Army

```
        10,
        Sort.Direction.ASC,
        "name"
);
```

`PageRequest.of()` requires:

- A non-negative page number
- A page size greater than zero

The last page may contain fewer elements:

```
Total elements = 45
Page size      = 20

Page 0 → 20 elements
Page 1 → 20 elements
Page 2 → 5 elements
```

At the API boundary, validate the page size and enforce a maximum value to prevent clients from requesting an unreasonably large page.

Sorting and pagination must work together

```
Pageable pageable = PageRequest.of(
        0,
        10,
        Sort.by("name").ascending()
);
```

This request contains:

```
Page number = 0
Page size   = 10
Sort        = name ascending
```

Coder Army

A stable and deterministic order is important because page boundaries depend on the ordering. If the primary sort field is not unique, adding a unique tie-breaker such as `id` can prevent inconsistent ordering between requests.

```
Sort sort = Sort.by("name").ascending()
        .and(Sort.by("id").ascending());
```

Coder Army

15. `Page<T>`

```
Page<Student> page = studentRepository.findAll(
        PageRequest.of(0, 10)
);
```

A `Page<Student>` contains both the current content and pagination metadata.

Common `Page` methods

```
List<Student> content = page.getContent();

int pageNumber = page.getNumber();
int pageSize = page.getSize();
int elementsInCurrentPage = page.getNumberOfElements();

long totalElements = page.getTotalElements();
int totalPages = page.getTotalPages();

boolean first = page.isFirst();
boolean last = page.isLast();
boolean hasNext = page.hasNext();
boolean hasPrevious = page.hasPrevious();

Sort sort = page.getSort();
```

Coder Army

The content contains only the entities in the current page. For a page size of 10, its size can be between 0 and 10.

`Page` extends `Slice` and adds:

- Total number of matching elements
- Total number of pages

---

16. Count Queries

Suppose the content query returns ten rows. Those ten rows alone cannot reveal whether the complete result contains 10, 11, 1,000, or 10 million rows.

To calculate `totalElements` and `totalPages`, a paged operation commonly needs two queries.

Content query

```
SELECT ...
FROM students
WHERE active = true
ORDER BY name
LIMIT 10 OFFSET 20;
```

Count query

```
SELECT COUNT(*)
FROM students
WHERE active = true;
```

For a simple derived method, Spring Data can derive both queries:

```
Page<Student> findByActiveTrue(Pageable pageable);
```

Custom count query with `@Query`

Coder Army

Complex JPQL can provide an explicit count query:

```
@Query(
    value = """
            SELECT s
            FROM Student s
            JOIN s.department d
            WHERE s.active = true
              AND d.name = :departmentName
            """,
    countQuery = """
            SELECT COUNT(s)
            FROM Student s
            JOIN s.department d
            WHERE s.active = true
              AND d.name = :departmentName
            """
)
Page<Student> findActiveStudentsByDepartment(
        @Param("departmentName") String departmentName,
        Pageable pageable
);
```

An explicit count query is particularly useful when the content query contains:

- Complex joins
- Grouping
- Projections
- Fetch joins
- Native SQL

The count query should reproduce the filtering logic but omit unnecessary ordering and fetching.

Coder Army

17. `Slice<T>`

A `Slice` represents the current chunk of data and whether another chunk is available.

```
Slice<Student> findByActiveTrue(Pageable pageable);
```

Common `Slice` methods

```
List<Student> content = slice.getContent();

int pageNumber = slice.getNumber();
int pageSize = slice.getSize();
int elementsInCurrentSlice = slice.getNumberOfElements();

boolean first = slice.isFirst();
boolean last = slice.isLast();
boolean hasNext = slice.hasNext();
boolean hasPrevious = slice.hasPrevious();
```

It does not provide:

```
getTotalElements()
getTotalPages()
```

`Page` versus `Slice`

Requirement	`Page<T>`	`Slice<T>`
Current content	Yes	Yes
Page number and size	Yes	Yes
Has next/previous	Yes	Yes
Total elements	Yes	No
Total pages	Yes	No

Coder Army

Requirement	`Page<T>`	`Slice<T>`
Count query commonly required	Yes	No

`Slice` can be more efficient when the client only needs to know whether more data is available.

```
Page
  → limited content query
  → often an additional count query

Slice
  → limited query
  → commonly fetches one extra row to determine hasNext
```

Use `Page` when the UI needs information such as “page 3 of 25.” Use `Slice` for experiences such as “load more” or infinite scrolling, where the exact total is unnecessary.

---

18. Complete Mental Model

When the application calls an inherited method:

```
studentRepository.findById(1L)
        ↓
Repository proxy receives the call
        ↓
SimpleJpaRepository handles the method
        ↓
EntityManager performs the JPA operation
        ↓
Hibernate checks the persistence context
        ↓
Hibernate executes SQL if required
        ↓
Database returns the result
```

When the application calls a derived query method:

```
studentRepository.findByEmail(email)
        ↓
```

Coder Army

```
Repository proxy receives the call
        ↓
Prepared repository query is selected
        ↓
Method arguments are bound
        ↓
EntityManager executes the query
        ↓
Hibernate generates and executes SQL
        ↓
Results are mapped to entities
```

Spring Data JPA therefore removes repetitive repository implementation code while continuing to use the same JPA and Hibernate machinery underneath.

Key Takeaways

1. JPA is a specification; Hibernate is a common JPA provider; Spring Data JPA is a repository abstraction built on top of JPA.
2. `JpaRepository<Student, Long>` captures both the entity type and its identifier type.
3. Spring creates a repository proxy at runtime; standard methods are primarily handled by `SimpleJpaRepository`.
4. `save()` chooses between `persist()` and `merge()` according to whether the entity is considered new.
5. Managed entities are updated through dirty checking, even without an explicit `save()` call.
6. Derived query methods are effective while their names remain readable.
7. JPQL works with entities and relationships; native SQL works with tables and columns.
8. Sorting should be explicit whenever order matters.
9. `Page` provides totals and commonly needs a count query; `Slice` omits totals and can be more efficient.
10. Spring Data JPA reduces boilerplate, but correct transactions, mappings, fetching, SQL design, and database performance still matter.

Coder Army

Introduction to Transactions

1. Database Transaction

A **database transaction** is a group of database operations that the database treats as one logical unit of work.

Consider a bank transfer. The initial state is:

```
Account A = ₹10,000
Account B = ₹5,000
```

To transfer ₹1,000 from Account A to Account B, the application executes two updates:

```sql
UPDATE account
SET balance = 9000
WHERE id = 1;

UPDATE account
SET balance = 6000
WHERE id = 2;
```

Technically, these are two separate SQL statements. From the business perspective, however, they represent one operation:

```
Transfer ₹1,000 from Account A to Account B
```

The complete operation should move the database from one valid state to another:

```
Before the transaction:

Account A = ₹10,000
Account B = ₹5,000
Total     = ₹15,000
```

Coder Army

```
After the transaction:

Account A = ₹9,000
Account B = ₹6,000
Total     = ₹15,000
```

A transaction is therefore not defined by the number of SQL statements. It is defined by the business operation whose steps must succeed or fail together.

> A transaction is a unit of work that must be completed fully or not applied at all.

---

2. ACID Properties

ACID describes the guarantees required for reliable transaction processing:

- **A — Atomicity**
- **C — Consistency**
- **I — Isolation**
- **D — Durability**

2.1 Atomicity

Atomicity means that all operations inside a transaction succeed together or all of them are discarded together.

In the transfer example:

```
Debit Account A
Credit Account B
```

Both operations must succeed. If Account A is debited but Account B cannot be credited, the debit must also be rolled back.

```
Both balances change, or neither balance changes.
```

Coder Army

Without atomicity, a failure between the two updates could cause money to disappear.

2.2 Consistency

Consistency means that a transaction should move the database from one valid state to another valid state.

Suppose the business rule is:

```
An account balance cannot be negative.
```

The current balance is:

```
Account A = ₹500
```

The application attempts to transfer ₹1,000. If the operation leaves the account in this state:

```
Account A = -₹500
```

the business rule has been violated and the database is now in an invalid state.

Consistency depends on correctly defined rules and their enforcement through application logic, database constraints, or both. A transaction does not invent these rules; it ensures that the committed result does not contain only part of the intended operation.

In the transfer example, consistency can include rules such as:

- Account balances must remain valid.
- The total amount of money must not change because of an internal transfer.
- The sender and receiver must both exist.

2.3 Isolation

Isolation deals with transactions executing concurrently.

Coder Army

Suppose Account A contains ₹10,000 and two transactions execute at approximately the same time:

```
Transaction T1: Withdraw ₹1,000
Transaction T2: Withdraw ₹8,000
```

Without sufficient isolation, both transactions may read the same initial balance:

```
T1 reads ₹10,000
T2 reads ₹10,000
```

They then calculate and write different values:

```
T1 writes ₹9,000
T2 writes ₹2,000
```

Depending on the execution order, one update may overwrite the other. The final balance may fail to reflect both withdrawals correctly.

Isolation answers the following question:

> What is one transaction allowed to observe while another transaction is still executing?

Isolation does not necessarily mean that only one transaction can execute at a time. Databases allow concurrency, but they control how concurrent operations interact.

2.4 Durability

Durability means that once a transaction has committed successfully, its changes should survive later failures.

Suppose the database confirms:

```
Transfer completed successfully.
```

Coder Army

Immediately afterward:

- The application crashes.
- The database process restarts.
- The server loses power.

After recovery, the committed transfer should still exist.

> Commit is a promise that the database has accepted the transaction as a permanent result.

---

3. ACID as One Combined Guarantee

The four ACID properties are not unrelated definitions. Together, they describe whether a database can safely execute a business operation despite failures and concurrency.

For the bank transfer:

Property	Guarantee
Atomicity	Both balances change, or neither changes.
Consistency	Money is not created or destroyed, and all business rules remain valid.
Isolation	Concurrent operations do not corrupt the transfer.
Durability	Once committed, the transfer survives subsequent failures.

Together, ACID answers one central question:

> Can the database safely move from one valid state to another despite failures and concurrent activity?

---

4. Transaction Boundary

A **transaction boundary** defines where a transaction begins and where it ends.

In Spring, a service method can represent the boundary of a business transaction:

CoderArmy

```
@Transactional
public void transfer(Long fromId, Long toId, BigDecimal amoun
t) {
    debitAccount(fromId, amount);
    creditAccount(toId, amount);
    recordTransfer(fromId, toId, amount);
}
```

Conceptually, the execution is:

```
Begin transaction
    ↓
Debit sender
    ↓
Credit receiver
    ↓
Record transfer
    ↓
Commit transaction
```

If an eligible failure occurs before successful completion:

```
Begin transaction
    ↓
Perform database operations
    ↓
Exception
    ↓
Rollback transaction
```

The transaction normally begins before the annotated method executes. It commits after the method completes successfully or rolls back when a qualifying exception leaves the method.

---

5. Begin, Commit and Rollback

5.1 Begin

Coder Army

`BEGIN` starts a database transaction.

At the JDBC level, manual transaction control usually begins by disabling autocommit:

```
connection.setAutoCommit(false);
```

With autocommit disabled, individual SQL statements are not committed automatically. The application must explicitly commit or roll back the transaction.

5.2 Commit

Commit means:

> Accept the transaction's changes as its final result.

At the JDBC level:

```
connection.commit();
```

After a successful commit, the changes become durable according to the guarantees of the database.

5.3 Rollback

Rollback means:

> Discard the changes made by the current uncommitted transaction.

At the JDBC level:

```
connection.rollback();
```

Rollback does not mean manually executing another `UPDATE` statement to restore every previous value. The database maintains the information required to undo or discard the uncommitted work.

Coder Army

6. Transaction Layers: JDBC, JPA, Hibernate and Spring

JDBC, JPA, Hibernate and Spring transactions are often presented as four different transaction systems. In a typical Spring application, they are better understood as different APIs and abstraction layers that control or coordinate the same underlying database transaction.

```
Service Method
    ↓
Spring Transaction Management
    ↓
JPA EntityManager / Hibernate Session
    ↓
JDBC Connection
    ↓
Database Transaction
```

6.1 JDBC Transaction

JDBC provides direct, low-level transaction control through a database connection:

```java
Connection connection = dataSource.getConnection();

try {
    connection.setAutoCommit(false);

    // Execute SQL operations

    connection.commit();
} catch (Exception exception) {
    connection.rollback();
    throw exception;
} finally {
    connection.close();
}
```

Coder Army

The developer is responsible for:

- Obtaining the connection.
- Disabling autocommit.
- Committing on success.
- Rolling back on failure.
- Closing the connection safely.
- Preserving the original exception correctly.

6.2 JPA Transaction

JPA provides transaction control through its persistence API, commonly using `EntityTransaction` in resource-local environments:

```
EntityTransaction transaction = entityManager.getTransaction
();

try {
    transaction.begin();

    // Persistence operations

    transaction.commit();
} catch (Exception exception) {
    transaction.rollback();
    throw exception;
}
```

JPA hides direct JDBC connection handling, but the application still contains transaction boilerplate when transactions are managed manually.

6.3 Hibernate Transaction

Hibernate exposes its own native transaction API through `Session` and `Transaction`:

Coder Army

```
Transaction transaction = session.beginTransaction();

try {
    // Hibernate operations
    transaction.commit();
} catch (Exception exception) {
    transaction.rollback();
    throw exception;
}
```

When Hibernate is used as the JPA provider, JPA operations are ultimately implemented by Hibernate, which then interacts with JDBC and the database.

6.4 Spring-Managed Transaction

Spring removes transaction-management boilerplate from business methods.

```
@Transactional
public void transfer(Long fromId, Long toId, BigDecimal amoun
t) {
    debitAccount(fromId, amount);
    creditAccount(toId, amount);
    recordTransfer(fromId, toId, amount);
}
```

The business method contains only business logic. Spring handles the surrounding transaction workflow:

```
Open or obtain required resources
Begin transaction
Execute business method
Commit on success
Roll back on eligible failure
Release resources
```

Spring does not replace the database transaction. It manages the lower-level transaction API on behalf of the application.

Coder Army

7. Spring's `@Transactional`

In a Spring Boot application, the commonly used annotation is:

```
import org.springframework.transaction.annotation.Transaction
al;
```

It tells Spring that the method or class should execute within a Spring-managed transaction.

7.1 Method-Level Transaction

Annotate a specific method when only selected operations require transaction management:

```
@Service
public class AccountService {

    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal a
mount) {
        // Business operations
    }
}
```

Here, the transaction applies to `transfer()`.

7.2 Class-Level Transaction

Annotate the class when all relevant public methods should use transactional behavior:

```
@Service
@Transactional
public class AccountService {

    public void transfer(...) {
```

```
    }

    public void closeAccount(...) {
    }

    public AccountResponse getAccount(...) {
    }
}
```

The class-level annotation provides a default transactional configuration for the class. A method-level annotation can provide more specific configuration for an individual method.

Transaction boundaries are generally placed in the **service layer** because a service method represents a complete business use case and may coordinate multiple repository operations.

---

8. Complete Mental Model

When Spring executes a transactional service method, the process can be understood as follows:

```
Caller invokes service method
    ↓
Spring starts or joins a transaction
    ↓
Repository operations use resources participating in that transaction
    ↓
Method completes successfully → Commit

                    or

Method fails with a rollback-eligible exception → Rollback
```

The key idea is:

> `@Transactional` defines the boundary of a business operation, while Spring coordinates the underlying persistence and database transaction APIs needed to execute that operation safely.

Coder Army

Coder Army

9. Key Takeaways

- A transaction groups operations that must behave as one logical unit.
- A business operation may require one SQL statement or many SQL statements.
- Atomicity ensures that partial work is not committed.
- Consistency protects valid rules and invariants.
- Isolation controls interaction between concurrent transactions.
- Durability preserves committed results after failures.
- Begin, commit and rollback define the transaction lifecycle.
- JDBC, JPA and Hibernate expose different transaction APIs over the persistence stack.
- Spring coordinates transaction management and removes repetitive boilerplate.
- `@Transactional` is most commonly applied at the service layer around a complete business operation.

Coder Army

Transactions: Propagation and Isolation in Spring

Spring's `@Transactional` annotation defines how a method participates in a transaction and how that transaction should behave.

```
@Transactional(
        propagation = Propagation.REQUIRED,
        isolation = Isolation.READ_COMMITTED,
        timeout = 5,
        readOnly = false,
        rollbackFor = IOException.class
)
```

The most important attributes are:

Attribute	Responsibility
`propagation`	Decides whether the method joins, creates, suspends, or rejects a transaction
`isolation`	Controls how concurrent transactions can observe one another's changes
`timeout`	Sets the maximum time allowed for the transaction
`readOnly`	Provides a hint that the transaction will only read data
`rollbackFor`	Adds exception types that should cause rollback

> `@Transactional` is normally applied through a Spring-generated proxy. Therefore, the method must be invoked through that proxy for transactional interception to occur.

Default `@Transactional` Behaviour

Writing:

Coder Army

```
@Transactional
public void performOperation() {
    // Business operations
}
```

uses the following defaults:

Setting	Default
Propagation	`Propagation.REQUIRED`
Isolation	`Isolation.DEFAULT`
Read-only	`false`
Timeout	Default of the underlying transaction infrastructure
Rollback	`RuntimeException` and `Error`

Default Propagation: `REQUIRED`

```
If a transaction exists → join it
If no transaction exists → create one
```

Default Isolation: `DEFAULT`

`Isolation.DEFAULT` tells Spring to use the default isolation level of the underlying transaction infrastructure, which is commonly determined by the database.

Default Read-Write Behaviour

```
readOnly = false
```

The transaction is treated as a normal read-write transaction.

Default Timeout

Spring uses the default timeout of the transaction infrastructure. This does not mean that a timeout can never occur. Related timeout rules may still come from

the transaction manager, database, connection pool, JDBC driver, query, or lock acquisition settings.

Coder Army

Commit and Rollback Rules

When Does a Transaction Commit?

A transaction normally commits when all three conditions are satisfied:

1. The transactional method completes normally.
2. The transaction has not been marked as rollback-only.
3. The actual commit operation succeeds.

Return values such as the following do not represent transaction failure:

```
false
null
Optional.empty()
ResponseEntity.badRequest()
```

They are ordinary Java return values. Spring does not inspect their business meaning to decide whether it should roll back.

Rollback for Runtime Exceptions

By default, an unchecked exception causes rollback when it escapes the transactional method.

```
@Transactional
public void transfer() {
    debit();
    throw new IllegalStateException("Transfer failed");
}
```

```
RuntimeException escapes the method
               ↓
```

Coder Army

```
Default rollback rule matches
                ↓
Transaction rolls back
```

Common examples include:

- `IllegalArgumentException`
- `IllegalStateException`
- `NullPointerException`
- `DataIntegrityViolationException`
- Custom subclasses of `RuntimeException`

Rollback for `Error`

An `Error` also triggers rollback by default. Examples include `OutOfMemoryError`, `StackOverflowError`, `AssertionError`, and `LinkageError`.

An `Error` usually represents a serious JVM or environment failure rather than an ordinary business problem, but it still matches Spring's default rollback policy.

Checked Exception Behaviour

A checked exception does not trigger rollback under Spring's traditional default policy.

```java
@Transactional
public void transfer() throws TransferException {
    debit();
    throw new TransferException("Transfer failed");
}
```

```java
public class TransferException extends Exception {
}
```

The default flow is:

Coder Army

```
Checked exception escapes the method
                ↓
Default rollback rule does not match
                ↓
Spring proceeds towards commit
```

This does not mean that checked exceptions are harmless. It only describes the default rollback rule.

Configuring `rollbackFor`

Use `rollbackFor` when a checked exception should cause rollback:

```
@Transactional(rollbackFor = TransferException.class)
public void transfer() throws TransferException {
    debit();
    throw new TransferException("Transfer failed");
}
```

Multiple exception types can be configured:

```
@Transactional(
        rollbackFor = {
                TransferException.class,
                IOException.class
        }
)
public void transfer() {
    // Business operations
}
```

Transaction Propagation

Propagation answers one central question:

Coder Army

> What should this transactional method do when it is called with or without an existing transaction?

Propagation Modes at a Glance

Propagation mode	If a transaction exists	If no transaction exists
`REQUIRED`	Join it	Create a transaction
`REQUIRES_NEW`	Suspend it and create a new transaction	Create a transaction
`SUPPORTS`	Join it	Run without a transaction
`MANDATORY`	Join it	Throw an exception
`NOT_SUPPORTED`	Suspend it and run without a transaction	Run without a transaction
`NEVER`	Throw an exception	Run without a transaction
`NESTED`	Create a savepoint inside it	Create a transaction

Transaction Suspension

Suspending a transaction does not temporarily commit it. It means that Spring pauses its association with the current transaction and its resources while another scope executes.

The suspended transaction has neither committed nor rolled back. It resumes after the inner scope completes.

Propagation modes that may suspend an existing transaction are:

- `REQUIRES_NEW`: suspends the outer transaction and starts a new transaction.
- `NOT_SUPPORTED`: suspends the outer transaction and runs without a transaction.

`Propagation.REQUIRED`

`REQUIRED` is Spring's default propagation mode.

```
@Transactional
public void placeOrder() {
}
```

Coder Army

is equivalent to:

```
@Transactional(propagation = Propagation.REQUIRED)
public void placeOrder() {
}
```

Its rule is simple:

```
If a transaction exists → join it
If no transaction exists → create one
```

Joining an Existing Transaction

Consider an outer method that calls three transactional services:

```
@Transactional
public void checkout() {
    orderService.createOrder();
    inventoryService.reserveStock();
    paymentService.createPaymentRecord();
}
```

If all three inner methods use `REQUIRED`, they participate in the same physical transaction:

```
Transaction A
├── createOrder()
├── reserveStock()
└── createPaymentRecord()
```

They normally share the same:

- Physical database transaction
- JDBC connection
- Transaction-bound `EntityManager`

Coder Army

- Persistence context
- Final commit or rollback decision

The inner methods create separate logical transaction scopes, but they do not create independent physical transactions.

Inner Transaction Settings

Suppose the outer method declares:

```
@Transactional(
        isolation = Isolation.READ_COMMITTED,
        timeout = 30
)
public void checkout() {
    inventoryService.reserveStock();
}
```

The inner method declares:

```
@Transactional(
        propagation = Propagation.REQUIRED,
        isolation = Isolation.SERIALIZABLE,
        timeout = 5
)
public void reserveStock() {
}
```

The inner method joins an already-running physical transaction. It therefore cannot normally replace that transaction's characteristics.

The effective physical transaction continues with the outer settings:

```
Isolation → READ_COMMITTED
Timeout   → 30 seconds
```

Coder Army

Outer Failure After Inner Success

```
@Transactional
public void placeOrder() {
    orderRepository.save(order);
    inventoryService.reserveStock();

    throw new IllegalStateException(
            "Failure after inventory reservation"
    );
}
```

Even if `reserveStock()` completes successfully, both operations roll back because they belong to the same physical transaction. An inner `REQUIRED` method cannot commit independently.

`UnexpectedRollbackException`

A subtle problem occurs when an inner `REQUIRED` method fails but the outer method catches the exception:

```
@Transactional
public void placeOrder() {
    orderRepository.save(new Order());

    try {
        inventoryService.reserveStock();
    } catch (InsufficientStockException exception) {
        System.out.println("Stock reservation failed");
    }

    System.out.println("Order method completed normally");
}
```

The execution flow is:

Coder Army

```
1. The outer method starts Transaction A.
2. reserveStock() joins Transaction A.
3. reserveStock() throws a RuntimeException.
4. Its transactional interceptor marks Transaction A as rollback-only.
5. The outer method catches the Java exception.
6. The outer method completes normally and attempts to commit.
7. Transaction A cannot commit because it is already rollback-only.
8. Spring rolls it back and throws UnexpectedRollbackException.
```

Catching the original exception prevents it from propagating further in Java. It does not remove the rollback-only state already placed on the shared transaction.

Demo: One Shared Transaction

```java
@Transactional
public void outer() {
    firstRepository.save(...);
    innerService.required();
    throw new RuntimeException();
}
```

```java
@Transactional
public void required() {
    secondRepository.save(...);
}
```

Both inserts roll back because both methods participate in the same transaction.

When to Use `REQUIRED`

`REQUIRED` is appropriate when all participating operations represent one business unit of work and must share one final outcome.

Typical examples include:

- Placing an order
- Transferring money

Coder Army

- Registering a student
- Creating an invoice
- Updating employee information
- Reserving inventory
- Cancelling a booking

`Propagation.REQUIRES_NEW`

`REQUIRES_NEW` always executes the method inside a new physical transaction.

```
If an outer transaction exists:
    Suspend it

Create a separate transaction

After the inner transaction completes:
    Resume the outer transaction
```

Execution Flow

```java
@Service
@RequiredArgsConstructor
public class OrderService {

    private final AuditService auditService;

    @Transactional
    public void placeOrder() {
        // Transaction A
        auditService.recordAttempt();
        // Transaction A resumes
    }
}
```

Coder Army

```java
@Service
@RequiredArgsConstructor
public class AuditService {

    private final AuditRepository auditRepository;

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void recordAttempt() {
        auditRepository.save(new AuditRecord());
    }
}
```

The flow is:

```
1. placeOrder() starts Transaction A.
2. recordAttempt() is invoked through the AuditService proxy.
3. Spring suspends Transaction A.
4. Spring starts Transaction B.
5. recordAttempt() executes inside Transaction B.
6. Transaction B commits or rolls back.
7. Spring releases Transaction B's resources.
8. Spring resumes Transaction A.
```

Separate Physical Resources

In a typical single-database JPA application, the transactions use separate resources:

```
Outer Transaction A
├── EntityManager A
├── Persistence Context A
└── JDBC Connection A

Inner Transaction B
├── EntityManager B
├── Persistence Context B
└── JDBC Connection B
```

Coder Army

Because the transactions are independent, the inner transaction may define its own isolation level, timeout, read-only status, and rollback rules.

Independent Commit and Rollback

Inner Commits, Outer Rolls Back

```
@Transactional
public void outer() {
    orderRepository.save(...);
    auditService.saveAudit();

    throw new RuntimeException();
}
```

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void saveAudit() {
    auditRepository.save(...);
}
```

Expected result:

```
Audit transaction → commits
Order transaction → rolls back
```

The audit remains committed because it completed in a separate physical transaction.

Inner Rolls Back, Outer Continues

```
@Transactional
public void outer() {
    orderRepository.save(...);

    try {
```

Coder Army

```
            auditService.saveAuditAndFail();
        } catch (RuntimeException exception) {
            System.out.println("Audit failed");
        }

        order.markCompleted();
    }
```

The inner transaction can roll back without marking the outer transaction as rollback-only. If the exception is caught and the outer transaction remains valid, the outer transaction can still commit.

If the exception is not caught, it reaches the outer transactional boundary. The outer transaction may then roll back because it received a `RuntimeException`, not because the two physical transactions share rollback state.

Visibility of Outer Uncommitted Data

Suppose Transaction A inserts a new order but has not committed it:

```
Transaction A:
INSERT order id = 101
Status: uncommitted
```

Transaction B starts through `REQUIRES_NEW` and attempts to load that order:

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void recordOrderAudit(Long orderId) {
    Order order = orderRepository.findById(orderId)
            .orElseThrow();
}
```

Transaction B generally cannot assume that it can see Transaction A's uncommitted insert. An independent transaction should not depend casually on uncommitted state owned by the suspended outer transaction.

Common Use Case: Audit Logging

Coder Army

`REQUIRES_NEW` is commonly used when an audit record must be preserved even if the main business transaction fails. It should be used deliberately because it needs an additional physical transaction and typically another database connection while the outer resources remain suspended.

Other Propagation Modes

`Propagation.SUPPORTS`

```
If a transaction exists → join it
If no transaction exists → run without a transaction
```

`SUPPORTS` means that a transaction is welcome but not required.

It can be suitable for:

- Read helper methods
- Calculations that may use transaction-bound entities
- Methods called both inside and outside larger workflows
- Infrastructure helpers that do not require atomic writes

Its behaviour can change depending on the caller, so it should not be used for a write operation that requires a guaranteed transactional boundary.

`Propagation.MANDATORY`

```
If a transaction exists → join it
If no transaction exists → throw an exception
```

`MANDATORY` creates a clear contract: the caller must establish the transaction.

For example, stock reservation may be designed only as a building block of a larger operation such as placing an order or replacing an order item. It should never execute independently.

Coder Army

```java
@Transactional(propagation = Propagation.MANDATORY)
public void reserveStock() {
}
```

Calling this method without an active transaction causes an exception. Calling it from a transactional service makes it join the existing transaction.

`MANDATORY` versus `REQUIRED`

```
REQUIRED  → join an existing transaction or create one
MANDATORY → join an existing transaction or fail
```

Use `REQUIRED` when the method can represent an independent business operation. Use `MANDATORY` when it should only be part of a transaction defined by its caller.

`Propagation.NOT_SUPPORTED`

```
If a transaction exists → suspend it and run without a transaction
If no transaction exists → run without a transaction
```

The method always executes non-transactionally.

`Propagation.NEVER`

```
If a transaction exists → throw an exception
If no transaction exists → run without a transaction
```

`NEVER` establishes the opposite contract to `MANDATORY`: the method must not execute inside a transaction.

`Propagation.NESTED`

`NESTED` normally does not create a second independent physical transaction. It uses the existing physical transaction and creates a database savepoint.

Coder Army

Savepoint

Suppose Transaction A has completed two operations:

```
Operation 1
Operation 2
SAVEPOINT S1
Operation 3
Operation 4
```

If a failure occurs after `S1`, the transaction can execute:

```
ROLLBACK TO SAVEPOINT S1;
```

This discards Operations 3 and 4 while retaining Operations 1 and 2. The outer transaction remains active and still owns the final commit or rollback decision.

If no outer transaction exists, `NESTED` behaves like `REQUIRED` and creates a new transaction because there is no existing transaction in which to create a savepoint.

> `NESTED` requires support from the chosen transaction manager and underlying resource. It is commonly associated with JDBC savepoints and should not be assumed to work identically in every JPA configuration.

Transaction Isolation

Propagation controls the relationship between transactional method calls. Isolation controls the relationship between transactions executing concurrently.

Concurrent execution is unavoidable because many users may place orders, transfer money, book seats, update inventory, and read reports at the same time.

The database must balance two competing objectives:

```
Correctness ↔ Concurrency
```

Stronger isolation offers more protection but can increase blocking, retries, deadlocks, and resource usage. Weaker isolation allows more concurrency but exposes the application to more anomalies.

Coder Army

Concurrency Problems

Dirty Read

A dirty read occurs when one transaction reads a change made by another transaction that has not committed.

Initial state:

```
Account A = ₹10,000
```

Time	Transaction T1	Transaction T2
1	Begins	
2	Updates balance to ₹2,000	
3		Begins
4		Reads balance as ₹2,000
5	Rolls back	
6	Balance returns to ₹10,000	Continues using ₹2,000

T2 used a value that never became part of the committed database state.

Non-Repeatable Read

A non-repeatable read occurs when a transaction reads the same row twice and receives different committed values.

Initial state:

```
Account A = ₹10,000
```

Time	Transaction T1	Transaction T2
1	Begins	
2	Reads balance as ₹10,000	
3		Begins

Coder Army

Time	Transaction T1	Transaction T2
4		Updates balance to ₹8,000
5		Commits
6	Reads the same row again	
7	Receives ₹8,000	

The second value is not dirty because T2 committed it. The problem is that the same row did not produce a repeatable value inside T1.

Phantom Read

A phantom read concerns a set of rows selected by a condition rather than a change to one previously read row.

T1 runs:

```
SELECT *
FROM employee
WHERE salary > 100000;
```

It receives three employees. T2 then inserts a new matching row and commits:

```
INSERT INTO employee(name, salary)
VALUES ('Employee D', 120000);
```

When T1 repeats the original query, it receives four employees. The additional matching row is the phantom.

Time	Transaction T1	Transaction T2
1	Begins	
2	Finds 3 matching employees	
3		Inserts a matching employee
4		Commits
5	Repeats the query	
6	Finds 4 matching employees	

Coder Army

How Databases Implement Isolation

Databases do not necessarily lock an entire table for every transaction. Depending on the database and isolation level, they may use:

- Row and range locks
- Shared and exclusive locks
- Multi-Version Concurrency Control (MVCC)
- Consistent snapshots
- Predicate locking
- Conflict detection
- Transaction aborts and retries

The same isolation level may therefore have different performance characteristics and edge-case behaviour across database systems.

Spring Isolation Levels

Spring provides the following options:

```
Isolation.DEFAULT
Isolation.READ_UNCOMMITTED
Isolation.READ_COMMITTED
Isolation.REPEATABLE_READ
Isolation.SERIALIZABLE
```

`Isolation.DEFAULT`

```
@Transactional(isolation = Isolation.DEFAULT)
public void performOperation() {
}
```

Spring does not request a different isolation level. It uses the default of the underlying transaction infrastructure.

Coder Army

Common database defaults include:

```
MySQL InnoDB → REPEATABLE READ
PostgreSQL   → READ COMMITTED
```

`DEFAULT` is not an additional database isolation level created by Spring.

`Isolation.READ_UNCOMMITTED`

```
@Transactional(isolation = Isolation.READ_UNCOMMITTED)
public void generateReport() {
}
```

This is the weakest standard isolation level. A transaction may observe another transaction's uncommitted changes.

```
Dirty read          → Possible
Non-repeatable read → Possible
Phantom read        → Possible
```

It is rarely suitable for business operations because decisions may be based on data that is later rolled back.

`Isolation.READ_COMMITTED`

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void performOperation() {
}
```

A transaction cannot read another transaction's uncommitted changes.

```
Dirty read          → Prevented
Non-repeatable read → Possible
Phantom read        → Possible
```

Coder Army

Each statement may see the latest data committed before that statement begins, so two reads inside one transaction can still produce different results.

`Isolation.REPEATABLE_READ`

```
@Transactional(isolation = Isolation.REPEATABLE_READ)
public void performOperation() {
}
```

In the standard isolation model:

```
Dirty read          → Prevented
Non-repeatable read → Prevented
Phantom read        → May occur
```

Actual behaviour depends on the database implementation. For example, an MVCC-based database may prevent some phantom-read scenarios even though the SQL standard permits them at this level.

`Isolation.SERIALIZABLE`

```
@Transactional(isolation = Isolation.SERIALIZABLE)
public void performCriticalOperation() {
}
```

`SERIALIZABLE` is the strongest standard isolation level. Concurrent transactions must produce a result equivalent to some valid serial order.

For two transactions, the result should be equivalent to either:

```
T1 completes, then T2 completes
```

or:

```
T2 completes, then T1 completes
```

Coder Army

The transactions may still overlap physically. The database can enforce serializable behaviour using locks, MVCC, predicate or range locks, conflict detection, or transaction abortion and retry.

```
Dirty read            → Prevented
Non-repeatable read   → Prevented
Phantom read          → Prevented
Serialization anomaly → Prevented
```

Isolation-Level Comparison

Isolation level	Dirty read	Non-repeatable read	Phantom read
`READ_UNCOMMITTED`	Possible	Possible	Possible
`READ_COMMITTED`	Prevented	Possible	Possible
`REPEATABLE_READ`	Prevented	Prevented	Standard permits it; the actual database may prevent it
`SERIALIZABLE`	Prevented	Prevented	Prevented

Choosing the Right Isolation Level

Do not choose the strongest isolation level automatically. Choose the weakest level that still preserves the correctness rules of the business operation.

Consider:

- What incorrect outcome can occur if data changes during the transaction?
- Does the method read the same row or query result more than once?
- Can two users update the same business data concurrently?
- Can the operation tolerate a retry?
- What are the database's actual guarantees at the chosen level?
- Would optimistic or pessimistic locking solve the specific problem more precisely?

Coder Army

Final Summary

Propagation

- `REQUIRED` joins an existing transaction or creates one.
- `REQUIRES_NEW` creates an independent physical transaction and suspends the outer one.
- `SUPPORTS` uses a transaction only when one already exists.
- `MANDATORY` requires the caller to provide a transaction.
- `NOT_SUPPORTED` always runs without a transaction.
- `NEVER` rejects execution inside a transaction.
- `NESTED` uses a savepoint inside an existing transaction when supported.

Isolation

- Isolation controls what concurrent transactions can observe.
- Dirty reads expose uncommitted data.
- Non-repeatable reads change the value of the same row between reads.
- Phantom reads change the set of rows returned by the same condition.
- Stronger isolation improves protection but can reduce concurrency.
- Actual behaviour depends on both the selected isolation level and the database implementation.

Rollback

- Normal completion generally leads to commit.
- `RuntimeException` and `Error` trigger rollback by default.
- Checked exceptions do not trigger rollback by default.
- Use `rollbackFor` when a checked exception must roll back the transaction.

- Catching an exception does not clear a rollback-only marker already placed on a shared transaction.

Coder Army

Coder Army

Spring Security Introduction

1. Security Foundations

What Application Security Means

Suppose we have an endpoint:

```
DELETE /api/users/57
```

From a normal Spring MVC perspective, we think:

```
Request
   ↓
Controller
   ↓
Service
   ↓
Repository
   ↓
Database
```

From a security perspective, several questions must be answered **before the business operation is allowed**:

```
Who is sending this request?

Has that identity been verified?

Is that identity allowed to delete users?

Is that identity allowed to delete this particular user?

Has the request been manipulated?

Was the request genuinely created by the user?
```

Coder Army

```
Could an attacker repeat or forge this request?
```

Application security means protecting the application's valuable resources and operations from unauthorized access, modification, destruction, disclosure, or misuse.

It is not limited to protecting data.

An application may need to protect:

- information;
- business operations;
- user identities;
- system availability;
- financial transactions;
- administrative capabilities;
- secrets and credentials;
- application reputation.

For example, consider a banking application.

```
Asset: Account balance
Asset: Transaction history
Asset: Transfer-money operation
Asset: Customer identity
Asset: Bank availability
Asset: Administrative access
```

An attacker may not need to steal information. Making the bank unavailable through excessive requests is also a security problem.

Security is not one feature

Security is a collection of controls.

Coder Army

```
Authentication
Authorization
Input validation
Session protection
Password protection
CSRF protection
CORS policies
Secure communication
Rate limiting
Audit logging
Error handling
Secret management
```

Spring Security primarily helps with:

```
Authentication
Authorization
Security-context management
Session security
Protection against several common web attacks
Integration with security protocols
```

It does not magically make all application code secure. For example, Spring Security will not automatically fix

- SQL injection caused by unsafe queries;
- insecure file uploads;
- weak business rules;
- secrets committed to Git;
- vulnerable dependencies;
- incorrect database permissions;
- sensitive information written into logs.

Therefore:

Coder Army

> Security is not something added only to the login page. It must exist wherever an untrusted actor can affect a valuable resource.

Assets, Threats, and Trust Boundaries

Before deciding how to secure an application, we need to understand three concepts:

```
Assets
Threats
Trust boundaries
```

Assets

An asset is anything valuable that should be protected.

Data assets

```
Passwords
Email addresses
Phone numbers
Health records
Payment information
Source code
API keys
Business documents
```

Operational assets

```
Creating an order
Transferring money
Approving a refund
Changing another user's password
```

Coder Army

```
Deleting an account
Publishing content
```

Infrastructure assets

```
Database
Server
Cloud account
Message broker
File storage
Deployment pipeline
```

Security assets

```
Session IDs
JWT signing keys
Encryption keys
OAuth client secrets
Password-reset tokens
CSRF tokens
```

Availability as an asset

Even when an attacker cannot access data, they might attempt to prevent legitimate users from accessing the application.

```
Thousands of expensive requests
Repeated login attempts
Resource exhaustion
Database connection exhaustion
```

Therefore, availability itself is an asset.

Coder Army

Threats

A threat is something that could harm an asset.

Examples:

```
An attacker stealing credentials
A malicious employee reading restricted data
A bot repeatedly guessing passwords
A user changing another user's order
A script flooding an endpoint
A browser sending an unwanted authenticated request
```

A threat is not the same thing as a vulnerability.

Vulnerability

A vulnerability is a weakness that makes an attack possible.

```
Threat:
An attacker wants to access an administrator account.

Vulnerability:
The application allows unlimited password attempts.

Attack:
The attacker runs an automated password-guessing tool.

Impact:
The administrator account is compromised.
```

A useful relationship is:

```
Threat + Vulnerability → Possible Attack
```

Trust Boundaries

Coder Army

A trust boundary is a point where information moves from one trust level to another.

Consider:

```
Browser
   ↓
Internet
   ↓
Spring Boot application
   ↓
Database
```

Every transition represents a possible trust boundary.

```
Browser → Server
Server → Database
Service A → Service B
Mobile application → API gateway
Public network → Internal network
External identity provider → Application
```

Why is the browser untrusted?

A common beginner assumption is:

> "The frontend only shows the delete button to admins, so normal users cannot delete anything."

But users control their browsers.

They can:

- change JavaScript;
- modify HTML;
- use Postman;

Coder Army

- use `curl`;
- manipulate request parameters;
- manually call hidden endpoints;
- replay old requests.

Suppose the frontend contains:

```
if (user.role==="ADMIN") {
    showDeleteButton();
}
```

This improves the interface, but it does not provide security.

A non-admin user can still send:

```
DELETE /api/users/57
```

Therefore, authorization must be enforced on the server.

Trust-boundary rule

Whenever data crosses a trust boundary, ask:

```
Who produced this data?
Can it be modified?
Should it be authenticated?
Should it be authorized?
Should it be validated?
Should it be encrypted?
```

Authentication

Authentication answers:

> Who are you, and what evidence proves that claim?

Suppose someone enters a company office and says:

Coder Army

> “I am Aditya.”

That statement is only an **identity claim**.

The security guard may ask for an employee card.

```
Claimed identity: Aditya
Evidence: Employee card
Verification: Card is checked
Result: Identity accepted or rejected
```

That process is authentication.

Authentication is a process

Authentication is the process of verifying an identity claim.

Username and password example

```
Claim:
"My username is aditya."

Credential:
"My password is secret123."

Verification:
Compare the supplied password with the stored encoded password.

Result:
Authentication succeeds or fails.
```

The username usually identifies the account. The password provides evidence that the requester is entitled to use that account.

Authentication factors

Authentication evidence commonly falls into three broad categories.

Something you know

Coder Army

```
Password
PIN
Security answer
```

Something you have

```
Smart card
Authenticator application
Private cryptographic key
```

Something you are

```
Fingerprint
Face
Iris
```

Multi-Factor Authentication (MFA)

Using two credentials from the same category is not necessarily strong multi-factor authentication.

```
Password + Authenticator application OTP
```

uses knowledge and possession.

In Spring Security, this authenticated representation is usually stored in an `Authentication` object.

Authorization

Authorization answers:

> What is this identity allowed to do?

Coder Army

Authentication establishes identity.

Authorization evaluates access.

Suppose the security guard has verified that a person is Aditya. That does not mean Aditya can enter every room in the building.

```
Authentication:
You are Aditya.

Authorization:
Aditya may enter the training room,
but may not enter the finance department.
```

Authorization as a decision

A proper authorization decision usually needs four pieces of information:

```
Subject
Action
Resource
Context
```

Subject

Who is trying to perform the action?

```
User 51
Administrator
Manager
Service account
Anonymous visitor
```

Action

What are they trying to do?

Coder Army

```
Read
Create
Update
Delete
Approve
Transfer
Download
```

Resource

What are they trying to access?

```
Order 101
Employee record 57
Bank account 9001
Admin dashboard
Course video
```

Context

Under what conditions is the request happening?

```
Is the user the owner?
Is the request inside business hours?
Is the account active?
Is the amount below the approval limit?
Has MFA been completed?
```

Therefore, authorization can be thought of as:

```
Can subject S
perform action A
```

```
on resource R
under context C?
```

Example:

```
Can employee 27
update
leave request 91
when employee 27 owns leave request 91?
```

Public access is also an authorization decision

Suppose an endpoint is public:

```
GET /api/courses/public
```

The application may allow everyone to access it.

That is still an authorization policy:

```
Authorization rule:
Permit all requests.
```

Authentication usually comes before authorization

A typical flow is:

```
Request
   ↓
Extract credentials
   ↓
Authenticate identity
   ↓
Determine authorities
   ↓
Evaluate authorization rule
   ↓
Allow or deny
```

Coder Army

CoderArmy

Authentication Failures vs. Security Attacks

An authentication failure and a security attack are not necessarily the same thing.

Authentication failure

An authentication failure means the presented evidence could not establish an acceptable identity.

Examples

```
Wrong password
Unknown username
Expired token
Invalid token signature
Malformed Authorization header
Disabled account
Expired account
Expired session
Incorrect OTP
```

Security attack

An attack involves intentional attempts to defeat, misuse, bypass, or exploit security controls.

Brute-force attack

An attacker tries many passwords for one account.

```
aditya / password1
aditya / password2
aditya / password3
aditya / password4
```

Each individual request produces an authentication failure.

The pattern across thousands of requests reveals an attack.

Coder Army

Credential stuffing

An attacker uses username-password combinations stolen from another website.

```
Stolen credential:
user@example.com / reusedPassword123
```

The password may even be correct.

Therefore, authentication could succeed even though the request is malicious.

This demonstrates an important point:

> Successful authentication does not prove that the human behind the request is legitimate. It proves only that acceptable credentials were presented.

Additional controls may be needed:

```
MFA
Anomaly detection
Login notifications
Device verification
Rate limiting
Credential-breach detection
```

Do not reveal unnecessary failure details

Suppose the application responds:

```
Username exists, but password is incorrect.
```

An attacker now knows that the account exists.

A safer external response is often:

```
Invalid username or password.
```

Coder Army

Internally, the application may log more precise information:

```
User not found
Password mismatch
Account disabled
Token expired
```

The external message and internal diagnostic information need not be identical.

---

2. Spring Security Identity Model

Spring Security's identity model uses the following core vocabulary:

The important concepts are:

```
Identity
Credentials
Principal
Authorities
Roles
Permissions
Authentication
SecurityContext
SecurityContextHolder
```

They are related, but they are not interchangeable.

Identity

Identity means the entity whose actions the system wants to recognize.

It may represent:

```
A human user
An administrator
A customer
Another application
A microservice
```

```
A scheduled job
An IoT device
```

An identity is broader than a database row.

Identity versus account

An account is a stored application record.

```
users table
-----------------------------------------------
id | username | password_hash | enabled
42 | aditya   | ...           | true
```

Identity is the conceptual subject.

Account is one way the application represents and manages that identity.

A person may have:

- multiple accounts;
- an employee identity;
- a customer identity;
- an administrator identity;

Credentials

Credentials are evidence used to prove an identity claim.

Examples:

```
Password
OTP
Private key
Certificate
API key
Bearer Token
```

Principal

Coder Army

Coder Army

The principal is the identity currently represented inside the security system.

Authorities

An authority represents a capability or high-level security attribute granted to a principal.

Examples:

```
COURSE_READ
COURSE_WRITE
USER_DELETE
PAYMENT_REFUND
ROLE_ADMIN
SCOPE_orders.read
```

In Spring Security, authorities are represented through `GrantedAuthority` objects and are stored inside the current `Authentication`. Spring describes authorities as high-level permissions such as roles and scopes.

Granted versus claimed authority

An authority should be trusted only when it has been assigned by a trusted authentication process.

A client should not be able to send:

```
{
  "username":"aditya",
  "role":"ADMIN"
}
```

and become an administrator merely because it claimed that role.

The role or authority must be:

- loaded from a trusted database;
- derived from a verified token;
- supplied by a trusted identity provider;

Coder Army

- assigned by a trusted `AuthenticationProvider`.

The `Authentication` API explicitly treats its authorities as values granted by the `AuthenticationManager`, not arbitrary client claims.

Roles

A role is a named grouping of responsibilities or access privileges.

Examples:

```
USER
INSTRUCTOR
MANAGER
ADMIN
SUPER_ADMIN
```

A role is usually broader than an individual permission.

```
ROLE_INSTRUCTOR
     ├── COURSE_READ
     ├── COURSE_CREATE
     ├── COURSE_UPDATE
     └── STUDENT_PROGRESS_READ
```

`Authentication`

`Authentication` is one of the most important Spring Security interfaces.

It has two different roles.

Role 1: Authentication request

Before credentials have been verified, an `Authentication` object can represent an authentication attempt.

Example:

```
UsernamePasswordAuthenticationToken
```

Coder Army

```
principal = "aditya"
credentials = "raw-password"
authorities = []
authenticated = false
```

This object means:

> Someone claims to be Aditya and has supplied this password.

It does not yet mean the claim is accepted.

Role 2: Authenticated principal

After successful verification, another `Authentication` object can represent the accepted identity.

```
principal = UserDetails("aditya")
credentials = null
authorities = [ROLE_USER, COURSE_READ]
authenticated = true
```

This object means:

> The authentication system has accepted this principal and granted these authorities.

Spring Security formally defines `Authentication` as both the token for an authentication request and the representation of an authenticated principal after processing.

Important `Authentication` information

Conceptually, it contains:

```
public interface Authentication {

    Object getPrincipal();

    Object getCredentials();
```

```
    Collection<? extends GrantedAuthority> getAuthorities();

    Object getDetails();

    boolean isAuthenticated();

    String getName();
}
```

`principal`

Represents the identity.

Before authentication:

```
"aditya"
```

After username-password authentication:

```
UserDetails object
```

`credentials`

The evidence proving the identity.

```
Password
Bearer token
Certificate
API key
```

Credentials may be removed after successful authentication.

`authorities`

The capabilities granted to the principal.

Coder Army

Coder Army

```
ROLE_USER
COURSE_READ
COURSE_WRITE
```

These are used later during authorization.

`details`

Additional information about the authentication request.

Depending on the mechanism, this may contain information such as:

```
Remote address
Session identifier
```

`isAuthenticated()`

This indicates whether the authentication token has been accepted as authenticated.

Conceptually:

```
Before verification:
isAuthenticated = false

After trusted verification:
isAuthenticated = true
```

Application code should not simply mark arbitrary client-created authentication objects as trusted. The authenticated result should normally come from the trusted authentication infrastructure.

Authentication transformation

```
Unauthenticated Authentication
┌──────────────────────────────────────┐
| principal   = "aditya"               |
| credentials = "raw-password"         |
```

Coder Army

```
| authorities = []                    |
| authenticated = false               |
└─────────────────────────────────────┘

                  |
                  | AuthenticationManager
                  | verifies credentials
                  ▼
Authenticated Authentication

┌─────────────────────────────────────┐
| principal   = UserDetails           |
| credentials = null                  |
| authorities = [ROLE_USER]           |
| authenticated = true                |
└─────────────────────────────────────┘
```

That authenticated object becomes the security identity used for the current request.

`SecurityContext`

The `SecurityContext` is a container that holds the current `Authentication`.

Conceptually:

```
public interface SecurityContext {

    Authentication getAuthentication();

    void setAuthentication(Authentication authentication);
}
```

Spring Security defines the `SecurityContext` as the object containing the current `Authentication`.

Why not store `Authentication` directly everywhere?

A wrapper provides a consistent place for request security state.

Instead of passing the current user manually:

Coder Army

```
courseService.createCourse(courseRequest,currentUser);
```

through every method, Spring Security establishes a request-associated security context that can be accessed where appropriate.

The security context can also be:

- loaded at the beginning of a request;
- updated after authentication;
- stored for future requests;
- cleared after request completion.

`SecurityContextHolder`

The `SecurityContextHolder` provides access to the current `SecurityContext`.

```
SecurityContext context =
        SecurityContextHolder.getContext();

Authentication authentication =
        context.getAuthentication();
```

Then:

```
String username = authentication.getName();

Object principal = authentication.getPrincipal();

Collection<? extends GrantedAuthority> authorities =
        authentication.getAuthorities();
```

Spring Security describes `SecurityContextHolder` as the central location where the framework stores details about the currently authenticated user.

Why is a holder necessary?

Consider this request:

Coder Army

```
HTTP request
    ↓
Security filters authenticate user
    ↓
Controller
    ↓
Service
    ↓
Method-security check
```

The authentication filter learns the user's identity.

Later components need access to that identity.

```
Authentication filter:
I authenticated Aditya.

Authorization component:
Who is the current user?

Controller:
Who submitted this request?

Audit service:
Which user performed this action?
```

The `SecurityContextHolder` gives these components access to the current security context without manually passing the user through every method.

Default `ThreadLocal` strategy

In servlet applications, one request is commonly processed by one thread at a time.

Spring Security therefore uses a `ThreadLocal` strategy by default.

Conceptually:

Coder Army

```
Thread handling request 1
    SecurityContext → Aditya

Thread handling request 2
    SecurityContext → Rohit
```

Each thread sees its own current context.

Spring Security documents that the default `SecurityContextHolder` strategy uses `ThreadLocal`, making the context available to methods running on the same thread. It also clears the context after request processing to prevent one request's identity from leaking into another request handled by a reused server thread.

Why clearing the context matters

Application servers reuse threads.

Without clearing:

```
Request 1
Thread 7
User = ADMIN
```

Then later:

```
Request 2
Same Thread 7
User = anonymous
```

If the old context remained, Request 2 might incorrectly inherit the administrator identity.

Therefore, the security context must be established and cleared around each request.

```
Request begins
    ↓
Load or create SecurityContext
```

Coder Army

```
    ↓
Process security and application logic
    ↓
Clear SecurityContextHolder
    ↓
Thread returns to pool
```

Complete identity-model relationship

```
Identity
    │
    │ claimed using
    ▼
Credentials
    │
    │ verified by authentication system
    ▼
Authentication
    ├── Principal
    ├── Credentials
    ├── Authorities
    └── Authentication status
            │
            │ stored inside
            ▼
SecurityContext
            │
            │ accessed through
            ▼
SecurityContextHolder
```

3. Authentication State

HTTP creates an important problem.

Coder Army

By default, HTTP requests are independent.

```
Request 1
POST /login

Request 2
GET /profile

Request 3
GET /orders

Request 4
POST /logout
```

The server must decide:

> How does Request 2 know that the user authenticated during Request 1?

There are two broad models:

```
Stateful authentication
Stateless authentication
```

Stateful Authentication

In stateful authentication, the server retains authentication-related state between requests.

A common example is session-based authentication.

```
User logs in once
      ↓
Server verifies credentials
      ↓
Server stores authentication in session
      ↓
Client receives session identifier
```

```
      ↓
Client sends session identifier on later requests
      ↓
Server retrieves stored authentication
```

First request: login

```
POST /login
Content-Type: application/x-www-form-urlencoded

username=aditya&password=secret
```

The application verifies the credentials.

```
Authentication successful
```

It then stores information representing the authenticated user in server-side session state.

Conceptually:

```
Session ID: ABC123

Session data:
    SecurityContext
        Authentication
            principal = aditya
            authorities = [ROLE_USER]
```

The response sends the session identifier to the browser, commonly using a cookie.

```
Set-Cookie: JSESSIONID=ABC123
```

Subsequent request

Coder Army

Coder Army

The browser sends:

```
GET /profile
Cookie: JSESSIONID=ABC123
```

The server uses the session identifier to retrieve the session.

```
ABC123
   ↓
Session
   ↓
SecurityContext
   ↓
Authentication for Aditya
```

The user does not need to send the password again.

Stateful does not mean all data is in the cookie

A common misunderstanding is:

> "The browser cookie contains the complete logged-in user."

In ordinary server-side session authentication, the cookie normally contains only a session identifier.

```
Browser:
JSESSIONID=ABC123

Server:
ABC123 → session containing SecurityContext
```

The important authentication state is held server-side.

The cookie allows the server to locate that state.

Where can session state be stored?

Session state may be kept in:

Coder Army

```
Application memory
Distributed cache
Redis
Database
External session store
```

Advantages of stateful authentication

User authenticates once

The user does not need to resend the original password on every request.

Immediate server-side invalidation

The server can remove or invalidate the session.

```
Delete session ABC123
```

Further requests using that session identifier fail.

Server controls session state

The application can record:

```
Last access time
Session expiry
Concurrent sessions
SecurityContext
Additional session attributes
```

Costs of stateful authentication

Server storage

Coder Army

Each active session consumes server-side resources.

Distributed-system complexity

When several servers handle requests, they need:

- shared session storage;
- sticky sessions;
- or another consistent session strategy.

Cookie does not necessarily mean session

Cookies may store many things:

```
Session identifier
Language preference
Theme preference
Analytics identifier
CSRF token
Authentication token
```

A token-based system can also place an access token inside a cookie.

Therefore:

> Seeing a cookie does not automatically mean the application uses server-side session authentication.

Session cookie as a bearer credential

Suppose an attacker steals:

```
JSESSIONID=ABC123
```

The attacker may not know the user's password.

But the session identifier may be enough to retrieve the authenticated session.

In practice, possession of the session cookie can grant access, so it must be protected like a credential.

Coder Army

Important protections include:

```
HTTPS
Secure cookie attribute
HttpOnly
SameSite policy
Session expiration
Session invalidation
Session ID rotation
```

Session fixation

Imagine an attacker somehow causes a victim to use a session identifier known to the attacker.

```
Attacker knows session ID = XYZ
Victim logs in using session XYZ
Session XYZ becomes authenticated
Attacker reuses XYZ
```

To reduce this risk, the application can change the session identifier when authentication succeeds.

```
Before login:
Session = XYZ

After login:
Session = NEW-789
```

Spring Security includes session-fixation protection and can change the session ID after authentication.

Stateless Authentication

In stateless authentication, the server does not rely on a stored login session to identify the user on later requests.

Coder Army

Each request carries the information needed for authentication.

```
Request 1
Authorization: Bearer TOKEN

Request 2
Authorization: Bearer TOKEN

Request 3
Authorization: Bearer TOKEN
```

For every request:

```
Extract credential
      ↓
Validate credential
      ↓
Create Authentication
      ↓
Place it in SecurityContext
      ↓
Authorize request
      ↓
Clear request context
```

The next request starts again.

Stateless does not mean no data exists anywhere

A common incorrect statement is:

> "A stateless system stores no state."

The application may still store:

```
Users
Passwords
```

Coder Army

```
Roles
Permissions
Signing keys
Refresh tokens
Revoked token records
Audit logs
Rate-limit counters
Business data
```

Stateless authentication specifically means:

> The server does not depend on a previously stored HTTP login session to reconstruct the user's authentication for the next request.

Tokens and Per-Request Authentication

A token is a credential presented by the client.

Example:

```
GET /api/profile
Authorization: Bearer eyJhbGciOi...
```

The server extracts the token and validates it.

```
Check signature
Check expiry
Check issuer
Check audience
Read subject
Determine authorities
```

It may then create:

```
Authentication
    principal = user 42
```

```
    authorities = [SCOPE_profile.read]
    authenticated = true
```

That authentication is used for only the current request unless the application explicitly persists it.

Self-Contained Token vs. Opaque Token

Self-contained token

A self-contained token contains signed information that the server can validate locally.

A JWT may contain claims such as:

```
{
  "sub":"aditya",
  "scope":"course.read course.write",
  "iat":1785980000,
  "exp":1785983600
}
```

The server validates the token using a trusted key.

It does not need to look up an HTTP session merely to identify the user.

Opaque token

An opaque token may look like:

```
a8f2b9137c45d...
```

The token itself does not reveal useful information to the application.

It is usually a random identifier.

The application may ask an authorization server or token store:

```
Is this token active?
Which user does it represent?
```

Coder Army

```
What scopes does it contain?
```

Opaque-token authentication can still be stateless from the application's HTTP-session perspective, even though a token service maintains token state.

```
Conceptually:

a8f2b9137c45d...
    ↓
Token database
    ↓
user = aditya
scope = course.read
active = true
expiry = 11:00
```

Bearer token

A bearer token effectively means:

> Whoever possesses this token may present it.

The word `Bearer` means that possession is enough.

The server generally does not require the client to prove that it is the original recipient of the token.

It is similar to cash:

```
Whoever possesses the cash can spend it.
```

```
The same observation applies to an ordinary
session cookie. Someone possessing a valid
JSESSIONID may be able to act as the user.
In that practical sense, a session identifier
behaves like a bearer credential.
```

Coder Army

Coder Army

Therefore, bearer tokens must be protected from:

```
Logging
URL leakage
Browser storage attacks
Network interception
Accidental sharing
Source-code commits
```

A valid token is not safe merely because it is a JWT.

Security still depends on:

```
Strong signing keys
Correct signature verification
Expiry validation
Issuer validation
Audience validation
Secure transport
Safe client storage
Appropriate scopes
```

An opaque token can be a bearer token

This is valid:

```
Authorization: Bearer a8f2b9137c45d...
```

Here:

```
Bearer = possession-based usage
Opaque = unreadable/reference format
```

The taxonomy

Coder Army

```
Access token
├── Opaque
│   └── Often used as a bearer token
│
└── Self-contained, often JWT
    └── Also often used as a bearer token
```

What Is a JWT?

JWT stands for:

```
JSON Web Token
```

A JWT is a **token format**.

Per-request flow

```
HTTP Request
    │
    │ Authorization: Bearer TOKEN
    ▼
Authentication filter
    │
    │ extracts token
    ▼
Token validator/provider
    │
    │ validates token
    ▼
Authenticated Authentication
    │
    ▼
SecurityContext
    │
```

Coder Army

```
    ▼
SecurityContextHolder
    |
    ▼
Authorization
    |
    ▼
Controller
```

At the end:

```
SecurityContextHolder cleared
```

On the next request, the token must be presented and processed again.

---

4. Authentication Mechanism vs. Session Policy

This is a fundamental distinction in Spring Security.

Authentication mechanism

The authentication mechanism answers:

> How are credentials obtained and verified?

Session policy

The session policy answers:

> After authentication succeeds, should the resulting SecurityContext be persisted in an HTTP session?

Form login describes how username and password are submitted.

```
POST /login

username=aditya&password=secret
```

Coder Army

The application may then:

Option A: Create a session

```
Form credentials
    ↓
Authenticate
    ↓
Store SecurityContext in session
    ↓
Send session cookie
```

This is the traditional stateful web application.

Option B: Issue a token

```
Form or JSON credentials
    ↓
Authenticate
    ↓
Generate access token
    ↓
Do not create login session
```

This can form part of a stateless API design.

Similarly, HTTP Basic is a credential transport mechanism

HTTP Basic sends credentials through the `Authorization` header.

```
Authorization: Basic YWRpdHlhOnNlY3JldA==
```

Usually the client presents the header repeatedly.

That naturally fits per-request authentication.

Coder Army

But the authentication mechanism and session policy are still conceptually separate. An application could authenticate Basic credentials and persist the resulting context unless configured not to do so.

Therefore:

```
HTTP Basic
    = credential transport mechanism

STATELESS
    = session persistence decision
```

JWT is not automatically stateless

A JWT is a token format.

Statelessness depends on how the application uses it.

Stateless use

```
JWT sent every request
JWT validated every request
No HTTP-session authentication used
```

Session-backed use

```
JWT validated once
Authentication stored in HTTP session
Later requests use session cookie
```

The second design uses a JWT during authentication but becomes session-based afterward.

Therefore:

> Token format does not determine session policy.

Conversely, a non-JWT opaque token can be processed on every request without creating an HTTP session.

Coder Army

Cookie does not automatically mean stateful

A stateless application may store a bearer token in a secure cookie.

```
Cookie: access_token=eyJhbGciOi...
```

On every request:

```
Read token from cookie
Validate token
Create Authentication
No server-side login session
```

This is still stateless with respect to authentication persistence, even though a cookie transports the token.

However, automatically submitted cookies introduce browser-specific security concerns such as CSRF, which must be addressed separately.

Stateful Request Lifecycle

Initial authentication

```
POST /login
    ↓
Username and password extracted
    ↓
Credentials authenticated
    ↓
Authentication created
    ↓
SecurityContext created
    ↓
SecurityContext saved in HttpSession
```

```
    ↓
Session cookie returned
```

Later request

```
GET /profile
Cookie: JSESSIONID=ABC123
    ↓
Session loaded
    ↓
SecurityContext loaded
    ↓
Authentication placed in SecurityContextHolder
    ↓
Authorization performed
    ↓
Controller executes
    ↓
SecurityContextHolder cleared
```

Stateless Request Lifecycle

```
GET /profile
Authorization: Bearer TOKEN
    ↓
Token extracted
    ↓
Token authenticated
    ↓
Authentication created
    ↓
SecurityContext created
    ↓
Authentication placed in SecurityContextHolder
    ↓
```

Coder Army

Coder Army

```
Authorization performed
    ↓
Controller executes
    ↓
SecurityContextHolder cleared
    ↓
No authentication saved in HTTP session
```

Next request:

```
Start the authentication process again
using the credential carried by that request.
```

---

5. Spring Boot Security Auto-Configuration

Spring Boot's job is not to implement authentication itself.

Spring Security provides the security framework.

Spring Boot's job is to observe what is available and create a sensible initial configuration.

A simplified mental model is:

```
Dependency added
      ↓
Spring Security classes found on classpath
      ↓
Spring Boot security auto-configuration activates
      ↓
Security beans are created
      ↓
Security filter chain is registered
      ↓
Every matching request passes through security
```

Spring Boot's servlet security defaults are primarily coordinated through:

Coder Army

```
SecurityAutoConfiguration

UserDetailsServiceAutoConfiguration
```

`SecurityAutoConfiguration` provides the default web-security setup, while `UserDetailsServiceAutoConfiguration` provides the default username-and-password user configuration.

Spring Boot is solving two different problems:

```
Problem 1:
Which requests should be protected,
and which authentication mechanisms should be available ?

Problem 2:
Which user can authenticate,
and where will that user be loaded from?
```

Those problems correspond conceptually to:

```
Default SecurityFilterChain

        +

Default UserDetailsService
```

The default security configuration

The effective default security configuration can be understood approximately as:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

    http.authorizeHttpRequests(
                authorize -> authorize
```

```
                                    .anyRequest()
                                    .authenticated()
            )
        .formLogin(Customizer.withDefaults())
        .httpBasic(Customizer.withDefaults());

    return http.build();
}
```

This means:

```
anyRequest().authenticated()
```

Every request requires authentication.

```
formLogin()
```

Username-password authentication through an HTML login form is available.

```
httpBasic()
```

Username-password authentication through the HTTP `Authorization` header is also available.

Spring Security documents this as the default filter-chain configuration: every request requires authentication, while both form login and HTTP Basic are enabled.

Auto-configuration is conditional

Spring Boot auto-configuration follows an important rule:

> Provide defaults only when the developer has not provided their own configuration.

Suppose Spring Boot asks:

Coder Army

Coder Army

```
Has the developer created a SecurityFilterChain bean?
```

If the answer is no:

```
Create the default web-security configuration.
```

If the answer is yes:

```
Back off and let the developer control web security.
```

Therefore, when we later write:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

    // Our configuration

}
```

Spring Boot stops supplying its default web-security rules.

But there is an important nuance:

> Creating a `SecurityFilterChain` bean replaces the default web-security configuration, but it does not automatically remove the generated default user.

To replace the generated user configuration as well, we normally provide one of the following:

```
UserDetailsService
AuthenticationProvider
AuthenticationManager
```

Spring Boot documents these as separate back-off conditions: a custom `SecurityFilterChain` replaces the default access configuration, while a custom user

Coder Army

or authentication component replaces the generated user setup.

So these are independent:

```
SecurityFilterChain
    → How requests are secured

UserDetailsService / AuthenticationProvider
    → How users are authenticated
```

---

6. Default User

Spring Boot needs at least one identity that can authenticate.

Without a user, the application would present a login page, but no credentials could ever succeed.

Therefore, Spring Boot creates a default in-memory user.

The default username is:

```
user
```

The user is stored in memory through a `UserDetailsService`, rather than being inserted into a database. Spring Boot's default user store contains one user, and the default username is `user`.

Conceptually:

```
InMemoryUserDetailsManager
└── UserDetails
    ├── username = "user"
    ├── password = generated password
    ├── authorities = ["ROLE_USER"]
    └── enabled = true
```

This user is not:

Coder Army

- saved in MySQL;
- saved through JPA;
- created by your controller;
- a permanent production account;
- related to a registration system.

It exists only because Spring Boot needs a development-time identity for testing its default security configuration.

What happens when we enter `user`?

The login request provides:

```
Principal claim:
user

Credential:
password entered in the login form
```

Spring Security then asks the configured user store:

```
Load the user whose username is "user".
```

The in-memory user store returns something conceptually like:

```
UserDetails
    username = user
    password = encoded stored password
    authorities = ROLE_USER
```

The submitted password is checked against the stored password representation.

If it matches:

```
Authentication successful
```

Spring Security creates an authenticated `Authentication` object:

```
Authentication
├── principal = UserDetails("user")
├── credentials = cleared
├── authorities = [ROLE_USER]
└── authenticated = true
```

That authentication is then placed in the `SecurityContext`.

Default role

The default user normally has:

```
ROLE_USER
```

Remember the distinction:

```
Role name:
USER

Granted authority:
ROLE_USER
```

For the default rule:

```
anyRequest().authenticated()
```

the actual role does not matter.

Spring Security is only asking:

```
Is there an accepted authenticated identity?
```

It is not yet asking:

Coder Army

Coder Army

```
Does the user have ROLE_ADMIN?
```

Therefore, the default user can access every endpoint after authentication because the default rule requires authentication, not a particular authority.

Configuring the default user through properties

For demonstration purposes, we can replace the generated credentials:

```
spring.security.user.name=aditya
spring.security.user.password=secret123
spring.security.user.roles=USER,ADMIN
```

Now the user becomes conceptually:

```
username = aditya
password = secret123
authorities = ROLE_USER, ROLE_ADMIN
```

7. Generated Password

When the application starts, the console displays something similar to:

```
Using generated security password:
78fa095d-3f4c-48b1-ad50-e24c31d5cf35

This generated password is for development use only.
Your security configuration must be updated before
running your application in production.
```

The generated value is the password that must be entered with:

```
Username: user
Password: generated value
```

Spring Boot generates a random default password and logs it at `WARN` level during startup. The documentation explicitly describes this password as development-only.

Coder Army

8. The CSRF `403 Forbidden` Surprise

Suppose a user successfully logs in and then sends:

```
POST /api/courses
```

They may receive:

```
403 Forbidden
```

They might conclude:

> "The user does not have permission."

But that is not necessarily the reason.

Spring Security enables CSRF protection by default for servlet web applications. Therefore, a state-changing request such as `POST`, `PUT`, `PATCH`, or `DELETE` may be rejected when it lacks the expected CSRF token.

The request may have:

```
Valid authenticated user
Valid ROLE_USER
Correct endpoint
```

but still fail because:

```
Required CSRF proof is missing
```

> **Key takeaway:** `401 Unauthorized` usually means acceptable authentication is missing or has failed. `403 Forbidden` means the request was understood but

> rejected. With Spring Security, a `403` on a state-changing request may be caused by a missing CSRF token—not only by insufficient roles or authorities.

Coder Army

Coder Army

Database Authentication and Password Security

From Default Authentication to Database Authentication

When the following dependency is added to a Spring Boot application, Spring Security automatically secures the application:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

By default, Spring Boot creates a user with the username `user`, prints a randomly generated password in the console, secures every endpoint, and provides form login and HTTP Basic authentication.

After successful authentication, Spring Security creates an `Authentication` object representing the currently authenticated user.

```
Authentication
-------------------------------
principal       = user
credentials     = protected/erased
authorities     = ROLE_USER
authenticated   = true
```

The important question is:

> Where does Spring Security obtain the information required to verify a user?

---

1. Authentication Requires a Source of Truth

Suppose a client submits:

Coder Army

```
username = aditya
password = secret123
```

Spring Security cannot automatically know whether these credentials are correct. It needs trusted information against which it can validate the claim.

```
User-submitted information          Trusted information
---------------------------         ---------------------------
username = aditya                   username = aditya
password = secret123                passwordHash = ...
                                    roles = ...
                                    accountStatus = ...
```

Authentication can therefore be understood as:

```
Claimed identity + credential
                ↓
      Validation against
                ↓
   Trusted user information
```

The trusted information can live in application memory, a relational database, MongoDB, LDAP, an external identity provider, or a separate authentication service.

In this application, the source of truth will be a MySQL database:

```
Spring Boot Application
           ↓
     MySQL Database
```

---

2. Why the Default User Is Not Enough

Coder Army

Spring Security's generated user is useful for learning and testing, but it is not a practical user-management system.

A real learning platform could have thousands of users. Every user may have their own identity, password, roles, and account status. This information must survive application restarts, server restarts, deployments, and application scaling.

The users must therefore be stored in persistent storage such as a database.

---

3. Why Spring Security Does Not Directly Understand Our Database

Every application has a different database design.

One application may use a `users` table with fields such as `id`, `email`, `password`, `phone`, `created_at`, and `enabled`. Another may use an `employees` table with `employee_id`, `corporate_email`, `credential_hash`, `department`, and `active`. Other systems may call the same concept `customers`, `members`, or `accounts` and may use a completely different database technology.

Spring Security cannot require every application to use the same table name, column names, or database design. Instead, it defines common security abstractions.

One of the most important abstractions is:

```
UserDetails
```

`UserDetails` represents the user information Spring Security needs during username-and-password authentication.

```
Application-specific database model
                ↓
            Adapter
                ↓
    Spring Security's UserDetails
```

This keeps Spring Security independent of the application's database structure.

Coder Army

4. Identity and Credentials

Authentication must answer two questions:

```
Who are you?          Can you prove it?
     ↓                       ↓
 Identity                Credential
```

For username-and-password authentication:

```
username or email  → identity
password           → credential
```

Username versus email

An application does not have to use a traditional username as the login identifier. It may use a username, email address, employee ID, phone number, or any other value that uniquely identifies an account.

For example, an application can use:

```
findByEmail(email);
```

instead of:

```
findByUsername(username);
```

Spring Security APIs use historical names such as `loadUserByUsername(...)` and `getUsername()`. However, the value can still represent an email address if that is the application's login design.

> The login identifier must uniquely identify exactly one account.

The following data would make authentication ambiguous:

```
ID    EMAIL
1     aditya@gmail.com
```

```
2     aditya@gmail.com
```

The login identifier should therefore have a unique database constraint. This example uses `username` to keep the terminology aligned with Spring Security's interfaces.

---

6. Designing the Authentication Database

The absolute minimum information required for username-and-password authentication is:

```
User
├── username
└── password
```

A minimal table could be:

```
users
---------------------------------
id          BIGINT PRIMARY KEY
username    VARCHAR UNIQUE
password    VARCHAR
```

In a realistic application, account status and roles are also useful.

---

7. Understanding `UserDetails`

A simplified view of Spring Security's `UserDetails` interface is:

```
public interface UserDetails {

    String getUsername();

    String getPassword();
```

Coder Army

```
    Collection<? extends GrantedAuthority> getAuthorities();

    boolean isAccountNonExpired();

    boolean isAccountNonLocked();

    boolean isCredentialsNonExpired();

    boolean isEnabled();
}
```

It contains three main categories of information:

1. Identity — `getUsername()`
2. Credential — `getPassword()`
3. Authorities and account status

Enabled and disabled accounts

An account may need to remain in the database even when it should no longer be allowed to authenticate.

```
User exists
     but
User cannot authenticate
```

For this requirement, the user table can contain an `enabled` field:

```
users
---------------------------------
id | username | password | enabled
```

Example:

Coder Army

Coder Army

```
1 | aditya | $2a$... | true
2 | rohit  | $2a$... | false
```

An enabled user may continue through authentication. A disabled user is not eligible for successful authentication.

Disabled versus locked

```
Disabled
→ The account is inactive.

Locked
→ The account exists, but authentication is temporarily block
ed.
```

An account might be locked after repeated failed login attempts without being permanently disabled.

Account expiration

`isAccountNonExpired()` represents whether the account itself is still valid. For example, a temporary employee's account may be valid only until the end of a contract.

Credential expiration

`isCredentialsNonExpired()` represents whether the user's credential is still valid. A company may, for example, require a password to be changed after 90 days.

```
Account expired
→ The account itself is no longer valid.

Credentials expired
→ The account may be valid, but its credential must be renewe
d.
```

Coder Army

> Spring Security provides hooks for these states, but every application does not need a separate database column for each one.

If an application does not support account expiration, its `UserDetails` implementation can simply return `true` from `isAccountNonExpired()`.

---

8. Initial `User` Entity

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    private boolean enabled = true;
}
```

The `unique = true` constraint is important because one username must identify only one account.

Domain `User` versus Spring Security `UserDetails`

The application's `User` entity and Spring Security's `UserDetails` abstraction belong to different layers.

```
Application/domain layer
```

Coder Army

```
User Entity
├── email
├── profile
├── createdAt
├── password
└── roles

          ↓ adapt

Spring Security layer

UserDetails
├── username
├── password
├── authorities
└── account status
```

The `User` entity models what the application wants to store. `UserDetails` models what Spring Security needs for authentication. They can be connected through an adapter without treating them as the same concept.

9. Introducing Roles

Authentication establishes who the user is. Authorization determines what that authenticated user is allowed to do.

A learning platform might define:

```
STUDENT  → View courses
TEACHER  → Create courses
ADMIN    → Delete users
```

A user may have one or more roles:

```
Aditya
├── ROLE_USER
```

```
└── ROLE_ADMIN

Rohit
└── ROLE_USER
```

Database structure

```
users
-----------------------------------------------
id | username | password | enabled

roles
-------------------
id | name

user_roles
--------------------
user_id | role_id
```

Example data:

```
users
1 | aditya | $2a$... | true
2 | rohit  | $2a$... | true

roles
1 | ROLE_USER
2 | ROLE_ADMIN

user_roles
1 | 1
1 | 2
2 | 1
```

Coder Army

The join-table entries mean that Aditya has both roles, while Rohit has only `ROLE_USER`.

Coder Army

10. JPA Entity Model for Roles

`Role` entity

```
@Entity
@Table(name = "roles")
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;
}
```

Updated `User` entity

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
```

Coder Army

```
    private String password;

    private boolean enabled = true;

    @ManyToMany
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles = new HashSet<>();
}
```

The mapping is unidirectional because the authentication use case needs `user.getRoles()` but does not currently require `role.getUsers()`.

A bidirectional relationship should be added only if the application genuinely needs navigation in both directions.

---

11. Repository Layer

The application needs a repository to retrieve users by their login identifier:

```
public interface UserRepository
        extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);
}
```

A role repository may also be useful during registration or role management:

```
public interface RoleRepository
        extends JpaRepository<Role, Long> {


```

```
    Optional<Role> findByName(String name);
}
```

At this stage, Spring Security still does not directly call `userRepository.findByUsername(...)`.

```
Database
   ↑
UserRepository
   ↑
Missing bridge
   ↑
Spring Security
```

The missing bridge is `UserDetailsService`.

`DaoAuthenticationProvider` uses `UserDetailsService` to load the username, encoded password, authorities, and account-status information required for authentication.

---

12. Why Raw Passwords Must Never Be Stored

Consider a registration request:

```
POST /register
Content-Type: application/json
```

```
{
  "username": "aditya",
  "password": "secret123"
}
```

The simplest implementation would save the password directly:

```
User user = new User();
user.setUsername(request.getUsername());
```

Coder Army

Coder Army

```
user.setPassword(request.getPassword());

userRepository.save(user);
```

The database would then contain:

```
id | username | password
------------------------------
1  | aditya   | secret123
```

This may work functionally, but it is extremely unsafe. If the database is compromised, every stored password is immediately exposed. The damage may extend beyond this application because users frequently reuse the same password on multiple services.

> An application should never intentionally store a user's raw password.

---

13. Why Password Encryption Is Not the Right Solution

Encryption is reversible:

```
Plaintext
   ↓ encryption + key
Ciphertext
   ↓ decryption + key
Plaintext
```

If passwords are encrypted, the application must keep a decryption key capable of recovering every original password. If an attacker obtains both the database and the key, all passwords can be recovered.

The more fundamental question is:

> Does the application ever need to recover a user's original password?

No. During login, the application only needs to determine whether the password entered now matches the password registered earlier.

Therefore, passwords need a one-way transformation rather than reversible encryption.

---

14. Password Hashing

Hashing changes the security model:

```
secret123
    ↓
One-way password function
    ↓
Encoded representation
```

The encoded representation is stored in the database. There should be no operation that converts it back into the original password.

During login, the system performs verification:

```
Entered raw password
          +
Stored encoded password
          ↓
Does the entered password correspond
to this stored representation?
          ↓
       true / false
```

Spring Security expresses this operation through:

```
passwordEncoder.matches(rawPassword, encodedPassword);
```

Coder Army

Coder Army

15. Why a Fast Hash Such as SHA-256 Is Not Enough

A general-purpose cryptographic hash such as SHA-256 is designed to be fast. Speed is valuable for many computing tasks, but it is dangerous for password storage.

If attackers steal a password-hash database, they can perform an offline guessing attack:

```
password
123456
qwerty
secret123
welcome123
...
```

For every guess, they calculate a hash and compare it with the stolen value. A very fast algorithm allows attackers to test an enormous number of guesses quickly.

Password-storage algorithms are deliberately designed to make every guess computationally expensive.

```
Fast general-purpose hash
→ Attackers can test guesses rapidly.

Adaptive password hash
→ Every guess requires more computation.
```

Spring Security supports adaptive one-way password functions such as BCrypt for this purpose.

---

16. BCrypt and `PasswordEncoder`

Spring Security provides `BCryptPasswordEncoder`, which implements the `PasswordEncoder` abstraction:

Coder Army

```
PasswordEncoder
       ↑
       | implements
BCryptPasswordEncoder
```

The encoder can be registered as a Spring bean:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Depending on the abstraction allows the application to change the concrete password-encoding strategy without rewriting all authentication code.

Core operations

```
String encoded = passwordEncoder.encode("secret123");
```

The result may look similar to:

```
$2a$10$...
```

The password can later be verified using:

```
passwordEncoder.matches("secret123", encoded); // true
```

An incorrect password produces:

```
passwordEncoder.matches("wrongPassword", encoded); // false
```

The application never needs to decrypt the stored value.

17. Salt and BCrypt

Coder Army

A stolen database can also be attacked using precomputed password-hash dictionaries, commonly associated with rainbow-table attacks.

```
password   → known hash 1
123456     → known hash 2
secret123  → known hash 3
admin123   → known hash 4
```

A salt adds unique random data to the password-hashing process.

```
Aditya
secret123 + randomSaltA
          ↓ BCrypt
       encodedValueA

Rohit
secret123 + randomSaltB
          ↓ BCrypt
       encodedValueB
```

Even though both users selected the same password, their stored BCrypt values are different.

BCrypt includes the salt and algorithm parameters inside the encoded value stored in the database. A separate salt column is therefore not required when using the standard BCrypt representation.

Practical demonstration

```
System.out.println(
    passwordEncoder.encode("secret123")
);

System.out.println(
```

```
    passwordEncoder.encode("secret123")
);
```

The two results should be different:

```
$2a$10$A......................
$2a$10$B......................
```

How does `matches()` work if every encoding is different?

`matches()` does not encode the raw password again with a new salt and compare the two strings:

```
passwordEncoder
    .encode(rawPassword)
    .equals(storedPassword); // Incorrect approach
```

That approach would fail because each new encoding normally uses a different random salt.

Instead, `matches()` reads the salt and other required parameters from the stored BCrypt value. It applies those parameters while verifying the submitted raw password.

```
passwordEncoder.matches(
    rawPassword,
    storedEncodedPassword
);
```

Spring Security hides the algorithm-specific verification details behind the `PasswordEncoder` interface.

---

18. Registration Flow from First Principles

Coder Army

Coder Army

The client sends:

```
POST /register
Content-Type: application/json
```

```
{
  "username": "aditya",
  "password": "secret123"
}
```

The registration flow is:

```
RegisterRequest
├── username = aditya
└── password = secret123
              ↓
       PasswordEncoder
              ↓
           BCrypt
              ↓
      Encoded password
              ↓
         User entity
              ↓
       UserRepository
              ↓
          Database
```

In short:

```
Raw password
     ↓
PasswordEncoder.encode()
     ↓
BCrypt-encoded password
```

Coder Army

```
      ↓
Database
```

> Password encoding occurs before the user entity is saved. The raw password must never be persisted.

---

19. Registration DTO

A simple request DTO can validate the incoming values:

```java
public class RegisterRequest {

    @NotBlank
    private String username;

    @NotBlank
    @Size(min = 8)
    private String password;
}
```

`@Size(min = 8)` demonstrates basic length validation. A production password policy may require additional rules, but password validation and password encoding solve different problems:

```
Validation
→ Decides whether the submitted password is acceptable.

Password encoding
→ Protects the password representation stored in the databas
e.
```

---

20. Registration Service

Coder Army

The service must check whether the username already exists, encode the raw password, create the entity, and then save it.

The repository can expose an existence query:

```
public interface UserRepository
        extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

    boolean existsByUsername(String username);
}
```

A minimal registration service can be written as:

```
@Service
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthService(
            UserRepository userRepository,
            PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    public User register(RegisterRequest request) {

        if (userRepository.existsByUsername(request.getUserna
me())) {
            throw new IllegalArgumentException(
                "Username is already registered"
            );
        }
```

Coder Army

```
        User user = new User();
        user.setUsername(request.getUsername());
        user.setPassword(
            passwordEncoder.encode(request.getPassword())
        );
        user.setEnabled(true);

        return userRepository.save(user);
    }
}
```

The critical line is:

```
user.setPassword(
    passwordEncoder.encode(request.getPassword())
);
```

The value received in the request is raw. The value saved in the database is BCrypt-encoded.

```
Request DTO              Database
------------------       -------------------------
secret123                $2a$10$...
raw password             encoded password
```

The raw password should exist only for the short time required to process the registration request. It should not be logged, returned in the response, or stored anywhere.

---

21. Complete Conceptual Picture

Registration and login use the password encoder differently.

During registration

Coder Army

```
Raw password
      ↓
PasswordEncoder.encode()
      ↓
Encoded password
      ↓
Database
```

22. Key Takeaways

- Authentication requires trusted user information from a source of truth.
- A database provides persistent storage for users, passwords, roles, and account status.
- Spring Security does not depend directly on an application-specific database structure.
- `UserDetails` represents the user information required by Spring Security.
- `UserDetailsService` forms the bridge between Spring Security and the application's repository layer.
- The login identifier must uniquely identify one account.
- The domain `User` entity and Spring Security's `UserDetails` are separate concepts.
- Raw passwords must never be stored.
- Password encryption is not appropriate because it is reversible.
- Fast general-purpose hashes are unsuitable for password storage.
- BCrypt is an adaptive, salted, one-way password-hashing algorithm.
- BCrypt stores the salt and required parameters inside its encoded output.
- `encode()` is used before saving a new password.
- `matches()` is used to verify a raw password against the stored encoded value.

- Registration must validate input, prevent duplicate identities, encode the password, and only then save the user.

Coder Army

Coder Army

Spring Security Database Authentication: Internals and Complete Login Flow

Spring Security performs authentication before the request reaches the controller. For database authentication, it connects the incoming username and password with our database through a set of filters, authentication abstractions, and user-loading contracts.

This chapter explains how those components fit together and how to implement the complete flow in a Spring Boot application.

---

1. Where Spring Security Works

Spring Security's servlet support is built on servlet filters.

```
HTTP Request
    ↓
Servlet Filters
    ↓
Spring Security Filters
    ↓
DispatcherServlet
    ↓
Controller
```

Security must run before the controller because the application first needs to determine:

- Who is making the request?
- Is the supplied identity valid?
- What roles or authorities does the user have?
- Is the user allowed to access the requested resource?

Coder Army

A simplified security-filter order looks like this:

```
Security context handling
        ↓
Exploit protection, such as CSRF
        ↓
Authentication filters
        ↓
Anonymous authentication
        ↓
Exception translation
        ↓
Authorization
```

The real filter chain can contain additional filters. The important principle is:

```
Protection → Authentication → Exception Handling → Authorization
```

Authentication must happen before authorization. Spring Security cannot decide whether a user has `ROLE_ADMIN` until it has first established who that user is.

---

2. `DelegatingFilterProxy`

`DelegatingFilterProxy` is a real servlet filter known to the servlet container. It does not implement all of Spring Security itself. Instead, it finds a filter bean managed by Spring and delegates the request to it.

Conceptually:

```java
public void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain chain
) {
    Filter springFilter = findFilterBeanInSpring();
    springFilter.doFilter(request, response, chain);
}
```

Coder Army

The connection is:

```
Tomcat
   ↓
DelegatingFilterProxy
   ↓
FilterChainProxy
```

> `DelegatingFilterProxy` connects the servlet-container lifecycle with Spring-managed filter beans.

---

3. `FilterChainProxy`

`FilterChainProxy` is the central filter of Spring Security's servlet architecture. It stands in front of one or more `SecurityFilterChain` objects.

```
FilterChainProxy
    ├── SecurityFilterChain for /api/**
    ├── SecurityFilterChain for /admin/**
    └── SecurityFilterChain for every other request
```

For every request, `FilterChainProxy` asks:

> Which `SecurityFilterChain` should process this request?

It then executes the filters belonging to the selected chain.

---

4. `SecurityFilterChain`

A `SecurityFilterChain` contains two things:

```
A request matcher
        +
An ordered list of security filters
```

Conceptually, the contract looks like this:

Coder Army

```
public interface SecurityFilterChain {

    boolean matches(HttpServletRequest request);

    List<Filter> getFilters();
}
```

For example:

```
SecurityFilterChain 1

Matcher:
    /api/**

Filters:
    SecurityContextHolderFilter
    BearerTokenAuthenticationFilter
    ExceptionTranslationFilter
    AuthorizationFilter
```

Another chain may be configured as:

```
SecurityFilterChain 2

Matcher:
    every request

Filters:
    SecurityContextHolderFilter
    CsrfFilter
    UsernamePasswordAuthenticationFilter
    BasicAuthenticationFilter
    AnonymousAuthenticationFilter
```

```
    ExceptionTranslationFilter
    AuthorizationFilter
```

The precise list depends on the features enabled through `HttpSecurity`.

Request matching

A `RequestMatcher` determines whether a chain applies to the current request. It can inspect properties such as:

- request path;
- HTTP method;
- headers;
- servlet path;
- parameters;
- dispatcher type.

For the request:

```
GET /api/users
```

the matcher `/api/**` returns `true`, while `/admin/**` returns `false`.

First matching chain wins

`FilterChainProxy` invokes only the first `SecurityFilterChain` that matches the request.

Suppose the chains are ordered as:

```
Chain 1: /api/**
Chain 2: /**
```

For `/api/users`, both patterns could match. Spring Security selects Chain 1 and stops searching. It does not combine the filters from both chains.

Coder Army

Coder Army

Therefore, chains should generally be ordered from the most specific matcher to the most general matcher.

```
1. /api/**
2. /admin/**
3. /**
```

If no chain matches, the request is not protected by a Spring Security filter chain. A fallback chain is therefore useful.

```
@Bean
SecurityFilterChain fallback(HttpSecurity http) throws Except
ion {
    http.authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated()
    );

    return http.build();
}
```

If no `securityMatcher(...)` is configured, the chain normally applies to every request.

---

5. `HttpSecurity`

`HttpSecurity` is a builder used to describe and construct a `SecurityFilterChain`.

```
http
    .csrf(Customizer.withDefaults())
    .formLogin(Customizer.withDefaults())
    .httpBasic(Customizer.withDefaults())
    .authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated()
    );
```

Each configuration method contributes to the final chain:

Coder Army

```
http.csrf(...)
    ↓
Configure CSRF protection

http.formLogin(...)
    ↓
Configure form-login filters and handlers

http.httpBasic(...)
    ↓
Configure HTTP Basic authentication

http.authorizeHttpRequests(...)
    ↓
Configure authorization rules
```

`HttpSecurity` describes:

- which security features are enabled;
- which filters are required;
- how those filters are configured;
- the order in which they execute;
- which authentication providers they use;
- which authorization rules apply.

Finally:

```
return http.build();
```

converts the builder configuration into a concrete `SecurityFilterChain`.

---

6. Authentication Filters

Different authentication mechanisms receive credentials in different forms.

Form login

Coder Army

```
POST /login
Content-Type: application/x-www-form-urlencoded

username=aditya&password=secret123
```

This request is handled by `UsernamePasswordAuthenticationFilter`.

HTTP Basic

```
Authorization: Basic YWRpdHlhOnNlY3JldDEyMw==
```

This request is handled by `BasicAuthenticationFilter`.

A filter does not authenticate every request

`UsernamePasswordAuthenticationFilter` normally looks for a form-login request such as:

```
POST /login
```

For an ordinary request such as:

```
GET /courses
```

the filter does not treat it as a new form-login attempt. It continues the chain.

```
chain.doFilter(request, response);
```

Therefore:

```
Authentication filter present in the chain
        ≠
Authentication attempt performed on every request
```

The filter first checks whether the request contains the authentication mechanism it understands.

7. `Authentication` and the Request Token

The HTTP request initially contains plain values:

```
username = aditya
password = secret123
```

Spring Security converts them into a standard authentication object. For username-and-password authentication, this is commonly a `UsernamePasswordAuthenticationToken`.

Before verification:

```
UsernamePasswordAuthenticationToken
├── principal = "aditya"
├── credentials = "secret123"
├── authorities = []
└── authenticated = false
```

At this stage, the token represents only a claim:

> Someone claims to be Aditya and presents `secret123` as proof.

It does not mean Spring Security has accepted the identity.

The unauthenticated token can be represented as:

```
UsernamePasswordAuthenticationToken token =
        UsernamePasswordAuthenticationToken.unauthenticated(
                username,
                password
        );
```

Here:

Coder Army

Coder Army

```
principal   = claimed identity
credentials = claimed proof
```

The authentication filter should not contain database or password-verification logic. Its main responsibilities are:

```
Extract credentials
        ↓
Create an Authentication request
        ↓
Send it to the authentication engine
```

---

8. `AuthenticationManager`

`AuthenticationManager` is the entry point into Spring Security's authentication engine.

Its central operation is:

```
public interface AuthenticationManager {

    Authentication authenticate(Authentication authenticatio
n)
            throws AuthenticationException;
}
```

It receives an unauthenticated request token and produces one of two outcomes:

```
Unauthenticated Authentication
        ↓
AuthenticationManager.authenticate(...)
        ↓
        ├── Success → Authenticated Authentication
        │
        └── Failure → AuthenticationException
```

Coder Army

`AuthenticationManager` is an interface. Its most common implementation is `ProviderManager`.

9. `ProviderManager` and `AuthenticationProvider`

An application can support multiple authentication mechanisms:

- database username and password;
- JWT;
- one-time passwords;
- LDAP;
- SAML;
- remember-me authentication;
- custom authentication mechanisms.

`ProviderManager` coordinates the available `AuthenticationProvider` objects.

```
ProviderManager
    ├── DaoAuthenticationProvider
    ├── JwtAuthenticationProvider
    ├── RememberMeAuthenticationProvider
    └── CustomOtpAuthenticationProvider
```

Each provider understands a particular authentication-token type. Conceptually, a provider exposes a check similar to:

```
boolean supports(Class<?> authenticationType);
```

For example:

```
DaoAuthenticationProvider
    → supports UsernamePasswordAuthenticationToken
```

Coder Army

```
JwtAuthenticationProvider
    → supports a JWT authentication token
```

When `ProviderManager` receives a `UsernamePasswordAuthenticationToken`, it finds a provider capable of authenticating that token.

```
AuthenticationManager
        ↓ coordinates
AuthenticationProvider
        ↓ performs mechanism-specific verification
```

For database-backed username-and-password authentication, the relevant provider is `DaoAuthenticationProvider`.

---

10. Connecting Our Database to Spring Security

Our application has its own `User` entity. Spring Security does not know its structure and should not depend directly on our persistence model.

Spring Security therefore uses two contracts:

```
UserDetails
    → Spring Security's view of one user

UserDetailsService
    → Strategy for finding that user
```

Our `User` entity

The application may care about fields such as:

```
id
username
password
email
firstName
lastName
```

```
profilePicture
createdAt
subscription
preferences
roles
```

Coder Army

Spring Security mainly needs:

- username;
- stored password;
- authorities;
- enabled state;
- account-expiration state;
- account-lock state;
- credential-expiration state.

---

11. `UserDetails`

`UserDetails` represents Spring Security's view of one user.

Conceptually:

```java
public interface UserDetails {

    String getUsername();

    String getPassword();

    Collection<? extends GrantedAuthority> getAuthorities();

    boolean isAccountNonExpired();

    boolean isAccountNonLocked();

    boolean isCredentialsNonExpired();
```

Coder Army

```
    boolean isEnabled();
}
```

Spring Security is effectively saying:

> Store users however you want, but provide an object that satisfies this contract.

Two different `User` classes

An application may contain:

```
com.coderarmy.entity.User
```

Spring Security also provides:

```
org.springframework.security.core.userdetails.User
```

They are not the same class.

```
Application User
    → Database entity and domain model

Spring Security User
    → Built-in UserDetails implementation
```

We do not have to use Spring Security's concrete `User` class. A custom adapter often makes the bridge easier to understand.

`CustomUserDetails`

```
public class CustomUserDetails implements UserDetails {

    private final User user;

    public CustomUserDetails(User user) {
```

Coder Army

```
        this.user = user;
    }

    @Override
    public String getUsername() {
        return user.getUsername();
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorit
ies() {
        return user.getRoles()
                .stream()
                .map(role -> new SimpleGrantedAuthority(role.
getName()))
                .toList();
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
```

```
    }

    @Override
    public boolean isEnabled() {
        return user.isEnabled();
    }
}
```

This class adapts our database entity to the interface Spring Security understands.

---

13. `UserDetailsService`

`UserDetailsService` answers one central question:

> Given a username, how should Spring Security load that user?

Its main method is:

```
UserDetails loadUserByUsername(String username);
```

The distinction is simple:

```
UserDetails
    → One user

UserDetailsService
    → How to find that user
```

Or:

```
UserDetailsService = Finder
UserDetails        = Result
```

`CustomUserDetailsService`

Coder Army

Coder Army

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsS
ervice {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository
                .findByUsername(username)
                .orElseThrow(() ->
                        new UsernameNotFoundException("User n
ot found")
                );

        return new CustomUserDetails(user);
    }
}
```

The final line means:

```
Convert our application's User
       ↓
into Spring Security's UserDetails view
```

If email is used as the login identity, the same contract can query by email:

```
@Override
public UserDetails loadUserByUsername(String email) {
    User user = userRepository
            .findByEmail(email)
            .orElseThrow(() ->
                    new UsernameNotFoundException("User not f
ound")
```

Coder Army

```
            );

    return new CustomUserDetails(user);
}
```

Is `CustomUserDetails` compulsory?

No. We can directly create Spring Security's built-in implementation:

```
return org.springframework.security.core.userdetails.User
        .withUsername(user.getUsername())
        .password(user.getPassword())
        .authorities(
                user.getRoles()
                        .stream()
                        .map(role -> new SimpleGrantedAuthori
ty(role.getName()))
                        .toList()
        )
        .disabled(!user.isEnabled())
        .build();
```

The custom class is useful when we want the mapping to be explicit or need access to additional domain-user properties.

---

14. `DaoAuthenticationProvider`

At this point, we have a user-loading flow:

```
Spring Security
      ↓ username
UserDetailsService
      ↓
UserRepository
      ↓
Database User
      ↓
```

Coder Army

```
CustomUserDetails
      ↓
UserDetails
```

We still need a component that can:

1. receive the authentication request;
2. load the user through `UserDetailsService`;
3. obtain the stored encoded password;
4. compare it with the submitted raw password;
5. check the account state;
6. return a trusted authentication result.

That component is `DaoAuthenticationProvider`.

```
                DaoAuthenticationProvider
                        /       \
                       /         \
                      ↓           ↓
          UserDetailsService  PasswordEncoder
                    ↓             ↓
            UserRepository  BCryptPasswordEncoder
                    ↓
                Database
```

`DaoAuthenticationProvider` is an `AuthenticationProvider` designed for username-and-password authentication using a `UserDetailsService` and a `PasswordEncoder`.

DAO stands for Data Access Object. The name reflects that user information is retrieved from a user-data source before credentials are verified.

---

15. Complete Internal Login Flow

The full flow can now be understood as one sequence.

```
HTTP Request
    ↓
Authentication Filter
```

Coder Army

```
    ↓
Unauthenticated UsernamePasswordAuthenticationToken
    ↓
AuthenticationManager
    ↓
ProviderManager
    ↓
DaoAuthenticationProvider
    ├── UserDetailsService → Repository → Database
    └── PasswordEncoder → Password verification
    ↓
Authenticated UsernamePasswordAuthenticationToken
    ↓
SecurityContext
    ↓
SecurityContextHolder
    ↓
Authorization
    ↓
Controller
```

Step 1: The filter extracts the credentials

For form login, `UsernamePasswordAuthenticationFilter` reads the submitted username and password. For HTTP Basic, `BasicAuthenticationFilter` reads them from the `Authorization` header.

Step 2: An unauthenticated token is created

```
UsernamePasswordAuthenticationToken

principal     = "aditya"
credentials   = "secret123"
authorities   = []
authenticated = false
```

The filter calls:

```
authenticationManager.authenticate(authenticationToken);
```

Step 3: `ProviderManager` selects a provider

Coder Army

`ProviderManager` checks which configured provider supports `UsernamePasswordAuthenticationToken`.

```
ProviderManager
      ↓
DaoAuthenticationProvider supports the token
```

Step 4: The username is extracted

The provider reads the claimed identity:

```
principal = "aditya"
```

It then asks the `UserDetailsService` for trusted user data:

```
userDetailsService.loadUserByUsername("aditya");
```

Step 5: The database is queried

Our service uses the repository:

```
User user = userRepository
        .findByUsername(username)
        .orElseThrow(() ->
                new UsernameNotFoundException("User not foun
d")
        );

return new CustomUserDetails(user);
```

The provider now has trusted information from the database, including the encoded password and authorities.

Step 6: Account status is checked

Coder Army

Spring Security evaluates values exposed by `UserDetails`:

```
isEnabled()
isAccountNonLocked()
isAccountNonExpired()
isCredentialsNonExpired()
```

For example, an account with:

```
enabled = false
```

cannot authenticate even if the submitted password is correct.

Step 7: The password is verified

The provider has two different values:

```
Login request:
    secret123

Database:
    $2a$10$...
```

It delegates password verification to the configured `PasswordEncoder`:

```
passwordEncoder.matches(
        rawPassword,
        userDetails.getPassword()
);
```

Conceptually:

```
secret123
    +
$2a$10$...
    ↓
BCryptPasswordEncoder.matches(...)
```

```
    ↓
true or false
```

The raw password is not encoded again and compared as a string. `matches(...)` performs the verification using the information stored in the BCrypt representation.

Step 8: A successful authentication result is created

If the user exists, the account is valid, and the password matches, the provider returns a trusted authentication object.

```
UsernamePasswordAuthenticationToken

principal:
    CustomUserDetails

credentials:
    erased or null

authorities:
    ROLE_USER

authenticated:
    true
```

The object has changed from an unverified claim into a verified identity.

```
Claim
  ↓ Authentication
Verified identity
```

Sensitive credential information is normally erased after successful authentication so the raw password is not retained longer than necessary.

Step 9: Authentication is stored in the security context

The successful result is placed into the current security context.

Coder Army

Coder Army

```
Authentication
      ↓
SecurityContext
      ↓
SecurityContextHolder
```

`SecurityContextHolder` provides access to the current request's authenticated security context.

Step 10: Authorization and controller execution continue

Spring Security can now evaluate rules such as:

```
.requestMatchers("/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()
```

If access is allowed, the request reaches the controller. The controller does not need to query the user or verify the password again.

---

16. What Happens When Authentication Fails?

User not found

```
throw new UsernameNotFoundException("User not found");
```

Wrong password

```
passwordEncoder.matches(
        "wrongPassword",
        storedEncodedPassword
);
```

returns `false`. Authentication fails, commonly through a `BadCredentialsException`.

Invalid account state

Authentication can also fail when the account is:

- disabled;
- locked;
- expired;
- using expired credentials.

In each case, no trusted authentication result is placed into the security context.

---

17. Form Login and HTTP Basic Share the Same Backend

Form login and HTTP Basic receive credentials differently, but both can use the same database-authentication engine.

```
Form Login
    ↓
UsernamePasswordAuthenticationFilter
    ↓
UsernamePasswordAuthenticationToken
    ↓
AuthenticationManager
    ↓
DaoAuthenticationProvider
    ↓
Database authentication
```

```
HTTP Basic
    ↓
BasicAuthenticationFilter
    ↓
UsernamePasswordAuthenticationToken
    ↓
AuthenticationManager
    ↓
DaoAuthenticationProvider
```

Coder Army

```
    ↓
Database authentication
```

The filters are different because the HTTP credential formats are different. The underlying username-and-password verification process is the same.

18. Registration and Login Are Separate Flows

Registration stores a new user. Login verifies an existing user.

Registration flow

```
Client
  ↓ username + raw password
AuthService
  ↓
PasswordEncoder.encode(...)
  ↓
BCrypt encoded password
  ↓
UserRepository
  ↓
Database
```

Login flow

```
Client
  ↓ username + raw password
Authentication filter
  ↓
AuthenticationManager
  ↓
DaoAuthenticationProvider
  ├── Load stored user
  └── PasswordEncoder.matches(...)
  ↓
Authenticated Authentication
```

Coder Army

Coder Army

Registration uses:

```
passwordEncoder.encode(rawPassword);
```

Login uses:

```
passwordEncoder.matches(rawPassword, storedEncodedPassword);
```

The server should assign the default role

A normal registration request should not be allowed to choose an administrative role.

Unsafe design:

```
{
  "username": "hacker",
  "password": "123",
  "role": "ROLE_ADMIN"
}
```

Instead, the server chooses the default role:

```
Role role = roleRepository
        .findByName("ROLE_USER")
        .orElseThrow();

user.getRoles().add(role);
```

The registration endpoint should also return a response DTO rather than the entity containing the stored password.

---

19. Project Structure

A possible structure is:

Coder Army

```
entity/
    User.java
    Role.java

repository/
    UserRepository.java
    RoleRepository.java

dto/
    RegisterRequest.java
    UserResponse.java

security/
    CustomUserDetails.java
    CustomUserDetailsService.java

service/
    AuthService.java

controller/
    AuthController.java
    TestController.java

config/
    SecurityConfig.java
```

20. Complete Implementation

`User` entity

```java
@Entity
@Table(name = "users")
@Getter
@Setter
@NoArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

Coder Army

```
    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;

    private boolean enabled = true;

    @ManyToMany
    @JoinTable(
            name = "user_roles",
            joinColumns = @JoinColumn(name = "user_id"),
            inverseJoinColumns = @JoinColumn(name = "role_i
d")
    )
    private Set<Role> roles = new HashSet<>();
}
```

`Role` entity

```
@Entity
@Table(name = "roles")
@Getter
@Setter
@NoArgsConstructor
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
```

```
    private String name;
}
```

Role names can be stored as:

```
ROLE_USER
ROLE_ADMIN
```

When using:

```
.hasRole("ADMIN")
```

Spring Security checks for the authority `ROLE_ADMIN`.

Repositories

```
public interface UserRepository extends JpaRepository<User, L
ong> {

    Optional<User> findByUsername(String username);
}
```

```
public interface RoleRepository extends JpaRepository<Role, L
ong> {

    Optional<Role> findByName(String name);
}
```

DTOs

```
public record RegisterRequest(
        String username,
```

Coder Army

```
        String password
) {}
```

```
public record UserResponse(
        Long id,
        String username
) {}
```

CustomUserDetails

```
public class CustomUserDetails implements UserDetails {

    private final User user;

    public CustomUserDetails(User user) {
        this.user = user;
    }

    @Override
    public String getUsername() {
        return user.getUsername();
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorit
ies() {
        return user.getRoles()
                .stream()
                .map(role -> new SimpleGrantedAuthority(role.
```

Coder Army

Coder Army

```
getName()))
                .toList();
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return user.isEnabled();
    }
}
```

CustomUserDetailsService

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsS
ervice {

    private final UserRepository userRepository;

    @Override
```

Coder Army

```
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository
                .findByUsername(username)
                .orElseThrow(() ->
                        new UsernameNotFoundException("User n
ot found")
                );

        return new CustomUserDetails(user);
    }
}
```

`PasswordEncoder` bean

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Visible `DaoAuthenticationProvider` bean

Spring Security can construct much of the username-and-password infrastructure automatically when the required beans are available. For learning the internal architecture, explicitly creating the provider makes the dependencies visible.

```
@Bean
public DaoAuthenticationProvider authenticationProvider(
        CustomUserDetailsService userDetailsService,
        PasswordEncoder passwordEncoder
) {
    DaoAuthenticationProvider provider =
            new DaoAuthenticationProvider(userDetailsServic
e);
```

```
        provider.setPasswordEncoder(passwordEncoder);

        return provider;
    }
```

This configuration directly shows:

```
DaoAuthenticationProvider
    ├── CustomUserDetailsService
    └── PasswordEncoder
```

SecurityConfig

```
@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider(
            CustomUserDetailsService userDetailsService,
            PasswordEncoder passwordEncoder
    ) {
        DaoAuthenticationProvider provider =
                new DaoAuthenticationProvider(userDetailsServ
ice);

        provider.setPasswordEncoder(passwordEncoder);

        return provider;
    }
```

Coder Army

Coder Army

```
    @Bean
    public SecurityFilterChain securityFilterChain(
            HttpSecurity http,
            DaoAuthenticationProvider provider
    ) throws Exception {

        http
            // Disabled only for the current classroom API de
mo.
            .csrf(csrf -> csrf.disable())

            .authenticationProvider(provider)

            .authorizeHttpRequests(auth -> auth
                    .requestMatchers("/register").permitAll()
                    .requestMatchers("/admin/**").hasRole("AD
MIN")
                    .anyRequest().authenticated()
            )

            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

Registration service

```
@Service
@RequiredArgsConstructor
public class AuthService {

    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
```

Coder Army

```
    private final PasswordEncoder passwordEncoder;

    public UserResponse register(RegisterRequest request) {
        User user = new User();

        user.setUsername(request.username());

        user.setPassword(
                passwordEncoder.encode(request.password())
        );

        user.setEnabled(true);

        Role role = roleRepository
                .findByName("ROLE_USER")
                .orElseThrow(() ->
                        new IllegalStateException("ROLE_USER
not found")
                );

        user.getRoles().add(role);

        User saved = userRepository.save(user);

        return new UserResponse(
                saved.getId(),
                saved.getUsername()
        );
    }
}
```

Registration controller

```
@RestController
@RequiredArgsConstructor
```

Coder Army

```
public class AuthController {

    private final AuthService authService;

    @PostMapping("/register")
    public UserResponse register(
            @RequestBody RegisterRequest request
    ) {
        return authService.register(request);
    }
}
```

21. CSRF and the Postman Demo

When CSRF protection is enabled, a state-changing request such as:

```
POST /register
```

can be rejected if Postman does not send a valid CSRF token.

For a short classroom API demonstration focused only on authentication, CSRF may be temporarily disabled:

```
.csrf(csrf -> csrf.disable())
```

> Disabling CSRF is a demo decision, not a universal production-security recommendation. The correct production configuration depends on how clients authenticate and whether browsers automatically attach credentials.

22. Testing an Authenticated Endpoint

Simple protected endpoint

Coder Army

```
@RestController
public class TestController {

    @GetMapping("/profile")
    public String profile() {
        return "Welcome to your profile";
    }
}
```

Because the configuration contains:

```
.anyRequest().authenticated()
```

`/profile` requires authentication.

Display the current `Authentication`

```
@GetMapping("/me")
public Map<String, Object> me(Authentication authentication)
{
    return Map.of(
            "username", authentication.getName(),
            "authorities", authentication.getAuthorities(),
            "authenticated", authentication.isAuthenticated()
    );
}
```

After successful authentication, the response may look like:

```
{
  "username": "aditya",
  "authorities": [
    {
      "authority": "ROLE_USER"
    }
```

```
  ],
  "authenticated": true
}
```

This endpoint makes the final authenticated object visible during the demonstration.

23. Suggested Demonstration Sequence

1. Create the default role

Ensure the `roles` table contains:

```
ROLE_USER
```

2. Register a user

```
POST /register
Content-Type: application/json

{
  "username": "aditya",
  "password": "secret123"
}
```

Show that the database contains an encoded password rather than the raw password.

3. Call a protected endpoint without credentials

```
GET /me
```

The request should not receive protected data.

Coder Army

Coder Army

4. Call it with correct HTTP Basic credentials

```
Username: aditya
Password: secret123
```

The endpoint should return the authenticated username and authorities.

5. Try a wrong password

The authentication attempt should fail even though the user exists.

6. Disable the account

Set:

```
enabled = false
```

Authentication should now fail even with the correct password.

This demonstrates that database authentication is more than password comparison; account state is also part of the decision.

---

24. Final Mental Model

The complete architecture can be remembered in four layers.

Layer 1: Receive credentials

```
UsernamePasswordAuthenticationFilter
or
BasicAuthenticationFilter
```

Layer 2: Represent the request

Coder Army

```
UsernamePasswordAuthenticationToken
```

Layer 3: Authenticate the request

```
AuthenticationManager
    ↓
ProviderManager
    ↓
DaoAuthenticationProvider
```

Layer 4: Load and verify the user

```
UserDetailsService → UserRepository → Database

PasswordEncoder → Password verification
```

After success:

```
Authenticated Authentication
        ↓
SecurityContext
        ↓
SecurityContextHolder
        ↓
Authorization
        ↓
Controller
```

Key Takeaways

1. Spring Security operates through servlet filters before the controller.
2. `DelegatingFilterProxy` bridges Tomcat and Spring-managed security components.
3. `FilterChainProxy` selects the first matching `SecurityFilterChain`.

4. `HttpSecurity` builds and configures that chain.
5. Authentication filters extract credentials; they do not perform database verification themselves.
6. `UsernamePasswordAuthenticationToken` represents both the unverified request and, later, the verified result.
7. `AuthenticationManager` is the entry point into the authentication engine.
8. `ProviderManager` delegates to a compatible `AuthenticationProvider`.
9. `DaoAuthenticationProvider` connects `UserDetailsService` with `PasswordEncoder`.
10. `UserDetails` represents one user; `UserDetailsService` explains how to find that user.
11. Registration uses `encode(...)`; login uses `matches(...)`.
12. After successful login, the authenticated object is placed in the `SecurityContextHolder` and authorization continues.

Coder Army

Coder Army

Spring Security JWT Authentication

From Stateful Sessions to Stateless JWT Authentication

1. The Problem: Authentication on Every Request

Consider a protected endpoint:

```
@GetMapping("/profile")
public String profile(Authentication authentication) {
    return "Logged in as: " + authentication.getName();
}
```

If the request contains valid credentials:

```
GET /profile
Authorization: Basic ...
```

authentication succeeds.

But a second request without credentials is a completely separate HTTP request:

```
GET /profile
```

Result:

```
401 Unauthorized
```

Why?

Coder Army

HTTP does not automatically remember that a previous request was authenticated.

Conceptually:

```
Request 1
GET /profile
Authorization: Basic ...
        ↓
Authentication succeeds

Request 2
GET /profile
        ↓
No credentials available
```

Authenticating one request does not automatically authenticate future requests.

The server therefore needs some identifier travelling with later requests if it wants to remember an authenticated client.

---

2. Session-Based Authentication

After a successful username/password login, Spring Security has an authenticated `Authentication` object.

Conceptually:

```
Authentication
├── Principal
├── Authorities
└── authenticated = true
```

Instead of discarding this authentication state after the request, the server can associate it with an HTTP session.

```
Server

Session ABC123
    ↓
SecurityContext
```

Coder Army

```
    ↓
Authentication
    ↓
aditya
ROLE_USER
```

The client receives only the session identifier:

```
ABC123
```

In a Spring Boot servlet application, the session ID commonly travels through a cookie:

```
Cookie: JSESSIONID=ABC123
```

Session Login Flow

```
CLIENT                          SERVER

username/password
        ──────────────────────────→

                            Authentication succeeds
                                    ↓
                              HttpSession
                                    ↓
                              SecurityContext
                                    ↓
                              Authentication

        ←──────────────────────────
        Set-Cookie:
        JSESSIONID=ABC123
```

Future request:

```
GET /profile
Cookie: JSESSIONID=ABC123
        ──────────────────────────→
```

Coder Army

```
Find session ABC123
        ↓
  SecurityContext
        ↓
  Authentication
        ↓
User authenticated
```

> `JSESSIONID` does not normally contain the username, password, or roles. It is essentially an identifier that points to server-side session state.

---

3. Session Management in Spring Security

A typical session-based configuration can look like:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity h
ttp)
        throws Exception {

    http
        .csrf(csrf -> csrf.disable())

        .authenticationProvider(authenticationProvider)

        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/register").permitAll()
            .anyRequest().authenticated()
        )

        .formLogin(Customizer.withDefaults())

        .httpBasic(Customizer.withDefaults())

        .sessionManagement(session ->
            session.sessionCreationPolicy(
```

```
                SessionCreationPolicy.IF_REQUIRED
            )
        );

    return http.build();
}
```

Spring Security can persist the `SecurityContext` using an `HttpSession`-backed `SecurityContextRepository`.

SessionCreationPolicy

`SessionCreationPolicy` controls how Spring Security behaves with HTTP sessions.

Policy	Creates Session?	Uses Existing Session?	Mental Model
`ALWAYS`	Yes	Yes	Always session-based
`IF_REQUIRED`	When needed	Yes	Create a session when security requires one
`NEVER`	No	Yes	Do not create a session, but use one if it already exists
`STATELESS`	No	No for restoring the `SecurityContext`	Authenticate each request independently

For session-based authentication:

```
SessionCreationPolicy.IF_REQUIRED
```

is a common choice.

For JWT authentication later:

```
SessionCreationPolicy.STATELESS
```

will be used.

Coder Army

Coder Army

4. Why Session Authentication Is Stateful

With a session, the request may contain only:

```
Cookie: JSESSIONID=ABC123
```

The server still has to remember what that session represents:

```
SERVER SESSION STORE

ABC123 → Aditya
XYZ456 → Rohit
PQR789 → Priya
```

Authentication state therefore exists on the server.

This is why session-based authentication is called:

> **Stateful Authentication**

---

5. Why JWT?

JWT stands for:

> **JSON Web Token**

The central idea is:

> Can a user authenticate once without requiring the server to maintain a separate session entry for every authenticated client?

With a session:

```
JSESSIONID = ABC123
        ↓
Session Store
        ↓
SecurityContext
        ↓
```

Coder Army

```
Authentication
        ↓
Aditya
```

The value `ABC123` tells us almost nothing by itself.

A JWT can instead carry claims such as:

```
{
  "sub": "aditya",
  "roles": ["ROLE_USER"],
  "iat": 1786570000,
  "exp": 1786573600
}
```

The server can use information inside the token to reconstruct authentication for each request.

```
JWT
 ↓
claims
 ↓
Authentication
 ↓
SecurityContext
```

The server no longer needs to preserve the user's `Authentication` object in an HTTP session.

---

6. The Security Problem With Client-Carried Data

Suppose the token contains:

```
{
  "sub": "aditya",
```

Coder Army

```
  "roles": ["ROLE_USER"]
}
```

What prevents a client from changing it to:

```
{
  "sub": "aditya",
  "roles": ["ROLE_ADMIN"]
}
```

The server needs to answer two questions:

- Was this information issued by a trusted party?
- Has the information been modified after it was issued?

That is the purpose of the **signature**.

---

7. Why Not Send Plain JSON?

A naive token could be:

```
{
  "sub": "aditya",
  "role": "ROLE_USER"
}
```

But raw JSON is inconvenient as a transport format:

- It contains spaces, braces, quotes, and possibly line breaks.
- The same JSON can be formatted differently.
- Tokens should be easy to transport as a compact string.
- Cryptographic signing needs a precise byte representation.

JWT therefore uses **Base64URL encoding** for its compact representation.

Coder Army

8. Base64URL Is Encoding, Not Encryption

For example:

```
{"sub":"42"}
```

can be represented as something like:

```
eyJzdWIiOiI0MiJ9
```

Base64URL provides a URL-safe textual representation of bytes and generally removes trailing padding.

The important point is:

> **Base64URL is encoding, not encryption.**

Anyone who gets the token can decode the header and payload.

No password or secret key is required merely to decode them.

Therefore, JWT payloads should not be treated as hidden data.

Base64URL also does not compress the payload; encoded data is generally larger than the original bytes.

---

9. Why Encoding Alone Is Insecure

Suppose the token is only:

```
Base64URL(payload)
```

Payload:

```
{
  "sub": "user-42",
```

Coder Army

```
  "role": "STUDENT"
}
```

An attacker could:

1. Decode the payload.
2. Change `STUDENT` to `ADMIN`.
3. Encode it again.
4. Send the modified value.

The server would have no cryptographic way to know whether the data was genuinely issued by it.

We need:

Integrity

The protected data has not been modified.

Authenticity

The protected data was created by someone possessing the trusted signing key.

---

10. Why a Plain Hash Is Not Enough

A naive idea might be:

```
token = payload + SHA256(payload)
```

or:

```
Base64URL(payload).SHA256(payload)
```

But an attacker can modify the payload and calculate a new SHA-256 hash themselves.

Coder Army

```
Original:
role = STUDENT

Attacker changes:
role = ADMIN

Attacker calculates:
SHA256(modified payload)
```

A plain hash can answer:

```
Does this hash match this data?
```

But it cannot answer:

```
Was this data produced by my trusted server?
```

To establish authenticity, the calculation must depend on a secret or private key that the attacker does not possess.

---

11. Structure of a Signed JWT

A signed JWT commonly has three parts:

```
HEADER.PAYLOAD.SIGNATURE
```

Example shape:

```
xxxxx.yyyyy.zzzzz
```

The header and payload are represented using Base64URL, and the signature bytes are also represented in Base64URL form.

11.1 Header

Coder Army

Example:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

`alg`

Specifies the cryptographic algorithm used to sign the token.

`typ`

Describes the token type.

11.2 Payload

Example:

```
{
  "iss": "https://auth.coderarmy.in",
  "sub": "user-42",
  "aud": "course-api",
  "role": "STUDENT",
  "iat": 1786570000,
  "exp": 1893456000
}
```

Common JWT Claims

Claim	Meaning
`iss`	Issuer — who issued the token
`sub`	Subject — who or what the token represents
`aud`	Audience — which application/API should accept it
`iat`	Issued At — when the token was created
`exp`	Expiration Time — when the token expires

Coder Army

Claim	Meaning
`jti`	Unique identifier for the token

The JWT standard defines registered claims, but not every claim is mandatory in every application. The application or security protocol decides which claims are required.

Custom Claims

Applications can also add their own claims:

```
{
  "roles": ["ROLE_USER"],
  "department": "engineering"
}
```

Because the payload is readable, avoid storing unnecessary or sensitive information such as:

```
Passwords
Secret API keys
Credit card numbers
Private personal information
```

11.3 Signature

The signature protects the encoded header and encoded payload.

```
Base64URL(header) + "." + Base64URL(payload)
                        ↓
                  signing input
```

Conceptually:

```
signature = cryptographic_sign(
    signingInput,
```

```
    signingKey
)
```

Final JWT:

```
Base64URL(header)
.
Base64URL(payload)
.
Base64URL(signature)
```

The signature covers both the header and the payload.

---

12. Symmetric JWT Signing With HS256

`HS256` means:

```
HMAC using SHA-256
```

Conceptually:

```
signature =
    HMAC-SHA256(
        secretKey,
        Base64URL(header) + "." + Base64URL(payload)
    )
```

The flow is:

```
headerEncoded.payloadEncoded
             +
         secret key
             ↓
       HMAC-SHA256
             ↓
         signature
```

Coder Army

The client receives:

```
headerEncoded.payloadEncoded.signature
```

---

13. What Happens If the Payload Changes?

Suppose the original payload contains:

```
"role": "STUDENT"
```

An attacker changes it to:

```
"role": "ADMIN"
```

The payload encoding changes, which means the signing input changes.

```
Original:
header.originalPayload

Modified:
header.modifiedPayload
```

Without the secret key, the attacker cannot calculate the correct HMAC for the modified input.

When the modified token reaches the server:

```
Server recalculates expected signature
using its secret
            ↓
Expected signature ≠ token signature
            ↓
Reject token
```

The JWT signature does **not physically prevent** someone from editing the token.

> It makes unauthorized modification detectable.

Coder Army

Coder Army

14. Symmetric Keys

With HS256:

```
Signing key = Verification key
```

The same secret is used to create and verify the HMAC.

```
Issuer:
HMAC(message, secret)

Verifier:
HMAC(message, same secret)
```

Advantage

- Simple
- Fast
- Convenient when one trusted application both issues and validates tokens

Limitation

Every service that verifies the token must know the same secret.

```
Authentication server ─┐
Course service        ├── same secret
Payment service       ┤
Admin service         ┘
```

Anyone who has the shared secret can not only verify tokens but can also create valid tokens.

If one service is compromised, the shared signing secret may also be compromised.

Coder Army

15. Asymmetric JWT Signing

Asymmetric cryptography uses a key pair:

```
Private key
Public key
```

The keys have different responsibilities:

```
Private key → creates signatures
Public key  → verifies signatures
```

Common JWT signature algorithms include:

```
RS256 → RSA signature with SHA-256
ES256 → ECDSA signature with SHA-256
```

For RS256, conceptually:

```
signature = Sign(
    privateKey,
    Base64URL(header) + "." + Base64URL(payload)
)
```

Verification:

```
Verify(
    publicKey,
    signingInput,
    signature
)
```

A typical architecture becomes:

```
Authentication server → private key + public key
Course service        → public key only
```

```
Payment service       → public key only
Admin service         → public key only
```

The resource services can verify signatures using the public key but cannot create new valid signatures.

This model is useful when one authorization server issues tokens and multiple APIs validate them.

---

16. Symmetric vs Asymmetric Signing

Property	HS256	RS256 / ES256
Key model	Shared secret	Private/public key pair
Signing	Secret key	Private key
Verification	Same secret key	Public key
Can verifier create tokens?	Yes, if it has the secret	No, if it only has the public key
Simple for one application	Yes	More setup
Useful for multiple independent APIs	Less ideal	Very useful

---

17. JWT Tampering Experiment

A useful way to understand JWT security is to create a token, decode it, modify its payload, and then verify the modified token.

Original Token Shape

```
A.B.C
```

Where:

Coder Army

Coder Army

```
A = encoded header
B = encoded payload
C = signature
```

Suppose the original payload contains:

```
{
  "sub": "aditya",
  "role": "STUDENT"
}
```

After changing the role to `ADMIN`, the new encoded payload becomes:

```
B2
```

If a tool has access to the secret, it can generate a valid new signature:

```
A.B2.C2
```

But a real attacker who does not know the secret cannot calculate `C2`.

The attacker's tampered token would instead look like:

```
A.B2.C
```

That is:

```
Original header
+
Modified payload
+
Original signature
```

The payload can still be decoded and may visibly show:

Coder Army

```
{
  "sub": "aditya",
  "role": "ADMIN"
}
```

But signature verification fails because the signature was created for the original payload.

```
Modified signing input
        ↓
Old signature no longer matches
        ↓
Invalid Signature
```

This demonstrates the central security property of a signed JWT:

> The payload is readable and editable, but unauthorized changes cause signature verification to fail.

---

18. JWT Authentication Flow in the Application

There are two major flows.

Login Flow

```
POST /auth/login

{
  "username": "aditya",
  "password": "123456"
}
        ↓
AuthenticationManager
        ↓
DaoAuthenticationProvider
        ↓
CustomUserDetailsService
```

Coder Army

```
        ↓
Database
        ↓
PasswordEncoder
        ↓
Authentication successful
        ↓
Generate JWT
        ↓
Return token
```

Future Protected Request

```
GET /profile

Authorization:
Bearer eyJhbGciOi...
        ↓
Extract JWT
        ↓
Validate JWT
        ↓
Identify user
        ↓
Create Authentication
        ↓
SecurityContext
        ↓
Authorization
        ↓
Controller
```

---

19. Add Spring Security Resource Server Support

Dependency:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</a
```

```
rtifactId>
</dependency>
```

Spring Security Resource Server can authenticate requests using bearer JWTs, including custom JWTs issued by the application itself.

---

20. `JwtEncoder` vs `JwtDecoder`

Spring Security separates token creation and token validation into two responsibilities.

```
JwtEncoder
    ↓
CREATE / SIGN JWT
```

```
JwtDecoder
    ↓
READ / VERIFY / VALIDATE JWT
```

For this application, HS256 is used:

```
same secret key
      ↓
sign JWT
      ↓
verify JWT
```

This keeps the demonstration simple because the same Spring Boot application both issues and validates the token.

---

21. Configure the JWT Secret

Generate a sufficiently long Base64-encoded secret:

Coder Army

Coder Army

```
openssl rand -base64 32
```

Configuration:

```
jwt.secret=${JWT_SECRET}
jwt.issuer=coderarmy-api
jwt.expiry=900
```

Here:

```
900 seconds = 15 minutes
```

For classroom demonstrations, the secret can temporarily be placed directly in `application.properties`.

For a real application:

> Do not commit signing secrets to Git.

---

22. Create the `SecretKey`

```
@Bean
public SecretKey jwtSecretKey(
        @Value("${jwt.secret}") String secret) {

    byte[] decodedKey =
            Base64.getDecoder().decode(secret);

    return new SecretKeySpec(
            decodedKey,
            "HmacSHA256"
    );
}
```

Coder Army

Conceptually:

```
Base64 configuration value
        ↓
decode
        ↓
raw key bytes
        ↓
SecretKey
```

The same `SecretKey` will be used by both the encoder and decoder.

---

23. Create the `JwtEncoder`

```java
@Bean
public JwtEncoder jwtEncoder(
        SecretKey secretKey) {

    return NimbusJwtEncoder
            .withSecretKey(secretKey)
            .algorithm(MacAlgorithm.HS256)
            .build();
}
```

Application dependency:

```
Application
    ↓
JwtEncoder
    ↓
NimbusJwtEncoder
```

The rest of the application depends on the `JwtEncoder` abstraction, while `NimbusJwtEncoder` provides the implementation.

Coder Army

24. Login DTOs

Login request:

```
public class LoginRequest {
    String username;
    String password;
}
```

Login response:

```
public record LoginResponse(
        String accessToken
) {}
```

Client request:

```
{
  "username": "aditya",
  "password": "123456"
}
```

Server response:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9..."
}
```

---

25. Create `JwtService`

```
@Service
public class JwtService {

    private final JwtEncoder jwtEncoder;
```

Coder Army

```
    @Value("${jwt.issuer}")
    private String issuer;

    @Value("${jwt.expiry}")
    private long expiry;

    public JwtService(JwtEncoder jwtEncoder) {
        this.jwtEncoder = jwtEncoder;
    }

    public String generateToken(
            Authentication authentication) {

        Instant now = Instant.now();

        List<String> authorities =
                authentication.getAuthorities()
                        .stream()
                        .map(GrantedAuthority::getAuthority)
                        .toList();

        JwtClaimsSet claims =
                JwtClaimsSet.builder()

                        .issuer(issuer)

                        .issuedAt(now)

                        .expiresAt(
                            now.plusSeconds(expiry)
                        )

                        .subject(
                            authentication.getName()
                        )
```

Coder Army

```
                        .claim(
                            "authorities",
                            authorities
                        )

                        .build();

        Jwt jwt =
                jwtEncoder.encode(
                    JwtEncoderParameters.from(claims)
                );

        return jwt.getTokenValue();
    }
}
```

26. Understanding `JwtService`

The service receives an already authenticated:

```
Authentication authentication
```

Conceptually:

```
Principal     → aditya
Authorities   → ROLE_USER
Authenticated → true
```

The service converts this successful authentication result into JWT claims.

`issuer`

Coder Army

```
.issuer(issuer)
```

Produces:

```
"iss": "coderarmy-api"
```

It identifies who issued the JWT.

The decoder can later be configured to accept tokens only from this issuer.

`subject`

```
.subject(authentication.getName())
```

Produces:

```
"sub": "aditya"
```

The subject identifies who the token represents.

`issuedAt`

```
.issuedAt(now)
```

Produces the:

```
iat
```

claim.

It records when the token was created.

`expiresAt`

Coder Army

```
.expiresAt(
    now.plusSeconds(expiry)
)
```

Produces the:

```
exp
```

claim.

After this time, the token should no longer be accepted.

---

27. Why Store Authorities in the JWT?

The application adds:

```
.claim(
    "authorities",
    authorities
)
```

Example payload:

```
{
  "authorities": [
    "ROLE_USER"
  ]
}
```

The JWT then contains enough information to recreate the user's authentication.

```
sub
 ↓
Who is the user?

authorities
```

Coder Army

```
 ↓
What authorities does the user have?
```

Subsequent requests do not need this flow:

```
JWT
 ↓
username
 ↓
database
 ↓
UserDetails
```

Instead:

```
JWT
├── sub = aditya
└── authorities = ROLE_USER
        ↓
Authentication
```

This is what makes the example stateless.

---

28. Create `/auth/login`

```java
@RestController
@RequestMapping("/auth")
public class AuthController {

    private final AuthenticationManager
            authenticationManager;

    private final JwtService jwtService;

    public AuthController(
            AuthenticationManager authenticationManager,
            JwtService jwtService) {
```

Coder Army

```
        this.authenticationManager =
                authenticationManager;

        this.jwtService = jwtService;
    }

    @PostMapping("/login")
    public LoginResponse login(
            @RequestBody LoginRequest request) {

        Authentication authenticationRequest =
                UsernamePasswordAuthenticationToken
                    .unauthenticated(
                        request.username(),
                        request.password()
                    );

        Authentication authentication =
                authenticationManager.authenticate(
                    authenticationRequest
                );

        String token =
                jwtService.generateToken(
                    authentication
                );

        return new LoginResponse(token);
    }
}
```

Login Authentication Flow

Coder Army

```
username + password
        ↓
UsernamePasswordAuthenticationToken
unauthenticated
        ↓
AuthenticationManager
        ↓
DaoAuthenticationProvider
        ↓
CustomUserDetailsService
        ↓
Database
        ↓
PasswordEncoder
        ↓
Authentication
authenticated = true
        ↓
JwtService
        ↓
JwtEncoder
        ↓
JWT
```

The username/password authentication mechanism remains the same as before.

JWT generation happens **after** authentication succeeds.

---

29. Authenticate Future Requests Using JWT

JWT creation is only half of the process.

When a token comes back from the client, the application must:

```
JWT string
      ↓
verify signature
      ↓
validate claims
      ↓
trusted Jwt object
```

Coder Army

This is the responsibility of `JwtDecoder`.

30. Create `JwtDecoder`

```
@Bean
public JwtDecoder jwtDecoder(
        SecretKey secretKey,
        @Value("${jwt.issuer}") String issuer) {

    NimbusJwtDecoder decoder =
            NimbusJwtDecoder
                .withSecretKey(secretKey)
                .macAlgorithm(MacAlgorithm.HS256)
                .build();

    decoder.setJwtValidator(
        JwtValidators.createDefaultWithIssuer(
            issuer
        )
    );

    return decoder;
}
```

The decoder uses the same HS256 secret key that was used by the encoder.

It also configures validation for the expected issuer.

31. What the Decoder Checks

For an incoming token:

```
HEADER.PAYLOAD.SIGNATURE
```

Coder Army

the decoder performs security checks conceptually like:

```
Is this structurally a valid JWT?
        ↓
Can the signature be verified
using the trusted secret key?
        ↓
Are the standard claims valid?
        ↓
Is the issuer correct?
        ↓
Is the token currently valid?
```

Only after these checks can the resulting claims be treated as trusted authentication data.

---

32. Map JWT Authorities Into Spring Security

Spring Security Resource Server commonly maps OAuth scopes such as:

```
{
  "scope": "read write"
}
```

into authorities like:

```
SCOPE_read
SCOPE_write
```

But this application stores:

```
ROLE_USER
ROLE_ADMIN
```

inside a custom JWT claim:

Coder Army

```
{
  "authorities": [
    "ROLE_USER"
  ]
}
```

Therefore, Spring must be told:

1. Read authorities from the `authorities` claim.
2. Do not add another prefix.

---

33. `JwtAuthenticationConverter`

```
@Bean
public JwtAuthenticationConverter
        jwtAuthenticationConverter() {

    JwtGrantedAuthoritiesConverter
            authoritiesConverter =
                new JwtGrantedAuthoritiesConverter();

    authoritiesConverter
            .setAuthoritiesClaimName(
                "authorities"
            );

    authoritiesConverter
            .setAuthorityPrefix("");

    JwtAuthenticationConverter
            authenticationConverter =
                new JwtAuthenticationConverter();

    authenticationConverter
```

```
            .setJwtGrantedAuthoritiesConverter(
                authoritiesConverter
            );

    return authenticationConverter;
}
```

The empty prefix is important because the token already contains:

```
ROLE_USER
```

The application wants exactly:

```
ROLE_USER
```

not an automatically prefixed value.

This keeps existing authorization rules compatible:

```
.hasRole("USER")
```

```
.hasRole("ADMIN")
```

---

34. Final `SecurityFilterChain`

```
@Bean
public SecurityFilterChain securityFilterChain(
        HttpSecurity http,
        JwtAuthenticationConverter
                jwtAuthenticationConverter)
        throws Exception {

    http
```

Coder Army

Coder Army

```
        .csrf(csrf ->
            csrf.disable()
        )

        .sessionManagement(session ->
            session.sessionCreationPolicy(
                SessionCreationPolicy.STATELESS
            )
        )

        .authorizeHttpRequests(auth ->
            auth

                .requestMatchers(
                    "/auth/login",
                    "/register"
                )
                .permitAll()

                .requestMatchers(
                    "/admin/**"
                )
                .hasRole("ADMIN")

                .anyRequest()
                .authenticated()
        )

        .oauth2ResourceServer(oauth2 ->
            oauth2.jwt(jwt ->
                jwt.jwtAuthenticationConverter(
                    jwtAuthenticationConverter
                )
            )
        );
```

```
    return http.build();
}
```

Coder Army

35. Why `SessionCreationPolicy.STATELESS`?

```
.sessionManagement(session ->
    session.sessionCreationPolicy(
        SessionCreationPolicy.STATELESS
    )
)
```

means Spring Security should not use the HTTP session to preserve authentication between requests.

Each protected request must carry its own authentication material, in this case:

```
Authorization: Bearer <JWT>
```

The server validates the token and reconstructs authentication for that request.

```
Request
  +
Bearer JWT
   ↓
JWT validation
   ↓
Authentication
   ↓
SecurityContext
   ↓
Authorization
```

After the request completes, the application does not rely on an HTTP session to remember that authentication for the next request.

Coder Army

36. Why `oauth2ResourceServer().jwt()` ?

```
.oauth2ResourceServer(oauth2 ->
    oauth2.jwt(jwt ->
        jwt.jwtAuthenticationConverter(
            jwtAuthenticationConverter
        )
    )
)
```

Spring Security packages bearer-token JWT authentication under its **Resource Server** support.

This support can protect an API using a custom JWT even when the application is not implementing a complete OAuth2 Authorization Server.

Conceptually:

```
Protected request
      ↓
Authorization: Bearer <JWT>
      ↓
Spring Security Resource Server
      ↓
JWT validation
      ↓
Authentication
```

---

37. Two Authentication Providers, Two Purposes

The application now uses two different authentication mechanisms.

Username/Password Login

```
DaoAuthenticationProvider
_________________________
```

Coder Army

```
understands:
username + password
```

It authenticates the user's login credentials.

JWT Authentication

```
JwtAuthenticationProvider
─────────────────────────
understands:
JWT
```

It authenticates future requests carrying bearer tokens.

Both mechanisms eventually produce the same core Spring Security abstraction:

```
Authentication
```

This keeps Spring Security's architecture consistent regardless of how the user proves their identity.

---

38. Complete End-to-End Flow

First Request: Login

```
POST /auth/login
username + password
        ↓
AuthenticationManager
        ↓
DaoAuthenticationProvider
        ↓
CustomUserDetailsService
        ↓
PasswordEncoder
        ↓
Authentication
        ↓
JwtService
```

```
        ↓
JwtEncoder
        ↓
Signed JWT
        ↓
Return token to client
```

Future Request

```
GET /profile
Authorization: Bearer <JWT>
        ↓
Bearer-token JWT authentication
        ↓
JwtDecoder
        ↓
Verify signature
        ↓
Validate claims
        ↓
JwtAuthenticationConverter
        ↓
Create Authentication
        ↓
SecurityContext
        ↓
Authorization rules
        ↓
Controller
```

---

39. Session Authentication vs JWT Authentication

Session-Based Authentication	JWT-Based Authentication
Server stores authentication state	Authentication information is reconstructed from the token
Client usually sends `JSESSIONID`	Client sends a bearer JWT
Session ID points to server-side state	Token carries claims
Commonly stateful	Can be stateless

Coder Army

Session-Based Authentication	JWT-Based Authentication
Server restores `SecurityContext` from session	Server validates token and creates authentication per request

Coder Army

40. Important JWT Takeaways

- JWT stands for **JSON Web Token**.
- A signed JWT commonly has three parts:

```
HEADER.PAYLOAD.SIGNATURE
```

- Header and payload are encoded, not encrypted.
- Anyone possessing a normal signed JWT can usually read its claims.
- The signature provides integrity and authenticity.
- The signature does not stop a person from editing the token; it makes unauthorized changes detectable.
- HS256 uses the same secret for signing and verification.
- Asymmetric signing uses a private key for signing and a public key for verification.
- Do not place passwords or sensitive secrets inside JWT payloads.
- `JwtEncoder` creates/signs JWTs.
- `JwtDecoder` verifies and validates incoming JWTs.
- JWT claims can be used to reconstruct Spring Security's `Authentication`.
- `SessionCreationPolicy.STATELESS` prevents Spring Security from relying on an HTTP session to preserve authentication between requests.
- Spring Security Resource Server support can validate custom bearer JWTs.
- `DaoAuthenticationProvider` handles username/password authentication.
- `JwtAuthenticationProvider` handles JWT authentication.

- Both ultimately produce an `Authentication` object.

Coder Army

41. Mental Model

```
LOGIN
──────────────────────────────────────────

Username + Password
        ↓
AuthenticationManager
        ↓
DaoAuthenticationProvider
        ↓
UserDetailsService + PasswordEncoder
        ↓
Authentication
        ↓
JwtEncoder
        ↓
Signed JWT
        ↓
Client


PROTECTED REQUEST
──────────────────────────────────────────

Client
  ↓
Authorization: Bearer <JWT>
  ↓
JWT verification + validation
  ↓
JwtAuthenticationProvider
  ↓
JwtAuthenticationConverter
  ↓
Authentication
  ↓
SecurityContext
  ↓
Authorization
  ↓
Controller
```

> With session authentication, the server remembers the authenticated user between requests. With stateless JWT authentication, each request brings the information needed to recreate and verify authentication.

Coder Army

Coder Army

OAuth 2.0, OpenID Connect and Google Login with Spring Boo

1. Moving Beyond Our Own Authentication System

Until now, our application controlled the complete security flow:

```
User sends username and password
            ↓
Spring Boot authenticates the user
            ↓
Application generates a JWT
            ↓
Client sends the JWT with future requests
```

Our API validates the token's signature and trusts the claims only when the signature is valid.

JWT solves an important problem:

> How can an API verify the information inside a token without trusting the client?

Now consider a different requirement:

```
Continue with Google
```

We cannot ask the user to send their Google password to our application:

```
POST /login-with-google

email=student@gmail.com
googlePassword=???
```

Our application should never receive, process or store the user's Google password.

Coder Army

2. The Delegated Access Problem

Suppose we build a productivity application and add this feature:

> Connect your Google Calendar.

Our application needs permission to read the user's calendar. A dangerous design would be:

```
User gives Google credentials to our application
                    ↓
Our application logs in to Google as the user
                    ↓
Our application accesses Google Calendar
```

This creates two major problems:

1. The user must share their Google password with another application.
2. The application receives far more access than it needs.

The same Google credentials could potentially expose Gmail, Drive, Photos, Calendar, YouTube and other services, even though the application only needs calendar access.

OAuth 2.0 solves this by replacing shared credentials with delegated, limited access:

```
User signs in directly with Google
                ↓
User allows the application to read the calendar
                ↓
Google gives the application an access token
                ↓
The application uses the token with Google Calendar API
```

> OAuth 2.0 is primarily a framework for delegated authorization.

Authorization answers:

Coder Army

```
What are you allowed to do?
```

3. The Four OAuth Roles

Role	Example	Responsibility
Resource Owner	User	Owns or controls access to the protected resource
Client	CoderArmy application	Requests delegated access on behalf of the user
Authorization Server	Google's authorization system	Authenticates the user, obtains consent and issues tokens
Resource Server	Google Calendar API	Hosts the protected resource and validates access tokens

Resource Owner

The resource owner is usually the user who controls the protected data.

Client

In OAuth terminology, **client does not necessarily mean browser**. It means the application requesting access.

A client can be a:

- Spring Boot application
- Mobile application
- Desktop application
- Browser-based application

Authorization Server

The authorization server performs tasks such as:

```
Authenticate the user
        ↓
```

Coder Army

```
Ask for authorization or consent
        ↓
Issue tokens
```

Resource Server

The resource server contains the protected resource. For example, Google Calendar API receives an access token with the request:

```
Authorization: Bearer <access-token>
```

It validates the token and decides whether the requested operation is allowed.

---

4. Scope and Consent

The authorization server must know exactly what the client wants to access. OAuth represents this requested access through **scopes**.

Conceptually, a calendar application could request:

```
calendar.read
```

instead of requesting unrestricted access to everything in the user's Google account.

> A scope represents the access being requested or granted.

Google can then display a consent screen:

```
CoderArmy App wants permission to:

✓ View your basic profile
✓ Read your calendar
```

The user can approve or reject the request. This is the consent step.

Coder Army

```
Client requests specific scopes
              ↓
Authorization server displays consent
              ↓
User approves or rejects the requested access
```

---

5. OAuth and OIDC Tokens

Access Token

After authorization, the client needs a credential it can present to the resource server. That credential is the **access token**.

```
Authorization Server
        ↓ access token
Client
        ↓ bearer token
Resource Server
```

> An access token is a credential used by a client to access a protected resource.

An access token may be a JWT, but OAuth 2.0 does not require it to be one. It can also be an opaque, random-looking value.

The client should generally treat an access token as an opaque credential unless the authorization server explicitly documents its format.

The Authentication Problem

OAuth solves delegated authorization:

```
What can this client access?
```

However, login requires an identity answer:

```
Who is the user who signed in?
```

Coder Army

OAuth 2.0 alone is not an identity protocol. **OpenID Connect**, commonly called OIDC, adds authentication and identity concepts on top of OAuth 2.0.

Protocol	Primary purpose	Main question
OAuth 2.0	Delegated authorization	What can this application access?
OpenID Connect	Authentication and identity	Who signed in?

ID Token

OIDC introduces the **ID token**. It is intended for the client and contains information about the authentication event and the user's identity.

An ID token is normally a signed JWT and may contain claims such as:

```
{
  "sub": "123456789",
  "iss": "https://accounts.google.com",
  "aud": "our-google-client-id",
  "name": "Aditya",
  "email": "adi@gmail.com"
}
```

Important claims include:

Claim	Meaning
`sub`	Stable subject identifier for the user at that provider
`iss`	Issuer that created the token
`aud`	Client for which the token was issued
`exp`	Expiration time

Refresh Token

Access tokens should expire. A refresh token may be issued so the client can obtain a new access token without forcing the user to repeat the complete login and consent flow.

Coder Army

```
Access token expires
        ↓
Client sends refresh token to authorization server
        ↓
Authorization server issues a new access token
```

A refresh token is sent to the authorization server's token endpoint, not to a normal resource API.

Token Comparison

Token	Used by	Sent to	Purpose
Access token	Client	Resource server	Access a protected API
ID token	Client	Client application	Verify authentication and obtain identity information
Refresh token	Client	Authorization server	Obtain a new access token

> An ID token is not a replacement for an access token. Do not use an ID token to call a protected API unless that API explicitly supports it.

---

6. Authorization Code Flow

The client must obtain tokens without casually exposing them through browser redirects. The Authorization Code Flow handles this through two stages:

1. The browser receives a short-lived authorization code.
2. The backend exchanges that code for tokens through a direct server-to-server request.

Step 1: Start the Authorization Request

The user visits the application and clicks:

```
Continue with Google
```

Coder Army

The application redirects the browser to Google's authorization endpoint.

The authorization request contains values such as:

```
client_id
redirect_uri
scope
response_type=code
state
```

`client_id`

Google gives the application a client ID when it is registered.

```
client_id = coderarmy-app-123
```

It identifies which application is making the request. A client ID is an identifier, not a secret.

`redirect_uri`

This is where Google sends the browser after completing the authentication and authorization process.

For the local Spring Boot application:

```
http://localhost:8080/login/oauth2/code/google
```

The redirect URI must exactly match an authorized URI registered with Google.

`scope`

For basic OIDC login, the application requests:

```
openid
profile
email
```

The `openid` scope tells the provider that this is an OpenID Connect request.

`response_type=code`

This tells the authorization server to return an authorization code rather than placing the final tokens directly in the browser redirect.

`state`

The client generates a random state value before the redirect:

```
Client creates state
        ↓
State travels with authorization request
        ↓
Authorization server returns the same state
        ↓
Client verifies it
```

The state value correlates the callback with the original request and helps protect the authorization flow against attacks such as login CSRF.

Step 2: Google Authenticates the User

The browser is now on Google's domain. Google can authenticate the user with a password, passkey, two-factor authentication or another supported mechanism.

The important security boundary is:

```
Google credentials go from the browser to Google.
They never go to our Spring Boot application.
```

Step 3: The User Gives Consent

Google displays the permissions requested by the application. For a basic login, these may include profile and email information. For a Calendar integration, the screen could also request calendar access.

After approval, Google knows that:

Coder Army

Coder Army

```
The user authenticated successfully
                    +
The user approved the requested access
```

Step 4: Google Returns an Authorization Code

Google redirects the browser to the registered callback:

```
http://localhost:8080/login/oauth2/code/google
    ?code=A7D91XYZ
    &state=abc123
```

The authorization code is a short-lived, temporary credential representing the successful authorization step.

```
Authorization code ≠ Access token
```

The code is not normally used to call a resource API. Its purpose is to be exchanged.

Step 5: The Backend Exchanges the Code for Tokens

The Spring Boot backend contacts Google's token endpoint:

```
Spring Boot backend
    ├── authorization code
    ├── redirect URI
    └── client authentication and/or PKCE proof
                ↓
Google token endpoint
                ↓
Access token + ID token + optional refresh token
```

Front Channel vs Back Channel

Coder Army

Channel	Communication path	Typical data
Front channel	Through the user's browser	Authorization request, code and state
Back channel	Directly between backend and token endpoint	Code exchange and tokens

The authorization code passes through the browser, but the final tokens are obtained through the back channel.

---

7. PKCE from First Principles

An attacker who steals an authorization code may try to exchange it before the legitimate client. PKCE binds the authorization request to the client instance that later performs the token exchange.

PKCE stands for:

```
Proof Key for Code Exchange
```

It is commonly pronounced **pixy**.

Creating the Proof

Before starting authorization, the client generates a long, random value:

```
code_verifier = a-long-random-secret
```

It hashes the verifier, normally using SHA-256:

```
SHA-256(code_verifier)
          ↓
code_challenge
```

During the Authorization Request

The client sends the `code_challenge` to the authorization server but keeps the `code_verifier` locally.

Coder Army

```
Client ── code_challenge ──→ Authorization server
```

The authorization server remembers the challenge associated with that request.

During the Token Exchange

The client later sends:

```
authorization_code
code_verifier
```

The authorization server hashes the received verifier and compares the result with the original challenge:

```
SHA-256(received code_verifier)
                ↓
        calculated challenge
                ↓
calculated challenge == original code_challenge?
```

If they match, the server can issue tokens. If they do not match, the request is rejected.

An attacker may steal the authorization code, but without the verifier the stolen code cannot be successfully exchanged.

> A client secret authenticates a confidential client application. PKCE binds the code exchange to the client instance that initiated the authorization request. They solve related but different problems.

---

8. Registering the Application with Google

Google must know which application is requesting login and which callback addresses it can trust.

```
Register application with Google
            ↓
```

Coder Army

```
Receive Client ID and Client Secret
            ↓
Register allowed redirect URI
```

For this local demo, the authorized redirect URI is:

```
http://localhost:8080/login/oauth2/code/google
```

Spring Security uses this default redirect URI template:

```
{baseUrl}/login/oauth2/code/{registrationId}
```

Here:

```
baseUrl       = http://localhost:8080
registrationId = google
```

---

9. Spring Boot Application Flow

The application uses Google for authentication but maintains its own internal user record:

```
Google authenticates the user
            ↓
Spring receives an OidcUser
            ↓
Look up provider + provider subject
       ┌────┴────┐
   Not found    Found
       ↓          ↓
    Insert    Update/load
       └────┬────┘
            ↓
Internal AppUser
            ↓
Spring Security session
```

Coder Army

Google owns the external identity. Our database owns application-specific information such as:

- Internal user ID
- Application role
- Account status
- Preferences
- Subscription or membership data

---

10. Project Structure

```
oauth2-demo
├── pom.xml
└── src
    └── main
        ├── java
        │   └── in.coderarmy.oauth2demo
        │       ├── OAuth2DemoApplication.java
        │       ├── config
        │       │   └── SecurityConfig.java
        │       ├── controller
        │       │   └── UserController.java
        │       ├── entity
        │       │   └── AppUser.java
        │       ├── repository
        │       │   └── AppUserRepository.java
        │       └── service
        │           ├── AppUserService.java
        │           └── CustomOidcUserService.java
        └── resources
            └── application.properties
```

---

11. Required Dependencies

The OAuth/OIDC client starter is:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId
```

```
>
</dependency>
```

The complete demo also needs Spring Web, Spring Data JPA and a database driver. For a minimal in-memory demo, the relevant dependencies are:

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-oauth2-client</artifa
ctId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

The OAuth2 client starter provides the Spring Security support required for OAuth2 Login. Adding `spring-boot-starter-security` separately is optional when the OAuth2 client starter is already present.

Why is our application called an OAuth client?

Coder Army

Coder Army

```
Spring Boot application
        ↓
Requests authentication and delegated access from Google
```

12. User Entity

```java
package in.coderarmy.oauth2demo.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    private String provider;

    @Column(name = "provider_subject")
    private String providerSubject;

    private String role;

    public AppUser() {
    }
```

Coder Army

```
    public AppUser(
            String name,
            String email,
            String provider,
            String providerSubject,
            String role
    ) {
        this.name = name;
        this.email = email;
        this.provider = provider;
        this.providerSubject = providerSubject;
        this.role = role;
    }

    // Generate getters and setters.
}
```

There is deliberately no password field. Google owns and verifies the Google password; our application never receives it.

Why Store `providerSubject`?

Google includes a `sub` claim in the ID token:

```
{
  "sub": "109324823948234",
  "email": "student@gmail.com",
  "name": "Rohit"
}
```

`sub` means **subject**. It is the provider's stable identifier for the user.

We therefore store:

```
provider = google
```

Coder Army

```
providerSubject = value of sub
```

The combination of provider and subject is a better external identity key than email because an email address may be changed, missing or unverified depending on the provider and scopes.

In a real database, this pair should normally be unique.

13. Repository

```java
package in.coderarmy.oauth2demo.repository;

import in.coderarmy.oauth2demo.entity.AppUser;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface AppUserRepository
        extends JpaRepository<AppUser, Long> {

    Optional<AppUser> findByProviderAndProviderSubject(
            String provider,
            String providerSubject
    );
}
```

This method asks:

```
Does our database already contain a user with:

provider = google
providerSubject = 123456
```

14. Application User Service

Coder Army

The signup operation happens inside our application after Google has authenticated the user.

```
package in.coderarmy.oauth2demo.service;

import in.coderarmy.oauth2demo.entity.AppUser;
import in.coderarmy.oauth2demo.repository.AppUserRepository;
import org.springframework.security.oauth2.core.oidc.user.Oid
cUser;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transaction
al;

import java.util.Optional;

@Service
public class AppUserService {

    private final AppUserRepository appUserRepository;

    public AppUserService(
            AppUserRepository appUserRepository
    ) {
        this.appUserRepository = appUserRepository;
    }

    @Transactional
    public AppUser registerOrUpdate(
            String provider,
            OidcUser oidcUser
    ) {
        String providerSubject = oidcUser.getSubject();
        String name = oidcUser.getClaimAsString("name");
        String email = oidcUser.getClaimAsString("email");

        Optional<AppUser> existingUser =
```

```
                appUserRepository
                        .findByProviderAndProviderSubject(
                                provider,
                                providerSubject
                        );

        if (existingUser.isPresent()) {
            AppUser user = existingUser.get();
            user.setName(name);
            user.setEmail(email);

            // The managed entity is updated through JPA dirt
y checking.
            return user;
        }

        AppUser newUser = new AppUser(
                name,
                email,
                provider,
                providerSubject,
                "USER"
        );

        return appUserRepository.save(newUser);
    }

    public Optional<AppUser> findByProviderAndSubject(
            String provider,
            String subject
    ) {
        return appUserRepository
                .findByProviderAndProviderSubject(
                        provider,
                        subject
                );
```

Coder Army

```
    }
}
```

The database flow is:

```
SELECT *
FROM app_user
WHERE provider = 'google'
  AND provider_subject = '123';
```

- If the user exists, update the latest profile details and load the internal account.
- If the user does not exist, insert a new `AppUser` with the default `USER` role.

This is the application's OAuth/OIDC signup process.

---

15. Custom OIDC User Service

Spring Security already knows how to communicate with an OIDC provider. We should preserve that built-in processing and add only our application-specific database logic.

```
Spring performs normal OIDC processing
                ↓
Spring returns an OidcUser
                ↓
Our code saves or updates the internal user
                ↓
Return the OidcUser to Spring Security
```

```
package in.coderarmy.oauth2demo.service;

import org.springframework.security.oauth2.client.oidc.userin
fo.OidcUserRequest;
import org.springframework.security.oauth2.client.oidc.userin
fo.OidcUserService;
```

Coder Army

Coder Army

```
import org.springframework.security.oauth2.client.userinfo.OA
uth2UserService;
import org.springframework.security.oauth2.core.OAuth2Authent
icationException;
import org.springframework.security.oauth2.core.oidc.user.Oid
cUser;
import org.springframework.stereotype.Service;

@Service
public class CustomOidcUserService implements
        OAuth2UserService<OidcUserRequest, OidcUser> {

    private final OidcUserService delegate;
    private final AppUserService appUserService;

    public CustomOidcUserService(
            AppUserService appUserService
    ) {
        this.appUserService = appUserService;
        this.delegate = new OidcUserService();
    }

    @Override
    public OidcUser loadUser(
            OidcUserRequest userRequest
    ) throws OAuth2AuthenticationException {

        // Let Spring perform the provider interaction and OI
DC processing.
        OidcUser oidcUser = delegate.loadUser(userRequest);

        String provider = userRequest
                .getClientRegistration()
                .getRegistrationId();

        // Run our application-specific signup/update logic.
```

```
            appUserService.registerOrUpdate(provider, oidcUser);

            return oidcUser;
        }
    }
```

The call below represents significant built-in processing:

```
OidcUser oidcUser = delegate.loadUser(userRequest);
```

Conceptually, by this stage:

```
Authorization code has been exchanged
                ↓
Access token and ID token are available
                ↓
Spring validates and processes OIDC information
                ↓
Spring may call the UserInfo endpoint
                ↓
Spring creates an OidcUser
```

---

16. Security Configuration

```
package in.coderarmy.oauth2demo.config;

import in.coderarmy.oauth2demo.service.CustomOidcUserService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.bui
lders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {
```

Coder Army

Coder Army

```
    @Bean
    public SecurityFilterChain securityFilterChain(
            HttpSecurity http,
            CustomOidcUserService customOidcUserService
    ) throws Exception {

        http
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/").permitAll()
                        .anyRequest().authenticated()
                )
                .oauth2Login(oauth -> oauth
                        .userInfoEndpoint(userInfo -> userInf
o
                                .oidcUserService(customOidcUs
erService)
                        )
                        .defaultSuccessUrl("/profile", true)
                );

        return http.build();
    }
}
```

The `oauth2Login()` configuration enables the OAuth 2.0 Login and OIDC flow. Spring Security manages the protocol endpoints involved in:

```
Authorization endpoint → redirect user to Google
Token endpoint         → exchange code for tokens
Redirection endpoint   → /login/oauth2/code/google
UserInfo endpoint      → obtain additional profile claims when required
```

17. Controller

Coder Army

```
package in.coderarmy.oauth2demo.controller;

import in.coderarmy.oauth2demo.entity.AppUser;
import in.coderarmy.oauth2demo.service.AppUserService;
import org.springframework.security.core.annotation.Authentic
ationPrincipal;
import org.springframework.security.oauth2.core.oidc.user.Oid
cUser;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestControlle
r;

import java.util.LinkedHashMap;
import java.util.Map;

@RestController
public class UserController {

    private final AppUserService appUserService;

    public UserController(
            AppUserService appUserService
    ) {
        this.appUserService = appUserService;
    }

    @GetMapping("/")
    public String home() {
        return """
                Public Home

                Login using:
                http://localhost:8080/oauth2/authorization/go
ogle

                """;
```

Coder Army

```
    }

    @GetMapping("/profile")
    public Map<String, Object> profile(
            @AuthenticationPrincipal OidcUser oidcUser
    ) {
        AppUser appUser = appUserService
                .findByProviderAndSubject(
                        "google",
                        oidcUser.getSubject()
                )
                .orElseThrow();

        Map<String, Object> response = new LinkedHashMap<>();

        response.put("internalUserId", appUser.getId());
        response.put("provider", appUser.getProvider());
        response.put("subject", oidcUser.getSubject());
        response.put("name", oidcUser.getClaimAsString("nam
e"));
        response.put("email", oidcUser.getClaimAsString("emai
l"));
        response.put("role", appUser.getRole());

        return response;
    }
}
```

After login, the response may look like:

```
{
  "internalUserId": 1,
  "provider": "google",
  "subject": "108293472983749823749",
  "name": "Rohit Negi",
  "email": "example@gmail.com",
```

```
  "role": "USER"
}
```

The data comes from two different systems:

Data	Source
`subject`, `name`, `email`	Google/OIDC claims
`internalUserId`, `role`	Our application database

18. Client Registration Properties

```
spring.application.name=oauth2-demo

# Google OAuth 2.0 / OIDC
spring.security.oauth2.client.registration.google.client-id=
${GOOGLE_CLIENT_ID}
spring.security.oauth2.client.registration.google.client-secr
et=${GOOGLE_CLIENT_SECRET}
spring.security.oauth2.client.registration.google.scope=openi
d,profile,email

# JPA demo configuration
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
```

Because Google is a provider recognized by Spring Boot, the normal authorization, token and UserInfo endpoint URLs do not need to be configured manually.

Do Not Hardcode the Client Secret

Keep the property placeholders in `application.properties` and supply the actual values through environment variables:

```
GOOGLE_CLIENT_ID
```

Coder Army

Coder Army

```
GOOGLE_CLIENT_SECRET
```

In IntelliJ IDEA:

```
Run
  → Edit Configurations
  → Select the Spring Boot application
  → Environment variables
```

Add both variables there. Spring resolves them at startup:

```
Environment variable
        ↓
${GOOGLE_CLIENT_ID}
        ↓
Spring client registration
```

Never commit a real client secret to Git or show it in a public recording.

---

19. Google Cloud Console Setup

Step 1: Create a Google Cloud Project

Open Google Cloud Console and create a project, for example:

```
spring-oauth-demo
```

Step 2: Open Google Auth Platform

Open **Google Auth Platform** for the project. If the project has not been configured before, select **Get Started**.

Step 3: Configure Branding

Enter the application details:

```
App name: CoderArmy OAuth Demo
```

Coder Army

```
User support email: your-email@gmail.com
```

Also provide the required developer contact information.

Step 4: Select the Audience

For a demo that should work with normal Gmail accounts, choose:

```
External
```

If the application remains in testing mode, add the Google accounts that will test the login flow as test users.

Step 5: Configure Data Access and Scopes

For basic OIDC login, request only:

```
openid
profile
email
```

Follow the principle of least privilege: request only the scopes the application genuinely needs.

Step 6: Create an OAuth Client

Open:

```
Google Auth Platform
  → Clients
  → Create Client
```

Choose:

```
Application type: Web application
```

Step 7: Authorized JavaScript Origins

Coder Army

This server-side Spring Boot demo does not use Google's browser-side JavaScript SDK, so **Authorized JavaScript origins** can remain empty.

Step 8: Add the Authorized Redirect URI

Add this exact URI:

```
http://localhost:8080/login/oauth2/code/google
```

The scheme, host, port and path must match. Even a small mismatch can cause a `redirect_uri_mismatch` error.

Step 9: Create the Client

Google generates:

```
Client ID
Client Secret
```

Store both as environment variables for the Spring Boot application.

---

20. Running and Testing the Application

Start the Spring Boot application and open:

```
http://localhost:8080/
```

To start Google login directly, visit:

```
http://localhost:8080/oauth2/authorization/google
```

The endpoint name contains the registration ID from the properties:

```
google
```

Coder Army

What Spring Does with This Request

```
GET /oauth2/authorization/google
                ↓
Spring finds ClientRegistration "google"
                ↓
Spring constructs the authorization request
                ↓
Spring sends a 302 redirect
                ↓
Browser opens Google's authorization endpoint
```

The generated request includes values such as:

```
client_id
redirect_uri
scope
state
response_type=code
```

Google Authentication

The user enters credentials on Google's domain:

```
Browser → Google
```

The credentials are not submitted to `localhost:8080`.

Callback and Code Exchange

After successful authentication and consent, Google redirects the browser to:

```
http://localhost:8080/login/oauth2/code/google
```

The callback contains a code and the returned state value:

```
/login/oauth2/code/google?code=ABC...&state=XYZ...
```

Coder Army

Spring validates the response and performs the back-channel code exchange with Google's token endpoint. It then processes the OIDC result and creates an `OidcUser`.

The `CustomOidcUserService` saves or updates the corresponding internal `AppUser`, and the success handler redirects to:

```
/profile
```

21. What Authenticates Later `/profile` Requests?

In this traditional server-side OAuth2 Login flow, the browser does **not** send Google's ID token with every request to `/profile`.

After the Google login completes:

```
Spring authenticates the user
            ↓
Authentication is stored in the SecurityContext
            ↓
The SecurityContext is associated with the HTTP session
            ↓
The browser sends the session cookie on later requests
```

This application is therefore session-based after login.

OAuth2 Login and OAuth2 Resource Server are different use cases:

Feature	Typical use
`oauth2Login()`	Log a browser user into the application and usually maintain a session
`oauth2ResourceServer().jwt()`	Protect an API that receives bearer access tokens

If a separate frontend or mobile application must call our API with bearer tokens, that requires a different architecture from this server-side session login demo.

22. Complete Flow Recap

Coder Army

```
1. User opens /oauth2/authorization/google
2. Spring redirects the browser to Google
3. Google authenticates the user
4. User approves the requested scopes
5. Google redirects back with an authorization code
6. Spring exchanges the code through the back channel
7. Spring validates and processes the OIDC response
8. Spring creates an OidcUser
9. CustomOidcUserService saves or updates AppUser
10. Spring stores the authenticated state in the session
11. The user accesses /profile as an authenticated user
```

Final Mental Model

```
Google password
→ stays with Google

Authorization code
→ short-lived value returned through the browser

Access token
→ used to access a protected resource

ID token
→ tells the client who authenticated

Refresh token
→ obtains new access tokens

provider + subject
→ links the Google identity to our internal user

session cookie
→ keeps the browser authenticated with our Spring application
```

> Google proves the external identity. Our application still owns its internal user record, roles and business rules.

Coder Army

Coder Army

Spring Testing with JUnit and Mockito

1. Why Testing Exists

Consider the following method:

```
public int calculateFinalPrice(
        int price,
        int discountPercentage) {

    return price -
            (price * discountPercentage / 100);
}
```

The method appears to work, but writing code and proving that it behaves correctly are two different activities.

We can manually test it with:

```
Price = 1000
Discount = 20
Expected result = 800
```

Even if this input returns `800`, several questions remain:

- What happens when the discount is `0`?
- What happens when the discount is `100`?
- What happens when the discount is negative?
- What happens when the discount is greater than `100`?
- What happens if someone changes the formula later?

Testing verifies that the behaviour of the code matches its requirements.

Coder Army

Manual Testing vs Automated Testing

Manual Testing

A developer manually:

1. Starts the application.
2. Prepares a request.
3. Calls the endpoint.
4. Inspects the response.
5. Decides whether the result is correct.

This can work for a small application, but it becomes slow and unreliable as the application grows.

Automated Testing

An automated test:

1. Provides a known input.
2. Executes the production code.
3. Compares the actual result with the expected result.
4. Reports whether the test passed or failed.

```
Known input
    ↓
Execute production code
    ↓
Actual output
    ↓
Compare with expected output
    ↓
Pass or fail
```

What Is a Test Case?

A test case contains three fundamental parts:

Coder Army

```
Initial condition
Action
Expected result
```

Example:

```
Initial condition:
Price is ₹1000 and discount is 20%

Action:
Calculate the final price

Expected result:
Final price should be ₹800
```

In testing terminology, this is commonly called **Arrange, Act and Assert**.

Arrange

Prepare the data and environment required by the test.

```
int price = 1000;
int discount = 20;
```

Act

Execute the behaviour being tested.

```
int actualPrice =
        calculator.calculateFinalPrice(price, discount);
```

Assert

Compare the actual result with the expected result.

```
assertEquals(800, actualPrice);
```

The complete test looks like this:

Coder Army

```
@Test
void shouldApplyDiscountToPrice() {

    // Arrange
    int price = 1000;
    int discount = 20;

    // Act
    int actualPrice =
            calculator.calculateFinalPrice(
                    price,
                    discount
            );

    // Assert
    assertEquals(800, actualPrice);
}
```

Tests Verify Behaviour

Assume the requirement is:

> A discount must be between 0 and 100.

This requirement introduces multiple behaviours:

- A valid discount calculates the final price.
- A discount below `0` is rejected.
- A discount above `100` is rejected.
- A negative price is rejected.

The production code can enforce these rules:

```
public int calculateFinalPrice(
        int price,
```

Coder Army

```
        int discountPercentage) {

    if (price < 0) {
        throw new IllegalArgumentException(
                "Price cannot be negative"
        );
    }

    if (discountPercentage < 0 ||
            discountPercentage > 100) {

        throw new IllegalArgumentException(
                "Discount must be between 0 and 100"
        );
    }

    return price -
            (price * discountPercentage / 100);
}
```

Tests should be written for the behaviours promised by a method, not simply for its lines of code.

Unit Tests and Integration Tests

Consider a common Spring Boot application flow:

```
Controller → Service → Repository → Database
```

Unit Test

A unit test verifies a small unit of behaviour in isolation.

For example, a test for `ProductService` should not require:

- The Spring `ApplicationContext`
- Tomcat

Coder Army

- A real database
- An HTTP request

If `ProductService` depends on `ProductRepository`, the real repository can be replaced with a controlled test double.

Integration Test

An integration test checks whether multiple real components work together.

Examples:

```
Repository + Hibernate + H2/MySQL
```

```
Controller + Service + Repository + Database
```

The important question is not only which annotation is used. The real question is:

> How much real application infrastructure participates in the test?

Test type	What is real?	Relative speed	Main purpose
Unit test	One class or unit of behaviour	Fastest	Verify business logic
Slice test	One application layer	Fast	Verify a specific Spring layer
Integration test	Multiple components	Slower	Verify that components work together
End-to-end test	Complete application flow	Slowest	Verify the complete user journey

Testing Pyramid

A healthy test suite generally contains:

```
Many fast unit tests
Fewer integration tests
Very few end-to-end tests
```

Coder Army

If every test starts Spring Boot and connects to a database, the test suite becomes slow. Developers are then less likely to run it frequently.

Unit tests provide fast feedback. Integration tests confirm that individual units are connected correctly. Both are necessary because they solve different problems.

What Should Be Unit Tested?

Prioritize code that contains behaviour or decisions:

- Business calculations
- Conditional branches
- Data transformations
- Validation rules
- Exception conditions
- Authorization decisions
- Service-layer orchestration
- Important edge cases

Usually avoid spending time unit-testing:

- Plain getters and setters
- Framework behaviour
- Built-in methods of `JpaRepository`
- Code with no meaningful decision or transformation

There is no need to prove that Spring Data's `save()` method works. The important part is whether the application calls `save()` under the correct business conditions.

---

2. JUnit Fundamentals

What Is JUnit?

JUnit provides:

Coder Army

- A programming model for declaring tests
- Annotations such as `@Test`
- Assertions such as `assertEquals()`
- Test discovery and execution
- Integration with Maven, Gradle and IDEs

The annotations used in modern Spring Boot tests belong to JUnit Jupiter:

```
org.junit.jupiter.api.Test
```

Testing Dependency

A Spring Boot project normally includes:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

The `test` scope means that these libraries are available for compiling and running tests, but they are not included as normal production dependencies.

`spring-boot-starter-test` provides commonly used testing libraries and utilities, including:

- JUnit
- Mockito
- AssertJ
- Hamcrest
- Spring Test
- JSON testing utilities

Coder Army

Test Directory Structure

Production code belongs in:

```
src/main/java
```

Test code belongs in:

```
src/test/java
```

The test package should normally mirror the production package:

```
src/main/java
└── com.example.demo.service
    └── ProductService.java

src/test/java
└── com.example.demo.service
    └── ProductServiceTest.java
```

This keeps the production class and its corresponding test easy to locate.

Writing the First JUnit Test

Production Class

```java
public class PriceCalculator {

    public int calculateFinalPrice(
            int price,
            int discountPercentage) {

        if (price < 0) {
            throw new IllegalArgumentException(
                    "Price cannot be negative"
            );
```

Coder Army

```
        }

        if (discountPercentage < 0 ||
                discountPercentage > 100) {

            throw new IllegalArgumentException(
                    "Discount must be between 0 and 100"
            );
        }

        return price -
                (price * discountPercentage / 100);
    }
}
```

Test Class

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class PriceCalculatorTest {

    private PriceCalculator calculator;

    @BeforeEach
    void setUp() {
        calculator = new PriceCalculator();
    }

    @Test
    void shouldApplyDiscountToPrice() {

        // Arrange
```

Coder Army

```
        int price = 1000;
        int discount = 20;

        // Act
        int actualPrice =
                calculator.calculateFinalPrice(
                        price,
                        discount
                );

        // Assert
        assertEquals(800, actualPrice);
    }
}
```

This test does not use:

```
@SpringBootTest
@Autowired
```

It also does not start the Spring `ApplicationContext`. `PriceCalculator` is a regular Java class, so it can be created with `new`.

Spring Boot applications are still Java applications. The complete Spring container should not be started when the behaviour can be tested directly.

Understanding `@Test`

```
@Test
void shouldApplyDiscountToPrice() {
}
```

`@Test` tells JUnit to discover and execute the method as a test case.

Test method names should describe the expected behaviour.

Good names:

Coder Army

```
shouldApplyDiscountToPrice()
shouldThrowExceptionWhenDiscountIsNegative()
shouldReturnOriginalPriceWhenDiscountIsZero()
```

Avoid vague names:

```
test1()
checkMethod()
calculatorTest()
```

Assertions

An assertion describes a condition that must be true for the test to pass.

`assertEquals()`

```
assertEquals(expectedValue, actualValue);
```

```
assertEquals(800, actualPrice);
```

`assertTrue()` and `assertFalse()`

```
assertTrue(result.isActive());
assertFalse(result.isDeleted());
```

`assertNull()` and `assertNotNull()`

```
assertNull(product.getId());
assertNotNull(savedProduct.getId());
```

`assertThrows()`

Use `assertThrows()` when an exception is the expected behaviour.

Coder Army

```
@Test
void shouldRejectDiscountAboveHundred() {

    IllegalArgumentException exception =
            assertThrows(
                    IllegalArgumentException.class,
                    () -> calculator.calculateFinalPrice(
                            1000,
                            120
                    )
            );

    assertEquals(
            "Discount must be between 0 and 100",
            exception.getMessage()
    );
}
```

The test passes only when the specified code throws `IllegalArgumentException`. If no exception is thrown, the test fails.

`@BeforeEach` and `@AfterEach`

`@BeforeEach`

`@BeforeEach` runs before every test method.

```
@BeforeEach
void setUp() {
    calculator = new PriceCalculator();
}
```

It is useful for preparation shared by multiple tests.

```
setUp()
testOne()
```

Coder Army

```
setUp()
testTwo()

setUp()
testThree()
```

Every test begins with a predictable state.

`@AfterEach`

`@AfterEach` runs after every test method.

```
@AfterEach
void cleanUp() {
    // Release test resources if required
}
```

It is useful when a test manually opens a resource that must be closed or cleaned. There is no need to add an empty `@AfterEach` merely because the annotation exists.

Running Tests

Tests can be run directly from IntelliJ.

To run the complete Maven test suite:

```
./mvnw test
```

If Maven is installed globally:

```
mvn test
```

```
Green → Expected behaviour was observed
Red   → Expected and actual behaviour did not match
```

A failing test provides useful information. It may mean that:

Coder Army

- The production code is incorrect.
- The test's expectation is incorrect.
- The requirement has changed.

---

3. Unit Testing the Service Layer with Mockito

The Dependency Problem

Consider the following service:

```
@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(
            ProductRepository productRepository) {

        this.productRepository = productRepository;
    }

    public Product getProductById(Long id) {

        return productRepository.findById(id)
                .orElseThrow(() ->
                        new ProductNotFoundException(
                                "Product not found: " + id
                        )
                );
    }

    public Product createProduct(Product product) {
```

Coder Army

```
        boolean alreadyExists =
                productRepository.existsByName(
                        product.getName()
                );

        if (alreadyExists) {
            throw new IllegalArgumentException(
                    "Product already exists"
            );
        }

        return productRepository.save(product);
    }
}
```

Repository:

```
public interface ProductRepository
        extends JpaRepository<Product, Long> {

    boolean existsByName(String name);
}
```

`ProductService` cannot be created without a `ProductRepository`:

```
ProductService service =
        new ProductService(???);
```

Using the real repository could also require:

- A Spring Data proxy
- `EntityManager`
- Hibernate
- `DataSource`

- A database

A failure could then come from the database or Hibernate configuration instead of the service logic. That would no longer be an isolated unit test.

The dependency should therefore be replaced with a controlled test double.

Test Doubles

A test double is an object used in place of a real dependency during testing.

Dummy

An object passed only because a method requires it. Its behaviour is not used by the test.

Stub

An object that returns predefined answers.

```
When findById(1) is called,
return this predefined product.
```

Fake

A simplified but working implementation, such as an in-memory repository backed by a `HashMap` instead of MySQL.

Mock

A configurable test double that can:

- Return predefined values
- Throw predefined exceptions
- Record method calls
- Verify how it was used

Mockito creates mocks without requiring manually written fake repository classes.

Configuring Mockito with JUnit

CoderArmy

Coder Army

```
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class ProductServiceTest {

    @Mock
    private ProductRepository productRepository;

    @InjectMocks
    private ProductService productService;
}
```

`@ExtendWith(MockitoExtension.class)`

JUnit knows how to execute tests, but it does not process Mockito annotations by itself.

`MockitoExtension` connects Mockito's lifecycle with JUnit Jupiter and initializes fields annotated with:

```
@Mock
@InjectMocks
```

`@Mock`

```
@Mock
private ProductRepository productRepository;
```

Mockito creates an object that implements `ProductRepository`. This object is not:

- A Spring bean
- A Spring Data proxy

Coder Army

- A real repository
- Connected to a database

It is a controlled replacement used only by the test.

`@InjectMocks`

```
@InjectMocks
private ProductService productService;
```

Mockito creates `ProductService` and injects the available mock into its constructor.

Conceptually, it performs something similar to:

```
ProductRepository repository =
        Mockito.mock(ProductRepository.class);

ProductService productService =
        new ProductService(repository);
```

`@InjectMocks` is a Mockito convenience mechanism. It is not Spring dependency injection.

Testing the Success Scenario

```
@ExtendWith(MockitoExtension.class)
class ProductServiceTest {

    @Mock
    private ProductRepository productRepository;

    @InjectMocks
    private ProductService productService;

    @Test
    void shouldReturnProductWhenProductExists() {
```

Coder Army

```
        // Arrange
        Product product =
                new Product(
                        1L,
                        "Laptop",
                        10
                );

        when(productRepository.findById(1L))
                .thenReturn(Optional.of(product));

        // Act
        Product result =
                productService.getProductById(1L);

        // Assert
        assertEquals(1L, result.getId());
        assertEquals("Laptop", result.getName());

        verify(productRepository)
                .findById(1L);
    }
}
```

The following stubbing instruction controls the mock repository:

```
when(productRepository.findById(1L))
        .thenReturn(Optional.of(product));
```

It means:

```
If findById(1L) is called on this mock repository,
return the predefined Product.
```

Coder Army

Mockito is not testing the repository. It is creating a predictable environment in which the service can be tested.

Stubbing, Assertion and Verification

These three concepts have different responsibilities.

Stubbing

```
when(repository.findById(1L))
        .thenReturn(Optional.of(product));
```

Defines how the dependency should behave during the test.

Assertion

```
assertEquals("Laptop", result.getName());
```

Checks the observable result produced by the service.

Verification

```
verify(repository).findById(1L);
```

Checks whether the service interacted with its dependency as expected.

```
when()   → Control the dependency
assert() → Check the result
verify() → Check the interaction
```

`verify()` does not need to be called for every mocked method. Use it when the interaction itself is part of the behaviour being tested.

Testing the Exception Scenario

Coder Army

```
@Test
void shouldThrowExceptionWhenProductDoesNotExist() {

    // Arrange
    when(productRepository.findById(99L))
            .thenReturn(Optional.empty());

    // Act
    ProductNotFoundException exception =
            assertThrows(
                    ProductNotFoundException.class,
                    () -> productService
                            .getProductById(99L)
            );

    // Assert
    assertEquals(
            "Product not found: 99",
            exception.getMessage()
    );

    verify(productRepository)
            .findById(99L);
}
```

The repository is instructed to report that the product is absent. The test then verifies that the service throws `ProductNotFoundException`.

This checks the application's business rule, not the database.

Testing Product Creation

Successful Creation

Coder Army

```java
@Test
void shouldCreateProductWhenNameIsUnique() {

    // Arrange
    Product request =
            new Product(
                    null,
                    "Keyboard",
                    20
            );

    Product savedProduct =
            new Product(
                    10L,
                    "Keyboard",
                    20
            );

    when(productRepository.existsByName("Keyboard"))
            .thenReturn(false);

    when(productRepository.save(request))
            .thenReturn(savedProduct);

    // Act
    Product result =
            productService.createProduct(request);

    // Assert
    assertEquals(10L, result.getId());
    assertEquals("Keyboard", result.getName());

    verify(productRepository)
            .existsByName("Keyboard");
```

```
    verify(productRepository)
            .save(request);
}
```

Duplicate Product

```
@Test
void shouldRejectProductWhenNameAlreadyExists() {

    // Arrange
    Product request =
            new Product(
                    null,
                    "Keyboard",
                    20
            );

    when(productRepository.existsByName("Keyboard"))
            .thenReturn(true);

    // Act and Assert
    assertThrows(
            IllegalArgumentException.class,
            () -> productService.createProduct(request)
    );

    verify(productRepository)
            .existsByName("Keyboard");

    verify(productRepository, never())
            .save(any(Product.class));
}
```

The following verification is important:

Coder Army

Coder Army

```
verify(productRepository, never())
        .save(any(Product.class));
```

The business rule states that a duplicate product must not be saved. The absence of the `save()` call is therefore part of the required behaviour.

Argument Matchers

Exact-Value Stubbing

```
when(productRepository.findById(1L))
        .thenReturn(Optional.of(product));
```

Flexible Stubbing

```
when(productRepository.save(any(Product.class)))
        .thenReturn(savedProduct);
```

Common Mockito matchers include:

```
any()
anyLong()
anyString()
eq(value)
```

Example:

```
verify(productRepository)
        .save(any(Product.class));
```

Use exact values when the exact argument matters. Use matchers when the test only cares about the argument's type or category.

When a matcher is used for one argument of a multi-argument method, use matchers for all arguments:

Coder Army

```
when(repository.search(
        eq("Laptop"),
        eq(true)
)).thenReturn(products);
```

4. Testing Spring MVC Controllers

What Should a Controller Test Verify?

Consider the following controller:

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(
            ProductService productService) {

        this.productService = productService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProduct(
            @PathVariable Long id) {

        Product product =
                productService.getProductById(id);

        return ResponseEntity.ok(product);
    }

    @PostMapping
```

Coder Army

```
    public ResponseEntity<Product> createProduct(
            @Valid
            @RequestBody Product product) {

        Product savedProduct =
                productService.createProduct(product);

        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(savedProduct);
    }
}
```

The controller is responsible for the HTTP contract:

- Which URL is accepted?
- Which HTTP method is accepted?
- How is JSON converted into Java?
- Are path variables and request parameters read correctly?
- Is validation executed?
- Which status code is returned?
- Which JSON is returned?
- How are exceptions converted into HTTP responses?
- Does security permit or reject the request?

A controller test should not repeat all service business-logic tests. That behaviour belongs in the service unit tests.

Why Not Call the Controller Method Directly?

It is possible to write:

```
ProductController controller =
        new ProductController(mockService);
```

Coder Army

```
ResponseEntity<Product> response =
        controller.getProduct(1L);
```

This tests the Java method, but it does not verify Spring MVC behaviour such as:

- `@GetMapping("/{id}")`
- `@PathVariable` conversion
- `@RequestBody` deserialization
- `@Valid`
- HTTP status handling
- JSON serialization
- Spring Security filters
- Global exception handlers

The controller should therefore be tested through Spring MVC's request-processing infrastructure.

`@WebMvcTest`

```
@WebMvcTest(ProductController.class)
class ProductControllerTest {
}
```

`@WebMvcTest` creates a limited Spring `ApplicationContext` containing components relevant to the web layer.

```
Loaded:

Controller
DispatcherServlet
Jackson JSON conversion
Validation
Controller advice
MVC configuration
Security filters
```

Coder Army

```
Not loaded:

Real service
Real repository
Hibernate
Database
Complete application
```

This is known as a **test slice**: only the part of the application required by the test is loaded.

`MockMvc`

```java
@Autowired
private MockMvc mockMvc;
```

`MockMvc` creates an HTTP-like request without starting a real web server.

```
Mock HTTP request
        ↓
Spring Security filter chain
        ↓
DispatcherServlet
        ↓
Controller
        ↓
HTTP-like response
```

There is no:

```
localhost
Port 8080
Running Tomcat server
Postman
```

Spring MVC still processes the request using its real web infrastructure.

`@Mock` vs `@MockitoBean`

The service unit test used:

Coder Army

```
@Mock
private ProductRepository productRepository;
```

That test did not have a Spring context, so Mockito created a normal mock field.

In a controller test, Spring creates `ProductController`, whose constructor requires a `ProductService` bean. The mock must therefore be placed inside Spring's test context:

```
@MockitoBean
private ProductService productService;
```

`@MockitoBean` creates or replaces the corresponding bean in the Spring test `ApplicationContext` with a Mockito mock.

Annotation	Managed by	Placed in the Spring context?
`@Mock`	Mockito	No
`@MockitoBean`	Spring Test and Mockito	Yes

For current Spring versions, use `@MockitoBean`. Older Spring Boot projects may use the now-deprecated `@MockBean`.

Testing a GET Endpoint

```
@WebMvcTest(ProductController.class)
class ProductControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private ProductService productService;

    @Test
    @WithMockUser
    void shouldReturnProductWhenProductExists()
```

Coder Army

```
            throws Exception {

        // Arrange
        Product product =
                new Product(
                        1L,
                        "Laptop",
                        10
                );

        when(productService.getProductById(1L))
                .thenReturn(product);

        // Act and Assert
        mockMvc.perform(
                        get("/api/products/1")
                                .accept(
                                        MediaType.APPLICATION
_JSON
                                )
                )
                .andExpect(
                        status().isOk()
                )
                .andExpect(
                        content().contentTypeCompatibleWith(
                                MediaType.APPLICATION_JSON
                        )
                )
                .andExpect(
                        jsonPath("$.id").value(1)
                )
                .andExpect(
                        jsonPath("$.name").value("Laptop")
                )
                .andExpect(
```

```
                        jsonPath("$.stock").value(10)
                );

        verify(productService)
                .getProductById(1L);
    }
}
```

The test sends:

```
GET /api/products/1
```

Spring MVC must:

1. Match the request with `@GetMapping("/{id}")`.
2. Extract `1` from the URL.
3. Convert it into a `Long`.
4. Call the controller method.
5. Serialize the returned `Product` into JSON.
6. Generate an HTTP response.

`perform()`

```
mockMvc.perform(
        get("/api/products/1")
);
```

Creates and executes the mock request.

`status()`

```
.andExpect(status().isOk());
```

Verifies that the response status is `200 OK`.

Coder Army

Coder Army

`content()`

```
.andExpect(
        content().contentTypeCompatibleWith(
                MediaType.APPLICATION_JSON
        )
);
```

Verifies that the response content represents JSON.

`jsonPath()`

```
.andExpect(
        jsonPath("$.name").value("Laptop")
);
```

For the following response:

```
{
  "id": 1,
  "name": "Laptop",
  "stock": 10
}
```

The JSON paths are:

```
$.id
$.name
$.stock
```

Testing a POST Endpoint

```
@Test
@WithMockUser
void shouldCreateProductWhenRequestIsValid()
```

```
        throws Exception {

    // Arrange
    Product savedProduct =
            new Product(
                    10L,
                    "Keyboard",
                    20
            );

    when(productService.createProduct(
            any(Product.class)
    )).thenReturn(savedProduct);

    String requestBody = """
            {
              "name": "Keyboard",
              "stock": 20
            }
            """;

    // Act and Assert
    mockMvc.perform(
                    post("/api/products")
                            .with(csrf())
                            .contentType(
                                    MediaType.APPLICATION_JSO
N
                            )
                            .content(requestBody)
            )
            .andExpect(
                    status().isCreated()
            )
            .andExpect(
                    jsonPath("$.id").value(10)
```

Coder Army

```
                )
                .andExpect(
                        jsonPath("$.name").value("Keyboard")
                )
                .andExpect(
                        jsonPath("$.stock").value(20)
                );

        verify(productService)
                .createProduct(any(Product.class));
}
```

This test verifies that:

- The `POST` mapping works.
- JSON is converted into a `Product` object.
- The controller calls the service.
- The response status is `201 Created`.
- The saved product is converted into response JSON.

For unsafe HTTP methods such as `POST`, `PUT` and `DELETE`, include:

```
.with(csrf())
```

when CSRF protection is enabled.

Handling Spring Security

If Spring Security protects the endpoint, an unauthenticated request may return:

```
401 Unauthorized
```

Use `@WithMockUser` to execute a test as an authenticated user:

Coder Army

Coder Army

```
@Test
@WithMockUser(
        username = "aditya",
        roles = "ADMIN"
)
void shouldAllowAdminToCreateProduct() {
}
```

For Spring Boot 4, add the technology-specific test starter:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security-test</artifactId
>
    <scope>test</scope>
</dependency>
```

5. Repository and Integration Testing

Why Repository Tests Use a Database

Mocking a repository does not prove that the real repository query works:

```
ProductRepository repository =
        mock(ProductRepository.class);

when(repository.findByNameIgnoreCase("laptop"))
        .thenReturn(Optional.of(product));
```

This only proves that Mockito returns the value configured in the test. It does not execute or validate the real query.

`@DataJpaTest`

Coder Army

```
@DataJpaTest
class ProductRepositoryTest {
}
```

`@DataJpaTest` loads the JPA-related slice of the application.

```
Loaded:

Entities
Spring Data repositories
Hibernate
EntityManager
DataSource
Embedded database

Not loaded:

Controllers
Normal service beans
Web server
Complete application
```

When an embedded database is present, Spring Boot configures it for the test. Data JPA tests are transactional and their changes are rolled back after each test.

Adding H2 for Tests

H2 is an embedded database that runs inside the test process.

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
</dependency>
```

Benefits of H2 for tests:

- No separate installation
- No database server to start

Coder Army

- Fast test execution
- A fresh and isolated test environment

Quick Comparison

Area under test	Main tool or annotation	Real components	Replaced components	Main confidence provided
Plain Java logic	JUnit and `@Test`	Class under test	None	Calculations, validation and edge cases behave correctly
Service layer	Mockito, `@Mock`, `@InjectMocks`	Service	Repository or other dependencies	Business logic and service orchestration behave correctly
MVC controller	`@WebMvcTest`, `MockMvc`, `@MockitoBean`	Spring MVC web layer	Service	HTTP mappings, JSON, status codes, validation and security behave correctly
Repository	`@DataJpaTest` and H2	Repository, Hibernate and database	Other application layers	Entity mappings and repository queries work correctly

Core Principle

Use the smallest test environment that can prove the behaviour being tested:

```
Plain Java behaviour → JUnit
Service with dependencies → JUnit + Mockito
Web-layer contract → @WebMvcTest + MockMvc
Repository query or mapping → @DataJpaTest + Database
```

Fast unit tests and focused integration tests work together to provide reliable feedback without making the test suite unnecessarily slow.

Coder Army

Coder Army

Building GateKeeper: API Management & Control Plane

In this part of the GateKeeper project, we start building the **API Management Service**.

GateKeeper consists of three Spring Boot microservices:

```
api-management-service
smart-gateway-service
analytics-service
```

Each service has a very different responsibility:

```
                    GateKeeper

      API Management   Smart Gateway      Analytics
      --------------   --------------     ---------
      Configuration    Live Traffic       Observations
      Control Plane    Data Plane         Reporting
```

The goal is not simply to create three Spring Boot applications. The important part is deciding **which problem each service owns**.

---

1. GateKeeper Architecture

API Management Service

The API Management Service stores the configuration that tells the Gateway **what it should do**.

Examples:

- Which organizations exist?

Coder Army

- Which routes belong to an organization?
- Where should a route forward traffic?
- Which HTTP methods are allowed?
- What is the rate limit?
- What is the response timeout?
- Is a route active?
- Which headers should be added or removed?

This service represents the **Control Plane** of GateKeeper.

Smart Gateway Service

The Smart Gateway handles the actual application traffic.

It reads configuration created through API Management and applies it to incoming requests.

Examples of its future responsibilities include:

- Route matching
- Request forwarding
- Rate limiting
- Header manipulation
- Timeout enforcement
- Idempotency checks

This service represents the **Data Plane**.

Analytics Service

The Analytics Service records and summarizes what happened while traffic passed through GateKeeper.

For example:

- Number of requests

Coder Army

- Response times
- Failed requests
- Rate-limit violations
- Route usage

Its responsibility is primarily **observation and reporting**.

---

2. Important Terminology

Before designing the database or APIs, we need a common vocabulary.

Organization

A company that uses GateKeeper.

For example:

```
FoodFast
QuickShop
```

In this project, an organization is also our **tenant**.

There is no separate `Tenant` entity.

We generally use:

- `organization` when talking about the database or business model
- `tenant` when talking about ownership and isolation

Management User

A human or machine identity allowed to configure or read GateKeeper.

Examples:

```
platform-admin
foodfast-admin
gateway-service
```

Coder Army

These users belong to GateKeeper itself.

Tenant Customer

A customer of an application using GateKeeper.

For example:

```
FoodFast customer
QuickShop customer
```

A FoodFast customer is **not a GateKeeper management user**.

This distinction becomes extremely important when implementing authentication.

Route

A rule describing how a public API path maps to a backend service.

Example:

```
/foodfast/orders/**
        ↓
FoodFast Order Service
```

A route can also contain policies such as:

- Allowed methods
- Rate limits
- Timeouts
- Header rules
- Idempotency configuration

Control Plane

The part of the system where Gateway configuration is created and managed.

In GateKeeper:

Coder Army

```
API Management Service = Control Plane
```

Data Plane

The part of the system through which real application requests travel.

In GateKeeper:

```
Smart Gateway Service = Data Plane
```

A simple way to remember the difference:

```
Control Plane → decides what should happen
Data Plane    → makes it happen
```

3. Understanding Multi-Tenancy

Suppose two companies are using the same GateKeeper installation.

FoodFast

```
Authentication Service :8091
Restaurant Service     :8092
Order Service          :8093
```

QuickShop

```
Authentication Service :8101
Catalog Service        :8102
Order Service          :8103
```

Both companies may expose an `/orders` API.

That immediately creates a problem.

Coder Army

How can one shared Gateway distinguish between:

```
FoodFast /orders
```

and

```
QuickShop /orders
```

One approach is to make the tenant part of the public path.

```
/foodfast/orders/**
/quickshop/orders/**
```

The first path segment now identifies the tenant.

```
/foodfast/...  → FoodFast
/quickshop/... → QuickShop
```

This also makes tenant ownership explicit while resolving routes.

---

4. Actors in the GateKeeper System

GateKeeper has four important types of actors.

```
PLATFORM_ADMIN
    ↓
Owns and operates GateKeeper

TENANT_ADMIN
    ↓
Manages one organization's Gateway configuration

GATEWAY_SERVICE
    ↓
Machine account used by Smart Gateway

TENANT CUSTOMER
```

```
    ↓
Uses FoodFast/QuickShop, not GateKeeper Management
```

Understanding these actors before designing authentication prevents multiple security problems later.

---

5. Platform Administrator

The `PLATFORM_ADMIN` operates GateKeeper itself.

It can perform operations such as:

- Create organizations
- Create the first administrator for an organization
- Activate organizations
- Suspend organizations
- Soft-delete organizations
- View organizations
- View infrastructure health

For example:

```
Platform Admin
     |
     +---- Create FoodFast
     |
     +---- Create QuickShop
```

However, the platform administrator is **not responsible for FoodFast's business operations**.

It should not:

- Place FoodFast orders
- Access customer accounts
- Authenticate FoodFast customers

Coder Army

- Manage FoodFast's restaurant data

GateKeeper manages API infrastructure, not the tenant application's business domain.

---

6. Tenant Administrator

Each organization has a `TENANT_ADMIN`.

For example:

```
FoodFast
    └── foodfast-admin

QuickShop
    └── quickshop-admin
```

A tenant administrator logs into GateKeeper Management and manages Gateway configuration belonging to **only its own organization**.

It can configure things such as:

- Routes
- Allowed HTTP methods
- Timeouts
- Rate limits
- Idempotency
- Header rules
- Route activation

For example:

```
foodfast-admin
      |
      └── manages FoodFast routes
```

It should never be able to modify:

Coder Army

Coder Army

```
QuickShop routes
```

This is the core idea behind **tenant isolation**.

For this project, the model is intentionally simplified to:

```
1 Organization → 1 Tenant Administrator
```

Features such as the following are outside the current scope:

- Multiple developers
- Viewer roles
- Invitations
- Billing users
- Subscription plans

---

7. Gateway Service Account

Smart Gateway also needs to communicate with API Management.

Instead of giving it a human account, we create a machine identity:

```
GATEWAY_SERVICE
```

Its job is simple:

```
Smart Gateway
      |
      | asks for active configuration
      ↓
API Management
```

The Gateway Service account can **read configuration**.

It cannot modify it.

Coder Army

For example:

```
Allowed:
GET active routes

Not allowed:
POST route
PUT route
DELETE route
```

This follows the principle of least privilege.

---

8. Tenant Customers Are Different From Management Users

This is one of the most important security boundaries in the project.

Suppose a FoodFast customer places an order.

That user might have a token issued by FoodFast:

```
FoodFast Customer
       |
       | FoodFast JWT
       ↓
Smart Gateway
       |
       | token passed forward
       ↓
FoodFast Backend
```

GateKeeper does **not** turn that customer into a GateKeeper management user.

The customer does not log into:

```
GateKeeper Management
```

Instead, the customer's credential belongs to FoodFast.

Coder Army

9. Who Validates What?

There are two different types of validation happening.

GateKeeper Validation

GateKeeper is responsible for Gateway policies.

It can ask questions such as:

- Does a route match this request?
- Is the route active?
- Has the rate limit been exceeded?
- Is this HTTP method allowed?
- Does an idempotent POST contain an `Idempotency-Key`?
- Which headers should be added?
- Which headers should be removed?
- What timeout should be used?

These are **Gateway concerns**.

Tenant Authentication

FoodFast is responsible for questions such as:

- Is this customer logged in?
- Is the JWT valid?
- Can this user access order `123`?
- Does this customer own the payment method?
- Can this user cancel this order?

These are **application concerns**.

Therefore:

Coder Army

```
GateKeeper
    ↓
Validates Gateway policy

FoodFast
    ↓
Validates FoodFast user identity and authorization
```

This creates a clean trust boundary.

---

10. Management Credentials Are Never Customer Credentials

Consider:

```
foodfast-admin
```

This account exists only to configure FoodFast inside GateKeeper.

It is not sent when a customer places an order.

The actual request looks conceptually like:

```
FoodFast Customer
      |
      | customer's FoodFast token
      ↓
Smart Gateway
      |
      ↓
FoodFast Backend
```

Not:

```
FoodFast Customer
      |
      | foodfast-admin credential ❌
```

```
      ↓
Smart Gateway
```

Administrative configuration and customer authentication are completely separate concerns.

---

11. Designing the Database From First Principles

Instead of starting by creating JPA entities, start with the questions the system needs to answer.

This helps the database structure emerge naturally from the business requirements.

---

12. Organizations

Question

> Which companies are using GateKeeper?

We need an `organizations` table.

```
organizations
    id
    name
    slug
    status
    created_at
    updated_at
    deleted_at
```

Example:

```
id       = 1
name     = FoodFast
```

Coder Army

```
slug     = foodfast
status   = ACTIVE
```

Coder Army

Why Have Both `name` and `slug`?

The name is meant for humans.

```
name = FoodFast
```

The slug is a URL-safe identity.

```
slug = foodfast
```

This makes routes like this possible:

```
/foodfast/orders/**
```

13. Organization Status

A simple boolean such as:

```
active = true/false
```

does not describe enough states.

Instead, an organization has a status.

```
PENDING
ACTIVE
SUSPENDED
DELETED
```

PENDING

The organization has been provisioned but is not ready to use the system.

ACTIVE

The organization can use GateKeeper Management and its Gateway configuration.

SUSPENDED

The organization temporarily cannot use the system.

The data still exists and the organization can potentially be reactivated later.

DELETED

The organization has been soft-deleted but its information can still be retained for historical purposes.

This is more expressive than a simple boolean.

---

14. Management Users and Roles

Question

> Who is allowed to operate the GateKeeper control plane?

We need roles such as:

```
PLATFORM_ADMIN
TENANT_ADMIN
GATEWAY_SERVICE
```

And a table for management identities:

```
management_users
    id
    organization_id
    role_id
    username
    password_hash
    enabled
```

Coder Army

Coder Army

15. Why Is `organization_id` Nullable?

Not every management user belongs to an organization.

For example:

```
platform-admin
    → organization_id = NULL

gateway-service
    → organization_id = NULL

foodfast-admin
    → organization_id = FoodFast

quickshop-admin
    → organization_id = QuickShop
```

Platform-level identities belong to GateKeeper itself.

Tenant administrators belong to a particular organization.

---

16. One Tenant Administrator Per Organization

For this version of GateKeeper, each organization has only one tenant administrator.

Therefore, a uniqueness constraint can be applied to:

```
management_users.organization_id
```

Conceptually:

```
FoodFast → maximum one tenant admin
QuickShop → maximum one tenant admin
```

Platform-level users still have:

```
organization_id = NULL
```

MySQL allows multiple `NULL` values in this type of unique constraint, so multiple platform-level accounts can coexist while still enforcing one organization-bound administrator.

---

17. API Routes

Question

> What information does Smart Gateway need to process a request?

That leads to the `api_routes` table.

```
api_routes
    organization_id
    route_id
    name
    path_pattern
    target_base_url
    target_path_prefix
    requests_per_minute
    response_timeout_ms
    idempotency_enabled
    active
    configuration_version
```

A route contains both:

```
Routing information
+
Gateway policy
```

---

18. Route Identity Is Tenant-Scoped

Coder Army

Coder Army

The important uniqueness rule is:

```
UNIQUE (organization_id, route_id)
```

This means `route_id` does not have to be globally unique.

For example:

```
FoodFast  + orders
QuickShop + orders
```

is perfectly valid.

Both companies can call their route:

```
orders
```

because the organization is part of its identity.

Conceptually:

```
Route Identity
=
Organization + Route ID
```

---

19. Route Allowed Methods

A route may allow multiple HTTP methods.

For example:

```
/orders

GET
POST
PUT
DELETE
```

Coder Army

Trying to store all of them in one route column would make the model harder to manage.

Therefore, allowed methods are represented separately:

```
route_allowed_methods
```

Conceptually:

```
api_route
    |
    +---- GET
    +---- POST
    +---- PUT
```

This creates a one-to-many relationship.

---

20. Route Header Rules

A route can also have multiple header rules.

For example:

```
Add X-Tenant-ID
Remove X-Internal-Header
Add X-Gateway-Version
```

Therefore, these rules are stored separately:

```
route_header_rules
```

Conceptually:

```
api_route
    |
    +---- header rule
```

```
    +---- header rule
    +---- header rule
```

Both allowed methods and header rules are children of the live route.

If the live route is deleted, these live child records can also be deleted.

---

21. Route Configuration Versioning

Now consider another problem.

Suppose a route changes multiple times.

```
Version 1
    ↓
Version 2
    ↓
Version 3
```

If we only update `api_routes`, the previous configuration disappears.

But configuration history can be extremely useful.

So we create:

```
route_configuration_versions
```

Containing information such as:

```
organization_id
route_id
version_number
change_type
configuration_snapshot
created_at
```

---

22. Every Route Mutation Creates a Version

Coder Army

Coder Army

A possible history may look like:

```
CREATE
    → Version 1
    → CREATED

PUT
    → Version 2
    → UPDATED

Deactivate
    → Version 3
    → DEACTIVATED

Activate
    → Version 4
    → ACTIVATED

DELETE
    → Version 5
    → DELETED
```

This gives us a historical record of how Gateway configuration evolved.

---

23. Why Configuration History Should Survive Route Deletion

A configuration version should not depend on the live route continuing to exist.

Suppose:

```
/orders
```

is deleted.

We may still need to know:

```
What configuration existed before deletion?
```

Therefore, `route_configuration_versions` deliberately does not depend on the live route through a foreign key that would cause its history to disappear when the route is removed.

The live state and historical state have different lifecycles.

```
Live Route
    → may be deleted

Route History
    → should remain
```

---

24. Control Plane Auditing

Versioning answers one important question:

> What did the route configuration look like?

But it does not completely answer:

> Who performed the operation?

For that, GateKeeper needs auditing.

We create:

```
control_plane_audit_records
```

with information such as:

```
organization_id
actor
action
resource_type
resource_id
correlation_id
duration_ms
result
```

Coder Army

```
details
created_at
```

An audit record can tell us things such as:

```
Actor       : foodfast-admin
Action      : UPDATE_ROUTE
Resource    : orders
Result      : SUCCESS
Duration    : 31 ms
```

---

25. Versioning vs Auditing

These two concepts are related but solve different problems.

Versioning

Answers:

```
What did the configuration look like?
```

For example:

```
Version 3 of the FoodFast orders route
```

Auditing

Answers:

```
Who attempted which operation, and what happened?
```

For example:

```
foodfast-admin attempted UPDATE_ROUTE
```

Coder Army

```
Result: SUCCESS
```

A useful mental model is:

```
Version History
    ↓
State history

Audit History
    ↓
Operator/action history
```

Both are valuable, but they should not be confused.

26. Final Database Relationship Model

The simplified relationship model becomes:

```
organizations
    |
    +---- 0..1 management_users
    |
    +---- many api_routes
               |
               +---- many route_allowed_methods
               |
               +---- many route_header_rules

organizations
    |
    +---- many route_configuration_versions
    |
    +---- many control_plane_audit_records
```

At this point, the database structure reflects the major business concepts instead of being a collection of arbitrary tables.

27. Design APIs Before Writing Controllers

Coder Army

Coder Army

Once the actors, ownership model, and database relationships are clear, we can design the API contract.

A good API should make two things obvious:

```
Who is making the request?
+
What operation are they allowed to perform?
```

Controllers should come **after** these boundaries are understood.

---

28. Public APIs

The publicly accessible management APIs begin with:

```
GET /health

POST /api/auth/login
```

Health Endpoint

```
GET /health
```

Allows us to check whether API Management is running.

Login Endpoint

```
POST /api/auth/login
```

Allows GateKeeper management users to authenticate.

These users include identities such as:

```
PLATFORM_ADMIN
TENANT_ADMIN
```

```
GATEWAY_SERVICE
```

depending on how authentication is configured.

29. Why There Is No Public Registration API

GateKeeper does not provide an endpoint such as:

```
POST /api/auth/register
```

The reason is architectural.

GateKeeper manages **management accounts**, not customer accounts.

Organizations and their administrators are provisioned through GateKeeper's controlled management flow.

A random user should not be able to register themselves as:

```
TENANT_ADMIN
```

or:

```
PLATFORM_ADMIN
```

Similarly, FoodFast customers do not register with GateKeeper.

Their accounts belong to FoodFast.

```
FoodFast Customer Registration
        ↓
FoodFast

GateKeeper Management Account Creation
        ↓
GateKeeper
```

Coder Army

These are two completely different identity systems.

30. Overall Request and Configuration Flow

The control-plane flow currently looks like:

```
Platform Administrator
        |
        | provisions
        ↓
     FoodFast
        |
        | gets
        ↓
FoodFast Tenant Admin
        |
        | configures routes
        ↓
API Management Service
        |
        | stores configuration
        ↓
      MySQL
```

Later, Smart Gateway can consume that configuration:

```
Smart Gateway
      |
      | GATEWAY_SERVICE identity
      ↓
API Management
      |
      | active route configuration
      ↓
Smart Gateway
```

Actual customer traffic follows a different path:

```
FoodFast Customer
      |
      | FoodFast credential
      ↓
Smart Gateway
```

Coder Army

Coder Army

```
      |
      | applies Gateway policies
      ↓
FoodFast Backend
      |
      | validates FoodFast customer
      ↓
Business Logic
```

This separation is the foundation of the GateKeeper architecture.

31. Control Plane vs Customer Traffic

It is important not to mix these two flows.

Management Flow

```
Tenant Admin
    ↓
GateKeeper API Management
    ↓
Create / Update Gateway Configuration
```

Authentication here belongs to **GateKeeper**.

Customer Flow

```
Tenant Customer
    ↓
Smart Gateway
    ↓
Tenant Backend
```

Authentication of the end user ultimately belongs to the **tenant application**.

This separation allows GateKeeper to manage infrastructure policies without taking ownership of every tenant's customer identity system.

Coder Army

32. Key Design Principles

1. A Microservice Must Own a Clear Problem

Creating separate Spring Boot applications does not automatically create a good architecture.

Each service needs a clear responsibility.

```
API Management → configuration
Smart Gateway  → request processing
Analytics      → observations
```

---

2. Separate Control Plane From Data Plane

Configuration management and live request processing solve different problems.

Keeping them separate makes the architecture easier to evolve.

---

3. Tenant Identity Must Be Explicit

Routes belong to organizations.

```
organization_id + route_id
```

Together identify a route.

This prevents one tenant's configuration from accidentally becoming another tenant's configuration.

---

4. Management Users Are Not Application Customers

```
foodfast-admin
```

and:

Coder Army

```
FoodFast customer
```

are completely different identities.

Mixing these concepts would create both architectural and security problems.

5. The Gateway Enforces Gateway Policy

GateKeeper deals with things such as:

```
Routing
Rate limiting
Timeouts
Headers
Idempotency
Route activation
```

The tenant backend continues to own application-level authorization.

6. Design the Database From Business Questions

Instead of starting with entities, ask:

```
Which companies use the system?
Who can operate it?
What configuration does the Gateway need?
What values can repeat?
How do we retain configuration history?
Who changed the configuration?
```

The tables then emerge naturally from the answers.

7. Live State and Historical State Have Different Lifecycles

Deleting a route should not automatically destroy its configuration history.

Historical data exists for a different purpose than the current live configuration.

Coder Army

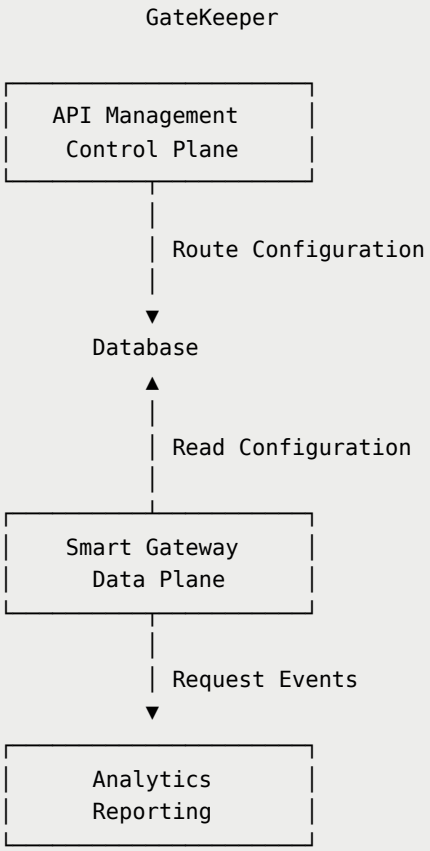
8. Versioning and Auditing Are Different

```
Versioning
→ What did the configuration look like?

Auditing
→ Who performed the operation and what happened?
```

A mature control plane usually needs both.

Final Architecture So Far



The important idea is that **API Management defines the behaviour of the Gateway, while Smart Gateway applies that behaviour to real traffic**.

With the ownership model, tenant boundaries, management identities, route model, version history, auditing, and API contract defined, the control plane now has a clear architectural foundation.

CoderArmy

Coder Army

GateKeeper-Architecture-Notes

GateKeeper — Custom API Gateway with Spring Boot

> **Lecture 1: Project overview, architecture and initial setup**
>
> This lecture covers the design of GateKeeper and the responsibilities of its services. The initial implementation milestone is creating the Spring Boot applications and verifying them with a simple health check. The gateway features below describe the planned system; their implementation belongs to upcoming lectures.

1. Why do we need an API gateway?

An application may start with one backend. The frontend communicates with one server, and that server handles authentication, requests and business logic.

When the backend is split into services, direct access becomes harder to manage. The client may need to know where each service runs, which service handles each operation and what changes when a service moves.

Several services may also repeat the same technical work:

- Validating authentication tokens
- Checking broad access permissions
- Applying request limits
- Creating request IDs
- Recording access information
- Handling technical errors

An API gateway provides one controlled entry point in front of these backend services. The client sends requests to the gateway, which applies the relevant rules and forwards each request to its destination.

Coder Army

What is an API gateway?

An **API gateway is a server between API clients and backend services**. It receives a request, applies entry-point policies, forwards it to the appropriate service and returns the response.

For the client, the gateway acts as the API server. For the backend service, it acts as the caller.

The three main jobs

Job	What it means
Routing	Select the target service using the request and route configuration.
Policy enforcement	Check authentication, permissions, request limits and other configured rules.
Reverse proxying	Send the HTTP request to the target and return its response.

Routing chooses the destination. Proxying performs the exchange.

Forward proxy and reverse proxy

A forward proxy acts on behalf of clients when they contact servers. A reverse proxy acts on behalf of servers when clients contact them.

An API gateway performs reverse proxying and adds API-specific policies around it.

2. What belongs in the gateway?

The gateway handles technical entry rules. Backend services remain responsible for business rules.

For example, the gateway can require an administrator role for a protected operation. The target service must still check whether that administrator is allowed to modify a particular resource.

Moving resource ownership checks and domain logic into the gateway would couple it to every backend service. Over time, it would become another large application that must change whenever business rules change.

Coder Army

> **The gateway controls entry. The target service validates the business operation.**

Related components

Component	Main responsibility	GateKeeper scope
API gateway	Route API traffic and apply entry policies.	Smart Gateway Service
Reverse proxy	Forward requests and return backend responses.	Part of the Smart Gateway
API management	Define, store and update routes and policies.	API Management Service
Load balancer	Select an instance of a service.	Excluded
Service discovery	Locate running service instances.	Excluded; target addresses are configured
Service mesh	Manage internal service-to-service traffic.	Excluded

3. What are we building?

GateKeeper is a backend-focused API gateway and API management platform built with Spring Boot, direct HTTP communication, Redis and Kafka.

The project demonstrates how gateway responsibilities work internally. It is an educational implementation with a defined scope, rather than a replacement for a mature production gateway.

Main applications

Application	Purpose
Smart Gateway Service	Handle and forward live client requests.
API Management Service	Manage users, routes and gateway policies.
Analytics Service	Process access events and provide request statistics.

Small Catalog and Order demo services will act as targets behind the gateway. Their purpose is to demonstrate forwarding and policies; they are not separate, full-sized business projects.

Coder Army

Planned capabilities

Area	Capabilities
Request identification	Generate or propagate a correlation ID.
Routing	Match active routes using configuration held in memory.
Security	Validate JWTs locally and enforce route-level role requirements.
Traffic control	Use Redis for rate limiting.
Duplicate request handling	Use Redis for configured POST idempotency.
Forwarding	Proxy HTTP requests with route-specific timeouts.
Transformation	Rewrite paths and add or remove selected headers.
Error handling	Return consistent gateway errors.
Analytics	Publish access events to Kafka for later processing.
Configuration	Store and version route settings; notify the gateway about changes.
Auditing	Record administrative actions using focused AOP auditing.

Dynamic routing

Dynamic routing means that routes are stored as configuration instead of being hard-coded into a large conditional block.

A route describes information such as its destination, allowed methods, authentication requirements, permitted roles, request limit, timeout and active status.

The Management Service stores these settings in MySQL. The Gateway loads active routes into memory and uses them for route matching. Updating a route does not require changing and rebuilding the Gateway's source code.

4. Data plane and control plane

Separating live traffic from configuration management is a central design decision.

Coder Army

Plane	Service	Responsibility
Data plane	Smart Gateway	Apply existing rules to live requests.
Control plane	API Management	Define and change those rules.

The data plane needs a short, predictable request path. The control plane handles administration and durable configuration storage.

Once the Gateway has loaded its routes, it can continue using that configuration during a temporary Management Service outage. New configuration loads and updates depend on the Management Service becoming available again.

5. Why build a custom gateway?

Using an API gateway is common and useful. Building a complete production gateway from scratch is a much larger commitment.

A production gateway may need high concurrency, connection pooling, streaming, multiple protocols, certificate management, service discovery, load balancing, resilience mechanisms, monitoring and ongoing security maintenance.

GateKeeper focuses on a smaller set of responsibilities so the underlying design is easier to understand. It brings together Spring Security, filters, interceptors, HTTP communication, Redis, Kafka, database ownership, exception handling and AOP in one project.

Features outside this project's scope

- Spring Cloud Gateway, Eureka and Spring Cloud Config
- Service discovery and load balancing
- Circuit breakers and automatic retries
- General response caching
- Request or response body transformation
- XML-to-JSON conversion
- WebSocket, gRPC and GraphQL proxying
- File uploads and streaming proxy support
- TLS termination

Coder Army

- API keys, billing and quota products
- Business-process orchestration

These features can be useful in other systems. They are excluded here to keep the project focused and manageable.

In particular, automatic retries can add delay and duplicate writes. Redis is used for rate limits and idempotency, not general caching of backend responses.

6. How the design controls latency

A gateway adds an extra network hop and processing work. It cannot provide zero additional latency.

GateKeeper keeps that extra work limited by avoiding unnecessary synchronous dependencies.

If the Gateway contacted separate authentication, routing, logging and analytics services for every request, each call would add time and another possible point of failure.

Instead, GateKeeper keeps routing and token validation local. Redis is consulted for policies that require shared state. The target service is then contacted over HTTP.

Work	Where it happens	Reason
Route matching	Gateway memory	Avoid a Management or database call for every request.
JWT validation	Gateway process	Verify the token using configured key material.
Role checks	Gateway process	Compare the authenticated roles with the loaded route policy.
Rate limiting	Redis	Maintain shared counters with atomic operations.
Idempotency checks	Redis	Coordinate duplicate requests using shared temporary state.
Path and header changes	Gateway process	Apply the loaded transformation rules locally.

Work	Where it happens	Reason
Analytics storage	Analytics Service	Process events outside the live response path.

Coder Army

Local JWT validation

The Management Service will authenticate users and issue signed JWTs. The Gateway will validate those tokens locally rather than calling the Management Service for every protected request.

This removes a recurring network call. It also means the Gateway is not fetching current user information on every request. Token expiry, key rotation and revocation are separate concerns; full revocation is outside this project's scope.

Routes in memory

MySQL remains the durable source of route configuration. The Gateway's in-memory registry is the copy used for fast request processing.

Routes are loaded at startup and refreshed when configuration changes. The Gateway does not read MySQL directly.

7. Redis: immediate shared state

Redis is a fast key-value data store. GateKeeper uses it for short-lived state needed while deciding whether a request can continue.

Why not use only Java memory?

Memory belongs to one running process. If multiple Gateway instances maintained separate counters, each instance would see only the requests it received.

Redis provides shared state that those instances can consult. Even with one Gateway running locally, this demonstrates the reason for separating shared runtime state from process memory.

Key concepts

Coder Army

Concept	Meaning
Key	Identifies a stored value.
Value	The counter, status or response information stored under a key.
TTL	A time limit after which the key expires automatically.
Atomic operation	A change performed as one indivisible operation.

Atomicity matters when requests arrive together. Separate reads and writes can allow two requests to act on the same old value. A Redis Lua script can group related counter and expiry operations into one atomic operation.

Rate limiting

Rate limiting restricts how many requests a user or client may make to a route within a time window.

Redis holds the counter. The Gateway updates it atomically, checks the configured limit and rejects requests that exceed it. Expiration removes counters after their useful window.

POST idempotency

Idempotency handling helps avoid repeating an operation when a client resends the same request.

For configured routes, Redis will hold temporary processing state and a saved successful response associated with an idempotency key. A matching completed request can receive the saved response without forwarding the operation again.

This is different from general response caching. It is tied to identifying a repeated operation. The detailed implementation and failure cases are reserved for later lectures.

Redis boundaries

Redis does not hold the authoritative route configuration or analytics history. MySQL and MongoDB have those responsibilities.

8. Kafka: asynchronous events

Coder Army

Kafka is a durable event-streaming platform. A producer writes an event to a topic, and a consumer reads it independently.

The consumer does not need to process the event immediately. Kafka retains records according to its configured retention, allowing consumers to process them later.

Key terms

Term	Meaning	GateKeeper example
Producer	Publishes events.	Gateway or Management Service
Topic	A named stream of events.	Access events or configuration events
Consumer	Reads and processes events.	Analytics Service or Gateway
Partition	An ordered subdivision of a topic.	Allows work to be distributed.
Offset	A record's position within a partition.	Tracks reading progress.
Consumer group	Consumers sharing the work of a topic.	Analytics consumers working together.

Access events

The Gateway publishes request outcome metadata to `gateway-access-events`. The Analytics Service consumes these events and stores them in MongoDB.

Useful metadata includes the event ID, correlation ID, route, request method, outcome, duration and whether a rate-limit rejection or idempotency replay occurred.

Passwords, JWTs, cookies and authorization headers must not be included in access events.

Configuration events

The Management Service publishes route-change notifications to `gateway-configuration-events`.

A notification identifies what changed. The Gateway then fetches the latest configuration from the Management Service and refreshes its in-memory registry.

Coder Army

The event is a notification; MySQL remains the source of truth through the Management Service.

Duplicate events

A consumer may receive an event again during failure recovery. The Analytics Service will use a unique `eventId` to avoid storing the same access event twice.

A `correlationId` serves a different purpose: it links work associated with one incoming request. An `eventId` identifies one event for deduplication.

Redis and Kafka serve different needs

Question	Redis	Kafka
What does it provide here?	Immediate shared state	Asynchronous event delivery
Typical data	Counters and idempotency state	Access and configuration events
Why does the Gateway use it?	Obtain a result needed for a policy decision	Notify another component about an outcome or change
How long is data retained?	Usually a short period controlled by TTL	According to topic retention

The Gateway does not wait for Analytics or MongoDB to finish processing an event. Publishing still has a cost, and producer failures need handling; an asynchronous send alone does not guarantee that events can never be lost.

9. Service responsibilities and ownership

Smart Gateway Service

The public entry point and data plane.

Its planned responsibilities are route matching, correlation IDs, JWT validation, role checks, rate limiting, idempotency, path and header rules, forwarding, timeouts, gateway errors and access-event publication.

It holds active routes in memory and uses Redis for temporary policy state. It loads configuration through the Management Service and contacts target services over HTTP.

Coder Army

It has no MySQL or MongoDB database and owns no business-domain tables.

API Management Service

The control plane for users, routes and policies.

It will manage credentials and roles, issue JWTs, store route configuration, maintain configuration versions and publish change notifications. AOP will support auditing of administrative actions.

It owns MySQL data for users, roles, routes, policy settings, versions and audit records. Other services must not directly modify its tables.

Analytics Service

Processes Gateway access events into operational information.

It will consume events, deduplicate them, store event documents and support statistics such as request counts, outcomes, average duration, slow routes, rate-limit rejections and idempotency replays.

It owns MongoDB data for access events and processed-event records. It does not change route configuration or participate in forwarding live requests.

Catalog and Order demo services

Small downstream services used to demonstrate routing, authorization, path rewriting, header propagation, timeouts and duplicate request handling.

Their business logic stays deliberately small. They remain responsible for their own business-level validation and authorization.

Infrastructure ownership

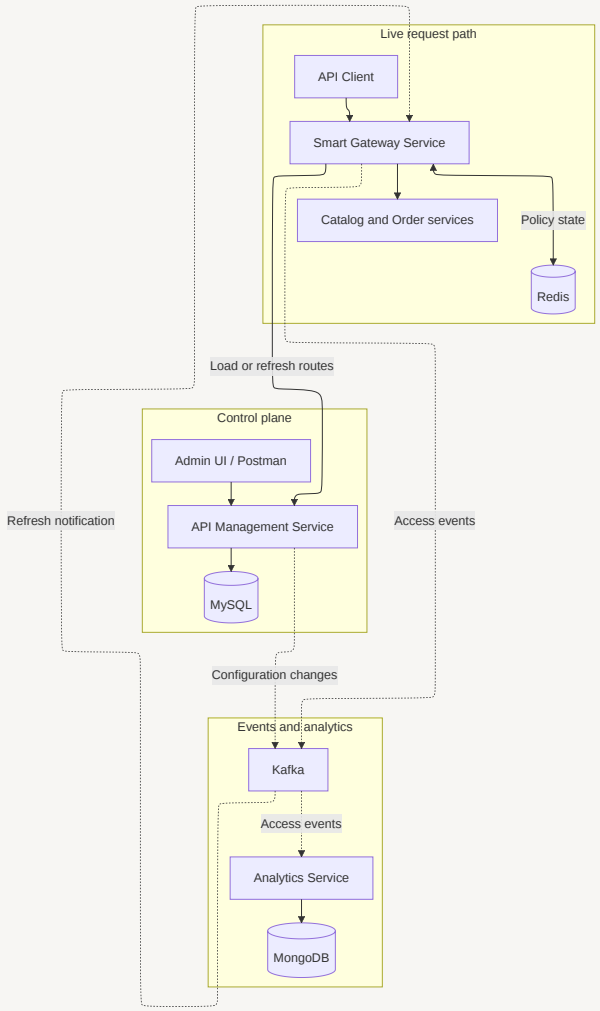
Component	Used or owned by	Data or purpose
MySQL	Owned by API Management	Durable users, roles, routes, versions and audits
MongoDB	Owned by Analytics	Access-event documents and processing records
Redis	Used by the Gateway’s policy layer	Rate-limit counters and idempotency state

Coder Army

Component	Used or owned by	Data or purpose
Kafka	Shared event infrastructure	Access events and route-change notifications

10. Complete architecture

The architecture separates live traffic, configuration management and analytics. Solid arrows show direct interactions; dashed arrows show event flow.



Live request path

The client contacts the Gateway. The Gateway uses local route and security information, consults Redis when a configured policy needs it, and forwards

permitted traffic to the target service. The target's response returns through the Gateway.

Management, MySQL, Analytics and MongoDB are not contacted for every forwarded request.

Configuration path

An administrator changes configuration through the Management Service. That service saves the change in MySQL and publishes a Kafka notification. The Gateway fetches the latest configuration and updates its local registry.

This path runs during startup and configuration refresh, not for every client request.

Analytics path

The Gateway publishes an access event to Kafka. The Analytics Service consumes it and stores the information in MongoDB. The client does not wait for this processing to complete.

Architectural boundaries

- Only the Smart Gateway forwards live target API traffic.
- The Gateway accesses route configuration through Management, not through MySQL.
- The Gateway does not call Analytics or MongoDB during forwarding.
- Analytics does not administer routes.
- Each service controls its own persistent data.
- Target services retain business-specific security checks.

11. What happens when a component fails?

Component unavailable	Intended effect
API Management	Already-loaded routes remain usable; administration and refreshes pause.

Coder Army

Coder Army

Component unavailable	Intended effect
Analytics	Forwarding continues; retained Kafka events can be processed later.
MongoDB	Forwarding continues; analytics storage is affected.
Target service	Requests to that target fail through the Gateway.
Slow target	The configured route timeout limits waiting.
Redis	Routes requiring Redis-backed policy decisions fail closed in this project.
Kafka	Event publication or delivery is affected; producer failures need explicit handling.

Fail closed means refusing a request when a required policy cannot be checked, rather than silently bypassing the policy.

Separation reduces unnecessary dependencies. It does not make the system immune to failures: a component required for the current request can still affect that request.

12. Initial setup and health check

The first implementation milestone uses three Spring Boot applications:

- `smart-gateway-service`
- `api-management-service`
- `analytics-service`

Each application should start independently, use a distinct port and answer a simple health-check request.

The health check confirms that the application can start and respond over HTTP. It does not, by itself, prove that Redis, Kafka, MySQL, MongoDB or the complete gateway workflow is ready.

The supplied script does not include the health-check implementation, so no controller code, endpoint path or port values are reproduced here.

Quick revision

Concept	Remember
API gateway	One controlled entry point: routing, policies and proxying.
Data plane	Smart Gateway handles live requests.
Control plane	API Management defines and stores the rules.
In-memory routes	Avoid repeated configuration lookups during forwarding.
Local JWT validation	Avoid an authentication-service call for every token.
Redis	Shared temporary state for rate limiting and idempotency.
Kafka	Access events and configuration-change notifications.
MySQL	Durable configuration owned by Management.
MongoDB	Analytics data owned by Analytics.
Latency	The gateway adds a hop but avoids unnecessary synchronous calls.
Business logic	Remains in target services.
Current milestone	Architecture, independent application startup and a basic health check.

Coder Army